Eema Complete End-to-End Test Master Plan

Authoritative planning baseline: **2026-09-09**, backend `b1c33b09ecb7b405a9113021586e03b996c45f1c`, frontend `8eb69a824772caf794945bdb3ac036a6654fb90a`; working-tree source inspected in both repositories. This is a test plan with a complete AUTH execution addendum dated 2026-09-10: **15 LOCAL PASS, 0 FAIL, 0 BLOCKED (rerun after pre-launch fixes; historical failures retained in the execution report)**. Subsequent domain execution sections and linked reports supersede baseline NOT RUN labels. PRICE-001–011 were audited and executed on 2026-09-15: **11 LOCAL PASS**, with PRICE-DEF-005 resolved on 2026-09-19 by the approved no-free-booking policy; see PRICE execution report. PAY-001–008 were audited and executed on 2026-09-15: **8 LOCAL PASS**, with live Mollie validation outstanding (the zero-price decision is now resolved; see PRICE addendum); see PAY execution report. PAYOUT-001–006 were audited and locally executed on 2026-09-19: **6 LOCAL PASS**, with the user-approved Eema v1 provider-cost allocation policy now implemented; see PAYOUT execution report. DEST-001–004 were executed on 2026-09-19: **4 LOCAL PASS**; see DEST execution report. DOC-001–004 are **4 LOCAL PASS** after the approved Unicode-name policy resolution; see DOC execution report. Existing automated means a test exists, not that its associated business scenario has passed. AUTH-001 results and newly added coverage are recorded in its companion guide; original suite inventories/counts below describe the 2026-09-09 baseline.

OPS-DEC-001 is resolved: important existing lifecycle notifications use a PostgreSQL transactional outbox. OPS-001–009 are **9 PASS / 0 FAIL / 0 BLOCKED** locally, with complete regression and packaged crash recovery passing. Historical **8 PASS / 0 FAIL / 1 BLOCKED** evidence is preserved. See OPS execution report. JOB-001–006 are also **6 PASS / 0 FAIL / 0 BLOCKED**, after shared-clock and bounded-backlog progress fixes. See JOB execution report. RACE-001–008 are **8 PASS / 0 FAIL / 0 BLOCKED** after reconciling current application-level race evidence. See RACE execution report. UX-001–035 have passing executed browser/client checks after five P2 fixes, with **24 PASS / 0 FAIL / 11 BLOCKED** under the approved representative AT model. The 11 selected scenarios require distinct real screen-reader flows; the other routes reuse shared coverage. The overall UX domain remains BLOCKED by UX-ENV-001 until all 11 actual AT flows pass; no actual AT execution is claimed. See UX execution report. Automatic continuation stops at UX; UXACT is not started.

Scope: current business workflows, UI, APIs, service contracts, persistent constraints, scheduled work and external-provider boundaries. The original planning baseline did not execute application changes or provider calls. Later authorized execution passes record their narrow source fixes, automated/browser evidence and remaining provider/product gates in their domain reports. Explicit characterization of a limitation is not approval of it for launch.

Use the system map first, then fixtures/runbook, domain scenarios and the release gate. Tables and parameterized scenarios are executable checklists: record each named variant separately, and fail the parent scenario if any required variant fails. Counts below count unique test IDs, not each permutation or browser viewport. An API role case and a unit case intentionally cover different layers of the same capability.

Contents

- System map and authoritative states
- Levels, priorities and environment matrix
- Fixture cookbook and local runbook
- Detailed scenarios
- Endpoint-by-role matrix
- Every frontend route and design QA
- Email lifecycle matrix
- Existing automated coverage
- Traceability and scope checklist

- Test coverage / product gaps discovered
- Regression packs and automation order
- Private-alpha gate and beta extension
- Counts and document verification

System map

Role	Current authority
anonymous	Active catalogue/detail/effective availability, public media, auth/recovery, Mollie webhook and health. Policy content currently requires login.
SCOUT	Own requests/bookings/inquiries; checkout and cancellation; own paid receipts; participant incidents; policy acceptance.
OWNER	Own venues/availability/production/media; verification and payout destination submission; request/inquiry decisions; confirmed cancellation; own payouts/statements and participant incidents.
ADMIN	Operational queues/reviews, booking read/accept/decline/cancel/full refund, payouts/liabilities, policy lifecycle, incident resolution; privileged receipts/statements. Not a universal substitute for OWNER-only endpoints.

Spring URLs below are relative to `http://localhost:8080/api`; Next routes use `http://localhost:3000`. No `/venues/[id]` Next page exists: public venue detail and calendar live inside `/`. No separate login/register page exists: AuthPanel is embedded.

Domain	Controller(s) / major paths	Authoritative states	Primary dependencies / frontend	Domain scenarios
Authentication	`AuthController`: `/auth/forgot-password`, `/auth/login`, `/auth/me`, `/auth/refresh`, `/auth/register`, `/auth/resend-verification`, `/auth/reset-password`, `/auth/verify-email`	UserRole; active/emailVerified; refresh revoked/expiry; recovery usedAt	AuthService / AccountRecoveryService; /, /verify-email, /forgot-password, /reset-password	AUTH-001–AUTH-015
Policies	`PolicyController`: `/policies/accept`, `/policies/current`, `/policies/status`, `/policies/{type}/current`	PolicyType, PolicyStatus; acceptance completeness	PolicyAcceptanceService; /policies	POL-001–POL-006
Policies	`AdminPolicyController`: `/admin/policies`, `/admin/policies/{id}/publish`, `/admin/policies/{id}/retire`	PolicyType, PolicyStatus; acceptance completeness	AdminPolicyService; /admin/policies	POL-001–POL-006

Domain	Controller(s) / major paths	Authoritative states	Primary dependencies / frontend	Domain scenarios
Venues / production profile / media	`VenueController`: `/venues`, `/venues/my`, `/venues/{id}`, `/venues/{id}/moderation`, `/venues/{id}/photos`, `/venues/{id}/submit-for-review`	VenueStatus, ProductionActivity, TrustEventType	VenueService / VenueModerationService; / (detail panel), /owner/venues, /owner/venues/new, /owner/venues/[id]	VEN-001–VEN-018
Media	`UploadController`: `/upload/files/{filename:.+}`, `/upload/venue-photo`	VenueStatus; ordered photoPaths	FileStorage / StorageService / S3PublicMediaStorage; /owner/venues/[id]; standalone upload API only	MEDIA-001–MEDIA-004
Availability	`VenueAvailabilityController`: `/venues/{venueId}/availability`, `/venues/{venueId}/availability/overrides`, `/venues/{venueId}/availability/overrides/{overrideId}`, `/venues/{venueId}/availability/settings`, `/venues/{venueId}/availability/weekly`	AvailabilityOverrideType, BookingStatus (blocking subset)	VenueAvailabilityService / VenueBookingLeadTimePolicy; / (calendar), /owner/venues/[id]	AVL-001–AVL-015
Matching	`MatchController`: `/match`	ProductionActivity; ParsedBrief requirement/optional fields	MatchService / GeminiAiService / VenueFilterBuilder; /	MATCH-001–MATCH-032
Booking / cancellation / refund	`BookingController`: `/bookings`, `/bookings/{id}`, `/bookings/{id}/accept`, `/bookings/{id}/cancel`, `/bookings/{id}/cancellation-preview`, `/bookings/{id}/decline`, `/bookings/{id}/refund`	BookingStatus, BookingEventType, CancellationPolicyTier, ActorType, RefundStatus	BookingService / CancellationService / RefundService; /bookings/[id], /scout/bookings, /owner/bookings; admin refund/cancel API only	BK-001–BK-008
Checkout	`CheckoutController`: `/bookings/{bookingId}/checkout`	BookingStatus, PaymentCreationStatus, RefundStatus, PaidPaymentResult	PaymentService / PaymentCreationCoordinator; /bookings/[id], /booked	PAY-001–PAY-008
Payment/refund callback	`WebhookController`: `/webhooks/mollie/payment`	BookingStatus, PaymentCreationStatus, RefundStatus, PaidPaymentResult	WebhookService / BookingService / RefundCoordinator; /booked (reads state); webhook API only	PAY-001–PAY-008

Domain	Controller(s) / major paths	Authoritative states	Primary dependencies / frontend	Domain scenarios
Inquiries	`AvailabilityInquiryController`: `/availability-inquiries`, `/availability-inquiries/mine`, `/availability-inquiries/owned-venues`, `/availability-inquiries/{id}/accept`, `/availability-inquiries/{id}/decline`, `/availability-inquiries/{id}/offer`	AvailabilityInquiryStatus, BookingStatus	AvailabilityInquiryService; / (calendar), /scout/inquiries, /owner/inquiries	INQ-001–INQ-004
Owner trust	`OwnerVerificationController`: `/owner/verification`, `/owner/verification/submit`	OwnerVerificationStatus, VerificationMethod, TrustEventType	OwnerVerificationService; /owner/verification	TRUST-001–TRUST-003
Owner trust admin	`AdminOwnerVerificationController`: `/admin/owner-verifications`, `/admin/owner-verifications/{id}`, `/admin/owner-verifications/{id}/approve`, `/admin/owner-verifications/{id}/reject`	OwnerVerificationStatus, VerificationMethod, TrustEventType	OwnerVerificationService; /admin/owner-verifications, /admin/owner-verifications/[id]	TRUST-001–TRUST-003
Moderation	`AdminVenueModerationController`: `/admin/venues`, `/admin/venues/{id}`, `/admin/venues/{id}/approve`, `/admin/venues/{id}/events`, `/admin/venues/{id}/reactivate`, `/admin/venues/{id}/request-changes`, `/admin/venues/{id}/suspend`	VenueStatus, ProductionActivity, TrustEventType	VenueModerationService; /admin/venues, /admin/venues/[id]	VEN-001–VEN-018
Owner payouts/statements	`OwnerPayoutController`: `/owner/payouts`, `/owner/payouts/{id}`, `/owner/payouts/{id}/statement`	OwnerPayoutStatus, OwnerPayoutHoldReason, OwnerFinancialLiabilityType/Status, ProviderCostResponsibility	OwnerPayoutService / FinancialDocumentService; /owner/payouts, /owner/payouts/[id]	PAYOUT-001–PAYOUT-006
Admin payouts	`AdminOwnerPayoutController`: `/admin/payouts`, `/admin/payouts/{id}`, `/admin/payouts/{id}/mark-paid`	OwnerPayoutStatus, OwnerPayoutHoldReason, OwnerFinancialLiabilityType/Status, ProviderCostResponsibility	OwnerPayoutService; /admin/payouts, /admin/payouts/[id]	PAYOUT-001–PAYOUT-006

Domain	Controller(s) / major paths	Authoritative states	Primary dependencies / frontend	Domain scenarios
Liabilities	`AdminOwnerFinancialLiabilityController`: `/admin/liabilities`, `/admin/liabilities/{id}`, `/admin/liabilities/{id}/settle`	OwnerPayoutStatus, OwnerPayoutHoldReason, OwnerFinancialLiabilityType/Status, ProviderCostResponsibility	OwnerFinancialLiabilityService; /admin/liabilities, /admin/liabilities/[id]	PAYOUT-001–PAYOUT-006
Payout destinations	`OwnerPayoutDestinationController`: `/owner/payout-destinations`	PayoutDestinationStatus/Type, TrustEventType	OwnerPayoutDestinationService; /owner/payout-destination	DEST-001–DEST-004
Payout destination review	`AdminPayoutDestinationController`: `/admin/payout-destinations`, `/admin/payout-destinations/{id}`, `/admin/payout-destinations/{id}/reject`, `/admin/payout-destinations/{id}/reveal`, `/admin/payout-destinations/{id}/verify`	PayoutDestinationStatus/Type, TrustEventType	OwnerPayoutDestinationService; /admin/payout-destinations, /admin/payout-destinations/[id]	DEST-001–DEST-004
Receipt	`FinancialDocumentController`: `/bookings/{id}/receipt`	FinancialDocumentType/Status, BookingStatus	FinancialDocumentService; /bookings/[id]	DOC-001–DOC-004
Incidents	`IncidentController`: `/bookings/{bookingId}/incidents`	IncidentCategory/Status, TrustEventType	IncidentService; /bookings/[id]	INC-001–INC-002
Incident review	`AdminIncidentController`: `/admin/incidents`, `/admin/incidents/{id}`, `/admin/incidents/{id}/close`, `/admin/incidents/{id}/resolve`, `/admin/incidents/{id}/start-review`	IncidentCategory/Status, TrustEventType	IncidentService; /admin/incidents, /admin/incidents/[id]	INC-001–INC-002
Booking operations	`AdminBookingOperationsController`: `/admin/bookings`, `/admin/bookings/{id}`, `/admin/bookings/{id}/events`	OperationalExceptionType/Severity, BookingEventType, ActorType	AdminOperationsService; /admin/bookings, /admin/bookings/[id]	ADMIN-001–ADMIN-011
Exception queue	`AdminOperationsExceptionController`: `/admin/operations/exceptions`	OperationalExceptionType/Severity, BookingEventType, ActorType	AdminOperationsService / OperationalThresholds; /admin/exceptions	ADMIN-001–ADMIN-011

Shared dependencies: PostgreSQL 16 native enums/arrays, row locks and exclusion/unique constraints; Flyway common V1–V14 and local-only V100–V103; JWT/database user authority; Clock in selected time policies (other services use OffsetDateTime.now); transactional notification intents and bounded asynchronous outbox dispatch; Mollie payment/refund/reporting client;

Gemini parser/ranker; Resend; local/S3 public media and private document stores; AES-GCM/HMAC bank protection. Optional Next Cala BFF is separately tested and has no Spring trust authority.

Authoritative enums and state coverage

State/event/actor values below come directly from every `entity/enums/*.java`. STATE tests require all values, including fixture-only values. Booking transition tests cover the full 8×8 allowed/forbidden matrix: REQUESTED may become AWAITING_PAYMENT/DECLINED/EXPIRED/UNAVAILABLE/CANCELLED; AWAITING_PAYMENT may become CONFIRMED/EXPIRED/CANCELLED; CONFIRMED may become REFUNDED/CANCELLED; all remaining statuses are terminal in BookingStateMachine. Refund callbacks preserve CANCELLED or EXPIRED for those special workflows.

Enum	Exact values	Exhaustive contract
ActorType	`SCOUT`, `OWNER`, `ADMIN`, `SYSTEM`, `WEBHOOK`	STATE-001
AvailabilityInquiryStatus	`OPEN`, `DECLINED`, `OFFERED`, `EXPIRED`, `ACCEPTED`	STATE-002
AvailabilityOverrideType	`AVAILABLE`, `BLOCKED`	STATE-003
BookingEventType	`REQUEST_SUBMITTED`, `REQUEST_ACCEPTED`, `REQUEST_DECLINED`, `REQUEST_EXPIRED`, `REQUEST_BECAME_UNAVAILABLE`, `APPROVAL_EXPIRED`, `BOOKING_CONFIRMED`, `PAYMENT_SUCCEEDED`, `PAYMENT_REFUNDED`, `PAYMENT_RECONCILED`, `REFUND_REQUESTED`, `REFUND_STATUS_CHANGED`, `LATE_PAYMENT_DETECTED`, `LATE_PAYMENT_REFUND_REQUESTED`, `LATE_PAYMENT_REFUND_COMPLETED`, `SCOUT_CANCELLED`, `OWNER_CANCELLED`, `ADMIN_CANCELLED`, `PAYOUT_CALCULATED`, `PAYOUT_READY`, `PAYOUT_MARKED_PAID`, `PAYOUT_CANCELLED`, `OWNER_LIABILITY_CREATED`, `OWNER_LIABILITY_SETTLED`, `BOOKING_RECEIPT_ISSUED`, `OWNER_PAYOUT_STATEMENT_ISSUED`	STATE-004
BookingStatus	`REQUESTED`, `AWAITING_PAYMENT`, `CONFIRMED`, `DECLINED`, `EXPIRED`, `UNAVAILABLE`, `CANCELLED`, `REFUNDED`	STATE-005
CancellationPolicyTier	`PRE_PAYMENT_WITHDRAWAL`, `GRACE_PERIOD`, `FLEXIBLE_FULL_REFUND`, `PARTIAL_REFUND`, `NON_REFUNDABLE`, `OWNER_FULL_REFUND`, `ADMIN_FULL_REFUND`	STATE-006
FinancialDocumentStatus	`ISSUED`	STATE-007
FinancialDocumentType	`BOOKING_RECEIPT`, `OWNER_PAYOUT_STATEMENT`	STATE-008
IncidentCategory	`PROPERTY_DAMAGE`, `ACCESS_PROBLEM`, `SAFETY`, `NOISE_NEIGHBOUR`, `MISREPRESENTATION`, `CONDUCT`, `OTHER`	STATE-009

Enum	Exact values	Exhaustive contract
IncidentStatus	`OPEN`, `UNDER_REVIEW`, `RESOLVED`, `CLOSED`	STATE-010
OperationalExceptionSeverity	`INFO`, `WARNING`, `CRITICAL`	STATE-011
OperationalExceptionType	`PAYOUT_READY_OVERDUE`, `PAYOUT_HELD`, `OWNER_LIABILITY_OPEN`, `REFUND_STUCK`, `PAYMENT_CREATION_STUCK`, `MOLLIE_FEE_UNRECONCILED`, `LATE_PAYMENT_REFUND_IN_PROGRESS`	STATE-012
OwnerFinancialLiabilityStatus	`OPEN`, `SETTLED`, `WAIVED`	STATE-013
OwnerFinancialLiabilityType	`PROVIDER_COST`, `OWNER_PAYOUT_RECOVERY`	STATE-014
OwnerPayoutHoldReason	`WAITING_FOR_BOOKING_END`, `WAITING_FOR_PROVIDER_COST`, `REFUND_IN_PROGRESS`, `WAITING_FOR_PAYOUT_DESTINATION`	STATE-015
OwnerPayoutStatus	`CALCULATED`, `HELD`, `READY`, `PAID`, `CANCELLED`	STATE-016
OwnerVerificationStatus	`UNVERIFIED`, `PENDING`, `VERIFIED`, `REJECTED`	STATE-017
PaymentCreationStatus	`NOT_STARTED`, `CREATING`, `CREATED`	STATE-018
PayoutDestinationStatus	`PENDING_VERIFICATION`, `VERIFIED`, `REJECTED`, `REPLACED`	STATE-019
PayoutDestinationType	`IBAN`	STATE-020
PolicyStatus	`DRAFT`, `PUBLISHED`, `RETIRED`	STATE-021
PolicyType	`TERMS_OF_SERVICE`, `PRIVACY_POLICY`, `ACCEPTABLE_USE_POLICY`, `PRODUCTION_POLICY`, `INSURANCE_REQUIREMENTS`	STATE-022
ProductionActivity	`PHOTO`, `FILM`, `VIDEO`, `INTERVIEW`, `COMMERCIAL`, `MUSIC_VIDEO`, `CONTENT_CREATION`	STATE-023
ProviderCostResponsibility	`NONE`, `OWNER_SETTLEMENT`, `OWNER`, `EEMA`	STATE-024
RefundStatus	`NONE`, `CREATING`, `QUEUED`, `PENDING`, `PROCESSING`, `REFUNDED`, `FAILED`, `CANCELED`	STATE-025

Enum	Exact values	Exhaustive contract
TrustEventType	OWNER_VERIFICATION_SUBMITTED, OWNER_VERIFICATION_APPROVED, OWNER_VERIFICATION_REJECTED, VENUE_SUBMITTED, VENUE_APPROVED, VENUE_CHANGES_REQUESTED, VENUE_SUSPENDED, VENUE_REACTIVATED, PAYOUT_DESTINATION_SUBMITTED, PAYOUT_DESTINATION_VERIFIED, PAYOUT_DESTINATION_REJECTED, PAYOUT_DESTINATION_REPLACED, PAYOUT_DESTINATION_REVEALED, PAYOUT_DESTINATION_SNAPSHOTTED, INCIDENT_REPORTED, INCIDENT_REVIEW_STARTED, INCIDENT_RESOLVED, INCIDENT_CLOSED	STATE-026
TrustSubjectType	OWNER_VERIFICATION, VENUE, PAYOUT_DESTINATION, INCIDENT	STATE-027
UserRole	SCOUT, OWNER, ADMIN	STATE-028
VenueStatus	DRAFT, PENDING, ACTIVE, SUSPENDED	STATE-029
VerificationMethod	MANUAL_REVIEW, VIDEO_CALL, BUSINESS_DOCUMENT, VENUE_CONTROL_EVIDENCE	STATE-030
BookingLifecycleEvent.Type	REQUEST_SUBMITTED, REQUEST_ACCEPTED, REQUEST_DECLINED, REQUEST_EXPIRED, REQUEST_BECAME_UNAVAILABLE, APPROVAL_EXPIRED	STATE-031
AvailabilityInquiryLifecycleEvent.Type	CREATED, OFFERED, DECLINED, OFFER_EXPIRED	STATE-032
BookingService.PaidPaymentResult	APPLIED, LATE_PAYMENT, DUPLICATE	STATE-033

Scheduled work

All six scheduled methods are covered by JOB cases: request/payment expiry (same job), inquiry offer expiry, payout readiness, auth recovery-token cleanup, Mollie fee reconciliation and workflow observability. There is no separate general refund-polling scheduler or durable email outbox. Provider callback/replay paths must reconcile refunds.

Metrics inventory (JOB-006, OPS-007, SEC-003/006 and AUTO observability): counters `eema.workflow.booking.approval.expired`, `eema.workflow.booking.request.expired`, `eema.workflow.booking.request.unavailable`, `eema.workflow.payment.late.detected`, `eema.workflow.refund.automatic.initiated`, `eema.workflow.inquiry.offer.expired`; gauges `eema.workflow.payment.creation.stuck`, `eema.workflow.refund.stuck`, `eema.workflow.mollie.fee.unreconciled`, `eema.workflow.payout.held`, `eema.workflow.payout.ready`, `eema.workflow.payout.overdue`, `eema.workflow.owner.liability.open`; recovery counters `eema.auth.email.verification.completed`, `eema.auth.password.reset.completed`. For each metric capture baseline, commit one eligible event or seed one qualifying row, scan/query, require expected delta/value, replay and ensure

no duplicate increment. Inspect meter tags: no user/booking/payment IDs. Read metric through ADMIN-only actuator endpoint. Counter reset on process restart is not a persistent financial ledger.

Levels, priorities and environment matrix

Level	Evidence required
UNIT	Explicit domain decision/validation/formatting assertions with controlled clock/mocks.
INTEGRATION	Collaborating services/controllers/storage or real database; say whether repositories are mocked.
API	Actual HTTP status/body/headers and actor/ownership checks; mock MVC evidence is narrower than live HTTP.
LOCAL E2E	Browser→Next→Spring→local PostgreSQL flow; external boundaries stubbed only through an available test harness.
STAGING E2E	Deployed origins, real sandbox providers, persistent staging database/storage and actual email delivery.
SECURITY	Negative access, object identity, input, secret, storage and transport probes.
CONCURRENCY	Independent transactions/workers released from a barrier; inspect committed final state.
MANUAL UX	Browser/keyboard/screen-reader/responsive/copy review with screenshots.
OPERATIONAL	Startup/jobs/metrics/logging/configuration/backup/recovery evidence.

P0 = auth/security, money, state integrity or release-blocking operations. P1 = core marketplace/trust/admin workflow. P2 = important interaction, presentation and nonfinancial edge behavior. P3 = secondary/polish or trivial existing harness checks. A failing security or financial invariant discovered in a P2 visual check is escalated to P0; priority never weakens a required assertion.

Environment	What can be proved	Limits / use
LOCAL MOCKED	Existing Java service/controller tests, parser/ranker fixtures, provider payloads, PDF rendering, crypto, fixed-clock boundaries	No database locking, actual browser/provider/delivery or bucket IAM proof.
LOCAL REAL POSTGRES	Testcontainers constraints/locking; local Docker browser/API/persistence; isolated backup/restore	Existing Testcontainer mocks are test-class scoped, not a browser-accessible fake-provider application profile. Browser payment/AI portions require a separately prepared isolated stub harness or must be BLOCKED locally. Direct quote can bypass Gemini for local request QA.
LOCAL MOLLIE TEST	Optional real TEST checkout and manual callback per backend README	Requires test key and explicitly enabled external testing; local callback POST causes real provider GET. Cannot prove public callback delivery without public routing. Never use live key. Not executed for this documentation task.
STAGING	Real Mollie TEST callbacks/refunds, Resend delivery, Gemini behavior, S3 IAM/durability, TLS/CORS/proxy/restarts	Mandatory for provider/hosting proof; fee reporting may remain BLOCKED if sandbox cannot expose authoritative cost.

Environment	What can be proved	Limits / use
PRIVATE ALPHA	Gate-approved staging-like release with controlled participants and operating support	Run P0/P1 regression and actual provider/support drills; use approved business inputs, not test legal content.
PRODUCTION SMOKE	Health, TLS/security headers, active catalogue, existing authorized history and navigation	Read-only and separately authorized. No fixture reset, synthetic legal publication, charge/refund, bank reveal or mark-paid smoke.

Each scenario has one primary environment for count clarity. LOCAL REAL POSTGRES scenarios should also be rehearsed in staging when they exercise browser/API behavior; LOCAL MOCKED cases remain fast CI tests. PROVIDER cases are the mandatory staging-only layer. PostgreSQL and staging requirements are not interchangeable.

Result contract for every scenario

Record `test ID / variant / commit pair / environment / tester / UTC time / PASS|FAIL|BLOCKED|NOT RUN / evidence / defect ID / cleanup`. PASS requires every stated expected UI/API/domain/provider/security observation, including no forbidden side effects. FAIL means an observed mismatch; BLOCKED means missing fixture, clock control, approved policy, provider or infrastructure. Never convert a documented limitation to a PASS on its acceptance criterion merely because source predicts it. UNIT scenarios explicitly say UI N/A; manual browser behavior is covered separately.

Financial and time oracles

`durationMinutes = Duration.between(startTime,endTime).toMinutes()`; `billableMinutes=max(durationMinutes, minimumBookingMinutes)`; `hourlyTotal=HALF_UP(hourlyRate × billableMinutes / 60, 2)`; `venuePriceEur=min(hourlyTotal, normalized fullDayRate)` when cap exists. There is no independent “must physically book minimum duration” rejection. Price snapshots remain immutable across owner edits/acceptance/checkout.

Cancellation grace is inclusive through confirmation+24h **only when shoot is strictly more than 48h away**. Outside grace: >7d full; exactly 7d through exactly 48h partial 50%; <48h zero. Owner/admin confirmed cancellation always full with required reason and different provider-cost responsibility. Use cancellation snapshots and instants across DST. Refund cents HALF_UP; retained=paid−refund; commission on retained uses stored percentage; owner entitlement cannot be negative.

Local profile commission is 4.00%, base configuration default is 20.00% (overridable). Capture effective profile/config for each run, do not silently standardize prices or rates. Booking response/payment deadlines are now+configured hours (24h default), **not capped to shoot start**; inquiry offer expiry is capped to slot start. Checkout exempts a valid accepted booking from fresh lead-time checks.

Fixture cookbook and local runbook

Use a unique `RUN` suffix for each run and keep a local fixture manifest mapping aliases to returned UUIDs. Do not use production people, email/phone/bank data. `.test` addresses are non-deliverable: local tests capture messages; staging delivery needs an approved nonpersonal QA mailbox on a controlled test domain.

Alias	Fictional account	State variants
Scout A	`scout-a+RUN@example.test`, QA Scout A	Owns booking/inquiry; create separate unverified/inactive copy.

Alias	Fictional account	State variants
Scout B	`scout-b+RUN@example.test`, QA Scout B	Unrelated ownership/competing scout.
Owner A	`owner-a+RUN@example.test`, QA Owner A	Owns standard venues; UNVERIFIED/PENDING/VERIFIED/REJECTED copies.
Owner B	`owner-b+RUN@example.test`, QA Owner B	Owns separate venue/bank/payout, never authorized for A.
Admin	`admin+RUN@example.test`, QA Admin	Provision via isolated local SQL fixture or existing staging admin process; public ADMIN registration prohibited.

Fixture password `QaFixture-2026!` is test-only. Phone can be omitted; use literal synthetic string `QA-PHONE-001` for unconstrained phone text tests, not a real number. Use structurally valid published test IBAN `NL91ABNA0417164300` only as a local/mock validation fixture and **never execute a transfer to it**; replacement test IBANs come from IbanPolicyTest. Do not assume a structurally valid test number belongs to a bank sandbox.

Set `T0` to a recorded UTC instant, seconds/nanos zero for minute tests. `D0` is its Europe/Amsterdam local date; `D14=D0+14 days`, `D15=D0+15 days`. Give all days identical weekly hours to avoid dependence on weekday. For DST derive the next zone transition with ZoneRules in existing Java test harness rather than hardcoding a date. API/browser tests cannot freeze all current services with the injected Clock: use far-future slots and controlled fixture timestamps for expiry, and existing unit tests for exact boundaries. Never change a shared host clock.

Venue fixture	Price / minimum / cap / lead	Distinct production profile	Weekly availability
Studio	EUR100/h; 60m; no cap; 120m	120m², crew12, PHOTO/VIDEO permitted, natural light yes, loading yes, bathrooms yes, sound no, blackout yes, ceiling3.5, parking yes/2, floor0, lift no, wheelchair yes	Every day 09–17
Warehouse	EUR80/h; 120m; cap500; 1440m	500m², crew40, FILM/COMMERCIAL, loading yes, ceiling6, sound no, natural light no, parking yes/5	Every day 08–20
Loft	EUR125/h; 180m; cap700; 2880m	150m², crew10, PHOTO/CONTENT_CREATION, natural light yes, floor3, lift no, wheelchair no, parking no/0	Every day 10–18
House/location	EUR90/h; 240m; cap400; 4320m	180m², crew8, PHOTO permitted/MUSIC_VIDEO prohibited, natural light yes, loading no, bathrooms yes, sound no	Every day 09–16
Sound studio	EUR110/h; 60m; cap650; 1440m	100m², crew15, INTERVIEW/FILM, natural light no, sound yes, bathrooms/loading yes, blackout yes	Every day 09–19

All named venues use neighbourhood `QA District`, timezone Europe/Amsterdam, description `Fictional QA venue`, productionRules `QA only: follow agreed production rules`, tags `["qa"]`, powerDetails `QA fixture power specification`, and one actually uploaded synthetic image. Supply remaining optional booleans explicitly for complete fixtures; create null/false variants separately. ADMIN approves only fixture-ready local venues. These are test data choices, not new business rules.

Start and inspect locally

Do not overwrite existing `.env` files or print secrets. Backend local startup uses repository README/Docker Compose. Java tests require JDK21/Maven on host; Docker builds can run normal application without host Java. These commands are instructions for a future authorized test run, not evidence they were executed here.

```
cd ~/workspace/Eema-backend
docker compose up --build -d
curl -sS http://localhost:8080/api/actuator/health
mvn test
mvn verify -Ppostgres-integration
# In another terminal:
cd ~/workspace/Eema-frontend
npm run dev
# TypeScript/lint/build checks (no browser-test framework currently installed):
npm run check
```

For LOCAL MOCKED do not configure external keys. Existing tests replace providers inside the test class; there is no documented mock-provider server profile for an entire browser session. For LOCAL REAL POSTGRES browser flows avoid matching via direct venue quote when Gemini unavailable, and stop payment portions at provider boundary unless a controlled test harness has been prepared. A simulated browser response alone proves UI only and must not be recorded as backend E2E.

Concrete API fixture sequence

Use a REST client with environment variables `API`, `SCOUT_TOKEN`, `OWNER_TOKEN`, `ADMIN_TOKEN`, `VENUE`, `BOOKING`; store tokens privately, never paste into the report. Set `API=http://localhost:8080/api`. HTTP examples use variable placeholders resolved by that client.

1. `POST {{API}}/auth/register` with `{"email":"scout-a+RUN@example.test","password":"QaFixture-2026!","fullName":"QA Scout A","role":"SCOUT"}`; save `user.id`, `accessToken`, `refreshToken`. Repeat owner and second personas with separate role/email.
2. Local notification fixture captures verification token; `POST /auth/verify-email {"token":"captured value"}`. SQL `email_verified=true` is allowed only for isolated non-email test setup, and is not proof verification worked. Local ADMIN setup: register a fixture account, then parameterized SQL update its `users.role` to `ADMIN` and email_verified true; login afresh. Follow existing staging admin runbook instead on shared environments.
3. Create five **local test-only** policies as ADMIN using `POST /admin/policies {"policyType":"TERMS_OF_SERVICE","version":"QA-RUN-v1","title":"QA fixture only","effectiveAt":"<T0 ISO offset>","content":"TEST FIXTURE — NOT LEGAL CONTENT"}`, repeating every PolicyType; publish each returned ID. Never publish these placeholders to real participant environments.
4. As scout/owner `GET /policies/current`; accept actual returned IDs using `POST /policies/accept {"policyVersionIds":["<ID1>","<ID2>","<ID3>","<ID4>","<ID5>"]}`.
5. As owner `POST /owner/verification/submit {"note":"QA review"}`; admin `POST /admin/owner-verifications/{id}/approve {"verificationMethod":"MANUAL_REVIEW","note":"QA fixture"}`.
6. Create Studio with payload below, capture `id` as VENUE. Upload multipart `file` to `/venues/{{VENUE}}/photos`; copy returned photoPaths. Save timezone/settings and weekly schedule. Submit `/venues/{{VENUE}}/submit-for-review`; admin approve `/admin/venues/{{VENUE}}/approve` with optional note.
7. Scout request payload below to `/bookings`, capture BOOKING. Owner `POST /bookings/{{BOOKING}}/accept` (no body); scout `POST /bookings/{{BOOKING}}/checkout` (no amount body) only under prepared mocked/test-provider environment.

8. Callback is `POST /webhooks/mollie/payment`, `Content-Type: application/x-www-form-urlencoded`, body `id=<authoritative mock/test payment ID>`. It fetches Mollie state; this is not a fake “paid=true” switch.

```
{
  "name":"QA Studio RUN", "neighbourhood":"QA District", "sizeM2":120,
  "hourlyRateEur":100.00, "minimumBookingMinutes":60, "minimumLeadTimeMinutes":120,
  "fullDayRateEur":null, "maxCrewSize":12,
  "permittedActivities":["PHOTO","VIDEO"], "prohibitedActivities":[],
  "naturalLight":true, "blackoutPossible":true, "ceilingHeightM":3.5,
  "loadingAccess":true, "parkingAvailable":true, "parkingSpaces":2, "floorNumber":0,
  "liftAvailable":false, "wheelchairAccessible":true, "bathroomsAvailable":true,
  "soundRecordingSuitable":false, "productionRules":"QA only: agreed production rules",
  "powerDetails":"QA fixture power specification", "tags":["qa"],
  "description":"Fictional QA venue", "clipUrl":null
}
```

`PUT /venues/{id}` requires the same complete body plus `"timezone":"Europe/Amsterdam","photoPaths":["<actual uploaded reference>"]`. Creation has no timezone or photoPaths fields. Weekly request: `{"intervals":[{"dayOfWeek":"MONDAY","startTime":"09:00","endTime":"17:00"}]}`; add remaining six days explicitly for the standard fixture. Override: `{"date":"<D14>","startTime":"11:00","endTime":"12:00","type":"BLOCKED","allDay":false}`; all-day uses null times. Booking request:

```
{
  "venueId":"<VENUE UUID>", "shootDate":"<D14 YYYY-MM-DD>",
  "startTime":"09:15", "endTime":"13:45", "rawBrief":"QA photo shoot with 4 people",
  "scoutContact":{"name":"QA Scout A","email":"scout-a+RUN@example.test","phone":null}
}
```

Inquiry create: `venueId, requestedDate, requestedStartTime, requestedEndTime, message`; offer: `offeredDate, offeredStartTime, offeredEndTime`. Cancel: optional scout `reason`, required owner/admin `reason`; incident: `category, description`; resolve: `resolutionNote`; destination: `accountHolderName, iban`; mark paid: `externalTransferReference`. Exact required/null/length constraints for **every** request DTO are embedded in INPUT cases. GET filters and method signatures are in API cases and endpoint matrix.

Observe and reset state

Use read-only SQL for evidence on: users, refresh_tokens, email_verification_tokens, password_reset_tokens; venues/venue_operating_hours/venue_availability_overrides; bookings/booking_events/availability_inquiries; owner_payouts/owner_financial_liabilities/owner_payout_destinations; financial_documents; policy_versions/policy_acceptances/booking_policy_versions; owner_verifications/marketplace_trust_events/incident_reports. Confirm exact column names against common migrations before scripting; inspect hashes/counts, never export raw bank ciphertext keys or auth credentials. Record source/target status, immutable snapshots, row/event count, updated timestamps and provider IDs privately.

Prefer a per-run disposable database and restore the baseline between destructive variants. Targeted deletion requires an explicit transaction and child-first dependency order from migrations; RESTRICT relationships should fail rather than be disabled. Existing Postgres test class truncates its own disposable users/policy fixtures with CASCADE, not shared data. A normal restart is

`docker compose restart backend`. **`docker compose down -v` permanently deletes all local database, uploaded-media and private-document volumes.** Only use it on disposable local stack after backup and then `docker compose up --build -d` to reseed. Compose hardcodes container names/ports: a project name alone does not isolate a parallel stack; use a prepared override or separate DB/host ports.

Backup/restore drill commands (disposable local only)

```
cd ~/workspace/Eema-backend
docker compose exec -T postgres pg_dump -U eema -d eema -Fc > /tmp/eema-qa-backup.dump
docker compose exec -T postgres createdb -U eema eema_qa_restore
docker compose exec -T postgres pg_restore -U eema -d eema_qa_restore --no-owner < /tmp/eema-qa-backup.dump
docker compose exec -T postgres psql -U eema -d eema_qa_restore -c 'SELECT version, success FROM flyway_schema_history ORDER BY installed_rank'
docker compose exec -T postgres psql -U eema -d eema_qa_restore -c 'SELECT status, count(*) FROM bookings GROUP BY status'
```

Start the same backend artifact with datasource pointing to `eema_qa_restore`, a separate port and copied fixture storage; disable emails/reporting/provider credentials. Validate health, representative authorized reads, bank decrypt using retained fixture keys and PDF hashes. Preserve source counts before dump. Restore object volumes separately; record recovery duration and data loss point, with acceptable RTO/RPO supplied by operations (not invented here). Cleanup: stop restore instance, explicitly verify database name, drop only `eema_qa_restore` after retaining evidence; remove private local dump when no longer required. Managed staging snapshot/PITR and object-store recovery still require staging rehearsal.

Detailed scenarios

Scenario families: AUTH · POL · TRUST · VEN · READY · MEDIA · AVL · MATCH · BK · INQ · PRICE · PAY · CANCEL · REF · PAYOUT · DEST · DOC · INC · ADMIN · SEC · EDGE · OPS · JOB · RACE · UX · UXACT · PROVIDER · CALA · EMAIL · JOURNEY · INPUT · API · STATE · AUTO

AUTH scenarios

Auth-input contract update (2026-09-10): register/login/resend/forgot email is trimmed and lowercased before validation; registration fullName trims outer whitespace only. Passwords and tokens remain exact. All six auth body DTOs reject unknown properties with safe 400 responses; only SCOUT and OWNER are self-service roles. These rules supersede older generic auth template assumptions. Database V1 already enforces unique LOWER(email); no migration added.

AUTH-001 — Register SCOUT

Test ID: AUTH-001. **Area:** AUTH. **Priority:** P0 (public account creation, role escalation and credential integrity). **Test level:** API, INTEGRATION, LOCAL E2E, SECURITY. **Result:** AUTH-001A **PASS** (2026-09-10, hardened contract, local API/integration/browser); AUTH-001B **NOT RUN** (staging delivery). See dated execution record. Re-executed in the complete AUTH pass; current evidence. Broader Unicode password safety fails separately in AUTH-004, not this ASCII fixture.

Roles involved: Anonymous visitor → newly registered SCOUT. A valid existing SCOUT token is an optional-header security subcase, not a denial case. OWNER registration remains AUTH-002. Registration does not accept an owner/scout resource UUID for authorization.

Preconditions: Backend and PostgreSQL healthy; frontend healthy for browser checks; `/api/auth/register` reachable; generated email absent. No existing user, verified email, policy acceptance, owner verification, venue or booking is required. Use local email-disabled configuration, or an explicitly authorized staging delivery mailbox for variant B. Never send provider delivery to `example.test`.

Test data: `RUN_ID=$(scripts/qa-run-id.sh)`, `scout-a+$RUN_ID@example.test`, full name `QA Scout A`, synthetic password `QaFixture-2026!`, role `SCOUT`. Generate a separate RUN_ID for API and browser registrations. No hard-coded user UUID or seed deletion. API normalization test submits two surrounding spaces plus the uppercase address and `QA Scout A ` ; expect canonical lowercase email and `QA Scout A`. Password bytes/characters are unchanged.

Exact steps:

1. Follow the companion guide's startup/health checks; generate the fresh fixture. Open `/` in a separate browser profile, choose **Sign in / register**, then the **Create account** tab.
2. Fill full name, choose **Scout**, enter email/password, and submit once. Observe one `POST /api/auth/register`; inspect status and field names without exporting token-bearing response bodies or HARs.
3. Confirm the registration panel closes to the marketplace at `/` when entered through the header with no pending matching brief. Check identity, Scout navigation and **Email not verified**.
4. Reload before access expiry. Confirm cached authenticated SCOUT state and `emailVerified=false` persist. Separately issue authenticated `GET /api/auth/me` and compare its user DTO. Do not claim reload itself invokes `/auth/me`.
5. Inspect safe database aggregates from the guide; run `mvn verify -Ppostgres-integration` for actual Argon2id matching, both token hash correspondences, expiry and transaction assertions. Do not try to recover verification secrets from the database.
6. Run the security/rejection subcases below with new fixtures where applicable. Sign out and remove temporary response evidence after recording sanitized results.

Expected UI behavior: `AuthProvider.completeSession` persists access/refresh tokens, expiresIn, user and storedAt under localStorage `eema:auth`; subscribers update immediately. Header shows `QA Scout A · Scout`, **My bookings**, **Inquiries**, **Email not verified**, **Sign out**. Registration through the header returns to idle marketplace; no route redirect. A pending matching brief is a separate flow. Browsing remains available. The API client blocks authenticated non-GET, non-admin mutations for an unverified account; backend verified-email/policy gates remain authoritative on their applicable endpoints. A **Policy review required** link depends on a successful policy-status response; registration neither requires nor creates acceptances. Reload reads cached session; compare `/auth/me` explicitly, without implying cache restoration validates the JWT.

Expected API/backend behavior: Anonymous `POST /api/auth/register` returns **201 Created**, JSON with exactly `accessToken` (nonempty signed JWT), `refreshToken` (nonempty opaque token), `tokenType` (`Bearer`), `expiresIn` (seconds; current 900), `user`. User fields are exactly `id` (UUID), `email` (lowercase, plus suffix preserved), `role=SCOUT`, `fullName=QA Scout A`, `phone=null`, `active=true`, `emailVerified=false`. No password/hash/verification token appears. JWT is issued for the new user, not an optional authenticated caller. `/api/auth/me` returns 200 and the same user. Record constructors invoke `EmailNormalizer` before Jakarta validation. Padded mixed-case email succeeds and persists trimmed/lowercase; internal whitespace and invalid format still fail `@Email`. Registration fullName is trimmed at its edges, preserving internal spaces. No Gmail-specific normalization.

Expected database/domain state: One new `users` row with returned UUID, normalized email, exact full_name, SCOUT, active=true, email_verified=false; password_hash is self-describing Argon2id (default memory 65536 KiB, iterations 3, parallelism 1), differs from raw password and satisfies `PasswordEncoder.matches(raw, hash)`. One `refresh_tokens` row for that user, revoked=false, expiry approximately registration time + configured 7 days, token_hash equal to lowercase SHA-256 hex of returned raw refresh token and unequal to the raw token. One `email_verification_tokens` row, used_at NULL, expiry approximately injected Clock time + configured 24 hours, token_hash equal to `OneTimeTokenCodec.hash` of the transient event token. Both token hashes have 64 hex characters; neither raw token is stored. No policy acceptance is implied. HTTP status, response fields and header copy are not database columns. Verification persistence failure rolls the user/token transaction back; post-commit delivery failure preserves committed state.

Expected email/provider behavior: Success publishes exactly one `EmailVerificationRequestedEvent(userId, rawToken)`. `AccountEmailEventListener` captures a credential-placeholder artifact synchronously BEFORE_COMMIT in the same transaction; the durable worker sends after commit. Notification recipient is the normalized email; subject **Verify your Eema email**; CTA `/verify-email?token=<transient-token>` at configured frontend base URL, with configured expiry and verification-required wording. Raw verification token is transient

notification content, never the auth response or a DB column. Provider failure cannot undo registration. OPS-DEC-001 adds durable retry; the outbox stores only a nonsecret derivation nonce/hash reference and credential-placeholder content, never the raw token or a credential-bearing URL. See OPS-008 for dispatch/recovery semantics.

- **AUTH-001A — LOCAL EMAIL DISABLED:** `EMAIL_NOTIFICATIONS_ENABLED=false`; token/event still created; notification adapter may execute, but `EmailService` skips Resend. Registration succeeds. This variant requires no real email delivery.
- **AUTH-001B — STAGING EMAIL ENABLED:** authorized controlled mailbox, configured sender/base URL and enabled Resend; verify one attempt, actual receipt, correct subject/CTA and expiry copy. Delivery is NOT RUN until proven in staging; synthetic local `.test` addresses cannot prove it. Do not click verification before completing the unverified-state checks.
- Rejected request: no new user/token records and no verification event/email attempt for that request. Duplicate registration leaves the first account's existing records unchanged.

Security expectation / focused subcases: These are assertion variants of AUTH-001, not extra master scenario IDs.

Subcase	Exact action	Expected result / side effects
Public endpoint	Submit valid SCOUT payload anonymously	201, new SCOUT
Optional authenticated actor	Repeat with fresh email and valid existing SCOUT Bearer token	201, new distinct user; caller identity does not override payload
ADMIN / unsupported role	Replace role with `ADMIN`, then `SUPERUSER`, using new addresses	400; no user, tokens or event
Unknown-field / mass assignment	Add each of `active`, `emailVerified`, `id`, `passwordHash`, `ownerVerificationStatus`, `createdAt`, or an arbitrary field separately to a valid payload	400 with safe existing JSON error; no user/token/event/email. Login, refresh, verify-email, resend-verification, forgot-password and reset-password are strict too; unrelated API JSON behavior is unchanged.
Duplicate / case collision	Register once, resubmit padded uppercase equivalent email	409; exactly one original user and one token of each kind; no extra event/email
Invalid inputs	Invalid email or internal email whitespace, 5-character password, 73-character password, 1-character fullName, missing role, malformed `{` body	400; no registration side effects. Password policy is 8–72 characters, not an invented complexity rule
Persistence fault	Integration test injects verification-token save failure	Safe 500; transaction rolls back user and both token types; no event/email
Post-commit notification fault	Integration test injects delivery adapter exception	201 persists; one attempt, no rollback; delivery itself failed

Cleanup/reset: Sign out, close isolated browser profile, discard raw responses/tokens; fixture rows can remain with RUN_ID. Preserve seed users/venues. Between full local suites only, run `scripts/reset-local-qa.sh --dry-run`, then explicitly `--confirm-destroy-local-qa` to delete all guarded local Compose database/media/document volumes and reseed. There is no snapshot-restore harness or targeted FK deletion step. Never apply this reset to staging. Testcontainers owns/discards its separate disposable database.

Automation recommendation: ALREADY AUTOMATED after this change: `RegistrationIntegrationTest` (11 methods) covers real HTTP/security, PostgreSQL/hash/expiry/atomicity, duplicate/invalid requests, strict unknown-field rejection, exact password spaces, normalized login/recovery and optional authenticated actor. `AuthInputContractTest` proves all six DTOs are strict, non-auth JSON stays tolerant, and secrets remain exact; `PostgresCriticalPathIntegrationTest.emailIndexRejectsConcurrentCaseVariedLogicalDuplicates` proves the existing lower(email) unique index under concurrency; `EmailServiceTest.disabledDeliveryNeverCallsProvider` proves disabled adapter does not touch Resend. Existing `AuthServiceTest.registerAndLoginIssueTokensWithoutPersistingRawRefreshToken` covers service token issuance; `AccountEmailNotificationServiceTest` covers content. MANUAL FRONTEND: rendered identity/navigation/reload/sign-out and explicit `/auth/me` comparison; one-off browser evidence does not constitute a committed browser suite. STAGING EMAIL ONLY: actual delivery/CTA. No browser framework added.

Environment requirement: LOCAL REAL POSTGRES for A and API/integration assertions; browser frontend at localhost:3000, backend localhost:8080/api. Testcontainers uses the existing `-Ppostgres-integration` profile and common migrations; local Compose additionally uses dev seeds. STAGING for B only. Neither provider-disabled local success nor simulated delivery failure proves actual deliverability.

Source/evidence: Backend `controller/AuthController`, `dto/request/{RegisterRequest,LoginRequest,EmailAddressRequest,StrictAuthRequest,EmailNormalizer}`, `dto/response/{AuthResponse,UserDto}`, `service/{AuthService,AccountRecoveryService,OneTimeTokenCodec,AccountEmailEventListener,AccountEmailNotificationService,EmailService}`, `security/{JwtService,JwtAuthenticationFilter}`, `security/SecurityConfig`, `entity/{User,RefreshToken,EmailVerificationToken}`, repositories, common migrations V1/V8, dev V100–V102, `exception/GlobalExceptionHandler`; frontend `components/auth/{AuthPanel,AuthProvider}`, `components/layout/AppHeader`, `app/page.tsx`, `lib/api/{auth,client}.ts`, `lib/auth/token.ts`. Reproduction/evidence: companion guide.

Pass/fail criteria: A passes only if local HTTP/DB/security/transaction checks and browser registration/reload/cleanup all meet the above, with dated sanitized evidence. B has its own delivery result. FAIL for a mismatch or unexpected side effect; BLOCKED for missing infrastructure/evidence; NOT RUN for unexecuted checks. Never infer B PASS from A or mark unrelated AUTH scenarios passed.

AUTH-002 — Register OWNER

Test ID: AUTH-002. **Area:** AUTH. **Priority:** P0. **Test level:** API, INTEGRATION, LOCAL E2E, SECURITY. **Result:** LOCAL PASS (2026-09-10); real delivery STAGING ONLY / NOT RUN where applicable.

Roles involved: Anonymous visitor registering OWNER.

Preconditions: Healthy local stack; fresh owner email; no verification/policy prerequisites.

Test data: Unique `scripts/qa-run-id.sh` email at example.test; fictional QA password, role and name. Independent fixture per destructive/expiry case; raw credentials kept only in test memory. Exact matrices are executable in the mapped tests.

Exact steps: 1. POST /api/auth/register with fresh padded email, QA Owner, password fixture, OWNER. 2. GET /api/auth/me and /api/owner/verification with returned access token. 3. Inspect dependent row counts. 4. Register/login OWNER through /; reload; sign out and open /owner/venues.

Expected UI behavior: OWNER identity, My venues/Verification/Payout details navigation and Email not verified; reload persists. Protected owner page renders sign-in after logout.

Expected API/backend behavior: 201 AuthResponse as AUTH-001 with user.role=OWNER, active=true, emailVerified=false. /me 200 agrees. Owner verification GET 200: status UNVERIFIED, id=null. No trust approval is implied.

Expected database/domain state: One OWNER and registration token rows; zero owner_verifications, venues, owner_payout_destinations and policy_acceptances for this user. GET must not create verification row.

Expected email/provider behavior: One verification event after successful registration; local disabled adapter skips provider. Staging delivery separate.

Security/authorization expectation: Public role allowlist; owner trust, bank ownership and email verification are independent.

Cleanup/reset: Close/sign out isolated browser; discard raw secrets; retain RUN_ID rows until deliberate guarded local full reset. Testcontainers discards its own database. Never delete seeded owners, disable FKs or restore a nonexistent snapshot.

Automation recommendation: AuthSessionIntegrationTest.ownerRegistrationCreatesNoTrustOrMarketplaceAggregates; browser OWNER journey. Execute rather than infer PASS from source. Human review concerns clarity/usability only; actual provider delivery is staging only.

Local/staging requirement: LOCAL REAL POSTGRES; browser at localhost:3000 with backend localhost:8080/api. Provider-disabled/mocked locally; no real email required for deterministic assertions.

Source/evidence: AuthController; AuthService; AccountRecoveryService; JwtService/JwtAuthenticationFilter; SecurityConfig/VerifiedUserPolicy; auth DTOs, repositories and V1/V8; frontend AuthPanel/AuthProvider/RoleGuard, token.ts/client.ts and recovery pages. Dated results, exact tests and sanitized artifacts in AUTH execution report.

Pass/fail criteria: LOCAL PASS only after mapped deterministic API/DB/client/browser assertions execute successfully. FAIL for contrary behavior; BLOCKED for missing execution infrastructure. Document characterized architecture limitations separately; staging delivery and human review have their own status.

AUTH-003 — Duplicate and normalized email

Test ID: AUTH-003. **Area:** AUTH. **Priority:** P0. **Test level:** API, INTEGRATION, LOCAL E2E, SECURITY. **Result:** LOCAL PASS (2026-09-10); real delivery STAGING ONLY / NOT RUN where applicable.

Roles involved: Anonymous visitor; isolated inactive duplicate fixture.

Preconditions: Fresh canonical email for initial success; mutate active only on generated fixture.

Test data: Unique `scripts/qa-run-id.sh` email at example.test; fictional QA password, role and name. Independent fixture per destructive/expiry case; raw credentials kept only in test memory. Exact matrices are executable in the mapped tests.

Exact steps: 1. Register padded mixed-case fresh address. 2. Resubmit exact address, uppercase and padded variants. 3. Set only this fixture inactive and retry duplicate. 4. Run concurrent SQL case-varied insert regression.

Expected UI behavior: API validation/duplicate message safe; registration does not replace the current session on failure.

Expected API/backend behavior: Initial 201; every canonical duplicate 409 including inactive original. Internal whitespace stays invalid. No plus alias stripping.

Expected database/domain state: One user and one registration token of each kind; failures add none. Existing LOWER(email) index prevents case-varied concurrent duplicates; no storage trim CHECK is claimed.

Expected email/provider behavior: Only first registration event; no duplicate event/send.

Security/authorization expectation: No privileged actor or unrelated-resource UUID required.

Cleanup/reset: Close/sign out isolated browser; discard raw secrets; retain RUN_ID rows until deliberate guarded local full reset. Testcontainers discards its own database. Never delete seeded owners, disable FKs or restore a nonexistent snapshot.

Automation recommendation: RegistrationIntegrationTest duplicate method; AuthSessionIntegrationTest.inactiveDuplicateStillConflicts; PostgresCriticalPathIntegrationTest email index race. Execute rather than infer PASS from source. Human review concerns clarity/usability only; actual provider delivery is staging only.

Local/staging requirement: LOCAL REAL POSTGRES; browser at localhost:3000 with backend localhost:8080/api. Provider-disabled/mocked locally; no real email required for deterministic assertions.

Source/evidence: AuthController; AuthService; AccountRecoveryService; JwtService/JwtAuthenticationFilter; SecurityConfig/VerifiedUserPolicy; auth DTOs, repositories and V1/V8; frontend AuthPanel/AuthProvider/RoleGuard, token.ts/client.ts and recovery pages. Dated results, exact tests and sanitized artifacts in AUTH execution report.

Pass/fail criteria: LOCAL PASS only after mapped deterministic API/DB/client/browser assertions execute successfully. FAIL for contrary behavior; BLOCKED for missing execution infrastructure. Document characterized architecture limitations separately; staging delivery and human review have their own status.

AUTH-004 — Registration validation

Test ID: AUTH-004. **Area:** AUTH. **Priority:** P0. **Test level:** API, INTEGRATION, LOCAL E2E, SECURITY. **Result:** LOCAL PASS (2026-09-10 rerun); historical BCrypt failure resolved with Argon2id.

Roles involved: Anonymous visitor.

Preconditions: Fresh emails for independent accepted boundaries; no account needed for rejections.

Test data: Unique `scripts/qa-run-id.sh` email at example.test; fictional QA password, role and name. Independent fixture per destructive/expiry case; raw credentials kept only in test memory. Exact matrices are executable in the mapped tests.

Exact steps: 1. Run registration field matrix: null/missing/blank/bad email; syntactically valid email lengths 254/255; names blank and 1/2/120/121; roles SCOUT/OWNER/ADMIN/unknown/missing; ASCII passwords 7/8/72/73 and surrounding spaces. 2. Probe 72 non-ASCII characters. 3. Send each forbidden extra property and malformed JSON. 4. Compare user/token/event counts.

Expected UI behavior: HTML limits block obvious invalid forms; safe backend validation shown for API-only boundaries. Password spaces remain meaningful.

Expected API/backend behavior: Valid boundaries 201; @NotBlank/@Email/@Size/role/strict JSON failures 400. Character limits are email254/name2–120/password8–72; 72 Unicode characters are accepted; distinct accepted passwords sharing the first72 UTF-8 bytes must not authenticate equivalently (AUTH-D01 regression). No complexity rule.

Expected database/domain state: Only accepted requests create user and token records. Invalid requests leave no partial account; Argon2id matching proven by shared registration tests.

Expected email/provider behavior: Accepted registrations publish verification; rejected requests publish none.

Security/authorization expectation: Unknown fields and ADMIN rejected; no raw password/hash/token details in errors.

Cleanup/reset: Close/sign out isolated browser; discard raw secrets; retain RUN_ID rows until deliberate guarded local full reset. Testcontainers discards its own database. Never delete seeded owners, disable FKs or restore a nonexistent snapshot.

Automation recommendation: RegistrationIntegrationTest; AuthSessionIntegrationTest.registrationBoundaryMatrix; AuthInputContractTest. Execute rather than infer PASS from source. Human review concerns clarity/usability only; actual provider delivery is staging only.

Local/staging requirement: LOCAL REAL POSTGRES; browser at localhost:3000 with backend localhost:8080/api. Provider-disabled/mocked locally; no real email required for deterministic assertions.

Source/evidence: AuthController; AuthService; AccountRecoveryService; JwtService/JwtAuthenticationFilter; SecurityConfig/VerifiedUserPolicy; auth DTOs, repositories and V1/V8; frontend AuthPanel/AuthProvider/RoleGuard, token.ts/client.ts and recovery pages. Dated results, exact tests and sanitized artifacts in AUTH execution report.

Pass/fail criteria: LOCAL PASS only after mapped deterministic API/DB/client/browser assertions execute successfully. FAIL for contrary behavior; BLOCKED for missing execution infrastructure. Document characterized architecture limitations separately; staging delivery and human review have their own status.

AUTH-005 — Login credentials and inactive account

Test ID: AUTH-005. **Area:** AUTH. **Priority:** P0. **Test level:** API, INTEGRATION, LOCAL E2E, SECURITY. **Result:** LOCAL PASS (2026-09-10); real delivery STAGING ONLY / NOT RUN where applicable.

Roles involved: Anonymous login to SCOUT/OWNER/isolated ADMIN account.

Preconditions: Existing active fixture, known password; no email/policy prerequisite. ADMIN fixture is created by isolated test setup, never public registration.

Test data: Unique `scripts/qa-run-id.sh` email at example.test; fictional QA password, role and name. Independent fixture per destructive/expiry case; raw credentials kept only in test memory. Exact matrices are executable in the mapped tests.

Exact steps: 1. Login each role with canonical/mixed/padded email. 2. Try wrong password and unknown email. 3. Set fixture inactive; try correct and wrong password. 4. Blank/malformed/extra-property bodies. 5. Inspect active refresh count after each. 6. Browser login both self-service roles, reload and logout.

Expected UI behavior: Correct identity/navigation; failures leave sign-in form safe and usable. Reload restores cached session; /me separately agrees.

Expected API/backend behavior: Success 200 AuthResponse; wrong and unknown 401 same Invalid email or password message. Inactive correct password403; inactive wrong password401. Invalid body400. /me anonymous401.

Expected database/domain state: Each successful login adds one hashed refresh row; no new user/verification token. Failed login adds none. Password hash unchanged.

Expected email/provider behavior: Login sends no lifecycle email.

Security/authorization expectation: Current intended inactive-account distinction is documented; timing equalization is not promised.

Cleanup/reset: Close/sign out isolated browser; discard raw secrets; retain RUN_ID rows until deliberate guarded local full reset. Testcontainers discards its own database. Never delete seeded owners, disable FKs or restore a nonexistent snapshot.

Automation recommendation: AuthSessionIntegrationTest.loginMatrixAndAnonymousMe; browser login/role journeys. Execute rather than infer PASS from source. Human review concerns clarity/usability only; actual provider delivery is staging only.

Local/staging requirement: LOCAL REAL POSTGRES; browser at localhost:3000 with backend localhost:8080/api. Provider-disabled/mocked locally; no real email required for deterministic assertions.

Source/evidence: AuthController; AuthService; AccountRecoveryService; JwtService/JwtAuthenticationFilter; SecurityConfig/VerifiedUserPolicy; auth DTOs, repositories and V1/V8; frontend AuthPanel/AuthProvider/RoleGuard, token.ts/client.ts and recovery pages. Dated results, exact tests and sanitized artifacts in AUTH execution report.

Pass/fail criteria: LOCAL PASS only after mapped deterministic API/DB/client/browser assertions execute successfully. FAIL for contrary behavior; BLOCKED for missing execution infrastructure. Document characterized architecture limitations separately; staging delivery and human review have their own status.

AUTH-006 — JWT and database role authority

Test ID: AUTH-006. **Area:** AUTH. **Priority:** P0. **Test level:** API, INTEGRATION, LOCAL E2E, SECURITY. **Result:** LOCAL PASS (2026-09-10); real delivery STAGING ONLY / NOT RUN where applicable.

Roles involved: Authenticated isolated user; anonymous invalid-token caller.

Preconditions: Disposable fixture and test-only JWT key; never modify seed/shared account roles.

Test data: Unique `scripts/qa-run-id.sh` email at example.test; fictional QA password, role and name. Independent fixture per destructive/expiry case; raw credentials kept only in test memory. Exact matrices are executable in the mapped tests.

Exact steps: 1. Check valid JWT subject/issuer/expiry. 2. GET /auth/me with bad signature, wrong issuer, expired token, malformed token, missing Bearer, invalid subject UUID. 3. Use signed arbitrary role claim for active SCOUT. 4. Promote isolated DB role to ADMIN then demote with same old token; call /admin/bookings/<unknown-fixture-UUID>. 5. Set inactive and retry /me.

Expected UI behavior: Client role cache can be stale; backend denies according to DB. No frontend privilege claim constitutes backend authorization.

Expected API/backend behavior: Invalid signature/issuer/expiry/subject/header401 safe. Signed role claim does not override DB. Admin detail403 as SCOUT,404 as authorized DB ADMIN for nonexistent booking,403 after demotion; /me200 reflects ADMIN after promotion. Separate admin-list500 is AUTH-D02. Inactive /me401.

Expected database/domain state: No token rotation/user mutation from reads. Only explicit test setup changes role/active; never authorization through claim text.

Expected email/provider behavior: No email.

Security/authorization expectation: Signature/issuer/expiry + current DB user/role/active are authoritative.

Cleanup/reset: Close/sign out isolated browser; discard raw secrets; retain RUN_ID rows until deliberate guarded local full reset. Testcontainers discards its own database. Never delete seeded owners, disable FKs or restore a nonexistent snapshot.

Automation recommendation: AuthSessionIntegrationTest.jwtValidationAndDatabaseAuthority. Execute rather than infer PASS from source. Human review concerns clarity/usability only; actual provider delivery is staging only.

Local/staging requirement: LOCAL REAL POSTGRES; browser at localhost:3000 with backend localhost:8080/api. Provider-disabled/mocked locally; no real email required for deterministic assertions.

Source/evidence: AuthController; AuthService; AccountRecoveryService; JwtService/JwtAuthenticationFilter; SecurityConfig/VerifiedUserPolicy; auth DTOs, repositories and V1/V8; frontend AuthPanel/AuthProvider/RoleGuard, token.ts/client.ts and recovery pages. Dated results, exact tests and sanitized artifacts in AUTH execution report.

Pass/fail criteria: LOCAL PASS only after mapped deterministic API/DB/client/browser assertions execute successfully. FAIL for contrary behavior; BLOCKED for missing execution infrastructure. Document characterized architecture limitations separately; staging delivery and human review have their own status.

AUTH-007 — Reload, logout and cross-tab state

Test ID: AUTH-007. **Area:** AUTH. **Priority:** P0. **Test level:** API, INTEGRATION, LOCAL E2E, SECURITY. **Result:** LOCAL PASS (2026-09-10); real delivery STAGING ONLY / NOT RUN where applicable.

Roles involved: SCOUT and OWNER in two same-origin tabs.

Preconditions: Fresh browser profile, two tabs sharing localStorage; unique users.

Test data: Unique `scripts/qa-run-id.sh` email at example.test; fictional QA password, role and name. Independent fixture per destructive/expiry case; raw credentials kept only in test memory. Exact matrices are executable in the mapped tests.

Exact steps: 1. Register/sign in A; load B. 2. Reload both and explicitly compare /me. 3. Sign out A; verify B storage/header update and protected page shows sign-in. 4. Refresh a retained token through API to characterize logout semantics, then discard it.

Expected UI behavior: Role identity and unverified state survive reload; storage event propagates sign-out; RoleGuard renders AuthPanel. No automatic /me call on reload is asserted.

Expected API/backend behavior: /me200 for valid access; captured refresh still works after frontend logout. No server logout/revoke endpoint exists.

Expected database/domain state: Logout does not delete/revoke rows. Separate logins create independent active refresh rows.

Expected email/provider behavior: No logout/login email.

Security/authorization expectation: Clearing local credentials is not server revocation; close isolated profile and discard retained token.

Cleanup/reset: Close/sign out isolated browser; discard raw secrets; retain RUN_ID rows until deliberate guarded local full reset. Testcontainers discards its own database. Never delete seeded owners, disable FKs or restore a nonexistent snapshot.

Automation recommendation: browser role/cross-tab journeys; AuthSessionIntegrationTest.independentSessionsAndRotation. Execute rather than infer PASS from source. Human review concerns clarity/usability only; actual provider delivery is staging only.

Local/staging requirement: LOCAL REAL POSTGRES; browser at localhost:3000 with backend localhost:8080/api. Provider-disabled/mocked locally; no real email required for deterministic assertions.

Source/evidence: AuthController; AuthService; AccountRecoveryService; JwtService/JwtAuthenticationFilter; SecurityConfig/VerifiedUserPolicy; auth DTOs, repositories and V1/V8; frontend AuthPanel/AuthProvider/RoleGuard, token.ts/client.ts and recovery pages. Dated results, exact tests and sanitized artifacts in AUTH execution report.

Pass/fail criteria: LOCAL PASS only after mapped deterministic API/DB/client/browser assertions execute successfully. FAIL for contrary behavior; BLOCKED for missing execution infrastructure. Document characterized architecture limitations separately; staging delivery and human review have their own status.

AUTH-008 — Refresh valid, expired, reused and revoked

Test ID: AUTH-008. **Area:** AUTH. **Priority:** P0. **Test level:** API, INTEGRATION, LOCAL E2E, SECURITY , CONCURRENCY. **Result:** LOCAL PASS (2026-09-10); real delivery STAGING ONLY / NOT RUN where applicable.

Roles involved: Public refresh-token holder.

Preconditions: Generated user and captured raw token in test memory; separate fixtures for expired/revoked/inactive cases.

Test data: Unique `scripts/qa-run-id.sh` email at example.test; fictional QA password, role and name. Independent fixture per destructive/expiry case; raw credentials kept only in test memory. Exact matrices are executable in the mapped tests.

Exact steps: 1. Register and login to create sessions A/B. 2. Refresh A; retry old A. 3. Verify B remains usable. 4. Test revoked, expired, unknown, blank and inactive tokens. 5. Race two HTTP refresh calls sharing one raw token.

Expected UI behavior: Client receives/stores current returned session; races covered separately in AUTH-010.

Expected API/backend behavior: Valid200 new token pair; old/revoked/expired/unknown401; blank400; inactive403. Concurrent HTTP rotation yields one200/one401. Refresh expiry compares isBefore(wall time), not injectable Clock equality.

Expected database/domain state: Old A revoked, new A active, B untouched until deliberately rotated. Failure adds no row. Rotation transaction/row lock authoritative.

Expected email/provider behavior: No email.

Security/authorization expectation: Refresh endpoint public but opaque credential required; hash only in PostgreSQL.

Cleanup/reset: Close/sign out isolated browser; discard raw secrets; retain RUN_ID rows until deliberate guarded local full reset. Testcontainers discards its own database. Never delete seeded owners, disable FKs or restore a nonexistent snapshot.

Automation recommendation: AuthSessionIntegrationTest.independentSessionsAndRotation / refreshFailureMatrix / concurrentHttpRefreshHasOneWinner; existing SQL race. Execute rather than infer PASS from source. Human review concerns clarity/usability only; actual provider delivery is staging only.

Local/staging requirement: LOCAL REAL POSTGRES; browser at localhost:3000 with backend localhost:8080/api. Provider-disabled/mocked locally; no real email required for deterministic assertions.

Source/evidence: AuthController; AuthService; AccountRecoveryService; JwtService/JwtAuthenticationFilter; SecurityConfig/VerifiedUserPolicy; auth DTOs, repositories and V1/V8; frontend AuthPanel/AuthProvider/RoleGuard, token.ts/client.ts and recovery pages. Dated results, exact tests and sanitized artifacts in AUTH execution report.

Pass/fail criteria: LOCAL PASS only after mapped deterministic API/DB/client/browser assertions execute successfully. FAIL for contrary behavior; BLOCKED for missing execution infrastructure. Document characterized architecture limitations separately; staging delivery and human review have their own status.

AUTH-009 — Frontend refresh failure and one retry

Test ID: AUTH-009. **Area:** AUTH. **Priority:** P0. **Test level:** API, INTEGRATION, LOCAL E2E, SECURITY. **Result:** LOCAL PASS (2026-09-10); real delivery STAGING ONLY / NOT RUN where applicable.

Roles involved: Authenticated SCOUT browser/client.

Preconditions: Valid session; intercept protected GET only or supply test-signed expired access JWT; keep refresh raw token exact.

Test data: Unique `scripts/qa-run-id.sh` email at example.test; fictional QA password, role and name. Independent fixture per destructive/expiry case; raw credentials kept only in test memory. Exact matrices are executable in the mapped tests.

Exact steps: 1. Protected request with expired JWT and valid refresh: expect one refresh and retry200. 2. Return second401 after successful refresh. 3. Independently make refresh fail401, network/offline, malformed JSON, missing required response fields. 4. Count original attempts/refresh calls and inspect session clearing. 5. Return a required OWNER read403 before, during and after another refresh. Verify the coalesced `/auth/me` check where needed, new-generation retry once, loaded owner page without reload, and no ADMIN copy. 6. With unchanged valid session, return real ownership403: preserve forbidden state; no resource retry. 7. Logout or switch account during a read: its old success must not be returned to the page.

Expected UI behavior: Session cleared for failed/malformed refresh only when that generation is still current; protected UX shows sign-in. A newer generation from another tab is preserved and adopted. Second401 retention is a documented limitation, not a PASS for auto-signout.

Expected API/backend behavior: One refresh and at most one retry per request. Required GET/HEAD403 may join recovery or make one coalesced `/auth/me` check; only generation change allows resource replay, never a403-based mutation replay. Failed refresh clears its still-current session; original401 surfaced. A superseding session is adopted and the original request retries once. Second401 after successful refresh is surfaced with newly stored session retained, per current code.

Expected database/domain state: Real success rotates one refresh row; intercepted failures have no backend mutation. Distinguish simulated transport from real PostgreSQL proof.

Expected email/provider behavior: No email.

Security/authorization expectation: No token contents in network evidence; per-request retry bounded.

Cleanup/reset: Close/sign out isolated browser; discard raw secrets; retain RUN_ID rows until deliberate guarded local full reset. Testcontainers discards its own database. Never delete seeded owners, disable FKs or restore a nonexistent snapshot.

Automation recommendation: scripts/qa/auth-client-check.cjs; browser expired-access refresh; backend refresh tests. Execute rather than infer PASS from source. Human review concerns clarity/usability only; actual provider delivery is staging only.

Local/staging requirement: LOCAL REAL POSTGRES; browser at localhost:3000 with backend localhost:8080/api. Provider-disabled/mocked locally; no real email required for deterministic assertions.

Source/evidence: AuthController; AuthService; AccountRecoveryService; JwtService/JwtAuthenticationFilter; SecurityConfig/VerifiedUserPolicy; auth DTOs, repositories and V1/V8; frontend AuthPanel/AuthProvider/RoleGuard, token.ts/client.ts and recovery pages. Dated results, exact tests and sanitized artifacts in AUTH execution report.

Pass/fail criteria: LOCAL PASS only after mapped deterministic API/DB/client/browser assertions execute successfully. FAIL for contrary behavior; BLOCKED for missing execution infrastructure. Document characterized architecture limitations separately; staging delivery and human review have their own status.

AUTH-010 — Same-tab versus cross-tab refresh race

Test ID: AUTH-010. **Area:** AUTH. **Priority:** P0. **Test level:** API, INTEGRATION, LOCAL E2E, SECURITY , CONCURRENCY. **Result:** LOCAL PASS (2026-09-10); real delivery STAGING ONLY / NOT RUN where applicable.

Roles involved: Two tabs sharing one account session.

Preconditions: Browser/client harness with controlled response ordering; disposable account.

Test data: Unique `scripts/qa-run-id.sh` email at example.test; fictional QA password, role and name. Independent fixture per destructive/expiry case; raw credentials kept only in test memory. Exact matrices are executable in the mapped tests.

Exact steps: 1. Five concurrent protected GETs in one tab all initially401. 2. Count one shared refresh and five retries. 3. In two tabs capture same original session and issue expired-token requests simultaneously. 4. Force winner response before loser response; record shared storage after losing401.

Expected UI behavior: The losing tab preserves the newer stored session, synchronizes its authenticated state and retries the original request at most once. A failed current generation still clears the session. Both rotations need not succeed.

Expected API/backend behavior: Same-tab one refresh promise. Cross-tab two refresh attempts may race; PostgreSQL has one winner. No inter-tab mutex exists. The client compares non-secret generation IDs with current storage before clearing; a stale failure adopts the newer session. A stale success must not overwrite a newer login or resurrect logout.

Expected database/domain state: One old revoked and one replacement active after same-token race; losing request creates no replacement.

Expected email/provider behavior: No email.

Security/authorization expectation: Characterization does not approve architecture; private-alpha decision on cross-tab logout behavior required.

Cleanup/reset: Close/sign out isolated browser; discard raw secrets; retain RUN_ID rows until deliberate guarded local full reset. Testcontainers discards its own database. Never delete seeded owners, disable FKs or restore a nonexistent snapshot.

Automation recommendation: auth-client-check.cjs independent contexts; browser two-tab real refresh race; concurrentHttpRefreshHasOneWinner. Execute rather than infer PASS from source. Human review concerns clarity/usability only; actual provider delivery is staging only.

Local/staging requirement: LOCAL REAL POSTGRES; browser at localhost:3000 with backend localhost:8080/api. Provider-disabled/mocked locally; no real email required for deterministic assertions.

Source/evidence: AuthController; AuthService; AccountRecoveryService; JwtService/JwtAuthenticationFilter; SecurityConfig/VerifiedUserPolicy; auth DTOs, repositories and V1/V8; frontend AuthPanel/AuthProvider/RoleGuard, token.ts/client.ts and recovery pages. Dated results, exact tests and sanitized artifacts in AUTH execution report.

Pass/fail criteria: LOCAL PASS only after mapped deterministic API/DB/client/browser assertions execute successfully. FAIL for contrary behavior; BLOCKED for missing execution infrastructure. Document characterized architecture limitations separately; staging delivery and human review have their own status.

AUTH-011 — Recovery enumeration resistance

Test ID: AUTH-011. **Area:** AUTH. **Priority:** P0. **Test level:** API, INTEGRATION, LOCAL E2E, SECURITY. **Result:** LOCAL PASS (2026-09-10); real delivery STAGING ONLY / NOT RUN where applicable.

Roles involved: Anonymous recovery requester.

Preconditions: Independent active unverified, active verified, inactive, and nonexistent canonical email fixtures.

Test data: Unique `scripts/qa-run-id.sh` email at example.test; fictional QA password, role and name. Independent fixture per destructive/expiry case; raw credentials kept only in test memory. Exact matrices are executable in the mapped tests.

Exact steps: 1. POST resend/forgot for each fixture using padded mixed-case input. 2. Compare status/body across eligibility variants. 3. Count user-specific outstanding token changes and events. 4. Probe unknown properties; inspect hashes.

Expected UI behavior: Forgot form always generic submitted message; resend notice generic. Timing equality not asserted.

Expected API/backend behavior: All valid email requests200 generic body per endpoint. Resend only active+unverified; forgot any active including verified. Invalid body400. No account-existence field or normalized-email echo.

Expected database/domain state: Eligible issuance invalidates prior outstanding same-kind token and creates one hashed token; ineligible case no changes. Password reset expiry60min, verification24h defaults.

Expected email/provider behavior: Eligible verification/reset event only; local mocked/disabled provider. Real deliveries staging only.

Security/authorization expectation: No enumeration beyond current generic response contract; secret values absent from error/log evidence.

Cleanup/reset: Close/sign out isolated browser; discard raw secrets; retain RUN_ID rows until deliberate guarded local full reset. Testcontainers discards its own database. Never delete seeded owners, disable FKs or restore a nonexistent snapshot.

Automation recommendation: AuthRecoveryIntegrationTest.enumerationEligibilityMatrix; browser forgot/resend. Execute rather than infer PASS from source. Human review concerns clarity/usability only; actual provider delivery is staging only.

Local/staging requirement: LOCAL REAL POSTGRES; browser at localhost:3000 with backend localhost:8080/api. Provider-disabled/mocked locally; no real email required for deterministic assertions.

Source/evidence: AuthController; AuthService; AccountRecoveryService; JwtService/JwtAuthenticationFilter; SecurityConfig/VerifiedUserPolicy; auth DTOs, repositories and V1/V8; frontend AuthPanel/AuthProvider/RoleGuard, token.ts/client.ts and recovery pages. Dated results, exact tests and sanitized artifacts in AUTH execution report.

Pass/fail criteria: LOCAL PASS only after mapped deterministic API/DB/client/browser assertions execute successfully. FAIL for contrary behavior; BLOCKED for missing execution infrastructure. Document characterized architecture limitations separately; staging delivery and human review have their own status.

AUTH-012 — Verification token lifecycle

Test ID: AUTH-012. **Area:** AUTH. **Priority:** P0. **Test level:** API, INTEGRATION, LOCAL E2E, SECURITY. **Result:** LOCAL PASS (2026-09-10); real delivery STAGING ONLY / NOT RUN where applicable.

Roles involved: Anonymous verification token holder.

Preconditions: Registration token captured from event in memory or isolated browser token fixture; account initially unverified.

Test data: Unique `scripts/qa-run-id.sh` email at example.test; fictional QA password, role and name. Independent fixture per destructive/expiry case; raw credentials kept only in test memory. Exact matrices are executable in the mapped tests.

Exact steps: 1. Verify valid token. 2. Reuse it after verification. 3. Try random, expired, inactive and replaced token while still unverified. 4. Resend and verify current replacement. 5. Check exact expiry equality with fixed-clock service test.

Expected UI behavior: /verify-email success and header refreshed through /me; missing/invalid/expired safe link message. Token absent from local/sessionStorage.

Expected API/backend behavior: Valid200 marks verified. Consumed token200 idempotent when user already verified (even if subsequently expired, provided active). Invalid/expired/inactive/replaced-before-verification400. No activation of inactive account.

Expected database/domain state: User email_verified true on success; token used_at set and outstanding verification tokens invalidated. Failure preserves unverified state. Raw token never persisted.

Expected email/provider behavior: Issuance/resend notifications only; verification itself sends no completion email in current implementation.

Security/authorization expectation: Possession credential is exact; no ownership UUID parameter.

Cleanup/reset: Close/sign out isolated browser; discard raw secrets; retain RUN_ID rows until deliberate guarded local full reset. Testcontainers discards its own database. Never delete seeded owners, disable FKs or restore a nonexistent snapshot.

Automation recommendation: AuthRecoveryIntegrationTest.verificationLifecycle; AccountRecoveryServiceTest exact boundary; browser verification matrix. Execute rather than infer PASS from source. Human review concerns clarity/usability only; actual provider delivery is staging only.

Local/staging requirement: LOCAL REAL POSTGRES; browser at localhost:3000 with backend localhost:8080/api. Provider-disabled/mocked locally; no real email required for deterministic assertions.

Source/evidence: AuthController; AuthService; AccountRecoveryService; JwtService/JwtAuthenticationFilter; SecurityConfig/VerifiedUserPolicy; auth DTOs, repositories and V1/V8; frontend AuthPanel/AuthProvider/RoleGuard, token.ts/client.ts and recovery pages. Dated results, exact tests and sanitized artifacts in AUTH execution report.

Pass/fail criteria: LOCAL PASS only after mapped deterministic API/DB/client/browser assertions execute successfully. FAIL for contrary behavior; BLOCKED for missing execution infrastructure. Document characterized architecture limitations separately; staging delivery and human review have their own status.

AUTH-013 — Verification UI and mutation enforcement

Test ID: AUTH-013. **Area:** AUTH. **Priority:** P0. **Test level:** API, INTEGRATION, LOCAL E2E, SECURITY. **Result:** LOCAL PASS (2026-09-10); real delivery STAGING ONLY / NOT RUN where applicable.

Roles involved: Unverified SCOUT/OWNER; verified counterpart.

Preconditions: Authenticated accounts, no accepted policies assumed; use valid mutation payloads or no-body endpoints to reach method security. Supply a directly seeded isolated ACTIVE venue for the post-verification policy gate; otherwise a missing venue legitimately returns404 first.

Test data: Unique `scripts/qa-run-id.sh` email at example.test; fictional QA password, role and name. Independent fixture per destructive/expiry case; raw credentials kept only in test memory. Exact matrices are executable in the mapped tests.

Exact steps: 1. Read /me and own lists. 2. Before verification call booking accept/decline/checkout and inquiry accept/decline, owner submission/upload: expect verified-email denial. 3. Verify token; repeat a representative owner submission and a policy-gated scout action. 4. Browser success updates cached user/header; missing/invalid/expired links safe.

Expected UI behavior: Unverified notice visible; verification success refreshes user and removes notice. Policy review remains independent; tokens not stored as frontend session data.

Expected API/backend behavior: Reads200 where role permits. Unverified mutations403 before domain lookup. Verified owner submission may progress to normal trust workflow; email verification does not accept policies. Separate policy gate may still403.

Expected database/domain state: Forbidden mutations no business changes. Verification changes email state only; allowed owner submit creates PENDING trust record in isolated fixture.

Expected email/provider behavior: Verification issue/resend and explicit allowed trust submission may trigger their own events; no automatic financial action.

Security/authorization expectation: ADMIN verified-email bypass follows policy but does not bypass other role/domain rules.

Cleanup/reset: Close/sign out isolated browser; discard raw secrets; retain RUN_ID rows until deliberate guarded local full reset. Testcontainers discards its own database. Never delete seeded owners, disable FKs or restore a nonexistent snapshot.

Automation recommendation: AuthRecoveryIntegrationTest.verifiedMutationBoundary; existing enforcement annotations; browser verification. Execute rather than infer PASS from source. Human review concerns clarity/usability only; actual provider delivery is staging only.

Local/staging requirement: LOCAL REAL POSTGRES; browser at localhost:3000 with backend localhost:8080/api. Provider-disabled/mocked locally; no real email required for deterministic assertions.

Source/evidence: AuthController; AuthService; AccountRecoveryService; JwtService/JwtAuthenticationFilter; SecurityConfig/VerifiedUserPolicy; auth DTOs, repositories and V1/V8; frontend AuthPanel/AuthProvider/RoleGuard, token.ts/client.ts and recovery pages. Dated results, exact tests and sanitized artifacts in AUTH execution report.

Pass/fail criteria: LOCAL PASS only after mapped deterministic API/DB/client/browser assertions execute successfully. FAIL for contrary behavior; BLOCKED for missing execution infrastructure. Document characterized architecture limitations separately; staging delivery and human review have their own status.

AUTH-014 — Password reset end to end

Test ID: AUTH-014. **Area:** AUTH. **Priority:** P0. **Test level:** API, INTEGRATION, LOCAL E2E, SECURITY. **Result:** LOCAL PASS (2026-09-10 rerun); historical BCrypt failure resolved with Argon2id.

Roles involved: Anonymous reset-token holder; authenticated browser session to clear.

Preconditions: Active generated account, multiple refresh sessions, reset token captured only in memory.

Test data: Unique `scripts/qa-run-id.sh` email at example.test; fictional QA password, role and name. Independent fixture per destructive/expiry case; raw credentials kept only in test memory. Exact matrices are executable in the mapped tests.

Exact steps: 1. Forgot password; inspect hashed reset token. 2. Test invalid/expired/used/inactive/replaced tokens. 3. Test new password lengths7/8/72/73 and surrounding spaces. 4. Browser missing link/mismatch/validation then valid submit. 5. Try old/new passwords and all old refresh tokens.

Expected UI behavior: Missing/invalid/expired link safe; mismatch no request; success shows Sign in CTA and clears this browser session (storage sync propagates same-origin logout).

Expected API/backend behavior: Reset200 consumes token, replaces Argon2id password, deletes all refresh records. Old password/refresh401; new login200; reused/invalid/expired/inactive400. Too-short/long400 without consuming valid token. ASCII spaces remain exact; distinct accepted Unicode passwords remain distinct even with the same first72 UTF-8 bytes (resolved pre-launch AUTH-D01 regression).

Expected database/domain state: New hash matches only new password; reset used_at marked/all outstanding invalidated; refresh rows physically deleted. Failures no password/token mutation.

Expected email/provider behavior: Forgot issuance email only; no password-reset-completed email implemented. Actual delivery staging only.

Security/authorization expectation: Reset token not stored client-side except transient URL; do not capture URL in screenshots/logs.

Cleanup/reset: Close/sign out isolated browser; discard raw secrets; retain RUN_ID rows until deliberate guarded local full reset. Testcontainers discards its own database. Never delete seeded owners, disable FKs or restore a nonexistent snapshot.

Automation recommendation: AuthRecoveryIntegrationTest.resetLifecycleAndBoundaries; browser reset matrix. Execute rather than infer PASS from source. Human review concerns clarity/usability only; actual provider delivery is staging only.

Local/staging requirement: LOCAL REAL POSTGRES; browser at localhost:3000 with backend localhost:8080/api. Provider-disabled/mocked locally; no real email required for deterministic assertions.

Source/evidence: AuthController; AuthService; AccountRecoveryService; JwtService/JwtAuthenticationFilter; SecurityConfig/VerifiedUserPolicy; auth DTOs, repositories and V1/V8; frontend AuthPanel/AuthProvider/RoleGuard, token.ts/client.ts and recovery pages. Dated results, exact tests and sanitized artifacts in AUTH execution report.

Pass/fail criteria: LOCAL PASS only after mapped deterministic API/DB/client/browser assertions execute successfully. FAIL for contrary behavior; BLOCKED for missing execution infrastructure. Document characterized architecture limitations separately; staging delivery and human review have their own status.

AUTH-015 — Issued access token after password reset

Test ID: AUTH-015. **Area:** AUTH. **Priority:** P0. **Test level:** API, INTEGRATION, LOCAL E2E, SECURITY. **Result:** LOCAL PASS (2026-09-10); real delivery STAGING ONLY / NOT RUN where applicable.

Roles involved: Holder of previously issued JWT for same active user.

Preconditions: Capture JWT in memory before successful reset; create expired test-signed JWT without changing production expiry.

Test data: Unique `scripts/qa-run-id.sh` email at example.test; fictional QA password, role and name. Independent fixture per destructive/expiry case; raw credentials kept only in test memory. Exact matrices are executable in the mapped tests.

Exact steps: 1. Reset password and verify old refresh invalid. 2. Call /auth/me using pre-reset unexpired access JWT. 3. Repeat with expired equivalent. 4. Deactivate fixture and repeat old unexpired JWT.

Expected UI behavior: Reset browser clears its own credentials; independent captured JWT remains usable until expiry. This is an architectural security observation requiring explicit staging acceptance.

Expected API/backend behavior: Pre-reset unexpired JWT still200 while active. Expired/inactive401. Reset does not track JWT/password version; no immediate access-token revocation claim.

Expected database/domain state: Reset deletes refresh rows; old JWT request does not recreate them. Account active flag independently enforced.

Expected email/provider behavior: No extra email.

Security/authorization expectation: Report current architecture without silently changing JWT/session design.

Cleanup/reset: Close/sign out isolated browser; discard raw secrets; retain RUN_ID rows until deliberate guarded local full reset. Testcontainers discards its own database. Never delete seeded owners, disable FKs or restore a nonexistent snapshot.

Automation recommendation: AuthRecoveryIntegrationTest.resetLifecycleAndBoundaries; AuthSessionIntegrationTest.jwtValidationAndDatabaseAuthority. Execute rather than infer PASS from source. Human review concerns clarity/usability only; actual provider delivery is staging only.

Local/staging requirement: LOCAL REAL POSTGRES; browser at localhost:3000 with backend localhost:8080/api. Provider-disabled/mocked locally; no real email required for deterministic assertions.

Source/evidence: AuthController; AuthService; AccountRecoveryService; JwtService/JwtAuthenticationFilter; SecurityConfig/VerifiedUserPolicy; auth DTOs, repositories and V1/V8; frontend AuthPanel/AuthProvider/RoleGuard, token.ts/client.ts and recovery pages. Dated results, exact tests and sanitized artifacts in AUTH execution report.

Pass/fail criteria: LOCAL PASS only after mapped deterministic API/DB/client/browser assertions execute successfully. FAIL for contrary behavior; BLOCKED for missing execution infrastructure. Document characterized architecture limitations separately; staging delivery and human review have their own status.

POL scenarios

POL-001 — Policy draft, publish, replacement and retire

Area: POL · **Priority:** P0 · **Test level:** API, PostgreSQL, browser where mapped · **Result:** LOCAL TECHNICAL PASS — see POL execution report

Roles involved: ADMIN creates/lists/publishes/retires. SCOUT and OWNER authenticate for current/status reads; they cannot administer policies. Anonymous is denied.

Preconditions: Disposable PostgreSQL migrated through V11; active authenticated ADMIN, SCOUT and OWNER. Policy endpoints do not require email verification or prior acceptance. Begin with no current policies in the disposable database.

Test data: Unique QA-<RUN_ID>-v1/v2 versions; every test policy says `QA TEST CONTENT — NOT LEGAL TERMS`.

Exact steps: For each of the five PolicyType values, POST /api/admin/policies with unique QA-<RUN_ID>-v1, exact plain QA text and future effectiveAt. Confirm draft is absent from GET /api/policies/{type}/current (404). POST /api/admin/policies/{id}/publish (200); create/publish v2; retire v2 (200). Create returns 201. Check current/status after each. Repeat admin list/create/publish/retire as SCOUT and OWNER (403), and anonymous (401). Denials leave state unchanged.

Expected API/UI behavior: Publication is immediate, including future effectiveAt. No scheduler. Replacement retires the previous published version. Retiring the current version leaves that type absent and completeness false. ADMIN needs no prior acceptance. Browser admin controls require confirmation for publication/retirement.

Expected database/domain state: Persist exact content, SHA-256 of its UTF-8 bytes, and non-null published_at; retain v1 content/hash after replacement and retirement. At most one PUBLISHED row per type (partial unique index). Hash is integrity metadata, not a digital signature.

Email/provider contract: AdminPolicyService and PolicyAcceptanceService publish no notification events or provider calls. Other domains retain their own lifecycle behavior.

Cleanup/reset: Testcontainers owns disposable automation databases. Browser suite uses a separate disposable local QA database and temporary Chrome profile. Retain history until disposing that whole QA environment; never manually delete published shared local/staging policies or assume a snapshot-restore harness.

Automation mapping: PolicyIntegrationTest.lifecycleAllFiveAndAuthorization; lifecycleAndAcceptancePublishNoDomainNotificationEvents; concurrentServicePublicationHasOneCurrentAndSafeConflict. Existing PostgresCriticalPathIntegrationTest.policyPublishRaceLeavesExactlyOneCurrentVersion. scripts/qa/policy-browser.cjs admin create/publish/replace/retire/reload.

Human/legal limitation: Production publication requires externally approved content. Local QA text is not legal approval.

Execution evidence: POL execution report, `docs/qa/evidence/policy/`. PASS requires mapped deterministic checks to execute successfully; legal approval remains separate. No H2, production fake policies, or live provider calls.

POL-002 — Policy immutability and unsupported edit

Area: POL · **Priority:** P0 · **Test level:** API, PostgreSQL, browser where mapped · **Result:** LOCAL TECHNICAL PASS — see POL execution report

Roles involved: ADMIN exercises unsupported methods and transitions. Global policy versions have no participant ownership variants.

Preconditions: Authenticated active ADMIN; disposable draft, published and retired versions. No email-verification or policy-acceptance prerequisite.

Test data: Unique QA-<RUN_ID>-v1/v2 versions; every test policy says `QA TEST CONTENT — NOT LEGAL TERMS`.

Exact steps: At each lifecycle state, attempt PUT/PATCH /api/admin/policies (405) and /api/admin/policies/{id} (404: no resource mapping). Attempt republish PUBLISHED, publish RETIRED, retire DRAFT and duplicate type/version (409). Exercise blank/overlong version/title/content, invalid type/date (400). Unknown fields currently ignored. Do not invent draft-edit or internal entity-update methods.

Expected API/UI behavior: No content edit endpoint exists, including drafts. Published/retired cards are read-only; replacement is a new draft. Unsupported requests are client errors, never 500. Effective date/title/version are not mutable via an API either.

Expected database/domain state: Content/hash on original rows remain stable. Type/version uniqueness and historical foreign keys remain enforced. Application immutability does not claim a database trigger that prohibits privileged manual SQL changes.

Email/provider contract: AdminPolicyService and PolicyAcceptanceService publish no notification events or provider calls. Other domains retain their own lifecycle behavior.

Cleanup/reset: Testcontainers owns disposable automation databases. Browser suite uses a separate disposable local QA database and temporary Chrome profile. Retain history until disposing that whole QA environment; never manually delete published shared local/staging policies or assume a snapshot-restore harness.

Automation mapping: PolicyIntegrationTest.immutableTransitionsAndRequestContract; lifecycleAllFiveAndAuthorization; replacementGatesCommitmentsAndPreservesBookingRightsAndSnapshots. Browser admin historical rows have no input/select/textarea editor.

Human/legal limitation: No legal decision is needed to validate technical immutability. Approved version/title conventions remain business input.

Execution evidence: POL execution report, `docs/qa/evidence/policy/`. PASS requires mapped deterministic checks to execute successfully; legal approval remains separate. No H2, production fake policies, or live provider calls.

POL-003 — Exact acceptance IDs and duplicates

Area: POL · **Priority:** P0 · **Test level:** API, PostgreSQL, browser where mapped · **Result:** LOCAL TECHNICAL PASS — see POL execution report

Roles involved: Authenticated SCOUT, OWNER and ADMIN can read and accept; anonymous receives 401. Acceptance is tied to the authenticated user, with no userId in the request.

Preconditions: Active authenticated accounts; no prior acceptance or email-verification prerequisite. Publish all five current types using isolated QA versions.

Test data: Unique QA-<RUN_ID>-v1/v2 versions; every test policy says `QA TEST CONTENT — NOT LEGAL TERMS`.

Exact steps: POST /api/policies/accept with {policyVersionIds:[UUID,...]}. Accept exact five (200), repeat (200), accept a subset (200, incomplete), and duplicate IDs (deduplicated Set). Empty, more than five distinct IDs, null ID, nonexistent, draft, retired or stale previous ID are 400. Distinguish API subset support from the UI, which requires selecting all currently unaccepted cards.

Expected API/UI behavior: All five types are required for status completeness: TERMS_OF_SERVICE, PRIVACY_POLICY, ACCEPTABLE_USE_POLICY, PRODUCTION_POLICY, INSURANCE_REQUIREMENTS. ADMIN can accept and receives normal completeness status but bypasses commitment enforcement. Missing publication cannot mean complete, even if all available versions were accepted.

Expected database/domain state: One acceptance per user/version; repeated acknowledgement preserves original row, timestamp and request metadata. Failed requests create no partial set. Booking snapshot set has four types, excluding PRIVACY_POLICY. No acceptance history is deleted when a replacement appears.

Email/provider contract: AdminPolicyService and PolicyAcceptanceService publish no notification events or provider calls. Other domains retain their own lifecycle behavior.

Cleanup/reset: Testcontainers owns disposable automation databases. Browser suite uses a separate disposable local QA database and temporary Chrome profile. Retain history until disposing that whole QA environment; never manually delete published shared local/staging policies or assume a snapshot-restore harness.

Automation mapping: PolicyIntegrationTest.exactAcceptanceSubsetsDuplicatesAndMissingTypes; duplicateConcurrentAcceptanceCommitsBothRequestsWithoutPartialSet (three barrier repetitions); lifecycleAndAcceptancePublishNoDomainNotificationEvents. Existing PolicyAcceptanceServiceTest plus browser persisted subset/all checks.

Human/legal limitation: Privacy wording remains review/acknowledgement, not invented consent. Whether role-specific required sets should differ is a business/legal decision.

Execution evidence: POL execution report, `docs/qa/evidence/policy/`. PASS requires mapped deterministic checks to execute successfully; legal approval remains separate. No H2, production fake policies, or live provider calls.

POL-004 — Policy replacement gating and history

Area: POL · **Priority:** P0 · **Test level:** API, PostgreSQL, browser where mapped · **Result:** LOCAL TECHNICAL PASS — see POL execution report

Roles involved: SCOUT/OWNER act on their eligible resources; ADMIN publishes replacement. Resource ownership matters for bookings/venues/inquiries, not global policy IDs.

Preconditions: Current v1 accepted by SCOUT and OWNER; verified email where independently required. Real created booking with four v1 references; active venue and valid availability. Paid-receipt/refund fixtures must independently meet financial eligibility. Payout form additionally requires verified owner status.

Test data: Unique QA-<RUN_ID>-v1/v2 versions; every test policy says `QA TEST CONTENT — NOT LEGAL TERMS`.

Exact steps: Publish replacement after v1 acceptance. Verify authenticated booking list/detail, incident history, cancellation preview, eligible cancellation/refund status, PDF receipt and owner payout history remain available; login and public browsing work. Attempt new booking, checkout, owner booking accept/decline, inquiry create/offer/decline/accept, payout-destination submit and venue submit-for-review: policy-specific 403. Accept replacement and retry new booking/owner decline; browser retries payout submission successfully. Replace/retire a booking-relevant type and compare original references.

Expected API/UI behavior: New commitments are gated; historical rights retain independent role/eligibility rules. Direct full refund initiation remains ADMIN-only; users retain eligible cancellation and refund-status access. Existing incident reads are ungated. No publication push/poll is implied: backend always checks new mutations; 403 dispatches eema:policy-stale and refreshes frontend policy status without replaying mutation.

Expected database/domain state: Original booking_policy_versions rows continue to point to original four version IDs after replacement and retirement. Privacy is absent. Denied actions leave aggregate state unchanged. Paid cancellation/refund is tested with a mocked provider response and real PostgreSQL persistence; this is not provider delivery verification.

Email/provider contract: AdminPolicyService and PolicyAcceptanceService publish no notification events or provider calls. Other domains retain their own lifecycle behavior.

Cleanup/reset: Testcontainers owns disposable automation databases. Browser suite uses a separate disposable local QA database and temporary Chrome profile. Retain history until disposing that whole QA environment; never manually delete published shared local/staging policies or assume a snapshot-restore harness.

Automation mapping: PolicyIntegrationTest.replacementGatesCommitmentsAndPreservesBookingRightsAndSnapshots; scripts/qa/policy-browser.cjs OWNER stale form/403/review/retry; scripts/qa/policy-client-check.cjs repeated policy-versus-ownership 403 distinction. Existing financial/cancellation/inquiry suites remain additional coverage.

Human/legal limitation: Policy services themselves emit no email; successful booking/cancellation/payout actions can emit their own existing events. Do not apply generic provider-silence expectations to those actions.

Execution evidence: POL execution report, `docs/qa/evidence/policy/`. PASS requires mapped deterministic checks to execute successfully; legal approval remains separate. No H2, production fake policies, or live provider calls.

POL-005 — Privacy wording and missing-set UI

Area: POL · **Priority:** P0 · **Test level:** API, PostgreSQL, browser where mapped · **Result:** LOCAL TECHNICAL PASS / LEGAL BLOCKED — see POL execution report

Roles involved: Authenticated SCOUT and OWNER policy centre; ADMIN prepares isolated versions and exercises admin UI. Anonymous sees sign-in and API reads return 401.

Preconditions: Disposable local backend and Chrome QA profile. Exercise no publications, one missing type, full set, persisted subset, all accepted, and replacement. Email verification is not required to review policies.

Test data: Unique QA-<RUN_ID>-v1/v2 versions; every test policy says `QA TEST CONTENT — NOT LEGAL TERMS`.

Exact steps: Open /policies for each set. Compare exact backend content with rendered text, including literal markup characters; assert no interpreted HTML. No unchecked version may be preselected. API subset acceptance is reflected as reviewed cards; UI selection of only part of the remaining set leaves submit disabled. Accept all remaining controls, reload, replace one policy, and verify explicit review is required again. On /admin/policies create/review draft, confirm publish, replace and retire; reload retains read-only history.

Expected API/UI behavior: No placeholder legal text is substituted. A missing type shows the incomplete required-set notice and must not show all-required-reviewed success. Backend allRequiredAccepted controls success, together with the complete displayed set. Privacy card says “I acknowledge that I have reviewed this privacy notice.” Plain React text rendering does not interpret HTML.

Expected database/domain state: UI acceptance must agree with exact-version records and backend completeness. No policy content is generated by the UI. Legal validity is not a database/software assertion.

Email/provider contract: AdminPolicyService and PolicyAcceptanceService publish no notification events or provider calls. Other domains retain their own lifecycle behavior.

Cleanup/reset: Testcontainers owns disposable automation databases. Browser suite uses a separate disposable local QA database and temporary Chrome profile. Retain history until disposing that whole QA environment; never manually delete published shared local/staging policies or assume a snapshot-restore harness.

Automation mapping: scripts/qa/policy-browser.cjs (existing AUTH CDP/ws harness, no new framework); sanitized screenshots and browser.json under docs/qa/evidence/policy. PolicyIntegrationTest covers authoritative completeness and accepted flags.

Human/legal limitation: LEGAL REVIEW REQUIRED: final content and privacy acknowledgement semantics remain unapproved. Technical UI results cannot turn this legal gate into PASS. Eli reviews understandability, review notices and deliberate admin publication.

Execution evidence: POL execution report, `docs/qa/evidence/policy/`. PASS requires mapped deterministic checks to execute successfully; legal approval remains separate. No H2, production fake policies, or live provider calls.

POL-006 — Acceptance publication race

Area: POL · **Priority:** P0 · **Test level:** API, PostgreSQL, browser where mapped · **Result:** LOCAL TECHNICAL PASS — see POL execution report

Roles involved: Authenticated accepting SCOUT (same production acceptance service as OWNER/ADMIN); ADMIN publishes. No unrelated ownership UUID variants.

Preconditions: Disposable Testcontainers PostgreSQL, V11 constraints, current published set and fresh unaccepted user. Transaction barriers/latches; no timing sleeps. Separate publication/acceptance race, duplicate acceptance race and competing publications.

Test data: Unique QA-<RUN_ID>-v1/v2 versions; every test policy says `QA TEST CONTENT — NOT LEGAL TERMS`.

Exact steps: Pause acceptance after its real current-set query, publish replacement, release acceptance. Assert old exact acceptances may persist; next policy gate rejects until replacement accepted. Force two identical HTTP acceptance requests past their initial absence check; both return 200 with one row per user/version and complete sets. Force two different drafts past the current-publication read; one publishes and the other safely conflicts (200/409), leaving one current version. Repeat each race three times.

Expected API/UI behavior: Acceptance records valid exact historical versions, not a perpetual complete flag. Completeness is recalculated for the next gated action. Concurrent duplicates use PostgreSQL ON CONFLICT DO NOTHING, without swallowing a uniqueness exception in an aborted transaction. Inserts have consistent version order. Competing publications retain the partial unique-index authority.

Expected database/domain state: No duplicate acceptance rows, no partial committed set, no rewritten/deleted stale history, no duplicate current publication. A losing publication transaction rolls back; losing draft remains draft. The existing SQL-only race supplements, but does not replace, service/HTTP transaction evidence.

Email/provider contract: AdminPolicyService and PolicyAcceptanceService publish no notification events or provider calls. Other domains retain their own lifecycle behavior.

Cleanup/reset: Testcontainers owns disposable automation databases. Browser suite uses a separate disposable local QA database and temporary Chrome profile. Retain history until disposing that whole QA environment; never manually delete published shared local/staging policies or assume a snapshot-restore harness.

Automation mapping: PolicyIntegrationTest.publicationDuringAcceptanceKeepsHistoryButNextGateRejects; duplicateConcurrentAcceptanceCommitsBothRequestsWithoutPartialSet; concurrentServicePublicationHasOneCurrentAndSafeConflict (each repeated three times). Existing PostgresCriticalPathIntegrationTest SQL publication race.

Human/legal limitation: No manual concurrency or row-count work for Eli. No software race test approves legal content.

Execution evidence: POL execution report, `docs/qa/evidence/policy/`. PASS requires mapped deterministic checks to execute successfully; legal approval remains separate. No H2, production fake policies, or live provider calls.

TRUST scenarios

TRUST-001 — Owner review lifecycle, audit and first-submission concurrency

Area: TRUST · **Priority:** P0 · **Test level:** HTTP, PostgreSQL, browser · **Result:** PASS (local software; staging/operations follow-ups separate)

Actual actors/prerequisites: Active OWNER; current DB email verification required for submission, not GET. ADMIN reviews; no admin email-verification prerequisite. Neither actor needs policy acceptance. Use unique RUN_ID fictional accounts; email delivery disabled locally.

Exact API journey: GET /api/owner/verification synthesizes UNVERIFIED with null ID before first submission without creating a row. POST /api/owner/verification/submit with optional QA note returns 200 PENDING. Duplicate PENDING submission returns 200 unchanged. ADMIN POST /api/admin/owner-verifications/{id}/reject with reason returns 200 REJECTED. Owner resubmits (200 PENDING, same ID); admin approves with MANUAL_REVIEW (200 VERIFIED).

Persistence/audit: One row per owner. Initial submission records ownerNote/submittedAt and version 0. Duplicate changes neither note nor timestamps/version and emits no event. Rejection records reviewer/reviewedAt/trimmed owner-visible reason, no method. Resubmission replaces ownerNote/submittedAt and clears all review fields. Approval records reviewer/time/method/optional note. Audit sequence SUBMITTED, REJECTED, SUBMITTED, APPROVED retains correct subject and OWNER/ADMIN actor; only rejection carries the reason. Current entity fields do not preserve previous owner/approval notes as separate audit evidence.

Browser: Owner UNVERIFIED → PENDING → reload → owner-visible rejection → resubmit → VERIFIED → reload. Admin queue/detail, explicit reject/approve confirmation. Owner consumes Spring's returned DTO; admin reloads from Spring. No locally invented status.

Notifications: Every legitimate submission/resubmission, rejection and approval emits an asynchronous AFTER_COMMIT lifecycle notification. Disabled EmailService skips delivery; listener failure cannot roll back the committed transition. Duplicate pending and denied/invalid commands emit none. Real Resend delivery remains staging-only.

Concurrency: Barrier-coordinated simultaneous first submissions use real PostgreSQL: both 200, same ID, one PENDING row and one submission audit/notification. Lock the existing owner before looking up the potentially absent verification. Reject/resubmit race must serialize to a valid lifecycle without mixed review fields.

Automation: OwnerTrustIntegrationTest lifecycle, simultaneousFirstSubmissionIsOneCommittedLifecycle, rejectionVersusResubmissionHasOnlyLegalOrderedOutcomes, notificationRunsAfterCommitAndFailureCannotRollBackTrust; TrustNotificationServiceTest and EmailServiceTest; scripts/qa/trust-browser.cjs.

Cleanup/limits: Dispose Testcontainers DB and dedicated browser QA DB/profile; discard session credentials. Never delete shared trust history or restore a fictional snapshot. Operations must define manual review criteria/SLA; software execution does not approve them. See docs/qa/TRUST_EXECUTION_REPORT.md and docs/qa/evidence/trust/.

TRUST-002 — Actor, transition and trust-boundary enforcement

Area: TRUST · **Priority:** P0 · **Test level:** HTTP, PostgreSQL, browser · **Result:** PASS (local software; staging/operations follow-ups separate)

Actual actors/prerequisites: OWNER owns the implicit current-user endpoint; no owner UUID parameter exists. ADMIN alone lists/gets/reviews admin records. Fixtures include verified/unverified OWNER, SCOUT, ADMIN and anonymous; no policy prerequisite.

Authorization: Owner GET: OWNER 200 including unverified email; SCOUT/ADMIN 403; anonymous 401. Owner submit: verified active OWNER 200, unverified OWNER/SCOUT/ADMIN 403, anonymous/inactive 401. Admin list/detail/approve/reject: ADMIN allowed, OWNER/SCOUT 403, anonymous 401. Verify controller and global /admin security. Current DB email/active/role state controls authorization even with an older token.

Transitions: VERIFIED submit is 409 and preserves review method/time. Only PENDING can be approved/rejected; VERIFIED, REJECTED and persisted schema-valid UNVERIFIED return 409. Logical no-row UNVERIFIED has no review ID; a nonexistent UUID returns 404. Repeat review is 409. Denied/invalid calls leave row/version/audit/notifications unchanged.

Separation/privacy: Approval must leave existing DRAFT/PENDING venues unchanged and create no payout destination/payout or identity/bank evidence. Owner DTO contains id, ownerId, status, submittedAt, reviewedAt, reviewNote, verificationMethod; no reviewer ID or private operations fields. Admin DTO adds ownerNote, reviewedBy, createdAt/updatedAt. Queue filters status/ownerId and caps page size at 100.

Concurrency/browser: Real PostgreSQL approve/approve and approve/reject races yield one 200 and one 409; exactly one consistent review/audit wins. Browser wrong-role guards, missing review, safe validation/session/network errors; stale admin decision refetches authoritative state and keeps actionable conflict text visible after PENDING controls disappear.

Automation: OwnerTrustIntegrationTest currentDbEmailRoleAndAdminSecurityAreAuthoritative, onlyPendingCanBeReviewedAndApprovalHasNoVenueOrBankSideEffects, queueFiltersProjectionAndDatabaseConstraints, competingAdminReviewsHaveOneWinner; existing security annotations/AUTH current-user tests; scripts/qa/trust-browser.cjs.

Cleanup/limits: Unique accounts in disposable PostgreSQL, isolated browser session; no deletion of shared audit history. No revoke/downgrade operation exists. Any future revocation or reviewer-note visibility change requires an operations/product decision.

TRUST-003 — Manual methods, input boundaries and neutral wording

Area: TRUST · **Priority:** P0 · **Test level:** parameterized HTTP/PostgreSQL, browser · **Result:** PASS (local software; staging/operations follow-ups separate)

Actual actors/prerequisites: Verified-email OWNER submits; ADMIN approves an independent PENDING owner per method. No policy acceptance prerequisite.

Exact contract: Supported required approval method: MANUAL_REVIEW, VIDEO_CALL, BUSINESS_DOCUMENT, VENUE_CONTROL_EVIDENCE. Each persists/returns exactly. Missing/null/unknown method returns 400 with no transition. Submit and approval notes are optional: null/empty/blank normalize to null; 1/1000 characters allowed, 1001 rejected (Java string length before trimming). Rejection reason is required nonblank, ≤1000; missing/null/empty/whitespace/1001 return 400, valid/1000 persist trimmed. Invalid input creates no audit/notification or partial review.

Unknown fields: Record DTO binding safely ignores unknown properties; injected ownerId/status/reviewedAt cannot assign entity state. Do not impose unrelated AUTH strict-JSON rules.

UI/semantics: Optional note and required reason/method controls enforce documented bounds; backend remains authoritative. Labels describe manual Eema review, not regulated KYC/KYB, government identity verification or bank verification. VENUE_CONTROL_EVIDENCE records a manual method without activating a venue. No sensitive evidence upload is supported; do not collect or attach real evidence for QA.

Notifications/storage: Valid transitions retain TRUST-001 behavior; invalid inputs emit none. Rejection text is escaped in HTML email. Database stores notes/method/timestamps and reviewer FK, not passport/face/business-document/bank files.

Automation: OwnerTrustIntegrationTest parameterized methods/submission/approval boundaries, invalidMethodsReasonsAndUnknownFieldsCannotAssignState; TrustNotificationServiceTest HTML escaping; scripts/qa/trust-browser.cjs actual form/review lifecycle.

Cleanup/limits: Dispose unique fictional QA fixtures with the whole dedicated DB/profile. Never delete shared trust history. Operations must define what each method proves, reviewer checklist, retention, SLA and owner-visible notes. No software PASS constitutes approval of an identity/KYC policy.

VEN scenarios

VEN-001 — Create an incomplete owner draft

Area: VEN · **Priority:** P1 · **Test level:** HTTP, PostgreSQL, browser · **Result:** PASS — local deterministic contract; human/staging/operations limits remain

Actors/prerequisites: Verified-email OWNER creates own venue; OWNER without owner verification/policies is eligible. SCOUT/ADMIN 403, unverified OWNER 403, anonymous 401.

Execution: POST /api/venues with minimum required name, neighbourhood, sizeM2, hourlyRateEur, minimumBookingMinutes, minimumLeadTimeMinutes and tags. Expect 201 DRAFT with server-assigned owner. Injected ownerId/status/reviewedAt/photoPaths/timezone cannot assign fields absent from CreateVenueRequest. Reload own detail and /venues/my.

Persistence/behavior: Nullable production facts remain unknown; empty photos and Europe/Amsterdam come from entity defaults. No weekly hours, overrides, moderation event or notification is created. Creating a draft requires neither policy acceptance nor owner review.

Automation: VenueDomainIntegrationTest.creationAuthorizationDefaultsAndServerOwnedFields; actual browser new-listing form.

Cleanup and limits: Unique RUN_ID fictional fixtures, disposable Testcontainers DB and isolated browser DB/profile/media. Dispose the whole dedicated environment; never delete shared venue/moderation history or restore a fictional snapshot. See docs/qa/VEN_EXECUTION_REPORT.md for evidence and human/staging/operations follow-ups.

VEN-002 — Full PUT replacement and immutable booking snapshots

Area: VEN · **Priority:** P1 · **Test level:** HTTP, PostgreSQL, browser · **Result:** PASS — local deterministic contract; human/staging/operations limits remain

Actors/prerequisites: Own verified-email OWNER or ADMIN (admin email exemption) may PUT; other OWNER/SCOUT 403, anonymous 401. No policy/owner-review gate for editing.

Execution: PUT /api/venues/{id} is full replacement: required base/pricing/timezone/tags/photoPaths must be present. Populate supported fields then explicitly null optional fields and empty lists; reload exact null/empty distinctions. Unsupported owner/status/audit fields are safely ignored by request DTO binding.

Persistence/behavior: No ownership/status injection; unauthorized requests leave row unchanged. Venue edits must preserve existing booking price/duration/billable/contact fields and booking policy references. Venue name is not asserted as an immutable booking snapshot. Current status may change only through the critical-owner-edit rule.

Automation: VenueDomainIntegrationTest.fullUpdateClearsOptionalDataAndCannotAssignOwnershipOrModeration; noncriticalEditsKeepApprovalAndBookingSnapshots; browser editor.

Cleanup and limits: Unique RUN_ID fictional fixtures, disposable Testcontainers DB and isolated browser DB/profile/media. Dispose the whole dedicated environment; never delete shared venue/moderation history or restore a fictional snapshot. See docs/qa/VEN_EXECUTION_REPORT.md for evidence and human/staging/operations follow-ups.

VEN-003 — Native activity arrays and readiness semantics

Area: VEN · **Priority:** P1 · **Test level:** HTTP, PostgreSQL, browser · **Result:** PASS — local deterministic contract; human/staging/operations limits remain

Actors/prerequisites: Same edit actors as VEN-002; verified OWNER may also supply activities during creation.

Execution: Round-trip null, empty and every ProductionActivity: PHOTO, FILM, VIDEO, INTERVIEW, COMMERCIAL, MUSIC_VIDEO, CONTENT_CREATION. Duplicate entries, overlapping permitted/prohibited entries, null elements and unknown enum values are invalid (400).

Persistence/behavior: PostgreSQL production_activity[] stores exact enum names; null means unanswered, [] explicitly none. Permitted null/empty both fail readiness. An explicit hard activity must be permitted and not prohibited. Verify SQL filtering as well as final evaluator.

Automation: VenueDomainIntegrationTest.activityArraysPreserveUnknownEmptyAndEveryEnumAndRejectContradictions; VenueProductionProfileValidatorTest; browser activity controls.

Cleanup and limits: Unique RUN_ID fictional fixtures, disposable Testcontainers DB and isolated browser DB/profile/media. Dispose the whole dedicated environment; never delete shared venue/moderation history or restore a fictional snapshot. See docs/qa/VEN_EXECUTION_REPORT.md for evidence and human/staging/operations follow-ups.

VEN-004 — Parking consistency and deterministic matching

Area: VEN · **Priority:** P1 · **Test level:** HTTP, PostgreSQL, browser · **Result:** PASS — local deterministic contract; human/staging/operations limits remain

Actors/prerequisites: Same edit actors as VEN-002. Deterministic matching uses controlled ParsedBrief, no live AI provider.

Execution: Parameterize parkingAvailable/spaces: false+null/0 allowed, false+positive 400; true+null/0/positive allowed; null+null/positive allowed. Reload exact values.

Persistence/behavior: Hard parking requires known true. A minimum spaces requirement also requires known true availability and sufficient known spaces, including a requested minimum of zero. Preference alone does not exclude unknown. The owner UI clears spaces when parking is changed away from Yes; that UI normalization is distinct from the broader backend contract.

Automation: VenueDomainIntegrationTest.parkingConsistencyAndHardRequirements; VenueHardRequirementEvaluatorTest; tri-state browser groups.

Cleanup and limits: Unique RUN_ID fictional fixtures, disposable Testcontainers DB and isolated browser DB/profile/media. Dispose the whole dedicated environment; never delete shared venue/moderation history or restore a fictional snapshot. See docs/qa/VEN_EXECUTION_REPORT.md for evidence and human/staging/operations follow-ups.

VEN-005 — Clip text and venue timezone contracts

Area: VEN · **Priority:** P1 · **Test level:** HTTP, PostgreSQL, browser · **Result:** PASS — local deterministic contract; human/staging/operations limits remain

Actors/prerequisites: Same edit actors as VEN-002; timezone also has owned/ADMIN availability settings PUT. Create DTO has no timezone.

Execution: Accept absolute HTTP/HTTPS clip text including localhost/internal hosts; reject javascript/data/file, relative, missing-host, malformed and >500 characters (400). Backend never fetches the clip. Current public detail has no clip-link renderer; owner input displays the stored value safely.

Persistence/behavior: Update timezone through listing PUT and PUT /api/venues/{id}/availability/settings. Europe/Amsterdam, America/New_York and other ZoneId-supported identifiers persist; invalid/blank/whitespace/case-invalid names return 400 unchanged. ZoneId also accepts fixed offsets; do not invent an IANA-only restriction. Availability, booking lead-time and existing cancellation tests cover downstream time interpretation.

Automation: VenueDomainIntegrationTest.clipAllowsHttpLinksWithoutServerFetch; unsafeOrMalformedClipsAreRejected; supportedZonesPersistAcrossBothRoutes; invalidZonesDoNotMutateEitherRoute; existing availability/lead-time tests.

Cleanup and limits: Unique RUN_ID fictional fixtures, disposable Testcontainers DB and isolated browser DB/profile/media. Dispose the whole dedicated environment; never delete shared venue/moderation history or restore a fictional snapshot. See docs/qa/VEN_EXECUTION_REPORT.md for evidence and human/staging/operations follow-ups.

VEN-006 — Critical owner edits and identical-save semantics

Area: VEN · **Priority:** P1 · **Test level:** HTTP, PostgreSQL, browser · **Result:** PASS — local deterministic contract; human/staging/operations limits remain

Actors/prerequisites: Own verified-email OWNER edits an independently ACTIVE, production-ready venue for each field. ADMIN override is characterized separately in VEN-007.

Execution: Critical fields exactly: maxCrewSize, permittedActivities, prohibitedActivities, naturalLight, blackoutPossible, ceilingHeightM, loadingAccess, parkingAvailable, parkingSpaces, floorNumber, liftAvailable, wheelchairAccessible, bathroomsAvailable, soundRecordingSuitable, productionRules, powerDetails, ordered photoPaths. Change one at a time. Exercise photo upload/addition, removal and reorder independently.

Persistence/behavior: ACTIVE → PENDING; submittedForReviewAt renewed, review metadata cleared; one VENUE_SUBMITTED audit/notification. Removed from public catalogue/unrelated detail/matching; owner can still read. Existing booking rows/policy references remain unchanged. Identical semantic numeric save remains ACTIVE: 3.5 and 3.50 are equal heights. Activity/photo order remains significant under current contract; null versus [] is distinct. Tag order is noncritical.

Automation: VenueDomainIntegrationTest.everyCriticalOwnerEditRequiresReviewAndPreservesBooking; photoListChangesRequireReview; numericallyIdenticalCeilingSaveDoesNotTriggerRereview; browser critical notice/media edits.

Cleanup and limits: Unique RUN_ID fictional fixtures, disposable Testcontainers DB and isolated browser DB/profile/media. Dispose the whole dedicated environment; never delete shared venue/moderation history or restore a fictional snapshot. See docs/qa/VEN_EXECUTION_REPORT.md for evidence and human/staging/operations follow-ups.

VEN-007 — Noncritical edits, availability and admin override

Area: VEN · **Priority:** P1 · **Test level:** HTTP, PostgreSQL, browser · **Result:** PASS — local deterministic contract; human/staging/operations limits remain

Actors/prerequisites: Own verified-email OWNER or ADMIN; ACTIVE fixtures, no new policy/owner-review prerequisites for editing.

Execution: Independently change name, neighbourhood, sizeM2, hourlyRateEur, minimumBookingMinutes, minimumLeadTimeMinutes, fullDayRateEur, tags, description, clipUrl, timezone, weekly availability and overrides.

Persistence/behavior: Remain ACTIVE with no moderation event. Required pricing/lead-time/timezone cannot be cleared or made invalid by DTO validation. Empty availability is permitted: ACTIVE means moderation approved, not guaranteed slots. Exact-interval matching excludes venues with no availability. ADMIN critical edits currently bypass owner-only re-review and can clear readiness fields while retaining ACTIVE: PRODUCT/OPERATIONS DECISION, do not silently alter that rule.

Automation: VenueDomainIntegrationTest.noncriticalEditsKeepApprovalAndBookingSnapshots; identicalAndAdminEditsAndEmptyAvailabilityCharacterizeApproval; mediaUploadLimitsAuthorizationAndAvailabilityAreIndependent; browser availability/pricing.

Cleanup and limits: Unique RUN_ID fictional fixtures, disposable Testcontainers DB and isolated browser DB/profile/media. Dispose the whole dedicated environment; never delete shared venue/moderation history or restore a fictional snapshot. See docs/qa/VEN_EXECUTION_REPORT.md for evidence and human/staging/operations follow-ups.

VEN-008 — Public visibility, own listings and tag behavior

Area: VEN · **Priority:** P1 · **Test level:** HTTP, PostgreSQL, browser · **Result:** PASS — local deterministic contract; human/staging/operations limits remain

Actors/prerequisites: Catalogue ACTIVE is public. Non-ACTIVE detail is own OWNER/ADMIN only; anonymous/SCOUT/other OWNER get 403. /my is OWNER only (including unverified email); anonymous/SCOUT/ADMIN get 403 at its method-security boundary.

Execution: GET /api/venues across DRAFT/PENDING/ACTIVE/SUSPENDED. No/blank tag is unfiltered; tag input trimmed/lowercased, stored tags are not normalized. Known lowercase matches; unknown and stored uppercase-only mismatch are characterized. GET /venues/{id} full actor/status matrix; missing UUID 404, malformed UUID 400.

Persistence/behavior: /venues/my is unpaged and returns only authenticated owner venues across statuses. Catalogue is unpaged ACTIVE only. No audit/notification writes from reads. Use real browser catalogue/detail and role guards without inventing ownership variants on admin queue endpoints.

Automation: VenueDomainIntegrationTest.visibilityStatusActorMatrixAndExactTagNormalization; browser SCOUT catalogue/detail and OWNER/ADMIN guards.

Cleanup and limits: Unique RUN_ID fictional fixtures, disposable Testcontainers DB and isolated browser DB/profile/media. Dispose the whole dedicated environment; never delete shared venue/moderation history or restore a fictional snapshot. See docs/qa/VEN_EXECUTION_REPORT.md for evidence and human/staging/operations follow-ups.

VEN-009 — Moderation lifecycle, metadata, audit and notifications

Area: VEN · **Priority:** P1 · **Test level:** HTTP, PostgreSQL, browser · **Result:** PASS — local deterministic contract; human/staging/operations limits remain

Actors/prerequisites: Submission: own OWNER, current verified email, owner VERIFIED, all current required policies, DRAFT and ready. ADMIN reviews without personal policy/email prerequisites.

Execution: Submit → PENDING; admin request-changes with required nonblank reason → DRAFT; owner fixes/resubmits → PENDING; admin approves → ACTIVE; suspend with reason → SUSPENDED; reactivate → ACTIVE. Browser confirmation and authoritative reload after transitions.

Persistence/behavior: Persist submittedForReviewAt, reviewedAt, reviewedBy and moderationReason according to current transition. Resubmission clears review fields; review/suspension/reactivation overwrite current review metadata but retain historical events. Audit sequence SUBMITTED, CHANGES_REQUESTED, SUBMITTED, APPROVED, SUSPENDED, REACTIVATED with exact OWNER/ADMIN actor and safe note. Each legitimate transition sends AFTER_COMMIT lifecycle event; disabled EmailService skips provider, invalid commands send none. Actual inbox delivery is staging-only.

Automation: VenueDomainIntegrationTest.fullModerationLifecycleMetadataAuditNotificationsAndAdminQueue; TrustNotificationServiceTest; real browser lifecycle.

Cleanup and limits: Unique RUN_ID fictional fixtures, disposable Testcontainers DB and isolated browser DB/profile/media. Dispose the whole dedicated environment; never delete shared venue/moderation history or restore a fictional snapshot. See docs/qa/VEN_EXECUTION_REPORT.md for evidence and human/staging/operations follow-ups.

VEN-010 — Revalidation, suspension commitments and concurrent decisions

Area: VEN · **Priority:** P1 · **Test level:** HTTP, PostgreSQL, browser · **Result:** PASS — local deterministic contract; human/staging/operations limits remain

Actors/prerequisites: Own OWNER submits/accepts requested bookings; ADMIN moderates. Each test establishes its actual policies/email/owner-review/readiness prerequisites independently.

Execution: Reject submission for unverified email, unverified owner, missing/stale policies, incomplete readiness, wrong status or wrong owner without mutation. Approval/reactivation recheck current readiness and owner verification. Use allowed pending/suspended edits to remove photo/rules; no invented trust-revocation endpoint. Admin approval does not recheck owner email/policy acceptance.

Persistence/behavior: Suspension removes public/matching access, blocks new booking/inquiry requests, and must block REQUESTED → AWAITING_PAYMENT acceptance (409 while venue lock held). Historical REQUESTED remains; previously AWAITING_PAYMENT remains payable until deadline; CONFIRMED remains confirmed. No automatic cancellations or booking snapshot rewrites. Real PostgreSQL barriers cover approve/approve, approve/request-changes, critical edit/approval and suspension/acceptance; serialize coherent state with safe conflicts and no contradictory events.

Automation: VenueDomainIntegrationTest.suspendedVenueCannotAcceptRequestedButKeepsAcceptedAndConfirmedHistory; submissionPrerequisitesAndReviewRevalidationUseSupportedTransitions; policiesAreRecheckedForSubmissionButNotAdminApproval; wrongStateRoleAndInvalidReasonHaveNoModerationEffects; three repeated concurrency methods.

Cleanup and limits: Unique RUN_ID fictional fixtures, disposable Testcontainers DB and isolated browser DB/profile/media. Dispose the whole dedicated environment; never delete shared venue/moderation history or restore a fictional snapshot. See docs/qa/VEN_EXECUTION_REPORT.md for evidence and human/staging/operations follow-ups.

VEN-011 — naturalLight: unknown/no/yes round trip

Area: VEN · **Priority:** P1 · **Test level:** parameterized HTTP/PostgreSQL + browser · **Result:** PASS — local deterministic contract; human/staging/operations limits remain

Actors/prerequisites: Own verified-email OWNER edits; ADMIN may edit; other OWNER/SCOUT denied. No owner-review/policy gate for saving a draft. Public detail fixture must be ACTIVE; required unknown facts cannot be approved through normal readiness checks.

Execution: Set naturalLight to null, false and true in turn. Owner Not specified / No / Yes control must produce JSON null / false / true, persist exactly to PostgreSQL, return exactly and reload unchanged. Public detailed profile renders neutral unknown, explicit negative and positive; sound uses suitable/not suitable, parking may show known space count. Cards may omit negative/unknown facts.

Matching: requiresNaturalLight=true admits only known true; false/null fail closed. prefersNaturalLight is a supported soft preference and must not exclude null. The Gemini prompt requests known matches above unknown above mismatch; actual provider ranking is staging/provider-dependent, not proved by deterministic unit scores.

Automation: Shared VenueDomainIntegrationTest.allEightBooleansPreserveThreeStatesThroughPostgresAndHardMatching; existing VenueHardRequirementEvaluatorTest/GeminiVenueRankerTest; shared browser tri-state loops. No eight duplicated test classes. Public unknown-required-fact fixtures explicitly characterize existing ADMIN override behavior.

Effects/cleanup/limits: DRAFT edits emit no moderation events; ACTIVE owner critical edits follow VEN-006. Use unique disposable DB/profile/media fixtures, never shared-history deletion. Human wording/usability and operations approval of admin overrides remain separate.

VEN-012 — blackoutPossible: unknown/no/yes round trip

Area: VEN · **Priority:** P1 · **Test level:** parameterized HTTP/PostgreSQL + browser · **Result:** PASS — local deterministic contract; human/staging/operations limits remain

Actors/prerequisites: Own verified-email OWNER edits; ADMIN may edit; other OWNER/SCOUT denied. No owner-review/policy gate for saving a draft. Public detail fixture must be ACTIVE; required unknown facts cannot be approved through normal readiness checks.

Execution: Set blackoutPossible to null, false and true in turn. Owner Not specified / No / Yes control must produce JSON null / false / true, persist exactly to PostgreSQL, return exactly and reload unchanged. Public detailed profile renders neutral unknown, explicit negative and positive; sound uses suitable/not suitable, parking may show known space count. Cards may omit negative/unknown facts.

Matching: requiresBlackout=true admits only known true; false/null fail closed. prefersBlackout is a supported soft preference and must not exclude null. The Gemini prompt requests known matches above unknown above mismatch; actual provider ranking is staging/provider-dependent, not proved by deterministic unit scores.

Automation: Shared VenueDomainIntegrationTest.allEightBooleansPreserveThreeStatesThroughPostgresAndHardMatching; existing VenueHardRequirementEvaluatorTest/GeminiVenueRankerTest; shared browser tri-state loops. No eight duplicated test classes. Public unknown-required-fact fixtures explicitly characterize existing ADMIN override behavior.

Effects/cleanup/limits: DRAFT edits emit no moderation events; ACTIVE owner critical edits follow VEN-006. Use unique disposable DB/profile/media fixtures, never shared-history deletion. Human wording/usability and operations approval of admin overrides remain separate.

VEN-013 — loadingAccess: unknown/no/yes round trip

Area: VEN · **Priority:** P1 · **Test level:** parameterized HTTP/PostgreSQL + browser · **Result:** PASS — local deterministic contract; human/staging/operations limits remain

Actors/prerequisites: Own verified-email OWNER edits; ADMIN may edit; other OWNER/SCOUT denied. No owner-review/policy gate for saving a draft. Public detail fixture must be ACTIVE; required unknown facts cannot be approved through normal readiness checks.

Execution: Set loadingAccess to null, false and true in turn. Owner Not specified / No / Yes control must produce JSON null / false / true, persist exactly to PostgreSQL, return exactly and reload unchanged. Public detailed profile renders neutral unknown, explicit negative and positive; sound uses suitable/not suitable, parking may show known space count. Cards may omit negative/unknown facts.

Matching: requiresLoadingAccess=true admits only known true; false/null fail closed. prefersLoadingAccess is a supported soft preference and must not exclude null. The Gemini prompt requests known matches above unknown above mismatch; actual provider ranking is staging/provider-dependent, not proved by deterministic unit scores.

Automation: Shared VenueDomainIntegrationTest.allEightBooleansPreserveThreeStatesThroughPostgresAndHardMatching; existing VenueHardRequirementEvaluatorTest/GeminiVenueRankerTest; shared browser tri-state loops. No eight duplicated test classes. Public unknown-required-fact fixtures explicitly characterize existing ADMIN override behavior.

Effects/cleanup/limits: DRAFT edits emit no moderation events; ACTIVE owner critical edits follow VEN-006. Use unique disposable DB/profile/media fixtures, never shared-history deletion. Human wording/usability and operations approval of admin overrides remain separate.

VEN-014 — parkingAvailable: unknown/no/yes round trip

Area: VEN · **Priority:** P1 · **Test level:** parameterized HTTP/PostgreSQL + browser · **Result:** PASS — local deterministic contract; human/staging/operations limits remain

Actors/prerequisites: Own verified-email OWNER edits; ADMIN may edit; other OWNER/SCOUT denied. No owner-review/policy gate for saving a draft. Public detail fixture must be ACTIVE; required unknown facts cannot be approved through normal readiness checks.

Execution: Set parkingAvailable to null, false and true in turn. Owner Not specified / No / Yes control must produce JSON null / false / true, persist exactly to PostgreSQL, return exactly and reload unchanged. Public detailed profile renders neutral unknown, explicit negative and positive; sound uses suitable/not suitable, parking may show known space count. Cards may omit negative/unknown facts.

Matching: requiresParking=true admits only known true; false/null fail closed. prefersParking is a supported soft preference and must not exclude null. The Gemini prompt requests known matches above unknown above mismatch; actual provider ranking is staging/provider-dependent, not proved by deterministic unit scores.

Automation: Shared VenueDomainIntegrationTest.allEightBooleansPreserveThreeStatesThroughPostgresAndHardMatching; existing VenueHardRequirementEvaluatorTest/GeminiVenueRankerTest; shared browser tri-state loops. No eight duplicated test classes. Public unknown-required-fact fixtures explicitly characterize existing ADMIN override behavior.

Effects/cleanup/limits: DRAFT edits emit no moderation events; ACTIVE owner critical edits follow VEN-006. Use unique disposable DB/profile/media fixtures, never shared-history deletion. Human wording/usability and operations approval of admin overrides remain separate.

VEN-015 — liftAvailable: unknown/no/yes round trip

Area: VEN · **Priority:** P1 · **Test level:** parameterized HTTP/PostgreSQL + browser · **Result:** PASS — local deterministic contract; human/staging/operations limits remain

Actors/prerequisites: Own verified-email OWNER edits; ADMIN may edit; other OWNER/SCOUT denied. No owner-review/policy gate for saving a draft. Public detail fixture must be ACTIVE; required unknown facts cannot be approved through normal readiness checks.

Execution: Set liftAvailable to null, false and true in turn. Owner Not specified / No / Yes control must produce JSON null / false / true, persist exactly to PostgreSQL, return exactly and reload unchanged. Public detailed profile renders neutral unknown, explicit negative and positive; sound uses suitable/not suitable, parking may show known space count. Cards may omit negative/unknown facts.

Matching: requiresLift=true admits only known true; false/null fail closed. prefersLift is a supported soft preference and must not exclude null. The Gemini prompt requests known matches above unknown above mismatch; actual provider ranking is staging/provider-dependent, not proved by deterministic unit scores.

Automation: Shared VenueDomainIntegrationTest.allEightBooleansPreserveThreeStatesThroughPostgresAndHardMatching; existing VenueHardRequirementEvaluatorTest/GeminiVenueRankerTest; shared browser tri-state loops. No eight duplicated test classes. Public unknown-required-fact fixtures explicitly characterize existing ADMIN override behavior.

Effects/cleanup/limits: DRAFT edits emit no moderation events; ACTIVE owner critical edits follow VEN-006. Use unique disposable DB/profile/media fixtures, never shared-history deletion. Human wording/usability and operations approval of admin overrides remain separate.

VEN-016 — wheelchairAccessible: unknown/no/yes round trip

Area: VEN · **Priority:** P1 · **Test level:** parameterized HTTP/PostgreSQL + browser · **Result:** PASS — local deterministic contract; human/staging/operations limits remain

Actors/prerequisites: Own verified-email OWNER edits; ADMIN may edit; other OWNER/SCOUT denied. No owner-review/policy gate for saving a draft. Public detail fixture must be ACTIVE; required unknown facts cannot be approved through normal readiness checks.

Execution: Set wheelchairAccessible to null, false and true in turn. Owner Not specified / No / Yes control must produce JSON null / false / true, persist exactly to PostgreSQL, return exactly and reload unchanged. Public detailed profile renders neutral unknown, explicit negative and positive; sound uses suitable/not suitable, parking may show known space count. Cards may omit negative/unknown facts.

Matching: requiresWheelchairAccessibility=true admits only known true; false/null fail closed. No dedicated preference flag exists for this field; do not invent one. The Gemini prompt requests known matches above unknown above mismatch; actual provider ranking is staging/provider-dependent, not proved by deterministic unit scores.

Automation: Shared VenueDomainIntegrationTest.allEightBooleansPreserveThreeStatesThroughPostgresAndHardMatching; existing VenueHardRequirementEvaluatorTest/GeminiVenueRankerTest; shared browser tri-state loops. No eight duplicated test classes. Public unknown-required-fact fixtures explicitly characterize existing ADMIN override behavior.

Effects/cleanup/limits: DRAFT edits emit no moderation events; ACTIVE owner critical edits follow VEN-006. Use unique disposable DB/profile/media fixtures, never shared-history deletion. Human wording/usability and operations approval of admin overrides remain separate.

VEN-017 — bathroomsAvailable: unknown/no/yes round trip

Area: VEN · **Priority:** P1 · **Test level:** parameterized HTTP/PostgreSQL + browser · **Result:** PASS — local deterministic contract; human/staging/operations limits remain

Actors/prerequisites: Own verified-email OWNER edits; ADMIN may edit; other OWNER/SCOUT denied. No owner-review/policy gate for saving a draft. Public detail fixture must be ACTIVE; required unknown facts cannot be approved through normal readiness checks.

Execution: Set bathroomsAvailable to null, false and true in turn. Owner Not specified / No / Yes control must produce JSON null / false / true, persist exactly to PostgreSQL, return exactly and reload unchanged. Public detailed profile renders neutral unknown, explicit negative and positive; sound uses suitable/not suitable, parking may show known space count. Cards may omit negative/unknown facts.

Matching: requiresBathrooms=true admits only known true; false/null fail closed. No dedicated preference flag exists for this field; do not invent one. The Gemini prompt requests known matches above unknown above mismatch; actual provider ranking is staging/provider-dependent, not proved by deterministic unit scores.

Automation: Shared VenueDomainIntegrationTest.allEightBooleansPreserveThreeStatesThroughPostgresAndHardMatching; existing VenueHardRequirementEvaluatorTest/GeminiVenueRankerTest; shared browser tri-state loops. No eight duplicated test classes. Public unknown-required-fact fixtures explicitly characterize existing ADMIN override behavior.

Effects/cleanup/limits: DRAFT edits emit no moderation events; ACTIVE owner critical edits follow VEN-006. Use unique disposable DB/profile/media fixtures, never shared-history deletion. Human wording/usability and operations approval of admin overrides remain separate.

VEN-018 — soundRecordingSuitable: unknown/no/yes round trip

Area: VEN · **Priority:** P1 · **Test level:** parameterized HTTP/PostgreSQL + browser · **Result:** PASS — local deterministic contract; human/staging/operations limits remain

Actors/prerequisites: Own verified-email OWNER edits; ADMIN may edit; other OWNER/SCOUT denied. No owner-review/policy gate for saving a draft. Public detail fixture must be ACTIVE; required unknown facts cannot be approved through normal readiness checks.

Execution: Set soundRecordingSuitable to null, false and true in turn. Owner Not specified / No / Yes control must produce JSON null / false / true, persist exactly to PostgreSQL, return exactly and reload unchanged. Public detailed profile renders neutral unknown, explicit negative and positive; sound uses suitable/not suitable, parking may show known space count. Cards may omit negative/unknown facts.

Matching: requiresSoundRecordingSuitable=true admits only known true; false/null fail closed. No dedicated preference flag exists for this field; do not invent one. The Gemini prompt requests known matches above unknown above mismatch; actual provider ranking is staging/provider-dependent, not proved by deterministic unit scores.

Automation: Shared VenueDomainIntegrationTest.allEightBooleansPreserveThreeStatesThroughPostgresAndHardMatching; existing VenueHardRequirementEvaluatorTest/GeminiVenueRankerTest; shared browser tri-state loops. No eight duplicated test classes. Public unknown-required-fact fixtures explicitly characterize existing ADMIN override behavior.

Effects/cleanup/limits: DRAFT edits emit no moderation events; ACTIVE owner critical edits follow VEN-006. Use unique disposable DB/profile/media fixtures, never shared-history deletion. Human wording/usability and operations approval of admin overrides remain separate.

READY scenarios

READY-001 — Readiness issue maxCrewSize

Area: READY · **Priority:** P1 · **Test level:** UNIT + service + PostgreSQL; representative browser · **Result:** PASS — local deterministic contract; UX/operations/staging limits remain

Actors/prerequisites: Validator has no actor. Workflow fixture: active verified-email owning OWNER, owner review VERIFIED, current policies accepted; ADMIN reviews/reactivates. No repeated unrelated ownership checks; VEN covers authorization.

Exact contract: Clear maxCrewSize to null: exact issue maxCrewSize. Restore 20: no issues. Zero/negative and DTO maximum 10,000 belong to numeric validation, not readiness presence.

Execution and effects: Start from the centralized complete fixture with no issues. Remove only this condition; assert exact single issue, no mutation; restore and assert empty. Submit/approve/reactivate must refuse readiness-incomplete fixtures before changing status, timestamps, trust events or notifications. Restore then transition successfully. Use real API/PG only for reachable absent states; never insert an impossible row or disable a constraint.

Automation: ReadyVenueFixture; VenueReadinessContractTest parameterization, full key set, purity, Boolean and numeric-boundary cases. VenueDomainIntegrationTest.readinessMissingFieldBlocksSubmitApproveAndReactivateUntilRestored covers persisted absence and restoration across all three transitions. ready-browser.cjs covers representative multi-issue, false, restore and stale-admin states. VEN provides matching/availability and critical-edit/approval concurrency evidence.

Cleanup and limits: Unique RUN_ID fictional fixtures, disposable Testcontainers and dedicated browser DB/profile/media. Dispose whole QA environments; never delete shared history or restore fictional snapshots. Failed readiness emits no lifecycle notification; successful transitions retain existing emails (delivery staging-only). Human checklist wording and existing ADMIN override decision remain separate. See docs/qa/READY_EXECUTION_REPORT.md.

READY-002 — Readiness issue permittedActivities

Area: READY · **Priority:** P1 · **Test level:** UNIT + service + PostgreSQL; representative browser · **Result:** PASS — local deterministic contract; UX/operations/staging limits remain

Actors/prerequisites: Validator has no actor. Workflow fixture: active verified-email owning OWNER, owner review VERIFIED, current policies accepted; ADMIN reviews/reactivates. No repeated unrelated ownership checks; VEN covers authorization.

Exact contract: Test null and [] independently: both return permittedActivities while preserving their distinct model values. Restore a supported activity: no issues. Reuse VEN contradiction/enum-array coverage.

Execution and effects: Start from the centralized complete fixture with no issues. Remove only this condition; assert exact single issue, no mutation; restore and assert empty. Submit/approve/reactivate must refuse readiness-incomplete fixtures before changing status, timestamps, trust events or notifications. Restore then transition successfully. Use real API/PG only for reachable absent states; never insert an impossible row or disable a constraint.

Automation: ReadyVenueFixture; VenueReadinessContractTest parameterization, full key set, purity, Boolean and numeric-boundary cases. VenueDomainIntegrationTest.readinessMissingFieldBlocksSubmitApproveAndReactivateUntilRestored covers persisted absence and restoration across all three transitions. ready-browser.cjs covers representative multi-issue, false, restore and stale-admin states. VEN provides matching/availability and critical-edit/approval concurrency evidence.

Cleanup and limits: Unique RUN_ID fictional fixtures, disposable Testcontainers and dedicated browser DB/profile/media. Dispose whole QA environments; never delete shared history or restore fictional snapshots. Failed readiness emits no lifecycle notification; successful transitions retain existing emails (delivery staging-only). Human checklist wording and existing ADMIN override decision remain separate. See docs/qa/READY_EXECUTION_REPORT.md.

READY-003 — Readiness issue productionRules

Area: READY · **Priority:** P1 · **Test level:** UNIT + service + PostgreSQL; representative browser · **Result:** PASS — local deterministic contract; UX/operations/staging limits remain

Actors/prerequisites: Validator has no actor. Workflow fixture: active verified-email owning OWNER, owner review VERIFIED, current policies accepted; ADMIN reviews/reactivates. No repeated unrelated ownership checks; VEN covers authorization.

Exact contract: Test null, empty and whitespace-only productionRules: exact issue productionRules. Restore nonblank QA text. No content-quality/minimum-prose policy is invented.

Execution and effects: Start from the centralized complete fixture with no issues. Remove only this condition; assert exact single issue, no mutation; restore and assert empty. Submit/approve/reactivate must refuse readiness-incomplete fixtures before changing status, timestamps, trust events or notifications. Restore then transition successfully. Use real API/PG only for reachable absent states; never insert an impossible row or disable a constraint.

Automation: ReadyVenueFixture; VenueReadinessContractTest parameterization, full key set, purity, Boolean and numeric-boundary cases. VenueDomainIntegrationTest.readinessMissingFieldBlocksSubmitApproveAndReactivateUntilRestored covers persisted absence and restoration across all three transitions. ready-browser.cjs covers representative multi-issue, false, restore and stale-admin states. VEN provides matching/availability and critical-edit/approval concurrency evidence.

Cleanup and limits: Unique RUN_ID fictional fixtures, disposable Testcontainers and dedicated browser DB/profile/media. Dispose whole QA environments; never delete shared history or restore fictional snapshots. Failed readiness emits no lifecycle notification; successful transitions retain existing emails (delivery staging-only). Human checklist wording and existing ADMIN override decision remain separate. See docs/qa/READY_EXECUTION_REPORT.md.

READY-004 — Readiness issue naturalLight

Area: READY · **Priority:** P1 · **Test level:** UNIT + service + PostgreSQL; representative browser · **Result:** PASS — local deterministic contract; UX/operations/staging limits remain

Actors/prerequisites: Validator has no actor. Workflow fixture: active verified-email owning OWNER, owner review VERIFIED, current policies accepted; ADMIN reviews/reactivates. No repeated unrelated ownership checks; VEN covers authorization.

Exact contract: null returns naturalLight; false and true are both answered and ready. Matching eligibility is separate (VEN coverage).

Execution and effects: Start from the centralized complete fixture with no issues. Remove only this condition; assert exact single issue, no mutation; restore and assert empty. Submit/approve/reactivate must refuse readiness-incomplete fixtures before changing status, timestamps, trust events or notifications. Restore then transition successfully. Use real API/PG only for reachable absent states; never insert an impossible row or disable a constraint.

Automation: ReadyVenueFixture; VenueReadinessContractTest parameterization, full key set, purity, Boolean and numeric-boundary cases. VenueDomainIntegrationTest.readinessMissingFieldBlocksSubmitApproveAndReactivateUntilRestored covers persisted absence and restoration across all three transitions. ready-browser.cjs covers representative multi-issue, false, restore and stale-admin states. VEN provides matching/availability and critical-edit/approval concurrency evidence.

Cleanup and limits: Unique RUN_ID fictional fixtures, disposable Testcontainers and dedicated browser DB/profile/media. Dispose whole QA environments; never delete shared history or restore fictional snapshots. Failed readiness emits no lifecycle notification; successful transitions retain existing emails (delivery staging-only). Human checklist wording and existing ADMIN override decision remain separate. See docs/qa/READY_EXECUTION_REPORT.md.

READY-005 — Readiness issue loadingAccess

Area: READY · **Priority:** P1 · **Test level:** UNIT + service + PostgreSQL; representative browser · **Result:** PASS — local deterministic contract; UX/operations/staging limits remain

Actors/prerequisites: Validator has no actor. Workflow fixture: active verified-email owning OWNER, owner review VERIFIED, current policies accepted; ADMIN reviews/reactivates. No repeated unrelated ownership checks; VEN covers authorization.

Exact contract: null returns loadingAccess; false and true are both answered and ready. No requirement to provide loading access is invented.

Execution and effects: Start from the centralized complete fixture with no issues. Remove only this condition; assert exact single issue, no mutation; restore and assert empty. Submit/approve/reactivate must refuse readiness-incomplete fixtures before changing status, timestamps, trust events or notifications. Restore then transition successfully. Use real API/PG only for reachable absent states; never insert an impossible row or disable a constraint.

Automation: ReadyVenueFixture; VenueReadinessContractTest parameterization, full key set, purity, Boolean and numeric-boundary cases. VenueDomainIntegrationTest.readinessMissingFieldBlocksSubmitApproveAndReactivateUntilRestored covers persisted absence and restoration across all three transitions. ready-browser.cjs covers representative multi-issue, false, restore and stale-admin states. VEN provides matching/availability and critical-edit/approval concurrency evidence.

Cleanup and limits: Unique RUN_ID fictional fixtures, disposable Testcontainers and dedicated browser DB/profile/media. Dispose whole QA environments; never delete shared history or restore fictional snapshots. Failed readiness emits no lifecycle notification; successful transitions retain existing emails (delivery staging-only). Human checklist wording and existing ADMIN override decision remain separate. See docs/qa/READY_EXECUTION_REPORT.md.

READY-006 — Readiness issue bathroomsAvailable

Area: READY · **Priority:** P1 · **Test level:** UNIT + service + PostgreSQL; representative browser · **Result:** PASS — local deterministic contract; UX/operations/staging limits remain

Actors/prerequisites: Validator has no actor. Workflow fixture: active verified-email owning OWNER, owner review VERIFIED, current policies accepted; ADMIN reviews/reactivates. No repeated unrelated ownership checks; VEN covers authorization.

Exact contract: null returns bathroomsAvailable; false and true are both answered and ready. Readiness does not mandate bathrooms.

Execution and effects: Start from the centralized complete fixture with no issues. Remove only this condition; assert exact single issue, no mutation; restore and assert empty. Submit/approve/reactivate must refuse readiness-incomplete fixtures before changing status, timestamps, trust events or notifications. Restore then transition successfully. Use real API/PG only for reachable absent states; never insert an impossible row or disable a constraint.

Automation: ReadyVenueFixture; VenueReadinessContractTest parameterization, full key set, purity, Boolean and numeric-boundary cases. VenueDomainIntegrationTest.readinessMissingFieldBlocksSubmitApproveAndReactivateUntilRestored covers persisted absence and restoration across all three transitions. ready-browser.cjs covers representative multi-issue, false, restore and stale-admin states. VEN provides matching/availability and critical-edit/approval concurrency evidence.

Cleanup and limits: Unique RUN_ID fictional fixtures, disposable Testcontainers and dedicated browser DB/profile/media. Dispose whole QA environments; never delete shared history or restore fictional snapshots. Failed readiness emits no lifecycle notification; successful transitions retain existing emails (delivery staging-only). Human checklist wording and existing ADMIN override decision remain separate. See docs/qa/READY_EXECUTION_REPORT.md.

READY-007 — Readiness issue soundRecordingSuitable

Area: READY · **Priority:** P1 · **Test level:** UNIT + service + PostgreSQL; representative browser · **Result:** PASS — local deterministic contract; UX/operations/staging limits remain

Actors/prerequisites: Validator has no actor. Workflow fixture: active verified-email owning OWNER, owner review VERIFIED, current policies accepted; ADMIN reviews/reactivates. No repeated unrelated ownership checks; VEN covers authorization.

Exact contract: null returns soundRecordingSuitable; false is a complete answer despite unsuitability for a sound hard requirement.

Execution and effects: Start from the centralized complete fixture with no issues. Remove only this condition; assert exact single issue, no mutation; restore and assert empty. Submit/approve/reactivate must refuse readiness-incomplete fixtures before changing status, timestamps, trust events or notifications. Restore then transition successfully. Use real API/PG only for reachable absent states; never insert an impossible row or disable a constraint.

Automation: ReadyVenueFixture; VenueReadinessContractTest parameterization, full key set, purity, Boolean and numeric-boundary cases. VenueDomainIntegrationTest.readinessMissingFieldBlocksSubmitApproveAndReactivateUntilRestored covers persisted absence and restoration across all three transitions. ready-browser.cjs covers representative multi-issue, false, restore and stale-admin states. VEN provides matching/availability and critical-edit/approval concurrency evidence.

Cleanup and limits: Unique RUN_ID fictional fixtures, disposable Testcontainers and dedicated browser DB/profile/media. Dispose whole QA environments; never delete shared history or restore fictional snapshots. Failed readiness emits no lifecycle notification; successful transitions retain existing emails (delivery staging-only). Human checklist wording and existing ADMIN override decision remain separate. See docs/qa/READY_EXECUTION_REPORT.md.

READY-008 — Readiness issue hourlyRateEur

Area: READY · **Priority:** P1 · **Test level:** UNIT + service; representative browser and reused VEN validation · **Result:** PASS — local deterministic contract; UX/operations/staging limits remain

Actors/prerequisites: Validator has no actor. Workflow fixture: active verified-email owning OWNER, owner review VERIFIED, current policies accepted; ADMIN reviews/reactivates. No repeated unrelated ownership checks; VEN covers authorization.

Exact contract: Null internal fixture returns hourlyRateEur. DB NOT NULL/positive and DTO 0.01–99,999,999.99 prevent missing/invalid live values; test HTTP/DTO validation separately. Readiness itself checks null, not positivity. fullDayRateEur remains optional.

Execution and effects: Start from the centralized complete fixture with no issues. Remove only this condition; assert exact single issue, no mutation; restore and assert empty. Submit/approve/reactivate must refuse readiness-incomplete fixtures before changing status, timestamps, trust events or notifications. Restore then transition successfully. Use real API/PG only for reachable absent states; never insert an impossible row or disable a constraint.

Automation: ReadyVenueFixture; VenueReadinessContractTest parameterization, full key set, purity, Boolean and numeric-boundary cases. Service parameterization proves defensive workflow gating; current DTO/DB restrictions are explicit. ready-browser.cjs covers representative multi-issue, false, restore and stale-admin states. VEN provides matching/availability and critical-edit/approval concurrency evidence.

Cleanup and limits: Unique RUN_ID fictional fixtures, disposable Testcontainers and dedicated browser DB/profile/media. Dispose whole QA environments; never delete shared history or restore fictional snapshots. Failed readiness emits no lifecycle notification; successful transitions retain existing emails (delivery staging-only). Human checklist wording and existing ADMIN override decision remain separate. See docs/qa/READY_EXECUTION_REPORT.md.

READY-009 — Readiness issue minimumBookingMinutes

Area: READY · **Priority:** P1 · **Test level:** UNIT + service; representative browser and reused VEN validation · **Result:** PASS — local deterministic contract; UX/operations/staging limits remain

Actors/prerequisites: Validator has no actor. Workflow fixture: active verified-email owning OWNER, owner review VERIFIED, current policies accepted; ADMIN reviews/reactivates. No repeated unrelated ownership checks; VEN covers authorization.

Exact contract: Null internal fixture returns minimumBookingMinutes. DB NOT NULL/positive; DTO 1–10,080. Zero/negative fail validation earlier, not a fabricated persisted readiness 409.

Execution and effects: Start from the centralized complete fixture with no issues. Remove only this condition; assert exact single issue, no mutation; restore and assert empty. Submit/approve/reactivate must refuse readiness-incomplete fixtures before changing status, timestamps, trust events or notifications. Restore then transition successfully. Use real API/PG

only for reachable absent states; never insert an impossible row or disable a constraint.

Automation: ReadyVenueFixture; VenueReadinessContractTest parameterization, full key set, purity, Boolean and numeric-boundary cases. Service parameterization proves defensive workflow gating; current DTO/DB restrictions are explicit. ready-browser.cjs covers representative multi-issue, false, restore and stale-admin states. VEN provides matching/availability and critical-edit/approval concurrency evidence.

Cleanup and limits: Unique RUN_ID fictional fixtures, disposable Testcontainers and dedicated browser DB/profile/media. Dispose whole QA environments; never delete shared history or restore fictional snapshots. Failed readiness emits no lifecycle notification; successful transitions retain existing emails (delivery staging-only). Human checklist wording and existing ADMIN override decision remain separate. See docs/qa/READY_EXECUTION_REPORT.md.

READY-010 — Readiness issue minimumLeadTimeMinutes

Area: READY · **Priority:** P1 · **Test level:** UNIT + service; representative browser and reused VEN validation · **Result:** PASS — local deterministic contract; UX/operations/staging limits remain

Actors/prerequisites: Validator has no actor. Workflow fixture: active verified-email owning OWNER, owner review VERIFIED, current policies accepted; ADMIN reviews/reactivates. No repeated unrelated ownership checks; VEN covers authorization.

Exact contract: Null returns minimumLeadTimeMinutes. Current DTO requires 60–43,200; V7 intentionally permits legacy NULL. Use internal service fixture for missing-value gating; do not bypass the live DTO. Test numeric boundaries separately.

Execution and effects: Start from the centralized complete fixture with no issues. Remove only this condition; assert exact single issue, no mutation; restore and assert empty. Submit/approve/reactivate must refuse readiness-incomplete fixtures before changing status, timestamps, trust events or notifications. Restore then transition successfully. Use real API/PG only for reachable absent states; never insert an impossible row or disable a constraint.

Automation: ReadyVenueFixture; VenueReadinessContractTest parameterization, full key set, purity, Boolean and numeric-boundary cases. Service parameterization proves defensive workflow gating; current DTO/DB restrictions are explicit. ready-browser.cjs covers representative multi-issue, false, restore and stale-admin states. VEN provides matching/availability and critical-edit/approval concurrency evidence.

Cleanup and limits: Unique RUN_ID fictional fixtures, disposable Testcontainers and dedicated browser DB/profile/media. Dispose whole QA environments; never delete shared history or restore fictional snapshots. Failed readiness emits no lifecycle notification; successful transitions retain existing emails (delivery staging-only). Human checklist wording and existing ADMIN override decision remain separate. See docs/qa/READY_EXECUTION_REPORT.md.

READY-011 — Readiness issue timezone

Area: READY · **Priority:** P1 · **Test level:** UNIT + service; representative browser and reused VEN validation · **Result:** PASS — local deterministic contract; UX/operations/staging limits remain

Actors/prerequisites: Validator has no actor. Workflow fixture: active verified-email owning OWNER, owner review VERIFIED, current policies accepted; ADMIN reviews/reactivates. No repeated unrelated ownership checks; VEN covers authorization.

Exact contract: Null/blank internal timezone returns timezone. DB forbids null/empty/space-only values; DTO rejects all blank values and media validation rejects invalid ZoneId before update. ZoneId also supports UTC/fixed offsets: do not impose an IANA-region-only rule. Readiness itself does not parse ZoneId.

Execution and effects: Start from the centralized complete fixture with no issues. Remove only this condition; assert exact single issue, no mutation; restore and assert empty. Submit/approve/reactivate must refuse readiness-incomplete fixtures before changing status, timestamps, trust events or notifications. Restore then transition successfully. Use real API/PG only for reachable absent states; never insert an impossible row or disable a constraint.

Automation: ReadyVenueFixture; VenueReadinessContractTest parameterization, full key set, purity, Boolean and numeric-boundary cases. Service parameterization proves defensive workflow gating; current DTO/DB restrictions are explicit. ready-browser.cjs covers representative multi-issue, false, restore and stale-admin states. VEN provides matching/availability and critical-edit/approval concurrency evidence.

Cleanup and limits: Unique RUN_ID fictional fixtures, disposable Testcontainers and dedicated browser DB/profile/media. Dispose whole QA environments; never delete shared history or restore fictional snapshots. Failed readiness emits no lifecycle notification; successful transitions retain existing emails (delivery staging-only). Human checklist wording and existing ADMIN override decision remain separate. See docs/qa/READY_EXECUTION_REPORT.md.

READY-012 — Readiness issue photoPaths

Area: READY · **Priority:** P1 · **Test level:** UNIT + service + PostgreSQL; representative browser · **Result:** PASS — local deterministic contract; UX/operations/staging limits remain

Actors/prerequisites: Validator has no actor. Workflow fixture: active verified-email owning OWNER, owner review VERIFIED, current policies accepted; ADMIN reviews/reactivates. No repeated unrelated ownership checks; VEN covers authorization.

Exact contract: Empty attached-reference list returns photoPaths and is reachable through PUT. Null is defensive internal-only coverage because DB/DTO prohibit it. One or multiple valid Eema photo references satisfy readiness; no object-storage lookup occurs.

Execution and effects: Start from the centralized complete fixture with no issues. Remove only this condition; assert exact single issue, no mutation; restore and assert empty. Submit/approve/reactivate must refuse readiness-incomplete fixtures before changing status, timestamps, trust events or notifications. Restore then transition successfully. Use real API/PG only for reachable absent states; never insert an impossible row or disable a constraint.

Automation: ReadyVenueFixture; VenueReadinessContractTest parameterization, full key set, purity, Boolean and numeric-boundary cases. VenueDomainIntegrationTest.readinessMissingFieldBlocksSubmitApproveAndReactivateUntilRestored covers persisted absence and restoration across all three transitions. ready-browser.cjs covers representative multi-issue, false, restore and stale-admin states. VEN provides matching/availability and critical-edit/approval concurrency evidence.

Cleanup and limits: Unique RUN_ID fictional fixtures, disposable Testcontainers and dedicated browser DB/profile/media. Dispose whole QA environments; never delete shared history or restore fictional snapshots. Failed readiness emits no lifecycle notification; successful transitions retain existing emails (delivery staging-only). Human checklist wording and existing ADMIN override decision remain separate. See docs/qa/READY_EXECUTION_REPORT.md.

MEDIA scenarios

MEDIA-001 — Valid image upload and durable reference

Area: MEDIA · **Priority:** P1 · **Test level:** UNIT, API, REAL POSTGRES, LOCAL FILESYSTEM, BROWSER · **Result:** PASS (LOCAL; staging/privacy decisions separate)

Actors/prerequisites: Standalone upload: authenticated, verified-email OWNER; ADMIN and SCOUT denied 403, anonymous 401. Scoped upload: own verified-email OWNER or ADMIN (email exemption); other OWNER/SCOUT 403, anonymous 401. Neither policies nor owner verification is required. Unique RUN_ID venues; no shared fixture reset.

Actual endpoints: POST `/api/upload/venue-photo`, multipart `file`, returns 200 `{message,url}` and stores only. POST `/api/venues/{id}/photos`, multipart `file`, returns 200 authoritative VenueDto after locked append and persistence. GET `/api/upload/files/{filename}` is public and proxies local/S3 bytes.

Steps/assertions: Upload real decodable JPEG (.jpg/.jpeg), PNG and WebP. Confirm generated UUID key, controlled reference, exact bytes/Content-Type/nosniff on public read, original filename discarded, scoped ordered DB attachment, and standalone no attachment. Reload editor and restart the production-Dockerfile backend with a dedicated named media volume; references and bytes survive. S3 mock verifies bucket/prefix/content type, conditional create and proxy read; it does not prove deployed storage.

Side effects: Ordinary DRAFT upload has no moderation event/email. Owner ACTIVE photo change invokes re-review and VENUE_SUBMITTED notification; ADMIN retains existing override behavior. Local emails disabled; actual delivery belongs to staging.

Automation/cleanup/limits: `MediaStorageContractTest`, `VenueDomainIntegrationTest#media*`, `scripts/qa/media-browser.cjs`. Disposable Testcontainers, temporary media root, dedicated compose volume/browser profile. Dispose whole QA environment, never shared media/history. Human gallery UX and deployed S3 durability/access remain separate; see `docs/qa/MEDIA_EXECUTION_REPORT.md`.

MEDIA-002 — Image content, size, multipart and public-path security

Area: MEDIA · **Priority:** P1 · **Test level:** PARAMETERIZED UNIT, API, REAL POSTGRES, REAL HTTP · **Result:** PASS (LOCAL; staging/privacy decisions separate)

Actors/prerequisites: Use an authorized actor from MEDIA-001 and an owned DRAFT to isolate validation. Reuse endpoint authorization matrix; there is no standalone resource-ownership UUID.

Steps/assertions: Both adapters reject empty, one-byte, truncated/fake image, GIF/SVG/BMP/text/octet-stream, missing/unsupported filename/type, MIME-extension mismatch and decoded-format mismatch before durable write. Validate JPEG/PNG with ImageIO and WebP with TwelveMonkeys; headers/extensions alone are not proof. Decode only after geometry checks: ≤10,000 pixels per side and ≤25 million pixels total, at most two decoders/JVM. No re-encoding, EXIF stripping or malware scanning is claimed.

Boundary layers: Default file limit 5,242,880 bytes. A decodable JPEG padded to exactly 5 MiB succeeds; +1 fails storage validation with 400 in service/MockMvc, and the real servlet rejects oversized multipart with 413. Missing part, malformed multipart and non-multipart upload requests return 400. Parameterize unsafe client filenames: generated UUID ignores all path components. Public nonexistent/invalid/traversal/private-document names cannot expose files; valid-shaped symlinks are refused. Missing object 404.

Persistence/side effects: Invalid and denied uploads leave no new object, photo reference, moderation event or lifecycle email. No raw provider error, bytes or credentials in client response. Browser errors explain corrupt/type/size failures; HTML accept attribute is UX only.

Automation/cleanup/limits: Shared binary fixtures and parameterized `MediaStorageContractTest`, `VenueDomainIntegrationTest#mediaAuthorizationInvalidContentAndMaxLimitNeverLeaveOrphans`, representative real-browser/HTTP runner. Temporary media/disposable DB; preserve shared objects. Pre-change fake bytes acceptance was a reproduced P1 defect, not acceptable behavior. Existing legacy objects are not retroactively validated. Privacy/EXIF and staging object permissions require separate decisions.

MEDIA-003 — Ordered photos, maximum, readiness and moderation

Area: MEDIA · **Priority:** P1 · **Test level:** API, REAL POSTGRES, BROWSER · **Result:** PASS (LOCAL; staging/privacy decisions separate)

Actors/prerequisites: Own verified-email OWNER or ADMIN for scoped upload/PUT; use independent ACTIVE fixture only for moderation case. ADMIN edit override is an existing operations decision, not changed here.

Steps/assertions: Upload A/B/C and persist `[A,B,C]`; first reference is primary in editor/card/detail. Full PUT `[C,A,B]` changes order without copying bytes. PUT `[B,C]` detaches A, with no physical deletion promise. Empty list restores photo readiness issue; first attachment removes it. Accept 1/11/12 photos and reject the thirteenth before storage. Reject duplicate references, external/file/traversal/non-UUID references and >12 arrays without mutation. Reference shape does not establish existence or exclusive ownership; do not invent that guarantee.

Moderation/concurrency: Owner ACTIVE add/remove/reorder → PENDING with existing re-review event/notification; historical booking behavior is covered by VEN. Ordinary DRAFT reorder/remove has no email. Real PostgreSQL barriers prove two successful simultaneous uploads both persist exactly once; at 11 one wins and one safely fails, with no orphan. Concurrent full PUT/upload serializes, but a later stale whole-array PUT may detach the uploaded reference; row locking does not provide client revision detection.

Automation/cleanup/limits: Reuse VEN photo criticality/READY-012 and add `VenueDomainIntegrationTest#mediaConcurrent*`, max-limit/reorder checks and representative browser primary/removal/12-photo journey. Repeat race groups; no timing sleeps. Dispose unique test environment. Orphans (standalone, detached, abandoned draft) remain an operations backlog; no GC or ownership table is introduced.

MEDIA-004 — Storage compensation after persistence failure

Area: MEDIA · **Priority:** P1 · **Test level:** SERVICE, REAL POSTGRES, S3 MOCK · **Result:** PASS (LOCAL; staging/privacy decisions separate)

Actors/prerequisites: Own verified-email OWNER with room for a photo, valid real image; use test-controlled persistence failure after storage succeeds. No provider credentials or shared-row deletion.

Steps/assertions: Scoped upload writes object, then save/saveAndFlush fails. Original persistence exception remains primary; deletion of the newly unattached object is attempted. Real PostgreSQL flush-then-fail proves transaction rollback: unchanged photo array/status/trust events and no after-commit notification. Repeat with delete failure; retain original error, log only safe venue/reference/error-class metadata, never report success. Mock S3 verifies deletion bucket/key.

Transaction limitation: Object storage is outside the DB transaction. Immediate save/flush compensation does not guarantee cleanup if transaction completion fails after the service returns, or process death/delete failure occurs. These are explicitly recorded orphan risks; no garbage collection is claimed.

Automation/cleanup/limits: `VenueServiceTest` persistence/delete-failure checks; `VenueDomainIntegrationTest#mediaAttachmentFailureAfterRealFlushRollsBackRowsAndDeletesNewObject`; S3 adapter mock contract. Temporary filesystem and disposable Testcontainers only. Real provider outages, credentials, lifecycle retention and orphan monitoring remain staging/operations work, not manual deterministic DB testing.

AVL scenarios

Shared AVL execution contract: Spring computes normalized weekly ∪ AVAILABLE, subtracts BLOCKED ∪ reserving bookings, clips by elapsed lead time in venue timezone, then normalizes. Intervals are ordered same-day half-open [start,end); adjacent union intervals merge. Empty week is empty base, not 24/7. ACTIVE means approved listing, not guaranteed current slots.

Actors and isolation: Public effective GET respects venue visibility (ACTIVE public; non-ACTIVE own OWNER/ADMIN). Settings GET is own OWNER/ADMIN; anonymous returns403 under the public GET route plus method guard. Mutation is own verified-email OWNER or ADMIN (email exemption); other OWNER/SCOUT403, anonymous401. No owner review/current policies needed for ordinary settings changes. Use unique RUN_ID fixtures, fixed test Clock, disposable Testcontainers and isolated browser DB/profile; never delete shared history or restore fictional snapshots. Schedule mutations have no lifecycle email. Booking/inquiry expiry retains real lifecycle events; email delivery is separate.

AVL-001 — Weekly full replacement

Area: AVL · **Priority:** P1 · **Test level:** API, REAL POSTGRES, BROWSER · **Result:** PASS (local; product/deployed checks separate)

Exact contract: PUT `/api/venues/{id}/availability/weekly` with `{intervals:[]}` clears all rows. Replace with one then multiple Monday ranges; parameterize every weekday. GET settings returns persisted rows sorted by stored day/start, not guaranteed Monday-first lexical order. Effective output agrees; AVAILABLE override can add a one-off interval to an empty week. No status/moderation mutation.

Automation: VenueDomainIntegrationTest#avlWeeklyReplacementAllWeekdays; owner browser. Shared actors, cleanup and limitations above apply. See `docs/qa/AVL_EXECUTION_REPORT.md`.

AVL-002 — Weekly validation and adjacency

Area: AVL · **Priority:** P1 · **Test level:** API, REAL POSTGRES · **Result:** PASS (local; product/deployed checks separate)

Exact contract: DTO requires intervals, max50; each interval has dayOfWeek/startTime/endTime. Missing day/time, reversed/equal/overnight, overlap and51 rows return400 without replacing old rows. Exactly50 valid rows accepted. Adjacent09:00–12:00 and12:00–15:00 persist legally and normalize to09:00–15:00. Wrong actors cannot mutate.

Automation: VenueDomainIntegrationTest#avlWeeklyValidationAndAuthorizationAreAtomic; all-weekday adjacency. Shared actors, cleanup and limitations above apply. See `docs/qa/AVL_EXECUTION_REPORT.md`.

AVL-003 — AVAILABLE/BLOCKED override algebra

Area: AVL · **Priority:** P1 · **Test level:** UNIT, API, REAL POSTGRES, BROWSER · **Result:** PASS (local; product/deployed checks separate)

Exact contract: POST overrides returns201. Base09:00–17:00 plus AVAILABLE08:00–10:00/16:00–18:00 minus BLOCKED11:00–12:00/16:00–19:00 yields08:00–11:00 and12:00–16:00. BLOCKED wins regardless of creation order. All-day AVAILABLE uses[00:00,23:59:59.999999999), then subtraction/clipping; all-day BLOCKED empties the date, with or without base hours. DELETE own override204; immediate requery uses remaining rows.

Automation: AvailabilityAlgebraTest; VenueDomainIntegrationTest#avlOverrides* / avlAllDay*; browser override matrix. Shared actors, cleanup and limitations above apply. See `docs/qa/AVL_EXECUTION_REPORT.md`.

AVL-004 — Override validation, duplicates and scoped deletion

Area: AVL · **Priority:** P1 · **Test level:** API, REAL POSTGRES · **Result:** PASS (local; product/deployed checks separate)

Exact contract: All-day with supplied times, partial missing a time, equal/reversed times return400. Duplicate AVAILABLE/BLOCKED and opposite types are legal: union/subtraction determines result, no invented uniqueness rule. Delete through own venue route using another venue override ID returns404 and retains that row. No rejected-action event/email.

Automation: VenueDomainIntegrationTest#avlOverridesUnionBlockedPrecedenceDuplicatesAndDelete. Shared actors, cleanup and limitations above apply. See `docs/qa/AVL_EXECUTION_REPORT.md`.

AVL-005 — Query range and visibility

Area: AVL · **Priority:** P1 · **Test level:** API, REAL POSTGRES · **Result:** PASS (local; product/deployed checks separate)

Exact contract: GET `/api/venues/{id}/availability?from=YYYY-MM-DD&to=YYYY-MM-DD` returns inclusive date list and venue timezone. Same day and93 days succeed;94, reversed/missing/malformed bounds400. Test all four venue statuses against anonymous/SCOUT/own OWNER/other OWNER/ADMIN. Empty public availability leaves ACTIVE unchanged. Settings and effective reads are separate authorization contracts.

Automation: VenueDomainIntegrationTest#avlQueryRangeVisibilityAndTimezoneChange / avlAllDay*. Shared actors, cleanup and limitations above apply. See `docs/qa/AVL_EXECUTION_REPORT.md`.

AVL-006 — Lead-time boundary, rounding and DST

Area: AVL · **Priority:** P1 · **Test level:** PARAMETERIZED UNIT, BROWSER REPRESENTATIVE · **Result:** PASS (local; product/deployed checks separate)

Exact contract: Fixed Clock with lead120: exact+120 allowed,+119 rejected; display clips earlier intervals and rounds fractional minutes upward. Round on the Instant timeline before comparing local dates:23:59:xx carries into tomorrow. In autumn overlap, requested wall time resolves earlier offset, so a threshold in the second occurrence clips beyond the overlap. Prove elapsed24h across spring/autumn; Java gap-shift/earlier-offset semantics are explicit, not new scheduling policy. Displayed starts must satisfy the lead policy.

Automation: AvailabilityAlgebraTest (including reproduced AVL-D01/D02); VenueBookingLeadTimePolicyTest; calendar client/browser. Shared actors, cleanup and limitations above apply. See `docs/qa/AVL_EXECUTION_REPORT.md`.

AVL-007 — Lead time between workflow stages

Area: AVL · **Priority:** P1 · **Test level:** SERVICE, REAL POSTGRES · **Result:** PASS (local; product/deployed checks separate)

Exact contract: Create a valid REQUESTED booking, advance injected availability Clock inside lead window, accept→409 with persisted EXPIRED and REQUEST_EXPIRED payload.data.reason=MINIMUM_LEAD_TIME_NOT_MET. Inquiry begins outside normal schedule, is OFFERED, then too-late acceptance→409, persisted EXPIRED, OFFER_EXPIRED and no booking. Accepted AWAITING_PAYMENT remains payable inside lead window while payment deadline is future. Do not expire accepted commitments solely due to lead time.

Automation: VenueDomainIntegrationTest#avlLeadExpiryCommitsForRequestAndInquiryButCheckoutIsExempt; existing PaymentServiceLeadTimeExemptionTest. Shared actors, cleanup and limitations above apply. See `docs/qa/AVL_EXECUTION_REPORT.md`.

Shared AVL-008–015 matrix: One parameterized suite covers all eight BookingStatus values. Base09:00–17:00 minus11:00–13:00 splits only for AWAITING_PAYMENT/CONFIRMED; full cover empties only those two. Unit shape matrix covers containment, start/end overlap and touching boundaries. PostgreSQL actual exclusion predicate must exactly match Java reservation policy, using tsrange `[)`. Overlapping non-reserving states coexist; reserving overlap rejected. Two real overlapping requests remain visible until acceptance; concurrent accepts yield one AWAITING_PAYMENT and one UNAVAILABLE, no500 or double reservation. Expiry reopens; confirmation stays blocked; cancellation/refund reopen via actual service transitions with provider transport mocked. A stale calendar request409 cannot force a reservation and the frontend refetches. VEN suspension rights/accepted commitments are reused, not changed.

AVL-008 — Booking subtraction REQUESTED

Area: AVL · **Priority:** P0 · **Test level:** PARAMETERIZED UNIT, REAL POSTGRES · **Result:** PASS (local; product/deployed checks separate)

Matrix row: REQUESTED → 09:00–17:00 unchanged, including full-cover fixture. Read-only subtraction fixture is distinct from a lifecycle transition; no email is caused by querying availability.

Automation: `AvailabilityAlgebraTest#statusAndHalfOpenSubtraction`, `VenueDomainIntegrationTest#avlAllBookingStatusesMatchDatabaseExclusion`, shared concurrent acceptance/transition tests and representative calendar browser journey. Shared matrix, actor/isolation contract and operations limitations above apply.

AVL-009 — Booking subtraction AWAITING_PAYMENT

Area: AVL · **Priority:** P0 · **Test level:** PARAMETERIZED UNIT, REAL POSTGRES · **Result:** PASS (local; product/deployed checks separate)

Matrix row: AWAITING_PAYMENT → 09:00–11:00 and13:00–17:00; full cover→empty. Read-only subtraction fixture is distinct from a lifecycle transition; no email is caused by querying availability.

Automation: `AvailabilityAlgebraTest#statusAndHalfOpenSubtraction`, `VenueDomainIntegrationTest#avlAllBookingStatusesMatchDatabaseExclusion`, shared concurrent acceptance/transition tests and representative calendar browser journey. Shared matrix, actor/isolation contract and operations limitations above apply.

AVL-010 — Booking subtraction CONFIRMED

Area: AVL · **Priority:** P0 · **Test level:** PARAMETERIZED UNIT, REAL POSTGRES · **Result:** PASS (local; product/deployed checks separate)

Matrix row: CONFIRMED → 09:00–11:00 and13:00–17:00; full cover→empty. Read-only subtraction fixture is distinct from a lifecycle transition; no email is caused by querying availability.

Automation: `AvailabilityAlgebraTest#statusAndHalfOpenSubtraction`, `VenueDomainIntegrationTest#avlAllBookingStatusesMatchDatabaseExclusion`, shared concurrent acceptance/transition tests and representative calendar browser journey. Shared matrix, actor/isolation contract and operations limitations above apply.

AVL-011 — Booking subtraction DECLINED

Area: AVL · **Priority:** P0 · **Test level:** PARAMETERIZED UNIT, REAL POSTGRES · **Result:** PASS (local; product/deployed checks separate)

Matrix row: DECLINED → 09:00–17:00 unchanged, including full-cover fixture. Read-only subtraction fixture is distinct from a lifecycle transition; no email is caused by querying availability.

Automation: `AvailabilityAlgebraTest#statusAndHalfOpenSubtraction`, `VenueDomainIntegrationTest#avlAllBookingStatusesMatchDatabaseExclusion`, shared concurrent acceptance/transition tests and representative calendar browser journey. Shared matrix, actor/isolation contract and operations limitations above apply.

AVL-012 — Booking subtraction EXPIRED

Area: AVL · **Priority:** P0 · **Test level:** PARAMETERIZED UNIT, REAL POSTGRES · **Result:** PASS (local; product/deployed checks separate)

Matrix row: EXPIRED → 09:00–17:00 unchanged, including full-cover fixture. Read-only subtraction fixture is distinct from a lifecycle transition; no email is caused by querying availability.

Automation: `AvailabilityAlgebraTest#statusAndHalfOpenSubtraction`, `VenueDomainIntegrationTest#avlAllBookingStatusesMatchDatabaseExclusion`, shared concurrent acceptance/transition tests and representative calendar browser journey. Shared matrix, actor/isolation contract and operations limitations above apply.

AVL-013 — Booking subtraction UNAVAILABLE

Area: AVL · **Priority:** P0 · **Test level:** PARAMETERIZED UNIT, REAL POSTGRES · **Result:** PASS (local; product/deployed checks separate)

Matrix row: UNAVAILABLE → 09:00–17:00 unchanged, including full-cover fixture. Read-only subtraction fixture is distinct from a lifecycle transition; no email is caused by querying availability.

Automation: `AvailabilityAlgebraTest#statusAndHalfOpenSubtraction`, `VenueDomainIntegrationTest#avlAllBookingStatusesMatchDatabaseExclusion`, shared concurrent acceptance/transition tests and representative calendar browser journey. Shared matrix, actor/isolation contract and operations limitations above apply.

AVL-014 — Booking subtraction CANCELLED

Area: AVL · **Priority:** P0 · **Test level:** PARAMETERIZED UNIT, REAL POSTGRES · **Result:** PASS (local; product/deployed checks separate)

Matrix row: CANCELLED → 09:00–17:00 unchanged, including full-cover fixture. Read-only subtraction fixture is distinct from a lifecycle transition; no email is caused by querying availability.

Automation: `AvailabilityAlgebraTest#statusAndHalfOpenSubtraction`, `VenueDomainIntegrationTest#avlAllBookingStatusesMatchDatabaseExclusion`, shared concurrent acceptance/transition tests and representative calendar browser journey. Shared matrix, actor/isolation contract and operations limitations above apply.

AVL-015 — Booking subtraction REFUNDED

Area: AVL · **Priority:** P0 · **Test level:** PARAMETERIZED UNIT, REAL POSTGRES · **Result:** PASS (local; product/deployed checks separate)

Matrix row: REFUNDED → 09:00–17:00 unchanged, including full-cover fixture. Read-only subtraction fixture is distinct from a lifecycle transition; no email is caused by querying availability.

Automation: `AvailabilityAlgebraTest#statusAndHalfOpenSubtraction`, `VenueDomainIntegrationTest#avlAllBookingStatusesMatchDatabaseExclusion`, shared concurrent acceptance/transition tests and representative calendar browser journey. Shared matrix, actor/isolation contract and operations limitations above apply.

MATCH scenarios

Shared MATCH contract (source audited 2026-09-14): authenticated discovery for SCOUT/OWNER/ADMIN, no verified-email/current-policy/owner-review prerequisite. Parser UNIT fixtures have no roles. Gemini maps intent; ACTIVE SQL plus Java decide eligibility; VenueAvailabilityService decides complete exact-interval availability; Gemini ranks only surviving IDs; booking/pricing remain authoritative. Direct venueId uses detail visibility, bypasses Gemini, and is not a reservation. Brief10–1000 required for both routes. No search writes or lifecycle email.

Isolation and evidence: parameterized controlled provider fixtures, RUN_ID venues, disposable Testcontainers PostgreSQL, existing isolated Chrome harness. No fictional snapshot restore or deletion of shared history. See docs/qa/MATCH_EXECUTION_REPORT.md and docs/qa/evidence/match/. Local PASS proves mapping/containment, never live Gemini extraction accuracy

or explanation truthfulness. Live language/model/privacy/operations approval remains separate.

MATCH-001 — Parser extraction: productionActivity

Area: MATCH · **Priority:** P1 · **Test level:** PARAMETERIZED UNIT · **Result:** PASS (local contract; live model quality not approved)

Exact contract: All seven ProductionActivity values map exactly from controlled JSON. Unknown enum fails safely; ambiguous model fixture null stays null. Prompt requires conservative activity inference, never EVENTS.

Automation: MatchingProviderContractTest (shared parser matrix); MatchingTransportContractTest. Shared actors, prerequisites, cleanup and live-provider limitations above apply.

MATCH-002 — Parser extraction: numPeople

Area: MATCH · **Priority:** P1 · **Test level:** PARAMETERIZED UNIT · **Result:** PASS (local contract; live model quality not approved)

Exact contract: numPeople must be a positive JSON integer within Java Integer range; zero/negative/fraction/string/overflow fail safely. Mapping a controlled12 is not proof of live understanding of “a dozen”.

Automation: MatchingProviderContractTest (shared parser matrix); MatchingTransportContractTest. Shared actors, prerequisites, cleanup and live-provider limitations above apply.

MATCH-003 — Parser extraction: requiresNaturalLight

Area: MATCH · **Priority:** P1 · **Test level:** PARAMETERIZED UNIT · **Result:** PASS (local contract; live model quality not approved)

Exact contract: requiresNaturalLight and prefersNaturalLight are distinct nullable Booleans. Hard true filters, false/null do not prohibit daylight. Weak language is preference by prompt contract.

Automation: MatchingProviderContractTest (shared parser matrix); MatchingTransportContractTest. Shared actors, prerequisites, cleanup and live-provider limitations above apply.

MATCH-004 — Parser extraction: requiresBlackout

Area: MATCH · **Priority:** P1 · **Test level:** PARAMETERIZED UNIT · **Result:** PASS (local contract; live model quality not approved)

Exact contract: requiresBlackout and prefersBlackout have distinct hard/preference semantics; true/false/null preserved, wrong scalar type rejected.

Automation: MatchingProviderContractTest (shared parser matrix); MatchingTransportContractTest. Shared actors, prerequisites, cleanup and live-provider limitations above apply.

MATCH-005 — Parser extraction: requiresLoadingAccess

Area: MATCH · **Priority:** P1 · **Test level:** PARAMETERIZED UNIT · **Result:** PASS (local contract; live model quality not approved)

Exact contract: requiresLoadingAccess and prefersLoadingAccess stay separate; unknown output does not invent loading access.

Automation: MatchingProviderContractTest (shared parser matrix); MatchingTransportContractTest. Shared actors, prerequisites, cleanup and live-provider limitations above apply.

MATCH-006 — Parser extraction: requiresParking

Area: MATCH · **Priority:** P1 · **Test level:** PARAMETERIZED UNIT · **Result:** PASS (local contract; live model quality not approved)

Exact contract: requiresParking and prefersParking stay separate; structured parking cannot be satisfied by a parking tag.

Automation: MatchingProviderContractTest (shared parser matrix); MatchingTransportContractTest. Shared actors, prerequisites, cleanup and live-provider limitations above apply.

MATCH-007 — Parser extraction: minimumParkingSpaces

Area: MATCH · **Priority:** P1 · **Test level:** PARAMETERIZED UNIT · **Result:** PASS (local contract; live model quality not approved)

Exact contract: minimumParkingSpaces is a nonnegative JSON integer. Zero remains an explicit minimum and requires parkingAvailable=true in deterministic filtering.

Automation: MatchingProviderContractTest (shared parser matrix); MatchingTransportContractTest. Shared actors, prerequisites, cleanup and live-provider limitations above apply.

MATCH-008 — Parser extraction: requiresWheelchairAccessibility

Area: MATCH · **Priority:** P1 · **Test level:** PARAMETERIZED UNIT · **Result:** PASS (local contract; live model quality not approved)

Exact contract: requiresWheelchairAccessibility is nullable Boolean. There is no dedicated wheelchair preference field; do not invent one.

Automation: MatchingProviderContractTest (shared parser matrix); MatchingTransportContractTest. Shared actors, prerequisites, cleanup and live-provider limitations above apply.

MATCH-009 — Parser extraction: requiresLift

Area: MATCH · **Priority:** P1 · **Test level:** PARAMETERIZED UNIT · **Result:** PASS (local contract; live model quality not approved)

Exact contract: requiresLift and prefersLift are separate nullable Booleans. False requirement is not a prohibition.

Automation: MatchingProviderContractTest (shared parser matrix); MatchingTransportContractTest. Shared actors, prerequisites, cleanup and live-provider limitations above apply.

MATCH-010 — Parser extraction: requiresBathrooms

Area: MATCH · **Priority:** P1 · **Test level:** PARAMETERIZED UNIT · **Result:** PASS (local contract; live model quality not approved)

Exact contract: requiresBathrooms is nullable Boolean; there is no dedicated bathroom preference field.

Automation: MatchingProviderContractTest (shared parser matrix); MatchingTransportContractTest. Shared actors, prerequisites, cleanup and live-provider limitations above apply.

MATCH-011 — Parser extraction: requiresSoundRecordingSuitable

Area: MATCH · **Priority:** P1 · **Test level:** PARAMETERIZED UNIT · **Result:** PASS (local contract; live model quality not approved)

Exact contract: requiresSoundRecordingSuitable is nullable Boolean; no soundproof/acoustic guarantee can be inferred.

Automation: MatchingProviderContractTest (shared parser matrix); MatchingTransportContractTest. Shared actors, prerequisites, cleanup and live-provider limitations above apply.

MATCH-012 — Parser extraction: minimumCeilingHeightM

Area: MATCH · **Priority:** P1 · **Test level:** PARAMETERIZED UNIT · **Result:** PASS (local contract; live model quality not approved)

Exact contract: minimumCeilingHeightM is positive numeric metres, BigDecimal comparison; null is absent requirement. String/zero/negative invalid.

Automation: MatchingProviderContractTest (shared parser matrix); MatchingTransportContractTest. Shared actors, prerequisites, cleanup and live-provider limitations above apply.

MATCH-013 — Parser extraction: requestedDate

Area: MATCH · **Priority:** P1 · **Test level:** PARAMETERIZED UNIT · **Result:** PASS (local contract; live model quality not approved)

Exact contract: ISO requestedDate maps strictly. Prompt only populates exact timing when one reliable date and both bounds exist. Date-only fixture does not run availability.

Automation: MatchingProviderContractTest (shared parser matrix); MatchingTransportContractTest. Shared actors, prerequisites, cleanup and live-provider limitations above apply.

MATCH-014 — Parser extraction: requestedStartTime/requestedEndTime

Area: MATCH · **Priority:** P1 · **Test level:** PARAMETERIZED UNIT · **Result:** PASS (local contract; live model quality not approved)

Exact contract: Local requestedStartTime/requestedEndTime map strictly. Equal/reversed pairs fail; no overnight support. Complete date/start/end invokes availability; partial fields do not.

Automation: MatchingProviderContractTest (shared parser matrix); MatchingTransportContractTest. Shared actors, prerequisites, cleanup and live-provider limitations above apply.

MATCH-015 — Parser extraction: relative timing

Area: MATCH · **Priority:** P1 · **Test level:** PARAMETERIZED UNIT · **Result:** PASS (local contract; live model quality not approved)

Exact contract: Prompt receives fixed reference date and Europe/Amsterdam. Controlled tomorrow response maps against that fixture; local tests do not prove model relative-date resolution.

Automation: MatchingProviderContractTest (shared parser matrix); MatchingTransportContractTest. Shared actors, prerequisites, cleanup and live-provider limitations above apply.

MATCH-016 — Parser extraction: ambiguous timing

Area: MATCH · **Priority:** P1 · **Test level:** PARAMETERIZED UNIT · **Result:** PASS (local contract; live model quality not approved)

Exact contract: “Tomorrow afternoon” and alternatives stay timingDatePreferences context in controlled response; prompt forbids fabricated bounds. Availability skips incomplete timing.

Automation: MatchingProviderContractTest (shared parser matrix); MatchingTransportContractTest. Shared actors, prerequisites, cleanup and live-provider limitations above apply.

MATCH-017 — Parser extraction: budget context

Area: MATCH · **Priority:** P1 · **Test level:** PARAMETERIZED UNIT · **Result:** PASS (local contract; live model quality not approved)

Exact contract: budgetMinEur/budgetMaxEur are nonnegative ordered numeric context. No total-price SQL predicate or authoritative budget-fit claim. Duration-aware quote remains pricing responsibility.

Automation: MatchingProviderContractTest (shared parser matrix); MatchingTransportContractTest. Shared actors, prerequisites, cleanup and live-provider limitations above apply.

MATCH-018 — Hard requirement activity

Area: MATCH · **Priority:** P0 · **Test level:** PARAMETERIZED UNIT, REAL POSTGRES · **Result:** PASS (local contract; live model quality not approved)

Exact contract: Activity must be known permitted; explicit prohibition wins even if also permitted. Null/empty permitted fails. Null prohibited permits a known allowed activity. Parameterize all seven enum values.

Automation: MatchingEligibilityContractTest + MatchingIntegrationTest#sqlAndJavaAgree (shared SQL/Java matrix). Shared actors, prerequisites, cleanup and live-provider limitations above apply.

MATCH-019 — Hard requirement maxCrewSize

Area: MATCH · **Priority:** P0 · **Test level:** PARAMETERIZED UNIT, REAL POSTGRES · **Result:** PASS (local contract; live model quality not approved)

Exact contract: numPeople12 requires maxCrewSize>=12;12/13 pass,11/null fail. SQL and Java agree.

Automation: MatchingEligibilityContractTest + MatchingIntegrationTest#sqlAndJavaAgree (shared SQL/Java matrix). Shared actors, prerequisites, cleanup and live-provider limitations above apply.

MATCH-020 — Hard requirement naturalLight

Area: MATCH · **Priority:** P0 · **Test level:** PARAMETERIZED UNIT, REAL POSTGRES · **Result:** PASS (local contract; live model quality not approved)

Exact contract: requiresNaturalLight=true requires naturalLight=true; false/null venue fails. Requirement false/null does not filter.

Automation: MatchingEligibilityContractTest + MatchingIntegrationTest#sqlAndJavaAgree (shared SQL/Java matrix). Shared actors, prerequisites, cleanup and live-provider limitations above apply.

MATCH-021 — Hard requirement blackoutPossible

Area: MATCH · **Priority:** P0 · **Test level:** PARAMETERIZED UNIT, REAL POSTGRES · **Result:** PASS (local contract; live model quality not approved)

Exact contract: requiresBlackout=true requires blackoutPossible=true; false/null fails. Preference alone retains all.

Automation: MatchingEligibilityContractTest + MatchingIntegrationTest#sqlAndJavaAgree (shared SQL/Java matrix). Shared actors, prerequisites, cleanup and live-provider limitations above apply.

MATCH-022 — Hard requirement loadingAccess

Area: MATCH · **Priority:** P0 · **Test level:** PARAMETERIZED UNIT, REAL POSTGRES · **Result:** PASS (local contract; live model quality not approved)

Exact contract: requiresLoadingAccess=true requires loadingAccess=true; false/null fails.

Automation: MatchingEligibilityContractTest + MatchingIntegrationTest#sqlAndJavaAgree (shared SQL/Java matrix). Shared actors, prerequisites, cleanup and live-provider limitations above apply.

MATCH-023 — Hard requirement parkingAvailable

Area: MATCH · **Priority:** P0 · **Test level:** PARAMETERIZED UNIT, REAL POSTGRES · **Result:** PASS (local contract; live model quality not approved)

Exact contract: requiresParking=true requires parkingAvailable=true; false/null fails; a tag does not replace the structured fact.

Automation: MatchingEligibilityContractTest + MatchingIntegrationTest#sqlAndJavaAgree (shared SQL/Java matrix). Shared actors, prerequisites, cleanup and live-provider limitations above apply.

MATCH-024 — Hard requirement parkingSpaces

Area: MATCH · **Priority:** P0 · **Test level:** PARAMETERIZED UNIT, REAL POSTGRES · **Result:** PASS (local contract; live model quality not approved)

Exact contract: minimumParkingSpaces N requires known spaces>=N AND parkingAvailable=true, including N=0. Equality inclusive.

Automation: MatchingEligibilityContractTest + MatchingIntegrationTest#sqlAndJavaAgree (shared SQL/Java matrix). Shared actors, prerequisites, cleanup and live-provider limitations above apply.

MATCH-025 — Hard requirement wheelchairAccessible

Area: MATCH · **Priority:** P0 · **Test level:** PARAMETERIZED UNIT, REAL POSTGRES · **Result:** PASS (local contract; live model quality not approved)

Exact contract: requiresWheelchairAccessibility=true requires wheelchairAccessible=true; false/null fails.

Automation: MatchingEligibilityContractTest + MatchingIntegrationTest#sqlAndJavaAgree (shared SQL/Java matrix). Shared actors, prerequisites, cleanup and live-provider limitations above apply.

MATCH-026 — Hard requirement liftAvailable

Area: MATCH · **Priority:** P0 · **Test level:** PARAMETERIZED UNIT, REAL POSTGRES · **Result:** PASS (local contract; live model quality not approved)

Exact contract: requiresLift=true requires liftAvailable=true; false/null fails; prefersLift alone never excludes.

Automation: MatchingEligibilityContractTest + MatchingIntegrationTest#sqlAndJavaAgree (shared SQL/Java matrix). Shared actors, prerequisites, cleanup and live-provider limitations above apply.

MATCH-027 — Hard requirement bathroomsAvailable

Area: MATCH · **Priority:** P0 · **Test level:** PARAMETERIZED UNIT, REAL POSTGRES · **Result:** PASS (local contract; live model quality not approved)

Exact contract: requiresBathrooms=true requires bathroomsAvailable=true; false/null fails, distinct from readiness completeness.

Automation: MatchingEligibilityContractTest + MatchingIntegrationTest#sqlAndJavaAgree (shared SQL/Java matrix). Shared actors, prerequisites, cleanup and live-provider limitations above apply.

MATCH-028 — Hard requirement soundRecordingSuitable

Area: MATCH · **Priority:** P0 · **Test level:** PARAMETERIZED UNIT, REAL POSTGRES · **Result:** PASS (local contract; live model quality not approved)

Exact contract: requiresSoundRecordingSuitable=true requires soundRecordingSuitable=true; false/null fails; no acoustic guarantee.

Automation: MatchingEligibilityContractTest + MatchingIntegrationTest#sqlAndJavaAgree (shared SQL/Java matrix). Shared actors, prerequisites, cleanup and live-provider limitations above apply.

MATCH-029 — Hard requirement ceilingHeightM

Area: MATCH · **Priority:** P0 · **Test level:** PARAMETERIZED UNIT, REAL POSTGRES · **Result:** PASS (local contract; live model quality not approved)

Exact contract: minimumCeilingHeightM3.5 requires known height>=3.5;3.49/null fail; BigDecimal-safe inclusive comparison.

Automation: MatchingEligibilityContractTest + MatchingIntegrationTest#sqlAndJavaAgree (shared SQL/Java matrix). Shared actors, prerequisites, cleanup and live-provider limitations above apply.

MATCH-030 — Preference ranking and zero candidates

Area: MATCH · **Priority:** P1 · **Test level:** UNIT, REAL POSTGRES, BROWSER REPRESENTATIVE · **Result:** PASS (local contract; live model quality not approved)

Exact contract: Production eligibility precedes ranking. Only ACTIVE candidates pass SQL, ordered hourly price ascending/size descending/name ascending; up to30 after Java filtering and before availability. Five soft preference fields retain match/unknown/mismatch candidates. No candidates skips ranker. Ranker receives catalogue facts only and returns at most five distinct eligible IDs; numeric scores clamp0–1. SQL currently materializes all predicate matches before limiting: record growth risk, not a fictitious SQL LIMIT.

Automation: GeminiAiServiceTest; MatchingProviderContractTest; MatchingIntegrationTest; MatchingBrowserIntegrationTest (opt-in real UI). Shared actors, prerequisites, cleanup and live-provider limitations above apply.

MATCH-031 — Availability-aware match and direct quote

Area: MATCH · **Priority:** P1 · **Test level:** UNIT, REAL POSTGRES, BROWSER REPRESENTATIVE · **Result:** PASS (local contract; live model quality not approved)

Exact contract: Exact complete date/start/end requires whole interval inside one authoritative available interval. PostgreSQL matrix covers partial/unavailable/BLOCKED/two blocking statuses/REQUESTED/lead/AVAILABLE/partial timing. Empty availability returns empty without relaxing. Direct venueId bypasses both Gemini calls; active accessible venue returns score1 and current hourly/minimum/optional full-day labels, not total price. Non-ACTIVE direct access is own OWNER/ADMIN only. No special booking bypass; AVL stale-reservation coverage reused.

Automation: GeminiAiServiceTest; MatchingProviderContractTest; MatchingIntegrationTest; MatchingBrowserIntegrationTest (opt-in real UI). Shared actors, prerequisites, cleanup and live-provider limitations above apply.

MATCH-032 — Ranker malicious/malformed response

Area: MATCH · **Priority:** P1 · **Test level:** UNIT, REAL POSTGRES, BROWSER REPRESENTATIVE · **Result:** PASS (local contract; live model quality not approved)

Exact contract: Unknown and filtered-out IDs never appear; filter membership and deduplicate before five-result limit. Invalid score types/nonfinite values and malformed roots/rows return safe502, not generic500 or raw provider body; ordinary finite numbers preserve existing0–1 clamping. Missing/null explanation stays empty. Code fences strip at transport boundary. No parser/ranker fallback is invented. User/venue text is untrusted prompt data; explanation renders as React text. Prompt semantic compliance remains live-model review. Header key transport and safe logging prevent application error leakage.

Automation: GeminiAiServiceTest; MatchingProviderContractTest; MatchingIntegrationTest; MatchingBrowserIntegrationTest (opt-in real UI). Shared actors, prerequisites, cleanup and live-provider limitations above apply.

BK scenarios

The BK contract is audited against BookingService, BookingStateMachine, BookingController, BookingAvailabilityValidator, BookingReservationPolicy and V1 exclusion constraint. Execution/evidence: BK execution report. Payment settlement, refunds and payouts remain separate domains.

Shared environment / cleanup: UNIT and real disposable PostgreSQL via `mvn verify -Ppostgres-integration`; unique fictional RUN_ID accounts/venues; fixed Clock for core deadlines; isolated Chrome profile. Never delete shared booking, policy or audit history. QA policies say “QA TEST CONTENT — NOT LEGAL TERMS”. No fake snapshot reset.

Shared prerequisites: SCOUT request and OWNER response need active account, verified email and all current required policies. ADMIN response is explicitly supported and exempt from email/policy acceptance, but ADMIN cannot create a scout request. Booking endpoints do not directly require owner verification. Request venue must be ACTIVE and interval satisfy availability/lead time. Read history remains available after policy replacement. Unit fixtures have no irrelevant role prerequisites.

Notification boundary: Request publication targets owner + scout; acceptance, decline, request expiry and unavailable target scout. After-commit listener contains delivery failures and suppresses stale status notifications. Denied/invalid operations do not publish lifecycle transitions, except deadline/lead-time acceptance conflicts deliberately commit request expiry. Request and acceptance create no provider payment/refund. Local email-disabled mode is not real delivery proof. Human UX and staging email/deployment review remain separate from local software results.

BK-001 — Standard request and immutable snapshots

Area: BK · **Priority:** P0 · **Levels:** UNIT / API / REAL POSTGRES / representative BROWSER · **Result:** LOCAL PASS (2026-09-14; human/staging separate)

Actors: SCOUT creates; current venue OWNER or ADMIN accepts; both participants/admin read.

Execution and assertions: POST /api/bookings with D14 09:15–13:45, request contact and complete brief → 201 REQUESTED. Selected duration 270; billable duration max(270, venue minimum). Compare quote with BookingPriceCalculator, not a second formula. Capture hourly/minimum/full-day, commission and both contact snapshots, then change venue pricing/live contacts and publish replacement policy. Historical snapshots/references remain unchanged after authorized acceptance. Checkout claim takes stored venuePriceEur; no live provider payment.

Contract notes: REQUESTED does not reserve. Four references: TERMS_OF_SERVICE, ACCEPTABLE_USE_POLICY, PRODUCTION_POLICY, INSURANCE_REQUIREMENTS. PRIVACY_POLICY excluded. Scout contact comes from request; owner contact from profile. Commission config is captured at creation, not recalculated on accept.

Automation: BookingCoreIntegrationTest.exact270MinutesAndHistoricalSnapshotsSurviveAcceptance (minimum 60/360); policyReplacementAndLiveEmailsDoNotRewriteBookingHistory; BookingPriceCalculatorTest; BookingBrowserIntegrationTest.

Cleanup and human limitation: Shared disposable-fixture strategy above; human UX/staging deployment and actual email delivery remain separate. See BK report for results and evidence.

BK-002 — Invalid creation and side-effect boundaries

Area: BK · **Priority:** P0 · **Levels:** UNIT / API / REAL POSTGRES / representative BROWSER · **Result:** LOCAL PASS (2026-09-14; human/staging separate)

Actors: SCOUT valid/invalid trust; OWNER and ADMIN creation denied; anonymous denied.

Execution and assertions: Parameterized HTTP matrix: malformed/null required data, blank brief/contact and reversed/equal interval → 400; past/too-soon/unavailable/blocked → 409; non-ACTIVE venue → 404; wrong role/unverified/stale policies → 403; anonymous → 401. Past date reaches lead-time validation first. No booking/event/notification/provider side effects on rejection. A short positive interval is legal with minimum billable minutes.

Contract notes: Do not flatten domain conflicts into 400. Email authority is current backend user state. Policy replacement is a prerequisite failure, not booking mutation.

Automation: BookingCoreIntegrationTest.rejectedCreationHasNoRowsEventsOrProviderEffects; exact270MinutesAndHistoricalSnapshotsSurviveAcceptance; POL/VEN/AVL prerequisite coverage; BookingAvailabilityValidatorTest.

Cleanup and human limitation: Shared disposable-fixture strategy above; human UX/staging deployment and actual email delivery remain separate. See BK report for results and evidence.

BK-003 — Overlapping REQUESTED bookings coexist

Area: BK · **Priority:** P0 · **Levels:** UNIT / API / REAL POSTGRES / representative BROWSER · **Result:** LOCAL PASS (2026-09-14; human/staging separate)

Actors: Two authenticated eligible SCOUT accounts; venue OWNER later accepts.

Execution and assertions: Create exact, partial and contained overlapping requests, plus adjacent requests; all return distinct 201 REQUESTED. Effective availability is unchanged until acceptance. Database permits these overlaps. Each successful request has REQUEST_SUBMITTED; no payment ID or reservation.

Contract notes: REQUESTED is non-blocking. Do not add a request-capacity rule or infer inventory from an email.

Automation: BookingCoreIntegrationTest.halfOpenCompetitorsOnlyBecomeUnavailable; VenueDomainIntegrationTest.avlAllBookingStatusesMatchDatabaseExclusion; BookingNotificationServiceTest.

Cleanup and human limitation: Shared disposable-fixture strategy above; human UX/staging deployment and actual email delivery remain separate. See BK report for results and evidence.

BK-004 — Owner response deadline and committed expiry

Area: BK · **Priority:** P0 · **Levels:** UNIT / API / REAL POSTGRES / representative BROWSER · **Result:** LOCAL PASS (2026-09-14; human/staging separate)

Actors: Eligible owning OWNER or ADMIN; fixed Clock and independent bookings at each boundary.

Execution and assertions: At T0 create request; ownerResponseExpiresAt = T0 + configured owner-response-window-hours (default24), not capped to shoot start. Accept one microsecond before → 200 AWAITING_PAYMENT. Equality/after → 409 while EXPIRED and REQUEST_EXPIRED commit once. Repeat accept/expiry worker; no duplicate event/notification. Reads never extend deadline.

Contract notes: Direct acceptance enforces expiry independently of scheduler. Request expiry differs from payment-window expiry and explicit decline. Lead-time expiry may occur before response deadline. No owner reminder/escalation is implemented.

Automation: BookingCoreIntegrationTest.exactResponseDeadlineCommitsExpiryBeforeConflict; persistedExpiryIsIdempotent (3 repetitions); PostgresCriticalPathIntegrationTest bounded SKIP LOCKED workers.

Cleanup and human limitation: Shared disposable-fixture strategy above; human UX/staging deployment and actual email delivery remain separate. See BK report for results and evidence.

BK-005 — Owner acceptance reserves and resolves competitors

Area: BK · **Priority:** P0 · **Levels:** UNIT / API / REAL POSTGRES / representative BROWSER · **Result:** LOCAL PASS (2026-09-14; human/staging separate)

Actors: Owning OWNER or ADMIN accepts; unrelated actors denied.

Execution and assertions: POST /api/bookings/{id}/accept → 200 AWAITING_PAYMENT. acceptedAt and paymentExpiresAt = acceptance + configured payment-window-hours (default24), not capped to shoot start. Lock venue then booking, flush reservation, resolve only half-open overlapping REQUESTED to UNAVAILABLE. Adjacent and different-day requests remain; terminal rows stay terminal. No Mollie call/payment ID at acceptance.

Contract notes: Exactly AWAITING_PAYMENT/CONFIRMED block in Java, availability and PostgreSQL. UNAVAILABLE is not owner decline. Two concurrent acceptances cannot reserve overlapping intervals. Failed competitor transaction rolls back winner and audit.

Automation: BookingCoreIntegrationTest.halfOpenCompetitorsOnlyBecomeUnavailable; actorBoundaryAndExplicitAdminDelegation; competitorFailureRollsBackFlushedWinnerAndAudit; afterCommitNotificationFailureCannotUndoReservation; VenueDomainIntegrationTest.avlConcurrentAcceptanceReservesOnceAndExpiryReopens (3 races).

Cleanup and human limitation: Shared disposable-fixture strategy above; human UX/staging deployment and actual email delivery remain separate. See BK report for results and evidence.

BK-006 — Acceptance rechecks changed conditions

Area: BK · **Priority:** P0 · **Levels:** UNIT / API / REAL POSTGRES / representative BROWSER · **Result:** LOCAL PASS (2026-09-14; human/staging separate)

Actors: Owning OWNER or ADMIN, current policies; request was valid when created.

Execution and assertions: Independently add BLOCKED override, advance inside lead window, suspend venue, or reserve overlapping slot before accept. BLOCKED/suspended/reserved conflict cannot promote request. Lead failure commits EXPIRED + REQUEST_EXPIRED with MINIMUM_LEAD_TIME_NOT_MET then 409. Suspension itself does not cancel historical commitments or invalidate existing AWAITING_PAYMENT checkout.

Contract notes: Existing VEN suspension fix is a required regression. Override/suspension conflict leaves REQUESTED; successful competing acceptance resolves overlapping requests to UNAVAILABLE. Do not invent automatic expiry/cancellation for every failure.

Automation: BookingCoreIntegrationTest.acceptanceRevalidatesCurrentConditions; VenueDomainIntegrationTest.suspendedVenueCannotAcceptRequestedButKeepsAcceptedAndConfirmedHistory; suspensionVersusRequestAcceptanceSerializesCommitmentBoundary; avlLeadExpiryCommitsForRequestAndInquiryButCheckoutIsExempt.

Cleanup and human limitation: Shared disposable-fixture strategy above; human UX/staging deployment and actual email delivery remain separate. See BK report for results and evidence.

BK-007 — Decline authorization and state transitions

Area: BK · **Priority:** P1 · **Levels:** UNIT / API / REAL POSTGRES / representative BROWSER · **Result:** LOCAL PASS (2026-09-14; human/staging separate)

Actors: Owning OWNER or ADMIN; other OWNER, both SCOUT roles and anonymous denied.

Execution and assertions: POST /api/bookings/{id}/decline → 200 DECLINED only from REQUESTED. Repeat →409, no duplicate event/email. Parameterize all seven other statuses:409 unchanged. Unauthorized →403; anonymous401. No payment/refund; availability stays open. Current decline does not recheck deadline, venue status or lead time.

Contract notes: REQUEST_DECLINED targets scout. UI must refetch on both review-action409 responses and display safe error, never force a local status.

Automation: BookingCoreIntegrationTest.actorBoundaryAndExplicitAdminDelegation; declineRejectsEveryNonRequestStateWithoutEffects; BookingStateMachineTest complete64-pair contract; BookingBrowserIntegrationTest stale decline and successful decline.

Cleanup and human limitation: Shared disposable-fixture strategy above; human UX/staging deployment and actual email delivery remain separate. See BK report for results and evidence.

BK-008 — Actor-scoped history and contact privacy

Area: BK · **Priority:** P0 · **Levels:** UNIT / API / REAL POSTGRES / representative BROWSER · **Result:** LOCAL PASS (2026-09-14; human/staging separate)

Actors: SCOUT participants; current venue OWNER; ADMIN operational read; unrelated users/anonymous denied.

Execution and assertions: GET /api/bookings returns SCOUT own, OWNER venue bookings, ADMIN all. Matrix of two scouts/two owners/three bookings. Detail participant/admin200; unrelated403; missing404; malformedUUID400; anonymous401. Inspect serialized response for absence of account secrets/bank data. Snapshot contacts remain historical after live contact changes.

Contract notes: Ordinary ADMIN list is supported, not just admin operations route. Shared BookingDto includes participant contacts and financial snapshot fields. No live User serialization. Contact visibility/retention is a product privacy decision, not permission to invent redaction.

Automation: BookingCoreIntegrationTest.ordinaryReadListsAreScopedAndDetailsDoNotSerializeUserSecrets; policyReplacementAndLiveEmailsDoNotRewriteBookingHistory; BookingServiceTest; BookingBrowserIntegrationTest.

Cleanup and human limitation: Shared disposable-fixture strategy above; human UX/staging deployment and actual email delivery remain separate. See BK report for results and evidence.

INQ scenarios

Execution: INQ execution report. Source is authoritative; the four IDs map to parameterized HTTP/PostgreSQL matrices and a representative real browser journey.

Shared fixtures/cleanup: Unique fictional QA users and venues, fixed Clock, disposable PostgreSQL Testcontainers, isolated Chrome profile; temporary auth configuration is mode 0600 and removed after execution. Do not delete shared inquiry/booking history or restore a nonexistent snapshot. No real payment or email provider calls. Unit notification tests and local disabled-email logs prove software behavior; deployed delivery is separate.

State model: OPEN → OFFERED/DECLINED; OFFERED → ACCEPTED/EXPIRED. OPEN and OFFERED are not reservations. Acceptance alone creates AWAITING_PAYMENT and a unique booking link atomically. Inquiry actions never write weekly operating hours or availability overrides. No custom offer price, offer revision/withdrawal, scout decline, OPEN expiry, or inquiry event-history table exists.

INQ-001 — Unavailable-slot inquiry creation

Area: INQ · **Priority:** P1 · **Test level:** API, REAL POSTGRES, BROWSER REPRESENTATIVE · **Result:** PASS locally — see execution report

Actors/prerequisites: Active, verified-email SCOUT with current policies; ACTIVE venue and valid interval meeting venue lead time. No scout owner-verification/payout requirement. OWNER/ADMIN 403, anonymous 401. GET mine is SCOUT-only; GET owned-venues OWNER-only. Lists are actor-scoped and do not require policy acceptance.

Contract: POST `/availability-inquiries` with venueId, requestedDate, requestedStartTime, requestedEndTime, message. Outside weekly 09–17, D14 18–20 returns 201 OPEN. Exact local requested times/message persist; DTO timezone is current venue metadata, not a historical timezone snapshot. Normally available 09–11 returns 400 directing normal booking requests, including with an overlapping non-blocking REQUESTED booking. A blocking booking may make an inquiry eligible, but cannot be bypassed during offer/accept.

Validation: Nonblank message max 1000 before service trim; null/empty/whitespace/1001→400, one character/1000→201. Browser empty input supplies “I'd like to ask about this time.” Invalid/missing date/time or same-day unordered interval→400; past/too-soon valid interval→409 via lead policy. Exact lead threshold allowed. Non-ACTIVE→404. Unknown command fields cannot set status/link.

Persistence/side effects: One OPEN row and CREATED lifecycle event/email to owner; no booking, reservation, calendar mutation or payment. Denied/invalid requests leave no row/event/notification. Dashboard DTO has no contact/private financial data; owner creation email currently includes scout contact information, documented for privacy review. React renders message as text.

Automation: InquiryIntegrationTest message, invalid creation, routing, exact lead, actor/read/privacy matrices; existing AvailabilityInquiryServiceTest and AvailabilityInquiryNotificationServiceTest; InquiryBrowserIntegrationTest calendar mode/fallback/OPEN. Shared cleanup above. Human review is wording and flow clarity, not binary/API edge cases.

INQ-002 — Owner offers same or different one-off slot

Area: INQ · **Priority:** P1 · **Test level:** API, REAL POSTGRES, BROWSER REPRESENTATIVE · **Result:** PASS locally — see execution report

Actors/prerequisites: Only the venue's active verified-email OWNER with current policies may POST `/{id}/offer`; other owner/scouts/ADMIN 403, anonymous 401. Existing OPEN inquiry, ACTIVE venue, ordered valid offered interval meeting lead time and no blocking overlap.

Contract: Body offeredDate/offeredStartTime/offeredEndTime; same or different slot allowed, including outside weekly hours or through calendar BLOCKED overrides. Existing AWAITING_PAYMENT/CONFIRMED overlap→409; other booking statuses do not block. Half-open adjacency is legal. Expiry is min(now+configured 48h default, offered slot start in venue timezone). Duplicate/non-OPEN offer→409 with no rewrite or expiry extension. Missing/malformed fields→400; past/too-soon→409. Suspended venue→409, no OFFERED transition.

Persistence/side effects: Preserve requested interval; persist offered interval/offeredAt/offerExpiresAt and OFFERED status. One OFFERED lifecycle notification to scout. No booking, price snapshot, reservation or calendar write. Offers serialize with venue moderation; an offer remains non-blocking even after successful save.

Automation: InquiryIntegrationTest blocking-status/adjacency, expiry cap, exact lead, invalid offer, all-state/action, current account/policy and suspension cases. Browser requested versus different offered time, reload, stale owner offer/decline conflict refetch with retained feedback. Shared cleanup above. Product decisions: price locking, temporary reservation and offer revision are not implemented by this test pass.

INQ-003 — Atomic offer acceptance and late conflict

Area: INQ · **Priority:** P0 · **Test level:** API, REAL POSTGRES, BROWSER REPRESENTATIVE · **Result:** PASS locally — see execution report

Actors/prerequisites: Only originating active verified-email SCOUT with current policies accepts via POST `/{id}/accept`; other scout/owners/ADMIN 403, anonymous 401. OFFERED, before expiry, ACTIVE venue, valid lead time and no blocking overlap. No ordinary weekly-availability requirement for the exception booking.

Contract: Returns 200 BookingDto directly AWAITING_PAYMENT; inquiry ACCEPTED with acceptedBookingId. Standard payment deadline defaults acceptance+24h; no owner-response deadline or second owner acceptance. Current authoritative venue price/commission/live participant contacts/four booking policy references are snapshotted at conversion. Offer is not a price lock. Normal shared checkout uses stored booking amount (mocked provider only).

Late outcomes: At/after offerExpiresAt→409, remains OFFERED until worker expires it. Lead failure→EXPIRED committed before 409. Occupied→409 remains OFFERED. Non-ACTIVE→404 remains OFFERED. Duplicate acceptance→409 with original single booking/link. Missing ID 404/malformed UUID 400. No failed path may leave ACCEPTED with a missing booking.

Persistence/concurrency: Same-inquiry lock plus venue lock serializes double acceptance, different overlapping offers, normal reservation and suspension. PostgreSQL exclusion permits exactly one blocking reservation. Added stricter disposable-DB failure injection proves booking/policy/audit writes roll back if inquiry save fails and no after-commit success notification is emitted. Calendar tables remain unchanged; the created booking now blocks inventory.

Events/UI: Only booking REQUEST_ACCEPTED event with source availability_inquiry and its scout payment notification; no duplicate inquiry-accepted event/email. Browser accepts, redirects `/bookings/{id}`, reloads AWAITING_PAYMENT; stale acceptance refetches and retains safe feedback. No real Mollie call or payment success claim.

Automation: InquiryIntegrationTest conversion snapshots/shared mocked checkout, deadlines, policy replacement, changed account authority, atomicity, repeated double/two-offer/normal-accept/suspension races; BK/AVL exclusion and checkout tests reused. InquiryBrowserIntegrationTest. Shared cleanup above. Product review: current acceptance-time price and live venue timezone semantics.

INQ-004 — Decline, expiry and scoped dashboard history

Area: INQ · **Priority:** P1 · **Test level:** API, REAL POSTGRES, BROWSER REPRESENTATIVE · **Result:** PASS locally — see execution report

Actors/prerequisites: Own verified-email/current-policy OWNER declines OPEN via POST `/{id}/decline`; no ADMIN delegation. Decline does not require ACTIVE because it creates no commitment. Other actors 403/anonymous 401. No decline reason field exists. Only due OFFERED rows expire.

Contract: OPEN→DECLINED with declinedAt; duplicate/other status→409 without extra notification. Worker `expireOffers(now)` locks at most 100 due offers with SKIP LOCKED; OFFERED→EXPIRED with expiredAt and one OFFER_EXPIRED notification. Repeated processing is idempotent. Default scan 60s; direct accept rejects overdue offers independently of scheduler. OPEN has no response deadline/expiry.

Persistence/notifications: Neither decline nor expiry creates booking/payment/refund/calendar changes. DECLINED email means inquiry declined, not booking declined. OFFER_EXPIRED means offer expired. Application lifecycle events are not a persisted inquiry timeline; booking audit begins at conversion.

Dashboard/privacy: `/scout/inquiries` shows own waiting/declined/offered/expired/accepted history and booking link; `/owner/inquiries` only owned-venue inquiries. Original requested and offered intervals both remain visible; expiry uses venue timezone even with a different browser timezone. No contact data is added to inquiry DTO/UI. Lists currently unpaginated.

Automation: InquiryIntegrationTest five-state×four-action matrix, actor-scoped reads, repeated expiry/idempotency; PostgresCriticalPathIntegrationTest inquiry SKIP LOCKED disjoint-worker coverage; existing notification recipient/template/failure tests; representative browser decline/stale refetch/offer/accept. Shared cleanup above. Human UX and real deployed email/scheduler behavior remain separately listed in execution report.

PRICE scenarios — audited 2026-09-15

Source authority: `BookingPriceCalculator`, venue DTO/entity money constraints, `BookingService`, `PaymentCreationCoordinator`, `BookingFinancials`, cancellation/payout accounting, receipt renderer and frontend `lib/pricing.ts`. Execution and defects: PRICE execution report; evidence: `docs/qa/evidence/price/`.

Shared contract: selected duration is same-date LocalTime minutes. Billable minutes = max(selected, minimum). Hourly total = rate × billable /60 rounded to two decimals HALF_UP. Optional cap is separately normalized to two decimals HALF_UP; final price = min(rounded hourly total, normalized cap). Cap changes money, never selected/billable minutes. Minimum is a billing minimum, not a minimum selectable interval. No hidden quarter/hour increments. Overnight intervals are unsupported. DST billing uses selected wall-clock minutes; elapsed-time scheduling remains a separate contract.

Execution: all eleven IDs map to one parameterized `PriceContractTest` shared fixture and `scripts/qa/price-client-check.cjs`; exact cent parity, no tolerance. No actors, authentication, policies, database, clock, emails or providers are required for pure math. Representative PRICE-001/006/007/010 use actual HTTP/real PostgreSQL in `PriceIntegrationTest` to verify immutable snapshots and mocked checkout. Browser representatives cover minimum, prorating, cap, uncapped price and server price winning over a stale preview. No live Mollie payment is claimed.

Money authority: preview is indicative. Spring snapshots price/rate/minimum/cap/durations/commission at normal request creation or inquiry acceptance. Acceptance never reprices an existing booking. Checkout ignores client monetary fields and uses `Booking.venuePriceEur`. Commission, cancellation, payout and receipt consume stored historical economics. Matching before duration only describes rate/minimum/optional cap.

Bounds: rate/cap DTOs require 0.01..99,999,999.99; minimum 1..10,080. Fractional-cent configuration remains accepted and now normalizes to PostgreSQL cents before returning a venue DTO. Sub-minute selected duration and a final price exceeding NUMERIC(10,2) fail safely with HTTP 400 before booking persistence. A fitting cap may bring a large uncapped calculation into range. Positive settings can still round to 0.00: documented accounting decision, not an invented minimum price or verified provider capability.

Cleanup: shared deterministic fixtures, unique fictional QA accounts/venues, disposable Testcontainers PostgreSQL, isolated Chrome profile and temporary auth config. Never delete shared booking history. Successful booking integration has existing BK lifecycle notifications; pure pricing has none; provider transport is mocked locally.

ID	Selected minutes	Billing minimum	EUR/hour	Cap EUR	Billable minutes	Expected EUR
PRICE-001	30	60	100	none	60	100.00
PRICE-002	60	60	100	none	60	100.00
PRICE-003	90	60	100	none	90	150.00
PRICE-004	270	60	100	none	270	450.00
PRICE-005	480	60	100	500	480	500.00
PRICE-006	30	240	100	200	240	200.00
PRICE-007	61	1	19.99	none	61	20.32

ID	Selected minutes	Billing minimum	EUR/hour	Cap EUR	Billable minutes	Expected EUR
PRICE-008	1	1	0.30	none	1	0.01
PRICE-009	299	1	100	500	299	498.33
PRICE-010	300	1	100	500	300	500.00
PRICE-011	301	1	100	500	301	500.00

PRICE-001 — 30 selected minutes, 60 minimum, 100/hour, cap none

Priority: P0 · **Level:** UNIT + FRONTEND CLIENT + REAL POSTGRES/API + MOCKED CHECKOUT · **Result:** PASS

Run shared fixture `PRICE-001` through Java calculator and actual frontend pricing module. Assert selected=30, billable=60, rounded hourly=100.00, final=100.00 exactly. Assert frontend returns the same cent amount. Shared snapshot, downstream and browser evidence applies as described above; no duplicated authorization matrix or database arithmetic implementation.

PRICE-002 — 60 selected minutes, 60 minimum, 100/hour, cap none

Priority: P0 · **Level:** UNIT + FRONTEND CLIENT · **Result:** PASS

Run shared fixture `PRICE-002` through Java calculator and actual frontend pricing module. Assert selected=60, billable=60, rounded hourly=100.00, final=100.00 exactly. Assert frontend returns the same cent amount. Shared snapshot, downstream and browser evidence applies as described above; no duplicated authorization matrix or database arithmetic implementation.

PRICE-003 — 90 selected minutes, 60 minimum, 100/hour, cap none

Priority: P0 · **Level:** UNIT + FRONTEND CLIENT · **Result:** PASS

Run shared fixture `PRICE-003` through Java calculator and actual frontend pricing module. Assert selected=90, billable=90, rounded hourly=150.00, final=150.00 exactly. Assert frontend returns the same cent amount. Shared snapshot, downstream and browser evidence applies as described above; no duplicated authorization matrix or database arithmetic implementation.

PRICE-004 — 270 selected minutes, 60 minimum, 100/hour, cap none

Priority: P0 · **Level:** UNIT + FRONTEND CLIENT · **Result:** PASS

Run shared fixture `PRICE-004` through Java calculator and actual frontend pricing module. Assert selected=270, billable=270, rounded hourly=450.00, final=450.00 exactly. Assert frontend returns the same cent amount. Shared snapshot, downstream and browser evidence applies as described above; no duplicated authorization matrix or database arithmetic implementation.

PRICE-005 — 480 selected minutes, 60 minimum, 100/hour, cap 500

Priority: P0 · **Level:** UNIT + FRONTEND CLIENT · **Result:** PASS

Run shared fixture `PRICE-005` through Java calculator and actual frontend pricing module. Assert selected=480, billable=480, rounded hourly=800.00, final=500.00 exactly. Assert frontend returns the same cent amount. Shared snapshot, downstream and browser evidence applies as described above; no duplicated authorization matrix or database arithmetic implementation.

PRICE-006 — 30 selected minutes, 240 minimum, 100/hour, cap 200

Priority: P0 · **Level:** UNIT + FRONTEND CLIENT + REAL POSTGRES/API + MOCKED CHECKOUT · **Result:** PASS

Run shared fixture `PRICE-006` through Java calculator and actual frontend pricing module. Assert selected=30, billable=240, rounded hourly=400.00, final=200.00 exactly. Assert frontend returns the same cent amount. Shared snapshot, downstream and browser evidence applies as described above; no duplicated authorization matrix or database arithmetic implementation.

PRICE-007 — 61 selected minutes, 1 minimum, 19.99/hour, cap none

Priority: P0 · **Level:** UNIT + FRONTEND CLIENT + REAL POSTGRES/API + MOCKED CHECKOUT · **Result:** PASS

Run shared fixture `PRICE-007` through Java calculator and actual frontend pricing module. Assert selected=61, billable=61, rounded hourly=20.32, final=20.32 exactly. Assert frontend returns the same cent amount. Shared snapshot, downstream and browser evidence applies as described above; no duplicated authorization matrix or database arithmetic implementation.

PRICE-008 — 1 selected minutes, 1 minimum, 0.30/hour, cap none

Priority: P0 · **Level:** UNIT + FRONTEND CLIENT · **Result:** PASS

Run shared fixture `PRICE-008` through Java calculator and actual frontend pricing module. Assert selected=1, billable=1, rounded hourly=0.01, final=0.01 exactly. Assert frontend returns the same cent amount. Shared snapshot, downstream and browser evidence applies as described above; no duplicated authorization matrix or database arithmetic implementation.

PRICE-009 — 299 selected minutes, 1 minimum, 100/hour, cap 500

Priority: P0 · **Level:** UNIT + FRONTEND CLIENT · **Result:** PASS

Run shared fixture `PRICE-009` through Java calculator and actual frontend pricing module. Assert selected=299, billable=299, rounded hourly=498.33, final=498.33 exactly. Assert frontend returns the same cent amount. Shared snapshot, downstream and browser evidence applies as described above; no duplicated authorization matrix or database arithmetic implementation.

PRICE-010 — 300 selected minutes, 1 minimum, 100/hour, cap 500

Priority: P0 · **Level:** UNIT + FRONTEND CLIENT + REAL POSTGRES/API + MOCKED CHECKOUT · **Result:** PASS

Run shared fixture `PRICE-010` through Java calculator and actual frontend pricing module. Assert selected=300, billable=300, rounded hourly=500.00, final=500.00 exactly. Assert frontend returns the same cent amount. Shared snapshot, downstream and browser evidence applies as described above; no duplicated authorization matrix or database arithmetic implementation.

PRICE-011 — 301 selected minutes, 1 minimum, 100/hour, cap 500

Priority: P0 · **Level:** UNIT + FRONTEND CLIENT · **Result:** PASS

Run shared fixture `PRICE-011` through Java calculator and actual frontend pricing module. Assert selected=301, billable=301, rounded hourly=501.67, final=500.00 exactly. Assert frontend returns the same cent amount. Shared snapshot, downstream and browser evidence applies as described above; no duplicated authorization matrix or database arithmetic implementation.

Local conclusion: canonical PRICE-001–011 pass unchanged. PRICE-DEF-005, previously an unresolved accounting decision, is resolved by the explicit Eema v1 policy: free bookings are not supported. After the unchanged exact-minute/minimum/hourly/HALF_UP/normalized-cap calculation, a final total below EUR 0.01 is rejected with HTTP 400 before booking persistence/payment creation. Inquiry acceptance remains coherently OFFERED on rejection. PRICE-008 remains valid at EUR 0.01. Checkout refuses legacy/corrupt stored amounts below EUR 0.01 without recomputation or provider interaction. Common migration V13 enforces the original-price invariant without changing legitimate zero paid/refund/payout amounts. Eight PostgreSQL and three service regression cases plus full backend verification passed; historical evidence and staging boundaries remain in the PRICE report.

PAY — Payment execution contract (audited 2026-09-15)

Authority: `PaymentCreationCoordinator`, `PaymentService`, `HttpMollieProviderClient`, `WebhookService`, locked `BookingService.processPaidPayment`, `RefundCoordinator`, and server-read frontend `/booked`. Full evidence, defects and limits: PAY execution report, `docs/qa/evidence/payment/`.

Three separate state machines: Booking `REQUESTED → AWAITING_PAYMENT → CONFIRMED`, or `AWAITING_PAYMENT → EXPIRED`. Payment creation `NOT_STARTED → CREATING → CREATED`. Refund `NONE → CREATING → QUEUED/PENDING/PROCESSING → REFUNDED`, with FAILED/CANCELED outcomes. A late paid booking stays EXPIRED while its refund progresses. No PENDING/PAID booking states, redirect confirmation, or second owner confirmation.

Shared execution: one parameterized `PaymentIntegrationTest` uses real PostgreSQL, fixed Clock and mocked Mollie; `PaymentTransportContractTest` verifies actual adapter payload/JSON handling and log redaction. Existing PRICE/BK/INQ/notification/payout coverage is referenced, not copied. `PaymentBrowserIntegrationTest` runs a representative real Chrome/UI/API journey with controlled provider navigation. Mocks do not establish real Mollie behavior.

Actors: checkout uses current active/verified user policy, centralized current policy acceptance and exact booking.scoutId ownership. No delegated owner/ADMIN checkout; normal unrelated actors403, anonymous401. An ADMIN policy exemption does not grant another scout's payment authority. Webhook is public form POST and fetches provider facts server-side.

Cleanup: unique fictional users/venues/bookings in disposable Testcontainers PostgreSQL; isolated browser profile and temporary auth config removed afterward. No shared staging history deletion, no fictitious fixture snapshot restoration. Real delivery, provider replay and deployed permissions remain staging checks.

PAY-001 — Checkout snapshot and association

Layer: UNIT / API / REAL POSTGRES. **Priority:** P0 payment integrity.

Eligible AWAITING_PAYMENT booking, deadline future. POST `/bookings/{id}/checkout` returns201 with server `bookingId`, `checkoutUrl`, `molliePaymentId`, `totalAmount`, `expiresIn`. Provider request uses stored `venuePriceEur`, EUR and server bookingId metadata. Current venue price changes and body/query amount/currency/total fields cannot change it. Configured redirect is passed; blank local webhook omitted; production HTTPS validation retained. Incomplete/mismatched response or unsafe checkout URL fails without attaching an ID. Stored/generated URL only; no client provider URL construction.

Evidence: `PaymentIntegrationTest`, transport/unit references and representative browser evidence mapped in PAY report.

PAY-002 — Checkout eligibility and exact deadline

Layer: PARAMETERIZED API / REAL POSTGRES. **Priority:** P0 payment integrity.

All eight BookingStatus values: only AWAITING_PAYMENT checks out; other states400. Wrong participant403, anonymous401, unverified database email403 despite stale token, missing/stale policies403 before provider call. Deadline before/equal/after:201/409/409. Discovered expiry commits EXPIRED and one APPROVAL_EXPIRED event before409; worker retries do not duplicate it. Accepted commitments remain checkout-eligible inside the venue lead-time window until payment deadline.

Evidence: `PaymentIntegrationTest`, transport/unit references and representative browser evidence mapped in PAY report.

PAY-003 — Durable creation identity and concurrency

Layer: REAL POSTGRES / REPEATED BARRIER RACE. **Priority:** P0 payment integrity.

Claim/key/start time commit before external creation. Concurrent CREATING callers may perform multiple same-key HTTP requests, but never create independent identities. Three barrier repetitions inspect the committed row from provider callback and assert identical keys/results. CREATED retry returns stored ID/URL without new create. Fresh coordinator instance reuses persisted CREATING/CREATED state. In-flight valid response arriving after expiry stores its association, leaves EXPIRED and returns409 so a later callback can be refunded. This is service-boundary durability proof, not a live process-crash/provider experiment.

Evidence: `PaymentIntegrationTest`, transport/unit references and representative browser evidence mapped in PAY report.

PAY-004 — Ambiguous creation and replay window

Layer: FIXED-CLOCK API / REAL POSTGRES. **Priority:** P0 payment integrity.

Transport timeout preserves CREATING, key and original start. At59 minutes and exactly60 minutes current local code permits same-key retry;61 minutes returns409 with no create call. One hour is a source constant, not configurable. No new identity after uncertainty. Existing PAYMENT_CREATION_STUCK exception queue/metric supports manual reconciliation. Mollie documentation says within one hour; exact provider-edge replay remains staging verification, not inferred from the local equality test.

Evidence: `PaymentIntegrationTest`, transport/unit references and representative browser evidence mapped in PAY report.

PAY-005 — Webhook integrity and safe failures

Layer: PARAMETERIZED API / UNIT / REAL POSTGRES. **Priority:** P0 payment integrity.

Public POST `/webhooks/mollie/payment` consumes form `id`. Extra paid/amount/currency/bookingId fields are ignored. Blank/missing/malformed ID400 before transport. Server fetch must match incoming ID, stored payment ID, bookingId metadata, exact two-decimal booking amount and EUR. Fractional/under/over/missing amounts, bad association, malformed provider JSON, timeout/5xx and missing status cannot confirm. Processing/integrity failures500 retain retry signal; known duplicate succeeds200. Logs retain safe ID/error class, never raw provider error body. Do not blanket-convert failures200.

Evidence: `PaymentIntegrationTest`, transport/unit references and representative browser evidence mapped in PAY report.

PAY-006 — Provider status and exactly-once confirmation

Layer: PARAMETERIZED API / REAL POSTGRES / REPEATED BARRIER RACE. **Priority:** P0 payment integrity.

open/pending/authorized/failed/canceled/expired do not confirm or mirror provider status into BookingStatus. Valid paid before Eema deadline directly confirms; equality/after take late-refund path. Saved price/commission determine paid financials; paidAt/confirmedAt, PAYMENT_SUCCEEDED/BOOKING_CONFIRMED and after-commit paid notification occur once. Three concurrent duplicate runs prove one economic effect and one payout ledger row. Unknown fee stays unreconciled and payout is not READY. A duplicate paid callback after a previously confirmed booking is cancelled preserves recorded cancellation economics and does not initiate a late refund. Confirmation emails target scout and owner; no real delivery claimed.

Evidence: `PaymentIntegrationTest`, transport/unit references and representative browser evidence mapped in PAY report.

PAY-007 — Late paid callback and full refund

Layer: REAL POSTGRES / FIXED CLOCK / REPEATED IDEMPOTENCY. **Priority:** P0 payment integrity.

Test expiry already persisted and overdue callback before worker. Both remain EXPIRED, record late detection once and initiate full saved-amount refund under durable refund key. No normal receipt/payable owner entitlement or inventory resurrection. Replacement reservation remains valid. Duplicate callbacks preserve completed economics; stale queued response cannot regress REFUNDED. Timeout/provider5xx preserves CREATING and same-key replay within safe window; old ambiguous claim stops for reconciliation. No cancellation percentage is applied to this safety refund. Refund delivery remains mocked.

Evidence: `PaymentIntegrationTest`, transport/unit references and representative browser evidence mapped in PAY report.

PAY-008 — Redirect is navigation only

Layer: BROWSER REPRESENTATIVE / REAL API / REAL POSTGRES. **Priority:** P0 payment integrity.

Real Pay now uses returned stored provider URL. Controlled provider return before webhook remains AWAITING_PAYMENT with manual Check payment status. Authenticated server refetch after authoritative callback shows CONFIRMED; late callback shows EXPIRED/refund. Query paid/success/status/bookingId cannot set state. `/booked` currently resolves only sessionStorage eema:bookingId: missing ID neutral, malformed safe400, unrelated safe403. Expired Pay now409 refetches authoritative EXPIRED, removes stale action and retains useful conflict. No frontend paid setter or real Mollie claim.

Evidence: `PaymentIntegrationTest`, transport/unit references and representative browser evidence mapped in PAY report.

CANCEL scenarios

Contract audit 2026-09-19. See CANCEL execution report. Tests use disposable PostgreSQL, mocked Mollie, injected time and fictional actors. Current policy acceptance is not a new gate on historical cancellation rights; mutation requires authoritative verified-user/ownership checks. No live provider or legal approval is implied. Prior generated scenario text is retained in cancellation historical evidence.

CANCEL-001 — Withdraw unpaid request or approval

Area: CANCEL · **Priority:** P0 · **Result:** LOCAL PASS (2026-09-19); see execution report.

Roles: SCOUT participant. **Environment:** LOCAL REAL POSTGRES; representative browser for CANCEL-003/004.

Steps and acceptance: Withdraw REQUESTED and AWAITING_PAYMENT with open/canceled provider state. Confirm CANCELLED, zero refund/no claim, and a replacement booking can reserve the released slot. A discovered authoritative paid payment first becomes CONFIRMED; missing paid-preview proof returns 409 and requires fresh consent. Provider ambiguity leaves withdrawal uncommitted.

Execution mapping: canonicalUnpaidWithdrawalReleasesReservation; paidPaymentDiscoveredDuringUnpaidWithdrawalRequiresFinancialPreview; unexpectedPaymentAfterWithdrawalIsRefundedWithoutResurrectingBooking.

Evidence: execution report, `docs/qa/evidence/cancellation/`. Testcontainers disposes fixtures; do not reset shared volumes. Actual Mollie refund and email delivery remain STAGING ONLY.

CANCEL-002 — Owner/admin full cancellation and reason limits

Area: CANCEL · **Priority:** P0 · **Result:** LOCAL PASS (2026-09-19); see execution report.

Roles: Owning OWNER and ADMIN; unrelated actors denied. **Environment:** LOCAL REAL POSTGRES; representative browser for CANCEL-003/004.

Steps and acceptance: Preview then cancel CONFIRMED as OWNER and ADMIN. Missing/blank reason returns 409; 1001 characters returns HTTP400; 1000 succeeds. Both return the full stored paid amount with zero owner entitlement; provider-cost responsibility is OWNER or EEMA respectively. Client-supplied refund fields do not control money. Denials preserve booking/events and call no provider.

Execution mapping: canonicalReasonBoundaryAndAuthoritativeFullRefund; canonicalDeniedActorsHaveNoSideEffectsAndZeroRefundIsDurable.

Evidence: execution report, `docs/qa/evidence/cancellation/`. Testcontainers disposes fixtures; do not reset shared volumes. Actual Mollie refund and email delivery remain STAGING ONLY.

CANCEL-003 — Authoritative preview, stale terms and renewed consent

Area: CANCEL · **Priority:** P0 · **Result:** LOCAL PASS (2026-09-19); see execution report.

Roles: SCOUT, owning OWNER, ADMIN. **Environment:** LOCAL REAL POSTGRES; representative browser for CANCEL-003/004.

Steps and acceptance: Preview supplies canCancel, tier, percentage, refund/retained amounts, relevant deadline and signed previewToken for CONFIRMED. Cancellation recomputes under booking lock. Missing/expired/tampered or actor/booking/economics-mismatched proof returns 409 CANCELLATION_TERMS_CHANGED without cancellation. Frontend refetches terms and requires another explicit confirmation; failed preview/session switch clears consent. Mutation 401 is not automatically replayed. Reload reflects authoritative cancellation/refund states. ADMIN remains API-only.

Execution mapping: CancellationPreviewProofTest; staleTermsConflictAndReplayCannotChangeEconomics; cancellation-browser.cjs.

Evidence: execution report, `docs/qa/evidence/cancellation/`. Testcontainers disposes fixtures; do not reset shared volumes. Actual Mollie refund and email delivery remain STAGING ONLY.

CANCEL-004 — Zero refund and idempotent cancellation

Area: CANCEL · **Priority:** P0 · **Result:** LOCAL PASS (2026-09-19); see execution report.

Roles: SCOUT participant. **Environment:** LOCAL REAL POSTGRES; representative browser for CANCEL-003/004.

Steps and acceptance: Inside nonrefundable tier, cancel with current proof: CANCELLED, refund 0.00, refund status NONE and no provider refund or claim. Duplicate cancellation leaves one cancellation event and unchanged economics. Positive ambiguous refunds reuse the original persisted amount/key within the exclusive one-hour replay window; no new economic refund. Refund completion does not rewrite CANCELLED.

Execution mapping: canonicalDeniedActorsHaveNoSideEffectsAndZeroRefundIsDurable; acceptedRefundResponseLossOrPersistenceFailureResumesFromDatabase; RefundCancellationContractTest; CancellationServiceTest.

Evidence: execution report, `docs/qa/evidence/cancellation/`. Testcontainers disposes fixtures; do not reset shared volumes. Actual Mollie refund and email delivery remain STAGING ONLY.

CANCEL-005 — Grace just before 24 hours

Area: CANCEL · **Priority:** P0 · **Result:** LOCAL PASS (2026-09-19); see execution report.

Roles / level: Booking SCOUT; deterministic policy/coordinator unit test with real signed proof. Persistence and HTTP authorization are covered by CANCEL-001–004.

Fixture: Confirmation 1439 elapsed minutes before cancellation; shoot 11520 elapsed minutes after cancellation. Run at `2026-06-01T10:00:00Z`, `2026-03-28T10:00:00Z`, and `2026-10-24T10:00:00Z`; convert shoot to Europe/Amsterdam before creating confirmation snapshots. Paid amount EUR100.01, snapshotted commission 20.00%, reconciled fee EUR1.00.

Steps and acceptance: Preview then cancel at fixed Clock with that signed proof. Expect `GRACE_PERIOD`, 100.00% refund; identical preview/mutation amounts and two-decimal HALF_UP results. Refund/retained/commission/gross settlement respectively are 100.01/0.00/0.00/0.00 for full, 50.01/50.00/10.00/40.00 for partial, 0.00/100.01/20.00/80.01 for zero. Owner payout is gross less known fee, clamped at zero. Paid snapshot remains unchanged; zero calls no refund coordinator. Verify grace/full/partial preview deadline, elapsed 168-hour and 48-hour snapshots, and equivalent instants expressed with another offset.

Execution mapping: `CancellationScenarioBoundaryTest.previewAndCancellationAgreeAtEveryBoundary`, named 005 row, three dates. Existing `BookingCancellationPolicyTest` retained.

Evidence: execution report, `docs/qa/evidence/cancellation/`. No database/provider reset or browser journey per arithmetic permutation. Real settlement is STAGING ONLY.

CANCEL-006 — Grace exactly 24 hours

Area: CANCEL · **Priority:** P0 · **Result:** LOCAL PASS (2026-09-19); see execution report.

Roles / level: Booking SCOUT; deterministic policy/coordinator unit test with real signed proof. Persistence and HTTP authorization are covered by CANCEL-001–004.

Fixture: Confirmation 1440 elapsed minutes before cancellation; shoot 4320 elapsed minutes after cancellation. Run at `2026-06-01T10:00:00Z`, `2026-03-28T10:00:00Z`, and `2026-10-24T10:00:00Z`; convert shoot to Europe/Amsterdam before creating confirmation snapshots. Paid amount EUR100.01, snapshotted commission 20.00%, reconciled fee EUR1.00.

Steps and acceptance: Preview then cancel at fixed Clock with that signed proof. Expect `GRACE_PERIOD`, 100.00% refund; identical preview/mutation amounts and two-decimal HALF_UP results. Refund/retained/commission/gross settlement respectively are 100.01/0.00/0.00/0.00 for full, 50.01/50.00/10.00/40.00 for partial, 0.00/100.01/20.00/80.01 for zero. Owner payout is gross less known fee, clamped at zero. Paid snapshot remains unchanged; zero calls no refund coordinator. Verify grace/full/partial preview deadline, elapsed 168-hour and 48-hour snapshots, and equivalent instants expressed with another offset.

Execution mapping: `CancellationScenarioBoundaryTest.previewAndCancellationAgreeAtEveryBoundary`, named 006 row, three dates. Existing `BookingCancellationPolicyTest` retained.

Evidence: execution report, `docs/qa/evidence/cancellation/`. No database/provider reset or browser journey per arithmetic permutation. Real settlement is STAGING ONLY.

CANCEL-007 — Grace just after 24 hours

Area: CANCEL · **Priority:** P0 · **Result:** LOCAL PASS (2026-09-19); see execution report.

Roles / level: Booking SCOUT; deterministic policy/coordinator unit test with real signed proof. Persistence and HTTP authorization are covered by CANCEL-001–004.

Fixture: Confirmation 1441 elapsed minutes before cancellation; shoot 4320 elapsed minutes after cancellation. Run at `2026-06-01T10:00:00Z`, `2026-03-28T10:00:00Z`, and `2026-10-24T10:00:00Z`; convert shoot to Europe/Amsterdam before creating confirmation snapshots. Paid amount EUR100.01, snapshotted commission 20.00%, reconciled fee EUR1.00.

Steps and acceptance: Preview then cancel at fixed Clock with that signed proof. Expect `PARTIAL_REFUND`, 50.00% refund; identical preview/mutation amounts and two-decimal HALF_UP results. Refund/retained/commission/gross settlement respectively are 100.01/0.00/0.00/0.00 for full, 50.01/50.00/10.00/40.00 for partial, 0.00/100.01/20.00/80.01 for zero. Owner payout is gross less known fee, clamped at zero. Paid snapshot remains unchanged; zero calls no refund coordinator. Verify grace/full/partial preview deadline, elapsed 168-hour and 48-hour snapshots, and equivalent instants expressed with another offset.

Execution mapping: `CancellationScenarioBoundaryTest.previewAndCancellationAgreeAtEveryBoundary`, named 007 row, three dates. Existing `BookingCancellationPolicyTest` retained.

Evidence: execution report, `docs/qa/evidence/cancellation/`. No database/provider reset or browser journey per arithmetic permutation. Real settlement is STAGING ONLY.

CANCEL-008 — More than seven days

Area: CANCEL · **Priority:** P0 · **Result:** LOCAL PASS (2026-09-19); see execution report.

Roles / level: Booking SCOUT; deterministic policy/coordinator unit test with real signed proof. Persistence and HTTP authorization are covered by CANCEL-001–004.

Fixture: Confirmation 2880 elapsed minutes before cancellation; shoot 10081 elapsed minutes after cancellation. Run at `2026-06-01T10:00:00Z`, `2026-03-28T10:00:00Z`, and `2026-10-24T10:00:00Z`; convert shoot to Europe/Amsterdam before creating confirmation snapshots. Paid amount EUR100.01, snapshotted commission 20.00%, reconciled fee EUR1.00.

Steps and acceptance: Preview then cancel at fixed Clock with that signed proof. Expect `FLEXIBLE_FULL_REFUND`, 100.00% refund; identical preview/mutation amounts and two-decimal HALF_UP results. Refund/retained/commission/gross settlement respectively are 100.01/0.00/0.00/0.00 for full, 50.01/50.00/10.00/40.00 for partial, 0.00/100.01/20.00/80.01 for zero. Owner

payout is gross less known fee, clamped at zero. Paid snapshot remains unchanged; zero calls no refund coordinator. Verify grace/full/partial preview deadline, elapsed 168-hour and 48-hour snapshots, and equivalent instants expressed with another offset.

Execution mapping: `CancellationScenarioBoundaryTest.previewAndCancellationAgreeAtEveryBoundary`, named 008 row, three dates. Existing `BookingCancellationPolicyTest` retained.

Evidence: execution report, `docs/qa/evidence/cancellation/`. No database/provider reset or browser journey per arithmetic permutation. Real settlement is STAGING ONLY.

CANCEL-009 — Exactly seven days

Area: CANCEL · **Priority:** P0 · **Result:** LOCAL PASS (2026-09-19); see execution report.

Roles / level: Booking SCOUT; deterministic policy/coordinator unit test with real signed proof. Persistence and HTTP authorization are covered by CANCEL-001–004.

Fixture: Confirmation 2880 elapsed minutes before cancellation; shoot 10080 elapsed minutes after cancellation. Run at `2026-06-01T10:00:00Z`, `2026-03-28T10:00:00Z`, and `2026-10-24T10:00:00Z`; convert shoot to Europe/Amsterdam before creating confirmation snapshots. Paid amount EUR100.01, snapshotted commission 20.00%, reconciled fee EUR1.00.

Steps and acceptance: Preview then cancel at fixed Clock with that signed proof. Expect `PARTIAL_REFUND`, 50.00% refund; identical preview/mutation amounts and two-decimal HALF_UP results. Refund/retained/commission/gross settlement respectively are 100.01/0.00/0.00/0.00 for full, 50.01/50.00/10.00/40.00 for partial, 0.00/100.01/20.00/80.01 for zero. Owner payout is gross less known fee, clamped at zero. Paid snapshot remains unchanged; zero calls no refund coordinator. Verify grace/full/partial preview deadline, elapsed 168-hour and 48-hour snapshots, and equivalent instants expressed with another offset.

Execution mapping: `CancellationScenarioBoundaryTest.previewAndCancellationAgreeAtEveryBoundary`, named 009 row, three dates. Existing `BookingCancellationPolicyTest` retained.

Evidence: execution report, `docs/qa/evidence/cancellation/`. No database/provider reset or browser journey per arithmetic permutation. Real settlement is STAGING ONLY.

CANCEL-010 — Between seven days and 48 hours

Area: CANCEL · **Priority:** P0 · **Result:** LOCAL PASS (2026-09-19); see execution report.

Roles / level: Booking SCOUT; deterministic policy/coordinator unit test with real signed proof. Persistence and HTTP authorization are covered by CANCEL-001–004.

Fixture: Confirmation 2880 elapsed minutes before cancellation; shoot 4320 elapsed minutes after cancellation. Run at `2026-06-01T10:00:00Z`, `2026-03-28T10:00:00Z`, and `2026-10-24T10:00:00Z`; convert shoot to Europe/Amsterdam before creating confirmation snapshots. Paid amount EUR100.01, snapshotted commission 20.00%, reconciled fee EUR1.00.

Steps and acceptance: Preview then cancel at fixed Clock with that signed proof. Expect `PARTIAL_REFUND`, 50.00% refund; identical preview/mutation amounts and two-decimal HALF_UP results. Refund/retained/commission/gross settlement respectively are 100.01/0.00/0.00/0.00 for full, 50.01/50.00/10.00/40.00 for partial, 0.00/100.01/20.00/80.01 for zero. Owner payout is gross less known fee, clamped at zero. Paid snapshot remains unchanged; zero calls no refund coordinator. Verify grace/full/partial preview deadline, elapsed 168-hour and 48-hour snapshots, and equivalent instants expressed with another offset.

Execution mapping: `CancellationScenarioBoundaryTest.previewAndCancellationAgreeAtEveryBoundary`, named 010 row, three dates. Existing `BookingCancellationPolicyTest` retained.

Evidence: execution report, `docs/qa/evidence/cancellation/`. No database/provider reset or browser journey per arithmetic permutation. Real settlement is STAGING ONLY.

CANCEL-011 — Exactly 48 hours

Area: CANCEL · **Priority:** P0 · **Result:** LOCAL PASS (2026-09-19); see execution report.

Roles / level: Booking SCOUT; deterministic policy/coordinator unit test with real signed proof. Persistence and HTTP authorization are covered by CANCEL-001–004.

Fixture: Confirmation 2880 elapsed minutes before cancellation; shoot 2880 elapsed minutes after cancellation. Run at `2026-06-01T10:00:00Z`, `2026-03-28T10:00:00Z`, and `2026-10-24T10:00:00Z`; convert shoot to Europe/Amsterdam before creating confirmation snapshots. Paid amount EUR100.01, snapshotted commission 20.00%, reconciled fee EUR1.00.

Steps and acceptance: Preview then cancel at fixed Clock with that signed proof. Expect `PARTIAL_REFUND`, 50.00% refund; identical preview/mutation amounts and two-decimal HALF_UP results. Refund/retained/commission/gross settlement respectively are 100.01/0.00/0.00/0.00 for full, 50.01/50.00/10.00/40.00 for partial, 0.00/100.01/20.00/80.01 for zero. Owner payout is gross less known fee, clamped at zero. Paid snapshot remains unchanged; zero calls no refund coordinator. Verify grace/full/partial preview deadline, elapsed 168-hour and 48-hour snapshots, and equivalent instants expressed with another offset.

Execution mapping: `CancellationScenarioBoundaryTest.previewAndCancellationAgreeAtEveryBoundary`, named 011 row, three dates. Existing `BookingCancellationPolicyTest` retained.

Evidence: execution report, `docs/qa/evidence/cancellation/`. No database/provider reset or browser journey per arithmetic permutation. Real settlement is STAGING ONLY.

CANCEL-012 — Just inside 48 hours

Area: CANCEL · **Priority:** P0 · **Result:** LOCAL PASS (2026-09-19); see execution report.

Roles / level: Booking SCOUT; deterministic policy/coordinator unit test with real signed proof. Persistence and HTTP authorization are covered by CANCEL-001–004.

Fixture: Confirmation 2880 elapsed minutes before cancellation; shoot 2879 elapsed minutes after cancellation. Run at `2026-06-01T10:00:00Z`, `2026-03-28T10:00:00Z`, and `2026-10-24T10:00:00Z`; convert shoot to Europe/Amsterdam before creating confirmation snapshots. Paid amount EUR100.01, snapshotted commission 20.00%, reconciled fee EUR1.00.

Steps and acceptance: Preview then cancel at fixed Clock with that signed proof. Expect `NON_REFUNDABLE`, 0.00% refund; identical preview/mutation amounts and two-decimal HALF_UP results. Refund/retained/commission/gross settlement respectively are 100.01/0.00/0.00/0.00 for full, 50.01/50.00/10.00/40.00 for partial, 0.00/100.01/20.00/80.01 for zero. Owner payout is gross less known fee, clamped at zero. Paid snapshot remains unchanged; zero calls no refund coordinator. Verify grace/full/partial preview deadline, elapsed 168-hour and 48-hour snapshots, and equivalent instants expressed with another offset.

Execution mapping: `CancellationScenarioBoundaryTest.previewAndCancellationAgreeAtEveryBoundary`, named 012 row, three dates. Existing `BookingCancellationPolicyTest` retained.

Evidence: execution report, `docs/qa/evidence/cancellation/`. No database/provider reset or browser journey per arithmetic permutation. Real settlement is STAGING ONLY.

CANCEL-013 — Grace does not override exactly 48 hours

Area: CANCEL · **Priority:** P0 · **Result:** LOCAL PASS (2026-09-19); see execution report.

Roles / level: Booking SCOUT; deterministic policy/coordinator unit test with real signed proof. Persistence and HTTP authorization are covered by CANCEL-001–004.

Fixture: Confirmation 60 elapsed minutes before cancellation; shoot 2880 elapsed minutes after cancellation. Run at `2026-06-01T10:00:00Z`, `2026-03-28T10:00:00Z`, and `2026-10-24T10:00:00Z`; convert shoot to Europe/Amsterdam before creating confirmation snapshots. Paid amount EUR100.01, snapshotted commission 20.00%, reconciled fee EUR1.00.

Steps and acceptance: Preview then cancel at fixed Clock with that signed proof. Expect `PARTIAL_REFUND`, 50.00% refund; identical preview/mutation amounts and two-decimal HALF_UP results. Refund/retained/commission/gross settlement respectively are 100.01/0.00/0.00/0.00 for full, 50.01/50.00/10.00/40.00 for partial, 0.00/100.01/20.00/80.01 for zero. Owner payout is gross less known fee, clamped at zero. Paid snapshot remains unchanged; zero calls no refund coordinator. Verify grace/full/partial preview deadline, elapsed 168-hour and 48-hour snapshots, and equivalent instants expressed with another offset.

Execution mapping: `CancellationScenarioBoundaryTest.previewAndCancellationAgreeAtEveryBoundary`, named 013 row, three dates. Existing `BookingCancellationPolicyTest` retained.

Evidence: execution report, `docs/qa/evidence/cancellation/`. No database/provider reset or browser journey per arithmetic permutation. Real settlement is STAGING ONLY.

CANCEL-014 — Grace does not override inside 48 hours

Area: CANCEL · **Priority:** P0 · **Result:** LOCAL PASS (2026-09-19); see execution report.

Roles / level: Booking SCOUT; deterministic policy/coordinator unit test with real signed proof. Persistence and HTTP authorization are covered by CANCEL-001–004.

Fixture: Confirmation 60 elapsed minutes before cancellation; shoot 2820 elapsed minutes after cancellation. Run at `2026-06-01T10:00:00Z`, `2026-03-28T10:00:00Z`, and `2026-10-24T10:00:00Z`; convert shoot to Europe/Amsterdam before creating confirmation snapshots. Paid amount EUR100.01, snapshotted commission 20.00%, reconciled fee EUR1.00.

Steps and acceptance: Preview then cancel at fixed Clock with that signed proof. Expect `NON_REFUNDABLE`, 0.00% refund; identical preview/mutation amounts and two-decimal HALF_UP results. Refund/retained/commission/gross settlement respectively are 100.01/0.00/0.00/0.00 for full, 50.01/50.00/10.00/40.00 for partial, 0.00/100.01/20.00/80.01 for zero. Owner payout is gross less known fee, clamped at zero. Paid snapshot remains unchanged; zero calls no refund coordinator. Verify grace/full/partial preview deadline, elapsed 168-hour and 48-hour snapshots, and equivalent instants expressed with another offset.

Execution mapping: `CancellationScenarioBoundaryTest.previewAndCancellationAgreeAtEveryBoundary`, named 014 row, three dates. Existing `BookingCancellationPolicyTest` retained.

Evidence: execution report, `docs/qa/evidence/cancellation/`. No database/provider reset or browser journey per arithmetic permutation. Real settlement is STAGING ONLY.

REF scenarios

REF-001 — Admin full-only refund and one policy-owned partial refund

Area: REF · **Priority:** P0 · **Test level:** real PostgreSQL API/service integration + separate transport/unit contracts · **Result:** PASS (local)

Roles: ADMIN permitted; OWNER/SCOUT 403; anonymous 401.

Fixture and steps: Create a CONFIRMED booking with successful EUR 125.50 payment. POST `/bookings/{id}/refund`; expect 202 BookingDto and exactly EUR 125.50 sent with a durably committed refund key. Change venue price and supply arbitrary amount/percentage/currency JSON: none influences the full-only endpoint, which has no request-body contract. Repeat all seven other BookingStatus values; expect 409 and identical booking, refund, payout, liability and audit snapshots, with no provider call. Missing paidAt or invalid paid amount must conflict before claim creation.

Partial comparison: Use one existing 50% cancellation result: its authoritative EUR 62.75 claim must produce exactly EUR 62.75 at the provider boundary. No custom partial admin API exists. Do not duplicate CANCEL policy boundaries.

UI/notifications: Existing booking/admin details expose persisted facts. Admin full refund has no mutation UI and no dedicated lifecycle email. Requested is not completed.

Evidence: REF execution report, sanitized execution evidence. `RefundIntegrationTest`, `RefundClaimContractTest`, shared `RefundCancellationContractTest` and separate `RefundTransportContractTest`. Local mocks prove Eema safety logic only. Real Mollie refunds, idempotency, statuses, settlement and actual emails remain staging-only.

REF-002 — Validated monotonic refund lifecycle and origin-sensitive completion

Area: REF · **Priority:** P0 · **Test level:** real PostgreSQL API/service integration + separate transport/unit contracts · **Result:** PASS (local)

Exact states: NONE, CREATING, QUEUED, PENDING, PROCESSING, REFUNDED, FAILED, CANCELED. NONE/CREATING are local states; provider `creating` is invalid. CREATING accepts any real provider state; QUEUED → PENDING/PROCESSING/REFUNDED/FAILED/CANCELED; PENDING → PROCESSING/REFUNDED/FAILED/CANCELED; PROCESSING → REFUNDED/FAILED. All three terminal states are absorbing; equality/stale regressions have no effects.

Provider checks: Validate payment ID, attached refund ID, persisted claim association, exact EUR amount and supported status before a transition. Parameterize missing/wrong fields, EUR currency, amount format and unknown status. Invalid facts must not complete or alter economics. The abstraction does not expose echoed claim metadata or list/search; before ID attachment it cannot independently distinguish an unrelated same-payment/same-amount response.

Completion: Admin CONFIRMED → REFUNDED; cancellation stays CANCELLED; late safety refund stays EXPIRED. Set refundedAt once only on accepted completion. Repeated terminal responses produce one audit completion and one payout recalculation; cancellation emits one completion notification, admin emits none. FAILED/CANCELED never claim successful money movement or permit automatic rekeying.

Evidence: REF execution report, sanitized execution evidence. `RefundIntegrationTest`, `RefundClaimContractTest`, shared `RefundCancellationContractTest` and separate `RefundTransportContractTest`. Local mocks prove Eema safety logic only. Real Mollie refunds, idempotency, statuses, settlement and actual emails remain staging-only.

REF-003 — Durable ambiguous creation and exclusive one-hour replay window

Area: REF · **Priority:** P0 · **Test level:** real PostgreSQL API/service integration + separate transport/unit contracts · **Result:** PASS (local)

Fixture and steps: Commit the claim, let the mock provider observe its database key and accept, then lose the response. Reconstruct the service boundary and retry from PostgreSQL. The same payment, amount and key must survive. Test fixed Clock at 3,599 / 3,600 / 3,601 seconds: only strictly inside one hour permits remote retry; at/after the boundary expect 409 and manual reconciliation with no new provider creation.

Errors: Transport failure and HTTP 408/409/429/5xx preserve CREATING. Representative definitive HTTP 422 enters FAILED, retains the identity and returns sanitized support-review text. No raw provider body. No force-success or arbitrary provider-ID mutation exists.

Concurrency/durability: Repeat ambiguous restart and two simultaneous requests three times. Real PostgreSQL and provider barriers must show one claim, one logical mock-provider refund and no concurrency 500. Multiple transports may reuse the same provider idempotency key. Reuse existing shared pre-call rollback/post-call persistence failure tests. V12 enforces amount <= paid amount and immutable claimed key/amount/payment and attached refund ID. Disposable databases only.

Evidence: REF execution report, sanitized execution evidence. `RefundIntegrationTest`, `RefundClaimContractTest`, shared `RefundCancellationContractTest` and separate `RefundTransportContractTest`. Local mocks prove Eema safety logic only. Real Mollie refunds, idempotency, statuses, settlement and actual emails remain staging-only.

REF-004 — Historical PAID payout, recovery liability and mark-paid race

Area: REF · **Priority:** P0 · **Test level:** real PostgreSQL API/service integration + separate transport/unit contracts · **Result:** PASS (local)

Fixture and steps: Pay EUR 125.50, reconcile actual provider fee EUR 1.50, end the interval, prepare verified destination and calculate READY. ADMIN marks EUR 98.90 paid; snapshot the entire payout row. Initiate full refund and accept provider completion. Repeat lookup and payout recalculation twice.

Expected accounting: PAID row remains byte-for-byte unchanged, including amount, paidAt, reference and destination snapshot. Exactly one positive EUR 98.90 OWNER_PAYOUT_RECOVERY liability; no new transfer and no negative payout. Refund completion is independent of later liability settlement.

Required race: With real PostgreSQL booking locks and barriers, repeat each serialization three times. Refund claim first holds the stale READY payout and mark-paid conflicts. Transfer first preserves PAID and completion creates recovery liability. Neither path rewrites transfer history.

Fee follow-up: A fee may reconcile after admin refund; owner entitlement remains zero and payout is CANCELLED if never paid. Unknown cost is not estimated. Keep the existing CANCEL/PAYOUT accounting policy; do not invent a waiver or debt rule.

Evidence: REF execution report, sanitized execution evidence. `RefundIntegrationTest`, `RefundClaimContractTest`, shared `RefundCancellationContractTest` and separate `RefundTransportContractTest`. Local mocks prove Eema safety logic only. Real Mollie refunds, idempotency, statuses, settlement and actual emails remain staging-only.

PAYOUT scenarios

Executed 2026-09-19 against the current contract. **6 LOCAL PASS, 0 FAIL, 0 BLOCKED.** See PAYOUT execution report and sanitized evidence. Historical generated scenarios and reproduced failures are retained under evidence/history. The user explicitly approved the Eema v1 allocation policy below; local tests do not verify real bank/provider operations.

PAYOUT-001 — Payout creation economics

Area: PAYOUT · **Priority:** P0 · **Test level:** UNIT, POSTGRESQL, BROWSER · **Result:** PASS (local)

Authoritative contract: A CONFIRMED or CANCELLED booking can acquire one payout ledger; other states do not create a new ledger. Stored retained gross, commission, reconciled fee and owner entitlement determine the ledger. Zero entitlement, refund or late payment cannot create a normal payable amount. PAID amounts, reference and routing remain historical; reversal is represented by a positive liability. Current venue pricing and client-supplied money are not authoritative.

Executed: Concurrent first calculation produces one payout and one PAYOUT_CALCULATED event. The fixture's stored EUR 125.50 gross and EUR 25.10 commission produce EUR 100.40 provisional entitlement, held for actual fee; EUR 1.23 reconciliation produces EUR 99.17. These are fixture snapshots, not a universal commission rate. Listing-price changes do not rewrite

them. Ineligible booking states, zero/late/cancelled economics and paid reversal reuse the payout unit suite and CANCEL/REF integration evidence. PostgreSQL rejects negative payout values and preserves financial foreign keys.

Evidence: `PayoutIntegrationTest.concurrentRecalculationCreatesOneLedgerAndOneAudit`, `OwnerPayoutServiceTest`, existing CANCEL/REF suites, owner/admin browser reads. No provider transfer is claimed.

PAYOUT-002 — Payout readiness precedence

Area: PAYOUT · **Priority:** P0 · **Test level:** UNIT, POSTGRESQL · **Result:** PASS (local)

Corrected state order: zero → CANCELLED; positive amount with active refund → HELD/REFUND_IN_PROGRESS; missing actual fee → HELD/WAITING_FOR_PROVIDER_COST; before venue-local shoot end → CALCULATED/WAITING_FOR_BOOKING_END; missing destination → HELD/WAITING_FOR_PAYOUT_DESTINATION; otherwise READY. Failed/canceled refunds are terminal rather than active holds.

Executed: Thirteen explicit combinations of blockers; nine end-time cases at −1/0/+1 seconds across ordinary, spring-DST and fall-DST dates. Readiness and worker use the injected Clock; equality is eligible. PostgreSQL proves a busy booking is skipped and picked up on the next scan, and independent workers claim disjoint booking locks. A separate worker test proves the exact end transition.

Defect fixed: PAYOUT-D4, P1: the worker previously locked payouts before waiting on bookings. It now claims booking IDs with FOR UPDATE OF b SKIP LOCKED and only then reads payout state, matching financial mutation lock order. Two original runs reproduced blocking; no artificial sleeps or claimed live multi-instance verification.

PAYOUT-003 — Fee reconciliation idempotency and responsibility

Area: PAYOUT · **Priority:** P0 · **Test level:** UNIT, POSTGRESQL · **Result:** PASS (local)

Executed: Concurrent EUR 1.23 / 1.230 reconciliation produces one authoritative result and one audit; EUR 1.24 correction conflicts without mutation. Null/negative fees and unpaid REQUESTED/CANCELLED bookings are rejected. Owner-caused cancellation creates only a positive PROVIDER_COST liability; EEMA responsibility does not create owner debt. CANCEL/REF evidence covers reconciliation after cancellation/refund and immutable historical financial records.

Defect fixed: PAYOUT-D2, P1: a cancelled but unpaid booking previously passed the service's status-only prerequisite. Prior payment now requires paidAt and a positive paid total, consistent with the existing reconciliation worker's candidate selection.

Resolved Eema v1 policy: NONE means no provider-cost allocation to the owner. EEMA means platform responsibility. OWNER_SETTLEMENT means limited recourse to unpaid settlement only: apply min(actual fee, nonnegative pre-fee entitlement), pay the remainder, and absorb excess at Eema without owner debt. OWNER means explicit responsibility: apply the same capped deduction and create only excess as an OPEN PROVIDER_COST liability. EUR 16 entitlement / EUR 25 fee gives zero payout and EUR 9 platform cost for OWNER_SETTLEMENT, versus EUR 9 owner liability for OWNER. Normal bookings retain their existing NULL responsibility with capped settlement allocation. Fees never reduce scout refunds.

Policy evidence: The production allocation helper is tested across all four responsibilities and below/equal/above/zero-entitlement cases. Real PostgreSQL repeats these 16 cases with concurrent reconciliation; actual scout/owner/admin cancellation converges for fee-before, fee-after and concurrent ordering. Repeats create one reconciliation event and at most one provider-cost liability. PAID history remains immutable; a late fee alone cannot create limited-recourse debt through OWNER_PAYOUT_RECOVERY. No redundant allocation columns or general ledger were added.

Paid-history correction: An already-paid OWNER settlement must not be charged again for cost already deducted at transfer. Derive the historical deduction from immutable gross, commission and net paid, capped at historical provider cost; retain it only while entitlement survives. EUR 16 entitlement / fee 5 / PAID 11 creates no new debt on recalculation. A true reversal to zero retains EUR 11 net recovery and EUR 5 explicit owner cost. NONE/EEMA historical platform costs cannot become limited-recourse recovery debt. Three unit and three PostgreSQL regressions cover these histories.

Operational boundary: Automated debt collection and liability netting are not implemented. Review this policy if Eema adopts Mollie Connect, delayed routing, automated payouts or a different merchant-of-record structure. Real Mollie fee/report matching remains staging-only.

PAYOUT-004 — Destination snapshot on READY

Area: PAYOUT · **Priority:** P0 · **Test level:** POSTGRESQL, BROWSER · **Result:** PASS (local)

Executed: Payout first becomes READY using verified A. Pending B does not redirect it. Real verification replaces A with B in the same transaction; repeated calculation keeps the old payout on A with one SNAPSHOTTED trust event. A later eligible payout uses B. Recording the original payout PAID and recalculating preserves its entire historical row. Owner reads reveal only the masked destination and no private transfer reference.

Defect fixed: PAYOUT-D3, P1: Hibernate could verify B before flushing A's replacement, violating the one-verified-destination index. Flush A's replacement before marking B VERIFIED; retain atomic rollback and the unique constraint. This proves the payout snapshot dependency; the subsequently completed DEST validation is recorded below.

PAYOUT-005 — Mark paid records external transfer

Area: PAYOUT · **Priority:** P0 · **Test level:** POSTGRESQL, BROWSER · **Result:** PASS (local)

Authoritative contract: ADMIN records a completed external/manual transfer; the endpoint does not send money. READY with a historically VERIFIED or REPLACED destination can become PAID. Trimmed identical references are idempotent; different references conflict. Blank and over-200-character references are rejected. HELD/CALCULATED/CANCELLED, missing/unverified routing and unauthorized actors cannot record payment.

Executed: Concurrent same/different references, one paid event, rollback with audit, client amount ignored, role/object scope, private reference exclusion and immutable paid history. Existing CANCEL/REF tests prove payment recording versus cancellation/refund serialization. Browser proves list/detail amounts, explicit disclaimer, authoritative refetch, reload consistency, held-state controls and two real stale-form 409 conflicts.

Defects fixed: PAYOUT-D1, P1: absent UUID filters caused admin financial list HTTP 500; build only supplied filter predicates. PAYOUT-D5, P2: conflict feedback disappeared when refetch hid the action form; render it outside the state-dependent form. Neither fix changes financial authority.

PAYOUT-006 — Liability settlement and WAIVED

Area: PAYOUT · **Priority:** P0 · **Test level:** POSTGRESQL, BROWSER · **Result:** PASS (local)

Executed: Both PROVIDER_COST and OWNER_PAYOUT_RECOVERY use stored positive amounts. ADMIN concurrent settlement changes OPEN to SETTLED with one event; exact retry is harmless. Owner/scout attempts are denied, arbitrary amount input is ignored, and booking/payout rows remain unchanged. Seeded WAIVED conflicts. Database rejects zero liabilities and duplicate booking/type rows. There is no public waive or amount-edit endpoint.

UI evidence: Real local settlement confirms that this records an external action without moving money, then refetches and survives reload. A clearly labeled controlled 409/WAIVED response tests preservation of conflict feedback; it is not evidence of an implemented waive workflow. PAYOUT-D1 and PAYOUT-D5 also affect this scenario and are fixed.

Continuation: The user-approved allocation policy and paid-history correction passed full regression. DEST-001–004 was subsequently completed; see its execution section and the current DOC continuation gate below.

DEST scenarios

Executed 2026-09-19 against the current source contract: **4 LOCAL PASS, 0 FAIL, 0 BLOCKED**. See DEST execution report and sanitized evidence. Historical generated assumptions and reproduced failures are preserved. Local verification does not prove real bank ownership, email delivery or deployed key custody.

DEST-001 — Destination submit and validation

Area: DEST · **Priority:** P0 · **Test level:** UNIT, POSTGRESQL, BROWSER · **Result:** PASS (local)

Corrected contract: OWNER submits for their own current identity; submission requires current policies, verified email and VERIFIED owner review. Safe owner history reads do not repeat those mutation prerequisites. Admin review follows admin authorization rather than owner prerequisites. Client owner ID, status, currency and type cannot choose persisted values: the service creates PENDING_VERIFICATION / EUR / IBAN.

Executed: Nonblank/max160 holder name; raw IBAN max80 before normalization; valid lowercase/spaces accepted, bad checksum, unsupported current country and wrong length rejected. Raw 80-character whitespace-padded valid input passes; 81 fails. The current country allowlist is authoritative, not a claim to support every SEPA country. All three submission prerequisites are independently denied without a destination or provider call. Owner/scout/admin/anonymous authorization and owner-scoped history are exercised. Same-owner normalized duplicates conflict across historical states; another owner may independently submit the same synthetic fixture.

Defects fixed: DEST-D1 (P1), concurrent duplicates returned 500 after racing the fingerprint check. Lock the stable owner row before submission so the loser gets 409 without duplicate state/audit. DEST-D3 (P0), account switching retained another owner's cached history. Key the owner component to current user identity; the browser proves old data disappears and the new owner's authoritative history loads.

DEST-002 — Encrypted persistence and masked access

Area: DEST · **Priority:** P0 · **Test level:** UNIT, POSTGRESQL, BROWSER · **Result:** PASS (local)

Executed: Real submission persists versioned AES-256-GCM ciphertext, keyed fingerprint and last4. Normal owner/admin list/detail responses exclude full bank value, ciphertext and fingerprint. Existing unit tests prove random ciphertext and stable fingerprint; tampered/malformed envelopes fail closed and missing/short configured keys fail initialization. PostgreSQL rejects invalid type/currency/last4/state, orphan owner references and duplicate historical owner/fingerprint rows. Neither browser storage nor sanitized evidence retains revealed bank values.

Boundary: Fictional bank fixtures and test-only keys are used locally. Production provisioning, custody/rotation and actual account-ownership review are operational/staging concerns. No electronic bank-verification claim or new storage scheme is introduced.

DEST-003 — Review replacement and rejection

Area: DEST · **Priority:** P0 · **Test level:** POSTGRESQL, BROWSER · **Result:** PASS (local)

Authoritative transitions: Submit → PENDING_VERIFICATION. Verify pending → VERIFIED and atomically replace the former VERIFIED destination. Reject pending → REJECTED with a required owner-visible reason. Pending/rejected replacements leave current verified routing active. Historical records cannot be resubmitted/restored through these endpoints; stale review conflicts. Review notes/reasons enforce their 1000-character limits.

Executed: Real sequential replacement/rejection, exact admin filters, three simultaneous different-destination reviews, verify-versus-reject on one record, rollback of both destination changes when audit fails, and one verified destination per owner. PAYOUT-004's prior regression proves historical READY/PAID routing stays on its original destination after replacement; the full payout suite is reused rather than duplicated. Browser review refetches authoritative state and rejection preserves the original verified destination.

Defect fixed: DEST-D2 (P1), simultaneous verification of different pending records raced the unique index and returned 500. Review now resolves owner identity, locks owner, then locks destination. Preserve the existing atomic replacement flush and database uniqueness rules. No schema change or new review power was added.

DEST-004 — Audited reveal and ephemeral frontend value

Area: DEST · **Priority:** P0 · **Test level:** POSTGRESQL, BROWSER · **Result:** PASS (local)

Executed: ADMIN-only POST reveal for PENDING_VERIFICATION / VERIFIED / REPLACED; REJECTED conflicts and GET is not supported. Successful response has Cache-Control no-store and Pragma no-cache; the REVEALED audit has the actor and no bank value. Audit failure prevents a successful reveal. Reads/reveal do not emit lifecycle notifications. Role denial preserves state and audit.

Browser evidence: Explicit confirmation, reveal/hide, no persistent storage of the value, clearing after review and reload/navigation, rejected-state controls, account switching, and delayed responses released after account switch/logout. Existing API actor checks reject old-session responses. The original harness attempted an intentionally disabled Cancel button during loading; that was corrected, not treated as a missing product feature.

Defect fixed: DEST-D4 (P1), an already-revealed value survived switching admin accounts without a new actor's deliberate reveal/audit. Key admin detail state to both current user and destination ID so disclosure, forms and cached results cannot cross those contexts.

Continuation: DEST is locally complete. Its prior PRICE-DEF-005 continuation blocker was resolved by the approved no-free-booking policy and full regression on 2026-09-19. DOC-001–004 was subsequently executed; see its results and current continuation gate below.

DOC scenarios

Executed against the current source contract: **4 LOCAL PASS, 0 FAIL, 0 BLOCKED**. Four prior P1 defects remain fixed, and DOC-DEC-001 is resolved by the approved embedded Unicode-name policy. Prior substitution evidence is retained in the execution report. See DOC execution report and sanitized evidence. No real provider or legal/tax approval is claimed.

DOC-001 — Receipt eligibility and ownership

Area: DOC · **Priority:** P0 · **Test level:** POSTGRESQL, BROWSER · **Result:** PASS (local)

Corrected contract: Authenticated booking SCOUT or ADMIN may download. OWNER and unrelated scouts are denied. Historical reads do not require newly verified email, current policy acceptance or owner trust. First issuance requires CONFIRMED/CANCELLED/REFUNDED plus paidAt, confirmedAt, positive totalPaid and no latePaymentDetectedAt. Previously issued bytes remain downloadable by authorized actors without recalculating current economics.

Executed: All three eligible and all five ineligible booking states; missing paid/confirmed timestamps, zero paid total and late-paid history; role/object/anonymous denial both before and after issuance; unverified/stale-policy historical reads. No denied issuance creates a document/object/audit or provider effect. Successful first issue creates one audit; repeated reads return identical bytes. Three barrier-coordinated concurrent request pairs each produce one persisted document, object and audit. HTTP response is an attachment, application/pdf, exact content length and no-store. The approved PRICE-DEF-005 minimum remains enforced upstream; no free-receipt lifecycle was introduced.

DOC-002 — Receipt content and immutability

Area: DOC · **Priority:** P0 · **Test level:** UNIT, POSTGRESQL, PDF, BROWSER · **Result:** PASS (local)

Authoritative content: RCT number, booking ID, paid date/status at issuance, venue metadata at first issue, saved scout contact and monetary/duration/rate/minimum/cap snapshots. Internal commission, provider cost, owner payout, liability, provider IDs and full bank values are excluded. It is a payment record, not a claim of tax-invoice compliance.

Executed: Extract actual generated PDF text and compare exact saved values. Change current venue name/rate/neighbourhood and cancel booking; subsequent file bytes stay identical. Browser receipt download and reload have identical SHA-256 and safe RCT filenames. No fresh lifecycle email or provider payment is issued by download. Initial repeated request identity is protected by PostgreSQL locks/uniqueness, not by a client cache.

DOC-003 — Payout statement content and scope

Area: DOC · **Priority:** P0 · **Test level:** UNIT, POSTGRESQL, PDF, BROWSER · **Result:** PASS (local)

Corrected contract: Same payout OWNER or ADMIN only. First issuance requires PAID, paidAt and nonblank external transfer reference. CALCULATED/HELD/READY/CANCELLED and legacy missing-reference rows conflict. New statement reads the immutable paid ledger and historical destination last4, not the owner's latest bank details. Owner/venue labels are captured at first issue and stored bytes are authoritative thereafter.

Executed: Exact PST/booking/payout references, complete transfer reference, masked historical REPLACED destination, gross/commission/provider cost/net payout and disclaimer. Unrelated owner/scout/anonymous denial; no internal liability or encrypted/full bank value. Owner/venue edits do not rewrite issued bytes. Three simultaneous first-download pairs each issue once. Browser statement download/reload preserves bytes and role controls. PAYOUT/DEST replacement suites supply the existing immutable-routing regression.

DOC-004 — Private storage failure, Unicode and layout

Area: DOC · **Priority:** P0 · **Test level:** UNIT, POSTGRESQL, PDF, BROWSER · **Result:** PASS (local; DOC-DEC-001 resolved)

Technical results: Storage failure, actual DB constraint failure after store, and transaction-commit failure return safe errors without issued metadata/audit; rollback attempts deletion of the new private object. Missing already-issued bytes fail without silent regeneration. Local opaque storage is inaccessible through public media routes. S3 request mocks prove private bucket/key, conditional writes and rejection of invalid references; real S3 remains staging-only.

Defects fixed: DOC-D1, local ATOMIC_MOVE could overwrite an existing object; completed bytes are now published atomically using a non-replacing hard link. DOC-D2, fixed-character wrapping clipped wide text and the 180-character limit truncated persisted references; layout now measures glyph widths, preserves full values and paginates before the footer. DOC-D3,

unsupported controls previously crashed rendering; the approved Unicode policy now rejects unsupported input safely before issuance, superseding the historical character fallback. DOC-D4, late transaction rollback did not clean the file; cleanup now follows transaction completion. All four are P1 and have passing regressions. The subsequent Unicode resolution uses template version 3 for new documents; existing issued files are unchanged.

Verified visuals: Wide names, complete 200-character transfer reference, maximal stored amounts, control/Unicode input, receipt/statement controls and missing-file error. No clipping/overlap remains in reviewed outputs. Browser delayed response after logout cannot trigger a download. Local publication requires filesystem hard-link support; deployment must verify that capability, private permissions and backups.

Approved v1 resolution: Financial documents preserve supported original Unicode names through controlled embedded font rendering; unsupported glyphs fail safely rather than being silently replaced. Bundled/subset PDType0 fonts: Noto Sans Regular/Bold, static Noto Sans SC derivative and Noto Naskh Arabic Regular, OFL1.1 notices retained. Font resources and ICU shaping are application controlled, with no host-font or remote lookup. New template3 applies only at first issue; historical PDFs remain byte-identical.

Executed Unicode evidence: José García, Zoë François, Daniël Müller, Łukasz Żółć, Νίκος, Алексей, 李雷, محمد and 李雷 García extract unchanged. Arabic connected appearance, CJK/mixed, Greek/Cyrillic, European diacritics and long wrapped fields were rendered and reviewed. Literal markup remains text. Receipt/statement cases retain exact saved money, masked destination and role isolation; unsupported names fail before any document/file/audit is created. Retry, rollback cleanup and repeated concurrent issuance regressions pass. Packaged JAR and production Docker runtime probes verify controlled resource loading and embedded fonts without network access.

Precise support boundary: PDFBox3.0.8 lacks full GPOS. ICU supplies standard Arabic presentation shaping and bidi; no universal Unicode/complex-script claim is made. Combining marks, variation selectors, format/join controls and unconfigured glyphs fail safely, without normalization or substitution. No new locale/template/tax semantics. Previously observed `??` output is archived as history, not accepted current behavior.

Historical continuation at DOC completion: INC-001–002 was next. Its subsequent execution and the current ADMIN decision are recorded below.

INC scenarios - audited 2026-09-20

Execution: `docs/qa/INC_EXECUTION_REPORT.md`; sanitized evidence: `docs/qa/evidence/inc/`. Two scenarios pass focused PostgreSQL and browser execution. Full regression and completion gate are recorded in the report. No live provider result is claimed.

INC-001 — Incident participants and plain text

Area: INC · **Priority:** P1 · **Test level:** API, LOCAL E2E · **Result:** PASS

Authoritative contract: Active authenticated booking scout, owning OWNER or ADMIN may create/list reports. Unrelated actors receive 403; anonymous/inactive receive 401; missing booking 404. No email verification, policy acceptance, owner review, booking status or date prerequisite applies. Admin intake is API-supported; no new admin report UI is required.

Executed: All seven categories for all three permitted actors; all eight booking states; unrelated/anonymous/inactive denial; absent fields/unknown category; blank, angle-bracket markup, 4000/4001, newline and Unicode descriptions. API validates raw Java-character length, then trims outer whitespace. Valid report persists OPEN with one INCIDENT_REPORTED trust event. Participant reads omit internal resolution-note values. Database foreign keys and category/status/description/resolution-shape constraints reject inconsistent rows.

UI/client: Real unverified scout without policy acceptances receives backend 400 for markup and 201 for valid Unicode text. Authoritative list survives reload and is visible to the owning owner. INC-DEF-001 fixed a frontend-only verification gate that incorrectly blocked support intake; the exception is limited to incident-report POST. Existing booking/venue verification guards remain. Seven focused client and thirteen AUTH client checks pass.

Isolation: No booking, refund, payout, liability, venue, booking policy snapshot or booking lifecycle notification change; no Mollie call. Separate accepted reports remain separate records, without an invented deduplication contract.

Evidence: IncidentIntegrationTest, IncidentBrowserIntegrationTest, inc-client-check.cjs, inc-browser.cjs; historical failed client/browser evidence retained. Disposable PostgreSQL and synthetic fixtures only. No incident email exists to verify.

INC-002 — Admin incident lifecycle and isolation

Area: INC · **Priority:** P1 · **Test level:** API, LOCAL E2E · **Result:** PASS

Authoritative contract: ADMIN only. OPEN → UNDER_REVIEW → RESOLVED → CLOSED; all skipped/repeated/reverse actions conflict with 409. No edit/delete/reopen operation. Resolution requires a nonblank plain-text note of at most 2000 Java characters. Admin endpoints expose the note; all participant-list DTOs hide it. No email-verification or policy prerequisite for administration.

Executed: Full state/action matrix; all non-admin and anonymous endpoint denials; note blank/markup/2000/2001/newline/Unicode/missing fields; missing incidents; status/category/booking filters and bounded pagination. Successful mutations append exactly one trust event. Injected trust-audit failure rolls back both creation and transition. Nine barrier-coordinated request pairs in independent transactions each produce one 200, one 409 and one audit event. Final persisted state is CLOSED.

Isolation/privacy: Populated payout/liability rows, booking/refund and venue snapshots, booking audit and policy references remain unchanged. Internal notes are not copied into trust-event notes or participant responses. No provider call or booking lifecycle notification.

UI: Eight grouped browser checks include stale start-review 409, explicit conflict feedback and authoritative refetch, resolution/closure/reload, participant note redaction and unrelated-scout denial. Screenshots visually reviewed. No optimistic financial or booking outcome is introduced.

Environment and review: Local PostgreSQL with fresh migrations and Hibernate validation. Elham should assess support-versus-financial-action wording, internal-note terminology and mobile usability. Deployed edge/authentication and monitoring remain staging work. No new product/business/legal decision was needed for this existing contract.

ADMIN scenarios - audited 2026-09-20

Execution report: `docs/qa/ADMIN_EXECUTION_REPORT.md`; evidence: `docs/qa/evidence/admin/`. Approved complete-backlog policy resolves ADMIN-DEC-001. Historical result was10 PASS,0 FAIL,1 BLOCKED because only200 of201 refunds were visible; history is preserved in the report. Current local scenario results:11 PASS,0 FAIL,0 BLOCKED. Database and13 browser checks pass, including202 exceptions across11 UI pages; full command gates and completion status are recorded in the resolution addendum.

Common environment: disposable PostgreSQL with V1–V13 and Hibernate validation, mocked providers, fixed Clock for thresholds, actual production frontend build for representative browser checks. Existing AUTH/POL/TRUST/VEN/PAY/REF/PAYOUT/DEST/INC reports remain dependencies. No legal content, provider result or operational SLA is approved by these software tests.

ADMIN-001 — Every admin queue pagination and filters

Area: ADMIN · **Priority:** P1 · **Test level:** API, LOCAL E2E · **Result:** PASS (approved complete-backlog policy, 2026-09-20)

Authoritative contract: Active ADMIN only; no admin email-verification/policy prerequisite. Seven principal paged queues plus booking events and operational exceptions use bounded page sizes up to100. Policies and venue moderation events are non-paged lists. Arbitrary client sort is ignored; repository-defined ordering wins.

Executed evidence: All paged queues have over20 fixtures; sizes1/20/100/101, negative/empty/extreme pages, supported filters independently and combined, malformed enums/UUIDs and SCOUT/OWNER/anonymous denial. Browser proves pagination/reload, Clear, and All overriding default status. ADMIN-D2 fixes offset overflow; ADMIN-D4 fixes stale controls and dropped empty default-status parameters.

Boundary / decision: Approved Eema v1: The ADMIN exception queue is a complete pageable operational backlog. Individual requests are bounded, but the total matching backlog is not silently capped. PostgreSQL UNION ALL/count/LIMIT/OFFSET returns only the requested page, with one-statement snapshot consistency. Global order is severity (CRITICAL/WARNING/INFO), detectedAt ascending nulls last, type, source-row UUID. Existing API has no type/severity filters. Stable fixtures of0/1/20/21/200/201/321 refunds and210 mixed exceptions prove complete unique traversal, counts and final/empty pages. Worker/metric batches remain separate and unchanged.

ADMIN-002 — Booking search and detail composition

Area: ADMIN · **Priority:** P1 · **Test level:** API, LOCAL E2E · **Result:** PASS

Authoritative contract: Booking detail composes persisted booking commission, provider fee, refund/cancellation, payout and liability values. The UI formats stored values without inventing financial calculations. Mollie payment ID input is trimmed.

Executed evidence: All BookingStatus, RefundStatus and PaymentCreationStatus values, optional/composite filters, exact stored amounts and linked payout/liability IDs, missing booking404 and unchanged persisted booking. Existing AdminBookingSearchIntegrationTest is reused.

Boundary / decision: No new pricing, refund, payout or admin-remediation capability.

ADMIN-003 — Booking event timeline payload allowlist

Area: ADMIN · **Priority:** P1 · **Test level:** API, LOCAL E2E · **Result:** PASS

Authoritative contract: Chronological paged booking events expose only allowlisted scalar JSON values. Unknown top-level keys and nested objects/arrays are excluded; immutable stored events are untouched. Missing booking404; nonadmin denied.

Executed evidence: 25 events across two pages, synthetic top-level and nested sensitive fields, numeric/currency preservation; independent unit/PostgreSQL/browser reproductions and regressions. ADMIN-D1 fixes shallow allowlist leakage. ADMIN-D5 fixes nonfunctional timeline Retry and browser verifies successful recovery.

Boundary / decision: Scalar-only output is a privacy boundary, not a destructive rewrite of historical payloads.

ADMIN-004 — Review mutation stale-state recovery

Area: ADMIN · **Priority:** P1 · **Test level:** API, LOCAL E2E · **Result:** PASS

Authoritative contract: Review mutations retain each domain's existing state machine and refetch authoritative state. Shared admin reads ignore superseded request generations. Sensitive payout destination reveal data is cleared by its existing action/conflict handling.

Executed evidence: TRUST/VEN/POL/DEST reports provide their review/conflict evidence; INC real browser again proves stale409/refetch and completed-state controls. The actual hook race reproduced stale overwrite and passes after ADMIN-D3. Filtered-navigation browser was already safe before that fix; no false browser reproduction claim is made.

Boundary / decision: No new review powers, automatic financial actions or policy-content approval.

ADMIN-005 — Exception PAYOUT_READY_OVERDUE

Area: ADMIN · **Priority:** P1 · **Test level:** API, LOCAL E2E · **Result:** PASS

Authoritative contract: READY payout is overdue only when readyAt is strictly before now minus configured72h default; severity WARNING, linked booking/payout IDs and READY_PAYOUT_NOT_RECORDED_PAID.

Executed evidence: Fixed Clock at one second before, exactly at and one second after cutoff; equality is excluded. Real PostgreSQL filtering and queue severity/age order; no mutation/provider call.

Boundary / decision: Complete backlog pagination follows the approved ADMIN-001 contract.

ADMIN-006 — Exception PAYOUT_HELD

Area: ADMIN · **Priority:** P1 · **Test level:** API, LOCAL E2E · **Result:** PASS

Authoritative contract: Every HELD payout is INFO, without age cutoff. Reason is PAYOUT_ plus WAITING_FOR_BOOKING_END, WAITING_FOR_PROVIDER_COST, REFUND_IN_PROGRESS or WAITING_FOR_PAYOUT_DESTINATION.

Executed evidence: All four hold reasons, authoritative payout/booking links, severity and read-only behavior. No arbitrary age test is invented for this condition.

Boundary / decision: Complete backlog pagination follows ADMIN-001.

ADMIN-007 — Exception OWNER_LIABILITY_OPEN

Area: ADMIN · **Priority:** P1 · **Test level:** API, LOCAL E2E · **Result:** PASS

Authoritative contract: OPEN owner liability is WARNING without age cutoff; links identify the booking and liability. Reason OWNER_LIABILITY_REQUIRES_RESOLUTION.

Executed evidence: Real persisted OPEN liability and linked ID; read leaves it OPEN and booking unchanged; severity/oldest-first ordering checked. Existing PAYOUT liability mutation tests remain separate.

Boundary / decision: Complete backlog pagination follows ADMIN-001. This read does not settle or waive debt.

ADMIN-008 — Exception REFUND_STUCK

Area: ADMIN · **Priority:** P0 · **Test level:** API, LOCAL E2E · **Result:** PASS

Authoritative contract: CREATING/QUEUED/PENDING/PROCESSING refunds are active; REFUND_STUCK is CRITICAL only when refundRequestedAt is strictly older than configured1440min default.

Executed evidence: Fixed-clock cutoff minus/equal/plus1second against real PostgreSQL; active status set shared with the fully enumerated late-refund predicate. Reason REFUND_EXCEEDED_STUCK_THRESHOLD and booking link; no provider retry or remediation.

Boundary / decision: Historical200-of201 evidence is retained. Approved regression now reaches all201 refunds uniquely with correct totals and a one-item final page at size20.

ADMIN-009 — Exception PAYMENT_CREATION_STUCK

Area: ADMIN · **Priority:** P1 · **Test level:** API, LOCAL E2E · **Result:** PASS

Authoritative contract: CREATING payment claim is WARNING only when paymentCreationStartedAt is strictly older than configured60min default. Reason PAYMENT_CREATION_EXCEEDED_STUCK_THRESHOLD.

Executed evidence: Fixed-clock cutoff equality and both sides; real exception browser follows the correct booking link. Read-only state/provider isolation; no transport replay added.

Boundary / decision: Complete backlog pagination follows ADMIN-001.

ADMIN-010 — Exception MOLLIE_FEE_UNRECONCILED

Area: ADMIN · **Priority:** P1 · **Test level:** API, LOCAL E2E · **Result:** PASS

Authoritative contract: Fee exception requires positive totalPaidEur, unreconciled fee and paidAt strictly older than configured1440min default in CONFIRMED, CANCELLED or REFUNDED. REFUNDED support supersedes the generated two-status assumption.

Executed evidence: Fixed-clock fee cutoff boundaries include REFUNDED; prior PAYOUT/provider-cost allocation evidence supplies historical fee-state regression. WARNING with AUTHORITATIVE_PROVIDER_FEE_UNAVAILABLE.

Boundary / decision: No fee estimate/provider lookup is introduced; actual provider reconciliation is staging work. Complete backlog pagination follows ADMIN-001.

ADMIN-011 — Exception LATE_PAYMENT_REFUND_IN_PROGRESS

Area: ADMIN · **Priority:** P0 · **Test level:** API, LOCAL E2E · **Result:** PASS

Authoritative contract: latePaymentDetectedAt plus active CREATING/QUEUED/PENDING/PROCESSING refund creates WARNING without age cutoff; terminal refund states are excluded.

Executed evidence: Every RefundStatus with late detection, linked booking and severity/age ordering. Existing PAY/REF financial state-machine evidence reused; read-only queue makes no provider call.

Boundary / decision: No new late-payment or refund semantics. Complete backlog pagination follows ADMIN-001.

WHAT ELHAM SHOULD TEST PERSONALLY: exception urgency/reason wording, linked-record discoverability, historical financial terminology and review/conflict clarity. **PRODUCT / OPERATIONS DECISIONS:** ADMIN-DEC-001 resolved by explicit complete-backlog policy; no new decision. **STAGING ONLY:** deployed monitoring, multi-instance operation and real provider reconciliation. Next domain after successful ADMIN command gates: SEC-001–007.

SEC scenarios - audited 2026-09-20

Execution report: `docs/qa/SEC_EXECUTION_REPORT.md`; evidence: `docs/qa/evidence/sec/`. Focused local execution passes19 PostgreSQL cases,8 security/correlation/email units,26 session-client cases and6 SEC browser checks; document browser regression passes8 checks. Full required command gates and final continuation status are recorded in the report. Historical pre-fix failures are retained.

SEC-001 — Object ownership tampering

Area: SEC · **Priority:** P0 · **Result:** PASS (local; full command gates in report)

Authoritative contract: Active user and role come from the database for each bearer request. Ownership and ADMIN exceptions remain endpoint-specific; verified email and policy acceptance are not universal read prerequisites.

Executed evidence: Prior AUTH/VEN/AVL/BK/INQ/PAYOUT/DEST/DOC/INC/ADMIN actor and UUID-tampering matrices are reused in full PostgreSQL regression. New signed-stale-role, admin demotion, inactive-user and forged verification-header checks prove current DB authority and no booking persistence on denial.

SEC-002 — Input injection and safe error handling

Area: SEC · **Priority:** P0 · **Result:** PASS (local; full command gates in report)

Authoritative contract: Inputs follow their actual DTO/domain contracts. Names, policy text, SQL-like quotes, Unicode/emoji, CR/LF and backslashes remain data where allowed. Incident angle brackets remain prohibited. React/email escape HTML.

Executed evidence: New six-value registration/policy/filter corpus, seeded nonempty SQL-filter queries, 10k name rejection and 10k policy-content acceptance, safe malformed-JSON response. Reuse prior brief/contact/reason/incident/filename bounds and TrustNotificationService HTML escaping. Browser proves stored script markup stays literal. Strict firewall CR/LF rejection now returns safe400 (SEC-DEF-003), without relaxing rejection.

SEC-003 — Token and credential leak audit

Area: SEC · **Priority:** P0 · **Result:** PASS (local; full command gates in report)

Authoritative contract: Auth responses and localStorage auth tokens remain intentional existing design. Normal DTOs and application logs must not contain passwords, raw auth/recovery tokens or provider credentials. Debug logging is included in the audit.

Executed evidence: SEC-DEF-001 independently reproduced synthetic secret leakage through malformed JWT-header messages and unexpected exception causes in units and real HTTP. Fixed logs retain safe error type/request ID only. Registration/recovery token scan passes against normal DTOs and captured logs. Reuse DEST encryption/reveal/no-store and MATCH/PAY/REF safe provider-error evidence. No production credential leak or real provider test is claimed.

SEC-004 — CORS, security headers and request ID

Area: SEC · **Priority:** P0 · **Result:** PASS (local; full command gates in report)

Authoritative contract: Configured origins only; allowed methods GET/POST/PUT/DELETE/OPTIONS/PATCH; allowed Authorization/Content-Type/X-Requested-With/X-Request-ID; expose X-Request-ID and Content-Disposition. nosniff, DENY, no-referrer and API CSP default-src none/frame-ancestors none. HSTS is opt-in and secure-request-only.

Executed evidence: Full filter-chain preflight tests reject unlisted/lookalike/null/wildcard origins, unsupported method/header. Real API responses expose configured headers. Safe correlation IDs survive; unsafe/Unicode/oversized IDs are replaced and MDC clears. CR/LF requests remain firewall-rejected with generic400. Local secure mock requests prove enabled HSTS; actual deployed TLS/edge behavior remains STAGING ONLY.

SEC-005 — Private document URL and cache isolation

Area: SEC · **Priority:** P0 · **Result:** PASS (local; full command gates in report)

Authoritative contract: Receipt/statement downloads remain individually authorized and private. Spring Security already supplies no-store; absence of explicit controller cache code was a stale generated concern. No public URL/storage reference is exposed.

Executed evidence: Reuse DOC47 PostgreSQL cases: actor matrix, guessed public-upload paths, immutable issuance and cache/header assertions. Document browser rerun passes8 checks including delayed download after logout, private receipt/statement controls and missing-file handling. No cache change or new storage architecture was needed.

SEC-006 — Framework and actuator exposure

Area: SEC · **Priority:** P0 · **Result:** PASS (local; full command gates in report)

Authoritative contract: Public health includes status and optional probe-group names, without component details. Metrics require ADMIN. Installed Swagger/OpenAPI routes require authentication; they are not assumed absent. Non-exposed env/configprops/heapdump actuator routes remain unavailable.

Executed evidence: Every role and anonymous health/metrics matrix, individual metric and no identifying gauge tags, authenticated Swagger200 versus anonymous401, disabled actuator404 for authenticated callers and unsupported admin method405/SCOUT403. Generated expectations of health status-only and authenticated Swagger404 were corrected from actual source/execution.

SEC-007 — Malformed sessions and local tampering

Area: SEC · **Priority:** P0 · **Result:** PASS (local; full command gates in report)

Authoritative contract: Cached sessions must have valid token/user shapes and finite positive expiry. Optional legacy cache metadata remains compatible; malformed metadata/storage access fails closed. Local user/role/email edits never override backend authority.

Executed evidence: SEC-DEF-002 reproduced malformed cached fullName and missing role in actual parser and browser. Fix passes26 session cases; AUTH13, INC7 and policy6 client regressions pass. Browser verifies malformed-cache sign-in recovery, forged ADMIN denial and logout/reload. No cookie/auth-storage redesign or automatic privileged action was introduced.

WHAT ELHAM SHOULD TEST PERSONALLY: session/error wording, recovery navigation, role/verification feedback and mobile usability. **PRODUCT / OPERATIONS DECISIONS:** none newly outstanding; existing localStorage design is retained and characterized. **STAGING ONLY:** deployed HTTPS/HSTS, edge CORS, WAF/rate limits, production logging and real provider transport. Local tests do not approve an operational SLA or verify deployed infrastructure.

EDGE scenarios

Approved v1 policies implemented and fully validated locally: 6 PASS / 0 FAIL / 0 BLOCKED. Focused units: 174; PostgreSQL: 25; representative browser: 4 checks. EDGE-DEF-001 and EDGE-DEC-001/002/003 are resolved. Earlier 3 PASS / 1 FAIL / 2 BLOCKED and original defect evidence are preserved under `docs/qa/evidence/edge/v1/history/`.

EDGE-001 — Sub-minute and midnight boundary

Area: EDGE · **Priority:** P1 · **Test level:** unit, real PostgreSQL HTTP/concurrency, representative browser · **Result:** PASS

Source / corrected assumptions: AVL already fixed midnight carry; PRICE already rejects less than one whole minute. These contracts remain unchanged.

Executed contract and outcome: Fixed Clock threshold 23:59:30 yields no interval on that date and permits 00:00 the next date. Sub-minute HTTP input fails with HTTP 400 without persistence, notification or provider interaction. Positive whole-minute durations preserve Duration.toMinutes truncation. This Hibernate/JDBC binding persists milliseconds; PostgreSQL supports microseconds but sub-millisecond ORM round-trip fidelity is not claimed. Frontend uses HH:mm; no arbitrary new precision ban.

Evidence: `docs/qa/EDGE_EXECUTION_REPORT.md`, `docs/qa/evidence/edge/v1/`, BookingTimePolicyTest and EdgeIntegrationTest; prior AVL/PRICE/MATCH/BK/INQ/CANCEL/PAYOUT/SEC reports. Fresh PostgreSQL V1–V13 and Hibernate validation; fixed Clock, synthetic fixtures, mocked providers.

Acceptance: local PASS, supported by required complete backend regression and frontend gates. No live provider or deployed infrastructure claim.

EDGE-002 — DST missing/repeated local times

Area: EDGE · **Priority:** P0 · **Test level:** unit, real PostgreSQL HTTP/concurrency, representative browser · **Result:** PASS

Source / corrected assumptions: EDGE-DEC-001 is approved: nonexistent local times are invalid; repeated autumn times use the earlier occurrence. Historical P0 EDGE-DEF-001 reproduced two local reservations aliasing after implicit normalization and a confirmed negative resolved interval. Evidence is retained.

Executed contract and outcome: BookingTimePolicy inspects ZoneRules valid offsets, rejects zero offsets with an actionable HTTP 400 and chooses the first/earlier offset explicitly. Resolved end must follow start. Weekly and AVAILABLE availability split around gap portions; matching cannot offer them. Direct booking and inquiry create/offer/accept reject gap endpoints atomically; legacy gap rows cannot be accepted or checked out. Amsterdam spring alias regression repeats three times, and gap inquiry rollback repeats three times. Autumn 02:30–03:30 resolves 00:30Z–02:30Z: 60 wall-clock billed minutes,120 elapsed minutes, consistent lead time, inquiry offer expiry, confirmation/cancellation snapshots and payout eligibility. Approved pricing is unchanged. No occurrence selector, overnight booking or migration.

Evidence: `docs/qa/EDGE_EXECUTION_REPORT.md`, `docs/qa/evidence/edge/v1/`, BookingTimePolicyTest and EdgeIntegrationTest; prior AVL/PRICE/MATCH/BK/INQ/CANCEL/PAYOUT/SEC reports. Fresh PostgreSQL V1–V13 and Hibernate validation; fixed Clock, synthetic fixtures, mocked providers.

Acceptance: local PASS, supported by required complete backend regression and frontend gates. No live provider or deployed infrastructure claim.

EDGE-003 — Matching filters and bounded candidate batches

Area: EDGE · **Priority:** P1 · **Test level:** unit, real PostgreSQL HTTP/concurrency, representative browser · **Result:** PASS

Source / corrected assumptions: EDGE-DEC-002 is approved: 30 is an internal query batch, not a semantic recall cap. Hard eligibility, visibility, ACTIVE status, budget, production requirements, lead time and exact availability remain authoritative.

Executed contract and outcome: SQL fetches at most 30 candidates per query ordered by hourly price ascending, size descending, name ascending and UUID ascending. Continue using raw batch offsets until 30 distinct eligible available venues or exhaustion. Candidate 31 is considered and returned when the first 30 are unavailable. Tests prove first-batch sufficiency causes no later query; 61 unavailable venues yield genuine empty; ties traverse once in stable order; ranker receives only final eligible available candidates. No full-catalogue materialization, hard-filter relaxation or new search infrastructure. AI preference ordering remains provider-dependent.

Evidence: `docs/qa/EDGE_EXECUTION_REPORT.md`, `docs/qa/evidence/edge/v1/`, BookingTimePolicyTest and EdgeIntegrationTest; prior AVL/PRICE/MATCH/BK/INQ/CANCEL/PAYOUT/SEC reports. Fresh PostgreSQL V1–V13 and Hibernate validation; fixed Clock, synthetic fixtures, mocked providers.

Acceptance: local PASS, supported by required complete backend regression and frontend gates. No live provider or deployed infrastructure claim.

EDGE-004 — Suspended request acceptance

Area: EDGE · **Priority:** P1 · **Test level:** unit, real PostgreSQL HTTP/concurrency, representative browser · **Result:** PASS

Source / corrected assumptions: VEN-D02 already fixed historical suspended acceptance; EDGE adds regression evidence.

Executed contract and outcome: Create REQUESTED while ACTIVE, suspend as ADMIN, then owning OWNER acceptance returns HTTP 409 with identical booking row/audit/notices and no provider call. Prior VEN/BK/AVL venue-lock and inventory races are reused through full regression. Existing accepted commitments and confirmed history remain preserved.

Evidence: `docs/qa/EDGE_EXECUTION_REPORT.md`, `docs/qa/evidence/edge/v1/`, BookingTimePolicyTest and EdgeIntegrationTest; prior AVL/PRICE/MATCH/BK/INQ/CANCEL/PAYOUT/SEC reports. Fresh PostgreSQL V1–V13 and Hibernate validation; fixed Clock, synthetic fixtures, mocked providers.

Acceptance: local PASS, supported by required complete backend regression and frontend gates. No live provider or deployed infrastructure claim.

EDGE-005 — Owner response deadline for both decisions

Area: EDGE · **Priority:** P1 · **Test level:** unit, real PostgreSQL HTTP/concurrency, representative browser · **Result:** PASS

Source / corrected assumptions: EDGE-DEC-003 is approved: ownerResponseExpiresAt applies equally to acceptance and decline. Scheduler timing cannot determine terminal business reason. Historical scheduler-dependent DECLINED versus EXPIRED is retained in evidence.

Executed contract and outcome: At deadline minus 1 microsecond decline returns HTTP 200 / DECLINED and REQUEST_DECLINED. At equality and plus 1 microsecond it commits EXPIRED and REQUEST_EXPIRED then returns HTTP 409 using the existing commit-before-conflict exception convention. Repeated commands/worker are idempotent. Real PostgreSQL latch-controlled decline-first and worker-first orderings repeat three times each and yield one committed expiry event, no decline event. Existing acceptance boundaries are retained. Frontend stale HTTP 409 refetch must show Response window expired, preserve conflict feedback, remove controls and survive reload. No new status, reminder or deadline cap.

Evidence: `docs/qa/EDGE_EXECUTION_REPORT.md`, `docs/qa/evidence/edge/v1/`, BookingTimePolicyTest and EdgeIntegrationTest; prior AVL/PRICE/MATCH/BK/INQ/CANCEL/PAYOUT/SEC reports. Fresh PostgreSQL V1–V13 and Hibernate validation; fixed Clock, synthetic fixtures, mocked providers.

Acceptance: local PASS, supported by required complete backend regression and frontend gates. No live provider or deployed infrastructure claim.

EDGE-006 — Unsupported compatibility service paths

Area: EDGE · **Priority:** P1 · **Test level:** unit, real PostgreSQL HTTP/concurrency, representative browser · **Result:** PASS

Source / corrected assumptions: HeroFeature is used by the homepage; BookerVerification has no importing production component. Service methods alone do not expose endpoints.

Executed contract and outcome: VenueService getAllVenues/deleteVenue have no production controller callers; public GET delegates to getAllActiveVenues. OWNER/ADMIN DELETE returns HTTP 405 without deletion; invented force-confirm/status endpoints return HTTP 404 without booking/event/provider effects. Existing REF endpoint is covered separately. Independently reachable optional Cala BFF remains outside EDGE certification and is tracked by CALA-001.

Evidence: `docs/qa/EDGE_EXECUTION_REPORT.md`, `docs/qa/evidence/edge/v1/`, BookingTimePolicyTest and EdgeIntegrationTest; prior AVL/PRICE/MATCH/BK/INQ/CANCEL/PAYOUT/SEC reports. Fresh PostgreSQL V1–V13 and Hibernate validation; fixed Clock, synthetic fixtures, mocked providers.

Acceptance: local PASS, supported by required complete backend regression and frontend gates. No live provider or deployed infrastructure claim.

EDGE gate: 6 PASS / 0 FAIL / 0 BLOCKED. No unresolved EDGE P0/P1 defect or product/business/legal decision. Required full regression and frontend gates passed. Locally clear to proceed automatically to OPS-001–009 under the user’s approved continuation mode.

OPS scenarios

OPS-001 — Fresh common migrations and Hibernate validation

Area: OPS · **Priority:** P0 · **Test level:** OPERATIONAL, INTEGRATION · **Result:** PASS

Roles involved: Operator-controlled disposable QA resources; scout/owner/admin only where actual booking/document/session access is exercised. No generic actor matrix for Flyway or maintenance.

Preconditions: Local Docker and packaged current release; isolated synthetic fixtures and disabled/mocked providers. Never reset shared local or staging history.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Start the packaged release against an empty disposable PostgreSQL database with only common migrations; compare exact versions and business-table emptiness, then restart and compare all table hashes.

Expected UI behavior: Where a browser is relevant, role controls and reload reflect persisted booking state and immutable receipt access. Database-only checks have no separate UI contract.

Expected API/backend behavior: Common V1–V14 apply once; Hibernate validation succeeds; users, venues, bookings and policy_versions remain empty. Restart changes no persisted rows.

Expected database/domain state: Common V1–V14 apply once; Hibernate validation succeeds; users, venues, bookings and policy_versions remain empty. Restart changes no persisted rows.

Expected email/provider behavior: Providers disabled or mocked; assert calls appropriate to the tested transition. No real provider delivery or settlement claim.

Security/authorization expectation: Preserve existing endpoint authorization and private-document access. Credentials, tokens, dumps and full bank details are excluded from evidence.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: OPS execution report, local recovery runbook, sanitized qa/evidence/ops; existing AUTH/POL/BK/PAY/REF/DEST/PAYOUT/DOC/ADMIN/EDGE reports.

Pass/fail criteria: PASS requires recorded local execution and required regression gates. OPS-DEC-001 now explicitly requires durable outbox dispatch; OPS-008 completion requires the new crash/retry/security assertions and complete regression. Real providers/deployed recovery remain separate staging requirements.

OPS-002 — Local dev chain and production exclusion

Area: OPS · **Priority:** P0 · **Test level:** OPERATIONAL, INTEGRATION · **Result:** PASS

Roles involved: Operator-controlled disposable QA resources; scout/owner/admin only where actual booking/document/session access is exercised. No generic actor matrix for Flyway or maintenance.

Preconditions: Local Docker and packaged current release; isolated synthetic fixtures and disabled/mocked providers. Never reset shared local or staging history.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Start the packaged local profile on a fresh disposable database; assert common1–13/dev100–103, deterministic development seeds and seed login. Audit prod common-only profile and exercise required production configuration rejection/acceptance.

Expected UI behavior: Where a browser is relevant, role controls and reload reflect persisted booking state and immutable receipt access. Database-only checks have no separate UI contract.

Expected API/backend behavior: Development fixtures belong only to local/dev migration locations. Production validates required configuration; unit validation is not a deployed S3/payment/email check. Neither common nor development migrations supply approved legal policy content.

Expected database/domain state: Development fixtures belong only to local/dev migration locations. Production validates required configuration; unit validation is not a deployed S3/payment/email check. Neither common nor development migrations supply approved legal policy content.

Expected email/provider behavior: Providers disabled or mocked; assert calls appropriate to the tested transition. No real provider delivery or settlement claim.

Security/authorization expectation: Preserve existing endpoint authorization and private-document access. Credentials, tokens, dumps and full bank details are excluded from evidence.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: OPS execution report, local recovery runbook, sanitized qa/evidence/ops; existing AUTH/POL/BK/PAY/REF/DEST/PAYOUT/DOC/ADMIN/EDGE reports.

Pass/fail criteria: PASS requires recorded local execution and required regression gates. OPS-DEC-001 now explicitly requires durable outbox dispatch; OPS-008 completion requires the new crash/retry/security assertions and complete regression. Real providers/deployed recovery remain separate staging requirements.

OPS-003 — Constraint and foreign-key behavior

Area: OPS · **Priority:** P0 · **Test level:** OPERATIONAL, INTEGRATION · **Result:** PASS

Roles involved: Operator-controlled disposable QA resources; scout/owner/admin only where actual booking/document/session access is exercised. No generic actor matrix for Flyway or maintenance.

Preconditions: Local Docker and packaged current release; isolated synthetic fixtures and disabled/mocked providers. Never reset shared local or staging history.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Reuse the freshly executed critical-path and financial/inventory domain PostgreSQL suites. Add policy/incident parent-delete rollback and actual restrict-FK catalog assertions.

Expected UI behavior: Where a browser is relevant, role controls and reload reflect persisted booking state and immutable receipt access. Database-only checks have no separate UI contract.

Expected API/backend behavior: Exclusion, financial identities, destination/document/policy uniqueness and referenced-parent restrictions remain enforced by PostgreSQL. A rejected deletion preserves parent and referencing rows.

Expected database/domain state: Exclusion, financial identities, destination/document/policy uniqueness and referenced-parent restrictions remain enforced by PostgreSQL. A rejected deletion preserves parent and referencing rows.

Expected email/provider behavior: Providers disabled or mocked; assert calls appropriate to the tested transition. No real provider delivery or settlement claim.

Security/authorization expectation: Preserve existing endpoint authorization and private-document access. Credentials, tokens, dumps and full bank details are excluded from evidence.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: OPS execution report, local recovery runbook, sanitized qa/evidence/ops; existing AUTH/POL/BK/PAY/REF/DEST/PAYOUT/DOC/ADMIN/EDGE reports.

Pass/fail criteria: PASS requires recorded local execution and required regression gates. OPS-DEC-001 now explicitly requires durable outbox dispatch; OPS-008 completion requires the new crash/retry/security assertions and complete regression. Real providers/deployed recovery remain separate staging requirements.

OPS-004 — Backup and isolated restore

Area: OPS · **Priority:** P0 · **Test level:** OPERATIONAL, INTEGRATION · **Result:** PASS

Roles involved: Operator-controlled disposable QA resources; scout/owner/admin only where actual booking/document/session access is exercised. No generic actor matrix for Flyway or maintenance.

Preconditions: Local Docker and packaged current release; isolated synthetic fixtures and disabled/mocked providers. Never reset shared local or staging history.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Back up synthetic populated PostgreSQL using pg_dump; restore to a separate DB. Copy public/private objects and recover synthetic bank encryption/fingerprint keys. Start the same packaged release, compare all table hashes and constraint definitions, then test documents/media, inquiry, destination reveal, refresh rotation and source isolation.

Expected UI behavior: Where a browser is relevant, role controls and reload reflect persisted booking state and immutable receipt access. Database-only checks have no separate UI contract.

Expected API/backend behavior: Restored rows/IDs and constraints match, allowing equivalent PostgreSQL expression formatting. Receipt/media bytes are identical; authorized access remains enforced; correct keys decrypt and wrong keys fail. Existing restored refresh rotates once; original source stays unchanged. Production backup/secret-escrow/RPO/RTO is staging/operations scope.

Expected database/domain state: Restored rows/IDs and constraints match, allowing equivalent PostgreSQL expression formatting. Receipt/media bytes are identical; authorized access remains enforced; correct keys decrypt and wrong keys fail. Existing restored refresh rotates once; original source stays unchanged. Production backup/secret-escrow/RPO/RTO is staging/operations scope.

Expected email/provider behavior: Providers disabled or mocked; assert calls appropriate to the tested transition. No real provider delivery or settlement claim.

Security/authorization expectation: Preserve existing endpoint authorization and private-document access. Credentials, tokens, dumps and full bank details are excluded from evidence.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: OPS execution report, local recovery runbook, sanitized qa/evidence/ops; existing AUTH/POL/BK/PAY/REF/DEST/PAYOUT/DOC/ADMIN/EDGE reports.

Pass/fail criteria: PASS requires recorded local execution and required regression gates. OPS-DEC-001 now explicitly requires durable outbox dispatch; OPS-008 completion requires the new crash/retry/security assertions and complete regression. Real providers/deployed recovery remain separate staging requirements.

OPS-005 — Backend restart and frontend reload

Area: OPS · **Priority:** P0 · **Test level:** OPERATIONAL, INTEGRATION · **Result:** PASS

Roles involved: Operator-controlled disposable QA resources; scout/owner/admin only where actual booking/document/session access is exercised. No generic actor matrix for Flyway or maintenance.

Preconditions: Local Docker and packaged current release; isolated synthetic fixtures and disabled/mocked providers. Never reset shared local or staging history.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Restart the actual packaged backend against its existing isolated database/files; compare all table hashes and receipt bytes. Reload scout/owner/admin deep links and fetch the receipt in the browser using the existing session.

Expected UI behavior: Where a browser is relevant, role controls and reload reflect persisted booking state and immutable receipt access. Database-only checks have no separate UI contract.

Expected API/backend behavior: Bookings, durable payment creation claims, sessions, policies, inquiry/document/destination rows and file bytes survive ordinary restart. Browser state agrees with PostgreSQL. Pending notification intents survive in PostgreSQL and are dispatched through the transactional outbox under OPS-008.

Expected database/domain state: Bookings, durable payment creation claims, sessions, policies, inquiry/document/destination rows and file bytes survive ordinary restart. Browser state agrees with PostgreSQL. Pending notification intents survive in PostgreSQL and are dispatched through the transactional outbox under OPS-008.

Expected email/provider behavior: Providers disabled or mocked; assert calls appropriate to the tested transition. No real provider delivery or settlement claim.

Security/authorization expectation: Preserve existing endpoint authorization and private-document access. Credentials, tokens, dumps and full bank details are excluded from evidence.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: OPS execution report, local recovery runbook, sanitized qa/evidence/ops; existing AUTH/POL/BK/PAY/REF/DEST/PAYOUT/DOC/ADMIN/EDGE reports.

Pass/fail criteria: PASS requires recorded local execution and required regression gates. OPS-DEC-001 now explicitly requires durable outbox dispatch; OPS-008 completion requires the new crash/retry/security assertions and complete regression. Real providers/deployed recovery remain separate staging requirements.

OPS-006 — Local reset and reseed rehearsal

Area: OPS · **Priority:** P0 · **Test level:** OPERATIONAL, INTEGRATION · **Result:** PASS

Roles involved: Operator-controlled disposable QA resources; scout/owner/admin only where actual booking/document/session access is exercised. No generic actor matrix for Flyway or maintenance.

Preconditions: Local Docker and packaged current release; isolated synthetic fixtures and disabled/mocked providers. Never reset shared local or staging history.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Run scripts/qa/ops-reset-rehearsal.py on a fresh uniquely named local QA Compose project. Assert project volume ownership, reset actual database/public/private volumes, recreate and verify missing markers, exact migrations, deterministic seeds and seed login. Separately verify packaged DB/object-directory reset in the recovery harness.

Expected UI behavior: Where a browser is relevant, role controls and reload reflect persisted booking state and immutable receipt access. Database-only checks have no separate UI contract.

Expected API/backend behavior: Only owned disposable resources are destroyed; shared eema-backend volumes are untouched. Database and file-volume markers disappear;10 venue seeds return; policy_versions stays empty. All owned containers/volumes and temporary keys/files are cleaned.

Expected database/domain state: Only owned disposable resources are destroyed; shared eema-backend volumes are untouched. Database and file-volume markers disappear;10 venue seeds return; policy_versions stays empty. All owned containers/volumes and temporary keys/files are cleaned.

Expected email/provider behavior: Providers disabled or mocked; assert calls appropriate to the tested transition. No real provider delivery or settlement claim.

Security/authorization expectation: Preserve existing endpoint authorization and private-document access. Credentials, tokens, dumps and full bank details are excluded from evidence.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: OPS execution report, local recovery runbook, sanitized qa/evidence/ops; existing AUTH/POL/BK/PAY/REF/DEST/PAYOUT/DOC/ADMIN/EDGE reports.

Pass/fail criteria: PASS requires recorded local execution and required regression gates. OPS-DEC-001 now explicitly requires durable outbox dispatch; OPS-008 completion requires the new crash/retry/security assertions and complete regression. Real providers/deployed recovery remain separate staging requirements.

OPS-007 — Fee metric versus exception consistency

Area: OPS · **Priority:** P0 · **Test level:** OPERATIONAL, INTEGRATION · **Result:** PASS

Roles involved: Operator-controlled disposable QA resources; scout/owner/admin only where actual booking/document/session access is exercised. No generic actor matrix for Flyway or maintenance.

Preconditions: Local Docker and packaged current release; isolated synthetic fixtures and disabled/mocked providers. Never reset shared local or staging history.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: With fixed Clock, compare fee gauge and the complete exception queue before/after205 qualifying paid-history rows across CONFIRMED/CANCELLED/REFUNDED. Test cutoff minus/equal/plus one microsecond and assert read-only snapshots.

Expected UI behavior: Where a browser is relevant, role controls and reload reflect persisted booking state and immutable receipt access. Database-only checks have no separate UI contract.

Expected API/backend behavior: Gauge and queue include the same positive-paid unreconciled rows strictly older than the shared cutoff. Equality and later are excluded. Results beyond200 are reachable; querying/scanning changes no booking state.

Expected database/domain state: Gauge and queue include the same positive-paid unreconciled rows strictly older than the shared cutoff. Equality and later are excluded. Results beyond200 are reachable; querying/scanning changes no booking state.

Expected email/provider behavior: Providers disabled or mocked; assert calls appropriate to the tested transition. No real provider delivery or settlement claim.

Security/authorization expectation: Preserve existing endpoint authorization and private-document access. Credentials, tokens, dumps and full bank details are excluded from evidence.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: OPS execution report, local recovery runbook, sanitized qa/evidence/ops; existing AUTH/POL/BK/PAY/REF/DEST/PAYOUT/DOC/ADMIN/EDGE reports.

Pass/fail criteria: PASS requires recorded local execution and required regression gates. OPS-DEC-001 now explicitly requires durable outbox dispatch; OPS-008 completion requires the new crash/retry/security assertions and complete regression. Real providers/deployed recovery remain separate staging requirements.

OPS-008 — Durable lifecycle notification dispatch and restart recovery

Area: OPS · **Priority:** P0 · **Test level:** UNIT, REAL POSTGRESQL, PACKAGED RESTART · **Result:** PASS

Approved contract: Eema v1 requires durable dispatch of important lifecycle notifications through a PostgreSQL transactional outbox. Business transactions never depend on provider email success. Dispatch is at-least-once; actual inbox delivery is not guaranteed.

Historical record: The prior implementation lost process-local work after domain commit and JVM death. OPS-008 was BLOCKED pending OPS-DEC-001, despite passing characterization. The decision is now approved; original report/crash evidence remains under `qa/evidence/ops/outbox/history` and prior OPS evidence. Do not rewrite that result as if durability existed previously.

Exact steps: Commit booking acceptance with dispatch disabled and verify a PENDING outbox row in the same PostgreSQL transaction; rollback and enqueue failure leave neither transition nor intent. Replay event publication and verify one logical message per recipient. Verify V13 upgrade, fresh V14 and Hibernate startup. Inject transient/permanent provider failures, exact retry/lease boundaries, exhaustion, claim death, two-worker competition, SKIP LOCKED progress and stale-worker completion. Simulate accepted-provider/lost-local-completion and retain the same idempotency identity/content on retry. Kill the packaged JVM after commit; restart with a loopback HTTP provider and prove dispatch without a new domain transition. Saturate the real bounded executor using latches. Verify auth credential absence, reconstruction, supersession, expiry and key rotation; inspect safe ADMIN queue and aggregate metrics.

Expected API/backend and database behavior: Synchronous BEFORE_COMMIT composition inserts notification intent in the domain transaction. PostgreSQL uniqueness prevents duplicate logical recipient/event rows. Short claimed transactions use SKIP LOCKED and a recoverable lease/fence. External transport never holds a business/database transaction. Defaults: batch size 20, one outstanding batch per instance, 8 attempts, 30-second doubling retry capped at 3600 seconds, 300-second leases and 20-second request timeout. Disabled provider leaves pending attempts untouched. Terminal failures retain history and appear in the existing ADMIN-only exception queue. OPS-DEF-001 executor aliases/backpressure remain preserved.

Payload/security: Persist only immutable existing email artifacts and aggregate/type/recipient identity, not entire mutable domain objects. Auth messages instead retain a nonsecret nonce, token-hash/user reference and credential-placeholder artifact. Reconstruct raw credentials only in memory from a versioned keyed derivation; enforce existing token expiry/use/account/recipient. Never persist raw auth credentials, credential-bearing URLs, API keys, refresh/access tokens, bank details or provider responses. No email body or recipient is exposed through exceptions/metric labels.

Provider boundary: Existing templates/recipients only. Stable per-outbox Resend idempotency header mitigates duplicate acceptance within the provider's documented 24-hour retention. Local completion loss may cause retry/duplicate dispatch; no exactly-once or inbox guarantee. Local tests use stubs only. Real Resend and deployed worker operations remain STAGING ONLY.

Evidence: OPS report, recovery runbook, `qa/evidence/ops/outbox`, OperationsNotificationIntegrationTest, OperationsRecoveryIntegrationTest, NotificationOutboxUnitTest, NotificationExecutorBindingTest, existing AUTH/BK/INQ/PAY/REF/TRUST/DEST/ADMIN regression authorities.

Pass/fail criteria: All locally required crash/transaction/security/claim assertions plus complete backend and applicable frontend gates must pass. No new lifecycle templates, notification preferences or terminal-retry admin powers are part of this change.

OPS-009 — Reporting pagination and unavailable fee

Area: OPS · **Priority:** P0 · **Test level:** OPERATIONAL, INTEGRATION · **Result:** PASS

Roles involved: Operator-controlled disposable QA resources; scout/owner/admin only where actual booking/document/session access is exercised. No generic actor matrix for Flyway or maintenance.

Preconditions: Local Docker and packaged current release; isolated synthetic fixtures and disabled/mocked providers. Never reset shared local or staging history.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Mock the real reporting HTTP client for missing token, unavailable fee, valid exact EUR, non-EUR, malformed fee/body, multiple pages, foreign origin, loop and provider failure. Exercise enabled/disabled reconciliation and continuation after per-booking failure/no-fee.

Expected UI behavior: Where a browser is relevant, role controls and reload reflect persisted booking state and immutable receipt access. Database-only checks have no separate UI contract.

Expected API/backend behavior: No fee is invented. Unsafe/invalid reporting is rejected/deferred; the next booking is processed; disabled job makes no provider/query calls. Same-origin pagination retrieves authoritative EUR fees. Real Mollie reporting entitlement/results remain staging-only.

Expected database/domain state: No fee is invented. Unsafe/invalid reporting is rejected/deferred; the next booking is processed; disabled job makes no provider/query calls. Same-origin pagination retrieves authoritative EUR fees. Real Mollie reporting entitlement/results remain staging-only.

Expected email/provider behavior: Providers disabled or mocked; assert calls appropriate to the tested transition. No real provider delivery or settlement claim.

Security/authorization expectation: Preserve existing endpoint authorization and private-document access. Credentials, tokens, dumps and full bank details are excluded from evidence.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: OPS execution report, local recovery runbook, sanitized qa/evidence/ops; existing AUTH/POL/BK/PAY/REF/DEST/PAYOUT/DOC/ADMIN/EDGE reports.

Pass/fail criteria: PASS requires recorded local execution and required regression gates. OPS-DEC-001 now explicitly requires durable outbox dispatch; OPS-008 completion requires the new crash/retry/security assertions and complete regression. Real providers/deployed recovery remain separate staging requirements.

JOB scenarios

Internal scheduled entrypoints; no generic user-role, email-verification or policy-acceptance actor matrix. The JOB execution report records source audit, corrected assumptions, historical defects and current validation.

JOB-001 — BookingApprovalExpirationJob.expireUnpaidApprovals

Area: JOB · **Priority:** P0 · **Test level:** REAL POSTGRESQL, CONCURRENCY, UNIT · **Result:** PASS

Authoritative contract: 60-second default scan; at most 100 REQUESTED rows and 100 AWAITING_PAYMENT rows per invocation. Injected Clock; expiry includes exact deadline. Request batch holds SKIP LOCKED locks transactionally. Approval candidate selection is short-lived and each mutation independently locks/rechecks its booking.

Exact local checks: Seed 101 due requests and 101 due approvals plus future/unrelated states. First scan expires 100 of each; later scans drain the remainder without duplicate audit, outbox or counters. Run competing actual workers with one row held by an independent transaction; the free rows progress and the held row expires later. Roll back a batch and verify no state/audit/outbox/counter outcome.

Boundaries and corrections: BK/PAY/EDGE expiry policies remain authoritative. Existing REQUEST_EXPIRED and APPROVAL_EXPIRED notifications are durable intents; the worker does not contact a payment provider. Do not claim the approval selector and all its mutations share one transaction.

Evidence/dependencies: ScheduledJobsIntegrationTest, existing job/observability tests, prior AUTH/BK/INQ/PAY/REF/PAYOUT/ADMIN/EDGE/OPS reports; `qa/evidence/job/`. Real PostgreSQL V1–V14 and Hibernate validation, fixed Clock, synthetic fixtures, provider stubs and deterministic barriers.

UI/security: Existing authoritative API reload/refetch and authorization remain covered by their domain reports. There is no new scheduler UI or permission. No raw credentials or financial provider responses are retained in evidence.

Acceptance: All locally required assertions and full backend regression must pass. Real provider/deployment behavior is separate; no staging result is inferred.

JOB-002 — AvailabilityInquiryExpirationJob.expireOffers

Area: JOB · **Priority:** P0 · **Test level:** REAL POSTGRESQL, CONCURRENCY, UNIT · **Result:** PASS

Authoritative contract: 60-second default scan; injected Clock and service transaction select at most 100 OFFERED rows whose offer_expires_at is at/before now, FOR UPDATE SKIP LOCKED. EXPIRED records store the scan instant.

Exact local checks: Seed 101 due offers plus a one-microsecond-future offer. Execute the actual job at a fixed advanced clock, verify 100 then the remainder under competing workers. Hold one due row with a latch/independent transaction and prove skip/recovery. Repeat without duplicate OFFER_EXPIRED intent.

Boundaries and corrections: INQ offer lifetime and slot-start cap remain unchanged. No new decline, reminder or booking path. Existing inquiry expiry intent is committed with the transition; provider delivery occurs through OPS outbox after commit.

Evidence/dependencies: ScheduledJobsIntegrationTest, existing job/observability tests, prior AUTH/BK/INQ/PAY/REF/PAYOUT/ADMIN/EDGE/OPS reports; `qa/evidence/job/`. Real PostgreSQL V1–V14 and Hibernate validation, fixed Clock, synthetic fixtures, provider stubs and deterministic barriers.

UI/security: Existing authoritative API reload/refetch and authorization remain covered by their domain reports. There is no new scheduler UI or permission. No raw credentials or financial provider responses are retained in evidence.

Acceptance: All locally required assertions and full backend regression must pass. Real provider/deployment behavior is separate; no staging result is inferred.

JOB-003 — OwnerPayoutReadinessJob.scan

Area: JOB · **Priority:** P0 · **Test level:** REAL POSTGRESQL, CONCURRENCY, UNIT · **Result:** PASS

Authoritative contract: 300-second default scan; at most 100 due CALCULATED/HELD payouts. Transactional booking-first SKIP LOCKED claims preserve PAYOUT lock order. A per-instance keyset cursor walks eligible_at/booking UUID, advances only after commit and wraps at the end. Restart safely revisits earlier rows.

Exact local checks: Verify 101-row batches, repeated audit idempotency, equality and busy/disjoint claims from PAYOUT regression. Place 100 persistently HELD rows before a later eligible payout and prove later progress. Roll back an actual scan: no READY/audit outcome and no cursor advancement. Verify no PAID transition or provider money movement.

Boundaries and corrections: Only existing readiness is recalculated: end time, fee reconciliation, refunds, destination and stored financial rules remain authoritative. READY/PAID/CANCELLED rows are excluded. Scan position is not a payout claim or financial record; no automatic transfer or new retry SLA.

Evidence/dependencies: ScheduledJobsIntegrationTest, existing job/observability tests, prior AUTH/BK/INQ/PAY/REF/PAYOUT/ADMIN/EDGE/OPS reports; `qa/evidence/job/`. Real PostgreSQL V1–V14 and Hibernate validation, fixed Clock, synthetic fixtures, provider stubs and deterministic barriers.

UI/security: Existing authoritative API reload/refetch and authorization remain covered by their domain reports. There is no new scheduler UI or permission. No raw credentials or financial provider responses are retained in evidence.

Acceptance: All locally required assertions and full backend regression must pass. Real provider/deployment behavior is separate; no staging result is inferred.

JOB-004 — AuthTokenCleanupJob.cleanup

Area: JOB · **Priority:** P0 · **Test level:** REAL POSTGRESQL, CONCURRENCY, UNIT · **Result:** PASS

Authoritative contract: 3600-second default scan; injected Clock, transactional deletion of at most 100 verification and 100 reset tokens. Eligible means used_at nonnull OR expires_at strictly before now. Refresh sessions are not housekeeping targets.

Exact local checks: Seed 101 expired tokens per table plus exact-expiry and future-unused tokens. Verify one bounded batch, concurrent remainder cleanup and repeat safety. Separately verify used tokens are removed even before expiry. Refresh-session count remains unchanged. Reuse OPS auth-outbox tests for missing/expired/used token safety.

Boundaries and corrections: Exact-expiry retention for one later cleanup pass is deliberate; redemption still rejects expired tokens at equality. Cleanup grants no extra validity and sends no email. Auth tokens remain hash-only; no raw credentials appear in evidence.

Evidence/dependencies: ScheduledJobsIntegrationTest, existing job/observability tests, prior AUTH/BK/INQ/PAY/REF/PAYOUT/ADMIN/EDGE/OPS reports; `qa/evidence/job/`. Real PostgreSQL V1–V14 and Hibernate validation, fixed Clock, synthetic fixtures, provider stubs and deterministic barriers.

UI/security: Existing authoritative API reload/refetch and authorization remain covered by their domain reports. There is no new scheduler UI or permission. No raw credentials or financial provider responses are retained in evidence.

Acceptance: All locally required assertions and full backend regression must pass. Real provider/deployment behavior is separate; no staging result is inferred.

JOB-005 — MollieFeeReconciliationJob.reconcile

Area: JOB · **Priority:** P0 · **Test level:** REAL POSTGRESQL, CONCURRENCY, UNIT · **Result:** PASS

Authoritative contract: Enabled only by existing configuration; 3600-second default delay and 60-second initial delay. At most 100 read-only provider reporting lookups per invocation, outside database transactions. Candidates are positive-paid unreconciled CONFIRMED/CANCELLED/REFUNDED bookings with a payment ID. Bounded keyset traversal and wrap prevent an unavailable oldest page from hiding later fees.

Exact local checks: Verify 101 successful rows with tied paid timestamps drain in bounded batches and unpaid rows remain unchanged. Place 100 unavailable older fees before one available later fee; repeated scans reach the later row and wrap to deferred rows when fees become available. Race independent workers through a reporting barrier; one persisted reconciliation/audit/payout outcome. Reuse disabled, absent/invalid fee, provider error, safe pagination and exact-money regressions.

Boundaries and corrections: No fee is invented. Provider lookup may repeat across workers; locked reconciliation is idempotent for the same amount and rejects conflicts. Scan cursor is an optimization, resets on restart, and never replaces authoritative financial state. Real Mollie reporting remains STAGING ONLY.

Evidence/dependencies: ScheduledJobsIntegrationTest, existing job/observability tests, prior AUTH/BK/INQ/PAY/REF/PAYOUT/ADMIN/EDGE/OPS reports; `qa/evidence/job/`. Real PostgreSQL V1–V14 and Hibernate validation, fixed Clock, synthetic fixtures, provider stubs and deterministic barriers.

UI/security: Existing authoritative API reload/refetch and authorization remain covered by their domain reports. There is no new scheduler UI or permission. No raw credentials or financial provider responses are retained in evidence.

Acceptance: All locally required assertions and full backend regression must pass. Real provider/deployment behavior is separate; no staging result is inferred.

JOB-006 — WorkflowObservability.scanStuckStates

Area: JOB · **Priority:** P0 · **Test level:** REAL POSTGRESQL, CONCURRENCY, UNIT · **Result:** PASS

Authoritative contract: 300-second default scan; full aggregate counts, not a 100-row mutation batch. Payment/refund/fee/overdue cutoffs are strict; HELD/READY/OPEN categories retain their existing predicates. Lifecycle counters increment AFTER_COMMIT.

Exact local checks: Seed more than 100 old payment creations, active refunds, HELD/READY/overdue payouts and OPEN liabilities. Concurrent scans produce complete gauges and preserve booking/payout rows byte-for-byte. Reuse OPS complete fee count beyond 200 and ADMIN/shared-threshold equality tests. Rollback/replay tests prove counters reflect committed

events.

Boundaries and corrections: Metrics do not remediate financial state or contact providers. No user/email/aggregate identifiers become labels. Process counters are operational telemetry, not a durable accounting ledger. Deployed scraping/alerts remain staging operations.

Evidence/dependencies: ScheduledJobsIntegrationTest, existing job/observability tests, prior AUTH/BK/INQ/PAY/REF/PAYOUT/ADMIN/EDGE/OPS reports; `qa/evidence/job/`. Real PostgreSQL V1–V14 and Hibernate validation, fixed Clock, synthetic fixtures, provider stubs and deterministic barriers.

UI/security: Existing authoritative API reload/refetch and authorization remain covered by their domain reports. There is no new scheduler UI or permission. No raw credentials or financial provider responses are retained in evidence.

Acceptance: All locally required assertions and full backend regression must pass. Real provider/deployment behavior is separate; no staging result is inferred.

RACE scenarios

The RACE execution report reconciles current application-level race evidence with the original SQL-only baseline. All mapped tests passed in the just-completed full PostgreSQL run on unchanged production/test source; no additional run or new test is claimed.

RACE-001 — Booking acceptance

Area: RACE · **Priority:** P0 · **Test level:** REAL POSTGRESQL, APPLICATION CONCURRENCY · **Result:** PASS

Corrected contract and executed outcome: Three HTTP venue-lock barrier repetitions produce 200/409, one AWAITING_PAYMENT and one UNAVAILABLE booking. Persisted availability excludes the winner and reopens after expiry. Venue-first/booking-second locking and PostgreSQL exclusion are separate authorities.

Execution mapping: `VenueDomainIntegrationTest.avlConcurrentAcceptanceReservesOnceAndExpiryReopens`; `PostgresCriticalPathIntegrationTest.overlappingRequestedRowsCoexistButConcurrentAcceptanceCannotDoubleReserve`. `qa/evidence/race/scenario-mapping.json` and `reused-full-suite.json` link the audited methods to actual successful class results.

Method: Independent transactions/connections, deterministic barriers/latches and persisted outcomes. Existing domain authorization, outbox and browser/refetch evidence is reused. No arbitrary sleep determines a race result.

Boundaries: Synthetic local fixtures and mocked/disabled providers. No real money movement, legal approval or deployed multi-instance/provider guarantee. Actor and side-effect expectations come from the actual domain, not a generic scout/owner matrix.

Acceptance: Local PASS with current full backend gates and unchanged frontend validation reused. No new P0/P1 defect or product decision. Historical raw SQL tests remain corroboration, not a substitute for application-level evidence.

RACE-002 — Blocking insert

Area: RACE · **Priority:** P0 · **Test level:** REAL POSTGRESQL, APPLICATION CONCURRENCY · **Result:** PASS

Corrected contract and executed outcome: Concurrent inquiry/inquiry and inquiry/normal acceptance serialize at the same venue reservation boundary: one reservation, safe 200/409 and no accepted inquiry without a booking. Duplicate same-offer requests create one booking/audit. A separate SQL race proves overlapping AWAITING_PAYMENT/CONFIRMED inserts cannot both commit; adjacent half-open intervals are allowed.

Execution mapping: `InquiryIntegrationTest.twoOffersAndNormalAcceptanceShareVenueReservationBoundary`; `InquiryIntegrationTest.concurrentSameOfferCreatesExactlyOneBooking`; `PostgresCriticalPathIntegrationTest.exclusionRejectsOneOverlappingBlockingInsert`. `qa/evidence/race/scenario-mapping.json` and `reused-full-suite.json` link the audited methods to actual successful class results.

Method: Independent transactions/connections, deterministic barriers/latches and persisted outcomes. Existing domain authorization, outbox and browser/refetch evidence is reused. No arbitrary sleep determines a race result.

Boundaries: Synthetic local fixtures and mocked/disabled providers. No real money movement, legal approval or deployed multi-instance/provider guarantee. Actor and side-effect expectations come from the actual domain, not a generic scout/owner matrix.

Acceptance: Local PASS with current full backend gates and unchanged frontend validation reused. No new P0/P1 defect or product decision. Historical raw SQL tests remain corroboration, not a substitute for application-level evidence.

RACE-003 — Refresh rotation

Area: RACE · **Priority:** P0 · **Test level:** REAL POSTGRESQL, APPLICATION CONCURRENCY · **Result:** PASS

Corrected contract and executed outcome: Concurrent HTTP use of one refresh credential yields 200/401, one revoked original and one live successor. The credential is hashed before the row-lock lookup. No authenticated-role/email-verification/policy prerequisite is invented. Existing auth client single-flight/session-race behavior remains authoritative.

Execution mapping: `AuthSessionIntegrationTest.concurrentHttpRefreshHasOneWinner`; `PostgresCriticalPathIntegrationTest.refreshTokenRotationLockAllowsOnlyOneWinner`. `qa/evidence/race/scenario-mapping.json` and `reused-full-suite.json` link the audited methods to actual successful class results.

Method: Independent transactions/connections, deterministic barriers/latches and persisted outcomes. Existing domain authorization, outbox and browser/refetch evidence is reused. No arbitrary sleep determines a race result.

Boundaries: Synthetic local fixtures and mocked/disabled providers. No real money movement, legal approval or deployed multi-instance/provider guarantee. Actor and side-effect expectations come from the actual domain, not a generic scout/owner matrix.

Acceptance: Local PASS with current full backend gates and unchanged frontend validation reused. No new P0/P1 defect or product decision. Historical raw SQL tests remain corroboration, not a substitute for application-level evidence.

RACE-004 — Payout uniqueness

Area: RACE · **Priority:** P0 · **Test level:** REAL POSTGRESQL, APPLICATION CONCURRENCY · **Result:** PASS

Corrected contract and executed outcome: Concurrent locked recalculation creates one payout ledger and one audit with exact stored financial inputs. Same manual paid-recording reference is idempotent; different reference conflicts with one PAID audit. No public payout-create API or automatic transfer is inferred from the internal calculation method.

Execution mapping: `PayoutIntegrationTest.concurrentRecalculationCreatesOneLedgerAndOneAudit`; `PayoutIntegrationTest.concurrentPaidRecordingIsIdempotentOrConflicts`; `PostgresCriticalPathIntegrationTest.payoutAndDocumentUniqueConstraintsHaveSingleAuthorities`. `qa/evidence/race/scenario-mapping.json` and `reused-full-suite.json` link the audited methods to actual successful class results.

Method: Independent transactions/connections, deterministic barriers/latches and persisted outcomes. Existing domain authorization, outbox and browser/refetch evidence is reused. No arbitrary sleep determines a race result.

Boundaries: Synthetic local fixtures and mocked/disabled providers. No real money movement, legal approval or deployed multi-instance/provider guarantee. Actor and side-effect expectations come from the actual domain, not a generic scout/owner matrix.

Acceptance: Local PASS with current full backend gates and unchanged frontend validation reused. No new P0/P1 defect or product decision. Historical raw SQL tests remain corroboration, not a substitute for application-level evidence.

RACE-005 — Verified destination

Area: RACE · **Priority:** P0 · **Test level:** REAL POSTGRESQL, APPLICATION CONCURRENCY · **Result:** PASS

Corrected contract and executed outcome: Distinct pending destinations for one owner may both be verified in serialized HTTP commands, both returning 200, ending with one VERIFIED and one REPLACED row. Same-record verify/reject has one decision and 200/409. Historical READY/PAID routing stays attached to its valid snapshot. A raw SQL uniqueness race is not the application replacement contract.

Execution mapping: `DestinationIntegrationTest.concurrentDistinctVerificationSerializesOwnerRouting`; `DestinationIntegrationTest.sameRecordVerifyRejectHasOneWinner`; `PostgresCriticalPathIntegrationTest.verifiedDestinationPartialIndexAndReplacementSemanticsHold`. `qa/evidence/race/scenario-mapping.json` and `reused-full-suite.json` link the audited methods to actual successful class results.

Method: Independent transactions/connections, deterministic barriers/latches and persisted outcomes. Existing domain authorization, outbox and browser/refetch evidence is reused. No arbitrary sleep determines a race result.

Boundaries: Synthetic local fixtures and mocked/disabled providers. No real money movement, legal approval or deployed multi-instance/provider guarantee. Actor and side-effect expectations come from the actual domain, not a generic scout/owner matrix.

Acceptance: Local PASS with current full backend gates and unchanged frontend validation reused. No new P0/P1 defect or product decision. Historical raw SQL tests remain corroboration, not a substitute for application-level evidence.

RACE-006 — Policy publication

Area: RACE · **Priority:** P0 · **Test level:** REAL POSTGRESQL, APPLICATION CONCURRENCY · **Result:** PASS

Corrected contract and executed outcome: Three HTTP competing replacement publications, synchronized after reading the current version, yield one 200/one 409, one current policy and a still-DRAFT losing target. A separate SQL race verifies first-publication uniqueness. Tests use synthetic policy text and do not approve legal content or production publication.

Execution mapping: `PolicyIntegrationTest.concurrentServicePublicationHasOneCurrentAndSafeConflict`; `PostgresCriticalPathIntegrationTest.policyPublishRaceLeavesExactlyOneCurrentVersion`. `qa/evidence/race/scenario-mapping.json` and `reused-full-suite.json` link the audited methods to actual successful class results.

Method: Independent transactions/connections, deterministic barriers/latches and persisted outcomes. Existing domain authorization, outbox and browser/refetch evidence is reused. No arbitrary sleep determines a race result.

Boundaries: Synthetic local fixtures and mocked/disabled providers. No real money movement, legal approval or deployed multi-instance/provider guarantee. Actor and side-effect expectations come from the actual domain, not a generic scout/owner matrix.

Acceptance: Local PASS with current full backend gates and unchanged frontend validation reused. No new P0/P1 defect or product decision. Historical raw SQL tests remain corroboration, not a substitute for application-level evidence.

RACE-007 — Document issuance

Area: RACE · **Priority:** P0 · **Test level:** REAL POSTGRESQL, APPLICATION CONCURRENCY · **Result:** PASS

Corrected contract and executed outcome: Three receipt and three statement concurrent HTTP first-download races return identical immutable bytes with one object/document/audit. Parent locks and document uniqueness serialize issuance. Storage/DB/late-commit failures leave no successful issue and clean up the new object. Existing Unicode-name and historical-read contracts are preserved.

Execution mapping: `DocumentIntegrationTest.simultaneousReceiptRequestsIssueOneObjectAndAudit`; `DocumentIntegrationTest.simultaneousStatementRequestsIssueOneObjectAndAudit`; `DocumentIntegrationTest.failedIssuanceNeverSucceedsAndCleansStoredObject`. `qa/evidence/race/scenario-mapping.json` and `reused-full-suite.json` link the audited methods to actual successful class results.

Method: Independent transactions/connections, deterministic barriers/latches and persisted outcomes. Existing domain authorization, outbox and browser/refetch evidence is reused. No arbitrary sleep determines a race result.

Boundaries: Synthetic local fixtures and mocked/disabled providers. No real money movement, legal approval or deployed multi-instance/provider guarantee. Actor and side-effect expectations come from the actual domain, not a generic scout/owner matrix.

Acceptance: Local PASS with current full backend gates and unchanged frontend validation reused. No new P0/P1 defect or product decision. Historical raw SQL tests remain corroboration, not a substitute for application-level evidence.

RACE-008 — SKIP LOCKED workers

Area: RACE · **Priority:** P0 · **Test level:** REAL POSTGRESQL, APPLICATION CONCURRENCY · **Result:** PASS

Corrected contract and executed outcome: Actual booking/inquiry/payout workers prove bounded selection, held-row skip/recovery, singular committed outcomes and rollback safety. Outbox workers prove one simultaneous claim, lease equality recovery and stale-fence rejection. Approval candidate locks are short-lived with per-row recheck; fee reporting may overlap read-only lookups. Do not invent a worker HTTP endpoint or require identical claim semantics for every job.

Execution mapping: `ScheduledJobsIntegrationTest.bookingWorkersAndBusyRowLeaveOneCommittedTransitionPerBooking`; `ScheduledJobsIntegrationTest.inquiryWorkerSkipsLockedOfferThenRecoversItWithoutDuplicateIntent`; `PayoutIntegrationTest.readinessWorkersClaimDisjointBookingsInIndependentTransactions`; `OperationsNotificationIntegrationTest.twoWorkersCannotSendTheSameClaimSimultaneously`; `OperationsNotificationIntegrationTest.expiredLeaseRecoversAtEqualityAndOldWorkerCannotComplete`. `qa/evidence/race/scenario-mapping.json` and `reused-full-suite.json` link the audited methods to actual successful class results.

Method: Independent transactions/connections, deterministic barriers/latches and persisted outcomes. Existing domain authorization, outbox and browser/refetch evidence is reused. No arbitrary sleep determines a race result.

Boundaries: Synthetic local fixtures and mocked/disabled providers. No real money movement, legal approval or deployed multi-instance/provider guarantee. Actor and side-effect expectations come from the actual domain, not a generic scout/owner matrix.

Acceptance: Local PASS with current full backend gates and unchanged frontend validation reused. No new P0/P1 defect or product decision. Historical raw SQL tests remain corroboration, not a substitute for application-level evidence.

UX scenarios

UX-001 — Page design, navigation and state QA: /admin/bookings/[id]

Area: UX · **Priority:** P2 · **Test level:** MANUAL UX, LOCAL E2E, SECURITY · **Result:** BLOCKED

Roles involved: ADMIN

Preconditions: Use the actual route role/auth contract. Verified email and current policies apply only to relevant marketplace mutations, not every page read or public recovery flow.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: 1. Open /admin/bookings/[id] directly and from every header/dashboard link as ADMIN; substitute owned fixture UUID for [id]. 2. Repeat anonymous and wrong-role access. 3. At 375px, 768px and 1440px inspect all cards/forms/tables/gallery/dialogs/financial breakdowns. 4. Keyboard-tab entire page, activate primary/secondary actions, close dialogs, check returned focus; verify labels/errors/loading/table headings structurally; actual screen-reader acceptance uses the shared 11-flow matrix rather than repeating this route mechanically. 5. Delay API 3 seconds; return empty collection or missing detail; then exercise current states OPEN, error, loading, ready. 6. Inject 400/401/403/404/409/500, offline, invalid JSON and valid JSON wrong shape into each page loader; restore and retry. 7. Use long UUID, max venue name/reason, Unicode and large money values; reload and navigate back. 8. Review each design item below and record screenshot/finding.

Expected UI behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. Visible focus, associated labels/errors, semantic headings/tables, meaningful button names, focus containment/return for dialogs, status not color-only, live loading/status announcement and contrast must be checked.

Expected API/backend behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. No private data flash before role resolution; errors contain no stack/provider output; independent failed panels do not hide valid sibling data. State/action visibility agrees with backend. Fail any uncovered case; static /admin home only loads auth/policy state, not its own result list.

Expected database/domain state: Read-only variants do not mutate; any invoked workflow must satisfy its domain scenario and endpoint matrix.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: Local route/state automation plus representative real screen-reader acceptance; see qa/UX_ASSISTIVE_TECHNOLOGY_CHECKLIST.md.

Local/staging requirement: Local browser with controlled response injection; real PostgreSQL workflow evidence reused. Shared representative actual-AT acceptance is pending (UX-ENV-001); the normal desktop/manual checklist is the approved execution path.

Source / existing evidence: frontend/app/admin/bookings/[id]/page.tsx

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

Representative AT assignment: AT-R09 requires actual execution on this route; result remains BLOCKED. See representative checklist.

Execution addendum (2026-09-21): Automated route checks pass. The domain-wide representative actual-AT gate is BLOCKED by UX-ENV-001; this does not require 35 duplicate reader runs. Browser accessibility-tree/keyboard checks are not actual screen-reader execution. Actual page/loader roles and states supersede generic matrices and Promise implementation labels. See UX execution report and route evidence. Historical generated wording is retained in evidence; no automatic continuation.

UX-002 — Page design, navigation and state QA: /admin/bookings

Area: UX · **Priority:** P2 · **Test level:** MANUAL UX, LOCAL E2E, SECURITY · **Result:** BLOCKED

Roles involved: ADMIN

Preconditions: Use the actual route role/auth contract. Verified email and current policies apply only to relevant marketplace mutations, not every page read or public recovery flow.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: 1. Open /admin/bookings directly and from every header/dashboard link as ADMIN; substitute owned fixture UUID for [id]. 2. Repeat anonymous and wrong-role access. 3. At 375px, 768px and 1440px inspect all cards/forms/tables/gallery/dialogs/financial breakdowns. 4. Keyboard-tab entire page, activate primary/secondary actions, close dialogs, check returned focus; verify labels/errors/loading/table headings structurally; actual screen-reader acceptance uses the shared 11-flow matrix rather than repeating this route mechanically. 5. Delay API 3 seconds; return empty collection or missing detail; then exercise current states ready. 6. Inject 400/401/403/404/409/500, offline, invalid JSON and valid JSON wrong shape into each page loader; restore and retry. 7. Use long UUID, max venue name/reason, Unicode and large money values; reload and navigate back. 8. Review each design item below and record screenshot/finding.

Expected UI behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. Visible focus, associated labels/errors, semantic headings/tables, meaningful button names, focus containment/return for dialogs, status not color-only, live loading/status announcement and contrast must be checked.

Expected API/backend behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. No private data flash before role resolution; errors contain no stack/provider output; independent failed panels do not hide valid sibling data. State/action visibility agrees with backend. Fail any uncovered case; static /admin home only loads auth/policy state, not its own result list.

Expected database/domain state: Read-only variants do not mutate; any invoked workflow must satisfy its domain scenario and endpoint matrix.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: Local route/state automation plus representative real screen-reader acceptance; see qa/UX_ASSISTIVE_TECHNOLOGY_CHECKLIST.md.

Local/staging requirement: Local browser with controlled response injection; real PostgreSQL workflow evidence reused. Shared representative actual-AT acceptance is pending (UX-ENV-001); the normal desktop/manual checklist is the approved execution path.

Source / existing evidence: frontend/app/admin/bookings/page.tsx

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

Representative AT assignment: AT-R08 requires actual execution on this route; result remains BLOCKED. See representative checklist.

Execution addendum (2026-09-21): Automated route checks pass. The domain-wide representative actual-AT gate is BLOCKED by UX-ENV-001; this does not require 35 duplicate reader runs. Browser accessibility-tree/keyboard checks are not actual screen-reader execution. Actual page/loader roles and states supersede generic matrices and Promise implementation labels. See UX execution report and route evidence. Historical generated wording is retained in evidence; no automatic continuation.

UX-003 — Page design, navigation and state QA: /admin/exceptions

Area: UX · **Priority:** P2 · **Test level:** MANUAL UX, LOCAL E2E, SECURITY · **Result:** BLOCKED

Roles involved: ADMIN

Preconditions: Use the actual route role/auth contract. Verified email and current policies apply only to relevant marketplace mutations, not every page read or public recovery flow.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: 1. Open /admin/exceptions directly and from every header/dashboard link as ADMIN; substitute owned fixture UUID for [id]. 2. Repeat anonymous and wrong-role access. 3. At 375px, 768px and 1440px inspect all cards/forms/tables/gallery/dialogs/financial breakdowns. 4. Keyboard-tab entire page, activate primary/secondary actions, close dialogs, check returned focus; verify labels/errors/loading/table headings structurally; actual screen-reader acceptance uses the shared 11-flow matrix rather than repeating this route mechanically. 5. Delay API 3 seconds; return empty collection or missing detail; then exercise current states ready. 6. Inject 400/401/403/404/409/500, offline, invalid JSON and valid JSON wrong shape into each page loader; restore and retry. 7. Use long UUID, max venue name/reason, Unicode and large money values; reload and navigate back. 8. Review each design item below and record screenshot/finding.

Expected UI behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. Visible focus, associated labels/errors, semantic headings/tables, meaningful button names, focus containment/return for dialogs, status not color-only, live loading/status announcement and contrast must be checked.

Expected API/backend behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. No private data flash before role resolution; errors contain no stack/provider output; independent failed panels do not hide valid sibling data. State/action visibility agrees with backend. Fail any uncovered case; static /admin home only loads auth/policy state, not its own result list.

Expected database/domain state: Read-only variants do not mutate; any invoked workflow must satisfy its domain scenario and endpoint matrix.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: Local route/state automation plus representative real screen-reader acceptance; see qa/UX_ASSISTIVE_TECHNOLOGY_CHECKLIST.md.

Local/staging requirement: Local browser with controlled response injection; real PostgreSQL workflow evidence reused. Shared representative actual-AT acceptance is pending (UX-ENV-001); the normal desktop/manual checklist is the approved execution path.

Source / existing evidence: frontend/app/admin/exceptions/page.tsx

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

Representative AT assignment: AT-R11 requires actual execution on this route; result remains BLOCKED. See representative checklist.

Execution addendum (2026-09-21): Automated route checks pass. The domain-wide representative actual-AT gate is BLOCKED by UX-ENV-001; this does not require 35 duplicate reader runs. Browser accessibility-tree/keyboard checks are not actual screen-reader execution. Actual page/loader roles and states supersede generic matrices and Promise implementation labels. See UX execution report and route evidence. Historical generated wording is retained in evidence; no automatic continuation.

UX-004 — Page design, navigation and state QA: /admin/incidents/[id]

Area: UX · **Priority:** P2 · **Test level:** MANUAL UX, LOCAL E2E, SECURITY · **Result:** PASS

Roles involved: ADMIN

Preconditions: Use the actual route role/auth contract. Verified email and current policies apply only to relevant marketplace mutations, not every page read or public recovery flow.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: 1. Open /admin/incidents/[id] directly and from every header/dashboard link as ADMIN; substitute owned fixture UUID for [id]. 2. Repeat anonymous and wrong-role access. 3. At 375px, 768px and 1440px inspect all cards/forms/tables/gallery/dialogs/financial breakdowns. 4. Keyboard-tab entire page, activate primary/secondary actions, close dialogs, check returned

focus; verify labels/errors/loading/table headings structurally; actual screen-reader acceptance uses the shared 11-flow matrix rather than repeating this route mechanically. 5. Delay API 3 seconds; return empty collection or missing detail; then exercise current states OPEN, RESOLVED, UNDER_REVIEW, ready. 6. Inject 400/401/403/404/409/500, offline, invalid JSON and valid JSON wrong shape into each page loader; restore and retry. 7. Use long UUID, max venue name/reason, Unicode and large money values; reload and navigate back. 8. Review each design item below and record screenshot/finding.

Expected UI behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. Visible focus, associated labels/errors, semantic headings/tables, meaningful button names, focus containment/return for dialogs, status not color-only, live loading/status announcement and contrast must be checked.

Expected API/backend behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. No private data flash before role resolution; errors contain no stack/provider output; independent failed panels do not hide valid sibling data. State/action visibility agrees with backend. Fail any uncovered case; static /admin home only loads auth/policy state, not its own result list.

Expected database/domain state: Read-only variants do not mutate; any invoked workflow must satisfy its domain scenario and endpoint matrix.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: Local route/state automation plus representative real screen-reader acceptance; see qa/UX_ASSISTIVE_TECHNOLOGY_CHECKLIST.md.

Local/staging requirement: Local browser with controlled response injection; real PostgreSQL workflow evidence reused. Shared representative actual-AT acceptance is pending (UX-ENV-001); the normal desktop/manual checklist is the approved execution path.

Source / existing evidence: frontend/app/admin/incidents/[id]/page.tsx

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

Representative AT assignment: No separate reader run is required on this route. PASS covers the corrected route scope; overall UX acceptance still depends on all 11 shared representatives. See representative checklist.

Execution addendum (2026-09-21): Automated route checks pass. The domain-wide representative actual-AT gate is BLOCKED by UX-ENV-001; this does not require 35 duplicate reader runs. Browser accessibility-tree/keyboard checks are not actual screen-reader execution. Actual page/loader roles and states supersede generic matrices and Promise implementation labels. See UX execution report and route evidence. Historical generated wording is retained in evidence; no automatic continuation.

UX-005 — Page design, navigation and state QA: /admin/incidents

Area: UX · **Priority:** P2 · **Test level:** MANUAL UX, LOCAL E2E, SECURITY · **Result:** PASS

Roles involved: ADMIN

Preconditions: Use the actual route role/auth contract. Verified email and current policies apply only to relevant marketplace mutations, not every page read or public recovery flow.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: 1. Open /admin/incidents directly and from every header/dashboard link as ADMIN; substitute owned fixture UUID for [id]. 2. Repeat anonymous and wrong-role access. 3. At 375px, 768px and 1440px inspect all cards/forms/tables/gallery/dialogs/financial breakdowns. 4. Keyboard-tab entire page, activate primary/secondary actions, close dialogs, check returned focus; verify labels/errors/loading/table headings structurally; actual screen-reader acceptance uses the shared 11-flow matrix rather than repeating this route mechanically. 5. Delay API 3 seconds; return empty collection or missing detail; then exercise current states OPEN, ready. 6. Inject 400/401/403/404/409/500, offline, invalid JSON and valid JSON wrong shape into each page loader; restore and retry. 7. Use long UUID, max venue name/reason, Unicode and large money values; reload and navigate back. 8. Review each design item below and record screenshot/finding.

Expected UI behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. Visible focus, associated labels/errors, semantic headings/tables, meaningful button names, focus containment/return for dialogs, status not color-only, live loading/status announcement and contrast must be checked.

Expected API/backend behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. No private data flash before role resolution; errors contain no stack/provider output; independent failed panels do not hide valid sibling data. State/action visibility agrees with backend. Fail any uncovered case; static /admin home only loads auth/policy state, not its own result list.

Expected database/domain state: Read-only variants do not mutate; any invoked workflow must satisfy its domain scenario and endpoint matrix.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: Local route/state automation plus representative real screen-reader acceptance; see qa/UX_ASSISTIVE_TECHNOLOGY_CHECKLIST.md.

Local/staging requirement: Local browser with controlled response injection; real PostgreSQL workflow evidence reused. Shared representative actual-AT acceptance is pending (UX-ENV-001); the normal desktop/manual checklist is the approved execution path.

Source / existing evidence: frontend/app/admin/incidents/page.tsx

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

Representative AT assignment: No separate reader run is required on this route. PASS covers the corrected route scope; overall UX acceptance still depends on all 11 shared representatives. See representative checklist.

Execution addendum (2026-09-21): Automated route checks pass. The domain-wide representative actual-AT gate is BLOCKED by UX-ENV-001; this does not require 35 duplicate reader runs. Browser accessibility-tree/keyboard checks are not actual screen-reader execution. Actual page/loader roles and states supersede generic matrices and Promise implementation labels. See UX execution report and route evidence. Historical generated wording is retained in evidence; no automatic continuation.

UX-006 — Page design, navigation and state QA: /admin/liabilities/[id]

Area: UX · **Priority:** P2 · **Test level:** MANUAL UX, LOCAL E2E, SECURITY · **Result:** PASS

Roles involved: ADMIN

Preconditions: Use the actual route role/auth contract. Verified email and current policies apply only to relevant marketplace mutations, not every page read or public recovery flow.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: 1. Open /admin/liabilities/[id] directly and from every header/dashboard link as ADMIN; substitute owned fixture UUID for [id]. 2. Repeat anonymous and wrong-role access. 3. At 375px, 768px and 1440px inspect all cards/forms/tables/gallery/dialogs/financial breakdowns. 4. Keyboard-tab entire page, activate primary/secondary actions, close dialogs, check returned focus; verify labels/errors/loading/table headings structurally; actual screen-reader acceptance uses the shared 11-flow matrix rather than repeating this route mechanically. 5. Delay API 3 seconds; return empty collection or missing detail; then exercise current states OPEN, ready. 6. Inject 400/401/403/404/409/500, offline, invalid JSON and valid JSON wrong shape into each page loader; restore and retry. 7. Use long UUID, max venue name/reason, Unicode and large money values; reload and navigate back. 8. Review each design item below and record screenshot/finding.

Expected UI behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. Visible focus, associated labels/errors, semantic headings/tables, meaningful button names, focus containment/return for dialogs, status not color-only, live loading/status announcement and contrast must be checked.

Expected API/backend behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. No private data flash before role resolution; errors contain no stack/provider output; independent failed panels do not hide valid sibling data. State/action visibility agrees with backend. Fail any uncovered case; static /admin home only loads auth/policy state, not its own result list.

Expected database/domain state: Read-only variants do not mutate; any invoked workflow must satisfy its domain scenario and endpoint matrix.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: Local route/state automation plus representative real screen-reader acceptance; see qa/UX_ASSISTIVE_TECHNOLOGY_CHECKLIST.md.

Local/staging requirement: Local browser with controlled response injection; real PostgreSQL workflow evidence reused. Shared representative actual-AT acceptance is pending (UX-ENV-001); the normal desktop/manual checklist is the approved execution path.

Source / existing evidence: frontend/app/admin/liabilities/[id]/page.tsx

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

Representative AT assignment: No separate reader run is required on this route. PASS covers the corrected route scope; overall UX acceptance still depends on all 11 shared representatives. See representative checklist.

Execution addendum (2026-09-21): Automated route checks pass. The domain-wide representative actual-AT gate is BLOCKED by UX-ENV-001; this does not require 35 duplicate reader runs. Browser accessibility-tree/keyboard checks are not actual screen-reader execution. Actual page/loader roles and states supersede generic matrices and Promise implementation labels. See UX execution report and route evidence. Historical generated wording is retained in evidence; no automatic continuation.

UX-007 — Page design, navigation and state QA: /admin/liabilities

Area: UX · **Priority:** P2 · **Test level:** MANUAL UX, LOCAL E2E, SECURITY · **Result:** PASS

Roles involved: ADMIN

Preconditions: Use the actual route role/auth contract. Verified email and current policies apply only to relevant marketplace mutations, not every page read or public recovery flow.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: 1. Open /admin/liabilities directly and from every header/dashboard link as ADMIN; substitute owned fixture UUID for [id]. 2. Repeat anonymous and wrong-role access. 3. At 375px, 768px and 1440px inspect all cards/forms/tables/gallery/dialogs/financial breakdowns. 4. Keyboard-tab entire page, activate primary/secondary actions, close dialogs, check returned focus; verify labels/errors/loading/table headings structurally; actual screen-reader acceptance uses the shared 11-flow matrix rather than repeating this route mechanically. 5. Delay API 3 seconds; return empty collection or missing detail; then exercise current states OPEN, ready. 6. Inject 400/401/403/404/409/500, offline, invalid JSON and valid JSON wrong shape into each page loader; restore and retry. 7. Use long UUID, max venue name/reason, Unicode and large money values; reload and navigate back. 8. Review each design item below and record screenshot/finding.

Expected UI behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. Visible focus, associated labels/errors, semantic headings/tables, meaningful button names, focus containment/return for dialogs, status not color-only, live loading/status announcement and contrast must be checked.

Expected API/backend behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. No private data flash before role resolution; errors contain no stack/provider output; independent failed panels do not hide valid sibling data. State/action visibility agrees with backend. Fail any uncovered case; static /admin home only loads auth/policy state, not its own result list.

Expected database/domain state: Read-only variants do not mutate; any invoked workflow must satisfy its domain scenario and endpoint matrix.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: Local route/state automation plus representative real screen-reader acceptance; see qa/UX_ASSISTIVE_TECHNOLOGY_CHECKLIST.md.

Local/staging requirement: Local browser with controlled response injection; real PostgreSQL workflow evidence reused. Shared representative actual-AT acceptance is pending (UX-ENV-001); the normal desktop/manual checklist is the approved execution path.

Source / existing evidence: frontend/app/admin/liabilities/page.tsx

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

Representative AT assignment: No separate reader run is required on this route. PASS covers the corrected route scope; overall UX acceptance still depends on all 11 shared representatives. See representative checklist.

Execution addendum (2026-09-21): Automated route checks pass. The domain-wide representative actual-AT gate is BLOCKED by UX-ENV-001; this does not require 35 duplicate reader runs. Browser accessibility-tree/keyboard checks are not actual screen-reader execution. Actual page/loader roles and states supersede generic matrices and Promise implementation labels. See UX execution report and route evidence. Historical generated wording is retained in evidence; no automatic continuation.

UX-008 — Page design, navigation and state QA: /admin/owner-verifications/[id]

Area: UX · **Priority:** P2 · **Test level:** MANUAL UX, LOCAL E2E, SECURITY · **Result:** PASS

Roles involved: ADMIN

Preconditions: Use the actual route role/auth contract. Verified email and current policies apply only to relevant marketplace mutations, not every page read or public recovery flow.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: 1. Open /admin/owner-verifications/[id] directly and from every header/dashboard link as ADMIN; substitute owned fixture UUID for [id]. 2. Repeat anonymous and wrong-role access. 3. At 375px, 768px and 1440px inspect all cards/forms/tables/gallery/dialogs/financial breakdowns. 4. Keyboard-tab entire page, activate primary/secondary actions, close dialogs, check returned focus; verify labels/errors/loading/table headings structurally; actual screen-reader acceptance uses the shared 11-flow matrix rather than repeating this route mechanically. 5. Delay API 3 seconds; return empty collection or missing detail; then exercise current states PENDING, ready. 6. Inject 400/401/403/404/409/500, offline, invalid JSON and valid JSON wrong shape into each page loader; restore and retry. 7. Use long UUID, max venue name/reason, Unicode and large money values; reload and navigate back. 8. Review each design item below and record screenshot/finding.

Expected UI behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. Visible focus, associated labels/errors, semantic headings/tables, meaningful button names, focus containment/return for dialogs, status not color-only, live loading/status announcement and contrast must be checked.

Expected API/backend behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. No private data flash before role resolution; errors contain no stack/provider output; independent failed panels do not hide valid sibling data. State/action visibility agrees with backend. Fail any uncovered case; static /admin home only loads auth/policy state, not its own result list.

Expected database/domain state: Read-only variants do not mutate; any invoked workflow must satisfy its domain scenario and endpoint matrix.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: Local route/state automation plus representative real screen-reader acceptance; see qa/UX_ASSISTIVE_TECHNOLOGY_CHECKLIST.md.

Local/staging requirement: Local browser with controlled response injection; real PostgreSQL workflow evidence reused. Shared representative actual-AT acceptance is pending (UX-ENV-001); the normal desktop/manual checklist is the approved execution path.

Source / existing evidence: frontend/app/admin/owner-verifications/[id]/page.tsx

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

Representative AT assignment: No separate reader run is required on this route. PASS covers the corrected route scope; overall UX acceptance still depends on all 11 shared representatives. See representative checklist.

Execution addendum (2026-09-21): Automated route checks pass. The domain-wide representative actual-AT gate is BLOCKED by UX-ENV-001; this does not require 35 duplicate reader runs. Browser accessibility-tree/keyboard checks are not actual screen-reader execution. Actual page/loader roles and states supersede generic matrices and Promise implementation labels. See UX execution report and route evidence. Historical generated wording is retained in evidence; no automatic continuation.

UX-009 — Page design, navigation and state QA: /admin/owner-verifications

Area: UX · **Priority:** P2 · **Test level:** MANUAL UX, LOCAL E2E, SECURITY · **Result:** PASS

Roles involved: ADMIN

Preconditions: Use the actual route role/auth contract. Verified email and current policies apply only to relevant marketplace mutations, not every page read or public recovery flow.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: 1. Open /admin/owner-verifications directly and from every header/dashboard link as ADMIN; substitute owned fixture UUID for [id]. 2. Repeat anonymous and wrong-role access. 3. At 375px, 768px and 1440px inspect all cards/forms/tables/gallery/dialogs/financial breakdowns. 4. Keyboard-tab entire page, activate primary/secondary actions, close dialogs, check returned focus; verify labels/errors/loading/table headings structurally; actual screen-reader acceptance uses the shared 11-flow matrix rather than repeating this route mechanically. 5. Delay API 3 seconds; return empty collection or missing detail; then exercise current states PENDING, ready. 6. Inject 400/401/403/404/409/500, offline, invalid JSON and valid JSON wrong shape into each page loader; restore and retry. 7. Use long UUID, max venue name/reason, Unicode and large money values; reload and navigate back. 8. Review each design item below and record screenshot/finding.

Expected UI behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. Visible focus, associated labels/errors, semantic headings/tables, meaningful button names, focus containment/return for dialogs, status not color-only, live loading/status announcement and contrast must be checked.

Expected API/backend behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. No private data flash before role resolution; errors contain no stack/provider output; independent failed panels do not hide valid sibling data. State/action visibility agrees with backend. Fail any uncovered case; static /admin home only loads auth/policy state, not its own result list.

Expected database/domain state: Read-only variants do not mutate; any invoked workflow must satisfy its domain scenario and endpoint matrix.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: Local route/state automation plus representative real screen-reader acceptance; see qa/UX_ASSISTIVE_TECHNOLOGY_CHECKLIST.md.

Local/staging requirement: Local browser with controlled response injection; real PostgreSQL workflow evidence reused. Shared representative actual-AT acceptance is pending (UX-ENV-001); the normal desktop/manual checklist is the approved execution path.

Source / existing evidence: frontend/app/admin/owner-verifications/page.tsx

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

Representative AT assignment: No separate reader run is required on this route. PASS covers the corrected route scope; overall UX acceptance still depends on all 11 shared representatives. See representative checklist.

Execution addendum (2026-09-21): Automated route checks pass. The domain-wide representative actual-AT gate is BLOCKED by UX-ENV-001; this does not require 35 duplicate reader runs. Browser accessibility-tree/keyboard checks are not actual screen-reader execution. Actual page/loader roles and states supersede generic matrices and Promise implementation labels. See UX execution report and route evidence. Historical generated wording is retained in evidence; no automatic continuation.

UX-010 — Page design, navigation and state QA: /admin

Area: UX · **Priority:** P2 · **Test level:** MANUAL UX, LOCAL E2E, SECURITY · **Result:** PASS

Roles involved: ADMIN

Preconditions: Use the actual route role/auth contract. Verified email and current policies apply only to relevant marketplace mutations, not every page read or public recovery flow.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: 1. Open /admin directly and from every header/dashboard link as ADMIN; substitute owned fixture UUID for [id]. 2. Repeat anonymous and wrong-role access. 3. At 375px, 768px and 1440px inspect all cards/forms/tables/gallery/dialogs/financial breakdowns. 4. Keyboard-tab entire page, activate primary/secondary actions, close dialogs, check returned focus; verify labels/errors/loading/table headings structurally; actual screen-reader acceptance uses the shared 11-flow matrix rather than repeating this route mechanically. 5. Delay API 3 seconds; return empty collection or missing detail; then exercise current states auth ready and role-gated content. 6. Inject 400/401/403/404/409/500, offline, invalid JSON and valid JSON wrong shape into each page loader; restore and retry. 7. Use long UUID, max venue name/reason, Unicode and large money values; reload and navigate back. 8. Review each design item below and record screenshot/finding.

Expected UI behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. Visible focus, associated labels/errors, semantic headings/tables, meaningful button names, focus containment/return for dialogs, status not color-only, live loading/status announcement and contrast must be checked.

Expected API/backend behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. No private data flash before role resolution; errors contain no stack/provider output; independent failed panels do not hide valid sibling data. State/action visibility agrees with backend. Fail any uncovered case; static /admin home only loads auth/policy state, not its own result list.

Expected database/domain state: Read-only variants do not mutate; any invoked workflow must satisfy its domain scenario and endpoint matrix.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: Local route/state automation plus representative real screen-reader acceptance; see qa/UX_ASSISTIVE_TECHNOLOGY_CHECKLIST.md.

Local/staging requirement: Local browser with controlled response injection; real PostgreSQL workflow evidence reused. Shared representative actual-AT acceptance is pending (UX-ENV-001); the normal desktop/manual checklist is the approved execution path.

Source / existing evidence: frontend/app/admin/page.tsx

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

Representative AT assignment: No separate reader run is required on this route. PASS covers the corrected route scope; overall UX acceptance still depends on all 11 shared representatives. See representative checklist.

Execution addendum (2026-09-21): Automated route checks pass. The domain-wide representative actual-AT gate is BLOCKED by UX-ENV-001; this does not require 35 duplicate reader runs. Browser accessibility-tree/keyboard checks are not actual screen-reader execution. Actual page/loader roles and states supersede generic matrices and Promise implementation labels. See UX execution report and route evidence. Historical generated wording is retained in evidence; no automatic continuation.

UX-011 — Page design, navigation and state QA: /admin/payout-destinations/[id]

Area: UX · **Priority:** P2 · **Test level:** MANUAL UX, LOCAL E2E, SECURITY · **Result:** BLOCKED

Roles involved: ADMIN

Preconditions: Use the actual route role/auth contract. Verified email and current policies apply only to relevant marketplace mutations, not every page read or public recovery flow.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: 1. Open /admin/payout-destinations/[id] directly and from every header/dashboard link as ADMIN; substitute owned fixture UUID for [id]. 2. Repeat anonymous and wrong-role access. 3. At 375px, 768px and 1440px inspect all cards/forms/tables/gallery/dialogs/financial breakdowns. 4. Keyboard-tab entire page, activate primary/secondary actions, close dialogs, check returned focus; verify labels/errors/loading/table headings structurally; actual screen-reader acceptance uses the shared 11-flow matrix rather than repeating this route mechanically. 5. Delay API 3 seconds; return empty collection or missing detail; then exercise current states PENDING_VERIFICATION, REJECTED, REPLACED, VERIFIED, ready. 6. Inject

400/401/403/404/409/500, offline, invalid JSON and valid JSON wrong shape into each page loader; restore and retry. 7. Use long UUID, max venue name/reason, Unicode and large money values; reload and navigate back. 8. Review each design item below and record screenshot/finding.

Expected UI behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. Visible focus, associated labels/errors, semantic headings/tables, meaningful button names, focus containment/return for dialogs, status not color-only, live loading/status announcement and contrast must be checked.

Expected API/backend behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. No private data flash before role resolution; errors contain no stack/provider output; independent failed panels do not hide valid sibling data. State/action visibility agrees with backend. Fail any uncovered case; static /admin home only loads auth/policy state, not its own result list.

Expected database/domain state: Read-only variants do not mutate; any invoked workflow must satisfy its domain scenario and endpoint matrix.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: Local route/state automation plus representative real screen-reader acceptance; see qa/UX_ASSISTIVE_TECHNOLOGY_CHECKLIST.md.

Local/staging requirement: Local browser with controlled response injection; real PostgreSQL workflow evidence reused. Shared representative actual-AT acceptance is pending (UX-ENV-001); the normal desktop/manual checklist is the approved execution path.

Source / existing evidence: frontend/app/admin/payout-destinations/[id]/page.tsx

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

Representative AT assignment: AT-R10 requires actual execution on this route; result remains BLOCKED. See representative checklist.

Execution addendum (2026-09-21): Automated route checks pass. The domain-wide representative actual-AT gate is BLOCKED by UX-ENV-001; this does not require 35 duplicate reader runs. Browser accessibility-tree/keyboard checks are not actual screen-reader execution. Actual page/loader roles and states supersede generic matrices and Promise implementation labels. See UX execution report and route evidence. Historical generated wording is retained in evidence; no automatic continuation.

UX-012 — Page design, navigation and state QA: /admin/payout-destinations

Area: UX · **Priority:** P2 · **Test level:** MANUAL UX, LOCAL E2E, SECURITY · **Result:** PASS

Roles involved: ADMIN

Preconditions: Use the actual route role/auth contract. Verified email and current policies apply only to relevant marketplace mutations, not every page read or public recovery flow.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: 1. Open /admin/payout-destinations directly and from every header/dashboard link as ADMIN; substitute owned fixture UUID for [id]. 2. Repeat anonymous and wrong-role access. 3. At 375px, 768px and 1440px inspect all cards/forms/tables/gallery/dialogs/financial breakdowns. 4. Keyboard-tab entire page, activate primary/secondary actions, close dialogs, check returned focus; verify labels/errors/loading/table headings structurally; actual screen-reader acceptance uses the shared 11-flow matrix rather than repeating this route mechanically. 5. Delay API 3 seconds; return empty collection or missing detail; then exercise current states PENDING_VERIFICATION, REJECTED, ready. 6. Inject 400/401/403/404/409/500, offline, invalid JSON and valid JSON wrong shape into each page loader; restore and retry. 7. Use long UUID, max venue name/reason, Unicode and large money values; reload and navigate back. 8. Review each design item below and record screenshot/finding.

Expected UI behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. Visible focus, associated labels/errors, semantic headings/tables, meaningful button names, focus containment/return for dialogs, status not color-only, live loading/status announcement and contrast must be checked.

Expected API/backend behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. No private data flash before role resolution; errors contain no stack/provider output; independent failed panels do not hide valid sibling data. State/action visibility agrees with backend. Fail any uncovered case; static /admin home only loads auth/policy state, not its own result list.

Expected database/domain state: Read-only variants do not mutate; any invoked workflow must satisfy its domain scenario and endpoint matrix.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: Local route/state automation plus representative real screen-reader acceptance; see qa/UX_ASSISTIVE_TECHNOLOGY_CHECKLIST.md.

Local/staging requirement: Local browser with controlled response injection; real PostgreSQL workflow evidence reused. Shared representative actual-AT acceptance is pending (UX-ENV-001); the normal desktop/manual checklist is the approved execution path.

Source / existing evidence: frontend/app/admin/payout-destinations/page.tsx

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

Representative AT assignment: No separate reader run is required on this route. PASS covers the corrected route scope; overall UX acceptance still depends on all 11 shared representatives. See representative checklist.

Execution addendum (2026-09-21): Automated route checks pass. The domain-wide representative actual-AT gate is BLOCKED by UX-ENV-001; this does not require 35 duplicate reader runs. Browser accessibility-tree/keyboard checks are not actual screen-reader execution. Actual page/loader roles and states supersede generic matrices and Promise implementation labels. See UX execution report and route evidence. Historical generated wording is retained in evidence; no automatic continuation.

UX-013 — Page design, navigation and state QA: /admin/payouts/[id]

Area: UX · **Priority:** P2 · **Test level:** MANUAL UX, LOCAL E2E, SECURITY · **Result:** PASS

Roles involved: ADMIN

Preconditions: Use the actual route role/auth contract. Verified email and current policies apply only to relevant marketplace mutations, not every page read or public recovery flow.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: 1. Open /admin/payouts/[id] directly and from every header/dashboard link as ADMIN; substitute owned fixture UUID for [id]. 2. Repeat anonymous and wrong-role access. 3. At 375px, 768px and 1440px inspect all cards/forms/tables/gallery/dialogs/financial breakdowns. 4. Keyboard-tab entire page, activate primary/secondary actions, close dialogs, check returned focus; verify labels/errors/loading/table headings structurally; actual screen-reader acceptance uses the shared 11-flow matrix rather than repeating this route mechanically. 5. Delay API 3 seconds; return empty collection or missing detail; then exercise current states HELD, READY, ready. 6. Inject 400/401/403/404/409/500, offline, invalid JSON and valid JSON wrong shape into each page loader; restore and retry. 7. Use long UUID, max venue name/reason, Unicode and large money values; reload and navigate back. 8. Review each design item below and record screenshot/finding.

Expected UI behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. Visible focus, associated labels/errors, semantic headings/tables, meaningful button names, focus containment/return for dialogs, status not color-only, live loading/status announcement and contrast must be checked.

Expected API/backend behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. No private data flash before role resolution; errors contain no stack/provider output; independent failed panels do not hide valid sibling data. State/action visibility agrees with backend. Fail any uncovered case; static /admin home only loads auth/policy state, not its own result list.

Expected database/domain state: Read-only variants do not mutate; any invoked workflow must satisfy its domain scenario and endpoint matrix.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: Local route/state automation plus representative real screen-reader acceptance; see qa/UX_ASSISTIVE_TECHNOLOGY_CHECKLIST.md.

Local/staging requirement: Local browser with controlled response injection; real PostgreSQL workflow evidence reused. Shared representative actual-AT acceptance is pending (UX-ENV-001); the normal desktop/manual checklist is the approved execution path.

Source / existing evidence: frontend/app/admin/payouts/[id]/page.tsx

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

Representative AT assignment: No separate reader run is required on this route. PASS covers the corrected route scope; overall UX acceptance still depends on all 11 shared representatives. See representative checklist.

Execution addendum (2026-09-21): Automated route checks pass. The domain-wide representative actual-AT gate is BLOCKED by UX-ENV-001; this does not require 35 duplicate reader runs. Browser accessibility-tree/keyboard checks are not actual screen-reader execution. Actual page/loader roles and states supersede generic matrices and Promise implementation labels. See UX execution report and route evidence. Historical generated wording is retained in evidence; no automatic continuation.

UX-014 — Page design, navigation and state QA: /admin/payouts

Area: UX · **Priority:** P2 · **Test level:** MANUAL UX, LOCAL E2E, SECURITY · **Result:** PASS

Roles involved: ADMIN

Preconditions: Use the actual route role/auth contract. Verified email and current policies apply only to relevant marketplace mutations, not every page read or public recovery flow.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: 1. Open /admin/payouts directly and from every header/dashboard link as ADMIN; substitute owned fixture UUID for [id]. 2. Repeat anonymous and wrong-role access. 3. At 375px, 768px and 1440px inspect all cards/forms/tables/gallery/dialogs/financial breakdowns. 4. Keyboard-tab entire page, activate primary/secondary actions, close dialogs, check returned focus; verify labels/errors/loading/table headings structurally; actual screen-reader acceptance uses the shared 11-flow matrix rather than repeating this route mechanically. 5. Delay API 3 seconds; return empty collection or missing detail; then exercise current states HELD, ready. 6. Inject 400/401/403/404/409/500, offline, invalid JSON and valid JSON wrong shape into each page loader; restore and retry. 7. Use long UUID, max venue name/reason, Unicode and large money values; reload and navigate back. 8. Review each design item below and record screenshot/finding.

Expected UI behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. Visible focus, associated labels/errors, semantic headings/tables, meaningful button names, focus containment/return for dialogs, status not color-only, live loading/status announcement and contrast must be checked.

Expected API/backend behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. No private data flash before role resolution; errors contain no stack/provider output; independent failed panels do not hide valid sibling data. State/action visibility agrees with backend. Fail any uncovered case; static /admin home only loads auth/policy state, not its own result list.

Expected database/domain state: Read-only variants do not mutate; any invoked workflow must satisfy its domain scenario and endpoint matrix.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: Local route/state automation plus representative real screen-reader acceptance; see qa/UX_ASSISTIVE_TECHNOLOGY_CHECKLIST.md.

Local/staging requirement: Local browser with controlled response injection; real PostgreSQL workflow evidence reused. Shared representative actual-AT acceptance is pending (UX-ENV-001); the normal desktop/manual checklist is the approved execution path.

Source / existing evidence: frontend/app/admin/payouts/page.tsx

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

Representative AT assignment: No separate reader run is required on this route. PASS covers the corrected route scope; overall UX acceptance still depends on all 11 shared representatives. See representative checklist.

Execution addendum (2026-09-21): Automated route checks pass. The domain-wide representative actual-AT gate is BLOCKED by UX-ENV-001; this does not require 35 duplicate reader runs. Browser accessibility-tree/keyboard checks are not actual screen-reader execution. Actual page/loader roles and states supersede generic matrices and Promise implementation labels. See UX execution report and route evidence. Historical generated wording is retained in evidence; no automatic continuation.

UX-015 — Page design, navigation and state QA: /admin/policies

Area: UX · **Priority:** P2 · **Test level:** MANUAL UX, LOCAL E2E, SECURITY · **Result:** PASS

Roles involved: ADMIN

Preconditions: Use the actual route role/auth contract. Verified email and current policies apply only to relevant marketplace mutations, not every page read or public recovery flow.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: 1. Open /admin/policies directly and from every header/dashboard link as ADMIN; substitute owned fixture UUID for [id]. 2. Repeat anonymous and wrong-role access. 3. At 375px, 768px and 1440px inspect all cards/forms/tables/gallery/dialogs/financial breakdowns. 4. Keyboard-tab entire page, activate primary/secondary actions, close dialogs, check returned focus; verify labels/errors/loading/table headings structurally; actual screen-reader acceptance uses the shared 11-flow matrix rather than repeating this route mechanically. 5. Delay API 3 seconds; return empty collection or missing detail; then exercise current states DRAFT, PUBLISHED, ready. 6. Inject 400/401/403/404/409/500, offline, invalid JSON and valid JSON wrong shape into each page loader; restore and retry. 7. Use long UUID, max venue name/reason, Unicode and large money values; reload and navigate back. 8. Review each design item below and record screenshot/finding.

Expected UI behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. Visible focus, associated labels/errors, semantic headings/tables, meaningful button names, focus containment/return for dialogs, status not color-only, live loading/status announcement and contrast must be checked.

Expected API/backend behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. No private data flash before role resolution; errors contain no stack/provider output; independent failed panels do not hide valid sibling data. State/action visibility agrees with backend. Fail any uncovered case; static /admin home only loads auth/policy state, not its own result list.

Expected database/domain state: Read-only variants do not mutate; any invoked workflow must satisfy its domain scenario and endpoint matrix.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: Local route/state automation plus representative real screen-reader acceptance; see qa/UX_ASSISTIVE_TECHNOLOGY_CHECKLIST.md.

Local/staging requirement: Local browser with controlled response injection; real PostgreSQL workflow evidence reused. Shared representative actual-AT acceptance is pending (UX-ENV-001); the normal desktop/manual checklist is the approved execution path.

Source / existing evidence: frontend/app/admin/policies/page.tsx

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

Representative AT assignment: No separate reader run is required on this route. PASS covers the corrected route scope; overall UX acceptance still depends on all 11 shared representatives. See representative checklist.

Execution addendum (2026-09-21): Automated route checks pass. The domain-wide representative actual-AT gate is BLOCKED by UX-ENV-001; this does not require 35 duplicate reader runs. Browser accessibility-tree/keyboard checks are not actual screen-reader execution. Actual page/loader roles and states supersede generic matrices and Promise implementation labels. See UX execution report and route evidence. Historical generated wording is retained in evidence; no automatic continuation.

UX-016 — Page design, navigation and state QA: /admin/venues/[id]

Area: UX · **Priority:** P2 · **Test level:** MANUAL UX, LOCAL E2E, SECURITY · **Result:** PASS

Roles involved: ADMIN

Preconditions: Use the actual route role/auth contract. Verified email and current policies apply only to relevant marketplace mutations, not every page read or public recovery flow.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: 1. Open /admin/venues/[id] directly and from every header/dashboard link as ADMIN; substitute owned fixture UUID for [id]. 2. Repeat anonymous and wrong-role access. 3. At 375px, 768px and 1440px inspect all cards/forms/tables/gallery/dialogs/financial breakdowns. 4. Keyboard-tab entire page, activate primary/secondary actions, close dialogs, check returned focus; verify labels/errors/loading/table headings structurally; actual screen-reader acceptance uses the shared 11-flow matrix rather than repeating this route mechanically. 5. Delay API 3 seconds; return empty collection or missing detail; then exercise current states ACTIVE, PENDING, SUSPENDED, ready. 6. Inject 400/401/403/404/409/500, offline, invalid JSON and valid JSON wrong shape into each page loader; restore and retry. 7. Use long UUID, max venue name/reason, Unicode and large money values; reload and navigate back. 8. Review each design item below and record screenshot/finding.

Expected UI behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. Visible focus, associated labels/errors, semantic headings/tables, meaningful button names, focus containment/return for dialogs, status not color-only, live loading/status announcement and contrast must be checked.

Expected API/backend behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. No private data flash before role resolution; errors contain no stack/provider output; independent failed panels do not hide valid sibling data. State/action visibility agrees with backend. Fail any uncovered case; static /admin home only loads auth/policy state, not its own result list.

Expected database/domain state: Read-only variants do not mutate; any invoked workflow must satisfy its domain scenario and endpoint matrix.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: Local route/state automation plus representative real screen-reader acceptance; see qa/UX_ASSISTIVE_TECHNOLOGY_CHECKLIST.md.

Local/staging requirement: Local browser with controlled response injection; real PostgreSQL workflow evidence reused. Shared representative actual-AT acceptance is pending (UX-ENV-001); the normal desktop/manual checklist is the approved execution path.

Source / existing evidence: frontend/app/admin/venues/[id]/page.tsx

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

Representative AT assignment: No separate reader run is required on this route. PASS covers the corrected route scope; overall UX acceptance still depends on all 11 shared representatives. See representative checklist.

Execution addendum (2026-09-21): Automated route checks pass. The domain-wide representative actual-AT gate is BLOCKED by UX-ENV-001; this does not require 35 duplicate reader runs. Browser accessibility-tree/keyboard checks are not actual screen-reader execution. Actual page/loader roles and states supersede generic matrices and Promise implementation labels. See UX execution report and route evidence. Historical generated wording is retained in evidence; no automatic continuation.

UX-017 — Page design, navigation and state QA: /admin/venues

Area: UX · **Priority:** P2 · **Test level:** MANUAL UX, LOCAL E2E, SECURITY · **Result:** PASS

Roles involved: ADMIN

Preconditions: Use the actual route role/auth contract. Verified email and current policies apply only to relevant marketplace mutations, not every page read or public recovery flow.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: 1. Open /admin/venues directly and from every header/dashboard link as ADMIN; substitute owned fixture UUID for [id]. 2. Repeat anonymous and wrong-role access. 3. At 375px, 768px and 1440px inspect all cards/forms/tables/gallery/dialogs/financial breakdowns. 4. Keyboard-tab entire page, activate primary/secondary actions, close dialogs, check returned focus; verify labels/errors/loading/table headings structurally; actual screen-reader acceptance uses the shared 11-flow matrix rather than repeating this route mechanically. 5. Delay API 3 seconds; return empty collection or missing detail; then exercise current states PENDING, SUSPENDED, ready. 6. Inject 400/401/403/404/409/500, offline, invalid JSON and valid JSON wrong shape into each page loader; restore and retry. 7. Use long UUID, max venue name/reason, Unicode and large money values; reload and navigate back. 8. Review each design item below and record screenshot/finding.

Expected UI behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. Visible focus, associated labels/errors, semantic headings/tables, meaningful button names, focus containment/return for dialogs, status not color-only, live loading/status announcement and contrast must be checked.

Expected API/backend behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. No private data flash before role resolution; errors contain no stack/provider output; independent failed panels do not hide valid sibling data. State/action visibility agrees with backend. Fail any uncovered case; static /admin home only loads auth/policy state, not its own result list.

Expected database/domain state: Read-only variants do not mutate; any invoked workflow must satisfy its domain scenario and endpoint matrix.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: Local route/state automation plus representative real screen-reader acceptance; see qa/UX_ASSISTIVE_TECHNOLOGY_CHECKLIST.md.

Local/staging requirement: Local browser with controlled response injection; real PostgreSQL workflow evidence reused. Shared representative actual-AT acceptance is pending (UX-ENV-001); the normal desktop/manual checklist is the approved execution path.

Source / existing evidence: frontend/app/admin/venues/page.tsx

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

Representative AT assignment: No separate reader run is required on this route. PASS covers the corrected route scope; overall UX acceptance still depends on all 11 shared representatives. See representative checklist.

Execution addendum (2026-09-21): Automated route checks pass. The domain-wide representative actual-AT gate is BLOCKED by UX-ENV-001; this does not require 35 duplicate reader runs. Browser accessibility-tree/keyboard checks are not actual screen-reader execution. Actual page/loader roles and states supersede generic matrices and Promise implementation labels. See UX execution report and route evidence. Historical generated wording is retained in evidence; no automatic continuation.

UX-018 — Page design, navigation and state QA: /booked

Area: UX · **Priority:** P2 · **Test level:** MANUAL UX, LOCAL E2E, SECURITY · **Result:** PASS

Roles involved: anonymous, SCOUT, OWNER, ADMIN as applicable

Preconditions: Use the actual route role/auth contract. Verified email and current policies apply only to relevant marketplace mutations, not every page read or public recovery flow.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: 1. Open /booked directly and from every header/dashboard link as anonymous, SCOUT, OWNER, ADMIN as applicable; substitute owned fixture UUID for [id]. 2. Repeat anonymous and wrong-role access. 3. At 375px, 768px and 1440px inspect all cards/forms/tables/gallery/dialogs/financial breakdowns. 4. Keyboard-tab entire page, activate primary/secondary actions, close dialogs, check returned focus; verify labels/errors/loading/table headings structurally; actual screen-reader acceptance uses the shared 11-flow matrix rather than repeating this route mechanically. 5. Delay API 3 seconds; return empty collection or missing detail; then exercise current states AWAITING_PAYMENT, auth, error, loading, missing, not-found, ready. 6. Inject 400/401/403/404/409/500, offline, invalid JSON and valid JSON wrong shape into each page loader; restore and retry. 7. Use long UUID, max venue name/reason, Unicode and large money values; reload and navigate back. 8. Review each design item below and record screenshot/finding.

Expected UI behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. Visible focus, associated labels/errors, semantic headings/tables, meaningful button names, focus containment/return for dialogs, status not color-only, live loading/status announcement and contrast must be checked.

Expected API/backend behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. No private data flash before role resolution; errors contain no stack/provider output; independent failed panels do not hide valid sibling data. State/action visibility agrees with backend. Fail any uncovered case; static /admin home only loads auth/policy state, not its own result list.

Expected database/domain state: Read-only variants do not mutate; any invoked workflow must satisfy its domain scenario and endpoint matrix.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: Local route/state automation plus representative real screen-reader acceptance; see qa/UX_ASSISTIVE_TECHNOLOGY_CHECKLIST.md.

Local/staging requirement: Local browser with controlled response injection; real PostgreSQL workflow evidence reused. Shared representative actual-AT acceptance is pending (UX-ENV-001); the normal desktop/manual checklist is the approved execution path.

Source / existing evidence: frontend/app/booked/page.tsx

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

Representative AT assignment: No separate reader run is required on this route. PASS covers the corrected route scope; overall UX acceptance still depends on all 11 shared representatives. See representative checklist.

Execution addendum (2026-09-21): Automated route checks pass. The domain-wide representative actual-AT gate is BLOCKED by UX-ENV-001; this does not require 35 duplicate reader runs. Browser accessibility-tree/keyboard checks are not actual screen-reader execution. Actual page/loader roles and states supersede generic matrices and Promise implementation labels. See UX execution report and route evidence. Historical generated wording is retained in evidence; no automatic continuation.

UX-019 — Page design, navigation and state QA: /bookings/[id]

Area: UX · **Priority:** P2 · **Test level:** MANUAL UX, LOCAL E2E, SECURITY · **Result:** BLOCKED

Roles involved: anonymous, SCOUT, OWNER, ADMIN as applicable

Preconditions: Use the actual route role/auth contract. Verified email and current policies apply only to relevant marketplace mutations, not every page read or public recovery flow.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: 1. Open /bookings/[id] directly and from every header/dashboard link as anonymous, SCOUT, OWNER, ADMIN as applicable; substitute owned fixture UUID for [id]. 2. Repeat anonymous and wrong-role access. 3. At 375px, 768px and 1440px inspect all cards/forms/tables/gallery/dialogs/financial breakdowns. 4. Keyboard-tab entire page, activate primary/secondary actions, close dialogs, check returned focus; verify labels/errors/loading/table headings structurally; actual screen-reader acceptance uses the shared 11-flow matrix rather than repeating this route mechanically. 5. Delay API 3 seconds; return empty collection or missing detail; then exercise current states AWAITING_PAYMENT, CANCELLED, CONFIRMED, DECLINED, EXPIRED, REFUNDED, REQUESTED, UNAVAILABLE, error, expired, forbidden, idle, loading, not-found, ready. 6. Inject 400/401/403/404/409/500, offline, invalid JSON and valid

JSON wrong shape into each page loader; restore and retry. 7. Use long UUID, max venue name/reason, Unicode and large money values; reload and navigate back. 8. Review each design item below and record screenshot/finding.

Expected UI behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. Visible focus, associated labels/errors, semantic headings/tables, meaningful button names, focus containment/return for dialogs, status not color-only, live loading/status announcement and contrast must be checked.

Expected API/backend behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. No private data flash before role resolution; errors contain no stack/provider output; independent failed panels do not hide valid sibling data. State/action visibility agrees with backend. Fail any uncovered case; static /admin home only loads auth/policy state, not its own result list.

Expected database/domain state: Read-only variants do not mutate; any invoked workflow must satisfy its domain scenario and endpoint matrix.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: Local route/state automation plus representative real screen-reader acceptance; see qa/UX_ASSISTIVE_TECHNOLOGY_CHECKLIST.md.

Local/staging requirement: Local browser with controlled response injection; real PostgreSQL workflow evidence reused. Shared representative actual-AT acceptance is pending (UX-ENV-001); the normal desktop/manual checklist is the approved execution path.

Source / existing evidence: frontend/app/bookings/[id]/page.tsx

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

Representative AT assignment: AT-R03 requires actual execution on this route; result remains BLOCKED. See representative checklist.

Execution addendum (2026-09-21): Automated route checks pass. The domain-wide representative actual-AT gate is BLOCKED by UX-ENV-001; this does not require 35 duplicate reader runs. Browser accessibility-tree/keyboard checks are not actual screen-reader execution. Actual page/loader roles and states supersede generic matrices and Promise implementation labels. See UX execution report and route evidence. Historical generated wording is retained in evidence; no automatic continuation.

UX-020 — Page design, navigation and state QA: /forgot-password

Area: UX · **Priority:** P2 · **Test level:** MANUAL UX, LOCAL E2E, SECURITY · **Result:** BLOCKED

Roles involved: anonymous, SCOUT, OWNER, ADMIN as applicable

Preconditions: Use the actual route role/auth contract. Verified email and current policies apply only to relevant marketplace mutations, not every page read or public recovery flow.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: 1. Open /forgot-password directly and from every header/dashboard link as anonymous, SCOUT, OWNER, ADMIN as applicable; substitute owned fixture UUID for [id]. 2. Repeat anonymous and wrong-role access. 3. At 375px, 768px and 1440px inspect all cards/forms/tables/gallery/dialogs/financial breakdowns. 4. Keyboard-tab entire page, activate primary/secondary actions, close dialogs, check returned focus; verify labels/errors/loading/table headings structurally; actual screen-reader acceptance uses the shared 11-flow matrix rather than repeating this route mechanically. 5. Delay API 3 seconds; return empty collection or missing detail; then exercise current states auth ready and role-gated content. 6. Inject 400/401/403/404/409/500, offline, invalid JSON and valid JSON wrong shape into each page loader; restore and retry. 7. Use long UUID, max venue name/reason, Unicode and large money values; reload and navigate back. 8. Review each design item below and record screenshot/finding.

Expected UI behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. Visible focus, associated labels/errors, semantic headings/tables, meaningful button names, focus containment/return for dialogs, status not color-only, live loading/status announcement and contrast must be checked.

Expected API/backend behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. No private data flash before role resolution; errors contain no stack/provider output; independent failed panels do not hide valid sibling data. State/action visibility agrees with backend. Fail any uncovered case; static /admin home only loads auth/policy state, not its own result list.

Expected database/domain state: Read-only variants do not mutate; any invoked workflow must satisfy its domain scenario and endpoint matrix.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: Local route/state automation plus representative real screen-reader acceptance; see qa/UX_ASSISTIVE_TECHNOLOGY_CHECKLIST.md.

Local/staging requirement: Local browser with controlled response injection; real PostgreSQL workflow evidence reused. Shared representative actual-AT acceptance is pending (UX-ENV-001); the normal desktop/manual checklist is the approved execution path.

Source / existing evidence: frontend/app/forgot-password/page.tsx

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

Representative AT assignment: AT-R01 requires actual execution on this route; result remains BLOCKED. See representative checklist.

Execution addendum (2026-09-21): Automated route checks pass. The domain-wide representative actual-AT gate is BLOCKED by UX-ENV-001; this does not require 35 duplicate reader runs. Browser accessibility-tree/keyboard checks are not actual screen-reader execution. Actual page/loader roles and states supersede generic matrices and Promise implementation labels. See UX execution report and route evidence. Historical generated wording is retained in evidence; no automatic continuation.

UX-021 — Page design, navigation and state QA: /owner/bookings

Area: UX · **Priority:** P2 · **Test level:** MANUAL UX, LOCAL E2E, SECURITY · **Result:** BLOCKED

Roles involved: OWNER

Preconditions: Use the actual route role/auth contract. Verified email and current policies apply only to relevant marketplace mutations, not every page read or public recovery flow.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: 1. Open /owner/bookings directly and from every header/dashboard link as OWNER; substitute owned fixture UUID for [id]. 2. Repeat anonymous and wrong-role access. 3. At 375px, 768px and 1440px inspect all cards/forms/tables/gallery/dialogs/financial breakdowns. 4. Keyboard-tab entire page, activate primary/secondary actions, close dialogs, check returned focus; verify labels/errors/loading/table headings structurally; actual screen-reader acceptance uses the shared 11-flow matrix rather than repeating this route mechanically. 5. Delay API 3 seconds; return empty collection or missing detail; then exercise current states REQUESTED, error, fulfilled, loading, rejected. 6. Inject 400/401/403/404/409/500, offline, invalid JSON and valid JSON wrong shape into each page loader; restore and retry. 7. Use long UUID, max venue name/reason, Unicode and large money values; reload and navigate back. 8. Review each design item below and record screenshot/finding.

Expected UI behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. Visible focus, associated labels/errors, semantic headings/tables, meaningful button names, focus containment/return for dialogs, status not color-only, live loading/status announcement and contrast must be checked.

Expected API/backend behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. No private data flash before role resolution; errors contain no stack/provider output; independent failed panels do not hide valid sibling data. State/action visibility agrees with backend. Fail any uncovered case; static /admin home only loads auth/policy state, not its own result list.

Expected database/domain state: Read-only variants do not mutate; any invoked workflow must satisfy its domain scenario and endpoint matrix.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: Local route/state automation plus representative real screen-reader acceptance; see qa/UX_ASSISTIVE_TECHNOLOGY_CHECKLIST.md.

Local/staging requirement: Local browser with controlled response injection; real PostgreSQL workflow evidence reused. Shared representative actual-AT acceptance is pending (UX-ENV-001); the normal desktop/manual checklist is the approved execution path.

Source / existing evidence: frontend/app/owner/bookings/page.tsx

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

Representative AT assignment: AT-R05 requires actual execution on this route; result remains BLOCKED. See representative checklist.

Execution addendum (2026-09-21): Automated route checks pass. The domain-wide representative actual-AT gate is BLOCKED by UX-ENV-001; this does not require 35 duplicate reader runs. Browser accessibility-tree/keyboard checks are not actual screen-reader execution. Actual page/loader roles and states supersede generic matrices and Promise implementation labels. See UX execution report and route evidence. Historical generated wording is retained in evidence; no automatic continuation.

UX-022 — Page design, navigation and state QA: /owner/inquiries

Area: UX · **Priority:** P2 · **Test level:** MANUAL UX, LOCAL E2E, SECURITY · **Result:** PASS

Roles involved: OWNER

Preconditions: Use the actual route role/auth contract. Verified email and current policies apply only to relevant marketplace mutations, not every page read or public recovery flow.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: 1. Open /owner/inquiries directly and from every header/dashboard link as OWNER; substitute owned fixture UUID for [id]. 2. Repeat anonymous and wrong-role access. 3. At 375px, 768px and 1440px inspect all cards/forms/tables/gallery/dialogs/financial breakdowns. 4. Keyboard-tab entire page, activate primary/secondary actions, close dialogs, check returned focus; verify labels/errors/loading/table headings structurally; actual screen-reader acceptance uses the shared 11-flow matrix rather than repeating this route mechanically. 5. Delay API 3 seconds; return empty collection or missing detail; then exercise current states OPEN, error, loading. 6. Inject 400/401/403/404/409/500, offline, invalid JSON and valid JSON wrong shape into each page loader; restore and retry. 7. Use long UUID, max venue name/reason, Unicode and large money values; reload and navigate back. 8. Review each design item below and record screenshot/finding.

Expected UI behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. Visible focus, associated labels/errors, semantic headings/tables, meaningful button names, focus containment/return for dialogs, status not color-only, live loading/status announcement and contrast must be checked.

Expected API/backend behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. No private data flash before role resolution; errors contain no stack/provider output; independent failed panels do not hide valid sibling data. State/action visibility agrees with backend. Fail any uncovered case; static /admin home only loads auth/policy state, not its own result list.

Expected database/domain state: Read-only variants do not mutate; any invoked workflow must satisfy its domain scenario and endpoint matrix.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: Local route/state automation plus representative real screen-reader acceptance; see qa/UX_ASSISTIVE_TECHNOLOGY_CHECKLIST.md.

Local/staging requirement: Local browser with controlled response injection; real PostgreSQL workflow evidence reused. Shared representative actual-AT acceptance is pending (UX-ENV-001); the normal desktop/manual checklist is the approved execution path.

Source / existing evidence: frontend/app/owner/inquiries/page.tsx

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

Representative AT assignment: No separate reader run is required on this route. PASS covers the corrected route scope; overall UX acceptance still depends on all 11 shared representatives. See representative checklist.

Execution addendum (2026-09-21): Automated route checks pass. The domain-wide representative actual-AT gate is BLOCKED by UX-ENV-001; this does not require 35 duplicate reader runs. Browser accessibility-tree/keyboard checks are not actual screen-reader execution. Actual page/loader roles and states supersede generic matrices and Promise implementation labels. See UX execution report and route evidence. Historical generated wording is retained in evidence; no automatic continuation.

UX-023 — Page design, navigation and state QA: /owner/payout-destination

Area: UX · **Priority:** P2 · **Test level:** MANUAL UX, LOCAL E2E, SECURITY · **Result:** BLOCKED

Roles involved: OWNER

Preconditions: Use the actual route role/auth contract. Verified email and current policies apply only to relevant marketplace mutations, not every page read or public recovery flow.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: 1. Open /owner/payout-destination directly and from every header/dashboard link as OWNER; substitute owned fixture UUID for [id]. 2. Repeat anonymous and wrong-role access. 3. At 375px, 768px and 1440px inspect all cards/forms/tables/gallery/dialogs/financial breakdowns. 4. Keyboard-tab entire page, activate primary/secondary actions, close dialogs, check returned focus; verify labels/errors/loading/table headings structurally; actual screen-reader acceptance uses the shared 11-flow matrix rather than repeating this route mechanically. 5. Delay API 3 seconds; return empty collection or missing detail; then exercise current states PENDING_VERIFICATION, REJECTED, REPLACED, VERIFIED, error, loading. 6. Inject 400/401/403/404/409/500, offline, invalid JSON and valid JSON wrong shape into each page loader; restore and retry. 7. Use long UUID, max venue name/reason, Unicode and large money values; reload and navigate back. 8. Review each design item below and record screenshot/finding.

Expected UI behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. Visible focus, associated labels/errors, semantic headings/tables, meaningful button names, focus containment/return for dialogs, status not color-only, live loading/status announcement and contrast must be checked.

Expected API/backend behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. No private data flash before role resolution; errors contain no stack/provider output; independent failed panels do not hide valid sibling data. State/action visibility agrees with backend. Fail any uncovered case; static /admin home only loads auth/policy state, not its own result list.

Expected database/domain state: Read-only variants do not mutate; any invoked workflow must satisfy its domain scenario and endpoint matrix.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: Local route/state automation plus representative real screen-reader acceptance; see qa/UX_ASSISTIVE_TECHNOLOGY_CHECKLIST.md.

Local/staging requirement: Local browser with controlled response injection; real PostgreSQL workflow evidence reused. Shared representative actual-AT acceptance is pending (UX-ENV-001); the normal desktop/manual checklist is the approved execution path.

Source / existing evidence: frontend/app/owner/payout-destination/page.tsx

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

Representative AT assignment: AT-R07 requires actual execution on this route; result remains BLOCKED. See representative checklist.

Execution addendum (2026-09-21): Automated route checks pass. The domain-wide representative actual-AT gate is BLOCKED by UX-ENV-001; this does not require 35 duplicate reader runs. Browser accessibility-tree/keyboard checks are not actual screen-reader execution. Actual page/loader roles and states supersede generic matrices and Promise implementation labels. See UX execution report and route evidence. Historical generated wording is retained in evidence; no automatic continuation.

UX-024 — Page design, navigation and state QA: /owner/payouts/[id]

Area: UX · **Priority:** P2 · **Test level:** MANUAL UX, LOCAL E2E, SECURITY · **Result:** PASS

Roles involved: OWNER

Preconditions: Use the actual route role/auth contract. Verified email and current policies apply only to relevant marketplace mutations, not every page read or public recovery flow.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: 1. Open /owner/payouts/[id] directly and from every header/dashboard link as OWNER; substitute owned fixture UUID for [id]. 2. Repeat anonymous and wrong-role access. 3. At 375px, 768px and 1440px inspect all cards/forms/tables/gallery/dialogs/financial breakdowns. 4. Keyboard-tab entire page, activate primary/secondary actions, close dialogs, check returned focus; verify labels/errors/loading/table headings structurally; actual screen-reader acceptance uses the shared 11-flow matrix rather than repeating this route mechanically. 5. Delay API 3 seconds; return empty collection or missing detail; then exercise current states PAID, fulfilled, ready, rejected. 6. Inject 400/401/403/404/409/500, offline, invalid JSON and valid JSON wrong shape into each page loader; restore and retry. 7. Use long UUID, max venue name/reason, Unicode and large money values; reload and navigate back. 8. Review each design item below and record screenshot/finding.

Expected UI behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. Visible focus, associated labels/errors, semantic headings/tables, meaningful button names, focus containment/return for dialogs, status not color-only, live loading/status announcement and contrast must be checked.

Expected API/backend behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. No private data flash before role resolution; errors contain no stack/provider output; independent failed panels do not hide valid sibling data. State/action visibility agrees with backend. Fail any uncovered case; static /admin home only loads auth/policy state, not its own result list.

Expected database/domain state: Read-only variants do not mutate; any invoked workflow must satisfy its domain scenario and endpoint matrix.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: Local route/state automation plus representative real screen-reader acceptance; see qa/UX_ASSISTIVE_TECHNOLOGY_CHECKLIST.md.

Local/staging requirement: Local browser with controlled response injection; real PostgreSQL workflow evidence reused. Shared representative actual-AT acceptance is pending (UX-ENV-001); the normal desktop/manual checklist is the approved execution path.

Source / existing evidence: frontend/app/owner/payouts/[id]/page.tsx

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

Representative AT assignment: No separate reader run is required on this route. PASS covers the corrected route scope; overall UX acceptance still depends on all 11 shared representatives. See representative checklist.

Execution addendum (2026-09-21): Automated route checks pass. The domain-wide representative actual-AT gate is BLOCKED by UX-ENV-001; this does not require 35 duplicate reader runs. Browser accessibility-tree/keyboard checks are not actual screen-reader execution. Actual page/loader roles and states supersede generic matrices and Promise implementation labels. See UX execution report and route evidence. Historical generated wording is retained in evidence; no automatic continuation.

UX-025 — Page design, navigation and state QA: /owner/payouts

Area: UX · **Priority:** P2 · **Test level:** MANUAL UX, LOCAL E2E, SECURITY · **Result:** PASS

Roles involved: OWNER

Preconditions: Use the actual route role/auth contract. Verified email and current policies apply only to relevant marketplace mutations, not every page read or public recovery flow.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: 1. Open /owner/payouts directly and from every header/dashboard link as OWNER; substitute owned fixture UUID for [id]. 2. Repeat anonymous and wrong-role access. 3. At 375px, 768px and 1440px inspect all cards/forms/tables/gallery/dialogs/financial breakdowns. 4. Keyboard-tab entire page, activate primary/secondary actions, close dialogs, check returned focus; verify labels/errors/loading/table headings structurally; actual screen-reader acceptance uses the shared 11-flow matrix rather than repeating this route mechanically. 5. Delay API 3 seconds; return empty collection or missing detail; then exercise current states error, fulfilled, loading, rejected. 6. Inject 400/401/403/404/409/500, offline, invalid JSON and valid JSON wrong shape into each page loader; restore and retry. 7. Use long UUID, max venue name/reason, Unicode and large money values; reload and navigate back. 8. Review each design item below and record screenshot/finding.

Expected UI behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. Visible focus, associated labels/errors, semantic headings/tables, meaningful button names, focus containment/return for dialogs, status not color-only, live loading/status announcement and contrast must be checked.

Expected API/backend behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. No private data flash before role resolution; errors contain no stack/provider output; independent failed panels do not hide valid sibling data. State/action visibility agrees with backend. Fail any uncovered case; static /admin home only loads auth/policy state, not its own result list.

Expected database/domain state: Read-only variants do not mutate; any invoked workflow must satisfy its domain scenario and endpoint matrix.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: Local route/state automation plus representative real screen-reader acceptance; see qa/UX_ASSISTIVE_TECHNOLOGY_CHECKLIST.md.

Local/staging requirement: Local browser with controlled response injection; real PostgreSQL workflow evidence reused. Shared representative actual-AT acceptance is pending (UX-ENV-001); the normal desktop/manual checklist is the approved execution path.

Source / existing evidence: frontend/app/owner/payouts/page.tsx

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

Representative AT assignment: No separate reader run is required on this route. PASS covers the corrected route scope; overall UX acceptance still depends on all 11 shared representatives. See representative checklist.

Execution addendum (2026-09-21): Automated route checks pass. The domain-wide representative actual-AT gate is BLOCKED by UX-ENV-001; this does not require 35 duplicate reader runs. Browser accessibility-tree/keyboard checks are not actual screen-reader execution. Actual page/loader roles and states supersede generic matrices and Promise implementation labels. See UX execution report and route evidence. Historical generated wording is retained in evidence; no automatic continuation.

UX-026 — Page design, navigation and state QA: /owner/venues/[id]

Area: UX · **Priority:** P2 · **Test level:** MANUAL UX, LOCAL E2E, SECURITY · **Result:** BLOCKED

Roles involved: OWNER

Preconditions: Use the actual route role/auth contract. Verified email and current policies apply only to relevant marketplace mutations, not every page read or public recovery flow.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: 1. Open /owner/venues/[id] directly and from every header/dashboard link as OWNER; substitute owned fixture UUID for [id]. 2. Repeat anonymous and wrong-role access. 3. At 375px, 768px and 1440px inspect all cards/forms/tables/gallery/dialogs/financial breakdowns. 4. Keyboard-tab entire page, activate primary/secondary actions, close dialogs, check returned focus; verify labels/errors/loading/table headings structurally; actual screen-reader acceptance uses the shared 11-flow matrix rather than repeating this route mechanically. 5. Delay API 3 seconds; return empty collection or missing detail; then exercise current states ACTIVE, PENDING, error, forbidden, loading, not-found. 6. Inject 400/401/403/404/409/500, offline, invalid JSON and valid JSON wrong shape into each page loader; restore and retry. 7. Use long UUID, max venue name/reason, Unicode and large money values; reload and navigate back. 8. Review each design item below and record screenshot/finding.

Expected UI behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. Visible focus, associated labels/errors, semantic headings/tables, meaningful button names, focus containment/return for dialogs, status not color-only, live loading/status announcement and contrast must be checked.

Expected API/backend behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. No private data flash before role resolution; errors contain no stack/provider output; independent failed panels do not hide valid sibling data. State/action visibility agrees with backend. Fail any uncovered case; static /admin home only loads auth/policy state, not its own result list.

Expected database/domain state: Read-only variants do not mutate; any invoked workflow must satisfy its domain scenario and endpoint matrix.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: Local route/state automation plus representative real screen-reader acceptance; see qa/UX_ASSISTIVE_TECHNOLOGY_CHECKLIST.md.

Local/staging requirement: Local browser with controlled response injection; real PostgreSQL workflow evidence reused. Shared representative actual-AT acceptance is pending (UX-ENV-001); the normal desktop/manual checklist is the approved execution path.

Source / existing evidence: frontend/app/owner/venues/[id]/page.tsx

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

Representative AT assignment: AT-R06 requires actual execution on this route; result remains BLOCKED. See representative checklist.

Execution addendum (2026-09-21): Automated route checks pass. The domain-wide representative actual-AT gate is BLOCKED by UX-ENV-001; this does not require 35 duplicate reader runs. Browser accessibility-tree/keyboard checks are not actual screen-reader execution. Actual page/loader roles and states supersede generic matrices and Promise implementation labels. See UX execution report and route evidence. Historical generated wording is retained in evidence; no automatic continuation.

UX-027 — Page design, navigation and state QA: /owner/venues/new

Area: UX · **Priority:** P2 · **Test level:** MANUAL UX, LOCAL E2E, SECURITY · **Result:** PASS

Roles involved: OWNER

Preconditions: Use the actual route role/auth contract. Verified email and current policies apply only to relevant marketplace mutations, not every page read or public recovery flow.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: 1. Open /owner/venues/new directly and from every header/dashboard link as OWNER; substitute owned fixture UUID for [id]. 2. Repeat anonymous and wrong-role access. 3. At 375px, 768px and 1440px inspect all cards/forms/tables/gallery/dialogs/financial breakdowns. 4. Keyboard-tab entire page, activate primary/secondary actions, close dialogs, check returned focus; verify labels/errors/loading/table headings structurally; actual screen-reader acceptance uses the shared 11-flow matrix rather than repeating this route mechanically. 5. Delay API 3 seconds; return empty collection or missing detail; then exercise current states auth ready and role-gated content. 6. Inject 400/401/403/404/409/500, offline, invalid JSON and valid JSON wrong shape into each page loader; restore and retry. 7. Use long UUID, max venue name/reason, Unicode and large money values; reload and navigate back. 8. Review each design item below and record screenshot/finding.

Expected UI behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. Visible focus, associated labels/errors, semantic headings/tables, meaningful button names, focus containment/return for dialogs, status not color-only, live loading/status announcement and contrast must be checked.

Expected API/backend behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. No private data flash before role resolution; errors contain no stack/provider output; independent failed panels do not hide valid sibling data. State/action visibility agrees with backend. Fail any uncovered case; static /admin home only loads auth/policy state, not its own result list.

Expected database/domain state: Read-only variants do not mutate; any invoked workflow must satisfy its domain scenario and endpoint matrix.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: Local route/state automation plus representative real screen-reader acceptance; see qa/UX_ASSISTIVE_TECHNOLOGY_CHECKLIST.md.

Local/staging requirement: Local browser with controlled response injection; real PostgreSQL workflow evidence reused. Shared representative actual-AT acceptance is pending (UX-ENV-001); the normal desktop/manual checklist is the approved execution path.

Source / existing evidence: frontend/app/owner/venues/new/page.tsx

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

Representative AT assignment: No separate reader run is required on this route. PASS covers the corrected route scope; overall UX acceptance still depends on all 11 shared representatives. See representative checklist.

Execution addendum (2026-09-21): Automated route checks pass. The domain-wide representative actual-AT gate is BLOCKED by UX-ENV-001; this does not require 35 duplicate reader runs. Browser accessibility-tree/keyboard checks are not actual screen-reader execution. Actual page/loader roles and states supersede generic matrices and Promise implementation labels. See UX execution report and route evidence. Historical generated wording is retained in evidence; no automatic continuation.

UX-028 — Page design, navigation and state QA: /owner/venues

Area: UX · **Priority:** P2 · **Test level:** MANUAL UX, LOCAL E2E, SECURITY · **Result:** PASS

Roles involved: OWNER

Preconditions: Use the actual route role/auth contract. Verified email and current policies apply only to relevant marketplace mutations, not every page read or public recovery flow.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: 1. Open /owner/venues directly and from every header/dashboard link as OWNER; substitute owned fixture UUID for [id]. 2. Repeat anonymous and wrong-role access. 3. At 375px, 768px and 1440px inspect all cards/forms/tables/gallery/dialogs/financial breakdowns. 4. Keyboard-tab entire page, activate primary/secondary actions, close dialogs, check returned focus; verify labels/errors/loading/table headings structurally; actual screen-reader acceptance uses the shared 11-flow matrix rather than repeating this route mechanically. 5. Delay API 3 seconds; return empty collection or missing detail; then exercise current states error, loading. 6. Inject 400/401/403/404/409/500, offline, invalid JSON and valid JSON wrong shape into each page loader; restore and retry. 7. Use long UUID, max venue name/reason, Unicode and large money values; reload and navigate back. 8. Review each design item below and record screenshot/finding.

Expected UI behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. Visible focus, associated labels/errors, semantic headings/tables, meaningful button names, focus containment/return for dialogs, status not color-only, live loading/status announcement and contrast must be checked.

Expected API/backend behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. No private data flash before role resolution; errors contain no stack/provider output; independent failed panels do not hide valid sibling data. State/action visibility agrees with backend. Fail any uncovered case; static /admin home only loads auth/policy state, not its own result list.

Expected database/domain state: Read-only variants do not mutate; any invoked workflow must satisfy its domain scenario and endpoint matrix.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: Local route/state automation plus representative real screen-reader acceptance; see qa/UX_ASSISTIVE_TECHNOLOGY_CHECKLIST.md.

Local/staging requirement: Local browser with controlled response injection; real PostgreSQL workflow evidence reused. Shared representative actual-AT acceptance is pending (UX-ENV-001); the normal desktop/manual checklist is the approved execution path.

Source / existing evidence: frontend/app/owner/venues/page.tsx

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

Representative AT assignment: No separate reader run is required on this route. PASS covers the corrected route scope; overall UX acceptance still depends on all 11 shared representatives. See representative checklist.

Execution addendum (2026-09-21): Automated route checks pass. The domain-wide representative actual-AT gate is BLOCKED by UX-ENV-001; this does not require 35 duplicate reader runs. Browser accessibility-tree/keyboard checks are not actual screen-reader execution. Actual page/loader roles and states supersede generic matrices and Promise implementation labels. See UX execution report and route evidence. Historical generated wording is retained in evidence; no automatic continuation.

UX-029 — Page design, navigation and state QA: /owner/verification

Area: UX · **Priority:** P2 · **Test level:** MANUAL UX, LOCAL E2E, SECURITY · **Result:** PASS

Roles involved: OWNER

Preconditions: Use the actual route role/auth contract. Verified email and current policies apply only to relevant marketplace mutations, not every page read or public recovery flow.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: 1. Open /owner/verification directly and from every header/dashboard link as OWNER; substitute owned fixture UUID for [id]. 2. Repeat anonymous and wrong-role access. 3. At 375px, 768px and 1440px inspect all cards/forms/tables/gallery/dialogs/financial breakdowns. 4. Keyboard-tab entire page, activate primary/secondary actions, close dialogs, check returned focus; verify labels/errors/loading/table headings structurally; actual screen-reader acceptance uses the shared 11-flow matrix rather than repeating this route mechanically. 5. Delay API 3 seconds; return empty collection or missing detail; then exercise current states REJECTED, UNVERIFIED, error, loading. 6. Inject 400/401/403/404/409/500, offline, invalid JSON and valid JSON wrong shape into each page loader; restore and retry. 7. Use long UUID, max venue name/reason, Unicode and large money values; reload and navigate back. 8. Review each design item below and record screenshot/finding.

Expected UI behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. Visible focus, associated labels/errors, semantic headings/tables, meaningful button names, focus containment/return for dialogs, status not color-only, live loading/status announcement and contrast must be checked.

Expected API/backend behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. No private data flash before role resolution; errors contain no stack/provider output; independent failed panels do not hide valid sibling data. State/action visibility agrees with backend. Fail any uncovered case; static /admin home only loads auth/policy state, not its own result list.

Expected database/domain state: Read-only variants do not mutate; any invoked workflow must satisfy its domain scenario and endpoint matrix.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: Local route/state automation plus representative real screen-reader acceptance; see qa/UX_ASSISTIVE_TECHNOLOGY_CHECKLIST.md.

Local/staging requirement: Local browser with controlled response injection; real PostgreSQL workflow evidence reused. Shared representative actual-AT acceptance is pending (UX-ENV-001); the normal desktop/manual checklist is the approved execution path.

Source / existing evidence: frontend/app/owner/verification/page.tsx

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

Representative AT assignment: No separate reader run is required on this route. PASS covers the corrected route scope; overall UX acceptance still depends on all 11 shared representatives. See representative checklist.

Execution addendum (2026-09-21): Automated route checks pass. The domain-wide representative actual-AT gate is BLOCKED by UX-ENV-001; this does not require 35 duplicate reader runs. Browser accessibility-tree/keyboard checks are not actual screen-reader execution. Actual page/loader roles and states supersede generic matrices and Promise implementation labels. See UX execution report and route evidence. Historical generated wording is retained in evidence; no automatic continuation.

UX-030 — Page design, navigation and state QA: /

Area: UX · **Priority:** P2 · **Test level:** MANUAL UX, LOCAL E2E, SECURITY · **Result:** PASS

Roles involved: anonymous, SCOUT, OWNER, ADMIN as applicable

Preconditions: Use the actual route role/auth contract. Verified email and current policies apply only to relevant marketplace mutations, not every page read or public recovery flow.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: 1. Open / directly and from every header/dashboard link as anonymous, SCOUT, OWNER, ADMIN as applicable; substitute owned fixture UUID for [id]. 2. Repeat anonymous and wrong-role access. 3. At 375px, 768px and 1440px inspect all cards/forms/tables/gallery/dialogs/financial breakdowns. 4. Keyboard-tab entire page, activate primary/secondary actions, close dialogs, check returned focus; verify labels/errors/loading/table headings structurally; actual screen-reader acceptance uses the shared 11-flow matrix rather than repeating this route mechanically. 5. Delay API 3 seconds; return empty collection or missing detail; then exercise current states auth ready and role-gated content. 6. Inject 400/401/403/404/409/500, offline, invalid JSON and valid JSON wrong shape into each page loader; restore and retry. 7. Use long UUID, max venue name/reason, Unicode and large money values; reload and navigate back. 8. Review each design item below and record screenshot/finding.

Expected UI behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. Visible focus, associated labels/errors, semantic headings/tables, meaningful button names, focus containment/return for dialogs, status not color-only, live loading/status announcement and contrast must be checked.

Expected API/backend behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. No private data flash before role resolution; errors contain no stack/provider output; independent failed panels do not hide valid sibling data. State/action visibility agrees with backend. Fail any uncovered case; static /admin home only loads auth/policy state, not its own result list.

Expected database/domain state: Read-only variants do not mutate; any invoked workflow must satisfy its domain scenario and endpoint matrix.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: Local route/state automation plus representative real screen-reader acceptance; see qa/UX_ASSISTIVE_TECHNOLOGY_CHECKLIST.md.

Local/staging requirement: Local browser with controlled response injection; real PostgreSQL workflow evidence reused. Shared representative actual-AT acceptance is pending (UX-ENV-001); the normal desktop/manual checklist is the approved execution path.

Source / existing evidence: frontend/app/page.tsx

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

Representative AT assignment: No separate reader run is required on this route. PASS covers the corrected route scope; overall UX acceptance still depends on all 11 shared representatives. See representative checklist.

Execution addendum (2026-09-21): Automated route checks pass. The domain-wide representative actual-AT gate is BLOCKED by UX-ENV-001; this does not require 35 duplicate reader runs. Browser accessibility-tree/keyboard checks are not actual screen-reader execution. Actual page/loader roles and states supersede generic matrices and Promise implementation labels. See UX execution report and route evidence. Historical generated wording is retained in evidence; no automatic continuation.

UX-031 — Page design, navigation and state QA: /policies

Area: UX · **Priority:** P2 · **Test level:** MANUAL UX, LOCAL E2E, SECURITY · **Result:** BLOCKED

Roles involved: anonymous, SCOUT, OWNER, ADMIN as applicable

Preconditions: Use the actual route role/auth contract. Verified email and current policies apply only to relevant marketplace mutations, not every page read or public recovery flow.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: 1. Open /policies directly and from every header/dashboard link as anonymous, SCOUT, OWNER, ADMIN as applicable; substitute owned fixture UUID for [id]. 2. Repeat anonymous and wrong-role access. 3. At 375px, 768px and 1440px inspect all cards/forms/tables/gallery/dialogs/financial breakdowns. 4. Keyboard-tab entire page, activate primary/secondary actions, close dialogs, check returned focus; verify labels/errors/loading/table headings structurally; actual screen-reader acceptance uses the shared 11-flow matrix rather than repeating this route mechanically. 5. Delay API 3 seconds; return empty collection or missing detail; then exercise current states error, loading, ready. 6. Inject 400/401/403/404/409/500, offline, invalid JSON and valid JSON wrong shape into each page loader; restore and retry. 7. Use long UUID, max venue name/reason, Unicode and large money values; reload and navigate back. 8. Review each design item below and record screenshot/finding.

Expected UI behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. Visible focus, associated labels/errors, semantic headings/tables, meaningful button names, focus containment/return for dialogs, status not color-only, live loading/status announcement and contrast must be checked.

Expected API/backend behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. No private data flash before role resolution; errors contain no stack/provider output; independent failed panels do not hide valid sibling data. State/action visibility agrees with backend. Fail any uncovered case; static /admin home only loads auth/policy state, not its own result list.

Expected database/domain state: Read-only variants do not mutate; any invoked workflow must satisfy its domain scenario and endpoint matrix.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: Local route/state automation plus representative real screen-reader acceptance; see qa/UX_ASSISTIVE_TECHNOLOGY_CHECKLIST.md.

Local/staging requirement: Local browser with controlled response injection; real PostgreSQL workflow evidence reused. Shared representative actual-AT acceptance is pending (UX-ENV-001); the normal desktop/manual checklist is the approved execution path.

Source / existing evidence: frontend/app/policies/page.tsx

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

Representative AT assignment: AT-R02 requires actual execution on this route; result remains BLOCKED. See representative checklist.

Execution addendum (2026-09-21): Automated route checks pass. The domain-wide representative actual-AT gate is BLOCKED by UX-ENV-001; this does not require 35 duplicate reader runs. Browser accessibility-tree/keyboard checks are not actual screen-reader execution. Actual page/loader roles and states supersede generic matrices and Promise implementation labels. See UX execution report and route evidence. Historical generated wording is retained in evidence; no automatic continuation.

UX-032 — Page design, navigation and state QA: /reset-password

Area: UX · **Priority:** P2 · **Test level:** MANUAL UX, LOCAL E2E, SECURITY · **Result:** PASS

Roles involved: anonymous, SCOUT, OWNER, ADMIN as applicable

Preconditions: Use the actual route role/auth contract. Verified email and current policies apply only to relevant marketplace mutations, not every page read or public recovery flow.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: 1. Open /reset-password directly and from every header/dashboard link as anonymous, SCOUT, OWNER, ADMIN as applicable; substitute owned fixture UUID for [id]. 2. Repeat anonymous and wrong-role access. 3. At 375px, 768px and 1440px inspect all cards/forms/tables/gallery/dialogs/financial breakdowns. 4. Keyboard-tab entire page, activate primary/secondary actions, close dialogs, check returned focus; verify labels/errors/loading/table headings structurally; actual screen-reader acceptance uses the shared 11-flow matrix rather than repeating this route mechanically. 5. Delay API 3 seconds; return empty collection or missing detail; then exercise current states invalid, success. 6. Inject 400/401/403/404/409/500, offline, invalid JSON and valid JSON wrong shape into each page loader; restore and retry. 7. Use long UUID, max venue name/reason, Unicode and large money values; reload and navigate back. 8. Review each design item below and record screenshot/finding.

Expected UI behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. Visible focus, associated labels/errors, semantic headings/tables, meaningful button names, focus containment/return for dialogs, status not color-only, live loading/status announcement and contrast must be checked.

Expected API/backend behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. No private data flash before role resolution; errors contain no stack/provider output; independent failed panels do not hide valid sibling data. State/action visibility agrees with backend. Fail any uncovered case; static /admin home only loads auth/policy state, not its own result list.

Expected database/domain state: Read-only variants do not mutate; any invoked workflow must satisfy its domain scenario and endpoint matrix.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: Local route/state automation plus representative real screen-reader acceptance; see qa/UX_ASSISTIVE_TECHNOLOGY_CHECKLIST.md.

Local/staging requirement: Local browser with controlled response injection; real PostgreSQL workflow evidence reused. Shared representative actual-AT acceptance is pending (UX-ENV-001); the normal desktop/manual checklist is the approved execution path.

Source / existing evidence: frontend/app/reset-password/page.tsx

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

Representative AT assignment: No separate reader run is required on this route. PASS covers the corrected route scope; overall UX acceptance still depends on all 11 shared representatives. See representative checklist.

Execution addendum (2026-09-21): Automated route checks pass. The domain-wide representative actual-AT gate is BLOCKED by UX-ENV-001; this does not require 35 duplicate reader runs. Browser accessibility-tree/keyboard checks are not actual screen-reader execution. Actual page/loader roles and states supersede generic matrices and Promise implementation labels. See UX execution report and route evidence. Historical generated wording is retained in evidence; no automatic continuation.

UX-033 — Page design, navigation and state QA: /scout/bookings

Area: UX · **Priority:** P2 · **Test level:** MANUAL UX, LOCAL E2E, SECURITY · **Result:** BLOCKED

Roles involved: SCOUT

Preconditions: Use the actual route role/auth contract. Verified email and current policies apply only to relevant marketplace mutations, not every page read or public recovery flow.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: 1. Open /scout/bookings directly and from every header/dashboard link as SCOUT; substitute owned fixture UUID for [id]. 2. Repeat anonymous and wrong-role access. 3. At 375px, 768px and 1440px inspect all cards/forms/tables/gallery/dialogs/financial breakdowns. 4. Keyboard-tab entire page, activate primary/secondary actions, close dialogs, check returned focus; verify labels/errors/loading/table headings structurally; actual screen-reader acceptance uses the shared 11-flow matrix rather than repeating this route mechanically. 5. Delay API 3 seconds; return empty collection or missing detail; then exercise current states error, loading. 6. Inject 400/401/403/404/409/500, offline, invalid JSON and valid JSON wrong shape into each page loader; restore and retry. 7. Use long UUID, max venue name/reason, Unicode and large money values; reload and navigate back. 8. Review each design item below and record screenshot/finding.

Expected UI behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. Visible focus, associated labels/errors, semantic headings/tables, meaningful button names, focus containment/return for dialogs, status not color-only, live loading/status announcement and contrast must be checked.

Expected API/backend behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. No private data flash before role resolution; errors contain no stack/provider output; independent failed panels do not hide valid sibling data. State/action visibility agrees with backend. Fail any uncovered case; static /admin home only loads auth/policy state, not its own result list.

Expected database/domain state: Read-only variants do not mutate; any invoked workflow must satisfy its domain scenario and endpoint matrix.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: Local route/state automation plus representative real screen-reader acceptance; see qa/UX_ASSISTIVE_TECHNOLOGY_CHECKLIST.md.

Local/staging requirement: Local browser with controlled response injection; real PostgreSQL workflow evidence reused. Shared representative actual-AT acceptance is pending (UX-ENV-001); the normal desktop/manual checklist is the approved execution path.

Source / existing evidence: frontend/app/scout/bookings/page.tsx

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

Representative AT assignment: AT-R04 requires actual execution on this route; result remains BLOCKED. See representative checklist.

Execution addendum (2026-09-21): Automated route checks pass. The domain-wide representative actual-AT gate is BLOCKED by UX-ENV-001; this does not require 35 duplicate reader runs. Browser accessibility-tree/keyboard checks are not actual screen-reader execution. Actual page/loader roles and states supersede generic matrices and Promise implementation labels. See UX execution report and route evidence. Historical generated wording is retained in evidence; no automatic continuation.

UX-034 — Page design, navigation and state QA: /scout/inquiries

Area: UX · **Priority:** P2 · **Test level:** MANUAL UX, LOCAL E2E, SECURITY · **Result:** PASS

Roles involved: SCOUT

Preconditions: Use the actual route role/auth contract. Verified email and current policies apply only to relevant marketplace mutations, not every page read or public recovery flow.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: 1. Open /scout/inquiries directly and from every header/dashboard link as SCOUT; substitute owned fixture UUID for [id]. 2. Repeat anonymous and wrong-role access. 3. At 375px, 768px and 1440px inspect all cards/forms/tables/gallery/dialogs/financial breakdowns. 4. Keyboard-tab entire page, activate primary/secondary actions, close dialogs, check returned focus; verify labels/errors/loading/table headings structurally; actual screen-reader acceptance uses the shared 11-flow matrix rather than repeating this route mechanically. 5. Delay API 3 seconds; return empty collection or missing detail; then exercise current states OFFERED, error, loading. 6. Inject 400/401/403/404/409/500, offline, invalid JSON and valid JSON wrong shape into each page loader; restore and retry. 7. Use long UUID, max venue name/reason, Unicode and large money values; reload and navigate back. 8. Review each design item below and record screenshot/finding.

Expected UI behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. Visible focus, associated labels/errors, semantic headings/tables, meaningful button names, focus containment/return for dialogs, status not color-only, live loading/status announcement and contrast must be checked.

Expected API/backend behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. No private data flash before role

resolution; errors contain no stack/provider output; independent failed panels do not hide valid sibling data. State/action visibility agrees with backend. Fail any uncovered case; static /admin home only loads auth/policy state, not its own result list.

Expected database/domain state: Read-only variants do not mutate; any invoked workflow must satisfy its domain scenario and endpoint matrix.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: Local route/state automation plus representative real screen-reader acceptance; see qa/UX_ASSISTIVE_TECHNOLOGY_CHECKLIST.md.

Local/staging requirement: Local browser with controlled response injection; real PostgreSQL workflow evidence reused. Shared representative actual-AT acceptance is pending (UX-ENV-001); the normal desktop/manual checklist is the approved execution path.

Source / existing evidence: frontend/app/scout/inquiries/page.tsx

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

Representative AT assignment: No separate reader run is required on this route. PASS covers the corrected route scope; overall UX acceptance still depends on all 11 shared representatives. See representative checklist.

Execution addendum (2026-09-21): Automated route checks pass. The domain-wide representative actual-AT gate is BLOCKED by UX-ENV-001; this does not require 35 duplicate reader runs. Browser accessibility-tree/keyboard checks are not actual screen-reader execution. Actual page/loader roles and states supersede generic matrices and Promise implementation labels. See UX execution report and route evidence. Historical generated wording is retained in evidence; no automatic continuation.

UX-035 — Page design, navigation and state QA: /verify-email

Area: UX · **Priority:** P2 · **Test level:** MANUAL UX, LOCAL E2E, SECURITY · **Result:** PASS

Roles involved: anonymous, SCOUT, OWNER, ADMIN as applicable

Preconditions: Use the actual route role/auth contract. Verified email and current policies apply only to relevant marketplace mutations, not every page read or public recovery flow.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: 1. Open /verify-email directly and from every header/dashboard link as anonymous, SCOUT, OWNER, ADMIN as applicable; substitute owned fixture UUID for [id]. 2. Repeat anonymous and wrong-role access. 3. At 375px, 768px and 1440px inspect all cards/forms/tables/gallery/dialogs/financial breakdowns. 4. Keyboard-tab entire page, activate primary/secondary actions, close dialogs, check returned focus; verify labels/errors/loading/table headings structurally; actual screen-reader acceptance uses the shared 11-flow matrix rather than repeating this route mechanically. 5. Delay API 3 seconds; return empty collection or missing detail; then exercise current states invalid, success, verifying. 6. Inject 400/401/403/404/409/500, offline, invalid JSON and valid JSON wrong shape into each page loader; restore and retry. 7. Use long UUID, max venue name/reason, Unicode and large money values; reload and navigate back. 8. Review each design item below and record screenshot/finding.

Expected UI behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. Visible focus, associated labels/errors, semantic headings/tables, meaningful button names, focus containment/return for dialogs, status not color-only, live loading/status announcement and contrast must be checked.

Expected API/backend behavior: Primary action obvious; role/context obvious; status understandable; next step clear; no unnecessary information; consistent copy; precise money labels; destructive actions distinct; mobile usable; polished loading/empty/error; consistent Eema typography, paper/ink/ochre palette and marketplace layout. No private data flash before role resolution; errors contain no stack/provider output; independent failed panels do not hide valid sibling data. State/action visibility agrees with backend. Fail any uncovered case; static /admin home only loads auth/policy state, not its own result list.

Expected database/domain state: Read-only variants do not mutate; any invoked workflow must satisfy its domain scenario and endpoint matrix.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: Local route/state automation plus representative real screen-reader acceptance; see qa/UX_ASSISTIVE_TECHNOLOGY_CHECKLIST.md.

Local/staging requirement: Local browser with controlled response injection; real PostgreSQL workflow evidence reused. Shared representative actual-AT acceptance is pending (UX-ENV-001); the normal desktop/manual checklist is the approved execution path.

Source / existing evidence: frontend/app/verify-email/page.tsx

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

Representative AT assignment: No separate reader run is required on this route. PASS covers the corrected route scope; overall UX acceptance still depends on all 11 shared representatives. See representative checklist.

Execution addendum (2026-09-21): Automated route checks pass. The domain-wide representative actual-AT gate is BLOCKED by UX-ENV-001; this does not require 35 duplicate reader runs. Browser accessibility-tree/keyboard checks are not actual screen-reader execution. Actual page/loader roles and states supersede generic matrices and Promise implementation labels. See UX execution report and route evidence. Historical generated wording is retained in evidence; no automatic continuation.

UXACT scenarios

UXACT-001 — Double submission: booking request

Area: UXACT · **Priority:** P0 · **Test level:** MANUAL UX, API, CONCURRENCY · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Open / with valid fixture. Delay mutation response 3 seconds; rapid double-click booking request, press Enter twice, then click again after a network timeout. Inspect requests and final record/provider/event count.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Busy control blocks ordinary repeated clicks; backend transition/idempotency prevents duplicate money/state actions where implemented. Creation endpoints have no universal idempotency key: duplicate distinct REQUESTED bookings/incidents/inquiries remain possible through replay; document failures rather than assuming universal guarantee.

Expected database/domain state: Busy control blocks ordinary repeated clicks; backend transition/idempotency prevents duplicate money/state actions where implemented. Creation endpoints have no universal idempotency key: duplicate distinct REQUESTED bookings/incidents/inquiries remain possible through replay; document failures rather than assuming universal guarantee.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: /

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

UXACT-002 — Double submission: inquiry

Area: UXACT · **Priority:** P0 · **Test level:** MANUAL UX, API, CONCURRENCY · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Open / with valid fixture. Delay mutation response 3 seconds; rapid double-click inquiry, press Enter twice, then click again after a network timeout. Inspect requests and final record/provider/event count.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Busy control blocks ordinary repeated clicks; backend transition/idempotency prevents duplicate money/state actions where implemented. Creation endpoints have no universal idempotency key: duplicate distinct REQUESTED bookings/incidents/inquiries remain possible through replay; document failures rather than assuming universal guarantee.

Expected database/domain state: Busy control blocks ordinary repeated clicks; backend transition/idempotency prevents duplicate money/state actions where implemented. Creation endpoints have no universal idempotency key: duplicate distinct REQUESTED bookings/incidents/inquiries remain possible through replay; document failures rather than assuming universal guarantee.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: /

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

UXACT-003 — Double submission: owner accept

Area: UXACT · **Priority:** P0 · **Test level:** MANUAL UX, API, CONCURRENCY · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Open /owner/bookings with valid fixture. Delay mutation response 3 seconds; rapid double-click owner accept, press Enter twice, then click again after a network timeout. Inspect requests and final record/provider/event count.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Busy control blocks ordinary repeated clicks; backend transition/idempotency prevents duplicate money/state actions where implemented. Creation endpoints have no universal idempotency key: duplicate distinct REQUESTED bookings/incidents/inquiries remain possible through replay; document failures rather than assuming universal guarantee.

Expected database/domain state: Busy control blocks ordinary repeated clicks; backend transition/idempotency prevents duplicate money/state actions where implemented. Creation endpoints have no universal idempotency key: duplicate distinct REQUESTED bookings/incidents/inquiries remain possible through replay; document failures rather than assuming universal guarantee.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: /owner/bookings

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

UXACT-004 — Double submission: owner decline

Area: UXACT · **Priority:** P0 · **Test level:** MANUAL UX, API, CONCURRENCY · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Open /owner/bookings with valid fixture. Delay mutation response 3 seconds; rapid double-click owner decline, press Enter twice, then click again after a network timeout. Inspect requests and final record/provider/event count.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Busy control blocks ordinary repeated clicks; backend transition/idempotency prevents duplicate money/state actions where implemented. Creation endpoints have no universal idempotency key: duplicate distinct REQUESTED bookings/incidents/inquiries remain possible through replay; document failures rather than assuming universal guarantee.

Expected database/domain state: Busy control blocks ordinary repeated clicks; backend transition/idempotency prevents duplicate money/state actions where implemented. Creation endpoints have no universal idempotency key: duplicate distinct REQUESTED bookings/incidents/inquiries remain possible through replay; document failures rather than assuming universal guarantee.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: /owner/bookings

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

UXACT-005 — Double submission: checkout

Area: UXACT · **Priority:** P0 · **Test level:** MANUAL UX, API, CONCURRENCY · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Open /bookings/[id] with valid fixture. Delay mutation response 3 seconds; rapid double-click checkout, press Enter twice, then click again after a network timeout. Inspect requests and final record/provider/event count.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Busy control blocks ordinary repeated clicks; backend transition/idempotency prevents duplicate money/state actions where implemented. Creation endpoints have no universal idempotency key: duplicate distinct REQUESTED bookings/incidents/inquiries remain possible through replay; document failures rather than assuming universal guarantee.

Expected database/domain state: Busy control blocks ordinary repeated clicks; backend transition/idempotency prevents duplicate money/state actions where implemented. Creation endpoints have no universal idempotency key: duplicate distinct REQUESTED bookings/incidents/inquiries remain possible through replay; document failures rather than assuming universal guarantee.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: /bookings/[id]

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

UXACT-006 — Double submission: cancel

Area: UXACT · **Priority:** P0 · **Test level:** MANUAL UX, API, CONCURRENCY · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Open /bookings/[id] with valid fixture. Delay mutation response 3 seconds; rapid double-click cancel, press Enter twice, then click again after a network timeout. Inspect requests and final record/provider/event count.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Busy control blocks ordinary repeated clicks; backend transition/idempotency prevents duplicate money/state actions where implemented. Creation endpoints have no universal idempotency key: duplicate distinct REQUESTED bookings/incidents/inquiries remain possible through replay; document failures rather than assuming universal guarantee.

Expected database/domain state: Busy control blocks ordinary repeated clicks; backend transition/idempotency prevents duplicate money/state actions where implemented. Creation endpoints have no universal idempotency key: duplicate distinct REQUESTED bookings/incidents/inquiries remain possible through replay; document failures rather than assuming universal guarantee.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: /bookings/[id]

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

UXACT-007 — Double submission: incident

Area: UXACT · **Priority:** P0 · **Test level:** MANUAL UX, API, CONCURRENCY · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Open /bookings/[id] with valid fixture. Delay mutation response 3 seconds; rapid double-click incident, press Enter twice, then click again after a network timeout. Inspect requests and final record/provider/event count.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Busy control blocks ordinary repeated clicks; backend transition/idempotency prevents duplicate money/state actions where implemented. Creation endpoints have no universal idempotency key: duplicate distinct REQUESTED bookings/incidents/inquiries remain possible through replay; document failures rather than assuming universal guarantee.

Expected database/domain state: Busy control blocks ordinary repeated clicks; backend transition/idempotency prevents duplicate money/state actions where implemented. Creation endpoints have no universal idempotency key: duplicate distinct REQUESTED bookings/incidents/inquiries remain possible through replay; document failures rather than assuming universal guarantee.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: /bookings/[id]

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

UXACT-008 — Double submission: policy acceptance

Area: UXACT · **Priority:** P0 · **Test level:** MANUAL UX, API, CONCURRENCY · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Open /policies with valid fixture. Delay mutation response 3 seconds; rapid double-click policy acceptance, press Enter twice, then click again after a network timeout. Inspect requests and final record/provider/event count.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Busy control blocks ordinary repeated clicks; backend transition/idempotency prevents duplicate money/state actions where implemented. Creation endpoints have no universal idempotency key: duplicate distinct REQUESTED bookings/incidents/inquiries remain possible through replay; document failures rather than assuming universal guarantee.

Expected database/domain state: Busy control blocks ordinary repeated clicks; backend transition/idempotency prevents duplicate money/state actions where implemented. Creation endpoints have no universal idempotency key: duplicate distinct REQUESTED bookings/incidents/inquiries remain possible through replay; document failures rather than assuming universal guarantee.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: /policies

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

UXACT-009 — Double submission: venue submission

Area: UXACT · **Priority:** P0 · **Test level:** MANUAL UX, API, CONCURRENCY · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Open /owner/venues/[id] with valid fixture. Delay mutation response 3 seconds; rapid double-click venue submission, press Enter twice, then click again after a network timeout. Inspect requests and final record/provider/event count.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Busy control blocks ordinary repeated clicks; backend transition/idempotency prevents duplicate money/state actions where implemented. Creation endpoints have no universal idempotency key: duplicate distinct REQUESTED bookings/incidents/inquiries remain possible through replay; document failures rather than assuming universal guarantee.

Expected database/domain state: Busy control blocks ordinary repeated clicks; backend transition/idempotency prevents duplicate money/state actions where implemented. Creation endpoints have no universal idempotency key: duplicate distinct REQUESTED bookings/incidents/inquiries remain possible through replay; document failures rather than assuming universal guarantee.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: /owner/venues/[id]

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

UXACT-010 — Double submission: owner verification

Area: UXACT · **Priority:** P0 · **Test level:** MANUAL UX, API, CONCURRENCY · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Open /owner/verification with valid fixture. Delay mutation response 3 seconds; rapid double-click owner verification, press Enter twice, then click again after a network timeout. Inspect requests and final record/provider/event count.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Busy control blocks ordinary repeated clicks; backend transition/idempotency prevents duplicate money/state actions where implemented. Creation endpoints have no universal idempotency key: duplicate distinct REQUESTED bookings/incidents/inquiries remain possible through replay; document failures rather than assuming universal guarantee.

Expected database/domain state: Busy control blocks ordinary repeated clicks; backend transition/idempotency prevents duplicate money/state actions where implemented. Creation endpoints have no universal idempotency key: duplicate distinct REQUESTED bookings/incidents/inquiries remain possible through replay; document failures rather than assuming universal guarantee.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: /owner/verification

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

UXACT-011 — Double submission: payout destination

Area: UXACT · **Priority:** P0 · **Test level:** MANUAL UX, API, CONCURRENCY · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Open /owner/payout-destination with valid fixture. Delay mutation response 3 seconds; rapid double-click payout destination, press Enter twice, then click again after a network timeout. Inspect requests and final record/provider/event count.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Busy control blocks ordinary repeated clicks; backend transition/idempotency prevents duplicate money/state actions where implemented. Creation endpoints have no universal idempotency key: duplicate distinct REQUESTED bookings/incidents/inquiries remain possible through replay; document failures rather than assuming universal guarantee.

Expected database/domain state: Busy control blocks ordinary repeated clicks; backend transition/idempotency prevents duplicate money/state actions where implemented. Creation endpoints have no universal idempotency key: duplicate distinct REQUESTED bookings/incidents/inquiries remain possible through replay; document failures rather than assuming universal guarantee.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: /owner/payout-destination

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

UXACT-012 — Double submission: admin verify/reject

Area: UXACT · **Priority:** P0 · **Test level:** MANUAL UX, API, CONCURRENCY · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Open /admin/payout-destinations/[id] with valid fixture. Delay mutation response 3 seconds; rapid double-click admin verify/reject, press Enter twice, then click again after a network timeout. Inspect requests and final record/provider/event count.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Busy control blocks ordinary repeated clicks; backend transition/idempotency prevents duplicate money/state actions where implemented. Creation endpoints have no universal idempotency key: duplicate distinct REQUESTED bookings/incidents/inquiries remain possible through replay; document failures rather than assuming universal guarantee.

Expected database/domain state: Busy control blocks ordinary repeated clicks; backend transition/idempotency prevents duplicate money/state actions where implemented. Creation endpoints have no universal idempotency key: duplicate distinct REQUESTED bookings/incidents/inquiries remain possible through replay; document failures rather than assuming universal guarantee.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: /admin/payout-destinations/[id]

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

UXACT-013 — Double submission: mark paid

Area: UXACT · **Priority:** P0 · **Test level:** MANUAL UX, API, CONCURRENCY · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Open /admin/payouts/[id] with valid fixture. Delay mutation response 3 seconds; rapid double-click mark paid, press Enter twice, then click again after a network timeout. Inspect requests and final record/provider/event count.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Busy control blocks ordinary repeated clicks; backend transition/idempotency prevents duplicate money/state actions where implemented. Creation endpoints have no universal idempotency key: duplicate distinct REQUESTED bookings/incidents/inquiries remain possible through replay; document failures rather than assuming universal guarantee.

Expected database/domain state: Busy control blocks ordinary repeated clicks; backend transition/idempotency prevents duplicate money/state actions where implemented. Creation endpoints have no universal idempotency key: duplicate distinct REQUESTED bookings/incidents/inquiries remain possible through replay; document failures rather than assuming universal guarantee.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: /admin/payouts/[id]

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

UXACT-014 — Double submission: settle liability

Area: UXACT · **Priority:** P0 · **Test level:** MANUAL UX, API, CONCURRENCY · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Open /admin/liabilities/[id] with valid fixture. Delay mutation response 3 seconds; rapid double-click settle liability, press Enter twice, then click again after a network timeout. Inspect requests and final record/provider/event count.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Busy control blocks ordinary repeated clicks; backend transition/idempotency prevents duplicate money/state actions where implemented. Creation endpoints have no universal idempotency key: duplicate distinct REQUESTED bookings/incidents/inquiries remain possible through replay; document failures rather than assuming universal guarantee.

Expected database/domain state: Busy control blocks ordinary repeated clicks; backend transition/idempotency prevents duplicate money/state actions where implemented. Creation endpoints have no universal idempotency key: duplicate distinct REQUESTED bookings/incidents/inquiries remain possible through replay; document failures rather than assuming universal guarantee.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: /admin/liabilities/[id]

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

UXACT-015 — Session expiry while browsing marketplace

Area: UXACT · **Priority:** P0 · **Test level:** MANUAL UX, SECURITY, LOCAL E2E · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Open flow for browsing marketplace; expire access token then refresh token in isolated fixture; invoke next read/mutation/download; reauthenticate and retry after authoritative reload.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Successful refresh continues once; failed refresh signs out and safe re-auth guidance shown. Unsaved fields and stale confirmations must be assessed; no duplicate financial operation or cross-account content after relogin.

Expected database/domain state: Successful refresh continues once; failed refresh signs out and safe re-auth guidance shown. Unsaved fields and stale confirmations must be assessed; no duplicate financial operation or cross-account content after relogin.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: AuthProvider, apiRequest, route/components

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

UXACT-016 — Session expiry while editing owner venue

Area: UXACT · **Priority:** P0 · **Test level:** MANUAL UX, SECURITY, LOCAL E2E · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Open flow for editing owner venue; expire access token then refresh token in isolated fixture; invoke next read/mutation/download; reauthenticate and retry after authoritative reload.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Successful refresh continues once; failed refresh signs out and safe re-auth guidance shown. Unsaved fields and stale confirmations must be assessed; no duplicate financial operation or cross-account content after relogin.

Expected database/domain state: Successful refresh continues once; failed refresh signs out and safe re-auth guidance shown. Unsaved fields and stale confirmations must be assessed; no duplicate financial operation or cross-account content after relogin.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: AuthProvider, apiRequest, route/components

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

UXACT-017 — Session expiry while checkout

Area: UXACT · **Priority:** P0 · **Test level:** MANUAL UX, SECURITY, LOCAL E2E · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Open flow for checkout; expire access token then refresh token in isolated fixture; invoke next read/mutation/download; reauthenticate and retry after authoritative reload.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Successful refresh continues once; failed refresh signs out and safe re-auth guidance shown. Unsaved fields and stale confirmations must be assessed; no duplicate financial operation or cross-account content after relogin.

Expected database/domain state: Successful refresh continues once; failed refresh signs out and safe re-auth guidance shown. Unsaved fields and stale confirmations must be assessed; no duplicate financial operation or cross-account content after relogin.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: AuthProvider, apiRequest, route/components

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

UXACT-018 — Session expiry while admin review or mark paid

Area: UXACT · **Priority:** P0 · **Test level:** MANUAL UX, SECURITY, LOCAL E2E · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Open flow for admin review or mark paid; expire access token then refresh token in isolated fixture; invoke next read/mutation/download; reauthenticate and retry after authoritative reload.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Successful refresh continues once; failed refresh signs out and safe re-auth guidance shown. Unsaved fields and stale confirmations must be assessed; no duplicate financial operation or cross-account content after relogin.

Expected database/domain state: Successful refresh continues once; failed refresh signs out and safe re-auth guidance shown. Unsaved fields and stale confirmations must be assessed; no duplicate financial operation or cross-account content after relogin.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: AuthProvider, apiRequest, route/components

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

UXACT-019 — Session expiry while private document download

Area: UXACT · **Priority:** P0 · **Test level:** MANUAL UX, SECURITY, LOCAL E2E · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Open flow for private document download; expire access token then refresh token in isolated fixture; invoke next read/mutation/download; reauthenticate and retry after authoritative reload.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Successful refresh continues once; failed refresh signs out and safe re-auth guidance shown. Unsaved fields and stale confirmations must be assessed; no duplicate financial operation or cross-account content after relogin.

Expected database/domain state: Successful refresh continues once; failed refresh signs out and safe re-auth guidance shown. Unsaved fields and stale confirmations must be assessed; no duplicate financial operation or cross-account content after relogin.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: AuthProvider, apiRequest, route/components

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

PROVIDER scenarios

PROVIDER-001 — Real staging Mollie

Area: PROVIDER · **Priority:** P0 · **Test level:** STAGING E2E, OPERATIONAL · **Result:** NOT RUN

Roles involved: QA sandbox personas, Admin/operator

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: After isolated staging is provisioned with test credentials by operations, repeat variants: payment create/open/paid; duplicate callback; full and partial refund; late payment; invalid amount/ID injected only in mock. Record sandbox provider IDs, sanitized request/response, timestamps and UI/domain evidence. Do not perform these calls during documentation work.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Provider-specific behavior agrees with local contract and lifecycle matrix; no live charge or transfer. If test mode cannot supply authoritative reporting data mark BLOCKED; mock success is not delivery/reporting/hosting proof.

Expected database/domain state: Provider-specific behavior agrees with local contract and lifecycle matrix; no live charge or transfer. If test mode cannot supply authoritative reporting data mark BLOCKED; mock success is not delivery/reporting/hosting proof.

Expected email/provider behavior: Mollie: collect sandbox provider-side confirmation and safe failure evidence; no production calls.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: STAGING ONLY

Local/staging requirement: STAGING

Source / existing evidence: MOLLIE

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

PROVIDER-002 — Real staging Resend

Area: PROVIDER · **Priority:** P0 · **Test level:** STAGING E2E, OPERATIONAL · **Result:** NOT RUN

Roles involved: QA sandbox personas, Admin/operator

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: After isolated staging is provisioned with test credentials by operations, repeat variants: delivery of every implemented EMAIL scenario; sender/domain, correct HTTPS deep link, inbox/spam and eventual delivery evidence. Record sandbox provider IDs, sanitized request/response, timestamps and UI/domain evidence. Do not perform these calls during documentation work.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Provider-specific behavior agrees with local contract and lifecycle matrix; no live charge or transfer. If test mode cannot supply authoritative reporting data mark BLOCKED; mock success is not delivery/reporting/hosting proof.

Expected database/domain state: Provider-specific behavior agrees with local contract and lifecycle matrix; no live charge or transfer. If test mode cannot supply authoritative reporting data mark BLOCKED; mock success is not delivery/reporting/hosting proof.

Expected email/provider behavior: Resend: collect sandbox provider-side confirmation and safe failure evidence; no production calls.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: STAGING ONLY

Local/staging requirement: STAGING

Source / existing evidence: RESEND

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

PROVIDER-003 — Real staging Gemini

Area: PROVIDER · **Priority:** P0 · **Test level:** STAGING E2E, OPERATIONAL · **Result:** NOT RUN

Roles involved: QA sandbox personas, Admin/operator

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: After isolated staging is provisioned with test credentials by operations, repeat variants: representative explicit/relative/ambiguous briefs; provider timeout/unavailable/malformed output; deterministic hard filters and zero-candidate ranker skip. Record sandbox provider IDs, sanitized request/response, timestamps and UI/domain evidence. Do not perform these calls during documentation work.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Provider-specific behavior agrees with local contract and lifecycle matrix; no live charge or transfer. If test mode cannot supply authoritative reporting data mark BLOCKED; mock success is not delivery/reporting/hosting proof.

Expected database/domain state: Provider-specific behavior agrees with local contract and lifecycle matrix; no live charge or transfer. If test mode cannot supply authoritative reporting data mark BLOCKED; mock success is not delivery/reporting/hosting proof.

Expected email/provider behavior: Gemini: collect sandbox provider-side confirmation and safe failure evidence; no production calls.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: STAGING ONLY

Local/staging requirement: STAGING

Source / existing evidence: GEMINI

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

PROVIDER-004 — Real staging S3

Area: PROVIDER · **Priority:** P0 · **Test level:** STAGING E2E, OPERATIONAL · **Result:** NOT RUN

Roles involved: QA sandbox personas, Admin/operator

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: After isolated staging is provisioned with test credentials by operations, repeat variants: public media retrieval, private PDF authorization, correct bucket separation, upload/issue failure and restart durability. Record sandbox provider IDs, sanitized request/response, timestamps and UI/domain evidence. Do not perform these calls during documentation work.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Provider-specific behavior agrees with local contract and lifecycle matrix; no live charge or transfer. If test mode cannot supply authoritative reporting data mark BLOCKED; mock success is not delivery/reporting/hosting proof.

Expected database/domain state: Provider-specific behavior agrees with local contract and lifecycle matrix; no live charge or transfer. If test mode cannot supply authoritative reporting data mark BLOCKED; mock success is not delivery/reporting/hosting proof.

Expected email/provider behavior: S3: collect sandbox provider-side confirmation and safe failure evidence; no production calls.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: STAGING ONLY

Local/staging requirement: STAGING

Source / existing evidence: S3

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

PROVIDER-005 — Real staging Mollie reporting

Area: PROVIDER · **Priority:** P0 · **Test level:** STAGING E2E, OPERATIONAL · **Result:** NOT RUN

Roles involved: QA sandbox personas, Admin/operator

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: After isolated staging is provisioned with test credentials by operations, repeat variants: actual fee visibility and reporting access with test payments; unavailable fee remains held and manual reconciliation limitation documented. Record sandbox provider IDs, sanitized request/response, timestamps and UI/domain evidence. Do not perform these calls during documentation work.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Provider-specific behavior agrees with local contract and lifecycle matrix; no live charge or transfer. If test mode cannot supply authoritative reporting data mark BLOCKED; mock success is not delivery/reporting/hosting proof.

Expected database/domain state: Provider-specific behavior agrees with local contract and lifecycle matrix; no live charge or transfer. If test mode cannot supply authoritative reporting data mark BLOCKED; mock success is not delivery/reporting/hosting proof.

Expected email/provider behavior: Mollie reporting: collect sandbox provider-side confirmation and safe failure evidence; no production calls.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: STAGING ONLY

Local/staging requirement: STAGING

Source / existing evidence: MOLLIE

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

CALA scenarios

CALA-001 — Optional company verification BFF

Area: CALA · **Priority:** P2 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: POST /api/verify-booker to Next origin with malformed JSON, {}, blank company and missing server key; then mock Company search success, detail failure, provider 500, timeout and quoted key configuration.

Expected UI behavior: BookerVerification is a compatibility component; verify import/reachability separately. Do not call this manual owner verification or KYC.

Expected API/backend behavior: Usually 200 verified=false on invalid/missing/provider failure; Company hit may return verified=true using search fallback. Server key never client-visible; response currently includes raw provider search/full data. No Spring owner verification/payment authorization side effects.

Expected database/domain state: Usually 200 verified=false on invalid/missing/provider failure; Company hit may return verified=true using search fallback. Server key never client-visible; response currently includes raw provider search/full data. No Spring owner verification/payment authorization side effects.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: Frontend app/api/verify-booker/route.ts, lib/cala.ts

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

EMAIL scenarios

EMAIL-001 — Registration verification

Area: EMAIL · **Priority:** P1 · **Test level:** UNIT, INTEGRATION · **Result:** NOT RUN

Roles involved: active registering/resending account

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Use corresponding domain scenario to commit Registration verification; capture EmailService.send arguments with local mock (do not enable real delivery). Compare text and HTML with fixture using Unicode and HTML-like name/reason. Replay lifecycle and roll back an independent event. Repeat actual delivery only under PROVIDER Resend scenario.

Expected UI behavior: Open captured CTA against matching frontend route under authorized test account; login preserves useful context.

Expected API/backend behavior: Recipient=active registering/resending account; subject exactly “Verify your Eema email”; CTA uses configured frontend base + /verify-email?token=…. Critical content: Raw token only in transactional link; expiry; account not yet verified. Escape user content; no keys/password/full IBAN; no false money claim. Token in verification/reset link is the intentional exception to no-secrets-in-mail.

Expected database/domain state: Notification does not change booking, refund, payout, trust state or amount; committed lifecycle is authoritative.

Expected email/provider behavior: LOCAL CONTENT TEST only. STAGING DELIVERY TEST is PROVIDER Resend: actual sender/domain, inbox, correct link and delivery evidence.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: AccountEmailNotificationService, BookingNotificationService, AvailabilityInquiryNotificationService, TrustNotificationService

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

EMAIL-002 — Password reset

Area: EMAIL · **Priority:** P1 · **Test level:** UNIT, INTEGRATION · **Result:** NOT RUN

Roles involved: active existing account

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Use corresponding domain scenario to commit Password reset; capture EmailService.send arguments with local mock (do not enable real delivery). Compare text and HTML with fixture using Unicode and HTML-like name/reason. Replay lifecycle and roll back an independent event. Repeat actual delivery only under PROVIDER Resend scenario.

Expected UI behavior: Open captured CTA against matching frontend route under authorized test account; login preserves useful context.

Expected API/backend behavior: Recipient=active existing account; subject exactly “Reset your Eema password”; CTA uses configured frontend base + /reset-password?token=…. Critical content: Generic request response, reset link and expiry; no plaintext password. Escape user content; no keys/password/full IBAN; no false money claim. Token in verification/reset link is the intentional exception to no-secrets-in-mail.

Expected database/domain state: Notification does not change booking, refund, payout, trust state or amount; committed lifecycle is authoritative.

Expected email/provider behavior: LOCAL CONTENT TEST only. STAGING DELIVERY TEST is PROVIDER Resend: actual sender/domain, inbox, correct link and delivery evidence.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: AccountEmailNotificationService, BookingNotificationService, AvailabilityInquiryNotificationService, TrustNotificationService

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

EMAIL-003 — Request submitted scout

Area: EMAIL · **Priority:** P1 · **Test level:** UNIT, INTEGRATION · **Result:** NOT RUN

Roles involved: scout contact snapshot

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Use corresponding domain scenario to commit Request submitted scout; capture EmailService.send arguments with local mock (do not enable real delivery). Compare text and HTML with fixture using Unicode and HTML-like name/reason. Replay lifecycle and roll back an independent event. Repeat actual delivery only under PROVIDER Resend scenario.

Expected UI behavior: Open captured CTA against matching frontend route under authorized test account; login preserves useful context.

Expected API/backend behavior: Recipient=scout contact snapshot; subject exactly “Your request was sent”; CTA uses configured frontend base + /bookings/{id}. Critical content: REQUESTED, not paid/confirmed, owner response deadline. Escape user content; no keys/password/full IBAN; no false money claim. Token in verification/reset link is the intentional exception to no-secrets-in-mail.

Expected database/domain state: Notification does not change booking, refund, payout, trust state or amount; committed lifecycle is authoritative.

Expected email/provider behavior: LOCAL CONTENT TEST only. STAGING DELIVERY TEST is PROVIDER Resend: actual sender/domain, inbox, correct link and delivery evidence.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: AccountEmailNotificationService, BookingNotificationService, AvailabilityInquiryNotificationService, TrustNotificationService

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

EMAIL-004 — Request submitted owner

Area: EMAIL · **Priority:** P1 · **Test level:** UNIT, INTEGRATION · **Result:** NOT RUN

Roles involved: owner contact snapshot

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Use corresponding domain scenario to commit Request submitted owner; capture EmailService.send arguments with local mock (do not enable real delivery). Compare text and HTML with fixture using Unicode and HTML-like name/reason. Replay lifecycle and roll back an independent event. Repeat actual delivery only under PROVIDER Resend scenario.

Expected UI behavior: Open captured CTA against matching frontend route under authorized test account; login preserves useful context.

Expected API/backend behavior: Recipient=owner contact snapshot; subject exactly “New booking request”; CTA uses configured frontend base + /bookings/{id}. Critical content: Scout contact/brief, unreserved slot and response deadline. Escape user content; no keys/password/full IBAN; no false money claim. Token in verification/reset link is the intentional exception to no-secrets-in-mail.

Expected database/domain state: Notification does not change booking, refund, payout, trust state or amount; committed lifecycle is authoritative.

Expected email/provider behavior: LOCAL CONTENT TEST only. STAGING DELIVERY TEST is PROVIDER Resend: actual sender/domain, inbox, correct link and delivery evidence.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: AccountEmailNotificationService, BookingNotificationService, AvailabilityInquiryNotificationService, TrustNotificationService

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

EMAIL-005 — Request accepted

Area: EMAIL · **Priority:** P1 · **Test level:** UNIT, INTEGRATION · **Result:** NOT RUN

Roles involved: scout contact snapshot

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Use corresponding domain scenario to commit Request accepted; capture EmailService.send arguments with local mock (do not enable real delivery). Compare text and HTML with fixture using Unicode and HTML-like name/reason. Replay lifecycle and roll back an independent event. Repeat actual delivery only under PROVIDER Resend scenario.

Expected UI behavior: Open captured CTA against matching frontend route under authorized test account; login preserves useful context.

Expected API/backend behavior: Recipient=scout contact snapshot; subject exactly “Your booking request was accepted”; CTA uses configured frontend base + /bookings/{id}. Critical content: AWAITING_PAYMENT, payment deadline and Pay now CTA. Escape user content; no keys/password/full IBAN; no false money claim. Token in verification/reset link is the intentional exception to no-secrets-in-mail.

Expected database/domain state: Notification does not change booking, refund, payout, trust state or amount; committed lifecycle is authoritative.

Expected email/provider behavior: LOCAL CONTENT TEST only. STAGING DELIVERY TEST is PROVIDER Resend: actual sender/domain, inbox, correct link and delivery evidence.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: AccountEmailNotificationService, BookingNotificationService, AvailabilityInquiryNotificationService, TrustNotificationService

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

EMAIL-006 — Request declined

Area: EMAIL · **Priority:** P1 · **Test level:** UNIT, INTEGRATION · **Result:** NOT RUN

Roles involved: scout contact snapshot

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Use corresponding domain scenario to commit Request declined; capture EmailService.send arguments with local mock (do not enable real delivery). Compare text and HTML with fixture using Unicode and HTML-like name/reason. Replay lifecycle and roll back an independent event. Repeat actual delivery only under PROVIDER Resend scenario.

Expected UI behavior: Open captured CTA against matching frontend route under authorized test account; login preserves useful context.

Expected API/backend behavior: Recipient=scout contact snapshot; subject exactly “Your booking request was declined”; CTA uses configured frontend base + /bookings/{id}. Critical content: DECLINED, no payment taken. Escape user content; no keys/password/full IBAN; no false money claim. Token in verification/reset link is the intentional exception to no-secrets-in-mail.

Expected database/domain state: Notification does not change booking, refund, payout, trust state or amount; committed lifecycle is authoritative.

Expected email/provider behavior: LOCAL CONTENT TEST only. STAGING DELIVERY TEST is PROVIDER Resend: actual sender/domain, inbox, correct link and delivery evidence.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: AccountEmailNotificationService, BookingNotificationService, AvailabilityInquiryNotificationService, TrustNotificationService

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

EMAIL-007 — Request expired

Area: EMAIL · **Priority:** P1 · **Test level:** UNIT, INTEGRATION · **Result:** NOT RUN

Roles involved: scout contact snapshot

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Use corresponding domain scenario to commit Request expired; capture EmailService.send arguments with local mock (do not enable real delivery). Compare text and HTML with fixture using Unicode and HTML-like name/reason. Replay lifecycle and roll back an independent event. Repeat actual delivery only under PROVIDER Resend scenario.

Expected UI behavior: Open captured CTA against matching frontend route under authorized test account; login preserves useful context.

Expected API/backend behavior: Recipient=scout contact snapshot; subject exactly “Your booking request expired”; CTA uses configured frontend base + /bookings/{id}. Critical content: No response by deadline; EXPIRED, not owner decline. Escape user content; no keys/password/full IBAN; no false money claim. Token in verification/reset link is the intentional exception to no-secrets-in-mail.

Expected database/domain state: Notification does not change booking, refund, payout, trust state or amount; committed lifecycle is authoritative.

Expected email/provider behavior: LOCAL CONTENT TEST only. STAGING DELIVERY TEST is PROVIDER Resend: actual sender/domain, inbox, correct link and delivery evidence.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: AccountEmailNotificationService, BookingNotificationService, AvailabilityInquiryNotificationService, TrustNotificationService

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

EMAIL-008 — Request unavailable

Area: EMAIL · **Priority:** P1 · **Test level:** UNIT, INTEGRATION · **Result:** NOT RUN

Roles involved: scout contact snapshot

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Use corresponding domain scenario to commit Request unavailable; capture EmailService.send arguments with local mock (do not enable real delivery). Compare text and HTML with fixture using Unicode and HTML-like name/reason. Replay lifecycle and roll back an independent event. Repeat actual delivery only under PROVIDER Resend scenario.

Expected UI behavior: Open captured CTA against matching frontend route under authorized test account; login preserves useful context.

Expected API/backend behavior: Recipient=scout contact snapshot; subject exactly “The requested time is no longer available”; CTA uses configured frontend base + /bookings/{id}. Critical content: Another reservation; UNAVAILABLE, not deliberate decline. Escape user content; no keys/password/full IBAN; no false money claim. Token in verification/reset link is the intentional exception to no-secrets-in-mail.

Expected database/domain state: Notification does not change booking, refund, payout, trust state or amount; committed lifecycle is authoritative.

Expected email/provider behavior: LOCAL CONTENT TEST only. STAGING DELIVERY TEST is PROVIDER Resend: actual sender/domain, inbox, correct link and delivery evidence.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: AccountEmailNotificationService, BookingNotificationService, AvailabilityInquiryNotificationService, TrustNotificationService

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

EMAIL-009 — Payment window expired

Area: EMAIL · **Priority:** P1 · **Test level:** UNIT, INTEGRATION · **Result:** NOT RUN

Roles involved: scout contact snapshot

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Use corresponding domain scenario to commit Payment window expired; capture EmailService.send arguments with local mock (do not enable real delivery). Compare text and HTML with fixture using Unicode and HTML-like name/reason. Replay lifecycle and roll back an independent event. Repeat actual delivery only under PROVIDER Resend scenario.

Expected UI behavior: Open captured CTA against matching frontend route under authorized test account; login preserves useful context.

Expected API/backend behavior: Recipient=scout contact snapshot; subject exactly “Your payment window expired”; CTA uses configured frontend base + /bookings/{id}. Critical content: Slot released, not confirmed. Escape user content; no keys/password/full IBAN; no false money claim. Token in verification/reset link is the intentional exception to no-secrets-in-mail.

Expected database/domain state: Notification does not change booking, refund, payout, trust state or amount; committed lifecycle is authoritative.

Expected email/provider behavior: LOCAL CONTENT TEST only. STAGING DELIVERY TEST is PROVIDER Resend: actual sender/domain, inbox, correct link and delivery evidence.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: AccountEmailNotificationService, BookingNotificationService, AvailabilityInquiryNotificationService, TrustNotificationService

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

EMAIL-010 — Payment confirmed scout

Area: EMAIL · **Priority:** P1 · **Test level:** UNIT, INTEGRATION · **Result:** NOT RUN

Roles involved: scout contact snapshot

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Use corresponding domain scenario to commit Payment confirmed scout; capture EmailService.send arguments with local mock (do not enable real delivery). Compare text and HTML with fixture using Unicode and HTML-like name/reason. Replay lifecycle and roll back an independent event. Repeat actual delivery only under PROVIDER Resend scenario.

Expected UI behavior: Open captured CTA against matching frontend route under authorized test account; login preserves useful context.

Expected API/backend behavior: Recipient=scout contact snapshot; subject exactly “Your Eema booking is confirmed”; CTA uses configured frontend base + /bookings/{id}. Critical content: Confirmed paid snapshot and correct date/time/contact. Escape user content; no keys/password/full IBAN; no false money claim. Token in verification/reset link is the intentional exception to no-secrets-in-mail.

Expected database/domain state: Notification does not change booking, refund, payout, trust state or amount; committed lifecycle is authoritative.

Expected email/provider behavior: LOCAL CONTENT TEST only. STAGING DELIVERY TEST is PROVIDER Resend: actual sender/domain, inbox, correct link and delivery evidence.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: AccountEmailNotificationService, BookingNotificationService, AvailabilityInquiryNotificationService, TrustNotificationService

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

EMAIL-011 — Payment confirmed owner

Area: EMAIL · **Priority:** P1 · **Test level:** UNIT, INTEGRATION · **Result:** NOT RUN

Roles involved: owner contact snapshot

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Use corresponding domain scenario to commit Payment confirmed owner; capture EmailService.send arguments with local mock (do not enable real delivery). Compare text and HTML with fixture using Unicode and HTML-like name/reason. Replay lifecycle and roll back an independent event. Repeat actual delivery only under PROVIDER Resend scenario.

Expected UI behavior: Open captured CTA against matching frontend route under authorized test account; login preserves useful context.

Expected API/backend behavior: Recipient=owner contact snapshot; subject exactly “Confirmed booking for your Eema venue”; CTA uses configured frontend base + /bookings/{id}. Critical content: Confirmed booking, no claim payout already transferred. Escape user content; no keys/password/full IBAN; no false money claim. Token in verification/reset link is the intentional exception to no-secrets-in-mail.

Expected database/domain state: Notification does not change booking, refund, payout, trust state or amount; committed lifecycle is authoritative.

Expected email/provider behavior: LOCAL CONTENT TEST only. STAGING DELIVERY TEST is PROVIDER Resend: actual sender/domain, inbox, correct link and delivery evidence.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: AccountEmailNotificationService, BookingNotificationService, AvailabilityInquiryNotificationService, TrustNotificationService

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

EMAIL-012 — Late payment

Area: EMAIL · **Priority:** P1 · **Test level:** UNIT, INTEGRATION · **Result:** NOT RUN

Roles involved: scout contact snapshot

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Use corresponding domain scenario to commit Late payment; capture EmailService.send arguments with local mock (do not enable real delivery). Compare text and HTML with fixture using Unicode and HTML-like name/reason. Replay lifecycle and roll back an independent event. Repeat actual delivery only under PROVIDER Resend scenario.

Expected UI behavior: Open captured CTA against matching frontend route under authorized test account; login preserves useful context.

Expected API/backend behavior: Recipient=scout contact snapshot; subject exactly “Your late Eema payment is being refunded”; CTA uses configured frontend base + /bookings/{id}. Critical content: EXPIRED/not confirmed, full refund initiated not completed. Escape user content; no keys/password/full IBAN; no false money claim. Token in verification/reset link is the intentional exception to no-secrets-in-mail.

Expected database/domain state: Notification does not change booking, refund, payout, trust state or amount; committed lifecycle is authoritative.

Expected email/provider behavior: LOCAL CONTENT TEST only. STAGING DELIVERY TEST is PROVIDER Resend: actual sender/domain, inbox, correct link and delivery evidence.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: AccountEmailNotificationService, BookingNotificationService, AvailabilityInquiryNotificationService, TrustNotificationService

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

EMAIL-013 — Cancellation scout notification

Area: EMAIL · **Priority:** P1 · **Test level:** UNIT, INTEGRATION · **Result:** NOT RUN

Roles involved: scout contact snapshot for all cancellation actors

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Use corresponding domain scenario to commit Cancellation scout notification; capture EmailService.send arguments with local mock (do not enable real delivery). Compare text and HTML with fixture using Unicode and HTML-like name/reason. Replay lifecycle and roll back an independent event. Repeat actual delivery only under PROVIDER Resend scenario.

Expected UI behavior: Open captured CTA against matching frontend route under authorized test account; login preserves useful context.

Expected API/backend behavior: Recipient=scout contact snapshot for all cancellation actors; subject exactly “Your Eema booking was cancelled”; CTA uses configured frontend base + /bookings/{id}. Critical content: Refund initiated versus no refund due; correct owner/admin/scout semantics. Escape user content; no keys/password/full IBAN; no false money claim. Token in verification/reset link is the intentional exception to no-secrets-in-mail.

Expected database/domain state: Notification does not change booking, refund, payout, trust state or amount; committed lifecycle is authoritative.

Expected email/provider behavior: LOCAL CONTENT TEST only. STAGING DELIVERY TEST is PROVIDER Resend: actual sender/domain, inbox, correct link and delivery evidence.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: AccountEmailNotificationService, BookingNotificationService, AvailabilityInquiryNotificationService, TrustNotificationService

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

EMAIL-014 — Scout cancellation owner notification

Area: EMAIL · **Priority:** P1 · **Test level:** UNIT, INTEGRATION · **Result:** NOT RUN

Roles involved: owner contact snapshot only when SCOUT cancelled

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Use corresponding domain scenario to commit Scout cancellation owner notification; capture EmailService.send arguments with local mock (do not enable real delivery). Compare text and HTML with fixture using Unicode and HTML-like name/reason. Replay lifecycle and roll back an independent event. Repeat actual delivery only under PROVIDER Resend scenario.

Expected UI behavior: Open captured CTA against matching frontend route under authorized test account; login preserves useful context.

Expected API/backend behavior: Recipient=owner contact snapshot only when SCOUT cancelled; subject exactly “A scout cancelled an Eema booking”; CTA uses configured frontend base + /bookings/{id}. Critical content: Settlement provisional pending fee reconciliation. Escape user content; no keys/password/full IBAN; no false money claim. Token in verification/reset link is the intentional exception to no-secrets-in-mail.

Expected database/domain state: Notification does not change booking, refund, payout, trust state or amount; committed lifecycle is authoritative.

Expected email/provider behavior: LOCAL CONTENT TEST only. STAGING DELIVERY TEST is PROVIDER Resend: actual sender/domain, inbox, correct link and delivery evidence.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: AccountEmailNotificationService, BookingNotificationService, AvailabilityInquiryNotificationService, TrustNotificationService

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

EMAIL-015 — Cancellation refund completed

Area: EMAIL · **Priority:** P1 · **Test level:** UNIT, INTEGRATION · **Result:** NOT RUN

Roles involved: scout contact snapshot

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Use corresponding domain scenario to commit Cancellation refund completed; capture EmailService.send arguments with local mock (do not enable real delivery). Compare text and HTML with fixture using Unicode and HTML-like name/reason. Replay lifecycle and roll back an independent event. Repeat actual delivery only under PROVIDER Resend scenario.

Expected UI behavior: Open captured CTA against matching frontend route under authorized test account; login preserves useful context.

Expected API/backend behavior: Recipient=scout contact snapshot; subject exactly “Your Eema refund has been processed”; CTA uses configured frontend base + /bookings/{id}. Critical content: Authoritative REFUNDED provider status; bank settlement timing may vary. Escape user content; no keys/password/full IBAN; no false money claim. Token in verification/reset link is the intentional exception to no-secrets-in-mail.

Expected database/domain state: Notification does not change booking, refund, payout, trust state or amount; committed lifecycle is authoritative.

Expected email/provider behavior: LOCAL CONTENT TEST only. STAGING DELIVERY TEST is PROVIDER Resend: actual sender/domain, inbox, correct link and delivery evidence.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: AccountEmailNotificationService, BookingNotificationService, AvailabilityInquiryNotificationService, TrustNotificationService

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

EMAIL-016 — Inquiry created

Area: EMAIL · **Priority:** P1 · **Test level:** UNIT, INTEGRATION · **Result:** NOT RUN

Roles involved: current venue owner account

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Use corresponding domain scenario to commit Inquiry created; capture EmailService.send arguments with local mock (do not enable real delivery). Compare text and HTML with fixture using Unicode and HTML-like name/reason. Replay lifecycle and roll back an independent event. Repeat actual delivery only under PROVIDER Resend scenario.

Expected UI behavior: Open captured CTA against matching frontend route under authorized test account; login preserves useful context.

Expected API/backend behavior: Recipient=current venue owner account; subject exactly “New availability inquiry”; CTA uses configured frontend base + /owner/inquiries. Critical content: Requested date/time, scout and message; not booking confirmation. Escape user content; no keys/password/full IBAN; no false money claim. Token in verification/reset link is the intentional exception to no-secrets-in-mail.

Expected database/domain state: Notification does not change booking, refund, payout, trust state or amount; committed lifecycle is authoritative.

Expected email/provider behavior: LOCAL CONTENT TEST only. STAGING DELIVERY TEST is PROVIDER Resend: actual sender/domain, inbox, correct link and delivery evidence.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: AccountEmailNotificationService, BookingNotificationService, AvailabilityInquiryNotificationService, TrustNotificationService

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

EMAIL-017 — Inquiry offered

Area: EMAIL · **Priority:** P1 · **Test level:** UNIT, INTEGRATION · **Result:** NOT RUN

Roles involved: current scout account

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Use corresponding domain scenario to commit Inquiry offered; capture EmailService.send arguments with local mock (do not enable real delivery). Compare text and HTML with fixture using Unicode and HTML-like name/reason. Replay lifecycle and roll back an independent event. Repeat actual delivery only under PROVIDER Resend scenario.

Expected UI behavior: Open captured CTA against matching frontend route under authorized test account; login preserves useful context.

Expected API/backend behavior: Recipient=current scout account; subject exactly “The owner offered you a time”; CTA uses configured frontend base + /scout/inquiries. Critical content: Offered/original intervals, timezone, expiry, Review offer CTA. Escape user content; no keys/password/full IBAN; no false money claim. Token in verification/reset link is the intentional exception to no-secrets-in-mail.

Expected database/domain state: Notification does not change booking, refund, payout, trust state or amount; committed lifecycle is authoritative.

Expected email/provider behavior: LOCAL CONTENT TEST only. STAGING DELIVERY TEST is PROVIDER Resend: actual sender/domain, inbox, correct link and delivery evidence.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: AccountEmailNotificationService, BookingNotificationService, AvailabilityInquiryNotificationService, TrustNotificationService

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

EMAIL-018 — Inquiry declined

Area: EMAIL · **Priority:** P1 · **Test level:** UNIT, INTEGRATION · **Result:** NOT RUN

Roles involved: current scout account

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Use corresponding domain scenario to commit Inquiry declined; capture EmailService.send arguments with local mock (do not enable real delivery). Compare text and HTML with fixture using Unicode and HTML-like name/reason. Replay lifecycle and roll back an independent event. Repeat actual delivery only under PROVIDER Resend scenario.

Expected UI behavior: Open captured CTA against matching frontend route under authorized test account; login preserves useful context.

Expected API/backend behavior: Recipient=current scout account; subject exactly “Your availability inquiry was declined”; CTA uses configured frontend base + /scout/inquiries. Critical content: No booking/payment created. Escape user content; no keys/password/full IBAN; no false money claim. Token in verification/reset link is the intentional exception to no-secrets-in-mail.

Expected database/domain state: Notification does not change booking, refund, payout, trust state or amount; committed lifecycle is authoritative.

Expected email/provider behavior: LOCAL CONTENT TEST only. STAGING DELIVERY TEST is PROVIDER Resend: actual sender/domain, inbox, correct link and delivery evidence.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: AccountEmailNotificationService, BookingNotificationService, AvailabilityInquiryNotificationService, TrustNotificationService

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

EMAIL-019 — Inquiry offer expired

Area: EMAIL · **Priority:** P1 · **Test level:** UNIT, INTEGRATION · **Result:** NOT RUN

Roles involved: current scout account

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Use corresponding domain scenario to commit Inquiry offer expired; capture EmailService.send arguments with local mock (do not enable real delivery). Compare text and HTML with fixture using Unicode and HTML-like name/reason. Replay lifecycle and roll back an independent event. Repeat actual delivery only under PROVIDER Resend scenario.

Expected UI behavior: Open captured CTA against matching frontend route under authorized test account; login preserves useful context.

Expected API/backend behavior: Recipient=current scout account; subject exactly “Your availability offer expired”; CTA uses configured frontend base + /scout/inquiries. Critical content: One-off offer no longer accept-able. Escape user content; no keys/password/full IBAN; no false money claim. Token in verification/reset link is the intentional exception to no-secrets-in-mail.

Expected database/domain state: Notification does not change booking, refund, payout, trust state or amount; committed lifecycle is authoritative.

Expected email/provider behavior: LOCAL CONTENT TEST only. STAGING DELIVERY TEST is PROVIDER Resend: actual sender/domain, inbox, correct link and delivery evidence.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: AccountEmailNotificationService, BookingNotificationService, AvailabilityInquiryNotificationService, TrustNotificationService

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

EMAIL-020 — Owner verification submitted

Area: EMAIL · **Priority:** P1 · **Test level:** UNIT, INTEGRATION · **Result:** NOT RUN

Roles involved: owner account

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Use corresponding domain scenario to commit Owner verification submitted; capture EmailService.send arguments with local mock (do not enable real delivery). Compare text and HTML with fixture using Unicode and HTML-like name/reason. Replay lifecycle and roll back an independent event. Repeat actual delivery only under PROVIDER Resend scenario.

Expected UI behavior: Open captured CTA against matching frontend route under authorized test account; login preserves useful context.

Expected API/backend behavior: Recipient=owner account; subject exactly “Owner verification submitted”; CTA uses configured frontend base + /owner/verification. Critical content: Manual review pending. Escape user content; no keys/password/full IBAN; no false money claim. Token in verification/reset link is the intentional exception to no-secrets-in-mail.

Expected database/domain state: Notification does not change booking, refund, payout, trust state or amount; committed lifecycle is authoritative.

Expected email/provider behavior: LOCAL CONTENT TEST only. STAGING DELIVERY TEST is PROVIDER Resend: actual sender/domain, inbox, correct link and delivery evidence.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: AccountEmailNotificationService, BookingNotificationService, AvailabilityInquiryNotificationService, TrustNotificationService

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

EMAIL-021 — Owner verification approved

Area: EMAIL · **Priority:** P1 · **Test level:** UNIT, INTEGRATION · **Result:** NOT RUN

Roles involved: owner account

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Use corresponding domain scenario to commit Owner verification approved; capture EmailService.send arguments with local mock (do not enable real delivery). Compare text and HTML with fixture using Unicode and HTML-like name/reason. Replay lifecycle and roll back an independent event. Repeat actual delivery only under PROVIDER Resend scenario.

Expected UI behavior: Open captured CTA against matching frontend route under authorized test account; login preserves useful context.

Expected API/backend behavior: Recipient=owner account; subject exactly “Your Eema owner verification was approved”; CTA uses configured frontend base + /owner/verification. Critical content: Can submit ready venue; not automatic bank verification. Escape user content; no keys/password/full IBAN; no false money claim. Token in verification/reset link is the intentional exception to no-secrets-in-mail.

Expected database/domain state: Notification does not change booking, refund, payout, trust state or amount; committed lifecycle is authoritative.

Expected email/provider behavior: LOCAL CONTENT TEST only. STAGING DELIVERY TEST is PROVIDER Resend: actual sender/domain, inbox, correct link and delivery evidence.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: AccountEmailNotificationService, BookingNotificationService, AvailabilityInquiryNotificationService, TrustNotificationService

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

EMAIL-022 — Owner verification rejected

Area: EMAIL · **Priority:** P1 · **Test level:** UNIT, INTEGRATION · **Result:** NOT RUN

Roles involved: owner account

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Use corresponding domain scenario to commit Owner verification rejected; capture EmailService.send arguments with local mock (do not enable real delivery). Compare text and HTML with fixture using Unicode and HTML-like name/reason. Replay lifecycle and roll back an independent event. Repeat actual delivery only under PROVIDER Resend scenario.

Expected UI behavior: Open captured CTA against matching frontend route under authorized test account; login preserves useful context.

Expected API/backend behavior: Recipient=owner account; subject exactly “Your Eema owner verification needs attention”; CTA uses configured frontend base + /owner/verification. Critical content: Safe owner-visible reason. Escape user content; no keys/password/full IBAN; no false money claim. Token in verification/reset link is the intentional exception to no-secrets-in-mail.

Expected database/domain state: Notification does not change booking, refund, payout, trust state or amount; committed lifecycle is authoritative.

Expected email/provider behavior: LOCAL CONTENT TEST only. STAGING DELIVERY TEST is PROVIDER Resend: actual sender/domain, inbox, correct link and delivery evidence.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: AccountEmailNotificationService, BookingNotificationService, AvailabilityInquiryNotificationService, TrustNotificationService

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

EMAIL-023 — Venue submitted

Area: EMAIL · **Priority:** P1 · **Test level:** UNIT, INTEGRATION · **Result:** NOT RUN

Roles involved: venue owner account

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Use corresponding domain scenario to commit Venue submitted; capture EmailService.send arguments with local mock (do not enable real delivery). Compare text and HTML with fixture using Unicode and HTML-like name/reason. Replay lifecycle and roll back an independent event. Repeat actual delivery only under PROVIDER Resend scenario.

Expected UI behavior: Open captured CTA against matching frontend route under authorized test account; login preserves useful context.

Expected API/backend behavior: Recipient=venue owner account; subject exactly “Venue submitted for review”; CTA uses configured frontend base + /owner/venues/{id}. Critical content: PENDING not live. Escape user content; no keys/password/full IBAN; no false money claim. Token in verification/reset link is the intentional exception to no-secrets-in-mail.

Expected database/domain state: Notification does not change booking, refund, payout, trust state or amount; committed lifecycle is authoritative.

Expected email/provider behavior: LOCAL CONTENT TEST only. STAGING DELIVERY TEST is PROVIDER Resend: actual sender/domain, inbox, correct link and delivery evidence.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: AccountEmailNotificationService, BookingNotificationService, AvailabilityInquiryNotificationService, TrustNotificationService

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

EMAIL-024 — Venue approved

Area: EMAIL · **Priority:** P1 · **Test level:** UNIT, INTEGRATION · **Result:** NOT RUN

Roles involved: venue owner account

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Use corresponding domain scenario to commit Venue approved; capture EmailService.send arguments with local mock (do not enable real delivery). Compare text and HTML with fixture using Unicode and HTML-like name/reason. Replay lifecycle and roll back an independent event. Repeat actual delivery only under PROVIDER Resend scenario.

Expected UI behavior: Open captured CTA against matching frontend route under authorized test account; login preserves useful context.

Expected API/backend behavior: Recipient=venue owner account; subject exactly “Your venue is live”; CTA uses configured frontend base + /owner/venues/{id}. Critical content: ACTIVE. Escape user content; no keys/password/full IBAN; no false money claim. Token in verification/reset link is the intentional exception to no-secrets-in-mail.

Expected database/domain state: Notification does not change booking, refund, payout, trust state or amount; committed lifecycle is authoritative.

Expected email/provider behavior: LOCAL CONTENT TEST only. STAGING DELIVERY TEST is PROVIDER Resend: actual sender/domain, inbox, correct link and delivery evidence.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: AccountEmailNotificationService, BookingNotificationService, AvailabilityInquiryNotificationService, TrustNotificationService

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

EMAIL-025 — Venue changes requested

Area: EMAIL · **Priority:** P1 · **Test level:** UNIT, INTEGRATION · **Result:** NOT RUN

Roles involved: venue owner account

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Use corresponding domain scenario to commit Venue changes requested; capture EmailService.send arguments with local mock (do not enable real delivery). Compare text and HTML with fixture using Unicode and HTML-like name/reason. Replay lifecycle and roll back an independent event. Repeat actual delivery only under PROVIDER Resend scenario.

Expected UI behavior: Open captured CTA against matching frontend route under authorized test account; login preserves useful context.

Expected API/backend behavior: Recipient=venue owner account; subject exactly “Changes requested for your venue”; CTA uses configured frontend base + /owner/venues/{id}. Critical content: DRAFT, safe reason. Escape user content; no keys/password/full IBAN; no false money claim. Token in verification/reset link is the intentional exception to no-secrets-in-mail.

Expected database/domain state: Notification does not change booking, refund, payout, trust state or amount; committed lifecycle is authoritative.

Expected email/provider behavior: LOCAL CONTENT TEST only. STAGING DELIVERY TEST is PROVIDER Resend: actual sender/domain, inbox, correct link and delivery evidence.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: AccountEmailNotificationService, BookingNotificationService, AvailabilityInquiryNotificationService, TrustNotificationService

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

EMAIL-026 — Venue suspended

Area: EMAIL · **Priority:** P1 · **Test level:** UNIT, INTEGRATION · **Result:** NOT RUN

Roles involved: venue owner account

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Use corresponding domain scenario to commit Venue suspended; capture EmailService.send arguments with local mock (do not enable real delivery). Compare text and HTML with fixture using Unicode and HTML-like name/reason. Replay lifecycle and roll back an independent event. Repeat actual delivery only under PROVIDER Resend scenario.

Expected UI behavior: Open captured CTA against matching frontend route under authorized test account; login preserves useful context.

Expected API/backend behavior: Recipient=venue owner account; subject exactly “Your venue was suspended”; CTA uses configured frontend base + /owner/venues/{id}. Critical content: No longer publicly bookable; no claim existing bookings cancelled. Escape user content; no keys/password/full IBAN; no false money claim. Token in verification/reset link is the intentional

exception to no-secrets-in-mail.

Expected database/domain state: Notification does not change booking, refund, payout, trust state or amount; committed lifecycle is authoritative.

Expected email/provider behavior: LOCAL CONTENT TEST only. STAGING DELIVERY TEST is PROVIDER Resend: actual sender/domain, inbox, correct link and delivery evidence.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: AccountEmailNotificationService, BookingNotificationService, AvailabilityInquiryNotificationService, TrustNotificationService

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

EMAIL-027 — Venue reactivated

Area: EMAIL · **Priority:** P1 · **Test level:** UNIT, INTEGRATION · **Result:** NOT RUN

Roles involved: venue owner account

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Use corresponding domain scenario to commit Venue reactivated; capture EmailService.send arguments with local mock (do not enable real delivery). Compare text and HTML with fixture using Unicode and HTML-like name/reason. Replay lifecycle and roll back an independent event. Repeat actual delivery only under PROVIDER Resend scenario.

Expected UI behavior: Open captured CTA against matching frontend route under authorized test account; login preserves useful context.

Expected API/backend behavior: Recipient=venue owner account; subject exactly “Your venue is live again”; CTA uses configured frontend base + /owner/venues/{id}. Critical content: ACTIVE after revalidation. Escape user content; no keys/password/full IBAN; no false money claim. Token in verification/reset link is the intentional exception to no-secrets-in-mail.

Expected database/domain state: Notification does not change booking, refund, payout, trust state or amount; committed lifecycle is authoritative.

Expected email/provider behavior: LOCAL CONTENT TEST only. STAGING DELIVERY TEST is PROVIDER Resend: actual sender/domain, inbox, correct link and delivery evidence.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: AccountEmailNotificationService, BookingNotificationService, AvailabilityInquiryNotificationService, TrustNotificationService

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

EMAIL-028 — Destination submitted

Area: EMAIL · **Priority:** P1 · **Test level:** UNIT, INTEGRATION · **Result:** NOT RUN

Roles involved: owner account

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Use corresponding domain scenario to commit Destination submitted; capture EmailService.send arguments with local mock (do not enable real delivery). Compare text and HTML with fixture using Unicode and HTML-like name/reason. Replay lifecycle and roll back an independent event. Repeat actual delivery only under PROVIDER Resend scenario.

Expected UI behavior: Open captured CTA against matching frontend route under authorized test account; login preserves useful context.

Expected API/backend behavior: Recipient=owner account; subject exactly “Payout destination submitted”; CTA uses configured frontend base + /owner/payout-destination. Critical content: Pending manual review; no full IBAN. Escape user content; no keys/password/full IBAN; no false money claim. Token in verification/reset link is the intentional exception to no-secrets-in-mail.

Expected database/domain state: Notification does not change booking, refund, payout, trust state or amount; committed lifecycle is authoritative.

Expected email/provider behavior: LOCAL CONTENT TEST only. STAGING DELIVERY TEST is PROVIDER Resend: actual sender/domain, inbox, correct link and delivery evidence.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: AccountEmailNotificationService, BookingNotificationService, AvailabilityInquiryNotificationService, TrustNotificationService

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

EMAIL-029 — Destination verified

Area: EMAIL · **Priority:** P1 · **Test level:** UNIT, INTEGRATION · **Result:** NOT RUN

Roles involved: owner account

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Use corresponding domain scenario to commit Destination verified; capture EmailService.send arguments with local mock (do not enable real delivery). Compare text and HTML with fixture using Unicode and HTML-like name/reason. Replay lifecycle and roll back an independent event. Repeat actual delivery only under PROVIDER Resend scenario.

Expected UI behavior: Open captured CTA against matching frontend route under authorized test account; login preserves useful context.

Expected API/backend behavior: Recipient=owner account; subject exactly “Payout destination verified”; CTA uses configured frontend base + /owner/payout-destination. Critical content: May be used for future eligible payouts, not paid. Escape user content; no keys/password/full IBAN; no false money claim. Token in verification/reset link is the intentional exception to no-secrets-in-mail.

Expected database/domain state: Notification does not change booking, refund, payout, trust state or amount; committed lifecycle is authoritative.

Expected email/provider behavior: LOCAL CONTENT TEST only. STAGING DELIVERY TEST is PROVIDER Resend: actual sender/domain, inbox, correct link and delivery evidence.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: AccountEmailNotificationService, BookingNotificationService, AvailabilityInquiryNotificationService, TrustNotificationService

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

EMAIL-030 — Destination rejected

Area: EMAIL · **Priority:** P1 · **Test level:** UNIT, INTEGRATION · **Result:** NOT RUN

Roles involved: owner account

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Use corresponding domain scenario to commit Destination rejected; capture EmailService.send arguments with local mock (do not enable real delivery). Compare text and HTML with fixture using Unicode and HTML-like name/reason. Replay lifecycle and roll back an independent event. Repeat actual delivery only under PROVIDER Resend scenario.

Expected UI behavior: Open captured CTA against matching frontend route under authorized test account; login preserves useful context.

Expected API/backend behavior: Recipient=owner account; subject exactly “Your payout destination needs attention”; CTA uses configured frontend base + /owner/payout-destination. Critical content: Safe reason, no full IBAN. Escape user content; no keys/password/full IBAN; no false money claim. Token in verification/reset link is the intentional exception to no-secrets-in-mail.

Expected database/domain state: Notification does not change booking, refund, payout, trust state or amount; committed lifecycle is authoritative.

Expected email/provider behavior: LOCAL CONTENT TEST only. STAGING DELIVERY TEST is PROVIDER Resend: actual sender/domain, inbox, correct link and delivery evidence.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: AccountEmailNotificationService, BookingNotificationService, AvailabilityInquiryNotificationService, TrustNotificationService

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

EMAIL-031 — Unsupported notification events remain explicit gaps

Area: EMAIL · **Priority:** P2 · **Test level:** INTEGRATION · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Trigger admin full refund initiated/completed, late-refund completion, owner/admin cancellation owner-side notification, destination replacement/reveal/snapshot, incident transition and payout mark-paid; capture email calls.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: No dedicated lifecycle email is currently implemented for these events. Owner gets cancellation email only for SCOUT cancellation; incidents have no email. Do not invent delivery requirements as current behavior; business owner decides needed communications.

Expected database/domain state: No dedicated lifecycle email is currently implemented for these events. Owner gets cancellation email only for SCOUT cancellation; incidents have no email. Do not invent delivery requirements as current behavior; business owner decides needed communications.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: Event listeners and notification service branches

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

JOURNEY scenarios

JOURNEY-001 — Scout happy path

Area: JOURNEY · **Priority:** P0 · **Test level:** LOCAL E2E, STAGING E2E, MANUAL UX · **Result:** NOT RUN

Roles involved: Scout A, Scout B, Owner A, Owner B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: 1. Register Scout A on /. 2. Capture local verification link and verify. 3. Review/accept five policies at /policies. 4. Browse or match Studio. 5. Open venue detail on / and availability. 6. Request D14 09:15–13:45. 7. Switch to Owner A /owner/bookings and accept. 8. Return Scout A /bookings/{id}, Pay now. 9. Apply mocked paid webhook locally or approved Mollie TEST staging checkout. 10. /booked authoritative CONFIRMED. 11. Download receipt and reopen /scout/bookings.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: REQUESTED→AWAITING_PAYMENT→CONFIRMED once; exact price/policy/contact snapshots; receipt private and immutable.

Expected database/domain state: REQUESTED→AWAITING_PAYMENT→CONFIRMED once; exact price/policy/contact snapshots; receipt private and immutable.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: Cross-domain journey; domain cases and endpoint matrix are mandatory subchecks

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

JOURNEY-002 — Owner happy path

Area: JOURNEY · **Priority:** P0 · **Test level:** LOCAL E2E, STAGING E2E, MANUAL UX · **Result:** NOT RUN

Roles involved: Scout A, Scout B, Owner A, Owner B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: 1. Register Owner A and verify email. 2. Accept current policies. 3. /owner/verification submit. 4. Admin approves manually. 5. /owner/payout-destination submit synthetic bank; Admin verifies. 6. /owner/venues/new create DRAFT. 7. Complete profile/photos/timezone/weekly availability. 8. Submit review; Admin approves. 9. Receive Scout request and accept. 10. Scout pays and backend confirms. 11. Advance ended fixture, reconcile mocked actual fee, run readiness. 12. Admin records separately simulated external transfer. 13. Owner sees PAID and downloads statement.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Trust gates respected; READY never presented as paid; statement only after recorded external transfer. No real money sent.

Expected database/domain state: Trust gates respected; READY never presented as paid; statement only after recorded external transfer. No real money sent.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: Cross-domain journey; domain cases and endpoint matrix are mandatory subchecks

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

JOURNEY-003 — Admin happy path

Area: JOURNEY · **Priority:** P0 · **Test level:** LOCAL E2E, STAGING E2E, MANUAL UX · **Result:** NOT RUN

Roles involved: Scout A, Scout B, Owner A, Owner B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: 1. Login pre-provisioned Admin. 2. Open every Operations card. 3. Review owner, venue and payout destination. 4. Inspect exception queue, booking detail/event timeline. 5. Start/resolve/close participant incident. 6. Mark READY payout paid with QA reference. 7. Settle OPEN liability. 8. Draft/publish test-only replacement policy in isolated local fixture.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: All admin transitions audited/scoped, no generic editing, external transfer is only recorded; policies not published as legal content to staging/production.

Expected database/domain state: All admin transitions audited/scoped, no generic editing, external transfer is only recorded; policies not published as legal content to staging/production.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: Cross-domain journey; domain cases and endpoint matrix are mandatory subchecks

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

JOURNEY-004 — Cancellation journey matrix

Area: JOURNEY · **Priority:** P0 · **Test level:** LOCAL E2E, STAGING E2E, MANUAL UX · **Result:** NOT RUN

Roles involved: Scout A, Scout B, Owner A, Owner B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Repeat full Scout path on five independent paid bookings: scout full refund, scout partial, scout zero, owner with reason, admin with reason. Preview then cancel; mock each provider completion; inspect both histories, ledger and receipt.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: 100/50/0 scout economics at documented boundaries; owner/admin full refund; CANCELLED with separate refund status; owner cost responsibility differs from admin.

Expected database/domain state: 100/50/0 scout economics at documented boundaries; owner/admin full refund; CANCELLED with separate refund status; owner cost responsibility differs from admin.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: Cross-domain journey; domain cases and endpoint matrix are mandatory subchecks

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

JOURNEY-005 — Inquiry alternate-slot journey

Area: JOURNEY · **Priority:** P0 · **Test level:** LOCAL E2E, STAGING E2E, MANUAL UX · **Result:** NOT RUN

Roles involved: Scout A, Scout B, Owner A, Owner B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Scout / calendar unavailable day → inquiry; Owner /owner/inquiries offers alternate D15 18–20; Scout /scout/inquiries accepts; normal /bookings/{id} checkout; apply paid webhook.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: One ACCEPTED inquiry linked to one AWAITING_PAYMENT→CONFIRMED booking; no permanent availability expansion.

Expected database/domain state: One ACCEPTED inquiry linked to one AWAITING_PAYMENT→CONFIRMED booking; no permanent availability expansion.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: Cross-domain journey; domain cases and endpoint matrix are mandatory subchecks

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

JOURNEY-006 — Competing scouts journey

Area: JOURNEY · **Priority:** P0 · **Test level:** LOCAL E2E, STAGING E2E, MANUAL UX · **Result:** NOT RUN

Roles involved: Scout A, Scout B, Owner A, Owner B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Scout A and B submit overlapping requests; Owner accepts A; B reloads booking/history and attempts checkout/accept against stale data.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: A reserves; B UNAVAILABLE, correct non-decline copy and no checkout; notification once for loser.

Expected database/domain state: A reserves; B UNAVAILABLE, correct non-decline copy and no checkout; notification once for loser.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: Cross-domain journey; domain cases and endpoint matrix are mandatory subchecks

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

JOURNEY-007 — Suspension journey

Area: JOURNEY · **Priority:** P0 · **Test level:** LOCAL E2E, STAGING E2E, MANUAL UX · **Result:** NOT RUN

Roles involved: Scout A, Scout B, Owner A, Owner B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Create active venue with confirmed booking and separate requested booking. Admin suspends; anonymously browse and attempt new request; read confirmed history; characterize acceptance of old request; admin reactivates after readiness check.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: New requests/catalogue blocked; historic confirmed record intact; known old-request acceptance gap explicitly recorded; reactivation revalidates.

Expected database/domain state: New requests/catalogue blocked; historic confirmed record intact; known old-request acceptance gap explicitly recorded; reactivation revalidates.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: Cross-domain journey; domain cases and endpoint matrix are mandatory subchecks

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

JOURNEY-008 — Recovery journey

Area: JOURNEY · **Priority:** P0 · **Test level:** LOCAL E2E, STAGING E2E, MANUAL UX · **Result:** NOT RUN

Roles involved: Scout A, Scout B, Owner A, Owner B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Scout requests reset; reset password in separate profile; test old refresh and unexpired JWT; reload original browser; login new password and resume eligible transaction.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Reset invalidates refresh tokens/local session; old access JWT persists until expiry as known gap; transactional state survives account recovery.

Expected database/domain state: Reset invalidates refresh tokens/local session; old access JWT persists until expiry as known gap; transactional state survives account recovery.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: Cross-domain journey; domain cases and endpoint matrix are mandatory subchecks

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

JOURNEY-009 — Policy update journey

Area: JOURNEY · **Priority:** P0 · **Test level:** LOCAL E2E, STAGING E2E, MANUAL UX · **Result:** NOT RUN

Roles involved: Scout A, Scout B, Owner A, Owner B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Scout accepts v1 and creates booking; admin publishes test-only v2; read historical booking/receipt, attempt new action; review/accept v2 and retry.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Historical snapshots unchanged; gated transaction resumes after exact current acceptance; no invented edit of historical terms.

Expected database/domain state: Historical snapshots unchanged; gated transaction resumes after exact current acceptance; no invented edit of historical terms.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: Cross-domain journey; domain cases and endpoint matrix are mandatory subchecks

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

INPUT scenarios

INPUT-001 — AcceptPoliciesRequest all field boundaries

Area: INPUT · **Priority:** P1 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Exact request schema:

```
public record AcceptPoliciesRequest(@NotEmpty @Size(max=5) Set<@NotNull UUID> policyVersionIds){}
```

Exact steps: Use the matching endpoint in the matrix and a valid body from the fixture cookbook. For EACH field in the exact DTO declaration below, send one changed value: omitted, null, wrong JSON type, blank, valid min/max, min−1/max+1; lists empty/one/max/max+1 and null element; numeric decimal/negative/overflow; supported versus unknown enum. Repeat through corresponding form. Nested ScoutContact/weekly items must receive the same checks.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected database/domain state: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/dto/request/AcceptPoliciesRequest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

INPUT-002 — CancelBookingRequest all field boundaries

Area: INPUT · **Priority:** P1 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Exact request schema:

```
public record CancelBookingRequest(
        @Size(max = 1000) String reason
) {
}
```

Exact steps: Use the matching endpoint in the matrix and a valid body from the fixture cookbook. For EACH field in the exact DTO declaration below, send one changed value: omitted, null, wrong JSON type, blank, valid min/max, min−1/max+1; lists empty/one/max/max+1 and null element; numeric decimal/negative/overflow; supported versus unknown enum. Repeat through corresponding form. Nested ScoutContact/weekly items must receive the same checks.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected database/domain state: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/dto/request/CancelBookingRequest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

INPUT-003 — CreateAvailabilityInquiryRequest all field boundaries

Area: INPUT · **Priority:** P1 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Exact request schema:

```
public record CreateAvailabilityInquiryRequest(
        @NotNull UUID venueId,
        @NotNull LocalDate requestedDate,
        @NotNull LocalTime requestedStartTime,
        @NotNull LocalTime requestedEndTime,
        @NotBlank @Size(max = 1000) String message) {}
```

Exact steps: Use the matching endpoint in the matrix and a valid body from the fixture cookbook. For EACH field in the exact DTO declaration below, send one changed value: omitted, null, wrong JSON type, blank, valid min/max, min−1/max+1; lists empty/one/max/max+1 and null element; numeric decimal/negative/overflow; supported versus unknown enum. Repeat through corresponding form. Nested ScoutContact/weekly items must receive the same checks.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected database/domain state: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/dto/request/CreateAvailabilityInquiryRequest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

INPUT-004 — CreateAvailabilityOverrideRequest all field boundaries

Area: INPUT · **Priority:** P1 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Exact request schema:

```
public record CreateAvailabilityOverrideRequest(@NotNull LocalDate date, LocalTime startTime, LocalTime endTime,
                                                 @NotNull AvailabilityOverrideType type, boolean allDay) {}
```

Exact steps: Use the matching endpoint in the matrix and a valid body from the fixture cookbook. For EACH field in the exact DTO declaration below, send one changed value: omitted, null, wrong JSON type, blank, valid min/max, min−1/max+1; lists empty/one/max/max+1 and null element; numeric decimal/negative/overflow; supported versus unknown enum. Repeat through corresponding form. Nested ScoutContact/weekly items must receive the same checks.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected database/domain state: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/dto/request/CreateAvailabilityOverrideRequest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

INPUT-005 — CreateBookingRequest all field boundaries

Area: INPUT · **Priority:** P1 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Exact request schema:

```
public record CreateBookingRequest(

        @NotNull(message = "venueId is required")
        UUID venueId,

        @NotNull(message = "shootDate is required")
        LocalDate shootDate,

        @NotNull(message = "startTime is required")
        LocalTime startTime,

        @NotNull(message = "endTime is required")
        LocalTime endTime,

        @NotBlank(message = "rawBrief cannot be blank")
        @Size(max = 1000)
        String rawBrief,

        @Valid
        @NotNull(message = "scoutContact is required")
        ScoutContactRequest scoutContact
) {
}
```

Exact steps: Use the matching endpoint in the matrix and a valid body from the fixture cookbook. For EACH field in the exact DTO declaration below, send one changed value: omitted, null, wrong JSON type, blank, valid min/max, min−1/max+1; lists empty/one/max/max+1 and null element; numeric decimal/negative/overflow; supported versus unknown enum. Repeat through corresponding form. Nested ScoutContact/weekly items must receive the same checks.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount;

record accepted-ignore behavior rather than assume 400.

Expected database/domain state: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/dto/request/CreateBookingRequest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

INPUT-006 — CreateIncidentRequest all field boundaries

Area: INPUT · **Priority:** P1 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Exact request schema:

```
public record CreateIncidentRequest(@NotNull IncidentCategory category,@NotBlank @Size(max=4000) @Pattern(regexp="^[^<>]*$",message="must not contain markup") String description){}
```

Exact steps: Use the matching endpoint in the matrix and a valid body from the fixture cookbook. For EACH field in the exact DTO declaration below, send one changed value: omitted, null, wrong JSON type, blank, valid min/max, min−1/max+1; lists empty/one/max/max+1 and null element; numeric decimal/negative/overflow; supported versus unknown enum. Repeat through corresponding form. Nested ScoutContact/weekly items must receive the same checks.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected database/domain state: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/dto/request/CreateIncidentRequest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

INPUT-007 — CreatePayoutDestinationRequest all field boundaries

Area: INPUT · **Priority:** P1 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Exact request schema:

```
public record CreatePayoutDestinationRequest(@NotBlank @Size(max=160) String accountHolderName,@NotBlank @Size(max=80) String iban){}
```

Exact steps: Use the matching endpoint in the matrix and a valid body from the fixture cookbook. For EACH field in the exact DTO declaration below, send one changed value: omitted, null, wrong JSON type, blank, valid min/max, min−1/max+1; lists empty/one/max/max+1 and null element; numeric decimal/negative/overflow; supported versus unknown enum. Repeat through corresponding form. Nested ScoutContact/weekly items must receive the same checks.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected database/domain state: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/dto/request/CreatePayoutDestinationRequest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

INPUT-008 — CreatePolicyVersionRequest all field boundaries

Area: INPUT · **Priority:** P1 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Exact request schema:

```
public record CreatePolicyVersionRequest(@NotNull PolicyType policyType,@NotBlank @Size(max=50) String version,
 @NotBlank @Size(max=200) String title,@NotNull OffsetDateTime effectiveAt,@NotBlank @Size(max=200000) String content){}
```

Exact steps: Use the matching endpoint in the matrix and a valid body from the fixture cookbook. For EACH field in the exact DTO declaration below, send one changed value: omitted, null, wrong JSON type, blank, valid min/max, min−1/max+1; lists empty/one/max/max+1 and null element; numeric decimal/negative/overflow; supported versus unknown enum. Repeat through corresponding form. Nested ScoutContact/weekly items must receive the same checks.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected database/domain state: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/dto/request/CreatePolicyVersionRequest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

INPUT-009 — CreateVenueRequest all field boundaries

Area: INPUT · **Priority:** P1 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Exact request schema:

```
public record CreateVenueRequest(

        @NotBlank
        @Size(min = 2, max = 120)
        String name,

        @NotBlank
        @Size(max = 100)
        String neighbourhood,

        @NotNull
        @Min(1)
        @Max(9999)
        Integer sizeM2,

        @NotNull
        @DecimalMin("0.01")
        @DecimalMax("99999999.99")
        BigDecimal hourlyRateEur,
```

```
        @NotNull @Min(1) @Max(10080)
        Integer minimumBookingMinutes,

        @NotNull @Min(60) @Max(43200)
        Integer minimumLeadTimeMinutes,

        @DecimalMin("0.01") @DecimalMax("99999999.99")
        BigDecimal fullDayRateEur,

        @Min(1) @Max(10000) Integer maxCrewSize,
        @Size(max = 7) List<@NotNull ProductionActivity> permittedActivities,
        @Size(max = 7) List<@NotNull ProductionActivity> prohibitedActivities,
        Boolean naturalLight,
        Boolean blackoutPossible,
        @DecimalMin("0.01") @DecimalMax("99.99") BigDecimal ceilingHeightM,
        Boolean loadingAccess,
        Boolean parkingAvailable,
        @Min(0) @Max(10000) Integer parkingSpaces,
        @Min(-20) @Max(200) Integer floorNumber,
        Boolean liftAvailable,
        Boolean wheelchairAccessible,
        Boolean bathroomsAvailable,
        Boolean soundRecordingSuitable,
        @Size(max = 5000) String productionRules,
        @Size(max = 2000) String powerDetails,

        @NotNull
        @Size(max = 20)
        List<
                @NotBlank
                @Size(max = 50)
                        String
                > tags,

        @Size(max = 5000)
        String description,

        @Size(max = 500)
        String clipUrl
) {
}
```

Exact steps: Use the matching endpoint in the matrix and a valid body from the fixture cookbook. For EACH field in the exact DTO declaration below, send one changed value: omitted, null, wrong JSON type, blank, valid min/max, min−1/max+1; lists empty/one/max/max+1 and null element; numeric decimal/negative/overflow; supported versus unknown enum. Repeat through corresponding form. Nested ScoutContact/weekly items must receive the same checks.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected database/domain state: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/dto/request/CreateVenueRequest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

INPUT-010 — EmailAddressRequest all field boundaries

Area: INPUT · **Priority:** P1 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Exact request schema:

```
public record EmailAddressRequest(@NotBlank @Email @Size(max = 254) String email) {}
```

Exact steps: Use the matching endpoint in the matrix and a valid body from the fixture cookbook. For EACH field in the exact DTO declaration below, send one changed value: omitted, null, wrong JSON type, blank, valid min/max, min−1/max+1; lists empty/one/max/max+1 and null element; numeric decimal/negative/overflow; supported versus unknown enum. Repeat through corresponding form. Nested ScoutContact/weekly items must receive the same checks.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected database/domain state: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/dto/request/EmailAddressRequest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

INPUT-011 — LoginRequest all field boundaries

Area: INPUT · **Priority:** P1 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Exact request schema:

```
public record LoginRequest(

        @NotBlank
        @Email
        String email,

        @NotBlank
        @Size(min = 8, max = 72)
        String password
```

```
) {
}
```

Exact steps: Use the matching endpoint in the matrix and a valid body from the fixture cookbook. For EACH field in the exact DTO declaration below, send one changed value: omitted, null, wrong JSON type, blank, valid min/max, min−1/max+1; lists empty/one/max/max+1 and null element; numeric decimal/negative/overflow; supported versus unknown enum. Repeat through corresponding form. Nested ScoutContact/weekly items must receive the same checks.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected database/domain state: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/dto/request/LoginRequest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

INPUT-012 — MarkOwnerPayoutPaidRequest all field boundaries

Area: INPUT · **Priority:** P1 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Exact request schema:

```
public record MarkOwnerPayoutPaidRequest(
        @NotBlank @Size(max = 200) String externalTransferReference
) {}
```

Exact steps: Use the matching endpoint in the matrix and a valid body from the fixture cookbook. For EACH field in the exact DTO declaration below, send one changed value: omitted, null, wrong JSON type, blank, valid min/max, min−1/max+1; lists empty/one/max/max+1 and null element; numeric decimal/negative/overflow; supported versus unknown enum. Repeat through corresponding form. Nested ScoutContact/weekly items must receive the same checks.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected database/domain state: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/dto/request/MarkOwnerPayoutPaidRequest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

INPUT-013 — MatchRequest all field boundaries

Area: INPUT · **Priority:** P1 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Exact request schema:

```
public record MatchRequest(
    @NotBlank(message = "Brief cannot be blank")
    @Size(min = 10, max = 1000, message = "Brief must be 10-1000 characters")
    String brief,

    UUID venueId  // optional: if provided, return a direct quote for this venue only
) {}
```

Exact steps: Use the matching endpoint in the matrix and a valid body from the fixture cookbook. For EACH field in the exact DTO declaration below, send one changed value: omitted, null, wrong JSON type, blank, valid min/max, min−1/max+1; lists empty/one/max/max+1 and null element; numeric decimal/negative/overflow; supported versus unknown enum. Repeat through corresponding form. Nested ScoutContact/weekly items must receive the same checks.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected database/domain state: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/dto/request/MatchRequest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

INPUT-014 — OfferAvailabilityInquiryRequest all field boundaries

Area: INPUT · **Priority:** P1 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Exact request schema:

```
public record OfferAvailabilityInquiryRequest(
        @NotNull LocalDate offeredDate,
        @NotNull LocalTime offeredStartTime,
        @NotNull LocalTime offeredEndTime) {}
```

Exact steps: Use the matching endpoint in the matrix and a valid body from the fixture cookbook. For EACH field in the exact DTO declaration below, send one changed value: omitted, null, wrong JSON type, blank, valid min/max, min−1/max+1; lists empty/one/max/max+1 and null element; numeric decimal/negative/overflow; supported versus unknown enum. Repeat through corresponding form. Nested ScoutContact/weekly items must receive the same checks.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected database/domain state: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/dto/request/OfferAvailabilityInquiryRequest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

INPUT-015 — OptionalNoteRequest all field boundaries

Area: INPUT · **Priority:** P1 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Exact request schema:

```
public record OptionalNoteRequest(@Size(max=1000) String note) {}
```

Exact steps: Use the matching endpoint in the matrix and a valid body from the fixture cookbook. For EACH field in the exact DTO declaration below, send one changed value: omitted, null, wrong JSON type, blank, valid min/max, min−1/max+1; lists empty/one/max/max+1 and null element; numeric decimal/negative/overflow; supported versus unknown enum. Repeat through corresponding form. Nested ScoutContact/weekly items must receive the same checks.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected database/domain state: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/dto/request/OptionalNoteRequest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

INPUT-016 — OwnerVerificationApproveRequest all field boundaries

Area: INPUT · **Priority:** P1 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Exact request schema:

```
public record OwnerVerificationApproveRequest(@NotNull VerificationMethod verificationMethod,
                                              @Size(max=1000) String note) {}
```

Exact steps: Use the matching endpoint in the matrix and a valid body from the fixture cookbook. For EACH field in the exact DTO declaration below, send one changed value: omitted, null, wrong JSON type, blank, valid min/max, min−1/max+1; lists empty/one/max/max+1 and null element; numeric decimal/negative/overflow; supported versus unknown enum. Repeat through corresponding form. Nested ScoutContact/weekly items must receive the same checks.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected database/domain state: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/dto/request/OwnerVerificationApproveRequest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

INPUT-017 — OwnerVerificationSubmitRequest all field boundaries

Area: INPUT · **Priority:** P1 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Exact request schema:

```
public record OwnerVerificationSubmitRequest(@Size(max=1000) String note) {}
```

Exact steps: Use the matching endpoint in the matrix and a valid body from the fixture cookbook. For EACH field in the exact DTO declaration below, send one changed value: omitted, null, wrong JSON type, blank, valid min/max, min−1/max+1; lists empty/one/max/max+1 and null element; numeric decimal/negative/overflow; supported versus unknown enum. Repeat through corresponding form. Nested ScoutContact/weekly items must receive the same checks.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected database/domain state: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/dto/request/OwnerVerificationSubmitRequest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

INPUT-018 — ReasonRequest all field boundaries

Area: INPUT · **Priority:** P1 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Exact request schema:

```
public record ReasonRequest(@NotBlank @Size(max=1000) String reason) {}
```

Exact steps: Use the matching endpoint in the matrix and a valid body from the fixture cookbook. For EACH field in the exact DTO declaration below, send one changed value: omitted, null, wrong JSON type, blank, valid min/max, min−1/max+1; lists empty/one/max/max+1 and null element; numeric decimal/negative/overflow; supported versus unknown enum. Repeat through corresponding form. Nested ScoutContact/weekly items must receive the same checks.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected database/domain state: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/dto/request/ReasonRequest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

INPUT-019 — RefreshRequest all field boundaries

Area: INPUT · **Priority:** P1 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Exact request schema:

```
public record RefreshRequest(

        @NotBlank
        String refreshToken

) {
}
```

Exact steps: Use the matching endpoint in the matrix and a valid body from the fixture cookbook. For EACH field in the exact DTO declaration below, send one changed value: omitted, null, wrong JSON type, blank, valid min/max, min−1/max+1; lists empty/one/max/max+1 and null element; numeric decimal/negative/overflow; supported versus unknown enum. Repeat through corresponding form. Nested ScoutContact/weekly items must receive the same checks.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected database/domain state: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/dto/request/RefreshRequest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

INPUT-020 — RegisterRequest all field boundaries

Area: INPUT · **Priority:** P1 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Exact request schema:

```
public record RegisterRequest(

        @NotBlank
        @Email
        @Size(max = 254)
```

```
        String email,

        @NotBlank
        @Size(min = 8, max = 72)
        String password,

        @NotBlank
        @Size(min = 2, max = 120)
        String fullName,

        @NotNull
        UserRole role

) {
}
```

Exact steps: Use the matching endpoint in the matrix and a valid body from the fixture cookbook. For EACH field in the exact DTO declaration below, send one changed value: omitted, null, wrong JSON type, blank, valid min/max, min−1/max+1; lists empty/one/max/max+1 and null element; numeric decimal/negative/overflow; supported versus unknown enum. Repeat through corresponding form. Nested ScoutContact/weekly items must receive the same checks.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected database/domain state: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/dto/request/RegisterRequest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

INPUT-021 — ResetPasswordRequest all field boundaries

Area: INPUT · **Priority:** P1 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Exact request schema:

```
public record ResetPasswordRequest(@NotBlank String token,
                                   @NotBlank @Size(min = 8, max = 72) String newPassword) {}
```

Exact steps: Use the matching endpoint in the matrix and a valid body from the fixture cookbook. For EACH field in the exact DTO declaration below, send one changed value: omitted, null, wrong JSON type, blank, valid min/max, min−1/max+1; lists empty/one/max/max+1 and null element; numeric decimal/negative/overflow; supported versus unknown enum. Repeat through corresponding form. Nested ScoutContact/weekly items must receive the same checks.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected database/domain state: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/dto/request/ResetPasswordRequest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

INPUT-022 — ResolveIncidentRequest all field boundaries

Area: INPUT · **Priority:** P1 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Exact request schema:

```
public record ResolveIncidentRequest(@NotBlank @Size(max=2000) @Pattern(regexp="^[^<>]*$",message="must not contain markup") String resolutionNote){}
```

Exact steps: Use the matching endpoint in the matrix and a valid body from the fixture cookbook. For EACH field in the exact DTO declaration below, send one changed value: omitted, null, wrong JSON type, blank, valid min/max, min−1/max+1; lists empty/one/max/max+1 and null element; numeric decimal/negative/overflow; supported versus unknown enum. Repeat through corresponding form. Nested ScoutContact/weekly items must receive the same checks.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected database/domain state: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/dto/request/ResolveIncidentRequest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

INPUT-023 — ScoutContactRequest all field boundaries

Area: INPUT · **Priority:** P1 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Exact request schema:

```
public record ScoutContactRequest(
    @NotBlank(message = "name is required")
    @Size(min = 2, max = 100, message = "name must be 2-100 characters")
    String name,

    @NotBlank(message = "email is required")
    @Email(message = "email must be valid")
    String email,

    String phone  // optional
) {}
```

Exact steps: Use the matching endpoint in the matrix and a valid body from the fixture cookbook. For EACH field in the exact DTO declaration below, send one changed value: omitted, null, wrong JSON type, blank, valid min/max, min−1/max+1; lists empty/one/max/max+1 and null element; numeric decimal/negative/overflow; supported versus unknown enum. Repeat through corresponding form. Nested ScoutContact/weekly items must receive the same checks.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected database/domain state: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/dto/request/ScoutContactRequest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

INPUT-024 — UpdateVenueAvailabilitySettingsRequest all field boundaries

Area: INPUT · **Priority:** P1 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Exact request schema:

```
public record UpdateVenueAvailabilitySettingsRequest(@NotBlank String timezone) {}
```

Exact steps: Use the matching endpoint in the matrix and a valid body from the fixture cookbook. For EACH field in the exact DTO declaration below, send one changed value: omitted, null, wrong JSON type, blank, valid min/max, min−1/max+1; lists empty/one/max/max+1 and null element; numeric decimal/negative/overflow; supported versus unknown enum. Repeat through corresponding form. Nested ScoutContact/weekly items must receive the same checks.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected database/domain state: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/dto/request/UpdateVenueAvailabilitySettingsRequest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

INPUT-025 — UpdateVenueRequest all field boundaries

Area: INPUT · **Priority:** P1 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Exact request schema:

```
public record UpdateVenueRequest(
        @NotBlank @Size(min = 2, max = 120) String name,
        @NotBlank @Size(max = 100) String neighbourhood,
        @NotNull @Min(1) @Max(9999) Integer sizeM2,
        @NotNull @DecimalMin("0.01") @DecimalMax("99999999.99") BigDecimal hourlyRateEur,
        @NotNull @Min(1) @Max(10080) Integer minimumBookingMinutes,
        @NotNull @Min(60) @Max(43200) Integer minimumLeadTimeMinutes,
        @DecimalMin("0.01") @DecimalMax("99999999.99") BigDecimal fullDayRateEur,
        @Min(1) @Max(10000) Integer maxCrewSize,
        @Size(max = 7) List<@NotNull ProductionActivity> permittedActivities,
        @Size(max = 7) List<@NotNull ProductionActivity> prohibitedActivities,
        Boolean naturalLight,
        Boolean blackoutPossible,
        @DecimalMin("0.01") @DecimalMax("99.99") BigDecimal ceilingHeightM,
        Boolean loadingAccess,
        Boolean parkingAvailable,
        @Min(0) @Max(10000) Integer parkingSpaces,
        @Min(-20) @Max(200) Integer floorNumber,
        Boolean liftAvailable,
        Boolean wheelchairAccessible,
        Boolean bathroomsAvailable,
        Boolean soundRecordingSuitable,
        @Size(max = 5000) String productionRules,
        @Size(max = 2000) String powerDetails,
        @NotNull @Size(max = 20) List<@NotBlank @Size(max = 50) String> tags,
        @Size(max = 5000) String description,
        @Size(max = 500) String clipUrl,
        @NotBlank @Size(max = 100) String timezone,
        @NotNull @Size(max = 12) List<@NotBlank @Size(max = 500) String> photoPaths
) {
}
```

Exact steps: Use the matching endpoint in the matrix and a valid body from the fixture cookbook. For EACH field in the exact DTO declaration below, send one changed value: omitted, null, wrong JSON type, blank, valid min/max, min−1/max+1; lists empty/one/max/max+1 and null element; numeric decimal/negative/overflow; supported versus unknown enum. Repeat through corresponding form. Nested ScoutContact/weekly items must receive the same checks.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected database/domain state: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/dto/request/UpdateVenueRequest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

INPUT-026 — UpdateWeeklyAvailabilityRequest all field boundaries

Area: INPUT · **Priority:** P1 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Exact request schema:

```
public record UpdateWeeklyAvailabilityRequest(@NotNull @Size(max = 50) List<@Valid WeeklyIntervalRequest> intervals) {}
```

Exact steps: Use the matching endpoint in the matrix and a valid body from the fixture cookbook. For EACH field in the exact DTO declaration below, send one changed value: omitted, null, wrong JSON type, blank, valid min/max, min−1/max+1; lists empty/one/max/max+1 and null element; numeric decimal/negative/overflow; supported versus unknown enum. Repeat through corresponding form. Nested ScoutContact/weekly items must receive the same checks.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected database/domain state: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/dto/request/UpdateWeeklyAvailabilityRequest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

INPUT-027 — VerifyEmailRequest all field boundaries

Area: INPUT · **Priority:** P1 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Exact request schema:

```
public record VerifyEmailRequest(@NotBlank String token) {}
```

Exact steps: Use the matching endpoint in the matrix and a valid body from the fixture cookbook. For EACH field in the exact DTO declaration below, send one changed value: omitted, null, wrong JSON type, blank, valid min/max, min−1/max+1; lists empty/one/max/max+1 and null element; numeric decimal/negative/overflow; supported versus unknown enum. Repeat through corresponding form. Nested ScoutContact/weekly items must receive the same checks.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected database/domain state: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/dto/request/VerifyEmailRequest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

INPUT-028 — WeeklyIntervalRequest all field boundaries

Area: INPUT · **Priority:** P1 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Exact request schema:

```
public record WeeklyIntervalRequest(@NotNull DayOfWeek dayOfWeek, @NotNull LocalTime startTime,
                                    @NotNull LocalTime endTime) {}
```

Exact steps: Use the matching endpoint in the matrix and a valid body from the fixture cookbook. For EACH field in the exact DTO declaration below, send one changed value: omitted, null, wrong JSON type, blank, valid min/max, min−1/max+1; lists empty/one/max/max+1 and null element; numeric decimal/negative/overflow; supported versus unknown enum. Repeat through corresponding form. Nested ScoutContact/weekly items must receive the same checks.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount;

record accepted-ignore behavior rather than assume 400.

Expected database/domain state: Bean validation constraints shown below apply before service invariants; valid boundary accepts subject to domain rules, invalid boundary 400 with field details and no mutation. Optional nullable fields must not be made required by QA. Frontend limits should not contradict backend. Unknown JSON fields must not control owner/status/amount; record accepted-ignore behavior rather than assume 400.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/dto/request/WeeklyIntervalRequest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API scenarios

API-001 — GET /admin/bookings: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `Page<AdminBookingSummaryDto> search(@RequestParam(required = false) UUID bookingId, @RequestParam(required = false) UUID venueId, @RequestParam(required = false) UUID scoutId, @RequestParam(required = false) BookingStatus status, @RequestParam(required = false) String molliePaymentId, @RequestParam(required = false) RefundStatus refundStatus, @RequestParam(required = false) PaymentCreationStatus paymentCreationStatus, @PageableDefault(size = 20) Pageable pageable)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send GET /api/admin/bookings with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/AdminBookingOperationsController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-002 — GET /admin/bookings/{id}: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `AdminBookingDetailDto detail(@PathVariable UUID id)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send GET /api/admin/bookings/{id} with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/AdminBookingOperationsController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-003 — GET /admin/bookings/{id}/events: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `Page<AdminBookingEventDto> events(@PathVariable UUID id, @PageableDefault(size = 20) Pageable pageable)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send GET /api/admin/bookings/{id}/events with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/AdminBookingOperationsController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-004 — GET /admin/incidents: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `Page<IncidentDto> list(@RequestParam(required=false)IncidentStatus status,@RequestParam(required=false)IncidentCategory category,@RequestParam(required=false)UUID bookingId,Pageable pageable)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send GET /api/admin/incidents with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/AdminIncidentController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-005 — GET /admin/incidents/{id}: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `IncidentDto get(@PathVariable UUID id)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send GET /api/admin/incidents/{id} with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/AdminIncidentController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-006 — POST /admin/incidents/{id}/start-review: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `IncidentDto start(@PathVariable UUID id)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send POST /api/admin/incidents/{id}/start-review with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/AdminIncidentController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-007 — POST /admin/incidents/{id}/resolve: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `IncidentDto resolve(@PathVariable UUID id,@Valid @RequestBody ResolveIncidentRequest r)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send POST /api/admin/incidents/{id}/resolve with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/AdminIncidentController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-008 — POST /admin/incidents/{id}/close: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `IncidentDto close(@PathVariable UUID id)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send POST /api/admin/incidents/{id}/close with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/AdminIncidentController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-009 — GET /admin/operations/exceptions: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `Page<OperationalExceptionDto> exceptions(@PageableDefault(size = 20) Pageable pageable)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send GET /api/admin/operations/exceptions with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/AdminOperationsExceptionController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-010 — GET /admin/liabilities: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `Page<OwnerFinancialLiabilityDto> list(@RequestParam(required = false) OwnerFinancialLiabilityStatus status, @RequestParam(required = false) OwnerFinancialLiabilityType type, @RequestParam(required = false) UUID ownerId, @RequestParam(required = false) UUID bookingId, @PageableDefault(size = 20) Pageable pageable)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send GET /api/admin/liabilities with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/AdminOwnerFinancialLiabilityController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-011 — GET /admin/liabilities/{id}: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `OwnerFinancialLiabilityDto get(@PathVariable UUID id)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send GET /api/admin/liabilities/{id} with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/AdminOwnerFinancialLiabilityController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-012 — POST /admin/liabilities/{id}/settle: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `OwnerFinancialLiabilityDto settle(@PathVariable UUID id)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send POST /api/admin/liabilities/{id}/settle with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/AdminOwnerFinancialLiabilityController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-013 — GET /admin/payouts: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `Page<OwnerPayoutDto> list(@RequestParam(required = false) OwnerPayoutStatus status, @RequestParam(required = false) UUID ownerId, @RequestParam(required = false) UUID bookingId, @RequestParam(required = false) UUID venueId, @PageableDefault(size = 20) Pageable pageable)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send GET /api/admin/payouts with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/AdminOwnerPayoutController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-014 — GET /admin/payouts/{id}: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `OwnerPayoutDto get(@PathVariable UUID id)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send GET /api/admin/payouts/{id} with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/AdminOwnerPayoutController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-015 — POST /admin/payouts/{id}/mark-paid: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `OwnerPayoutDto markPaid(@PathVariable UUID id, @Valid @RequestBody MarkOwnerPayoutPaidRequest request)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send POST /api/admin/payouts/{id}/mark-paid with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/AdminOwnerPayoutController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-016 — GET /admin/owner-verifications: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `Page<AdminOwnerVerificationDto> list(@RequestParam(required=false) OwnerVerificationStatus status, @RequestParam(required=false) UUID ownerId, Pageable pageable)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send GET /api/admin/owner-verifications with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/AdminOwnerVerificationController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-017 — GET /admin/owner-verifications/{id}: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `AdminOwnerVerificationDto get(@PathVariable UUID id)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send GET /api/admin/owner-verifications/{id} with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset

fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/AdminOwnerVerificationController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-018 — POST /admin/owner-verifications/{id}/approve: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `AdminOwnerVerificationDto approve(@PathVariable UUID id,@Valid @RequestBody OwnerVerificationApproveRequest request)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send POST /api/admin/owner-verifications/{id}/approve with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/AdminOwnerVerificationController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-019 — POST /admin/owner-verifications/{id}/reject: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `AdminOwnerVerificationDto reject(@PathVariable UUID id,@Valid @RequestBody ReasonRequest request)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send POST /api/admin/owner-verifications/{id}/reject with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/AdminOwnerVerificationController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-020 — GET /admin/payout-destinations: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `Page<PayoutDestinationDto> list(@RequestParam(required=false)PayoutDestinationStatus status,@RequestParam(required=false)UUID ownerId,@RequestParam(required=false)UUID destinationId,@PageableDefault(size=20)Pageable pageable)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send GET /api/admin/payout-destinations with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/AdminPayoutDestinationController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-021 — GET /admin/payout-destinations/{id}: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `PayoutDestinationDto get(@PathVariable UUID id)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send GET /api/admin/payout-destinations/{id} with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/AdminPayoutDestinationController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-022 — POST /admin/payout-destinations/{id}/verify: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `PayoutDestinationDto verify(@PathVariable UUID id,@Valid @RequestBody(required=false)OptionalNoteRequest request)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send POST /api/admin/payout-destinations/{id}/verify with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/AdminPayoutDestinationController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-023 — POST /admin/payout-destinations/{id}/reject: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `PayoutDestinationDto reject(@PathVariable UUID id,@Valid @RequestBody ReasonRequest request)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send POST /api/admin/payout-destinations/{id}/reject with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/AdminPayoutDestinationController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-024 — POST /admin/payout-destinations/{id}/reveal: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `ResponseEntity<PayoutDestinationRevealDto> reveal(@PathVariable UUID id)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send POST /api/admin/payout-destinations/{id}/reveal with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/AdminPayoutDestinationController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-025 — GET /admin/policies: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `List<PolicyVersionDto> list()`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send GET /api/admin/policies with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/AdminPolicyController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-026 — POST /admin/policies: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `PolicyVersionDto create(@Valid @RequestBody CreatePolicyVersionRequest r)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send POST /api/admin/policies with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 201. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 201. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/AdminPolicyController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-027 — POST /admin/policies/{id}/publish: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `PolicyVersionDto publish(@PathVariable UUID id)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send POST /api/admin/policies/{id}/publish with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/AdminPolicyController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-028 — POST /admin/policies/{id}/retire: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `PolicyVersionDto retire(@PathVariable UUID id)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send POST /api/admin/policies/{id}/retire with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/AdminPolicyController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-029 — GET /admin/venues: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `Page<AdminVenueModerationDto> list(@RequestParam(required=false) VenueStatus status, @RequestParam(required=false) UUID ownerId,@RequestParam(required=false) UUID venueId,Pageable pageable)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send GET /api/admin/venues with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/AdminVenueModerationController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-030 — GET /admin/venues/{id}: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `AdminVenueModerationDto get(@PathVariable UUID id)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send GET /api/admin/venues/{id} with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/AdminVenueModerationController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-031 — GET /admin/venues/{id}/events: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `List<TrustEventDto> events(@PathVariable UUID id)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send GET /api/admin/venues/{id}/events with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/AdminVenueModerationController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-032 — POST /admin/venues/{id}/approve: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `AdminVenueModerationDto approve(@PathVariable UUID id,@Valid @RequestBody(required=false) OptionalNoteRequest request)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send POST /api/admin/venues/{id}/approve with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/AdminVenueModerationController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-033 — POST /admin/venues/{id}/request-changes: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `AdminVenueModerationDto changes(@PathVariable UUID id,@Valid @RequestBody ReasonRequest request)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send POST /api/admin/venues/{id}/request-changes with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation

reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/AdminVenueModerationController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-034 — POST /admin/venues/{id}/suspend: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `AdminVenueModerationDto suspend(@PathVariable UUID id,@Valid @RequestBody ReasonRequest request)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send POST /api/admin/venues/{id}/suspend with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/AdminVenueModerationController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-035 — POST /admin/venues/{id}/reactivate: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `AdminVenueModerationDto reactivate(@PathVariable UUID id,@Valid @RequestBody(required=false) OptionalNoteRequest request)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send POST /api/admin/venues/{id}/reactivate with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 200. ADMIN only; lifecycle/body validation still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/AdminVenueModerationController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-036 — POST /auth/register: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `ResponseEntity<AuthResponse> register(@Valid @RequestBody RegisterRequest request)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send POST /api/auth/register with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 201 ; 201 ; 201 ; 201. Public auth operations still require valid credentials/token; registration 201, invalid credentials 401, inactive 403, duplicates 409. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 201 ; 201 ; 201 ; 201. Public auth operations still require valid credentials/token; registration 201, invalid credentials 401, inactive 403, duplicates 409. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/AuthController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-037 — POST /auth/login: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `ResponseEntity<AuthResponse> login(@Valid @RequestBody LoginRequest request)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send POST /api/auth/login with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 200 ; 200 ; 200 ; 200. Public auth operations still require valid credentials/token; registration 201, invalid credentials 401, inactive 403, duplicates 409. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 200 ; 200 ; 200 ; 200. Public auth operations still require valid credentials/token; registration 201, invalid credentials 401, inactive 403, duplicates 409. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/AuthController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-038 — POST /auth/refresh: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `ResponseEntity<AuthResponse> refresh(@Valid @RequestBody RefreshRequest request)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send POST /api/auth/refresh with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 200 ; 200 ; 200 ; 200. Public auth operations still require valid credentials/token; registration 201, invalid credentials 401, inactive 403, duplicates 409. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 200 ; 200 ; 200 ; 200. Public auth operations still require valid credentials/token; registration 201, invalid credentials 401, inactive 403, duplicates 409. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/AuthController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-039 — POST /auth/verify-email: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `ResponseEntity<MessageResponse> verifyEmail(@Valid @RequestBody VerifyEmailRequest request)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send POST /api/auth/verify-email with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 200 ; 200 ; 200 ; 200. Public auth operations still require valid credentials/token; registration 201, invalid credentials 401, inactive 403, duplicates 409. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 200 ; 200 ; 200 ; 200. Public auth operations still require valid credentials/token; registration 201, invalid credentials 401, inactive 403, duplicates 409. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/AuthController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-040 — POST /auth/resend-verification: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `ResponseEntity<MessageResponse> resendVerification(@Valid @RequestBody EmailAddressRequest request)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send POST /api/auth/resend-verification with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 200 ; 200 ; 200 ; 200. Public auth operations still require valid credentials/token; registration 201, invalid credentials 401, inactive 403, duplicates 409. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 200 ; 200 ; 200 ; 200. Public auth operations still require valid credentials/token; registration 201, invalid credentials 401, inactive 403, duplicates 409. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/AuthController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-041 — POST /auth/forgot-password: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `ResponseEntity<MessageResponse> forgotPassword(@Valid @RequestBody EmailAddressRequest request)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send POST /api/auth/forgot-password with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 200 ; 200 ; 200 ; 200. Public auth operations still require valid credentials/token; registration 201, invalid credentials 401, inactive 403, duplicates 409. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 200 ; 200 ; 200 ; 200. Public auth operations still require valid credentials/token; registration 201, invalid credentials 401, inactive 403, duplicates 409. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/AuthController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-042 — POST /auth/reset-password: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `ResponseEntity<MessageResponse> resetPassword(@Valid @RequestBody ResetPasswordRequest request)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send POST /api/auth/reset-password with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 200 ; 200 ; 200 ; 200. Public auth operations still require valid credentials/token; registration 201, invalid credentials 401, inactive 403, duplicates 409. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 200 ; 200 ; 200 ; 200. Public auth operations still require valid credentials/token; registration 201, invalid credentials 401, inactive 403, duplicates 409. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/AuthController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-043 — GET /auth/me: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `ResponseEntity<UserDto> getCurrentUser(Authentication authentication)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send GET /api/auth/me with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 200 ; 200 ; 200. Public auth operations still require valid credentials/token; registration 201, invalid credentials 401, inactive 403, duplicates 409. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 200 ; 200 ; 200. Public auth operations still require valid credentials/token; registration 201, invalid credentials 401, inactive 403, duplicates 409. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/AuthController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-044 — POST /availability-inquiries: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `AvailabilityInquiryDto create(@Valid @RequestBody CreateAvailabilityInquiryRequest request)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send POST /api/availability-inquiries with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 201 ; 403 ; 403. Roles explicit; mutation email/current policies; expired/invalid state conflict, unavailable inquiry restrictions. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 201 ; 403 ; 403. Roles explicit; mutation email/current policies; expired/invalid state conflict, unavailable inquiry restrictions. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/AvailabilityInquiryController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-045 — GET /availability-inquiries/mine: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `List<AvailabilityInquiryDto> mine()`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send GET /api/availability-inquiries/mine with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 200 ; 403 ; 403. Roles explicit; mutation email/current policies; expired/invalid state conflict, unavailable inquiry restrictions. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 200 ; 403 ; 403. Roles explicit; mutation email/current policies; expired/invalid state conflict, unavailable inquiry restrictions. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/AvailabilityInquiryController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-046 — GET /availability-inquiries/owned-venues: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `List<AvailabilityInquiryDto> ownedVenueInquiries()`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send GET /api/availability-inquiries/owned-venues with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 200 own / 403 other ; 403. Roles explicit; mutation email/current policies; expired/invalid state conflict, unavailable inquiry restrictions. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 200 own / 403 other ; 403. Roles explicit; mutation email/current policies; expired/invalid state conflict, unavailable inquiry restrictions. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/AvailabilityInquiryController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-047 — POST /availability-inquiries/{id}/decline: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `AvailabilityInquiryDto decline(@PathVariable UUID id)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send POST /api/availability-inquiries/{id}/decline with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 200 own / 403 other ; 403. Roles explicit; mutation email/current policies; expired/invalid state conflict, unavailable inquiry restrictions. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 200 own / 403 other ; 403. Roles explicit; mutation email/current policies; expired/invalid state conflict, unavailable inquiry restrictions. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/AvailabilityInquiryController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-048 — POST /availability-inquiries/{id}/offer: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `AvailabilityInquiryDto offer(@PathVariable UUID id, @Valid @RequestBody OfferAvailabilityInquiryRequest request)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send POST /api/availability-inquiries/{id}/offer with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 200 own / 403 other ; 403. Roles explicit; mutation email/current policies; expired/invalid state conflict, unavailable inquiry restrictions. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 200 own / 403 other ; 403. Roles explicit; mutation email/current policies; expired/invalid state conflict, unavailable inquiry restrictions. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/AvailabilityInquiryController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-049 — POST /availability-inquiries/{id}/accept: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `BookingDto accept(@PathVariable UUID id)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send POST /api/availability-inquiries/{id}/accept with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 200 ; 403 ; 403. Roles explicit; mutation email/current policies; expired/invalid state conflict, unavailable inquiry restrictions. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 200 ; 403 ; 403. Roles explicit; mutation email/current policies; expired/invalid state conflict, unavailable inquiry restrictions. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/AvailabilityInquiryController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-050 — POST /bookings: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `ResponseEntity<BookingDto> createBooking(@Valid @RequestBody CreateBookingRequest request)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send POST /api/bookings with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 201 ; 403 ; 403. SCOUT only; verified email and policies; nonactive venue 404, unavailable 409. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 201 ; 403 ; 403. SCOUT only; verified email and policies; nonactive venue 404, unavailable 409. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/BookingController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-051 — GET /bookings: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `ResponseEntity<List<BookingDto>> getBookings()`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send GET /api/bookings with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 200 own / 403 other ; 200 own / 403 other ; 200. List scoped; detail missing 404, malformed 400; preview canCancel false outside valid states. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 200 own / 403 other ; 200 own / 403 other ; 200. List scoped; detail missing 404, malformed 400; preview canCancel false outside valid states. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/BookingController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-052 — GET /bookings/{id}: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `ResponseEntity<BookingDto> getBooking(@PathVariable UUID id)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send GET /api/bookings/{id} with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 200 own / 403 other ; 200 own / 403 other ; 200. List scoped; detail missing 404, malformed 400; preview canCancel false outside valid states. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 200 own / 403 other ; 200 own / 403 other ; 200. List scoped; detail missing 404, malformed 400; preview canCancel false outside valid states. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/BookingController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-053 — POST /bookings/{id}/accept: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `ResponseEntity<BookingDto> acceptRequest(@PathVariable UUID id)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send POST /api/bookings/{id}/accept with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 200 own / 403 other ; 200. Owner or ADMIN; verified email for marketplace actors, policies except ADMIN; state/deadline 409. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 200 own / 403 other ; 200. Owner or ADMIN; verified email for marketplace actors, policies except ADMIN; state/deadline 409. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/BookingController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-054 — POST /bookings/{id}/decline: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `ResponseEntity<BookingDto> declineRequest(@PathVariable UUID id)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send POST /api/bookings/{id}/decline with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 200 own / 403 other ; 200. Owner or ADMIN; verified email for marketplace actors, policies except ADMIN; state/deadline 409. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 200 own / 403 other ; 200. Owner or ADMIN; verified email for marketplace actors, policies except ADMIN; state/deadline 409. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/BookingController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-055 — POST /bookings/{id}/cancel: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `ResponseEntity<BookingDto> cancelBooking( @PathVariable UUID id, @Valid @RequestBody(required = false) CancelBookingRequest request)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send POST /api/bookings/{id}/cancel with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 200 own / 403 other ; 200 own / 403 other ; 200. Scout unpaid/confirmed; owner/admin confirmed with reason only; unsupported state/missing owner reason 409; email gate, no policy gate. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 200 own / 403 other ; 200 own / 403 other ; 200. Scout unpaid/confirmed; owner/admin confirmed with reason only; unsupported state/missing owner reason 409; email gate, no policy gate. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/BookingController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-056 — GET /bookings/{id}/cancellation-preview: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `ResponseEntity<CancellationPreviewDto> cancellationPreview(@PathVariable UUID id)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send GET /api/bookings/{id}/cancellation-preview with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 200 own / 403 other ; 200 own / 403 other ; 200. List scoped; detail missing 404, malformed 400; preview canCancel false outside valid states. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 200 own / 403 other ; 200 own / 403 other ; 200. List scoped; detail missing 404, malformed 400; preview canCancel false outside valid states. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/BookingController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-057 — POST /bookings/{id}/refund: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `ResponseEntity<BookingDto> refundBooking(@PathVariable UUID id)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send POST /api/bookings/{id}/refund with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 202. ADMIN full refund only; verified-user policy ADMIN exemption; confirmed required 409. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 403 ; 202. ADMIN full refund only; verified-user policy ADMIN exemption; confirmed required 409. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/BookingController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-058 — POST /bookings/{bookingId}/checkout: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `ResponseEntity<CheckoutResponse> createCheckout(@PathVariable java.util.UUID bookingId)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send POST /api/bookings/{bookingId}/checkout with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 201 own / 403 other ; 403 ; 403. Correct requesting scout; AWAITING_PAYMENT only 400, expired 409; policies/email. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 201 own / 403 other ; 403 ; 403. Correct requesting scout; AWAITING_PAYMENT only 400, expired 409; policies/email. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/CheckoutController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-059 — GET /bookings/{id}/receipt: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `ResponseEntity<byte[]> receipt(@PathVariable UUID id)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send GET /api/bookings/{id}/receipt with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 200 own / 403 other ; 403 ; 200. First issuance eligibility 409; PDF response, no JSON storage path. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 200 own / 403 other ; 403 ; 200. First issuance eligibility 409; PDF response, no JSON storage path. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/FinancialDocumentController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-060 — POST /bookings/{bookingId}/incidents: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `IncidentDto report(@PathVariable UUID bookingId,@Valid @RequestBody CreateIncidentRequest r)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send POST /api/bookings/{bookingId}/incidents with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 201/200 own;403 other ; 201/200 own;403 other ; 201/200. POST 201, GET 200; no backend email/policy gate, frontend generic mutation gate may be stricter. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 201/200 own;403 other ; 201/200 own;403 other ; 201/200. POST 201, GET 200; no backend email/policy gate, frontend generic mutation gate may be stricter. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/IncidentController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-061 — GET /bookings/{bookingId}/incidents: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `List<IncidentDto> list(@PathVariable UUID bookingId)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send GET /api/bookings/{bookingId}/incidents with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 201/200 own;403 other ; 201/200 own;403 other ; 201/200. POST 201, GET 200; no backend email/policy gate, frontend generic mutation gate may be stricter. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 201/200 own;403 other ; 201/200 own;403 other ; 201/200. POST 201, GET 200; no backend email/policy gate, frontend generic mutation gate may be stricter. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat

arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/IncidentController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-062 — POST /match: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `ResponseEntity<MatchResponse> search( @Valid @RequestBody MatchRequest request)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send POST /api/match with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 200 ; 200 ; 200. Authenticated all roles in backend; direct quote ownership still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 200 ; 200 ; 200. Authenticated all roles in backend; direct quote ownership still applies. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/MatchController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-063 — GET /owner/payouts: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `Page<OwnerPayoutSummaryDto> list(@PageableDefault(size = 20) Pageable pageable)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send GET /api/owner/payouts with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 200 own / 403 other ; 403. ADMIN allowed statement only; owner list/detail explicitly excludes ADMIN. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 200 own / 403 other ; 403. ADMIN allowed statement only; owner list/detail explicitly excludes ADMIN. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/OwnerPayoutController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-064 — GET /owner/payouts/{id}: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `OwnerPayoutSummaryDto detail(@PathVariable UUID id)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send GET /api/owner/payouts/{id} with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 200 own / 403 other ; 403. ADMIN allowed statement only; owner list/detail explicitly excludes ADMIN. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 200 own / 403 other ; 403. ADMIN allowed statement only; owner list/detail explicitly excludes ADMIN. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/OwnerPayoutController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-065 — GET /owner/payouts/{id}/statement: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `ResponseEntity<byte[]> statement(@PathVariable UUID id)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send GET /api/owner/payouts/{id}/statement with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 200 own / 403 other ; 200. ADMIN allowed statement only; owner list/detail explicitly excludes ADMIN. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 200 own / 403 other ; 200. ADMIN allowed statement only; owner list/detail explicitly excludes ADMIN. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/OwnerPayoutController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-066 — GET /owner/payout-destinations: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `List<PayoutDestinationDto> list()`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send GET /api/owner/payout-destinations with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 200 ; 403. OWNER only; submit email, policies and owner verification required. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 200 ; 403. OWNER only; submit email, policies and owner verification required. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/OwnerPayoutDestinationController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-067 — POST /owner/payout-destinations: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `PayoutDestinationDto submit(@Valid @RequestBody CreatePayoutDestinationRequest request)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send POST /api/owner/payout-destinations with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 200 ; 403. OWNER only; submit email, policies and owner verification required. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 200 ; 403. OWNER only; submit email, policies and owner verification required. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/OwnerPayoutDestinationController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-068 — GET /owner/verification: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `OwnerVerificationDto get()`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send GET /api/owner/verification with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 200 ; 403. OWNER only; submit requires verified email; review state 409. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 200 ; 403. OWNER only; submit requires verified email; review state 409. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/OwnerVerificationController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-069 — POST /owner/verification/submit: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `OwnerVerificationDto submit(@Valid @RequestBody(required=false) OwnerVerificationSubmitRequest request)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send POST /api/owner/verification/submit with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 200 ; 403. OWNER only; submit requires verified email; review state 409. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 200 ; 403. OWNER only; submit requires verified email; review state 409. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/OwnerVerificationController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-070 — GET /policies/current: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `List<PolicyVersionDto> current()`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send GET /api/policies/current with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 200 ; 200 ; 200. Policies are authenticated, not public; invalid current ID 400, missing type 404. No backend verified-email annotation on accept. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 200 ; 200 ; 200. Policies are authenticated, not public; invalid current ID 400, missing type 404. No backend verified-email annotation on accept. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/PolicyController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-071 — GET /policies/{type}/current: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `PolicyVersionDto current(@PathVariable PolicyType type)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send GET /api/policies/{type}/current with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 200 ; 200 ; 200. Policies are authenticated, not public; invalid current ID 400, missing type 404. No backend verified-email annotation on accept. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 200 ; 200 ; 200. Policies are authenticated, not public; invalid current ID 400, missing type 404. No backend verified-email annotation on accept. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/PolicyController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-072 — GET /policies/status: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `PolicyStatusDto status()`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send GET /api/policies/status with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 200 ; 200 ; 200. Policies are authenticated, not public; invalid current ID 400, missing type 404. No backend verified-email annotation on accept. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 200 ; 200 ; 200. Policies are authenticated, not public; invalid current ID 400, missing type 404. No backend verified-email annotation on accept. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/PolicyController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-073 — POST /policies/accept: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `PolicyStatusDto accept(@Valid @RequestBody AcceptPoliciesRequest request,HttpServletRequest http)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send POST /api/policies/accept with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 200 ; 200 ; 200. Policies are authenticated, not public; invalid current ID 400, missing type 404. No backend verified-email annotation on accept. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 200 ; 200 ; 200. Policies are authenticated, not public; invalid current ID 400, missing type 404. No backend verified-email annotation on accept. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/PolicyController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-074 — POST /upload/venue-photo: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `ResponseEntity<Map<String, String>> uploadVenuePhoto(@RequestParam("file") MultipartFile file)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send POST /api/upload/venue-photo with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 200 ; 403. Public media GET; standalone upload OWNER verified email; not scoped to venue. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 200 ; 403. Public media GET; standalone upload OWNER verified email; not scoped to venue. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/UploadController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-075 — GET /upload/files/{filename:.+}: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `ResponseEntity<Resource> serveFile(@PathVariable String filename)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send GET /api/upload/files/{filename:.+} with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 200 ; 200 ; 200 ; 200. Public media GET; standalone upload OWNER verified email; not scoped to venue. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 200 ; 200 ; 200 ; 200. Public media GET; standalone upload OWNER verified email; not scoped to venue. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/UploadController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-076 — GET /venues/{venueId}/availability: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `VenueAvailabilityDto getAvailability(@PathVariable UUID venueId, @RequestParam LocalDate from, @RequestParam LocalDate to)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send GET /api/venues/{venueId}/availability with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 200 ACTIVE / 403 other ; 200 ACTIVE / 403 other ; 200 ACTIVE or own / 403 other ; 200. Visibility enforced via VenueService; range 400. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 200 ACTIVE / 403 other ; 200 ACTIVE / 403 other ; 200 ACTIVE or own / 403 other ; 200. Visibility enforced via VenueService; range 400. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/VenueAvailabilityController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-077 — GET /venues/{venueId}/availability/settings: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `VenueAvailabilitySettingsDto getSettings(@PathVariable UUID venueId)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send GET /api/venues/{venueId}/availability/settings with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 403 ; 403 ; 200 own / 403 other ; 200. Owner/admin scope; mutations verified email; no policy gate; foreign override 404. Anonymous settings GET reaches method guard and 403 controller advice. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 403 ; 403 ; 200 own / 403 other ; 200. Owner/admin scope; mutations verified email; no policy gate; foreign override 404. Anonymous settings GET reaches method guard and 403 controller advice. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/VenueAvailabilityController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-078 — PUT /venues/{venueId}/availability/settings: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `VenueAvailabilitySettingsDto updateSettings(@PathVariable UUID venueId, @Valid @RequestBody UpdateVenueAvailabilitySettingsRequest request)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send PUT /api/venues/{venueId}/availability/settings with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 200 own / 403 other ; 200. Owner/admin scope; mutations verified email; no policy gate; foreign override 404. Anonymous settings GET reaches method guard and 403 controller advice. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 200 own / 403 other ; 200. Owner/admin scope; mutations verified email; no policy gate; foreign override 404. Anonymous settings GET reaches method guard and 403 controller advice. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/VenueAvailabilityController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-079 — PUT /venues/{venueId}/availability/weekly: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `VenueAvailabilitySettingsDto replaceWeekly(@PathVariable UUID venueId, @Valid @RequestBody UpdateWeeklyAvailabilityRequest request)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send PUT /api/venues/{venueId}/availability/weekly with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 200 own / 403 other ; 200. Owner/admin scope; mutations verified email; no policy gate; foreign override 404. Anonymous settings GET reaches method guard and 403 controller advice. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 200 own / 403 other ; 200. Owner/admin scope; mutations verified email; no policy gate; foreign override 404. Anonymous settings GET reaches method guard and 403 controller advice. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/VenueAvailabilityController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-080 — POST /venues/{venueId}/availability/overrides: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `AvailabilityOverrideDto addOverride(@PathVariable UUID venueId, @Valid @RequestBody CreateAvailabilityOverrideRequest request)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send POST /api/venues/{venueId}/availability/overrides with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 201 own / 403 other ; 201. Owner/admin scope; mutations verified email; no policy gate; foreign override 404. Anonymous settings GET reaches method guard and 403 controller advice. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 201 own / 403 other ; 201. Owner/admin scope; mutations verified email; no policy gate; foreign override 404. Anonymous settings GET reaches method guard and 403 controller advice. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/VenueAvailabilityController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-081 — DELETE /venues/{venueId}/availability/overrides/{overrideId}: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `void deleteOverride(@PathVariable UUID venueId, @PathVariable UUID overrideId)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send DELETE /api/venues/{venueId}/availability/overrides/{overrideId} with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 204 own / 403 other ; 204. Owner/admin scope; mutations verified email; no policy gate; foreign override 404. Anonymous settings GET reaches method guard and 403 controller advice. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 204 own / 403 other ; 204. Owner/admin scope; mutations verified email; no policy gate; foreign override 404. Anonymous settings GET reaches method guard and 403 controller advice. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/VenueAvailabilityController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-082 — GET /venues: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `ResponseEntity<List<VenueListDto>> getAllVenues( @RequestParam(required = false) String tag)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send GET /api/venues with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 200 ; 200 ; 200 ; 200. ACTIVE only catalogue; tag filter. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 200 ; 200 ; 200 ; 200. ACTIVE only catalogue; tag filter. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/VenueController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-083 — GET /venues/{id}: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `ResponseEntity<VenueDto> getVenueById(@PathVariable UUID id)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send GET /api/venues/{id} with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 200 ACTIVE / 403 other ; 200 ACTIVE / 403 other ; 200 ACTIVE or own / 403 other ; 200. Nonexistent 404. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 200 ACTIVE / 403 other ; 200 ACTIVE / 403 other ; 200 ACTIVE or own / 403 other ; 200. Nonexistent 404. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/VenueController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-084 — POST /venues: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `ResponseEntity<VenueDto> createVenue(@Valid @RequestBody CreateVenueRequest request)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send POST /api/venues with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 201 ; 403. Verified email required; no policy gate on draft creation. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 201 ; 403. Verified email required; no policy gate on draft creation. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/VenueController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-085 — GET /venues/my: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `ResponseEntity<List<VenueDto>> getMyVenues()`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send GET /api/venues/my with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between

actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 403 ; 403 ; 200 ; 403. Public GET matcher permits filter-chain access; method guard denies anonymous with AccessDeniedException handled as 403 by GlobalExceptionHandler. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 403 ; 403 ; 200 ; 403. Public GET matcher permits filter-chain access; method guard denies anonymous with AccessDeniedException handled as 403 by GlobalExceptionHandler. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/VenueController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-086 — PUT /venues/{id}: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `ResponseEntity<VenueDto> updateVenue(@PathVariable UUID id, @Valid @RequestBody UpdateVenueRequest request)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send PUT /api/venues/{id} with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 200 own / 403 other ; 200. Verified email, owner scope; no policy gate on ordinary update/photos. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 200 own / 403 other ; 200. Verified email, owner scope; no policy gate on ordinary update/photos. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/VenueController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-087 — POST /venues/{id}/photos: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `ResponseEntity<VenueDto> uploadAndAttachPhoto(@PathVariable UUID id, @RequestParam("file") MultipartFile file)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send POST /api/venues/{id}/photos with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 200 own / 403 other ; 200. Verified email, owner scope; no policy gate on ordinary update/photos. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 200 own / 403 other ; 200. Verified email, owner scope; no policy gate on ordinary update/photos. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/VenueController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-088 — POST /venues/{id}/submit-for-review: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `VenueModerationDto submitForReview(@PathVariable UUID id)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send POST /api/venues/{id}/submit-for-review with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 200 own / 403 other ; 403. Verified email, policies, owner verification and readiness required. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 401 ; 403 ; 200 own / 403 other ; 403. Verified email, policies, owner verification and readiness required. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/VenueController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-089 — GET /venues/{id}/moderation: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `VenueModerationDto moderation(@PathVariable UUID id)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send GET /api/venues/{id}/moderation with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 403 ; 403 ; 200 own / 403 other ; 200. Public GET matcher plus method guard; AccessDeniedException handled as 403 by controller advice. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 403 ; 403 ; 200 own / 403 other ; 200. Public GET matcher plus method guard; AccessDeniedException handled as 403 by controller advice. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/VenueController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

API-090 — POST /webhooks/mollie/payment: role/ownership/status contract

Area: API · **Priority:** P0 · **Test level:** API, SECURITY · **Result:** NOT RUN

Roles involved: anonymous, Scout A/B, Owner A/B, Admin

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Controller signature: `ResponseEntity<Void> handlePaymentWebhook(@RequestParam("id") String paymentId)`. Request DTO validation scenarios INPUT-* define field bounds.

Exact steps: 1. Select a fresh valid fixture for this operation using domain recipe and request DTO below. 2. Send POST /api/webhooks/mollie/payment with valid body/query and no token, then Scout A, Owner A and ADMIN tokens. 3. Repeat each allowed object call using Scout B/Owner B-owned ID, nonexistent UUID, malformed UUID and invalid body. 4. For mutation reset fixture between actors; replay in wrong state and double-submit. 5. Compare response, database rows, audit and provider call count; follow mapped UI route if exposed.

Expected UI behavior: Mapped route displays authoritative response; API-only operations have no current control. Role guards are UX only; direct API denial is mandatory.

Expected API/backend behavior: Role cells anonymous / SCOUT / OWNER / ADMIN: 200 ; 200 ; 200 ; 200. Public provider callback; form id validated and remote Mollie state fetched; malformed id 400, integrity/provider processing failure may be 500. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected database/domain state: Role cells anonymous / SCOUT / OWNER / ADMIN: 200 ; 200 ; 200 ; 200. Public provider callback; form id validated and remote Mollie state fetched; malformed id 400, integrity/provider processing failure may be 500. Standard invalid body/type 400, missing resource 404, business state conflict 409 where service defines it. Do not treat arbitrary 4xx/5xx as success.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/controller/WebhookController.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

STATE scenarios

STATE-001 — ActorType exhaustive state representation

Area: STATE · **Priority:** P0 · **Test level:** INTEGRATION, API, MANUAL UX · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: For each value SCOUT, OWNER, ADMIN, SYSTEM, WEBHOOK, create an isolated entity/event/DTO fixture, serialize and round-trip through PostgreSQL mapping and its response/UI badge or admin filter where exposed. For lifecycle enums call each supported outgoing transition and every unsupported target; for event/actor enums trigger the corresponding domain event and inspect actor/payload. Use fixture-only states if there is no application transition.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Every ActorType value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected database/domain state: Every ActorType value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/entity/enums/ActorType.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

STATE-002 — AvailabilityInquiryStatus exhaustive state representation

Area: STATE · **Priority:** P1 · **Test level:** INTEGRATION, API, MANUAL UX · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: For each value OPEN, DECLINED, OFFERED, EXPIRED, ACCEPTED, create an isolated entity/event/DTO fixture, serialize and round-trip through PostgreSQL mapping and its response/UI badge or admin filter where exposed. For lifecycle enums call each supported outgoing transition and every unsupported target; for event/actor enums trigger the corresponding domain event and inspect actor/payload. Use fixture-only states if there is no application transition.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Every AvailabilityInquiryStatus value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected database/domain state: Every AvailabilityInquiryStatus value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/entity/enums/AvailabilityInquiryStatus.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

STATE-003 — AvailabilityOverrideType exhaustive state representation

Area: STATE · **Priority:** P1 · **Test level:** INTEGRATION, API, MANUAL UX · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: For each value AVAILABLE, BLOCKED, create an isolated entity/event/DTO fixture, serialize and round-trip through PostgreSQL mapping and its response/UI badge or admin filter where exposed. For lifecycle enums call each supported outgoing transition and every unsupported target; for event/actor enums trigger the corresponding domain event and inspect actor/payload. Use fixture-only states if there is no application transition.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Every AvailabilityOverrideType value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected database/domain state: Every AvailabilityOverrideType value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/entity/enums/AvailabilityOverrideType.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

STATE-004 — BookingEventType exhaustive state representation

Area: STATE · **Priority:** P0 · **Test level:** INTEGRATION, API, MANUAL UX · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: For each value REQUEST_SUBMITTED, REQUEST_ACCEPTED, REQUEST_DECLINED, REQUEST_EXPIRED, REQUEST_BECAME_UNAVAILABLE, APPROVAL_EXPIRED, BOOKING_CONFIRMED, PAYMENT_SUCCEEDED, PAYMENT_REFUNDED, PAYMENT_RECONCILED, REFUND_REQUESTED, REFUND_STATUS_CHANGED, LATE_PAYMENT_DETECTED, LATE_PAYMENT_REFUND_REQUESTED, LATE_PAYMENT_REFUND_COMPLETED, SCOUT_CANCELLED, OWNER_CANCELLED, ADMIN_CANCELLED, PAYOUT_CALCULATED, PAYOUT_READY, PAYOUT_MARKED_PAID, PAYOUT_CANCELLED, OWNER_LIABILITY_CREATED, OWNER_LIABILITY_SETTLED, BOOKING_RECEIPT_ISSUED, OWNER_PAYOUT_STATEMENT_ISSUED, create an isolated entity/event/DTO fixture, serialize and round-trip through PostgreSQL mapping and its response/UI badge or admin filter where exposed. For lifecycle enums call each supported outgoing transition and every unsupported target; for event/actor enums trigger the corresponding domain event and inspect actor/payload. Use fixture-only states if there is no application transition.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Every BookingEventType value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected database/domain state: Every BookingEventType value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/entity/enums/BookingEventType.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

STATE-005 — BookingStatus exhaustive state representation

Area: STATE · **Priority:** P0 · **Test level:** INTEGRATION, API, MANUAL UX · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: For each value REQUESTED, AWAITING_PAYMENT, CONFIRMED, DECLINED, EXPIRED, UNAVAILABLE, CANCELLED, REFUNDED, create an isolated entity/event/DTO fixture, serialize and round-trip through PostgreSQL mapping and its response/UI badge or admin filter where exposed. For lifecycle enums call each supported outgoing transition and every unsupported target; for event/actor enums trigger the corresponding domain event and inspect actor/payload. Use fixture-only states if there is no application transition.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Every BookingStatus value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected database/domain state: Every BookingStatus value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/entity/enums/BookingStatus.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

STATE-006 — CancellationPolicyTier exhaustive state representation

Area: STATE · **Priority:** P0 · **Test level:** INTEGRATION, API, MANUAL UX · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: For each value PRE_PAYMENT_WITHDRAWAL, GRACE_PERIOD, FLEXIBLE_FULL_REFUND, PARTIAL_REFUND, NON_REFUNDABLE, OWNER_FULL_REFUND, ADMIN_FULL_REFUND, create an isolated entity/event/DTO fixture, serialize and round-trip through PostgreSQL mapping and its response/UI badge or admin filter where exposed. For lifecycle enums call each supported outgoing transition and every unsupported target; for event/actor enums trigger the corresponding domain event and inspect actor/payload. Use fixture-only states if there is no application transition.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Every CancellationPolicyTier value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected database/domain state: Every CancellationPolicyTier value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/entity/enums/CancellationPolicyTier.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

STATE-007 — FinancialDocumentStatus exhaustive state representation

Area: STATE · **Priority:** P1 · **Test level:** INTEGRATION, API, MANUAL UX · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: For each value ISSUED, create an isolated entity/event/DTO fixture, serialize and round-trip through PostgreSQL mapping and its response/UI badge or admin filter where exposed. For lifecycle enums call each supported outgoing transition and every unsupported target; for event/actor enums trigger the corresponding domain event and inspect actor/payload. Use fixture-only states if there is no application transition.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Every FinancialDocumentStatus value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected database/domain state: Every FinancialDocumentStatus value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/entity/enums/FinancialDocumentStatus.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

STATE-008 — FinancialDocumentType exhaustive state representation

Area: STATE · **Priority:** P1 · **Test level:** INTEGRATION, API, MANUAL UX · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: For each value BOOKING_RECEIPT, OWNER_PAYOUT_STATEMENT, create an isolated entity/event/DTO fixture, serialize and round-trip through PostgreSQL mapping and its response/UI badge or admin filter where exposed. For lifecycle enums call each supported outgoing transition and every unsupported target; for event/actor enums trigger the corresponding domain event and inspect actor/payload. Use fixture-only states if there is no application transition.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Every FinancialDocumentType value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected database/domain state: Every FinancialDocumentType value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/entity/enums/FinancialDocumentType.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

STATE-009 — IncidentCategory exhaustive state representation

Area: STATE · **Priority:** P1 · **Test level:** INTEGRATION, API, MANUAL UX · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: For each value PROPERTY_DAMAGE, ACCESS_PROBLEM, SAFETY, NOISE_NEIGHBOUR, MISREPRESENTATION, CONDUCT, OTHER, create an isolated entity/event/DTO fixture, serialize and round-trip through PostgreSQL mapping and its response/UI badge or admin filter where exposed. For lifecycle enums call each supported outgoing transition and every unsupported target; for event/actor enums trigger the corresponding domain event and inspect actor/payload. Use fixture-only states if there is no application transition.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Every IncidentCategory value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected database/domain state: Every IncidentCategory value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/entity/enums/IncidentCategory.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

STATE-010 — IncidentStatus exhaustive state representation

Area: STATE · **Priority:** P1 · **Test level:** INTEGRATION, API, MANUAL UX · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: For each value OPEN, UNDER_REVIEW, RESOLVED, CLOSED, create an isolated entity/event/DTO fixture, serialize and round-trip through PostgreSQL mapping and its response/UI badge or admin filter where exposed. For lifecycle enums call each supported outgoing transition and every unsupported target; for event/actor enums trigger the corresponding domain event and inspect actor/payload. Use fixture-only states if there is no application transition.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Every IncidentStatus value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected database/domain state: Every IncidentStatus value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/entity/enums/IncidentStatus.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

STATE-011 — OperationalExceptionSeverity exhaustive state representation

Area: STATE · **Priority:** P1 · **Test level:** INTEGRATION, API, MANUAL UX · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: For each value INFO, WARNING, CRITICAL, create an isolated entity/event/DTO fixture, serialize and round-trip through PostgreSQL mapping and its response/UI badge or admin filter where exposed. For lifecycle enums call each supported outgoing transition and every unsupported target; for event/actor enums trigger the corresponding domain event and inspect actor/payload. Use fixture-only states if there is no application transition.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Every OperationalExceptionSeverity value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected database/domain state: Every OperationalExceptionSeverity value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/entity/enums/OperationalExceptionSeverity.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

STATE-012 — OperationalExceptionType exhaustive state representation

Area: STATE · **Priority:** P1 · **Test level:** INTEGRATION, API, MANUAL UX · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: For each value PAYOUT_READY_OVERDUE, PAYOUT_HELD, OWNER_LIABILITY_OPEN, REFUND_STUCK, PAYMENT_CREATION_STUCK, MOLLIE_FEE_UNRECONCILED, LATE_PAYMENT_REFUND_IN_PROGRESS, create an isolated entity/event/DTO fixture, serialize and round-trip through PostgreSQL mapping and its response/UI badge or admin filter where exposed. For lifecycle enums call each supported outgoing transition and every unsupported target; for event/actor enums trigger the corresponding domain event and inspect actor/payload. Use fixture-only states if there is no application transition.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Every OperationalExceptionType value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected database/domain state: Every OperationalExceptionType value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/entity/enums/OperationalExceptionType.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

STATE-013 — OwnerFinancialLiabilityStatus exhaustive state representation

Area: STATE · **Priority:** P0 · **Test level:** INTEGRATION, API, MANUAL UX · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: For each value OPEN, SETTLED, WAIVED, create an isolated entity/event/DTO fixture, serialize and round-trip through PostgreSQL mapping and its response/UI badge or admin filter where exposed. For lifecycle enums call each supported outgoing transition and every unsupported target; for event/actor enums trigger the corresponding domain event and inspect actor/payload. Use fixture-only states if there is no application transition.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Every OwnerFinancialLiabilityStatus value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected database/domain state: Every OwnerFinancialLiabilityStatus value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/entity/enums/OwnerFinancialLiabilityStatus.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

STATE-014 — OwnerFinancialLiabilityType exhaustive state representation

Area: STATE · **Priority:** P0 · **Test level:** INTEGRATION, API, MANUAL UX · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: For each value PROVIDER_COST, OWNER_PAYOUT_RECOVERY, create an isolated entity/event/DTO fixture, serialize and round-trip through PostgreSQL mapping and its response/UI badge or admin filter where exposed. For lifecycle enums call each supported outgoing transition and every unsupported target; for event/actor enums trigger the corresponding domain event and inspect actor/payload. Use fixture-only states if there is no application transition.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Every OwnerFinancialLiabilityType value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected database/domain state: Every OwnerFinancialLiabilityType value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/entity/enums/OwnerFinancialLiabilityType.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

STATE-015 — OwnerPayoutHoldReason exhaustive state representation

Area: STATE · **Priority:** P0 · **Test level:** INTEGRATION, API, MANUAL UX · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: For each value WAITING_FOR_BOOKING_END, WAITING_FOR_PROVIDER_COST, REFUND_IN_PROGRESS, WAITING_FOR_PAYOUT_DESTINATION, create an isolated entity/event/DTO fixture, serialize and round-trip through PostgreSQL mapping and its response/UI badge or admin filter where exposed. For lifecycle enums call each supported outgoing transition and every unsupported target; for event/actor enums trigger the corresponding domain event and inspect actor/payload. Use fixture-only states if there is no application transition.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Every OwnerPayoutHoldReason value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected database/domain state: Every OwnerPayoutHoldReason value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/entity/enums/OwnerPayoutHoldReason.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

STATE-016 — OwnerPayoutStatus exhaustive state representation

Area: STATE · **Priority:** P0 · **Test level:** INTEGRATION, API, MANUAL UX · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: For each value CALCULATED, HELD, READY, PAID, CANCELLED, create an isolated entity/event/DTO fixture, serialize and round-trip through PostgreSQL mapping and its response/UI badge or admin filter where exposed. For lifecycle enums call each supported outgoing transition and every unsupported target; for event/actor enums trigger the corresponding domain event and inspect actor/payload. Use fixture-only states if there is no application transition.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Every OwnerPayoutStatus value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected database/domain state: Every OwnerPayoutStatus value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/entity/enums/OwnerPayoutStatus.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

STATE-017 — OwnerVerificationStatus exhaustive state representation

Area: STATE · **Priority:** P1 · **Test level:** INTEGRATION, API, MANUAL UX · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: For each value UNVERIFIED, PENDING, VERIFIED, REJECTED, create an isolated entity/event/DTO fixture, serialize and round-trip through PostgreSQL mapping and its response/UI badge or admin filter where exposed. For lifecycle enums call each supported outgoing transition and every unsupported target; for event/actor enums trigger the corresponding domain event and inspect actor/payload. Use fixture-only states if there is no application transition.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Every OwnerVerificationStatus value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected database/domain state: Every OwnerVerificationStatus value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/entity/enums/OwnerVerificationStatus.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

STATE-018 — PaymentCreationStatus exhaustive state representation

Area: STATE · **Priority:** P0 · **Test level:** INTEGRATION, API, MANUAL UX · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: For each value NOT_STARTED, CREATING, CREATED, create an isolated entity/event/DTO fixture, serialize and round-trip through PostgreSQL mapping and its response/UI badge or admin filter where exposed. For lifecycle enums call each supported outgoing transition and every unsupported target; for event/actor enums trigger the corresponding domain event

and inspect actor/payload. Use fixture-only states if there is no application transition.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Every PaymentCreationStatus value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected database/domain state: Every PaymentCreationStatus value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/entity/enums/PaymentCreationStatus.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

STATE-019 — PayoutDestinationStatus exhaustive state representation

Area: STATE · **Priority:** P0 · **Test level:** INTEGRATION, API, MANUAL UX · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: For each value PENDING_VERIFICATION, VERIFIED, REJECTED, REPLACED, create an isolated entity/event/DTO fixture, serialize and round-trip through PostgreSQL mapping and its response/UI badge or admin filter where exposed. For lifecycle enums call each supported outgoing transition and every unsupported target; for event/actor enums trigger the corresponding domain event and inspect actor/payload. Use fixture-only states if there is no application transition.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Every PayoutDestinationStatus value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not

client-writable.

Expected database/domain state: Every PayoutDestinationStatus value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/entity/enums/PayoutDestinationStatus.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

STATE-020 — PayoutDestinationType exhaustive state representation

Area: STATE · **Priority:** P0 · **Test level:** INTEGRATION, API, MANUAL UX · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: For each value IBAN, create an isolated entity/event/DTO fixture, serialize and round-trip through PostgreSQL mapping and its response/UI badge or admin filter where exposed. For lifecycle enums call each supported outgoing transition and every unsupported target; for event/actor enums trigger the corresponding domain event and inspect actor/payload. Use fixture-only states if there is no application transition.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Every PayoutDestinationType value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected database/domain state: Every PayoutDestinationType value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/entity/enums/PayoutDestinationType.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

STATE-021 — PolicyStatus exhaustive state representation

Area: STATE · **Priority:** P0 · **Test level:** INTEGRATION, API, MANUAL UX · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: For each value DRAFT, PUBLISHED, RETIRED, create an isolated entity/event/DTO fixture, serialize and round-trip through PostgreSQL mapping and its response/UI badge or admin filter where exposed. For lifecycle enums call each supported outgoing transition and every unsupported target; for event/actor enums trigger the corresponding domain event and inspect actor/payload. Use fixture-only states if there is no application transition.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Every PolicyStatus value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected database/domain state: Every PolicyStatus value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/entity/enums/PolicyStatus.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

STATE-022 — PolicyType exhaustive state representation

Area: STATE · **Priority:** P0 · **Test level:** INTEGRATION, API, MANUAL UX · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: For each value TERMS_OF_SERVICE, PRIVACY_POLICY, ACCEPTABLE_USE_POLICY, PRODUCTION_POLICY, INSURANCE_REQUIREMENTS, create an isolated entity/event/DTO fixture, serialize and round-trip through PostgreSQL mapping and its response/UI badge or admin filter where exposed. For lifecycle enums call each supported outgoing transition and every unsupported target; for event/actor enums trigger the corresponding domain event and inspect actor/payload. Use fixture-only states if there is no application transition.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Every PolicyType value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected database/domain state: Every PolicyType value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/entity/enums/PolicyType.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

STATE-023 — ProductionActivity exhaustive state representation

Area: STATE · **Priority:** P1 · **Test level:** INTEGRATION, API, MANUAL UX · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: For each value PHOTO, FILM, VIDEO, INTERVIEW, COMMERCIAL, MUSIC_VIDEO, CONTENT_CREATION, create an isolated entity/event/DTO fixture, serialize and round-trip through PostgreSQL mapping and its response/UI badge or admin filter where exposed. For lifecycle enums call each supported outgoing transition and every unsupported target; for event/actor enums trigger the corresponding domain event and inspect actor/payload. Use fixture-only states if there is no application transition.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Every ProductionActivity value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected database/domain state: Every ProductionActivity value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/entity/enums/ProductionActivity.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

STATE-024 — ProviderCostResponsibility exhaustive state representation

Area: STATE · **Priority:** P1 · **Test level:** INTEGRATION, API, MANUAL UX · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: For each value NONE, OWNER_SETTLEMENT, OWNER, EEMA, create an isolated entity/event/DTO fixture, serialize and round-trip through PostgreSQL mapping and its response/UI badge or admin filter where exposed. For lifecycle enums call each supported outgoing transition and every unsupported target; for event/actor enums trigger the corresponding domain event and inspect actor/payload. Use fixture-only states if there is no application transition.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Every ProviderCostResponsibility value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected database/domain state: Every ProviderCostResponsibility value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/entity/enums/ProviderCostResponsibility.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

STATE-025 — RefundStatus exhaustive state representation

Area: STATE · **Priority:** P0 · **Test level:** INTEGRATION, API, MANUAL UX · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: For each value NONE, CREATING, QUEUED, PENDING, PROCESSING, REFUNDED, FAILED, CANCELED, create an isolated entity/event/DTO fixture, serialize and round-trip through PostgreSQL mapping and its response/UI badge or admin filter where exposed. For lifecycle enums call each supported outgoing transition and every unsupported target; for event/actor enums trigger the corresponding domain event and inspect actor/payload. Use fixture-only states if there is no application transition.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Every RefundStatus value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected database/domain state: Every RefundStatus value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/entity/enums/RefundStatus.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

STATE-026 — TrustEventType exhaustive state representation

Area: STATE · **Priority:** P1 · **Test level:** INTEGRATION, API, MANUAL UX · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: For each value OWNER_VERIFICATION_SUBMITTED, OWNER_VERIFICATION_APPROVED, OWNER_VERIFICATION_REJECTED, VENUE_SUBMITTED, VENUE_APPROVED, VENUE_CHANGES_REQUESTED, VENUE_SUSPENDED, VENUE_REACTIVATED, PAYOUT_DESTINATION_SUBMITTED, PAYOUT_DESTINATION_VERIFIED, PAYOUT_DESTINATION_REJECTED, PAYOUT_DESTINATION_REPLACED, PAYOUT_DESTINATION_REVEALED, PAYOUT_DESTINATION_SNAPSHOTTED, INCIDENT_REPORTED, INCIDENT_REVIEW_STARTED, INCIDENT_RESOLVED, INCIDENT_CLOSED, create an isolated entity/event/DTO fixture, serialize and round-trip through PostgreSQL mapping and its response/UI badge or admin filter where exposed. For lifecycle enums call each supported outgoing transition and every unsupported target; for event/actor enums trigger the corresponding domain event and inspect actor/payload. Use fixture-only states if there is no application transition.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Every TrustEventType value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected database/domain state: Every TrustEventType value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/entity/enums/TrustEventType.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

STATE-027 — TrustSubjectType exhaustive state representation

Area: STATE · **Priority:** P1 · **Test level:** INTEGRATION, API, MANUAL UX · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: For each value OWNER_VERIFICATION, VENUE, PAYOUT_DESTINATION, INCIDENT, create an isolated entity/event/DTO fixture, serialize and round-trip through PostgreSQL mapping and its response/UI badge or admin filter where exposed. For lifecycle enums call each supported outgoing transition and every unsupported target; for event/actor enums trigger the corresponding domain event and inspect actor/payload. Use fixture-only states if there is no application transition.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Every TrustSubjectType value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected database/domain state: Every TrustSubjectType value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/entity/enums/TrustSubjectType.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

STATE-028 — UserRole exhaustive state representation

Area: STATE · **Priority:** P0 · **Test level:** INTEGRATION, API, MANUAL UX · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: For each value SCOUT, OWNER, ADMIN, create an isolated entity/event/DTO fixture, serialize and round-trip through PostgreSQL mapping and its response/UI badge or admin filter where exposed. For lifecycle enums call each supported outgoing transition and every unsupported target; for event/actor enums trigger the corresponding domain event and inspect actor/payload. Use fixture-only states if there is no application transition.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Every UserRole value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected database/domain state: Every UserRole value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/entity/enums/UserRole.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

STATE-029 — VenueStatus exhaustive state representation

Area: STATE · **Priority:** P1 · **Test level:** INTEGRATION, API, MANUAL UX · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: For each value DRAFT, PENDING, ACTIVE, SUSPENDED, create an isolated entity/event/DTO fixture, serialize and round-trip through PostgreSQL mapping and its response/UI badge or admin filter where exposed. For lifecycle enums call each supported outgoing transition and every unsupported target; for event/actor enums trigger the corresponding domain event and inspect actor/payload. Use fixture-only states if there is no application transition.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Every VenueStatus value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected database/domain state: Every VenueStatus value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/entity/enums/VenueStatus.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

STATE-030 — VerificationMethod exhaustive state representation

Area: STATE · **Priority:** P1 · **Test level:** INTEGRATION, API, MANUAL UX · **Result:** NOT RUN

Roles involved: Scout A, Owner A; repeat denial with Scout B, Owner B and anonymous

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: For each value MANUAL_REVIEW, VIDEO_CALL, BUSINESS_DOCUMENT, VENUE_CONTROL_EVIDENCE, create an isolated entity/event/DTO fixture, serialize and round-trip through PostgreSQL mapping and its response/UI badge or admin filter where exposed. For lifecycle enums call each supported outgoing transition and every unsupported target; for event/actor enums trigger the corresponding domain event and inspect actor/payload. Use fixture-only states if there is no application transition.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Every VerificationMethod value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected database/domain state: Every VerificationMethod value round-trips without silently defaulting. Unknown string rejected; nullable field stays null where optional. Enum presence does not imply a public mutation (WAIVED, event-only types, SYSTEM/WEBHOOK). Money-state semantics and forbidden transitions follow the domain cases; financial audit is not client-writable.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/main/java/com/eema/backend/entity/enums/VerificationMethod.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

STATE-031 — BookingLifecycleEvent.Type internal branch coverage

Area: STATE · **Priority:** P0 · **Test level:** UNIT, INTEGRATION · **Result:** NOT RUN

Roles involved: SYSTEM, WEBHOOK; captured participants

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Invoke the lifecycle event/result branch for each REQUEST_SUBMITTED, REQUEST_ACCEPTED, REQUEST_DECLINED, REQUEST_EXPIRED, REQUEST_BECAME_UNAVAILABLE, APPROVAL_EXPIRED in service/listener fixtures; inspect matching notification/event/counter behavior; repeat event after state has advanced and after rollback.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Internal enum is not user-input status; each branch preserves state semantics, skips stale notification where guarded and emits only intended after-commit effects. Paid result APPLIED confirms, LATE_PAYMENT initiates refund while EXPIRED, DUPLICATE adds no payment effect.

Expected database/domain state: Internal enum is not user-input status; each branch preserves state semantics, skips stale notification where guarded and emits only intended after-commit effects. Paid result APPLIED confirms, LATE_PAYMENT initiates refund while EXPIRED, DUPLICATE adds no payment effect.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/main/java/com/eema/backend/event/BookingLifecycleEvent.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

STATE-032 — AvailabilityInquiryLifecycleEvent.Type internal branch coverage

Area: STATE · **Priority:** P0 · **Test level:** UNIT, INTEGRATION · **Result:** NOT RUN

Roles involved: SYSTEM, WEBHOOK; captured participants

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Invoke the lifecycle event/result branch for each CREATED, OFFERED, DECLINED, OFFER_EXPIRED in service/listener fixtures; inspect matching notification/event/counter behavior; repeat event after state has advanced and after rollback.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Internal enum is not user-input status; each branch preserves state semantics, skips stale notification where guarded and emits only intended after-commit effects. Paid result APPLIED confirms, LATE_PAYMENT initiates refund while EXPIRED, DUPLICATE adds no payment effect.

Expected database/domain state: Internal enum is not user-input status; each branch preserves state semantics, skips stale notification where guarded and emits only intended after-commit effects. Paid result APPLIED confirms, LATE_PAYMENT initiates refund while EXPIRED, DUPLICATE adds no payment effect.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/main/java/com/eema/backend/event/AvailabilityInquiryLifecycleEvent.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

STATE-033 — BookingService.PaidPaymentResult internal branch coverage

Area: STATE · **Priority:** P0 · **Test level:** UNIT, INTEGRATION · **Result:** NOT RUN

Roles involved: SYSTEM, WEBHOOK; captured participants

Preconditions: Local stack healthy; relevant actor active, email verified and current policies accepted unless varied.

Test data: Standard fixtures below; use a fresh aggregate for each independent variant.

Exact steps: Invoke the lifecycle event/result branch for each APPLIED, LATE_PAYMENT, DUPLICATE in service/listener fixtures; inspect matching notification/event/counter behavior; repeat event after state has advanced and after rollback.

Expected UI behavior: Show the authoritative returned state or a safe actionable error; refresh/reload must agree. No optimistic money or status claim.

Expected API/backend behavior: Internal enum is not user-input status; each branch preserves state semantics, skips stale notification where guarded and emits only intended after-commit effects. Paid result APPLIED confirms, LATE_PAYMENT initiates refund while EXPIRED, DUPLICATE adds no payment effect.

Expected database/domain state: Internal enum is not user-input status; each branch preserves state semantics, skips stale notification where guarded and emits only intended after-commit effects. Paid result APPLIED confirms, LATE_PAYMENT initiates refund while EXPIRED, DUPLICATE adds no payment effect.

Expected email/provider behavior: No email/provider operation unless the lifecycle email matrix explicitly names this transition; capture calls and verify none on rejection.

Security/authorization expectation: Use the endpoint role matrix and a different owner/scout UUID; denial must leave domain state unchanged.

Cleanup/reset steps: Restore the isolated fixture snapshot; clear browser session and discard captured tokens. Never delete shared staging history.

Automation recommendation: SHOULD AUTOMATE

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/main/java/com/eema/backend/service/BookingService.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO scenarios

AUTO-001 — SimpleTest#testTrue

Area: AUTO · **Priority:** P3 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=SimpleTest#testTrue`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        assertTrue(true);
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/SimpleTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-002 — AsyncConfigurationTest#createsBoundedNamedExecutor

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=AsyncConfigurationTest#createsBoundedNamedExecutor`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        ThreadPoolTaskExecutor executor = (ThreadPoolTaskExecutor) new AsyncConfiguration().notificationExecutor(2, 4, 25);
        assertThat(executor.getCorePoolSize()).isEqualTo(2);
        assertThat(executor.getMaxPoolSize()).isEqualTo(4);
        assertThat(executor.getThreadPoolExecutor().getQueue().remainingCapacity()).isEqualTo(25);
        assertThat(executor.getThreadNamePrefix()).isEqualTo("eema-async-");
        executor.shutdown();
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/config/AsyncConfigurationTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-003 — FlywayLocationTest#developmentSeedsAreNotInCommonProductionLocation

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=FlywayLocationTest#developmentSeedsAreNotInCommonProductionLocation`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        ClassLoader loader = getClass().getClassLoader();
        assertNull(loader.getResource("db/migration/common/V100__development_seed.sql"));
        assertNotNull(loader.getResource("db/migration/dev/V100__development_seed.sql"));
        assertNotNull(loader.getResource("db/migration/common/V1__initial_schema.sql"));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/config/FlywayLocationTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-004 — ProductionConfigurationValidatorTest#rejectsLocalStorageInProduction

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=ProductionConfigurationValidatorTest#rejectsLocalStorageInProduction`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        MockEnvironment environment = productionEnvironment();
        ProductionConfigurationValidator validator = new ProductionConfigurationValidator(environment, properties("local", strongSecret(),
List.of("https://app.eema.example")));
        assertThatThrownBy(() -> validator.run(new DefaultApplicationArguments())).hasMessageContaining("provider must be s3");
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/config/ProductionConfigurationValidatorTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-005 — ProductionConfigurationValidatorTest#rejectsWeakJwtInProduction

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=ProductionConfigurationValidatorTest#rejectsWeakJwtInProduction`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        MockEnvironment environment = productionEnvironment();
        ProductionConfigurationValidator validator = new ProductionConfigurationValidator(environment,
                properties("s3", Base64.getEncoder().encodeToString("short".getBytes()), List.of("https://app.eema.example")));
        assertThatThrownBy(() -> validator.run(new DefaultApplicationArguments())).hasMessageContaining("256 bits");
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/config/ProductionConfigurationValidatorTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-006 — ProductionConfigurationValidatorTest#rejectsWildcardCorsInProduction

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=ProductionConfigurationValidatorTest#rejectsWildcardCorsInProduction`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        MockEnvironment environment = productionEnvironment();
        ProductionConfigurationValidator validator = new ProductionConfigurationValidator(environment,
                properties("s3", strongSecret(), List.of("*")));
        assertThatThrownBy(() -> validator.run(new DefaultApplicationArguments())).hasMessageContaining("CORS_ALLOWED_ORIGINS");
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/config/ProductionConfigurationValidatorTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-007 — ProductionConfigurationValidatorTest#rejectsSharedBuckets

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=ProductionConfigurationValidatorTest#rejectsSharedBuckets`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        MockEnvironment environment = productionEnvironment().withProperty("S3_BUCKET_PRIVATE_DOCUMENTS", "media");
        ProductionConfigurationValidator validator = new ProductionConfigurationValidator(environment,
                properties("s3", strongSecret(), List.of("https://app.eema.example")));
        assertThatThrownBy(() -> validator.run(new DefaultApplicationArguments())).hasMessageContaining("different S3 buckets");
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/config/ProductionConfigurationValidatorTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-008 — ProductionConfigurationValidatorTest#requiresS3Credentials

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=ProductionConfigurationValidatorTest#requiresS3Credentials`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        MockEnvironment environment = productionEnvironment().withProperty("S3_SECRET_KEY", "");
        ProductionConfigurationValidator validator = new ProductionConfigurationValidator(environment,
                properties("s3", strongSecret(), List.of("https://app.eema.example")));
        assertThatThrownBy(() -> validator.run(new DefaultApplicationArguments())).hasMessageContaining("S3_SECRET_KEY");
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/config/ProductionConfigurationValidatorTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-009 — AdminOperationsSurfaceTest#everyAdminControllerHasDefenseInDepthRoleGuard

Area: AUTO · **Priority:** P1 · **Test level:** UNIT, INTEGRATION · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=AdminOperationsSurfaceTest#everyAdminControllerHasDefenseInDepthRoleGuard`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        ADMIN_CONTROLLERS.forEach(type -> assertThat(type.getAnnotation(PreAuthorize.class))
                .as(type.getSimpleName()).isNotNull()
                .extracting(PreAuthorize::value).isEqualTo("hasRole('ADMIN')"));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/controller/AdminOperationsSurfaceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-010 — AdminOperationsSurfaceTest#surfaceContainsNoGenericPutPatchOrBookingStatusMutation

Area: AUTO · **Priority:** P1 · **Test level:** UNIT, INTEGRATION · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=AdminOperationsSurfaceTest#surfaceContainsNoGenericPutPatchOrBookingStatusMutation`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this

named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        for (Class<?> type : ADMIN_CONTROLLERS) {
            assertThat(type.getDeclaredMethods()).noneMatch(m -> m.isAnnotationPresent(PutMapping.class)
                    || m.isAnnotationPresent(PatchMapping.class));
        }
        Set<String> postPaths = new HashSet<>();
        for (Class<?> type : ADMIN_CONTROLLERS) for (Method method : type.getDeclaredMethods()) {
            PostMapping mapping = method.getAnnotation(PostMapping.class);
            if (mapping != null) postPaths.addAll(Arrays.asList(mapping.value()));
        }
        assertThat(postPaths).containsExactlyInAnyOrder("/{id}/mark-paid", "/{id}/settle");
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/controller/AdminOperationsSurfaceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-011 — AdminOwnerFinancialLiabilityControllerTest#scoutAndOwnerAreDeniedAllLiabilityOperations

Area: AUTO · **Priority:** P0 · **Test level:** UNIT, INTEGRATION · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=AdminOwnerFinancialLiabilityControllerTest#scoutAndOwnerAreDeniedAllLiabilityOperations`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        for (UserRole role : new UserRole[]{UserRole.SCOUT, UserRole.OWNER}) {
            when(current.getCurrentUser()).thenReturn(User.builder().role(role).build());
            assertThatThrownBy(() -> controller.list(null, null, null, null,
                    org.springframework.data.domain.Pageable.unpaged())).isInstanceOf(ForbiddenException.class);
            assertThatThrownBy(() -> controller.get(UUID.randomUUID())).isInstanceOf(ForbiddenException.class);
            assertThatThrownBy(() -> controller.settle(UUID.randomUUID())).isInstanceOf(ForbiddenException.class);
        }
        verifyNoInteractions(service);
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/controller/AdminOwnerFinancialLiabilityControllerTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-012 — AdminOwnerPayoutControllerTest#scoutAndOwnerCannotReadOrMarkPayouts

Area: AUTO · **Priority:** P0 · **Test level:** UNIT, INTEGRATION · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=AdminOwnerPayoutControllerTest#scoutAndOwnerCannotReadOrMarkPayouts`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        for (UserRole role : new UserRole[]{UserRole.SCOUT, UserRole.OWNER}) {
            when(current.getCurrentUser()).thenReturn(User.builder().id(UUID.randomUUID()).role(role).build());
            assertThatThrownBy(() -> controller.list(null, null, null, null,
org.springframework.data.domain.Pageable.unpaged())).isInstanceOf(ForbiddenException.class);
            assertThatThrownBy(() -> controller.get(UUID.randomUUID())).isInstanceOf(ForbiddenException.class);
            assertThatThrownBy(() -> controller.markPaid(UUID.randomUUID(), new MarkOwnerPayoutPaidRequest("bank-1")))
                    .isInstanceOf(ForbiddenException.class);
        }
        verifyNoInteractions(service);
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/controller/AdminOwnerPayoutControllerTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-013 — AdminOwnerPayoutControllerTest#adminCanUseOperationsEndpoints

Area: AUTO · **Priority:** P0 · **Test level:** UNIT, INTEGRATION · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=AdminOwnerPayoutControllerTest#adminCanUseOperationsEndpoints`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        User admin = User.builder().id(UUID.randomUUID()).role(UserRole.ADMIN).build();
        when(current.getCurrentUser()).thenReturn(admin);
        UUID id = UUID.randomUUID();
        controller.list(null, null, null, null, org.springframework.data.domain.Pageable.unpaged());
        controller.get(id);
        controller.markPaid(id, new MarkOwnerPayoutPaidRequest("bank-1"));
        verify(service).list(eq(null), eq(null), eq(null), eq(null), any());
        verify(service).get(id);
        verify(service).markPaid(id, "bank-1", admin);
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/controller/AdminOwnerPayoutControllerTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-014 — FinancialDocumentControllerTest#authorizedEndpointReturnsPdfAttachmentWithoutStoragePath

Area: AUTO · **Priority:** P0 · **Test level:** UNIT, INTEGRATION · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=FinancialDocumentControllerTest#authorizedEndpointReturnsPdfAttachmentWithoutStoragePath`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        FinancialDocumentService documents = mock(FinancialDocumentService.class);
        CurrentUserService current = mock(CurrentUserService.class);
        FinancialDocumentController controller = new FinancialDocumentController(documents, current);
        UUID id = UUID.randomUUID(); User scout = User.builder().id(UUID.randomUUID()).role(UserRole.SCOUT).build();
        when(current.getCurrentUser()).thenReturn(scout);
        when(documents.bookingReceipt(id, scout)).thenReturn(new FinancialDocumentService.DocumentDownload(
                "%PDF".getBytes(), "RCT-2026-000001", "eema-booking-receipt-RCT-2026-000001.pdf"));
        ResponseEntity<byte[]> response = controller.receipt(id);
        assertThat(response.getHeaders().getContentType()).isEqualTo(MediaType.APPLICATION_PDF);
        assertThat(response.getHeaders().getFirst(HttpHeaders.CONTENT_DISPOSITION))
                .isEqualTo("attachment; filename=\"eema-booking-receipt-RCT-2026-000001.pdf\"")
                .doesNotContain("/tmp", "booking-receipts/");
        assertThat(response.getBody()).startsWith("%PDF".getBytes());
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/controller/FinancialDocumentControllerTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-015 — TrustWorkflowSecuritySurfaceTest#adminQueuesAndActionsAreAdminOnly

Area: AUTO · **Priority:** P0 · **Test level:** UNIT, INTEGRATION · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=TrustWorkflowSecuritySurfaceTest#adminQueuesAndActionsAreAdminOnly`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        assertThat(AdminOwnerVerificationController.class.getAnnotation(PreAuthorize.class).value()).isEqualTo("hasRole('ADMIN')");
        assertThat(AdminVenueModerationController.class.getAnnotation(PreAuthorize.class).value()).isEqualTo("hasRole('ADMIN')");
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/controller/TrustWorkflowSecuritySurfaceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-016 — TrustWorkflowSecuritySurfaceTest#ownerVerificationAndSubmissionAreOwnerOnly

Area: AUTO · **Priority:** P0 · **Test level:** UNIT, INTEGRATION · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=TrustWorkflowSecuritySurfaceTest#ownerVerificationAndSubmissionAreOwnerOnly`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        assertThat(OwnerVerificationController.class.getAnnotation(PreAuthorize.class).value()).contains("OWNER");
        assertThat(VenueController.class.getMethod("submitForReview",java.util.UUID.class).getAnnotation(PreAuthorize.class).value())
                .contains("OWNER").contains("verifiedUserPolicy");
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/controller/TrustWorkflowSecuritySurfaceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-017 — VerifiedEmailEnforcementSurfaceTest#launchBlockerMarketplaceMutationsUseCentralVerifiedUserPolicy

Area: AUTO · **Priority:** P0 · **Test level:** UNIT, INTEGRATION · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=VerifiedEmailEnforcementSurfaceTest#launchBlockerMarketplaceMutationsUseCentralVerifiedUserPolicy`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        assertVerified(BookingController.class, "createBooking", CreateBookingRequest.class);
        assertVerified(AvailabilityInquiryController.class, "create", CreateAvailabilityInquiryRequest.class);
        assertVerified(VenueController.class, "createVenue", CreateVenueRequest.class);
        assertVerified(CheckoutController.class, "createCheckout", java.util.UUID.class);
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/controller/VerifiedEmailEnforcementSurfaceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-018 — PostgresCriticalPathIntegrationTest#commonFlywayChainAndHibernateValidationStartFromEmptyDatabase

Area: AUTO · **Priority:** P0 · **Test level:** INTEGRATION · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn verify -Ppostgres-integration -Dit.test=PostgresCriticalPathIntegrationTest#commonFlywayChainAndHibernateValidationStartFromEmptyDatabase`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        List<Integer> versions=jdbc.queryForList("select version::int from flyway_schema_history where success order by installed_rank",Integer.class);
        assertTrue(versions.containsAll(List.of(1,2,3,4,5,6,7,8,9,10,11)), ()->"common versions="+versions);
        assertTrue(versions.stream().noneMatch(v->v>=100), ()->"development migration leaked: "+versions);
        assertEquals("bookings_no_overlapping_reservations", jdbc.queryForObject("select conname from pg_constraint where
conname='bookings_no_overlapping_reservations'",String.class));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/test/java/com/eema/backend/integration/PostgresCriticalPathIntegrationTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-019 — PostgresCriticalPathIntegrationTest#nativeEnumsAndProductionActivityArraysRoundTripThroughHibernate

Area: AUTO · **Priority:** P0 · **Test level:** INTEGRATION · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn verify -Ppostgres-integration -Dit.test=PostgresCriticalPathIntegrationTest#nativeEnumsAndProductionActivityArraysRoundTripThroughHibernate`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        Fixture f=fixture();
        jdbc.update("update venues set status='PENDING'::venue_status, permitted_activities=array['FILM','PHOTO']::production_activity[] where
id=?",f.venue);
        Venue venue=venues.findById(f.venue).orElseThrow();
        assertEquals(VenueStatus.PENDING,venue.getStatus());
assertEquals(List.of(ProductionActivity.FILM,ProductionActivity.PHOTO),venue.getPermittedActivities());
        UUID booking=insertBooking(f,"REQUESTED","09:00","10:00");
        assertEquals(BookingStatus.REQUESTED,bookings.findById(booking).orElseThrow().getStatus());
        assertEquals(RefundStatus.NONE,bookings.findById(booking).orElseThrow().getRefundStatus());
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/test/java/com/eema/backend/integration/PostgresCriticalPathIntegrationTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-020 — PostgresCriticalPathIntegrationTest#exclusionRejectsOneOverlappingBlockingInsert

Area: AUTO · **Priority:** P0 · **Test level:** INTEGRATION, CONCURRENCY · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn verify -Ppostgres-integration -Dit.test=PostgresCriticalPathIntegrationTest#exclusionRejectsOneOverlappingBlockingInsert`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        Fixture f=fixture(); RaceResult result=race(
                c->insertBooking(c,f,"AWAITING_PAYMENT","09:00","12:00"),
                c->insertBooking(c,f,"CONFIRMED","10:00","11:00"));
        assertEquals(1,result.successes(),result::diagnostic); assertEquals(1,result.failures(),result::diagnostic);
        assertEquals(1,jdbc.queryForObject("select count(*) from bookings",Integer.class));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/test/java/com/eema/backend/integration/PostgresCriticalPathIntegrationTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-021 — PostgresCriticalPathIntegrationTest#halfOpenBackToBackReservationsBothPersist

Area: AUTO · **Priority:** P0 · **Test level:** INTEGRATION · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn verify -Ppostgres-integration -Dit.test=PostgresCriticalPathIntegrationTest#halfOpenBackToBackReservationsBothPersist`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        Fixture f=fixture();insertBooking(f,"AWAITING_PAYMENT","09:00","12:00");insertBooking(f,"CONFIRMED","12:00","15:00");
        assertEquals(2,jdbc.queryForObject("select count(*) from bookings",Integer.class));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/test/java/com/eema/backend/integration/PostgresCriticalPathIntegrationTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-022 — PostgresCriticalPathIntegrationTest#overlappingRequestedRowsCoexistButConcurrentAcceptanceCannotDoubleReserve

Area: AUTO · **Priority:** P0 · **Test level:** INTEGRATION, CONCURRENCY · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn verify -Ppostgres-integration -Dit.test=PostgresCriticalPathIntegrationTest#overlappingRequestedRowsCoexistButConcurrentAcceptanceCannotDoubleReserve`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        Fixture f=fixture();UUID a=insertBooking(f,"REQUESTED","09:00","12:00"),b=insertBooking(f,"REQUESTED","10:00","11:00");
        RaceResult result=race(c->updateStatus(c,a,"AWAITING_PAYMENT"),c->updateStatus(c,b,"AWAITING_PAYMENT"));
        assertEquals(1,result.successes(),result::diagnostic);
        assertEquals(1,jdbc.queryForObject("select count(*) from bookings where status='AWAITING_PAYMENT'",Integer.class));
        assertEquals(1,jdbc.queryForObject("select count(*) from bookings where status='REQUESTED'",Integer.class));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/test/java/com/eema/backend/integration/PostgresCriticalPathIntegrationTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-023 — PostgresCriticalPathIntegrationTest#twoSkipLockedWorkersReceiveDisjointBoundedBookingBatches

Area: AUTO · **Priority:** P0 · **Test level:** INTEGRATION, CONCURRENCY · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn verify -Ppostgres-integration -Dit.test=PostgresCriticalPathIntegrationTest#twoSkipLockedWorkersReceiveDisjointBoundedBookingBatches`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        Fixture f=fixture();for(int i=0;i<5;i++)insertBooking(f,"AWAITING_PAYMENT",String.format("0%d:00",i),String.format("0%d:30",i));
        SkipLockedResult r=skipLocked("select id from bookings where status='AWAITING_PAYMENT' order by created_at for update skip locked limit 3");
        assertTrue(Collections.disjoint(r.first,r.second),r::toString);assertEquals(5,new HashSet<UUID>()
{{addAll(r.first);addAll(r.second);}}.size());assertTrue(r.first.size()<=3&&r.second.size()<=3);
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/test/java/com/eema/backend/integration/PostgresCriticalPathIntegrationTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-024 — PostgresCriticalPathIntegrationTest#inquiryAndPayoutSkipLockedQueriesAreActuallyDisjoint

Area: AUTO · **Priority:** P0 · **Test level:** INTEGRATION, CONCURRENCY · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn verify -Ppostgres-integration -Dit.test=PostgresCriticalPathIntegrationTest#inquiryAndPayoutSkipLockedQueriesAreActuallyDisjoint`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        Fixture f=fixture();UUID booking=insertBooking(f,"CONFIRMED","09:00","10:00");
        for(int i=0;i<4;i++)jdbc.update("insert into
availability_inquiries(venue_id,scout_id,requested_date,requested_start_time,requested_end_time,message,status,offered_date,offered_start_time,offered_end_
time,offer_expires_at,offered_at) values(?,?,current_date+1,'09:00','10:00','test','OFFERED',current_date+1,'09:00','10:00',now()-interval '1
minute',now())",f.venue,f.scout);
        SkipLockedResult inquiries=skipLocked("select id from availability_inquiries where status='OFFERED' and offer_expires_at<=now() order by
offer_expires_at for update skip locked limit 2");
        assertTrue(Collections.disjoint(inquiries.first,inquiries.second));assertEquals(4,inquiries.first.size()+inquiries.second.size());
        jdbc.update("insert into
owner_payouts(booking_id,owner_id,venue_id,gross_retained_amount_eur,commission_eur,provider_cost_eur,payout_amount_eur,status,eligible_at)
values(?,?,?,100,20,0,80,'CALCULATED',now()-interval '1 minute')",booking,f.owner,f.venue);
        SkipLockedResult payouts=skipLocked("select id from owner_payouts where status in('CALCULATED','HELD') and eligible_at<=now() order by eligible_at
for update skip locked limit 1");
        assertEquals(1,payouts.first.size()+payouts.second.size());assertTrue(Collections.disjoint(payouts.first,payouts.second));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/test/java/com/eema/backend/integration/PostgresCriticalPathIntegrationTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-025 — PostgresCriticalPathIntegrationTest#refreshTokenRotationLockAllowsOnlyOneWinner

Area: AUTO · **Priority:** P0 · **Test level:** INTEGRATION, CONCURRENCY · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn verify -Ppostgres-integration -Dit.test=PostgresCriticalPathIntegrationTest#refreshTokenRotationLockAllowsOnlyOneWinner`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        Fixture f=fixture();String hash="a".repeat(64);jdbc.update("insert into refresh_tokens(user_id,token_hash,expires_at) values(?,?,now()+interval '1 hour')",f.scout,hash);
        RaceResult r=race(c->rotate(c,hash),c->rotate(c,hash));assertEquals(1,r.successes(),r::diagnostic);
        assertTrue(jdbc.queryForObject("select revoked from refresh_tokens where token_hash=?",Boolean.class,hash));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/test/java/com/eema/backend/integration/PostgresCriticalPathIntegrationTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-026 — PostgresCriticalPathIntegrationTest#payoutAndDocumentUniqueConstraintsHaveSingleAuthorities

Area: AUTO · **Priority:** P0 · **Test level:** INTEGRATION, CONCURRENCY · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn verify -Ppostgres-integration -Dit.test=PostgresCriticalPathIntegrationTest#payoutAndDocumentUniqueConstraintsHaveSingleAuthorities`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        Fixture f=fixture();UUID booking=insertBooking(f,"CONFIRMED","09:00","10:00");
        RaceResult payout=race(c->insertPayout(c,f,booking),c->insertPayout(c,f,booking));assertEquals(1,payout.successes(),payout::diagnostic);
        RaceResult receipt=race(c->insertReceipt(c,f,booking,"REC-1","private/a.pdf"),c->insertReceipt(c,f,booking,"REC-
2","private/b.pdf"));assertEquals(1,receipt.successes(),receipt::diagnostic);
        assertEquals(1,jdbc.queryForObject("select count(*) from financial_documents where booking_id=?",Integer.class,booking));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/test/java/com/eema/backend/integration/PostgresCriticalPathIntegrationTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-027 — PostgresCriticalPathIntegrationTest#verifiedDestinationPartialIndexAndReplacementSemanticsHold

Area: AUTO · **Priority:** P0 · **Test level:** INTEGRATION · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn verify -Ppostgres-integration -Dit.test=PostgresCriticalPathIntegrationTest#verifiedDestinationPartialIndexAndReplacementSemanticsHold`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        Fixture f=fixture();UUID first=destination(f,"PENDING_VERIFICATION","A"),second=destination(f,"PENDING_VERIFICATION","B");
        RaceResult r=race(c->verifyDestination(c,first),c->verifyDestination(c,second));assertEquals(1,r.successes(),r::diagnostic);
        UUID winner=jdbc.queryForObject("select id from owner_payout_destinations where status='VERIFIED'",UUID.class);
        UUID loser=winner.equals(first)?second:first;
        jdbc.update("update owner_payout_destinations set status='REPLACED',replaced_at=now() where id=?",winner);
        jdbc.update("update owner_payout_destinations set status='VERIFIED',verified_at=now(),verified_by=? where id=?",f.owner,loser);
```

```
        assertEquals(1,jdbc.queryForObject("select count(*) from owner_payout_destinations where owner_id=? and status='VERIFIED'",Integer.class,f.owner));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/test/java/com/eema/backend/integration/PostgresCriticalPathIntegrationTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-028 — PostgresCriticalPathIntegrationTest#readyPayoutDestinationSnapshotDoesNotFollowReplacement

Area: AUTO · **Priority:** P0 · **Test level:** INTEGRATION · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn verify -Ppostgres-integration -Dit.test=PostgresCriticalPathIntegrationTest#readyPayoutDestinationSnapshotDoesNotFollowReplacement`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        Fixture f=fixture();UUID booking=insertBooking(f,"CONFIRMED","09:00","10:00"),old=destination(f,"VERIFIED","A");insertPayout(null,f,booking);
        jdbc.update("update owner_payouts set status='READY',ready_at=now(),payout_destination_id=? where booking_id=?",old,booking);
        UUID replacement=destination(f,"PENDING_VERIFICATION","B");jdbc.update("update owner_payout_destinations set status='REPLACED',replaced_at=now()
```

```
where id=?",old);jdbc.update("update owner_payout_destinations set status='VERIFIED',verified_at=now(),verified_by=? where id=?",f.owner,replacement);
        assertEquals(old,jdbc.queryForObject("select payout_destination_id from owner_payouts where booking_id=?",UUID.class,booking));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/test/java/com/eema/backend/integration/PostgresCriticalPathIntegrationTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-029 — PostgresCriticalPathIntegrationTest#policyPublishRaceLeavesExactlyOneCurrentVersion

Area: AUTO · **Priority:** P0 · **Test level:** INTEGRATION, CONCURRENCY · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn verify -Ppostgres-integration -Dit.test=PostgresCriticalPathIntegrationTest#policyPublishRaceLeavesExactlyOneCurrentVersion`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        UUID a=draftPolicy("1"),b=draftPolicy("2");RaceResult r=race(c->publish(c,a),c->publish(c,b));assertEquals(1,r.successes(),r::diagnostic);
        assertEquals(1,jdbc.queryForObject("select count(*) from policy_versions where policy_type='TERMS_OF_SERVICE' and
```

```
status='PUBLISHED'",Integer.class));
        assertEquals(2,jdbc.queryForObject("select count(*) from policy_versions",Integer.class));
        assertNotNull(policies.findByPolicyTypeAndStatus(PolicyType.TERMS_OF_SERVICE,PolicyStatus.PUBLISHED).orElseThrow().getContentHash());
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/test/java/com/eema/backend/integration/PostgresCriticalPathIntegrationTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-030 — PostgresCriticalPathIntegrationTest#policyAcceptanceBookingSnapshotStaysHistoricalAndIncidentHasNoFinancialSideEffects

Area: AUTO · **Priority:** P0 · **Test level:** INTEGRATION · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn verify -Ppostgres-integration -Dit.test=PostgresCriticalPathIntegrationTest#policyAcceptanceBookingSnapshotStaysHistoricalAndIncidentHasNoFinancialSideEffects`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        Fixture f=fixture();Map<String,UUID> current=new HashMap<>();for(String
```

```
type:List.of("TERMS_OF_SERVICE","PRIVACY_POLICY","ACCEPTABLE_USE_POLICY","PRODUCTION_POLICY","INSURANCE_REQUIREMENTS")){UUID
id=UUID.randomUUID();jdbc.update("insert into policy_versions(id,policy_type,version,title,effective_at,published_at,content,content_hash,status)
values(?,?, '1','Approved fixture',now(),now(),'fixture',?,'PUBLISHED')",id,type,"f".repeat(64));jdbc.update("insert into
policy_acceptances(user_id,policy_version_id) values(?,?)",f.scout,id);current.put(type,id);}
        UUID booking=insertBooking(f,"REQUESTED","09:00","10:00");for(String
type:List.of("TERMS_OF_SERVICE","ACCEPTABLE_USE_POLICY","PRODUCTION_POLICY","INSURANCE_REQUIREMENTS"))jdbc.update("insert into booking_policy_versions
values(?,?,?)",booking,type,current.get(type));
        List<UUID> before=jdbc.queryForList("select policy_version_id from booking_policy_versions where booking_id=? order by
policy_type",UUID.class,booking);jdbc.update("update policy_versions set status='RETIRED' where policy_type='TERMS_OF_SERVICE'");draftPolicy("2");
        jdbc.update("insert into incident_reports(booking_id,reported_by_user_id,category,description) values(?,?, 'OTHER','Operational
fixture')",booking,f.scout);
        assertEquals(before,jdbc.queryForList("select policy_version_id from booking_policy_versions where booking_id=? order by
policy_type",UUID.class,booking));
        assertEquals("REQUESTED",jdbc.queryForObject("select status::text from bookings where
id=?",String.class,booking));assertEquals("NONE",jdbc.queryForObject("select refund_status from bookings where
id=?",String.class,booking));assertEquals(0,jdbc.queryForObject("select count(*) from owner_payouts where booking_id=?",Integer.class,booking));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL REAL POSTGRES

Source / existing evidence: src/test/java/com/eema/backend/integration/PostgresCriticalPathIntegrationTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-031 — JwtServiceTest#validatesSignatureExpiryAndExpectedIssuer

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=JwtServiceTest#validatesSignatureExpiryAndExpectedIssuer`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        AppProperties properties = new AppProperties(
                new AppProperties.JwtProperties(Base64.getEncoder().encodeToString(KEY), 15, 7),
                null, null, null, null, null, null, null, null);
        JwtService service = new JwtService(properties);
        service.init();
        User user = User.builder().id(UUID.randomUUID()).email("scout@example.com").role(UserRole.SCOUT).build();
        assertEquals("eema", service.validateToken(service.generateAccessToken(user)).getIssuer());

        String wrongIssuer = Jwts.builder().setSubject(user.getId().toString()).setIssuer("other")
                .setExpiration(new Date(System.currentTimeMillis() + 60_000))
                .signWith(Keys.hmacShaKeyFor(KEY)).compact();
        assertThrows(IncorrectClaimException.class, () -> service.validateToken(wrongIssuer));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/security/JwtServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-032 — RequestCorrelationFilterTest#propagatesSafeRequestId

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=RequestCorrelationFilterTest#propagatesSafeRequestId`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        MockHttpServletRequest request = new MockHttpServletRequest();
        request.addHeader(RequestCorrelationFilter.HEADER, "edge-request_123");
        MockHttpServletResponse response = new MockHttpServletResponse();
        filter.doFilter(request, response, (req, res) -> assertThat(org.slf4j.MDC.get("requestId")).isEqualTo("edge-request_123"));
        assertThat(response.getHeader(RequestCorrelationFilter.HEADER)).isEqualTo("edge-request_123");
        assertThat(org.slf4j.MDC.get("requestId")).isNull();
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/security/RequestCorrelationFilterTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-033 — RequestCorrelationFilterTest#replacesUnsafeRequestId

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=RequestCorrelationFilterTest#replacesUnsafeRequestId`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        MockHttpServletRequest request = new MockHttpServletRequest();
        request.addHeader(RequestCorrelationFilter.HEADER, "bad header\nvalue");
        MockHttpServletResponse response = new MockHttpServletResponse();
        filter.doFilter(request, response, (req, res) -> {});
        assertThat(response.getHeader(RequestCorrelationFilter.HEADER)).matches("[0-9a-f-]{36}");
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/security/RequestCorrelationFilterTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-034 — AccountEmailNotificationServiceTest#verificationUsesAuthoritativeEmailAndFrontendVerificationRoute

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=AccountEmailNotificationServiceTest#verificationUsesAuthoritativeEmailAndFrontendVerificationRoute`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        UUID id = UUID.randomUUID();
        when(users.findById(id)).thenReturn(Optional.of(User.builder().id(id).email("current@example.com").build()));

        service.sendVerification(id, "raw+/token");

        verify(email).send(eq("current@example.com"), eq("Verify your Eema email"),
                contains("https://app.eema.example/verify-email?token=raw%2B%2Ftoken"),
                contains("/verify-email?token=raw%2B%2Ftoken"));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/AccountEmailNotificationServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-035 — AccountEmailNotificationServiceTest#resetUsesAuthoritativeEmailAndFrontendResetRoute

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=AccountEmailNotificationServiceTest#resetUsesAuthoritativeEmailAndFrontendResetRoute`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        UUID id = UUID.randomUUID();
        when(users.findById(id)).thenReturn(Optional.of(User.builder().id(id).email("current@example.com").build()));

        service.sendPasswordReset(id, "raw-token");

        verify(email).send(eq("current@example.com"), eq("Reset your Eema password"),
                contains("https://app.eema.example/reset-password?token=raw-token"),
                contains("/reset-password?token=raw-token"));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/AccountEmailNotificationServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-036 — AccountRecoveryServiceTest#issueStoresOnlyHashAndPublishesRawTokenAfterTransactionBoundary

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=AccountRecoveryServiceTest#issueStoresOnlyHashAndPublishesRawTokenAfterTransactionBoundary`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        service.issueVerification(user);
        verify(verification).save(argThat(token -> token.getTokenHash().equals("hashed-token")
                && !token.getTokenHash().contains("raw-secret-token")
                && token.getExpiresAt().equals(OffsetDateTime.parse("2026-08-28T10:00:00Z"))));
        verify(events).publishEvent(new EmailVerificationRequestedEvent(user.getId(), "raw-secret-token"));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/AccountRecoveryServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-037 — AccountRecoveryServiceTest#validVerificationIsSingleUseAndSameUsedTokenIsIdempotent

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=AccountRecoveryServiceTest#validVerificationIsSingleUseAndSameUsedTokenIsIdempotent`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        EmailVerificationToken token = EmailVerificationToken.builder().user(user).tokenHash("hashed-token")
                .expiresAt(OffsetDateTime.parse("2026-08-28T10:00:00Z")).build();
        when(verification.findByTokenHashForUpdate("hashed-token")).thenReturn(Optional.of(token));
        assertThat(service.verifyEmail("raw-secret-token").message()).contains("verified");
        assertThat(user.isEmailVerified()).isTrue();
        assertThat(token.getUsedAt()).isNotNull();
        assertThat(service.verifyEmail("raw-secret-token").message()).contains("verified");
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/AccountRecoveryServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-038 — AccountRecoveryServiceTest#expiredAndInactiveVerificationAreRejectedWithoutActivation

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=AccountRecoveryServiceTest#expiredAndInactiveVerificationAreRejectedWithoutActivation`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        EmailVerificationToken expired = EmailVerificationToken.builder().user(user).tokenHash("hashed-token")
                .expiresAt(OffsetDateTime.parse("2026-08-27T09:59:59Z")).build();
        when(verification.findByTokenHashForUpdate("hashed-token")).thenReturn(Optional.of(expired));
        assertThatThrownBy(() -> service.verifyEmail("raw-secret-token")).isInstanceOf(BadRequestException.class);
        user.setActive(false);
        expired.setExpiresAt(OffsetDateTime.parse("2026-08-28T10:00:00Z"));
        assertThatThrownBy(() -> service.verifyEmail("raw-secret-token")).isInstanceOf(BadRequestException.class);
        assertThat(user.isActive()).isFalse();
        assertThat(user.isEmailVerified()).isFalse();
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/AccountRecoveryServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-039 — AccountRecoveryServiceTest#resendIsGenericAndDoesNothingForVerifiedOrUnknownAccount

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=AccountRecoveryServiceTest#resendIsGenericAndDoesNothingForVerifiedOrUnknownAccount`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        when(users.findByEmailIgnoreCase("user@example.com")).thenReturn(Optional.of(user));
        assertThat(service.resendVerification(" USER@example.com ").message()).isEqualTo(AccountRecoveryService.RESEND_MESSAGE);
        verify(verification).invalidateOutstanding(eq(user.getId()), any());
        clearInvocations(verification, events);
        user.setEmailVerified(true);
        service.resendVerification("user@example.com");
        when(users.findByEmailIgnoreCase("missing@example.com")).thenReturn(Optional.empty());
        assertThat(service.resendVerification("missing@example.com").message()).isEqualTo(AccountRecoveryService.RESEND_MESSAGE);
        verifyNoInteractions(verification);
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/AccountRecoveryServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-040 — AccountRecoveryServiceTest#forgotIsGenericAndResetChangesPasswordAndRevokesEveryRefreshSession

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=AccountRecoveryServiceTest#forgotIsGenericAndResetChangesPasswordAndRevokesEveryRefreshSession`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        when(users.findByEmailIgnoreCase("user@example.com")).thenReturn(Optional.of(user));
        assertThat(service.forgotPassword("user@example.com").message()).isEqualTo(AccountRecoveryService.FORGOT_MESSAGE);
        verify(resets).save(argThat(token -> token.getTokenHash().equals("hashed-token")
                && token.getExpiresAt().equals(OffsetDateTime.parse("2026-08-27T11:00:00Z"))));
        verify(events).publishEvent(new PasswordResetRequestedEvent(user.getId(), "raw-secret-token"));

        PasswordResetToken reset = PasswordResetToken.builder().user(user).tokenHash("hashed-token")
                .expiresAt(OffsetDateTime.parse("2026-08-27T11:00:00Z")).build();
        when(resets.findByTokenHashForUpdate("hashed-token")).thenReturn(Optional.of(reset));
```

```
        when(passwords.encode("new-password")).thenReturn("new-hash");
        service.resetPassword("raw-secret-token", "new-password");
        assertThat(user.getPasswordHash()).isEqualTo("new-hash");
        assertThat(reset.getUsedAt()).isNotNull();
        verify(refresh).deleteAllByUserId(user.getId());
        assertThatThrownBy(() -> service.resetPassword("raw-secret-token", "new-password"))
                .isInstanceOf(BadRequestException.class);
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/AccountRecoveryServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-041 — AccountRecoveryServiceTest#unknownForgotResponseMatchesExistingAndExpiredResetIsRejected

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=AccountRecoveryServiceTest#unknownForgotResponseMatchesExistingAndExpiredResetIsRejected`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        when(users.findByEmailIgnoreCase("missing@example.com")).thenReturn(Optional.empty());
        assertThat(service.forgotPassword("missing@example.com").message()).isEqualTo(AccountRecoveryService.FORGOT_MESSAGE);
        PasswordResetToken expired = PasswordResetToken.builder().user(user).tokenHash("hashed-token")
                .expiresAt(OffsetDateTime.parse("2026-08-27T09:00:00Z")).build();
        when(resets.findByTokenHashForUpdate("hashed-token")).thenReturn(Optional.of(expired));
        assertThatThrownBy(() -> service.resetPassword("raw-secret-token", "new-password"))
                .isInstanceOf(BadRequestException.class);
        verifyNoInteractions(refresh);
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/AccountRecoveryServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-042 — AdminOperationsServiceTest#eventTimelineIsChronologicalPagedAndRedactsUnknownSensitiveKeys

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=AdminOperationsServiceTest#eventTimelineIsChronologicalPagedAndRedactsUnknownSensitiveKeys`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        UUID bookingId = UUID.randomUUID();
        when(bookings.findById(bookingId)).thenReturn(Optional.of(Booking.builder().id(bookingId).build()));
        BookingEvent event = BookingEvent.builder().id(UUID.randomUUID()).bookingId(bookingId)
                .eventType(BookingEventType.PAYMENT_SUCCEEDED).actorType(ActorType.SYSTEM).actorName("SYSTEM")
                .payload(new BookingEventPayload(new LinkedHashMap<>(Map.of(
                        "amount", "100.00", "apiKey", "secret", "rawMollieResponse", "hidden")))).build();
        when(events.findPageByBookingIdOrderByCreatedAtAsc(eq(bookingId), any()))
                .thenReturn(new PageImpl<>(List.of(event)));
        var page = service.events(bookingId, PageRequest.of(0, 250));
        assertThat(page.getContent().getFirst().payload()).containsEntry("amount", "100.00")
                .doesNotContainKeys("apiKey", "rawMollieResponse");
        verify(events).findPageByBookingIdOrderByCreatedAtAsc(eq(bookingId),
                argThat(p -> p.getPageSize() == 100));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/AdminOperationsServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-043 — AdminOperationsServiceTest#exceptionQueueAggregatesAllRequiredOperationalCategories

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=AdminOperationsServiceTest#exceptionQueueAggregatesAllRequiredOperationalCategories`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        UUID bookingId = UUID.randomUUID();
        OffsetDateTime old = OffsetDateTime.now().minusDays(10);
        OwnerPayout held = OwnerPayout.builder().id(UUID.randomUUID()).bookingId(bookingId)
                .status(OwnerPayoutStatus.HELD).holdReason(OwnerPayoutHoldReason.WAITING_FOR_PROVIDER_COST).updatedAt(old).build();
        OwnerPayout overdue = OwnerPayout.builder().id(UUID.randomUUID()).bookingId(bookingId)
                .status(OwnerPayoutStatus.READY).readyAt(old).build();
        OwnerFinancialLiability liability = OwnerFinancialLiability.builder().id(UUID.randomUUID()).bookingId(bookingId)
                .status(OwnerFinancialLiabilityStatus.OPEN).createdAt(old).build();
        Booking booking = Booking.builder().id(bookingId).refundRequestedAt(old).paymentCreationStartedAt(old)
                .paidAt(old).latePaymentDetectedAt(old).build();
        when(payouts.findByStatusOrderByCreatedAtAsc(eq(OwnerPayoutStatus.HELD), any())).thenReturn(List.of(held));
        when(payouts.findByStatusAndReadyAtBeforeOrderByReadyAtAsc(eq(OwnerPayoutStatus.READY), any(), any())).thenReturn(List.of(overdue));
        when(liabilities.findByStatusOrderByCreatedAtAsc(eq(OwnerFinancialLiabilityStatus.OPEN), any())).thenReturn(List.of(liability));
        when(bookings.findStuckRefunds(anyList(), any(), any())).thenReturn(List.of(booking));
        when(bookings.findStuckPaymentCreations(eq(PaymentCreationStatus.CREATING), any(), any())).thenReturn(List.of(booking));
        when(bookings.findOldUnreconciledMollieFees(any(), any())).thenReturn(List.of(booking));
        when(bookings.findLatePaymentRefundsInProgress(anyList(), any())).thenReturn(List.of(booking));

        Set<OperationalExceptionType> types = service.exceptions(PageRequest.of(0, 100)).stream()
                .map(OperationalExceptionDto::type).collect(java.util.stream.Collectors.toSet());
        assertThat(types).containsExactlyInAnyOrder(OperationalExceptionType.values());
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/AdminOperationsServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-044 — AdminOperationsServiceTest#bookingSearchForwardsControlledFiltersAndCapsPageSize

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=AdminOperationsServiceTest#bookingSearchForwardsControlledFiltersAndCapsPageSize`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        UUID id = UUID.randomUUID();
        when(bookings.adminSearch(any(), any(), any(), any(), any(), any(), any(), any()))
                .thenReturn(Page.empty());
        service.searchBookings(id, null, null, BookingStatus.CONFIRMED, " tr_1 ", RefundStatus.NONE,
                PaymentCreationStatus.CREATED, PageRequest.of(0, 500));
        verify(bookings).adminSearch(eq(id), isNull(), isNull(), eq(BookingStatus.CONFIRMED), eq("tr_1"),
                eq(RefundStatus.NONE), eq(PaymentCreationStatus.CREATED), argThat(p -> p.getPageSize() == 100));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/AdminOperationsServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-045 — AdminOperationsServiceTest#bookingDetailCombinesAuthoritativeRefundFeePayoutAndLiabilityState

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=AdminOperationsServiceTest#bookingDetailCombinesAuthoritativeRefundFeePayoutAndLiabilityState`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        UUID id = UUID.randomUUID();
        Booking booking = Booking.builder().id(id).venueId(UUID.randomUUID()).scoutId(UUID.randomUUID())
                .status(BookingStatus.CONFIRMED).commissionPercentage(new BigDecimal("20.00"))
                .commissionEur(new BigDecimal("200.00")).mollieFeeEur(new BigDecimal("15.00"))
                .mollieFeeReconciled(true).refundStatus(RefundStatus.PROCESSING)
                .refundAmountEur(new BigDecimal("500.00")).build();
        OwnerPayout payout = OwnerPayout.builder().id(UUID.randomUUID()).bookingId(id).build();
        OwnerFinancialLiability liability = OwnerFinancialLiability.builder().id(UUID.randomUUID()).bookingId(id).build();
        when(bookings.findById(id)).thenReturn(Optional.of(booking));
        when(payouts.findByBookingId(id)).thenReturn(Optional.of(payout));
        when(liabilities.findByBookingIdOrderByCreatedAtAsc(id)).thenReturn(List.of(liability));
        var payoutDto = new com.eema.backend.dto.response.OwnerPayoutDto(payout.getId(), id, null, null, "EUR",
```

```
                null, null, null, null, null, null, null, null, null, null, null, null, null, null);
        var liabilityDto = new com.eema.backend.dto.response.OwnerFinancialLiabilityDto(liability.getId(), null,
                id, null, null, null, null, null, null);
        when(payoutService.dto(payout)).thenReturn(payoutDto);
        when(liabilityService.dto(liability)).thenReturn(liabilityDto);
        var detail = service.bookingDetail(id);
        assertThat(detail.mollieFeeEur()).isEqualByComparingTo("15.00");
        assertThat(detail.refundStatus()).isEqualTo(RefundStatus.PROCESSING);
        assertThat(detail.refundAmountEur()).isEqualByComparingTo("500.00");
        assertThat(detail.payout()).isSameAs(payoutDto);
        assertThat(detail.liabilities()).containsExactly(liabilityDto);
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/AdminOperationsServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-046 — AuthServiceTest#loginRejectsInactiveUser

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=AuthServiceTest#loginRejectsInactiveUser`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        User user = User.builder().email("x@example.com").passwordHash("hash").role(UserRole.SCOUT).active(false).build();
        when(users.findByEmailIgnoreCase("x@example.com")).thenReturn(Optional.of(user));
        when(passwords.matches("password1", "hash")).thenReturn(true);
        assertThrows(ForbiddenException.class, () -> service.login(new LoginRequest("x@example.com", "password1")));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/AuthServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-047 — AuthServiceTest#registerAndLoginIssueTokensWithoutPersistingRawRefreshToken

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=AuthServiceTest#registerAndLoginIssueTokensWithoutPersistingRawRefreshToken`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        when(passwords.encode("password1")).thenReturn("hash");
        when(jwt.generateAccessToken(any())).thenReturn("access");
        when(jwt.generateRefreshToken()).thenReturn("raw-refresh");
        when(jwt.hashRefreshToken("raw-refresh")).thenReturn("hashed-refresh");
        assertEquals("raw-refresh", service.register(new RegisterRequest(
                "New@Example.com", "password1", "New Scout", UserRole.SCOUT)).refreshToken());
        verify(recovery).issueVerification(argThat(user -> !user.isEmailVerified()));
        verify(tokens).save(argThat(token -> "hashed-refresh".equals(token.getTokenHash())));

        User active = User.builder().email("new@example.com").passwordHash("hash").role(UserRole.SCOUT).active(true).build();
        when(users.findByEmailIgnoreCase("new@example.com")).thenReturn(Optional.of(active));
        when(passwords.matches("password1", "hash")).thenReturn(true);
        assertEquals("access", service.login(new LoginRequest("new@example.com", "password1")).accessToken());
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/AuthServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-048 — AuthServiceTest#refreshRotatesAndRejectsReuse

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=AuthServiceTest#refreshRotatesAndRejectsReuse`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        User user = User.builder().id(UUID.randomUUID()).email("x@example.com").role(UserRole.SCOUT).active(true).build();
        RefreshToken token = RefreshToken.builder().user(user).tokenHash("hash")
                .expiresAt(OffsetDateTime.now().plusDays(1)).build();
        when(jwt.hashRefreshToken("raw-token")).thenReturn("hash");
        when(tokens.findByTokenHashForUpdate("hash")).thenReturn(Optional.of(token));
        when(jwt.generateAccessToken(user)).thenReturn("access");
        when(jwt.generateRefreshToken()).thenReturn("new-refresh");
        when(jwt.hashRefreshToken("new-refresh")).thenReturn("new-hash");

        assertEquals("new-refresh", service.refresh(new RefreshRequest("raw-token")).refreshToken());
        assertTrue(token.isRevoked());
        assertThrows(UnauthorizedException.class, () -> service.refresh(new RefreshRequest("raw-token")));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/AuthServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-049 — AvailabilityInquiryNotificationServiceTest#ownerReceivesNewInquiryWithExistingOwnerRoute

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=AvailabilityInquiryNotificationServiceTest#ownerReceivesNewInquiryWithExistingOwnerRoute`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        service.sendLifecycle(inquiryId, AvailabilityInquiryLifecycleEvent.Type.CREATED);

        verify(email).send(eq("owner@example.test"), eq("New availability inquiry"),
                contains("Could you open early?"), contains("https://app.eema.test/owner/inquiries"));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/AvailabilityInquiryNotificationServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-050 — AvailabilityInquiryNotificationServiceTest#scoutReceivesOfferWithDeadlineAndExistingScoutRoute

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=AvailabilityInquiryNotificationServiceTest#scoutReceivesOfferWithDeadlineAndExistingScoutRoute`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        inquiry.setStatus(AvailabilityInquiryStatus.OFFERED);
        inquiry.setOfferedDate(LocalDate.of(2026, 8, 26));
        inquiry.setOfferedStartTime(LocalTime.of(10, 0));
        inquiry.setOfferedEndTime(LocalTime.of(12, 0));
        inquiry.setOfferExpiresAt(OffsetDateTime.parse("2026-08-25T18:00:00+02:00"));

        service.sendLifecycle(inquiryId, AvailabilityInquiryLifecycleEvent.Type.OFFERED);

        verify(email).send(eq("scout@example.test"), eq("The owner offered you a time"),
                contains("25 August 2026 at 18:00 CEST"), contains("https://app.eema.test/scout/inquiries"));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/AvailabilityInquiryNotificationServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-051 — AvailabilityInquiryNotificationServiceTest#scoutReceivesDeclineAndOfferExpiry

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=AvailabilityInquiryNotificationServiceTest#scoutReceivesDeclineAndOfferExpiry`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        inquiry.setStatus(AvailabilityInquiryStatus.DECLINED);
        service.sendLifecycle(inquiryId, AvailabilityInquiryLifecycleEvent.Type.DECLINED);
        verify(email).send(eq("scout@example.test"), eq("Your availability inquiry was declined"), anyString(), anyString());

        clearInvocations(email);
        inquiry.setStatus(AvailabilityInquiryStatus.EXPIRED);
        inquiry.setOfferedDate(LocalDate.of(2026, 8, 26));
        inquiry.setOfferedStartTime(LocalTime.of(10, 0));
        inquiry.setOfferedEndTime(LocalTime.of(12, 0));
        service.sendLifecycle(inquiryId, AvailabilityInquiryLifecycleEvent.Type.OFFER_EXPIRED);
        verify(email).send(eq("scout@example.test"), eq("Your availability offer expired"), anyString(), anyString());
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/AvailabilityInquiryNotificationServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-052 — AvailabilityInquiryNotificationServiceTest#providerFailureDoesNotEscapeNotificationBoundary

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=AvailabilityInquiryNotificationServiceTest#providerFailureDoesNotEscapeNotificationBoundary`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        doThrow(new EmailService.EmailDeliveryException("failed", new RuntimeException()))
                .when(email).send(anyString(), anyString(), anyString(), anyString());
        assertDoesNotThrow(() -> service.sendLifecycle(inquiryId, AvailabilityInquiryLifecycleEvent.Type.CREATED));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/AvailabilityInquiryNotificationServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-053 — AvailabilityInquiryServiceTest#inquiryIsAllowedForUnavailableInterval

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=AvailabilityInquiryServiceTest#inquiryIsAllowedForUnavailableInterval`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        when(currentUserService.getCurrentUser()).thenReturn(scout);
        when(venues.findById(venueId)).thenReturn(Optional.of(venue));
        when(availabilityService.getAvailableIntervals(venueId, futureDate)).thenReturn(List.of());
        when(inquiries.save(any())).thenAnswer(invocation -> invocation.getArgument(0));

        var result = service.create(new CreateAvailabilityInquiryRequest(
                venueId, futureDate, time(7), time(9), "Could the studio open early?"));

        assertEquals(AvailabilityInquiryStatus.OPEN, result.status());
        verify(inquiries).save(any(AvailabilityInquiry.class));
        verify(eventPublisher).publishEvent(new AvailabilityInquiryLifecycleEvent(
                result.id(), AvailabilityInquiryLifecycleEvent.Type.CREATED));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/AvailabilityInquiryServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-054 — AvailabilityInquiryServiceTest#inquiryIsRejectedWhenNormalAvailabilityAlreadyContainsInterval

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=AvailabilityInquiryServiceTest#inquiryIsRejectedWhenNormalAvailabilityAlreadyContainsInterval`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        when(currentUserService.getCurrentUser()).thenReturn(scout);
        when(venues.findById(venueId)).thenReturn(Optional.of(venue));
        when(availabilityService.getAvailableIntervals(venueId, futureDate))
                .thenReturn(List.of(new com.eema.backend.dto.response.AvailabilityIntervalDto(time(8), time(18))));

        assertThrows(BadRequestException.class, () -> service.create(new CreateAvailabilityInquiryRequest(
                venueId, futureDate, time(9), time(11), "Already available")));
```

```
        verifyNoInteractions(inquiries);
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/AvailabilityInquiryServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-055 — AvailabilityInquiryServiceTest#unauthorizedOwnerCannotRespond

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=AvailabilityInquiryServiceTest#unauthorizedOwnerCannotRespond`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        AvailabilityInquiry inquiry = openInquiry();
        User otherOwner = User.builder().id(UUID.randomUUID()).role(UserRole.OWNER).build();
        when(currentUserService.getCurrentUser()).thenReturn(otherOwner);
        when(inquiries.findByIdForUpdate(inquiry.getId())).thenReturn(Optional.of(inquiry));
```

```
        when(venues.findById(venueId)).thenReturn(Optional.of(venue));

        assertThrows(ForbiddenException.class, () -> service.decline(inquiry.getId()));
        assertEquals(AvailabilityInquiryStatus.OPEN, inquiry.getStatus());
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/AvailabilityInquiryServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-056 — AvailabilityInquiryServiceTest#ownerDeclineEmitsOneLifecycleEventAndDuplicateIsRejected

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=AvailabilityInquiryServiceTest#ownerDeclineEmitsOneLifecycleEventAndDuplicateIsRejected`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        AvailabilityInquiry inquiry = openInquiry();
```

```
        when(currentUserService.getCurrentUser()).thenReturn(owner);
        when(inquiries.findByIdForUpdate(inquiry.getId())).thenReturn(Optional.of(inquiry));
        when(venues.findById(venueId)).thenReturn(Optional.of(venue));

        service.decline(inquiry.getId());
        assertThrows(ConflictException.class, () -> service.decline(inquiry.getId()));

        verify(eventPublisher, times(1)).publishEvent(new AvailabilityInquiryLifecycleEvent(
                inquiry.getId(), AvailabilityInquiryLifecycleEvent.Type.DECLINED));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/AvailabilityInquiryServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-057 — AvailabilityInquiryServiceTest#ownerCannotOfferIntervalConflictingWithBooking

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=AvailabilityInquiryServiceTest#ownerCannotOfferIntervalConflictingWithBooking`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        AvailabilityInquiry inquiry = openInquiry();
        Booking conflict = Booking.builder().startTime(time(10)).endTime(time(12)).build();
        when(currentUserService.getCurrentUser()).thenReturn(owner);
        when(inquiries.findByIdForUpdate(inquiry.getId())).thenReturn(Optional.of(inquiry));
        when(venues.findById(venueId)).thenReturn(Optional.of(venue));
        when(bookings.findBlockingBookings(venueId, futureDate)).thenReturn(List.of(conflict));

        assertThrows(ConflictException.class, () -> service.offer(inquiry.getId(),
                new OfferAvailabilityInquiryRequest(futureDate, time(11), time(13))));
        assertEquals(AvailabilityInquiryStatus.OPEN, inquiry.getStatus());
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/AvailabilityInquiryServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-058 — AvailabilityInquiryServiceTest#inquiryAndOwnerOfferCannotBypassLeadTime

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=AvailabilityInquiryServiceTest#inquiryAndOwnerOfferCannotBypassLeadTime`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        when(currentUserService.getCurrentUser()).thenReturn(scout);
        when(venues.findById(venueId)).thenReturn(Optional.of(venue));
        doThrow(new VenueBookingLeadTimePolicy.MinimumLeadTimeNotMetException("too late"))
                .when(leadTimePolicy).validate(venue, futureDate, time(7));
        assertThrows(VenueBookingLeadTimePolicy.MinimumLeadTimeNotMetException.class,
                () -> service.create(new CreateAvailabilityInquiryRequest(venueId, futureDate, time(7), time(9), "Request")));

        AvailabilityInquiry inquiry = openInquiry();
        when(currentUserService.getCurrentUser()).thenReturn(owner);
        when(inquiries.findByIdForUpdate(inquiry.getId())).thenReturn(Optional.of(inquiry));
        assertThrows(VenueBookingLeadTimePolicy.MinimumLeadTimeNotMetException.class,
                () -> service.offer(inquiry.getId(), new OfferAvailabilityInquiryRequest(futureDate, time(7), time(9))));
        assertEquals(AvailabilityInquiryStatus.OPEN, inquiry.getStatus());
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/AvailabilityInquiryServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-059 — AvailabilityInquiryServiceTest#offeredSlotThatBecomesTooLateExpiresWithoutCreatingBooking

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=AvailabilityInquiryServiceTest#offeredSlotThatBecomesTooLateExpiresWithoutCreatingBooking`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        AvailabilityInquiry inquiry = offeredInquiry();
        when(currentUserService.getCurrentUser()).thenReturn(scout);
        when(inquiries.findByIdForUpdate(inquiry.getId())).thenReturn(Optional.of(inquiry));
        when(venues.findById(venueId)).thenReturn(Optional.of(venue));
        when(leadTimePolicy.isSatisfied(venue, inquiry.getOfferedDate(), inquiry.getOfferedStartTime())).thenReturn(false);
        when(leadTimePolicy.notMet(venue)).thenReturn(
                new VenueBookingLeadTimePolicy.MinimumLeadTimeNotMetException("too late"));

        assertThrows(VenueBookingLeadTimePolicy.MinimumLeadTimeNotMetException.class,
                () -> service.accept(inquiry.getId()));
        assertEquals(AvailabilityInquiryStatus.EXPIRED, inquiry.getStatus());
        verifyNoInteractions(bookingService);
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/AvailabilityInquiryServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-060 — AvailabilityInquiryServiceTest#offerExpiresAndCannotBeAccepted

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=AvailabilityInquiryServiceTest#offerExpiresAndCannotBeAccepted`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        AvailabilityInquiry inquiry = offeredInquiry();
        inquiry.setOfferExpiresAt(OffsetDateTime.now().minusMinutes(1));
        when(currentUserService.getCurrentUser()).thenReturn(scout);
        when(inquiries.findByIdForUpdate(inquiry.getId())).thenReturn(Optional.of(inquiry));

        assertThrows(ConflictException.class, () -> service.accept(inquiry.getId()));
        assertEquals(AvailabilityInquiryStatus.OFFERED, inquiry.getStatus());
        verifyNoInteractions(bookingService);
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/AvailabilityInquiryServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-061 — AvailabilityInquiryServiceTest#onlyInquiryScoutCanAcceptOffer

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=AvailabilityInquiryServiceTest#onlyInquiryScoutCanAcceptOffer`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        AvailabilityInquiry inquiry = offeredInquiry();
        User anotherScout = User.builder().id(UUID.randomUUID()).role(UserRole.SCOUT).build();
        when(currentUserService.getCurrentUser()).thenReturn(anotherScout);
        when(inquiries.findByIdForUpdate(inquiry.getId())).thenReturn(Optional.of(inquiry));

        assertThrows(ForbiddenException.class, () -> service.accept(inquiry.getId()));
        verifyNoInteractions(bookingService);
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/AvailabilityInquiryServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-062 — AvailabilityInquiryServiceTest#ownOfferCreatesApprovedBookingAndDoesNotChangeWeeklyAvailability

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=AvailabilityInquiryServiceTest#ownOfferCreatesApprovedBookingAndDoesNotChangeWeeklyAvailability`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        AvailabilityInquiry inquiry = offeredInquiry();
        UUID bookingId = UUID.randomUUID();
        BookingDto result = bookingDto(bookingId);
        when(currentUserService.getCurrentUser()).thenReturn(scout);
        when(inquiries.findByIdForUpdate(inquiry.getId())).thenReturn(Optional.of(inquiry));
        when(venues.findById(venueId)).thenReturn(Optional.of(venue));
        when(bookings.findBlockingBookings(venueId, futureDate)).thenReturn(List.of());
        when(bookingService.createApprovedExceptionBooking(venueId, scoutId, futureDate, time(7), time(9), inquiry.getMessage()))
                .thenReturn(result);

        assertSame(result, service.accept(inquiry.getId()));
        assertEquals(AvailabilityInquiryStatus.ACCEPTED, inquiry.getStatus());
```

```
        assertEquals(bookingId, inquiry.getAcceptedBookingId());
        verifyNoInteractions(availabilityService);
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/AvailabilityInquiryServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-063 — AvailabilityInquiryServiceTest#offeringOneOffSlotDoesNotModifyWeeklyAvailability

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=AvailabilityInquiryServiceTest#offeringOneOffSlotDoesNotModifyWeeklyAvailability`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        AvailabilityInquiry inquiry = openInquiry();
        when(currentUserService.getCurrentUser()).thenReturn(owner);
        when(inquiries.findByIdForUpdate(inquiry.getId())).thenReturn(Optional.of(inquiry));
```

```
        when(venues.findById(venueId)).thenReturn(Optional.of(venue));
        when(bookings.findBlockingBookings(venueId, futureDate)).thenReturn(List.of());

        var result = service.offer(inquiry.getId(), new OfferAvailabilityInquiryRequest(futureDate, time(7), time(9)));

        assertEquals(AvailabilityInquiryStatus.OFFERED, result.status());
        assertEquals(time(7), result.offeredStartTime());
        assertThrows(ConflictException.class, () -> service.offer(inquiry.getId(),
                new OfferAvailabilityInquiryRequest(futureDate, time(7), time(9))));
        verify(eventPublisher).publishEvent(new AvailabilityInquiryLifecycleEvent(
                inquiry.getId(), AvailabilityInquiryLifecycleEvent.Type.OFFERED));
        verifyNoInteractions(availabilityService);
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/AvailabilityInquiryServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-064 — AvailabilityInquiryServiceTest#scheduledExpiryReleasesOutstandingOffers

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=AvailabilityInquiryServiceTest#scheduledExpiryReleasesOutstandingOffers`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        AvailabilityInquiry inquiry = offeredInquiry();
        OffsetDateTime now = OffsetDateTime.now();
        when(inquiries.lockDueOffers(now, 100)).thenReturn(List.of(inquiry));

        assertEquals(1, service.expireOffers(now));
        assertEquals(AvailabilityInquiryStatus.EXPIRED, inquiry.getStatus());
        assertEquals(now, inquiry.getExpiredAt());
        verify(eventPublisher).publishEvent(new AvailabilityInquiryLifecycleEvent(
                inquiry.getId(), AvailabilityInquiryLifecycleEvent.Type.OFFER_EXPIRED));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/AvailabilityInquiryServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-065 — BookingApprovalExpirationJobTest#scheduledScanExpiresApprovedUnpaidBookings

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingApprovalExpirationJobTest#scheduledScanExpiresApprovedUnpaidBookings`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        BookingRepository repository = mock(BookingRepository.class);
        BookingService service = mock(BookingService.class);
        UUID id = UUID.randomUUID();
        when(repository.lockDueAwaitingPaymentBookings(any(), eq(100)))
                .thenReturn(List.of(Booking.builder().id(id).build()));
        new BookingApprovalExpirationJob(repository, service).expireUnpaidApprovals();
        verify(service).expireDueRequests(any(), eq(100));
        verify(service).expireApproval(eq(id), any());
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingApprovalExpirationJobTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-066 — BookingAvailabilityValidatorTest#acceptsSlotContainedInEffectiveAvailability

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingAvailabilityValidatorTest#acceptsSlotContainedInEffectiveAvailability`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        LocalDate date = LocalDate.now(java.time.ZoneId.of("Europe/Amsterdam")).plusDays(1);
        when(availability.getAvailableIntervalsExcludingBooking(venueId, date, null)).thenReturn(List.of(interval(9, 17)));
        assertDoesNotThrow(() -> validator.validate(venue, date, time(10), time(12)));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingAvailabilityValidatorTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-067 — BookingAvailabilityValidatorTest#rejectsSlotOutsideEffectiveAvailability

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingAvailabilityValidatorTest#rejectsSlotOutsideEffectiveAvailability`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        LocalDate date = LocalDate.now(java.time.ZoneId.of("Europe/Amsterdam")).plusDays(1);
        when(availability.getAvailableIntervalsExcludingBooking(venueId, date, null)).thenReturn(List.of(interval(9, 17)));
        assertThrows(ConflictException.class, () -> validator.validate(venue, date, time(8), time(12)));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingAvailabilityValidatorTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-068 — BookingAvailabilityValidatorTest#rejectsInvalidVenueTimezone

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingAvailabilityValidatorTest#rejectsInvalidVenueTimezone`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test

and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        venue.setTimezone("Amsterdam/+1");
        assertThrows(BadRequestException.class, () -> validator.validate(venue, LocalDate.now(), time(9), time(10)));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingAvailabilityValidatorTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-069 — BookingCancellationCoordinatorTest#scoutWithdrawsRequestedAndAwaitingPaymentWithoutRefund

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingCancellationCoordinatorTest#scoutWithdrawsRequestedAndAwaitingPaymentWithoutRefund`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        User scout = user(scoutId, UserRole.SCOUT);
        assertThat(coordinator.cancel(bookingId, scout, null, now).refundClaim()).isNull();
        assertThat(booking.getStatus()).isEqualTo(BookingStatus.CANCELLED);
        assertThat(booking.getCancellationPolicyTier())
                .isEqualTo(CancellationPolicyTier.PRE_PAYMENT_WITHDRAWAL);
        verifyNoInteractions(refunds);

        booking.setStatus(BookingStatus.AWAITING_PAYMENT);
        assertThat(coordinator.cancel(bookingId, scout, null, now).refundClaim()).isNull();
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingCancellationCoordinatorTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-070 — BookingCancellationCoordinatorTest#ownerCanCancelOwnConfirmedBookingButNotAnotherVenue

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingCancellationCoordinatorTest#ownerCanCancelOwnConfirmedBookingButNotAnotherVenue`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML

for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        prepareConfirmed();
        RefundCoordinator.Claim claim = new RefundCoordinator.Claim(
                bookingId, "tr_1", new BigDecimal("1000.00"), UUID.randomUUID());
        when(refunds.claimCancellation(eq(bookingId), eq(new BigDecimal("1000.00")),
                eq(ownerId), anyString(), eq(ActorType.OWNER))).thenReturn(claim);
        assertThat(coordinator.cancel(bookingId, user(ownerId, UserRole.OWNER), "Venue unavailable", now)
                .refundClaim()).isSameAs(claim);
        assertThat(booking.getOwnerPayoutEur()).isZero();
        assertThat(booking.getProviderCostResponsibility()).isEqualTo(ProviderCostResponsibility.OWNER);

        booking.setStatus(BookingStatus.CONFIRMED);
        assertThrows(ForbiddenException.class, () -> coordinator.cancel(
                bookingId, user(UUID.randomUUID(), UserRole.OWNER), "Reason", now));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingCancellationCoordinatorTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-071 — BookingCancellationCoordinatorTest#adminRequiresReasonAndReceivesFullRefundPolicy

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingCancellationCoordinatorTest#adminRequiresReasonAndReceivesFullRefundPolicy`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        prepareConfirmed();
        User admin = user(UUID.randomUUID(), UserRole.ADMIN);
        assertThrows(ConflictException.class, () -> coordinator.cancel(bookingId, admin, " ", now));
        when(refunds.claimCancellation(eq(bookingId), any(), eq(admin.getId()), anyString(), eq(ActorType.ADMIN)))
                .thenReturn(new RefundCoordinator.Claim(bookingId, "tr_1", new BigDecimal("1000.00"), UUID.randomUUID()));
        coordinator.cancel(bookingId, admin, "Safety issue", now);
        assertThat(booking.getCancellationPolicyTier()).isEqualTo(CancellationPolicyTier.ADMIN_FULL_REFUND);
        assertThat(booking.getProviderCostResponsibility()).isEqualTo(ProviderCostResponsibility.EEMA);
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingCancellationCoordinatorTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-072 — BookingCancellationPolicyTest#gracePeriodRequiresConfirmationWithin24HoursAndShootMoreThan48HoursAway

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingCancellationPolicyTest#gracePeriodRequiresConfirmationWithin24HoursAndShootMoreThan48HoursAway`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        OffsetDateTime now = OffsetDateTime.parse("2026-06-01T10:00:00Z");
        Booking insideGrace = confirmed(now.minusHours(1), now.plusDays(5));
        Booking outsideGrace = confirmed(now.minusHours(25), now.plusDays(5));

        assertDecision(insideGrace, now, CancellationPolicyTier.GRACE_PERIOD, "100.00");
        assertDecision(outsideGrace, now, CancellationPolicyTier.PARTIAL_REFUND, "50.00");
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingCancellationPolicyTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-073 — BookingCancellationPolicyTest#exactSevenDaysAndExact48HoursUsePartialTier

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingCancellationPolicyTest#exactSevenDaysAndExact48HoursUsePartialTier`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        OffsetDateTime now = OffsetDateTime.parse("2026-06-01T10:00:00Z");
        assertDecision(confirmed(now.minusDays(30), now.plusDays(8)), now,
                CancellationPolicyTier.FLEXIBLE_FULL_REFUND, "100.00");
        assertDecision(confirmed(now.minusDays(30), now.plusDays(7)), now,
                CancellationPolicyTier.PARTIAL_REFUND, "50.00");
        assertDecision(confirmed(now.minusDays(30), now.plusHours(48)), now,
                CancellationPolicyTier.PARTIAL_REFUND, "50.00");
        assertDecision(confirmed(now.minusDays(30), now.plusHours(48).minusSeconds(1)), now,
                CancellationPolicyTier.NON_REFUNDABLE, "0.00");
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingCancellationPolicyTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-074 — BookingCancellationPolicyTest#cancellationEconomicsUseRetainedAmountAndCommissionSnapshot

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingCancellationPolicyTest#cancellationEconomicsUseRetainedAmountAndCommissionSnapshot`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        OffsetDateTime now = OffsetDateTime.parse("2026-06-01T10:00:00Z");
        var full = policy.evaluate(confirmed(now.minusDays(30), now.plusDays(8)), ActorType.SCOUT, now);
        assertThat(full.refundAmountEur()).isEqualByComparingTo("1000.00");
        assertThat(full.retainedAmountEur()).isEqualByComparingTo("0.00");
        assertThat(full.eemaCommissionAfterCancellationEur()).isEqualByComparingTo("0.00");
        assertThat(full.ownerGrossSettlementEur()).isEqualByComparingTo("0.00");

        var partial = policy.evaluate(confirmed(now.minusDays(30), now.plusDays(5)), ActorType.SCOUT, now);
        assertThat(partial.refundAmountEur()).isEqualByComparingTo("500.00");
        assertThat(partial.retainedAmountEur()).isEqualByComparingTo("500.00");
        assertThat(partial.eemaCommissionAfterCancellationEur()).isEqualByComparingTo("100.00");
        assertThat(partial.ownerGrossSettlementEur()).isEqualByComparingTo("400.00");

        var none = policy.evaluate(confirmed(now.minusDays(30), now.plusHours(24)), ActorType.SCOUT, now);
        assertThat(none.refundAmountEur()).isEqualByComparingTo("0.00");
        assertThat(none.eemaCommissionAfterCancellationEur()).isEqualByComparingTo("200.00");
        assertThat(none.ownerGrossSettlementEur()).isEqualByComparingTo("800.00");
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingCancellationPolicyTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-075 — BookingCancellationPolicyTest#ownerAndAdminAlwaysProduceFullCustomerRefundAndZeroSettlement

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingCancellationPolicyTest#ownerAndAdminAlwaysProduceFullCustomerRefundAndZeroSettlement`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        OffsetDateTime now = OffsetDateTime.parse("2026-06-01T10:00:00Z");
        Booking booking = confirmed(now.minusDays(30), now.plusHours(1));

        var owner = policy.evaluate(booking, ActorType.OWNER, now);
        assertThat(owner.policyTier()).isEqualTo(CancellationPolicyTier.OWNER_FULL_REFUND);
        assertThat(owner.refundAmountEur()).isEqualByComparingTo("1000.00");
        assertThat(owner.ownerGrossSettlementEur()).isZero();
```

```
        assertThat(owner.providerCostResponsibility()).isEqualTo(ProviderCostResponsibility.OWNER);

        var admin = policy.evaluate(booking, ActorType.ADMIN, now);
        assertThat(admin.policyTier()).isEqualTo(CancellationPolicyTier.ADMIN_FULL_REFUND);
        assertThat(admin.refundAmountEur()).isEqualByComparingTo("1000.00");
        assertThat(admin.ownerGrossSettlementEur()).isZero();
        assertThat(admin.providerCostResponsibility()).isEqualTo(ProviderCostResponsibility.EEMA);
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingCancellationPolicyTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-076 — BookingCancellationPolicyTest#snapshotUsesVenueTimezoneAcrossAmsterdamDstTransition

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingCancellationPolicyTest#snapshotUsesVenueTimezoneAcrossAmsterdamDstTransition`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        Booking booking = baseBooking(LocalDate.of(2026, 3, 29), LocalTime.of(10, 0));
        OffsetDateTime confirmed = OffsetDateTime.parse("2026-03-01T12:00:00Z");
        policy.snapshotAtConfirmation(booking, AMSTERDAM, confirmed);
        Instant shootInstant = booking.getShootDate().atTime(booking.getStartTime())
                .atZone(AMSTERDAM).toInstant();

        assertThat(Duration.between(booking.getPartialRefundUntil().toInstant(), shootInstant))
                .isEqualTo(Duration.ofHours(48));
        assertThat(Duration.between(booking.getFullRefundUntil().toInstant(), shootInstant))
                .isEqualTo(Duration.ofDays(7));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingCancellationPolicyTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-077 — BookingCancellationPolicyTest#moneyRoundingIsHalfUpAtTwoDecimals

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingCancellationPolicyTest#moneyRoundingIsHalfUpAtTwoDecimals`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        Booking booking = baseBooking(LocalDate.of(2026, 6, 6), LocalTime.of(10, 0));
        booking.setTotalPaidEur(new BigDecimal("100.01"));
        OffsetDateTime now = OffsetDateTime.parse("2026-06-01T10:00:00Z");
        policy.snapshotAtConfirmation(booking, AMSTERDAM, now.minusDays(30));

        var decision = policy.evaluate(booking, ActorType.SCOUT, now);
        assertThat(decision.refundAmountEur()).isEqualByComparingTo("50.01");
        assertThat(decision.retainedAmountEur()).isEqualByComparingTo("50.00");
        assertThat(decision.eemaCommissionAfterCancellationEur()).isEqualByComparingTo("10.00");
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingCancellationPolicyTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-078 — BookingFinancialsTest#calculatesCommissionAndActualFeePayout

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingFinancialsTest#calculatesCommissionAndActualFeePayout`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        BigDecimal commission = financials.commission(new BigDecimal("100.00"), new BigDecimal("20.00"));
        assertEquals(new BigDecimal("20.00"), commission);
        assertEquals(new BigDecimal("79.51"),
                financials.ownerPayout(new BigDecimal("100.00"), commission, new BigDecimal("0.49")));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingFinancialsTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-079 — BookingFinancialsTest#roundsMoneyExplicitlyAndEnforcesInvariants

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingFinancialsTest#roundsMoneyExplicitlyAndEnforcesInvariants`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        assertEquals(new BigDecimal("12.35"), financials.money(new BigDecimal("12.345")));
        assertThrows(IllegalArgumentException.class,
                () -> financials.commission(BigDecimal.ONE, new BigDecimal("100.01")));
        assertThrows(IllegalArgumentException.class,
                () -> financials.ownerPayout(BigDecimal.TEN, new BigDecimal("9.00"), new BigDecimal("2.00")));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingFinancialsTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-080 — BookingNotificationServiceTest#usesSnapshotRecipientsAndRendersConfirmedDetails

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingNotificationServiceTest#usesSnapshotRecipientsAndRendersConfirmedDetails`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        service.sendPaymentReceived(bookingId);

        ArgumentCaptor<String> recipient = ArgumentCaptor.forClass(String.class);
        ArgumentCaptor<String> subject = ArgumentCaptor.forClass(String.class);
        ArgumentCaptor<String> text = ArgumentCaptor.forClass(String.class);
        ArgumentCaptor<String> html = ArgumentCaptor.forClass(String.class);
        verify(email, times(2)).send(recipient.capture(), subject.capture(), text.capture(), html.capture());

        assertTrue(recipient.getAllValues().contains("snapshot-scout@example.test"));
        assertTrue(recipient.getAllValues().contains("snapshot-owner@example.test"));
        String all = String.join(" ", subject.getAllValues()) + String.join(" ", text.getAllValues()) + String.join(" ", html.getAllValues());
        assertTrue(all.contains("25 August 2026"));
        assertTrue(all.contains("09:00–17:00"));
        assertTrue(all.contains("https://app.eema.test/bookings/" + bookingId));
        assertTrue(all.contains("Status: CONFIRMED") || all.contains(">CONFIRMED<"));
        assertTrue(all.toLowerCase().contains("booking is confirmed"));
        assertFalse(all.toLowerCase().contains("owner payout"));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingNotificationServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-081 — BookingNotificationServiceTest#scoutFailureStillAttemptsOwner

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingNotificationServiceTest#scoutFailureStillAttemptsOwner`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        doThrow(new EmailService.EmailDeliveryException("failed", new RuntimeException()))
                .when(email).send(eq("snapshot-scout@example.test"), eq("Payment received for your Eema booking"), anyString(), anyString());
        service.sendPaymentReceived(bookingId);
        verify(email).send(eq("snapshot-owner@example.test"), eq("Confirmed booking for your Eema venue"), anyString(), anyString());
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingNotificationServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-082 — BookingNotificationServiceTest#latePaymentMessageSaysNotConfirmedAndRefundInitiated

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingNotificationServiceTest#latePaymentMessageSaysNotConfirmedAndRefundInitiated`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        Booking late = bookings.findById(bookingId).orElseThrow();
        late.setStatus(BookingStatus.EXPIRED);
        late.setLatePaymentDetectedAt(java.time.OffsetDateTime.now());
        late.setRefundStatus(RefundStatus.CREATING);

        service.sendLatePaymentRefundInitiated(bookingId);

        ArgumentCaptor<String> text = ArgumentCaptor.forClass(String.class);
        verify(email).send(eq("snapshot-scout@example.test"),
                eq("Your late Eema payment is being refunded"), text.capture(), anyString());
        assertTrue(text.getValue().contains("booking was not confirmed"));
        assertTrue(text.getValue().contains("full refund has been initiated"));
        assertFalse(text.getValue().contains("refund has been completed"));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingNotificationServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-083 — BookingNotificationServiceTest#cancellationEmailDistinguishesInitiatedFromCompletedRefund

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingNotificationServiceTest#cancellationEmailDistinguishesInitiatedFromCompletedRefund`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        Booking booking = bookings.findById(bookingId).orElseThrow();
        booking.setStatus(BookingStatus.CANCELLED);
        booking.setCancelledByActorType(ActorType.OWNER);
        booking.setCancellationPolicyTier(CancellationPolicyTier.OWNER_FULL_REFUND);
        booking.setCancellationRefundAmountEur(new BigDecimal("125.00"));
        booking.setRefundStatus(RefundStatus.QUEUED);

        service.sendCancellation(bookingId);

        verify(email).send(eq("snapshot-scout@example.test"), eq("Your Eema booking was cancelled"),
                contains("€125.00 has been initiated"), anyString());
```

```
        verify(email, never()).send(anyString(), contains("processed"), anyString(), anyString());
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingNotificationServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-084 — BookingNotificationServiceTest#authoritativeCancellationRefundCompletionUsesProcessedCopy

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingNotificationServiceTest#authoritativeCancellationRefundCompletionUsesProcessedCopy`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        Booking booking = bookings.findById(bookingId).orElseThrow();
        booking.setStatus(BookingStatus.CANCELLED);
        booking.setCancellationPolicyTier(CancellationPolicyTier.PARTIAL_REFUND);
        booking.setRefundStatus(RefundStatus.REFUNDED);
```

```
        booking.setRefundAmountEur(new BigDecimal("62.50"));

        service.sendCancellationRefundCompleted(bookingId);

        verify(email).send(eq("snapshot-scout@example.test"),
                eq("Your Eema refund has been processed"), contains("Bank settlement timing may vary"), anyString());
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingNotificationServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-085 — BookingNotificationServiceTest#ownerAndScoutAreNotifiedWhenRequestIsSubmitted

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingNotificationServiceTest#ownerAndScoutAreNotifiedWhenRequestIsSubmitted`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        Booking booking = bookings.findById(bookingId).orElseThrow();
        booking.setStatus(BookingStatus.REQUESTED);
        booking.setRawBrief("Editorial shoot with a small crew");
        booking.setOwnerResponseExpiresAt(OffsetDateTime.parse("2026-08-26T18:00:00+02:00"));

        service.sendLifecycle(bookingId, BookingLifecycleEvent.Type.REQUEST_SUBMITTED);

        verify(email).send(eq("snapshot-owner@example.test"), eq("New booking request"),
                contains("Editorial shoot"), contains("https://app.eema.test/bookings/" + bookingId));
        verify(email).send(eq("snapshot-scout@example.test"), eq("Your request was sent"), anyString(), anyString());
        verify(email).send(eq("snapshot-scout@example.test"), eq("Your request was sent"),
                contains("owner has until 26 August 2026 at 18:00 CEST"), anyString());
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingNotificationServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-086 — BookingNotificationServiceTest#scoutAcceptanceEmailContainsDeadlinePayCtaAndDeepLink

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingNotificationServiceTest#scoutAcceptanceEmailContainsDeadlinePayCtaAndDeepLink`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        Booking booking = bookings.findById(bookingId).orElseThrow();
        booking.setStatus(BookingStatus.AWAITING_PAYMENT);
        booking.setPaymentExpiresAt(OffsetDateTime.parse("2026-08-26T18:00:00+02:00"));

        service.sendLifecycle(bookingId, BookingLifecycleEvent.Type.REQUEST_ACCEPTED);

        verify(email).send(eq("snapshot-scout@example.test"), eq("Your booking request was accepted"),
                contains("26 August 2026 at 18:00 CEST"), contains(">Pay now</a>"));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingNotificationServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-087 — BookingNotificationServiceTest#scoutReceivesDeclineAndApprovalExpiryMessages

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingNotificationServiceTest#scoutReceivesDeclineAndApprovalExpiryMessages`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        Booking booking = bookings.findById(bookingId).orElseThrow();
        booking.setStatus(BookingStatus.DECLINED);
        service.sendLifecycle(bookingId, BookingLifecycleEvent.Type.REQUEST_DECLINED);
        verify(email).send(eq("snapshot-scout@example.test"), eq("Your booking request was declined"),
                contains("Status: DECLINED"), anyString());

        clearInvocations(email);
        booking.setStatus(BookingStatus.EXPIRED);
        service.sendLifecycle(bookingId, BookingLifecycleEvent.Type.APPROVAL_EXPIRED);
        verify(email).send(eq("snapshot-scout@example.test"), eq("Your payment window expired"),
                contains("slot was released"), anyString());
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingNotificationServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-088 — BookingNotificationServiceTest#lifecycleEmailFailureIsContained

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingNotificationServiceTest#lifecycleEmailFailureIsContained`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        Booking booking = bookings.findById(bookingId).orElseThrow();
        booking.setStatus(BookingStatus.DECLINED);
        doThrow(new EmailService.EmailDeliveryException("failed", new RuntimeException()))
                .when(email).send(anyString(), anyString(), anyString(), anyString());

        assertDoesNotThrow(() -> service.sendLifecycle(bookingId, BookingLifecycleEvent.Type.REQUEST_DECLINED));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingNotificationServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-089 — BookingNotificationServiceTest#requestExpiryAndUnavailableUseDistinctNonDeclineWording

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingNotificationServiceTest#requestExpiryAndUnavailableUseDistinctNonDeclineWording`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        Booking booking = bookings.findById(bookingId).orElseThrow();
        booking.setStatus(BookingStatus.EXPIRED);
        service.sendLifecycle(bookingId, BookingLifecycleEvent.Type.REQUEST_EXPIRED);
        verify(email).send(eq("snapshot-scout@example.test"), eq("Your booking request expired"),
                contains("owner did not respond"), anyString());

        clearInvocations(email);
        booking.setStatus(BookingStatus.UNAVAILABLE);
        service.sendLifecycle(bookingId, BookingLifecycleEvent.Type.REQUEST_BECAME_UNAVAILABLE);
        verify(email).send(eq("snapshot-scout@example.test"), eq("The requested time is no longer available"),
                argThat(body -> body.contains("Another booking reserved") && body.contains("did not explicitly decline")), anyString());
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingNotificationServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-090 — BookingOverlapRuleTest#detectsOverlappingStart

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingOverlapRuleTest#detectsOverlappingStart`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{ assertTrue(overlaps(8, 10, 9, 12)); }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingOverlapRuleTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-091 — BookingOverlapRuleTest#detectsOverlappingEnd

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingOverlapRuleTest#detectsOverlappingEnd`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{ assertTrue(overlaps(11, 13, 9, 12)); }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingOverlapRuleTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-092 — BookingOverlapRuleTest#detectsCandidateContainingBooking

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingOverlapRuleTest#detectsCandidateContainingBooking`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{ assertTrue(overlaps(8, 15, 9, 12)); }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingOverlapRuleTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-093 — BookingOverlapRuleTest#detectsCandidateInsideBooking

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingOverlapRuleTest#detectsCandidateInsideBooking`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{ assertTrue(overlaps(10, 11, 9, 12)); }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingOverlapRuleTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-094 — BookingOverlapRuleTest#permitsBackToBackBefore

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingOverlapRuleTest#permitsBackToBackBefore`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{ assertFalse(overlaps(9, 12, 12, 15)); }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingOverlapRuleTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-095 — BookingOverlapRuleTest#permitsBackToBackAfter

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingOverlapRuleTest#permitsBackToBackAfter`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{ assertFalse(overlaps(12, 15, 9, 12)); }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingOverlapRuleTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-096 — BookingOverlapRuleTest#permitsSeparatedBooking

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingOverlapRuleTest#permitsSeparatedBooking`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{ assertFalse(overlaps(15, 16, 9, 12)); }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingOverlapRuleTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-097 — BookingPriceCalculatorTest#appliesMinimumExactMinutesAndFullDayCap

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingPriceCalculatorTest#appliesMinimumExactMinutesAndFullDayCap`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        assertPrice("700.00", "175.00", 240, "1200.00", 9, 11);
        assertPrice("700.00", "175.00", 240, "1200.00", 9, 13);
        assertPrice("875.00", "175.00", 240, "1200.00", 9, 14);
        assertPrice("1200.00", "175.00", 240, "1200.00", 9, 16);
        assertPrice("1225.00", "175.00", 240, null, 9, 16);
        assertPrice("787.50", "175.00", 240, null, LocalTime.of(9, 0), LocalTime.of(13, 30));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingPriceCalculatorTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-098 — BookingPriceCalculatorTest#capAlsoAppliesToMinimumDerivedTotalAndRoundsHalfUpToCents

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingPriceCalculatorTest#capAlsoAppliesToMinimumDerivedTotalAndRoundsHalfUpToCents`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        assertPrice("1200.00", "500.00", 240, "1200.00", 9, 11);
        assertPrice("0.17", "10.00", 1, null, LocalTime.of(9, 0), LocalTime.of(9, 1));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingPriceCalculatorTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-099 — BookingPriceCalculatorTest#exposesAuthoritativeQuoteDetails

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingPriceCalculatorTest#exposesAuthoritativeQuoteDetails`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        var quote = calculator.calculate(new BigDecimal("175"), 240, new BigDecimal("1200"),
                LocalTime.of(9, 0), LocalTime.of(11, 0));
        assertEquals(120, quote.durationMinutes());
        assertEquals(240, quote.billableMinutes());
        assertEquals(new BigDecimal("700.00"), quote.calculatedHourlyTotal());
        assertEquals(new BigDecimal("700.00"), quote.finalPriceEur());
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingPriceCalculatorTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-100 — BookingPriceCalculatorTest#rejectsInvalidConfigurationAndIntervals

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingPriceCalculatorTest#rejectsInvalidConfigurationAndIntervals`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        assertThrows(IllegalArgumentException.class, () -> calculator.calculate(BigDecimal.ZERO, 60, null,
                LocalTime.NOON, LocalTime.of(13, 0)));
        assertThrows(IllegalArgumentException.class, () -> calculator.calculate(BigDecimal.ONE, 0, null,
                LocalTime.NOON, LocalTime.of(13, 0)));
        assertThrows(IllegalArgumentException.class, () -> calculator.calculate(BigDecimal.ONE, 60, BigDecimal.ZERO,
                LocalTime.NOON, LocalTime.of(13, 0)));
        assertThrows(RuntimeException.class, () -> calculator.calculate(BigDecimal.ONE, 60, null,
                LocalTime.NOON, LocalTime.NOON));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingPriceCalculatorTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-101 — BookingServiceTest#scoutCanReadOwnBookingButNotAnotherScoutsBooking

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingServiceTest#scoutCanReadOwnBookingButNotAnotherScoutsBooking`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        when(bookingRepository.findById(bookingId)).thenReturn(Optional.of(booking));
        User scout = User.builder().id(scoutId).role(UserRole.SCOUT).build();
        when(currentUserService.getCurrentUser()).thenReturn(scout);
        assertEquals(bookingId, service.getBookingForCurrentUser(bookingId).id());

        when(currentUserService.getCurrentUser())
                .thenReturn(User.builder().id(UUID.randomUUID()).role(UserRole.SCOUT).build());
        assertThrows(ForbiddenException.class, () -> service.getBookingForCurrentUser(bookingId));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-102 — BookingServiceTest#rejectsWebhookWhosePaymentIdDoesNotMatchStoredPayment

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingServiceTest#rejectsWebhookWhosePaymentIdDoesNotMatchStoredPayment`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        when(bookingRepository.findByIdForUpdate(bookingId)).thenReturn(Optional.of(booking));
        MolliePaymentResponse payment = paidPayment("tr_wrong", "100.00");
        assertThrows(IllegalStateException.class, () -> service.processPaidPayment(payment));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-103 — BookingServiceTest#rejectsWebhookWhoseAmountDoesNotMatchDatabasePrice

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingServiceTest#rejectsWebhookWhoseAmountDoesNotMatchDatabasePrice`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        when(bookingRepository.findByIdForUpdate(bookingId)).thenReturn(Optional.of(booking));
        MolliePaymentResponse payment = paidPayment("tr_expected", "99.99");
        assertThrows(IllegalStateException.class, () -> service.processPaidPayment(payment));
        assertEquals(BookingStatus.AWAITING_PAYMENT, booking.getStatus());
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-104 — BookingServiceTest#duplicatePaidWebhookDoesNotCreateAnotherEvent

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingServiceTest#duplicatePaidWebhookDoesNotCreateAnotherEvent`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        booking.setStatus(BookingStatus.CONFIRMED);
        when(bookingRepository.findByIdForUpdate(bookingId)).thenReturn(Optional.of(booking));
        service.processPaidPayment(paidPayment("tr_expected", "100.00"));
        verifyNoInteractions(bookingEventRepository);
        verifyNoInteractions(eventPublisher);
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-105 — BookingServiceTest#firstPaidTransitionPublishesNotificationOnceAndDuplicateDoesNot

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingServiceTest#firstPaidTransitionPublishesNotificationOnceAndDuplicateDoesNot`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        when(bookingRepository.findByIdForUpdate(bookingId)).thenReturn(Optional.of(booking));
        service.processPaidPayment(paidPayment("tr_expected", "100.00"));
        service.processPaidPayment(paidPayment("tr_expected", "100.00"));
        assertEquals(BookingStatus.CONFIRMED, booking.getStatus());
        verify(eventPublisher, times(1)).publishEvent(new BookingPaidEvent(bookingId));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-106 — BookingServiceTest#ownerAndAdminCanReadAuthorizedBooking

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingServiceTest#ownerAndAdminCanReadAuthorizedBooking`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        UUID ownerId = UUID.randomUUID();
        User owner = User.builder().id(ownerId).role(UserRole.OWNER).build();
        Venue venue = Venue.builder().id(booking.getVenueId()).owner(owner).name("Venue").build();
        when(bookingRepository.findById(bookingId)).thenReturn(Optional.of(booking));
        when(venueRepository.findById(booking.getVenueId())).thenReturn(Optional.of(venue));
        when(currentUserService.getCurrentUser()).thenReturn(owner);
        assertEquals(bookingId, service.getBookingForCurrentUser(bookingId).id());

        when(currentUserService.getCurrentUser()).thenReturn(User.builder().id(UUID.randomUUID()).role(UserRole.ADMIN).build());
        assertEquals(bookingId, service.getBookingForCurrentUser(bookingId).id());
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-107 — BookingServiceTest#databaseConflictFromConcurrentAttemptCannotCreateSecondBooking

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingServiceTest#databaseConflictFromConcurrentAttemptCannotCreateSecondBooking`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        User owner = User.builder().id(UUID.randomUUID()).email("owner@example.com").fullName("Owner").role(UserRole.OWNER).build();
        User scout = User.builder().id(scoutId).email("scout@example.com").role(UserRole.SCOUT).build();
        Venue venue = Venue.builder().id(booking.getVenueId()).owner(owner).name("Venue").status(VenueStatus.ACTIVE)
                .timezone("Europe/Amsterdam")
                .hourlyRateEur(new BigDecimal("25.00")).minimumBookingMinutes(240)
                .fullDayRateEur(new BigDecimal("100.00")).build();
        when(venueRepository.findById(venue.getId())).thenReturn(Optional.of(venue));
        when(userRepository.findById(scoutId)).thenReturn(Optional.of(scout));
        when(bookingRepository.saveAndFlush(any(Booking.class)))
                .thenThrow(new DataIntegrityViolationException("exclusion constraint"));
        CreateBookingRequest request = new CreateBookingRequest(venue.getId(), LocalDate.now().plusDays(1),
                java.time.LocalTime.of(9, 0), java.time.LocalTime.of(17, 0), "Brief",
                new ScoutContactRequest("Scout", "scout@example.com", null));
```

```
        assertThrows(ConflictException.class, () -> service.createBooking(request, scoutId));
        verifyNoInteractions(bookingEventRepository);
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-108 — BookingServiceTest#requestAcceptPaymentEndsConfirmedWithoutSecondOwnerAction

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingServiceTest#requestAcceptPaymentEndsConfirmedWithoutSecondOwnerAction`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        User owner = ownerForBooking();
        prepareRequestedBooking(owner);
```

```
        assertEquals(BookingStatus.AWAITING_PAYMENT, service.acceptRequest(bookingId).status());
        verifyNoInteractions(priceCalculator);
        service.processPaidPayment(paidPayment("tr_expected", "100.00"));
        assertEquals(BookingStatus.CONFIRMED, booking.getStatus());
        service.processPaidPayment(paidPayment("tr_expected", "100.00"));
        verify(eventPublisher, times(1)).publishEvent(new BookingPaidEvent(bookingId));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-109 — BookingServiceTest#scoutCreatesRequestWithSelectedTimesWithoutStartingPayment

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingServiceTest#scoutCreatesRequestWithSelectedTimesWithoutStartingPayment`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        User owner = ownerForBooking();
        User scout = User.builder().id(scoutId).email("scout@example.com").role(UserRole.SCOUT).build();
        Venue venue = Venue.builder().id(booking.getVenueId()).owner(owner).name("Venue").status(VenueStatus.ACTIVE)
                .timezone("Europe/Amsterdam")
                .hourlyRateEur(new BigDecimal("175.00")).minimumBookingMinutes(240)
                .fullDayRateEur(new BigDecimal("1200.00")).build();
        when(venueRepository.findById(venue.getId())).thenReturn(Optional.of(venue));
        when(userRepository.findById(scoutId)).thenReturn(Optional.of(scout));
        when(bookingRepository.saveAndFlush(any(Booking.class))).thenAnswer(invocation -> {
            Booking saved = invocation.getArgument(0);
            saved.setId(bookingId);
            return saved;
        });
        CreateBookingRequest request = new CreateBookingRequest(venue.getId(), LocalDate.now().plusDays(1),
                LocalTime.of(10, 0), LocalTime.of(14, 0), "Brief",
                new ScoutContactRequest("Scout", "scout@example.com", null));

        var result = service.createBooking(request, scoutId);
        assertEquals(BookingStatus.REQUESTED, result.status());
        assertEquals(LocalTime.of(10, 0), result.startTime());
        assertEquals(LocalTime.of(14, 0), result.endTime());
        assertEquals(new BigDecimal("700.00"), result.venuePriceEur());
        assertEquals(new BigDecimal("175.00"), result.hourlyRateEurSnapshot());
        assertEquals(240, result.minimumBookingMinutesSnapshot());
        assertEquals(240, result.durationMinutes());
        assertEquals(240, result.billableMinutes());
        assertEquals(null, result.molliePaymentId());
        org.junit.jupiter.api.Assertions.assertNotNull(result.ownerResponseExpiresAt());
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-110 — BookingServiceTest#ownerCanDeclineRequestAndDoubleAcceptanceIsRejected

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingServiceTest#ownerCanDeclineRequestAndDoubleAcceptanceIsRejected`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        User owner = ownerForBooking();
        prepareRequestedBooking(owner);
        assertEquals(BookingStatus.DECLINED, service.declineRequest(bookingId).status());

        booking.setStatus(BookingStatus.REQUESTED);
        service.acceptRequest(bookingId);
        assertThrows(ConflictException.class, () -> service.acceptRequest(bookingId));
        verify(eventPublisher, times(1)).publishEvent(new BookingLifecycleEvent(
                bookingId, BookingLifecycleEvent.Type.REQUEST_ACCEPTED));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-111 — BookingServiceTest#unauthorizedOwnerCannotAcceptRequest

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingServiceTest#unauthorizedOwnerCannotAcceptRequest`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        User actualOwner = ownerForBooking();
        prepareRequestedBooking(actualOwner);
        when(currentUserService.getCurrentUser()).thenReturn(
                User.builder().id(UUID.randomUUID()).email("other@example.com").role(UserRole.OWNER).build());
        assertThrows(ForbiddenException.class, () -> service.acceptRequest(bookingId));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-112 — BookingServiceTest#acceptedRequestMakesOnlyOverlappingRequestsUnavailable

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingServiceTest#acceptedRequestMakesOnlyOverlappingRequestsUnavailable`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        User owner = ownerForBooking();
        prepareRequestedBooking(owner);
        Booking overlapping = Booking.builder().id(UUID.randomUUID()).venueId(booking.getVenueId())
                .status(BookingStatus.REQUESTED).shootDate(booking.getShootDate())
                .startTime(LocalTime.of(16, 0)).endTime(LocalTime.of(18, 0)).build();
        when(bookingRepository.findOverlappingRequestedForUpdate(booking.getVenueId(), booking.getShootDate(),
                booking.getStartTime(), booking.getEndTime(), bookingId)).thenReturn(java.util.List.of(overlapping));

        service.acceptRequest(bookingId);

        assertEquals(BookingStatus.AWAITING_PAYMENT, booking.getStatus());
        assertEquals(BookingStatus.UNAVAILABLE, overlapping.getStatus());
        verify(eventPublisher).publishEvent(new BookingLifecycleEvent(
```

```
                overlapping.getId(), BookingLifecycleEvent.Type.REQUEST_BECAME_UNAVAILABLE));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-113 — BookingServiceTest#acceptanceAfterOwnerDeadlineExpiresRequestAndFailsClearly

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingServiceTest#acceptanceAfterOwnerDeadlineExpiresRequestAndFailsClearly`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        User owner = ownerForBooking();
        prepareRequestedBooking(owner);
        booking.setOwnerResponseExpiresAt(OffsetDateTime.now().minusMinutes(1));
```

```
        assertThrows(com.eema.backend.exception.RequestDeadlineExpiredException.class,
                () -> service.acceptRequest(bookingId));
        assertEquals(BookingStatus.EXPIRED, booking.getStatus());
        verify(availabilityValidator, never()).validate(any(), any(), any(), any(), any());
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-114 — BookingServiceTest#acceptanceThatHasBecomeTooLateExpiresRequest

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingServiceTest#acceptanceThatHasBecomeTooLateExpiresRequest`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        User owner = ownerForBooking();
```

```
        prepareRequestedBooking(owner);
        Venue venue = venueRepository.findByIdForUpdate(booking.getVenueId()).orElseThrow();
        VenueBookingLeadTimePolicy.MinimumLeadTimeNotMetException failure =
                new VenueBookingLeadTimePolicy.MinimumLeadTimeNotMetException("Minimum lead time not met");
        when(leadTimePolicy.isSatisfied(venue, booking.getShootDate(), booking.getStartTime())).thenReturn(false);
        when(leadTimePolicy.notMet(venue)).thenReturn(failure);

        assertEquals(failure, assertThrows(VenueBookingLeadTimePolicy.MinimumLeadTimeNotMetException.class,
                () -> service.acceptRequest(bookingId)));
        assertEquals(BookingStatus.EXPIRED, booking.getStatus());
        verify(availabilityValidator, never()).validate(any(), any(), any(), any(), any());
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-115 — BookingServiceTest#dueRequestExpirationIsIdempotentAndIgnoresAcceptedRows

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingServiceTest#dueRequestExpirationIsIdempotentAndIgnoresAcceptedRows`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        OffsetDateTime now = OffsetDateTime.now();
        Booking due = Booking.builder().id(UUID.randomUUID()).status(BookingStatus.REQUESTED)
                .ownerResponseExpiresAt(now.minusSeconds(1)).build();
        when(bookingRepository.lockDueRequestedBookings(now, 100)).thenReturn(java.util.List.of(due));
        assertEquals(1, service.expireDueRequests(now, 100));
        assertEquals(BookingStatus.EXPIRED, due.getStatus());
        when(bookingRepository.lockDueRequestedBookings(now, 100)).thenReturn(java.util.List.of());
        assertEquals(0, service.expireDueRequests(now, 100));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-116 — BookingServiceTest#approvedRequestExpiresAndLatePaymentIsMarkedForRefund

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingServiceTest#approvedRequestExpiresAndLatePaymentIsMarkedForRefund`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        User owner = ownerForBooking();
        prepareRequestedBooking(owner);
        service.acceptRequest(bookingId);
        OffsetDateTime afterDeadline = booking.getPaymentExpiresAt().plusSeconds(1);
        service.expireApproval(bookingId, afterDeadline);
        assertEquals(BookingStatus.EXPIRED, booking.getStatus());
        var outcome = service.processPaidPayment(paidPayment("tr_expected", "100.00"));
        assertEquals(BookingService.PaidPaymentResult.LATE_PAYMENT, outcome.result());
        assertEquals(BookingStatus.EXPIRED, booking.getStatus());
        assertEquals(new BigDecimal("100.00"), booking.getTotalPaidEur());
        verify(eventPublisher, never()).publishEvent(any(BookingPaidEvent.class));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-117 — BookingServiceTest#paymentAfterDeadlineExpiresBookingEvenBeforeSchedulerRuns

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingServiceTest#paymentAfterDeadlineExpiresBookingEvenBeforeSchedulerRuns`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        booking.setStatus(BookingStatus.AWAITING_PAYMENT);
        booking.setPaymentExpiresAt(OffsetDateTime.now().minusSeconds(1));
        when(bookingRepository.findByIdForUpdate(bookingId)).thenReturn(Optional.of(booking));

        var outcome = service.processPaidPayment(paidPayment("tr_expected", "100.00"));

        assertEquals(BookingService.PaidPaymentResult.LATE_PAYMENT, outcome.result());
        assertEquals(BookingStatus.EXPIRED, booking.getStatus());
        verify(eventPublisher, never()).publishEvent(any(BookingPaidEvent.class));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-118 — BookingServiceTest#acceptedInquiryBookingPublishesOnlyAcceptedBookingLifecycleEmail

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingServiceTest#acceptedInquiryBookingPublishesOnlyAcceptedBookingLifecycleEmail`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        User owner = ownerForBooking();
        User scout = User.builder().id(scoutId).email("scout@example.com").fullName("Scout")
                .role(UserRole.SCOUT).build();
        Venue venue = Venue.builder().id(booking.getVenueId()).owner(owner).name("Studio")
                .timezone("Europe/Amsterdam").status(VenueStatus.ACTIVE)
                .hourlyRateEur(new BigDecimal("25.00"))
                .minimumBookingMinutes(240).fullDayRateEur(new BigDecimal("100.00")).build();
        when(venueRepository.findById(venue.getId())).thenReturn(Optional.of(venue));
        when(userRepository.findById(scoutId)).thenReturn(Optional.of(scout));
        when(bookingRepository.findBlockingBookings(venue.getId(), LocalDate.now().plusDays(3))).thenReturn(java.util.List.of());
        when(bookingRepository.saveAndFlush(any(Booking.class))).thenAnswer(invocation -> {
            Booking saved = invocation.getArgument(0);
            saved.setId(bookingId);
            return saved;
        });

        var result = service.createApprovedExceptionBooking(venue.getId(), scoutId,
                LocalDate.now().plusDays(3), LocalTime.of(7, 0), LocalTime.of(9, 0), "Exception time");

        assertEquals(BookingStatus.AWAITING_PAYMENT, result.status());
        verify(eventPublisher, times(1)).publishEvent(new BookingLifecycleEvent(
```

```
                bookingId, BookingLifecycleEvent.Type.REQUEST_ACCEPTED));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-119 — BookingStateMachineTest#permitsDefinedTransitions

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingStateMachineTest#permitsDefinedTransitions`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        assertDoesNotThrow(() -> machine.requireTransition(BookingStatus.REQUESTED, BookingStatus.AWAITING_PAYMENT));
        assertDoesNotThrow(() -> machine.requireTransition(BookingStatus.REQUESTED, BookingStatus.DECLINED));
        assertDoesNotThrow(() -> machine.requireTransition(BookingStatus.REQUESTED, BookingStatus.EXPIRED));
        assertDoesNotThrow(() -> machine.requireTransition(BookingStatus.REQUESTED, BookingStatus.UNAVAILABLE));
```

```
        assertDoesNotThrow(() -> machine.requireTransition(BookingStatus.AWAITING_PAYMENT, BookingStatus.CONFIRMED));
        assertDoesNotThrow(() -> machine.requireTransition(BookingStatus.AWAITING_PAYMENT, BookingStatus.EXPIRED));
        assertDoesNotThrow(() -> machine.requireTransition(BookingStatus.CONFIRMED, BookingStatus.REFUNDED));
        assertDoesNotThrow(() -> machine.requireTransition(BookingStatus.CONFIRMED, BookingStatus.CANCELLED));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingStateMachineTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-120 — BookingStateMachineTest#rejectsUserSkippingPaymentAndTerminalMutations

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=BookingStateMachineTest#rejectsUserSkippingPaymentAndTerminalMutations`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        assertThrows(ConflictException.class,
```

```
                () -> machine.requireTransition(BookingStatus.REQUESTED, BookingStatus.CONFIRMED));
        assertThrows(ConflictException.class,
                () -> machine.requireTransition(BookingStatus.CANCELLED, BookingStatus.CONFIRMED));
        assertThrows(ConflictException.class,
                () -> machine.requireTransition(BookingStatus.UNAVAILABLE, BookingStatus.AWAITING_PAYMENT));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/BookingStateMachineTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-121 — CancellationServiceTest#openAwaitingPaymentIsCancelledAtMollieBeforeWithdrawal

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=CancellationServiceTest#openAwaitingPaymentIsCancelledAtMollieBeforeWithdrawal`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        Booking booking = Booking.builder().id(bookingId).status(BookingStatus.AWAITING_PAYMENT)
                .molliePaymentId("tr_open").build();
        arrange(booking);
        when(mollie.getPayment("tr_open")).thenReturn(payment("tr_open", "open"));
        when(mollie.cancelPayment("tr_open")).thenReturn(payment("tr_open", "canceled"));
        when(coordinator.cancel(bookingId, scout, null, now))
                .thenReturn(new BookingCancellationCoordinator.CancellationResult(bookingId, null));

        service.cancel(bookingId, null);

        verify(mollie).cancelPayment("tr_open");
        verifyNoInteractions(refunds);
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/CancellationServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-122 — CancellationServiceTest#paidProviderStateIsAppliedBeforeConfirmedCancellationPolicy

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=CancellationServiceTest#paidProviderStateIsAppliedBeforeConfirmedCancellationPolicy`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        Booking booking = Booking.builder().id(bookingId).status(BookingStatus.AWAITING_PAYMENT)
                .molliePaymentId("tr_paid").build();
        arrange(booking);
        MolliePaymentResponse paid = payment("tr_paid", "paid");
        when(mollie.getPayment("tr_paid")).thenReturn(paid);
        RefundCoordinator.Claim claim = new RefundCoordinator.Claim(
                bookingId, "tr_paid", new BigDecimal("100.00"), UUID.randomUUID());
        when(coordinator.cancel(bookingId, scout, null, now))
                .thenReturn(new BookingCancellationCoordinator.CancellationResult(bookingId, claim));

        service.cancel(bookingId, null);

        var order = inOrder(bookingService, coordinator, refunds);
        order.verify(bookingService).processPaidPayment(paid);
        order.verify(coordinator).cancel(bookingId, scout, null, now);
        order.verify(refunds).requestCancellationRefund(claim);
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/CancellationServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-123 — CancellationServiceTest#zeroRefundCancellationNeverCallsRefundProvider

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=CancellationServiceTest#zeroRefundCancellationNeverCallsRefundProvider`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        Booking booking = Booking.builder().id(bookingId).status(BookingStatus.CONFIRMED).build();
        arrange(booking);
        when(coordinator.cancel(bookingId, scout, null, now))
                .thenReturn(new BookingCancellationCoordinator.CancellationResult(bookingId, null));

        service.cancel(bookingId, new CancelBookingRequest(null));

        verifyNoInteractions(mollie, refunds);
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/CancellationServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-124 — CancellationServiceTest#duplicateCancellationResumesAmbiguousRefundWithPersistedClaim

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=CancellationServiceTest#duplicateCancellationResumesAmbiguousRefundWithPersistedClaim`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        Booking booking = Booking.builder().id(bookingId).status(BookingStatus.CANCELLED).build();
        when(current.getCurrentUser()).thenReturn(scout);
        when(clock.now()).thenReturn(now);
        when(bookings.findById(bookingId)).thenReturn(Optional.of(booking));
        when(coordinator.preview(bookingId, scout, now)).thenReturn(new CancellationPreviewDto(
                bookingId, null, BigDecimal.ZERO, BigDecimal.ZERO, BigDecimal.ZERO, null, false));

        service.cancel(bookingId, null);

        verify(refunds).resumeCancellationRefund(bookingId);
        verify(coordinator, never()).cancel(any(), any(), any(), any());
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/CancellationServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-125 — CriticalBookingWorkflowTest#requestApprovalCheckoutAndAuthoritativePaymentCompleteOneReservedBooking

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=CriticalBookingWorkflowTest#requestApprovalCheckoutAndAuthoritativePaymentCompleteOneReservedBooking`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        Booking booking = createRequest();
        assertEquals(BookingStatus.REQUESTED, booking.getStatus());
        assertFalse(BookingReservationPolicy.blocksAvailability(booking.getStatus()));

        when(current.getCurrentUser()).thenReturn(owner);
        bookingService.acceptRequest(bookingId);
        assertEquals(BookingStatus.AWAITING_PAYMENT, booking.getStatus());
        assertTrue(BookingReservationPolicy.blocksAvailability(booking.getStatus()));

        PaymentCreationCoordinator coordinator = new PaymentCreationCoordinator(bookings, new BookingFinancials());
        PaymentService payments = new PaymentService(coordinator, mollie);
        MolliePaymentResponse created = payment("open", true);
```

```
        when(mollie.createPayment(eq(bookingId), eq(new BigDecimal("100.00")), any())).thenReturn(created);
        var checkout = payments.createPayment(bookingId, scout.getId());
        assertEquals("tr_workflow", checkout.molliePaymentId());
        assertEquals(PaymentCreationStatus.CREATED, booking.getPaymentCreationStatus());

        MolliePaymentResponse paid = payment("paid", false);
        when(mollie.getPayment("tr_workflow")).thenReturn(paid);
        WebhookService webhook = new WebhookService(mollie, bookingService, refunds);
        webhook.handlePaymentWebhook("tr_workflow");
        webhook.handlePaymentWebhook("tr_workflow");

        assertEquals(BookingStatus.CONFIRMED, booking.getStatus());
        assertTrue(BookingReservationPolicy.blocksAvailability(booking.getStatus()));
        verify(events).publishEvent(new BookingLifecycleEvent(bookingId, BookingLifecycleEvent.Type.REQUEST_SUBMITTED));
        verify(events).publishEvent(new BookingLifecycleEvent(bookingId, BookingLifecycleEvent.Type.REQUEST_ACCEPTED));
        verify(events, times(1)).publishEvent(new BookingPaidEvent(bookingId));
        verify(audit, atLeastOnce()).save(argThat(event -> event.getEventType() == BookingEventType.PAYMENT_SUCCEEDED));
        verify(audit, atLeastOnce()).save(argThat(event -> event.getEventType() == BookingEventType.BOOKING_CONFIRMED));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/CriticalBookingWorkflowTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-126 — CriticalBookingWorkflowTest#declineAndExpiryReleaseSlotsAndRejectCheckout

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=CriticalBookingWorkflowTest#declineAndExpiryReleaseSlotsAndRejectCheckout`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        Booking declined = createRequest();
        when(current.getCurrentUser()).thenReturn(owner);
        bookingService.declineRequest(bookingId);
        assertEquals(BookingStatus.DECLINED, declined.getStatus());
        assertFalse(BookingReservationPolicy.blocksAvailability(declined.getStatus()));
        PaymentCreationCoordinator coordinator = new PaymentCreationCoordinator(bookings, new BookingFinancials());
        assertThrows(com.eema.backend.exception.BadRequestException.class,
                () -> coordinator.claim(bookingId, scout.getId()));
        verify(events).publishEvent(new BookingLifecycleEvent(bookingId, BookingLifecycleEvent.Type.REQUEST_DECLINED));

        resetForNewRequest();
        Booking expired = createRequest();
        when(current.getCurrentUser()).thenReturn(owner);
        bookingService.acceptRequest(bookingId);
        bookingService.expireApproval(bookingId, expired.getPaymentExpiresAt().plusSeconds(1));
        assertEquals(BookingStatus.EXPIRED, expired.getStatus());
        assertFalse(BookingReservationPolicy.blocksAvailability(expired.getStatus()));
        assertThrows(com.eema.backend.exception.BadRequestException.class,
                () -> coordinator.claim(bookingId, scout.getId()));
        verify(events).publishEvent(new BookingLifecycleEvent(bookingId, BookingLifecycleEvent.Type.APPROVAL_EXPIRED));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/CriticalBookingWorkflowTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-127 — CriticalBookingWorkflowTest#paidCallbackAfterDeadlineKeepsSlotReleasedAndInitiatesOneRefund

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=CriticalBookingWorkflowTest#paidCallbackAfterDeadlineKeepsSlotReleasedAndInitiatesOneRefund`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        Booking booking = createRequest();
        when(current.getCurrentUser()).thenReturn(owner);
        bookingService.acceptRequest(bookingId);
        PaymentCreationCoordinator paymentCoordinator = new PaymentCreationCoordinator(bookings, new BookingFinancials());
        when(mollie.createPayment(eq(bookingId), any(), any())).thenReturn(payment("open", true));
        new PaymentService(paymentCoordinator, mollie).createPayment(bookingId, scout.getId());
        booking.setPaymentExpiresAt(OffsetDateTime.now().minusSeconds(1));

        RefundCoordinator refundCoordinator = new RefundCoordinator(bookings, audit, new BookingFinancials(),
                new BookingStateMachine(), events, payouts);
        RefundService refundService = new RefundService(current, refundCoordinator, mollie, bookingService);
        MollieRefundResponse queued = new MollieRefundResponse("re_late", "queued", "tr_workflow",
                new MolliePaymentResponse.Amount("100.00", "EUR"));
        when(mollie.getPayment("tr_workflow")).thenReturn(payment("paid", false));
```

```
        when(mollie.createRefund(eq("tr_workflow"), eq(bookingId), eq(new BigDecimal("100.00")), any()))
                .thenReturn(queued);
        when(bookings.findByMolliePaymentId("tr_workflow")).thenReturn(Optional.of(booking));
        when(mollie.getRefund("tr_workflow", "re_late")).thenReturn(queued);

        WebhookService webhook = new WebhookService(mollie, bookingService, refundService);
        webhook.handlePaymentWebhook("tr_workflow");
        webhook.handlePaymentWebhook("tr_workflow");

        assertEquals(BookingStatus.EXPIRED, booking.getStatus());
        assertEquals(RefundStatus.QUEUED, booking.getRefundStatus());
        assertFalse(BookingReservationPolicy.blocksAvailability(booking.getStatus()));
        verify(mollie, times(1)).createRefund(eq("tr_workflow"), eq(bookingId), any(), any());
        verify(events, never()).publishEvent(any(BookingPaidEvent.class));
        verify(events, times(1)).publishEvent(new LatePaymentDetectedEvent(bookingId));
        verify(events, times(1)).publishEvent(new LatePaymentRefundInitiatedEvent(bookingId));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/CriticalBookingWorkflowTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-128 — EmailServiceTest#disabledEmailNeedsNoCredentials

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=EmailServiceTest#disabledEmailNeedsNoCredentials`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        assertDoesNotThrow(() -> new EmailService(properties(false, null, null)));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/EmailServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-129 — EmailServiceTest#enabledEmailRequiresCredentials

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=EmailServiceTest#enabledEmailRequiresCredentials`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace

the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        assertThrows(IllegalStateException.class, () -> new EmailService(properties(true, "", "")));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/EmailServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-130 — FinancialDocumentConcurrencyContractTest#issuanceLocksAuthoritativeAggregateRowsBeforeUniqueDocumentCreation

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=FinancialDocumentConcurrencyContractTest#issuanceLocksAuthoritativeAggregateRowsBeforeUniqueDocumentCreation`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        Lock bookingLock = BookingRepository.class.getMethod("findByIdForUpdate", UUID.class).getAnnotation(Lock.class);
        Lock payoutLock = OwnerPayoutRepository.class.getMethod("findByIdForUpdate", UUID.class).getAnnotation(Lock.class);
        assertThat(bookingLock).isNotNull();
        assertThat(bookingLock.value()).isEqualTo(LockModeType.PESSIMISTIC_WRITE);
        assertThat(payoutLock).isNotNull();
        assertThat(payoutLock.value()).isEqualTo(LockModeType.PESSIMISTIC_WRITE);
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/FinancialDocumentConcurrencyContractTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-131 — FinancialDocumentPdfRendererTest#receiptContainsPaymentSnapshotAndExcludesInternalEconomics

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=FinancialDocumentPdfRendererTest#receiptContainsPaymentSnapshotAndExcludesInternalEconomics`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        Booking b = booking(); Venue venue = venue();
        String text = text(renderer.bookingReceipt("RCT-2026-000001", b, venue));
        assertThat(text).contains("Booking receipt", "RCT-2026-000001", b.getId().toString(),
                "Studio North", "27 Aug 2026", "10:00 - 14:00", "EUR 1000.00", "Total paid");
        assertThat(text).doesNotContain("commission", "Owner payout", "Mollie fee", "provider cost");
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/FinancialDocumentPdfRendererTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-132 — FinancialDocumentPdfRendererTest#payoutStatementUsesExactLedgerValuesAndTransferReference

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=FinancialDocumentPdfRendererTest#payoutStatementUsesExactLedgerValuesAndTransferReference`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        Booking b = booking(); Venue venue = venue(); User owner = venue.getOwner();
        OwnerPayout p = OwnerPayout.builder().id(UUID.randomUUID()).bookingId(b.getId()).venueId(venue.getId())
                .ownerId(owner.getId()).status(OwnerPayoutStatus.PAID).paidAt(OffsetDateTime.parse("2026-08-28T12:00:00Z"))
                .externalTransferReference("bank-transfer-42").grossRetainedAmountEur(new BigDecimal("500.00"))
                .commissionEur(new BigDecimal("100.00")).providerCostEur(new BigDecimal("15.00"))
                .payoutAmountEur(new BigDecimal("385.00")).build();
        String text = text(renderer.payoutStatement("PST-2026-000001", p, b, venue, owner));
        assertThat(text).contains("Owner payout statement", "PST-2026-000001", "bank-transfer-42",
                "EUR 500.00", "EUR 100.00", "EUR 15.00", "EUR 385.00",
                "not presented as a tax invoice");
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/FinancialDocumentPdfRendererTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-133 — FinancialDocumentServiceTest#confirmedScoutReceiptIssuesOnceAndAdminCanReuseIt

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=FinancialDocumentServiceTest#confirmedScoutReceiptIssuesOnceAndAdminCanReuseIt`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        User scout = User.builder().id(scoutId).role(UserRole.SCOUT).build();
        var first = service.bookingReceipt(bookingId, scout);
        var second = service.bookingReceipt(bookingId, User.builder().id(UUID.randomUUID()).role(UserRole.ADMIN).build());
        assertThat(first.documentNumber()).matches("RCT-\\d{4}-000001");
        assertThat(second.documentNumber()).isEqualTo(first.documentNumber());
        verify(renderer, times(1)).bookingReceipt(any(), same(booking), same(venue));
        verify(events, times(1)).save(argThat(e -> e.getEventType() == BookingEventType.BOOKING_RECEIPT_ISSUED));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/FinancialDocumentServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-134 — FinancialDocumentServiceTest#documentNumbersAreUniqueAndHistoricalReceiptIsNotRewritten

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=FinancialDocumentServiceTest#documentNumbersAreUniqueAndHistoricalReceiptIsNotRewritten`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        User scout = User.builder().id(scoutId).role(UserRole.SCOUT).build();
        var first = service.bookingReceipt(bookingId, scout);
        booking.setStatus(BookingStatus.CANCELLED);
        var historical = service.bookingReceipt(bookingId, scout);
        assertThat(historical.documentNumber()).isEqualTo(first.documentNumber());

        UUID secondId = UUID.randomUUID();
        Booking second = Booking.builder().id(secondId).venueId(venueId).scoutId(scoutId)
                .status(BookingStatus.CONFIRMED).confirmedAt(OffsetDateTime.now()).paidAt(OffsetDateTime.now())
                .totalPaidEur(BigDecimal.ONE).build();
        when(bookings.findByIdForUpdate(secondId)).thenReturn(Optional.of(second));
        when(documents.findByDocumentTypeAndBookingId(FinancialDocumentType.BOOKING_RECEIPT, secondId))
                .thenReturn(Optional.empty());
        receipt.set(null);
        var next = service.bookingReceipt(secondId, scout);
        assertThat(next.documentNumber()).matches("RCT-\\d{4}-000002").isNotEqualTo(first.documentNumber());
        verify(renderer, times(2)).bookingReceipt(any(), any(), any());
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/FinancialDocumentServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-135 — FinancialDocumentServiceTest#unpaidStatesAndUnauthorizedPeopleCannotObtainReceipt

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=FinancialDocumentServiceTest#unpaidStatesAndUnauthorizedPeopleCannotObtainReceipt`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        for (BookingStatus status : List.of(BookingStatus.REQUESTED, BookingStatus.AWAITING_PAYMENT)) {
            booking.setStatus(status);
            assertThatThrownBy(() -> service.bookingReceipt(bookingId,
                    User.builder().id(scoutId).role(UserRole.SCOUT).build())).isInstanceOf(ConflictException.class);
        }
        booking.setStatus(BookingStatus.CONFIRMED);
        assertThatThrownBy(() -> service.bookingReceipt(bookingId,
                User.builder().id(UUID.randomUUID()).role(UserRole.SCOUT).build())).isInstanceOf(ForbiddenException.class);
        assertThatThrownBy(() -> service.bookingReceipt(bookingId,
                User.builder().id(ownerId).role(UserRole.OWNER).build())).isInstanceOf(ForbiddenException.class);
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/FinancialDocumentServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-136 — FinancialDocumentServiceTest#confirmedPaymentRemainsReceiptEligibleAfterCancellationBeforeFirstDownload

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=FinancialDocumentServiceTest#confirmedPaymentRemainsReceiptEligibleAfterCancellationBeforeFirstDownload`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        booking.setStatus(BookingStatus.CANCELLED);
        var result = service.bookingReceipt(bookingId, User.builder().id(scoutId).role(UserRole.SCOUT).build());
        assertThat(result.documentNumber()).startsWith("RCT-");
        verify(renderer).bookingReceipt(any(), same(booking), same(venue));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/FinancialDocumentServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-137 — FinancialDocumentServiceTest#onlyPaidPayoutUsesExactLedgerAndIssuesOneStatement

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=FinancialDocumentServiceTest#onlyPaidPayoutUsesExactLedgerAndIssuesOneStatement`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        User owner = venue.getOwner();
        var result = service.payoutStatement(payout.getId(), owner);
        assertThat(result.documentNumber()).matches("PST-\\d{4}-000001");
        verify(renderer).payoutStatement(any(), same(payout), same(booking), same(venue), same(owner));
        service.payoutStatement(payout.getId(), User.builder().id(UUID.randomUUID()).role(UserRole.ADMIN).build());
        verify(renderer, times(1)).payoutStatement(any(), any(), any(), any(), any());
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/FinancialDocumentServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-138 — FinancialDocumentServiceTest#readyHeldScoutAndUnrelatedOwnerCannotObtainStatement

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=FinancialDocumentServiceTest#readyHeldScoutAndUnrelatedOwnerCannotObtainStatement`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        for (OwnerPayoutStatus status : List.of(OwnerPayoutStatus.READY, OwnerPayoutStatus.HELD)) {
            payout.setStatus(status);
            assertThatThrownBy(() -> service.payoutStatement(payout.getId(), venue.getOwner())).isInstanceOf(ConflictException.class);
        }
        payout.setStatus(OwnerPayoutStatus.PAID);
        assertThatThrownBy(() -> service.payoutStatement(payout.getId(),
                User.builder().id(UUID.randomUUID()).role(UserRole.OWNER).build())).isInstanceOf(ForbiddenException.class);
        assertThatThrownBy(() -> service.payoutStatement(payout.getId(),
                User.builder().id(UUID.randomUUID()).role(UserRole.SCOUT).build())).isInstanceOf(ForbiddenException.class);
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/FinancialDocumentServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-139 — HttpMollieProviderClientTest#omitsWebhookUrlForManualLocalWebhookMode

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=HttpMollieProviderClientTest#omitsWebhookUrlForManualLocalWebhookMode`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        Map<String, Object> body = createPaymentAndCaptureBody("");

        assertThat(body).doesNotContainKey("webhookUrl");
        assertThat(body).containsEntry("redirectUrl", "http://localhost:3000/booked");
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/HttpMollieProviderClientTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-140 — HttpMollieProviderClientTest#sendsWebhookUrlWhenPublicTunnelIsConfigured

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=HttpMollieProviderClientTest#sendsWebhookUrlWhenPublicTunnelIsConfigured`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        Map<String, Object> body = createPaymentAndCaptureBody(
                "https://tunnel.example/api/webhooks/mollie/payment");

        assertThat(body).containsEntry("webhookUrl", "https://tunnel.example/api/webhooks/mollie/payment");
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/HttpMollieProviderClientTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-141 — IbanPolicyTest#normalizesAndAcceptsValidSepaIbans

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=IbanPolicyTest#normalizesAndAcceptsValidSepaIbans`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{assertThat(policy.normalizeAndValidate("nl91 abna 0417 1643 00")).isEqualTo("NL91ABNA0417164300");assertThat(policy.normalizeAndValidate("DE89 3704 0044 0532 0130 00")).isEqualTo("DE89370400440532013000");}
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/IbanPolicyTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-142 — IbanPolicyTest#rejectsChecksumAndUnsupportedCountry

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=IbanPolicyTest#rejectsChecksumAndUnsupportedCountry`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{assertThatThrownBy(()->policy.normalizeAndValidate("NL90ABNA0417164300")).isInstanceOf(BadRequestException.class);assertThatThrownBy(()->policy.normalizeAndValidate("GB82WEST12345698765432")).isInstanceOf(BadRequestException.class);}
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/IbanPolicyTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-143 — IncidentReportTest#transitionsAreNarrowAndExplicit

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=IncidentReportTest#transitionsAreNarrowAndExplicit`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{IncidentReport incident=IncidentReport.builder().category(IncidentCategory.OTHER).description("Operational
report").status(IncidentStatus.OPEN).build();incident.startReview();assertEquals(IncidentStatus.UNDER_REVIEW,incident.getStatus());OffsetDateTime
resolved=OffsetDateTime.now();incident.resolve("Handled
separately",resolved);assertEquals(IncidentStatus.RESOLVED,incident.getStatus());assertEquals(resolved,incident.getResolvedAt());incident.close();assertEqu
als(IncidentStatus.CLOSED,incident.getStatus());}
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/IncidentReportTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-144 — IncidentReportTest#cannotResolveBeforeReview

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=IncidentReportTest#cannotResolveBeforeReview`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{IncidentReport incident=IncidentReport.builder().category(IncidentCategory.ACCESS_PROBLEM).description("Could not
enter").status(IncidentStatus.OPEN).build();assertThrows(IllegalStateException.class,()->incident.resolve("Reviewed",OffsetDateTime.now()));}
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/IncidentReportTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-145 — LocalPrivateDocumentStorageTest#storesOpaqueRelativeReferencesAndRejectsTraversal

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=LocalPrivateDocumentStorageTest#storesOpaqueRelativeReferencesAndRejectsTraversal`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        LocalPrivateDocumentStorage storage = new LocalPrivateDocumentStorage(directory.toString());
        byte[] pdf = "%PDF-private".getBytes(java.nio.charset.StandardCharsets.US_ASCII);
        assertThat(storage.store("booking-receipts/id.pdf", pdf)).isEqualTo("booking-receipts/id.pdf");
        assertThat(storage.load("booking-receipts/id.pdf")).isEqualTo(pdf);
        assertThatThrownBy(() -> storage.load("../uploads/public.pdf")).isInstanceOf(IllegalArgumentException.class);
        assertThatThrownBy(() -> storage.load(directory.resolve("absolute.pdf").toString())).isInstanceOf(IllegalArgumentException.class);
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/LocalPrivateDocumentStorageTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-146 — LocalPrivateDocumentStorageTest#publicUploadControllerHasNoPrivateDocumentStorageDependency

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=LocalPrivateDocumentStorageTest#publicUploadControllerHasNoPrivateDocumentStorageDependency`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        assertThat(java.util.Arrays.stream(com.eema.backend.controller.UploadController.class.getDeclaredFields())
                .map(java.lang.reflect.Field::getType)).doesNotContain(PrivateDocumentStorage.class);
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/LocalPrivateDocumentStorageTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-147 — MollieFeeReconciliationJobTest#reconcilesAuthoritativeFeeOncePerPendingBooking

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=MollieFeeReconciliationJobTest#reconcilesAuthoritativeFeeOncePerPendingBooking`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        BookingRepository repository = mock(BookingRepository.class);
        MollieFeeReportingClient client = mock(MollieFeeReportingClient.class);
        BookingService service = mock(BookingService.class);
        AppProperties properties = new AppProperties(null, null, null,
                new AppProperties.MollieProperties(null, null, null, "token", true, 1000, 1000),
                null, null, null, null, null);
        Booking booking = Booking.builder().id(UUID.randomUUID()).molliePaymentId("tr_1").build();
        when(repository.findPendingFeeReconciliation(any())).thenReturn(List.of(booking), List.of());
        when(client.findAuthoritativePaymentFee("tr_1")).thenReturn(Optional.of(new BigDecimal("0.49")));
        MollieFeeReconciliationJob job = new MollieFeeReconciliationJob(properties, repository, client, service);
        job.reconcile();
        job.reconcile();
        verify(service, times(1)).reconcileMollieFee(booking.getId(), new BigDecimal("0.49"));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/MollieFeeReconciliationJobTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-148 — OwnerFinancialLiabilityServiceTest#settleUsesStoredAmountAuditsAndIsIdempotent

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=OwnerFinancialLiabilityServiceTest#settleUsesStoredAmountAuditsAndIsIdempotent`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        UUID id = UUID.randomUUID();
        OwnerFinancialLiability liability = OwnerFinancialLiability.builder().id(id).bookingId(UUID.randomUUID())
                .ownerId(UUID.randomUUID()).type(OwnerFinancialLiabilityType.PROVIDER_COST)
                .amountEur(new BigDecimal("15.00")).status(OwnerFinancialLiabilityStatus.OPEN).build();
        when(repository.findByIdForUpdate(id)).thenReturn(Optional.of(liability));
        when(repository.save(any())).thenAnswer(i -> i.getArgument(0));
        User admin = User.builder().id(UUID.randomUUID()).email("admin@example.test").role(UserRole.ADMIN).build();

        assertThat(service.settle(id, admin).status()).isEqualTo(OwnerFinancialLiabilityStatus.SETTLED);
        assertThat(liability.getSettledAt()).isNotNull();
        verify(events).save(argThat(e -> e.getEventType() == BookingEventType.OWNER_LIABILITY_SETTLED
                && e.getActorType() == ActorType.ADMIN));
        service.settle(id, admin);
        verify(events, times(1)).save(any());
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/OwnerFinancialLiabilityServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-149 — OwnerFinancialLiabilityServiceTest#nonAdminCannotSettleAndWaivedCannotBeSettled

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=OwnerFinancialLiabilityServiceTest#nonAdminCannotSettleAndWaivedCannotBeSettled`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        assertThatThrownBy(() -> service.settle(UUID.randomUUID(), User.builder().role(UserRole.OWNER).build()))
                .isInstanceOf(ForbiddenException.class);
        UUID id = UUID.randomUUID();
        when(repository.findByIdForUpdate(id)).thenReturn(Optional.of(OwnerFinancialLiability.builder()
                .id(id).status(OwnerFinancialLiabilityStatus.WAIVED).build()));
        assertThatThrownBy(() -> service.settle(id, User.builder().role(UserRole.ADMIN).build()))
                .isInstanceOf(ConflictException.class);
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/OwnerFinancialLiabilityServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-150 — OwnerPayoutServiceTest#otherwiseEligiblePayoutWaitsForVerifiedDestination

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=OwnerPayoutServiceTest#otherwiseEligiblePayoutWaitsForVerifiedDestination`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        when(destinations.findByOwnerIdAndStatus(ownerId, PayoutDestinationStatus.VERIFIED)).thenReturn(Optional.empty());
        OwnerPayout payout = service.recalculateForBooking(bookingId);
        assertThat(payout.getStatus()).isEqualTo(OwnerPayoutStatus.HELD);
        assertThat(payout.getHoldReason()).isEqualTo(OwnerPayoutHoldReason.WAITING_FOR_PAYOUT_DESTINATION);
        assertThat(payout.getPayoutDestination()).isNull();
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/OwnerPayoutServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-151 — OwnerPayoutServiceTest#normalEconomicsBecomeReadyOnlyAfterEndAndAuthoritativeFee

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=OwnerPayoutServiceTest#normalEconomicsBecomeReadyOnlyAfterEndAndAuthoritativeFee`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        OwnerPayout ready = service.recalculateForBooking(bookingId);
        assertThat(ready.getGrossRetainedAmountEur()).isEqualByComparingTo("1000.00");
        assertThat(ready.getCommissionEur()).isEqualByComparingTo("200.00");
        assertThat(ready.getProviderCostEur()).isEqualByComparingTo("15.00");
        assertThat(ready.getPayoutAmountEur()).isEqualByComparingTo("785.00");
        assertThat(ready.getStatus()).isEqualTo(OwnerPayoutStatus.READY);

        stored.set(null);
        booking.setShootDate(LocalDate.now().plusDays(1));
        assertThat(service.recalculateForBooking(bookingId).getStatus()).isEqualTo(OwnerPayoutStatus.CALCULATED);
        stored.set(null);
        booking.setShootDate(LocalDate.now().minusDays(1));
```

```
        booking.setMollieFeeReconciled(false);
        booking.setMollieFeeEur(BigDecimal.ZERO);
        booking.setOwnerPayoutEur(new BigDecimal("800.00"));
        OwnerPayout held = service.recalculateForBooking(bookingId);
        assertThat(held.getStatus()).isEqualTo(OwnerPayoutStatus.HELD);
        assertThat(held.getHoldReason()).isEqualTo(OwnerPayoutHoldReason.WAITING_FOR_PROVIDER_COST);
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/OwnerPayoutServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-152 — OwnerPayoutServiceTest#scoutPartialCancellationUsesAuthoritativeSnapshot

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=OwnerPayoutServiceTest#scoutPartialCancellationUsesAuthoritativeSnapshot`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        booking.setStatus(BookingStatus.CANCELLED);
        booking.setCancellationPolicyTier(CancellationPolicyTier.PARTIAL_REFUND);
        booking.setCancellationRetainedAmountEur(new BigDecimal("500.00"));
        booking.setCommissionEur(new BigDecimal("100.00"));
        booking.setOwnerPayoutEur(new BigDecimal("385.00"));
        booking.setProviderCostResponsibility(ProviderCostResponsibility.OWNER_SETTLEMENT);
        OwnerPayout payout = service.recalculateForBooking(bookingId);
        assertThat(payout.getGrossRetainedAmountEur()).isEqualByComparingTo("500.00");
        assertThat(payout.getPayoutAmountEur()).isEqualByComparingTo("385.00");
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/OwnerPayoutServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-153 — OwnerPayoutServiceTest#ownerCancellationCreatesProviderCostLiabilityWithoutNegativePayout

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=OwnerPayoutServiceTest#ownerCancellationCreatesProviderCostLiabilityWithoutNegativePayout`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        booking.setStatus(BookingStatus.CANCELLED);
        booking.setCancellationPolicyTier(CancellationPolicyTier.OWNER_FULL_REFUND);
        booking.setCancellationRetainedAmountEur(BigDecimal.ZERO);
        booking.setCommissionEur(BigDecimal.ZERO);
        booking.setOwnerPayoutEur(BigDecimal.ZERO);
        booking.setProviderCostResponsibility(ProviderCostResponsibility.OWNER);
        booking.setCancellationProviderCostEur(new BigDecimal("15.00"));
        booking.setCancellationProviderCostReconciled(true);
        OwnerPayout payout = service.recalculateForBooking(bookingId);
        assertThat(payout.getStatus()).isEqualTo(OwnerPayoutStatus.CANCELLED);
        assertThat(payout.getPayoutAmountEur()).isZero();
        verify(liabilities).save(argThat(l -> l.getType() == OwnerFinancialLiabilityType.PROVIDER_COST
                && l.getAmountEur().compareTo(new BigDecimal("15.00")) == 0));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/OwnerPayoutServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-154 — OwnerPayoutServiceTest#adminCancellationIsCancelledWithoutOwnerLiability

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=OwnerPayoutServiceTest#adminCancellationIsCancelledWithoutOwnerLiability`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        booking.setStatus(BookingStatus.CANCELLED);
        booking.setCancellationPolicyTier(CancellationPolicyTier.ADMIN_FULL_REFUND);
        booking.setCancellationRetainedAmountEur(BigDecimal.ZERO);
        booking.setCommissionEur(BigDecimal.ZERO);
        booking.setOwnerPayoutEur(BigDecimal.ZERO);
        booking.setProviderCostResponsibility(ProviderCostResponsibility.EEMA);
        assertThat(service.recalculateForBooking(bookingId).getStatus()).isEqualTo(OwnerPayoutStatus.CANCELLED);
        verify(liabilities, never()).save(any());
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/OwnerPayoutServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-155 — OwnerPayoutServiceTest#latePaymentCancelsAnyDefensivePayoutRecord

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=OwnerPayoutServiceTest#latePaymentCancelsAnyDefensivePayoutRecord`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        booking.setStatus(BookingStatus.CANCELLED);
        booking.setLatePaymentDetectedAt(OffsetDateTime.now());
        assertThat(service.recalculateForBooking(bookingId).getStatus()).isEqualTo(OwnerPayoutStatus.CANCELLED);
        assertThat(stored.get().getPayoutAmountEur()).isZero();
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/OwnerPayoutServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-156 — OwnerPayoutServiceTest#paidHistoryIsImmutableAndReversalCreatesRecoveryLiability

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=OwnerPayoutServiceTest#paidHistoryIsImmutableAndReversalCreatesRecoveryLiability`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        OwnerPayout paid = service.recalculateForBooking(bookingId);
        paid.setStatus(OwnerPayoutStatus.PAID);
        paid.setPaidAt(OffsetDateTime.now());
        paid.setExternalTransferReference("bank-1");
        booking.setStatus(BookingStatus.REFUNDED);
        booking.setOwnerPayoutEur(BigDecimal.ZERO);

        OwnerPayout result = service.recalculateForBooking(bookingId);
        assertThat(result.getPayoutAmountEur()).isEqualByComparingTo("785.00");
        assertThat(result.getStatus()).isEqualTo(OwnerPayoutStatus.PAID);
        verify(liabilities).save(argThat(l -> l.getType() == OwnerFinancialLiabilityType.OWNER_PAYOUT_RECOVERY
                && l.getAmountEur().compareTo(new BigDecimal("785.00")) == 0));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/OwnerPayoutServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-157 — OwnerPayoutServiceTest#markPaidIsIdempotentOnlyForSameReference

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=OwnerPayoutServiceTest#markPaidIsIdempotentOnlyForSameReference`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        OwnerPayout ready = service.recalculateForBooking(bookingId);
        when(payouts.findByIdForUpdate(ready.getId())).thenReturn(Optional.of(ready));
        User admin = User.builder().id(UUID.randomUUID()).email("admin@example.test").role(UserRole.ADMIN).build();
        assertThat(service.markPaid(ready.getId(), " transfer-42 ", admin).status()).isEqualTo(OwnerPayoutStatus.PAID);
        assertThat(service.markPaid(ready.getId(), "transfer-42", admin).externalTransferReference()).isEqualTo("transfer-42");
        assertThatThrownBy(() -> service.markPaid(ready.getId(), "different", admin)).isInstanceOf(ConflictException.class);
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/OwnerPayoutServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-158 — OwnerPayoutServiceTest#adminListForwardsAllFiltersAndCapsPageSize

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=OwnerPayoutServiceTest#adminListForwardsAllFiltersAndCapsPageSize`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        UUID venueFilter = UUID.randomUUID();
        when(payouts.search(any(), any(), any(), any(), any())).thenReturn(Page.empty());
        service.list(OwnerPayoutStatus.HELD, ownerId, bookingId, venueFilter, PageRequest.of(0, 500));
        verify(payouts).search(eq(OwnerPayoutStatus.HELD), eq(ownerId), eq(bookingId), eq(venueFilter),
                argThat(p -> p.getPageSize() == 100));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/OwnerPayoutServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-159 — OwnerPayoutServiceTest#ownerHistoryIsScopedAndDoesNotExposeTransferReference

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=OwnerPayoutServiceTest#ownerHistoryIsScopedAndDoesNotExposeTransferReference`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        OwnerPayout payout = OwnerPayout.builder().id(UUID.randomUUID()).ownerId(ownerId).bookingId(bookingId)
                .venueId(venueId).status(OwnerPayoutStatus.PAID).externalTransferReference("private-bank-ref").build();
        when(payouts.findOwnerPayouts(eq(ownerId), any())).thenReturn(new PageImpl<>(List.of(payout)));
        var result = service.listForOwner(ownerId, PageRequest.of(0, 20)).getContent().getFirst();
        assertThat(result.id()).isEqualTo(payout.getId());
        assertThat(result.statementAvailable()).isTrue();
        assertThat(Arrays.stream(result.getClass().getRecordComponents()).map(java.lang.reflect.RecordComponent::getName))
                .doesNotContain("externalTransferReference", "ownerId", "updatedAt");
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/OwnerPayoutServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-160 — OwnerVerificationServiceTest#ownerSubmitsAndCannotDowngradeVerifiedState

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=OwnerVerificationServiceTest#ownerSubmitsAndCannotDowngradeVerifiedState`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        when(current.getCurrentUser()).thenReturn(owner);when(repo.findByOwnerIdForUpdate(owner.getId())).thenReturn(Optional.empty());
        var submitted=service.submit(new OwnerVerificationSubmitRequest("Venue control evidence available"));
        assertThat(submitted.status()).isEqualTo(OwnerVerificationStatus.PENDING);
        OwnerVerification verified=OwnerVerification.builder().id(submitted.id()).owner(owner).status(OwnerVerificationStatus.VERIFIED).build();
        when(repo.findByOwnerIdForUpdate(owner.getId())).thenReturn(Optional.of(verified));
        assertThatThrownBy(()->service.submit(new OwnerVerificationSubmitRequest(null))).isInstanceOf(ConflictException.class);
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/OwnerVerificationServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-161 — OwnerVerificationServiceTest#adminApprovesPendingAndRejectsPendingWithReason

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=OwnerVerificationServiceTest#adminApprovesPendingAndRejectsPendingWithReason`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        when(current.getCurrentUser()).thenReturn(admin);
        OwnerVerification pending=OwnerVerification.builder().id(UUID.randomUUID()).owner(owner).status(OwnerVerificationStatus.PENDING).build();
        when(repo.findByIdForUpdate(pending.getId())).thenReturn(Optional.of(pending));
        assertThat(service.approve(pending.getId(),new
OwnerVerificationApproveRequest(VerificationMethod.VIDEO_CALL,"checked")).status()).isEqualTo(OwnerVerificationStatus.VERIFIED);
        pending.setStatus(OwnerVerificationStatus.PENDING);pending.setReviewedAt(null);pending.setReviewedBy(null);
        assertThat(service.reject(pending.getId(),new ReasonRequest("Authority evidence unclear")).status()).isEqualTo(OwnerVerificationStatus.REJECTED);
        assertThat(pending.getReviewNote()).isEqualTo("Authority evidence unclear");
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/OwnerVerificationServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-162 — PaymentCreationCoordinatorTest#concurrentClaimsReuseDurableProviderKeyAndCompletionIsIdempotent

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=PaymentCreationCoordinatorTest#concurrentClaimsReuseDurableProviderKeyAndCompletionIsIdempotent`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        UUID id = UUID.randomUUID();
        UUID scoutId = UUID.randomUUID();
        Booking booking = Booking.builder().id(id).scoutId(scoutId).status(BookingStatus.AWAITING_PAYMENT)
                .paymentExpiresAt(OffsetDateTime.now().plusHours(1)).venuePriceEur(new BigDecimal("100.00")).build();
        when(repository.findByIdForUpdate(id)).thenReturn(Optional.of(booking));

        PaymentCreationCoordinator.Claim first = coordinator.claim(id, scoutId);
        PaymentCreationCoordinator.Claim concurrent = coordinator.claim(id, scoutId);
        assertEquals(first.idempotencyKey(), concurrent.idempotencyKey());
        assertEquals(PaymentCreationStatus.CREATING, booking.getPaymentCreationStatus());

        MolliePaymentResponse response = new MolliePaymentResponse("tr_one", "open",
                new MolliePaymentResponse.Amount("100.00", "EUR"),
                new MolliePaymentResponse.MollieLinks(new MolliePaymentResponse.MollieLink("https://checkout")),
                java.util.Map.of("bookingId", id.toString()));
        assertEquals("tr_one", coordinator.complete(first, response).molliePaymentId());
        assertEquals("tr_one", coordinator.complete(concurrent, response).molliePaymentId());
```

```
        assertEquals(PaymentCreationStatus.CREATED, booking.getPaymentCreationStatus());
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/PaymentCreationCoordinatorTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-163 — PaymentCreationCoordinatorTest#refusesUnsafeRetryOutsideMollieReplayWindow

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=PaymentCreationCoordinatorTest#refusesUnsafeRetryOutsideMollieReplayWindow`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        UUID id = UUID.randomUUID();
        UUID scoutId = UUID.randomUUID();
        Booking booking = Booking.builder().id(id).scoutId(scoutId).status(BookingStatus.AWAITING_PAYMENT)
                .venuePriceEur(BigDecimal.TEN).paymentCreationStatus(PaymentCreationStatus.CREATING)
```

```
                .paymentExpiresAt(OffsetDateTime.now().plusHours(1))
                .paymentIdempotencyKey(UUID.randomUUID()).paymentCreationStartedAt(OffsetDateTime.now().minusHours(2)).build();
        when(repository.findByIdForUpdate(id)).thenReturn(Optional.of(booking));
        assertThrows(ConflictException.class, () -> coordinator.claim(id, scoutId));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/PaymentCreationCoordinatorTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-164 — PaymentCreationCoordinatorTest#checkoutBeforeOwnerApprovalIsRejected

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=PaymentCreationCoordinatorTest#checkoutBeforeOwnerApprovalIsRejected`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        UUID id = UUID.randomUUID();
```

```
        UUID scoutId = UUID.randomUUID();
        Booking booking = Booking.builder().id(id).scoutId(scoutId).status(BookingStatus.REQUESTED).build();
        when(repository.findByIdForUpdate(id)).thenReturn(Optional.of(booking));
        assertThrows(BadRequestException.class, () -> coordinator.claim(id, scoutId));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/PaymentCreationCoordinatorTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-165 — PaymentCreationCoordinatorTest#checkoutAfterApprovalExpirationIsRejected

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=PaymentCreationCoordinatorTest#checkoutAfterApprovalExpirationIsRejected`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        UUID id = UUID.randomUUID();
```

```
        UUID scoutId = UUID.randomUUID();
        Booking booking = Booking.builder().id(id).scoutId(scoutId).status(BookingStatus.AWAITING_PAYMENT)
                .paymentExpiresAt(OffsetDateTime.now().minusMinutes(1)).build();
        when(repository.findByIdForUpdate(id)).thenReturn(Optional.of(booking));
        assertThrows(ConflictException.class, () -> coordinator.claim(id, scoutId));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/PaymentCreationCoordinatorTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-166 — PaymentServiceLeadTimeExemptionTest#acceptedBookingCanCheckoutWithoutReapplyingLeadTime

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=PaymentServiceLeadTimeExemptionTest#acceptedBookingCanCheckoutWithoutReapplyingLeadTime`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        PaymentCreationCoordinator coordinator = mock(PaymentCreationCoordinator.class);
        MollieProviderClient mollie = mock(MollieProviderClient.class);
        UUID bookingId = UUID.randomUUID();
        UUID scoutId = UUID.randomUUID();
        UUID key = UUID.randomUUID();
        var claim = new PaymentCreationCoordinator.Claim(bookingId, new BigDecimal("100.00"), key, null);
        when(coordinator.claim(bookingId, scoutId)).thenReturn(claim);
        MolliePaymentResponse payment = new MolliePaymentResponse("tr_1", "open",
                new MolliePaymentResponse.Amount("100.00", "EUR"),
                new MolliePaymentResponse.MollieLinks(new MolliePaymentResponse.MollieLink("https://pay.example/1")),
                java.util.Map.of("bookingId", bookingId.toString()));
        CheckoutResponse checkout = new CheckoutResponse(bookingId, "https://pay.example/1", "tr_1",
                new BigDecimal("100.00"), 3600L);
        when(mollie.createPayment(bookingId, claim.amount(), claim.idempotencyKey())).thenReturn(payment);
        when(coordinator.complete(claim, payment)).thenReturn(checkout);

        assertThat(new PaymentService(coordinator, mollie).createPayment(bookingId, scoutId)).isSameAs(checkout);
        verify(mollie).createPayment(bookingId, claim.amount(), claim.idempotencyKey());
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/PaymentServiceLeadTimeExemptionTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-167 — PayoutDestinationCryptoTest#authenticatedEncryptionUsesRandomNonceAndRoundTrips

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=PayoutDestinationCryptoTest#authenticatedEncryptionUsesRandomNonceAndRoundTrips`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{var c=crypto();String
one=c.encrypt("NL91ABNA0417164300"),two=c.encrypt("NL91ABNA0417164300");assertThat(one).doesNotContain("NL91").isNotEqualTo(two);assertThat(c.decrypt(one))
.isEqualTo("NL91ABNA0417164300");assertThat(c.fingerprint("NL91ABNA0417164300")).isEqualTo(c.fingerprint("NL91ABNA0417164300"));}
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/PayoutDestinationCryptoTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-168 — PayoutDestinationCryptoTest#tamperingFailsAuthentication

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=PayoutDestinationCryptoTest#tamperingFailsAuthentication`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{var c=crypto();String value=c.encrypt("NL91ABNA0417164300");String[] parts=value.split(":",3);byte[]
ciphertext=Base64.getUrlDecoder().decode(parts[2]);ciphertext[0]^=1;String
tampered=parts[0]+":"+parts[1]+":"+Base64.getUrlEncoder().withoutPadding().encodeToString(ciphertext);assertThatThrownBy(()-
>c.decrypt(tampered)).isInstanceOf(IllegalStateException.class);}
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/PayoutDestinationCryptoTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-169 — PolicyAcceptanceServiceTest#publishingANewVersionMakesTheOldAcceptanceIncomplete

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=PolicyAcceptanceServiceTest#publishingANewVersionMakesTheOldAcceptanceIncomplete`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{List<PolicyVersion>
v1=allCurrent("1");when(versions.findByStatusOrderByPolicyTypeAsc(PolicyStatus.PUBLISHED)).thenReturn(v1);when(acceptances.findPolicyVersionIdByUserId(user
.getId())).thenReturn(ids(v1));assertTrue(service.status(user).allRequiredAccepted());List<PolicyVersion> v2=new ArrayList<>
(v1);v2.set(0,published(v1.get(0).getPolicyType(),"2"));when(versions.findByStatusOrderByPolicyTypeAsc(PolicyStatus.PUBLISHED)).thenReturn(v2);assertFalse(
service.status(user).allRequiredAccepted());assertEquals(1,service.status(user).missing().size());}
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/PolicyAcceptanceServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-170 — PolicyAcceptanceServiceTest#duplicateAcceptanceIsIdempotent

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=PolicyAcceptanceServiceTest#duplicateAcceptanceIsIdempotent`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{List<PolicyVersion> current=allCurrent("1");PolicyVersion
p=current.get(0);when(versions.findAllById(Set.of(p.getId()))).thenReturn(List.of(p));when(versions.findByStatusOrderByPolicyTypeAsc(PolicyStatus.PUBLISHED
)).thenReturn(current);when(acceptances.existsByUserIdAndPolicyVersionId(user.getId(),p.getId())).thenReturn(true);when(acceptances.findPolicyVersionIdByUs
erId(user.getId())).thenReturn(Set.of(p.getId()));service.accept(user,Set.of(p.getId()),"req",null);verify(acceptances,never()).saveAndFlush(any());}
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/PolicyAcceptanceServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-171 — PolicyAcceptanceServiceTest#snapshotReferencesExactFourCurrentVersions

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=PolicyAcceptanceServiceTest#snapshotReferencesExactFourCurrentVersions`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{List<PolicyVersion>
current=allCurrent("1");when(versions.findByStatusOrderByPolicyTypeAsc(PolicyStatus.PUBLISHED)).thenReturn(current);when(acceptances.findPolicyVersionIdByU
serId(user.getId())).thenReturn(ids(current));service.snapshotForBooking(UUID.randomUUID(),user);verify(snapshots,times(4)).save(any());}
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/PolicyAcceptanceServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-172 — RefundCoordinatorLatePaymentTest#duplicateClaimReusesOneProviderIdempotencyKeyAndOneNotification

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=RefundCoordinatorLatePaymentTest#duplicateClaimReusesOneProviderIdempotencyKeyAndOneNotification`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        var first = coordinator.claimLatePayment(bookingId).orElseThrow();
        var duplicate = coordinator.claimLatePayment(bookingId).orElseThrow();
```

```
        assertEquals(first.idempotencyKey(), duplicate.idempotencyKey());
        assertEquals(RefundStatus.CREATING, booking.getRefundStatus());
        verify(publisher, times(1)).publishEvent(new LatePaymentRefundInitiatedEvent(bookingId));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/RefundCoordinatorLatePaymentTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-173 — RefundCoordinatorLatePaymentTest#authoritativeRefundCompletionKeepsBookingExpired

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=RefundCoordinatorLatePaymentTest#authoritativeRefundCompletionKeepsBookingExpired`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        var claim = coordinator.claimLatePayment(bookingId).orElseThrow();
        coordinator.recordProviderState(claim, new MollieRefundResponse("re_1", "refunded", "tr_late",
```

```
                new MolliePaymentResponse.Amount("100.00", "EUR")));

        assertEquals(BookingStatus.EXPIRED, booking.getStatus());
        assertEquals(RefundStatus.REFUNDED, booking.getRefundStatus());
        assertNotNull(booking.getRefundedAt());
        assertEquals(new BigDecimal("0.00"), booking.getOwnerPayoutEur());
        assertTrue(coordinator.claimLatePayment(bookingId).isEmpty());
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/RefundCoordinatorLatePaymentTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-174 — RefundCoordinatorLatePaymentTest#cancellationRefundCompletionIsIdempotentAndKeepsCancelledStatus

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=RefundCoordinatorLatePaymentTest#cancellationRefundCompletionIsIdempotentAndKeepsCancelledStatus`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        booking.setStatus(BookingStatus.CANCELLED);
        booking.setCancellationPolicyTier(CancellationPolicyTier.PARTIAL_REFUND);
        booking.setRefundStatus(RefundStatus.CREATING);
        booking.setRefundAmountEur(new BigDecimal("50.00"));
        booking.setRefundIdempotencyKey(UUID.randomUUID());
        var claim = new RefundCoordinator.Claim(bookingId, "tr_late", new BigDecimal("50.00"),
                booking.getRefundIdempotencyKey());
        var response = new MollieRefundResponse("re_partial", "refunded", "tr_late",
                new MolliePaymentResponse.Amount("50.00", "EUR"));

        coordinator.recordProviderState(claim, response);
        coordinator.recordProviderState(claim, response);

        assertEquals(BookingStatus.CANCELLED, booking.getStatus());
        assertEquals(RefundStatus.REFUNDED, booking.getRefundStatus());
        verify(publisher, times(1)).publishEvent(new CancellationRefundCompletedEvent(bookingId));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/RefundCoordinatorLatePaymentTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-175 — RefundServiceTest#onlyAdminMayRefund

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=RefundServiceTest#onlyAdminMayRefund`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        when(current.getCurrentUser()).thenReturn(User.builder().id(UUID.randomUUID()).role(UserRole.OWNER).build());
        assertThrows(ForbiddenException.class, () -> service.requestFullRefund(UUID.randomUUID()));
        verifyNoInteractions(coordinator, mollie);
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/RefundServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-176 — RefundServiceTest#definitiveProviderRejectionIsRecordedAndSanitized

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=RefundServiceTest#definitiveProviderRejectionIsRecordedAndSanitized`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        UUID bookingId = UUID.randomUUID();
        User admin = User.builder().id(UUID.randomUUID()).email("admin@example.com").role(UserRole.ADMIN).build();
        RefundCoordinator.Claim claim = new RefundCoordinator.Claim(bookingId, "tr_1", new BigDecimal("100.00"), UUID.randomUUID());
        when(current.getCurrentUser()).thenReturn(admin);
        when(coordinator.claim(eq(bookingId), eq(admin.getId()), anyString())).thenReturn(claim);
        when(mollie.createRefund(anyString(), eq(bookingId), any(), any()))
                .thenThrow(HttpClientErrorException.create(org.springframework.http.HttpStatus.UNPROCESSABLE_ENTITY, "", null, null, null));
        assertThrows(BadRequestException.class, () -> service.requestFullRefund(bookingId));
        verify(coordinator).markRejected(bookingId);
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/RefundServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-177 — RefundServiceTest#latePaymentProviderFailureRetainsCoordinatorStateForSafeRetry

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=RefundServiceTest#latePaymentProviderFailureRetainsCoordinatorStateForSafeRetry`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        UUID bookingId = UUID.randomUUID();
        RefundCoordinator.Claim claim = new RefundCoordinator.Claim(
                bookingId, "tr_late", new BigDecimal("100.00"), UUID.randomUUID());
        when(coordinator.claimLatePayment(bookingId)).thenReturn(java.util.Optional.of(claim));
        when(mollie.createRefund(anyString(), eq(bookingId), any(), eq(claim.idempotencyKey())))
                .thenThrow(new org.springframework.web.client.ResourceAccessException("timeout"));

        assertThrows(BadRequestException.class, () -> service.requestLatePaymentRefund(bookingId));
        verify(coordinator, never()).markRejected(bookingId);
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/RefundServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-178 — RefundServiceTest#cancellationRefundUsesExactClaimedPartialAmount

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=RefundServiceTest#cancellationRefundUsesExactClaimedPartialAmount`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        UUID bookingId = UUID.randomUUID();
        RefundCoordinator.Claim claim = new RefundCoordinator.Claim(
                bookingId, "tr_partial", new BigDecimal("500.00"), UUID.randomUUID());
        MollieRefundResponse response = new MollieRefundResponse(
                "re_1", "queued", "tr_partial",
                new MolliePaymentResponse.Amount("500.00", "EUR"));
        when(mollie.createRefund("tr_partial", bookingId, new BigDecimal("500.00"), claim.idempotencyKey()))
                .thenReturn(response);

        service.requestCancellationRefund(claim);

        verify(coordinator).recordProviderState(claim, response);
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/RefundServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-179 — RefundServiceTest#duplicateCancellationClaimDoesNotCreateAnotherProviderRefund

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=RefundServiceTest#duplicateCancellationClaimDoesNotCreateAnotherProviderRefund`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        UUID bookingId = UUID.randomUUID();
        User scout = User.builder().id(UUID.randomUUID()).role(UserRole.SCOUT).build();
        when(current.getCurrentUser()).thenReturn(scout);
        assertThrows(ForbiddenException.class, () -> service.requestFullRefund(bookingId));
        verifyNoInteractions(mollie);
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/RefundServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-180 — S3StorageTest#publicUploadUsesConfiguredPublicBucketAndGeneratedKey

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=S3StorageTest#publicUploadUsesConfiguredPublicBucketAndGeneratedKey`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        S3PublicMediaStorage storage = new S3PublicMediaStorage(client, properties(), "venue-media", "media/");
        String reference = storage.store(new MockMultipartFile("file", "../../photo.jpg", "image/jpeg", new byte[]{1, 2}));
        assertThat(reference).matches("/api/upload/files/[0-9a-f-]{36}\\.jpg");
        verify(client).putObject(org.mockito.ArgumentMatchers.argThat((PutObjectRequest request) ->
                request.bucket().equals("venue-media") && request.key().matches("media/[0-9a-f-]{36}\\.jpg")),
any(software.amazon.awssdk.core.sync.RequestBody.class));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/S3StorageTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-181 — S3StorageTest#rejectsOversizedAndUnsupportedImages

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=S3StorageTest#rejectsOversizedAndUnsupportedImages`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        S3PublicMediaStorage storage = new S3PublicMediaStorage(client, properties(), "venue-media", "media/");
        assertThatThrownBy(() -> storage.store(new MockMultipartFile("file", "x.gif", "image/gif", new byte[]{1})))
                .isInstanceOf(IllegalArgumentException.class);
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/S3StorageTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-182 — S3StorageTest#privateStorageRejectsTraversal

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=S3StorageTest#privateStorageRejectsTraversal`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        S3PrivateDocumentStorage storage = new S3PrivateDocumentStorage(client, "documents", "private/");
        assertThatThrownBy(() -> storage.store("../receipt.pdf", new byte[]{1})).isInstanceOf(IllegalArgumentException.class);
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/S3StorageTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-183 — TrustNotificationServiceTest#sendsVerificationApprovalAndSafeRejectionReason

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=TrustNotificationServiceTest#sendsVerificationApprovalAndSafeRejectionReason`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{OwnerVerification
v=OwnerVerification.builder().id(UUID.randomUUID()).owner(owner).build();when(verifications.findById(v.getId())).thenReturn(Optional.of(v));
        service.send(new TrustNotificationEvent(TrustEventType.OWNER_VERIFICATION_APPROVED,v.getId(),null));
        service.send(new TrustNotificationEvent(TrustEventType.OWNER_VERIFICATION_REJECTED,v.getId(),"Need venue authority evidence"));
        verify(email).send(eq(owner.getEmail()),contains("approved"),anyString(),anyString());
        verify(email).send(eq(owner.getEmail()),contains("attention"),contains("Need venue authority evidence"),contains("Need venue authority
evidence"));}
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/TrustNotificationServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-184 — TrustNotificationServiceTest#sendsApprovalChangesAndSuspensionMessages

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=TrustNotificationServiceTest#sendsApprovalChangesAndSuspensionMessages`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{Venue v=Venue.builder().id(UUID.randomUUID()).owner(owner).name("Canal Studio").build();when(venues.findById(v.getId())).thenReturn(Optional.of(v));
        service.send(new TrustNotificationEvent(TrustEventType.VENUE_APPROVED,v.getId(),null));
        service.send(new TrustNotificationEvent(TrustEventType.VENUE_CHANGES_REQUESTED,v.getId(),"Add safety rules"));
        service.send(new TrustNotificationEvent(TrustEventType.VENUE_SUSPENDED,v.getId(),"Safety review"));
        verify(email).send(eq(owner.getEmail()),eq("Your venue is live"),anyString(),anyString());
        verify(email).send(eq(owner.getEmail()),contains("Changes requested"),contains("Add safety rules"),contains("Add safety rules"));
        verify(email).send(eq(owner.getEmail()),contains("suspended"),contains("Safety review"),contains("Safety review"));}
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/TrustNotificationServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-185 — VenueAvailabilityServiceTest#ownerMayManageOwnVenueAndTimezoneIsValidated

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=VenueAvailabilityServiceTest#ownerMayManageOwnVenueAndTimezoneIsValidated`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        when(currentUsers.getCurrentUser()).thenReturn(User.builder().id(ownerId).role(UserRole.OWNER).build());
        assertEquals("Europe/Paris", service.updateSettings(venueId,
                new UpdateVenueAvailabilitySettingsRequest("Europe/Paris")).timezone());
        assertThrows(BadRequestException.class, () -> service.updateSettings(venueId,
                new UpdateVenueAvailabilitySettingsRequest("UTC+Amsterdam")));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/VenueAvailabilityServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-186 — VenueAvailabilityServiceTest#ownerCannotManageAnotherOwnersVenue

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=VenueAvailabilityServiceTest#ownerCannotManageAnotherOwnersVenue`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        when(currentUsers.getCurrentUser()).thenReturn(User.builder().id(UUID.randomUUID()).role(UserRole.OWNER).build());
        assertThrows(ForbiddenException.class, () -> service.updateSettings(venueId,
                new UpdateVenueAvailabilitySettingsRequest("Europe/Amsterdam")));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/VenueAvailabilityServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-187 — VenueAvailabilityServiceTest#scoutCannotMutateAvailability

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=VenueAvailabilityServiceTest#scoutCannotMutateAvailability`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        when(currentUsers.getCurrentUser()).thenReturn(User.builder().id(UUID.randomUUID()).role(UserRole.SCOUT).build());
        assertThrows(ForbiddenException.class, () -> service.replaceWeekly(venueId,
                new UpdateWeeklyAvailabilityRequest(List.of())));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/VenueAvailabilityServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-188 — VenueAvailabilityServiceTest#rejectsInvalidAndOverlappingWeeklyIntervals

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=VenueAvailabilityServiceTest#rejectsInvalidAndOverlappingWeeklyIntervals`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        when(currentUsers.getCurrentUser()).thenReturn(User.builder().id(ownerId).role(UserRole.OWNER).build());
        assertThrows(BadRequestException.class, () -> service.replaceWeekly(venueId,
                weekly(new WeeklyIntervalRequest(DayOfWeek.MONDAY, time(17), time(9)))));
        assertThrows(BadRequestException.class, () -> service.replaceWeekly(venueId,
                weekly(new WeeklyIntervalRequest(DayOfWeek.MONDAY, time(8), time(12)),
                        new WeeklyIntervalRequest(DayOfWeek.MONDAY, time(11), time(15)))));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/VenueAvailabilityServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-189 — VenueAvailabilityServiceTest#calculatesWeeklyAndEmptyDay

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=VenueAvailabilityServiceTest#calculatesWeeklyAndEmptyDay`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        LocalDate monday = LocalDate.of(2026, 8, 24);
        when(weekly.findByVenueIdAndDayOfWeekOrderByStartTime(venueId, DayOfWeek.MONDAY))
                .thenReturn(List.of(hours(DayOfWeek.MONDAY, 9, 17)));
        when(overrides.findByVenueIdAndDateOrderByStartTime(venueId, monday)).thenReturn(List.of());
        assertEquals(List.of(interval(9, 17)), service.getAvailableIntervals(venueId, monday));

        LocalDate tuesday = monday.plusDays(1);
        when(weekly.findByVenueIdAndDayOfWeekOrderByStartTime(venueId, DayOfWeek.TUESDAY)).thenReturn(List.of());
        when(overrides.findByVenueIdAndDateOrderByStartTime(venueId, tuesday)).thenReturn(List.of());
        assertTrue(service.getAvailableIntervals(venueId, tuesday).isEmpty());
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/VenueAvailabilityServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-190 — VenueAvailabilityServiceTest#blockedOverrideSubtractsPartialRange

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=VenueAvailabilityServiceTest#blockedOverrideSubtractsPartialRange`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        LocalDate date = LocalDate.of(2026, 8, 25);
        when(weekly.findByVenueIdAndDayOfWeekOrderByStartTime(venueId, date.getDayOfWeek()))
                .thenReturn(List.of(hours(date.getDayOfWeek(), 9, 17)));
        when(overrides.findByVenueIdAndDateOrderByStartTime(venueId, date))
                .thenReturn(List.of(override(date, AvailabilityOverrideType.BLOCKED, 12, 14)));
        assertEquals(List.of(interval(9, 12), interval(14, 17)), service.getAvailableIntervals(venueId, date));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/VenueAvailabilityServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-191 — VenueAvailabilityServiceTest#availableOverrideAddsAndNormalizesOutsideWeeklyHours

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=VenueAvailabilityServiceTest#availableOverrideAddsAndNormalizesOutsideWeeklyHours`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        LocalDate date = LocalDate.of(2026, 8, 27);
        when(weekly.findByVenueIdAndDayOfWeekOrderByStartTime(venueId, date.getDayOfWeek()))
                .thenReturn(List.of(hours(date.getDayOfWeek(), 9, 17)));
        when(overrides.findByVenueIdAndDateOrderByStartTime(venueId, date))
                .thenReturn(List.of(override(date, AvailabilityOverrideType.AVAILABLE, 7, 23)));
        assertEquals(List.of(interval(7, 23)), service.getAvailableIntervals(venueId, date));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/VenueAvailabilityServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-192 — VenueAvailabilityServiceTest#leadTimeRemovesClipsAndPreservesIntervals

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=VenueAvailabilityServiceTest#leadTimeRemovesClipsAndPreservesIntervals`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        venue.setMinimumLeadTimeMinutes(1440);
        service = new VenueAvailabilityService(venues, weekly, overrides, currentUsers, venueService, bookings,
                new VenueBookingLeadTimePolicy(Clock.fixed(Instant.parse("2026-08-27T08:00:00Z"), ZoneOffset.UTC)));
        LocalDate before = LocalDate.of(2026, 8, 27);
        LocalDate boundary = LocalDate.of(2026, 8, 28);
        LocalDate after = LocalDate.of(2026, 8, 29);
        for (LocalDate date : List.of(before, boundary, after)) {
            when(weekly.findByVenueIdAndDayOfWeekOrderByStartTime(venueId, date.getDayOfWeek()))
                    .thenReturn(List.of(hours(date.getDayOfWeek(), 9, 18)));
            when(overrides.findByVenueIdAndDateOrderByStartTime(venueId, date)).thenReturn(List.of());
        }
        assertTrue(service.getAvailableIntervals(venueId, before).isEmpty());
        assertEquals(List.of(interval(10, 18)), service.getAvailableIntervals(venueId, boundary));
        assertEquals(List.of(interval(9, 18)), service.getAvailableIntervals(venueId, after));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/VenueAvailabilityServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-193 — VenueAvailabilityServiceTest#blocksPartOfSpecialAvailabilityWhenNoWeeklyHoursExist

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=VenueAvailabilityServiceTest#blocksPartOfSpecialAvailabilityWhenNoWeeklyHoursExist`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        LocalDate date = LocalDate.of(2026, 8, 30);
        when(weekly.findByVenueIdAndDayOfWeekOrderByStartTime(venueId, date.getDayOfWeek())).thenReturn(List.of());
        when(overrides.findByVenueIdAndDateOrderByStartTime(venueId, date)).thenReturn(List.of(
                override(date, AvailabilityOverrideType.AVAILABLE, 8, 20),
                override(date, AvailabilityOverrideType.BLOCKED, 10, 11)));
        assertEquals(List.of(interval(8, 10), interval(11, 20)), service.getAvailableIntervals(venueId, date));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/VenueAvailabilityServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-194 — VenueAvailabilityServiceTest#confirmedBookingSplitsAvailabilityAndCancelledBookingDoesNotBlock

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=VenueAvailabilityServiceTest#confirmedBookingSplitsAvailabilityAndCancelledBookingDoesNotBlock`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        LocalDate date = LocalDate.of(2026, 9, 7);
        when(weekly.findByVenueIdAndDayOfWeekOrderByStartTime(venueId, date.getDayOfWeek()))
                .thenReturn(List.of(hours(date.getDayOfWeek(), 8, 20)));
        when(overrides.findByVenueIdAndDateOrderByStartTime(venueId, date)).thenReturn(List.of());
        Booking confirmed = Booking.builder().status(com.eema.backend.entity.enums.BookingStatus.CONFIRMED)
                .startTime(time(12)).endTime(time(15)).build();
        Booking cancelled = Booking.builder().status(com.eema.backend.entity.enums.BookingStatus.CANCELLED)
                .startTime(time(9)).endTime(time(10)).build();
        when(bookings.findBlockingBookings(venueId, date)).thenReturn(List.of(cancelled, confirmed));
```

```
        assertEquals(List.of(interval(8, 12), interval(15, 20)), service.getAvailableIntervals(venueId, date));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/VenueAvailabilityServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-195 — VenueAvailabilityServiceTest#onlyReservationStatusesBlockEffectiveAvailability

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=VenueAvailabilityServiceTest#onlyReservationStatusesBlockEffectiveAvailability`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        LocalDate date = LocalDate.of(2026, 9, 8);
        when(weekly.findByVenueIdAndDayOfWeekOrderByStartTime(venueId, date.getDayOfWeek()))
                .thenReturn(List.of(hours(date.getDayOfWeek(), 8, 20)));
        when(overrides.findByVenueIdAndDateOrderByStartTime(venueId, date)).thenReturn(List.of());
```

```
        for (var status : List.of(com.eema.backend.entity.enums.BookingStatus.REQUESTED,
                com.eema.backend.entity.enums.BookingStatus.DECLINED,
                com.eema.backend.entity.enums.BookingStatus.EXPIRED,
                com.eema.backend.entity.enums.BookingStatus.UNAVAILABLE)) {
            when(bookings.findBlockingBookings(venueId, date)).thenReturn(List.of(
                    Booking.builder().status(status).startTime(time(10)).endTime(time(12)).build()));
            assertEquals(List.of(interval(8, 20)), service.getAvailableIntervals(venueId, date), status.name());
        }
        for (var status : List.of(com.eema.backend.entity.enums.BookingStatus.AWAITING_PAYMENT,
                com.eema.backend.entity.enums.BookingStatus.CONFIRMED)) {
            when(bookings.findBlockingBookings(venueId, date)).thenReturn(List.of(
                    Booking.builder().status(status).startTime(time(10)).endTime(time(12)).build()));
            assertEquals(List.of(interval(8, 10), interval(12, 20)), service.getAvailableIntervals(venueId, date), status.name());
        }
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/VenueAvailabilityServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-196 — VenueAvailabilityServiceTest#rejectsExcessiveDateRange

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=VenueAvailabilityServiceTest#rejectsExcessiveDateRange`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        LocalDate from = LocalDate.of(2026, 1, 1);
        assertThrows(BadRequestException.class, () -> service.getAvailability(venueId, from, from.plusDays(93)));
        verifyNoInteractions(venueService);
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/VenueAvailabilityServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-197 — VenueBookingLeadTimePolicyTest#allowsTwentyFiveHoursAndExactBoundary

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=VenueBookingLeadTimePolicyTest#allowsTwentyFiveHoursAndExactBoundary`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test

and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        assertThat(policy.isSatisfied(venue, LocalDate.of(2026, 8, 28), LocalTime.of(11, 0))).isTrue();
        assertThat(policy.isSatisfied(venue, LocalDate.of(2026, 8, 28), LocalTime.of(10, 0))).isTrue();
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/VenueBookingLeadTimePolicyTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-198 — VenueBookingLeadTimePolicyTest#rejectsOneMinuteInsideBoundary

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=VenueBookingLeadTimePolicyTest#rejectsOneMinuteInsideBoundary`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        assertThat(policy.isSatisfied(venue, LocalDate.of(2026, 8, 28), LocalTime.of(9, 59))).isFalse();
        assertThatThrownBy(() -> policy.validate(venue, LocalDate.of(2026, 8, 28), LocalTime.of(9, 59)))
                .isInstanceOf(VenueBookingLeadTimePolicy.MinimumLeadTimeNotMetException.class)
                .hasMessageContaining("24 hours");
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/VenueBookingLeadTimePolicyTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-199 — VenueBookingLeadTimePolicyTest#usesElapsedTimeAcrossAmsterdamDstChange

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=VenueBookingLeadTimePolicyTest#usesElapsedTimeAcrossAmsterdamDstChange`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        Instant beforeSpringChange = Instant.parse("2026-03-28T09:00:00Z"); // 10:00 CET
        VenueBookingLeadTimePolicy dstPolicy = new VenueBookingLeadTimePolicy(Clock.fixed(beforeSpringChange, ZoneOffset.UTC));
        assertThat(dstPolicy.isSatisfied(venue, LocalDate.of(2026, 3, 29), LocalTime.of(10, 0))).isFalse();
        assertThat(dstPolicy.isSatisfied(venue, LocalDate.of(2026, 3, 29), LocalTime.of(11, 0))).isTrue();
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/VenueBookingLeadTimePolicyTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-200 — VenueHardRequirementEvaluatorTest#crewCapacityMustBeKnownAndMeetTheRequirement

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=VenueHardRequirementEvaluatorTest#crewCapacityMustBeKnownAndMeetTheRequirement`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        ParsedBrief brief = fromJson("{\"numPeople\":20}");

        assertThat(evaluator.isEligible(venueWithCrew(20), brief)).isTrue();
        assertThat(evaluator.isEligible(venueWithCrew(19), brief)).isFalse();
        assertThat(evaluator.isEligible(venueWithCrew(null), brief)).isFalse();
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/VenueHardRequirementEvaluatorTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-201 — VenueHardRequirementEvaluatorTest#explicitActivityMustBePermittedAndNotProhibited

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=VenueHardRequirementEvaluatorTest#explicitActivityMustBePermittedAndNotProhibited`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        ParsedBrief brief = fromJson("{\"productionActivity\":\"INTERVIEW\"}");

        Venue permitted = venue();
        when(permitted.getPermittedActivities()).thenReturn(List.of(ProductionActivity.INTERVIEW));
        assertThat(evaluator.isEligible(permitted, brief)).isTrue();

        Venue prohibited = venue();
        when(prohibited.getPermittedActivities()).thenReturn(List.of(ProductionActivity.INTERVIEW));
        when(prohibited.getProhibitedActivities()).thenReturn(List.of(ProductionActivity.INTERVIEW));
        assertThat(evaluator.isEligible(prohibited, brief)).isFalse();

        assertThat(evaluator.isEligible(venue(), brief)).isFalse();

        Venue omittedFromKnownPermissions = venue();
        when(omittedFromKnownPermissions.getPermittedActivities()).thenReturn(List.of(ProductionActivity.PHOTO));
        assertThat(evaluator.isEligible(omittedFromKnownPermissions, brief)).isFalse();
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/VenueHardRequirementEvaluatorTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-202 — VenueHardRequirementEvaluatorTest#requiredBooleanFactsFailClosedWhenFalseOrUnknown

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=VenueHardRequirementEvaluatorTest#requiredBooleanFactsFailClosedWhenFalseOrUnknown`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        assertRequiredBoolean("natural light");
        assertRequiredBoolean("loading access");
        assertRequiredBoolean("sound suitability");
        assertRequiredBoolean("wheelchair access");
        assertRequiredBoolean("lift");
        assertRequiredBoolean("bathrooms");
        assertRequiredBoolean("blackout");
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/VenueHardRequirementEvaluatorTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-203 — VenueHardRequirementEvaluatorTest#minimumCeilingHeightMustBeKnownAndHighEnough

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=VenueHardRequirementEvaluatorTest#minimumCeilingHeightMustBeKnownAndHighEnough`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        ParsedBrief brief = fromJson("{\"minimumCeilingHeightM\":4.50}");

        assertThat(evaluator.isEligible(venueWithCeiling("4.50"), brief)).isTrue();
        assertThat(evaluator.isEligible(venueWithCeiling("4.49"), brief)).isFalse();
        assertThat(evaluator.isEligible(venueWithCeiling(null), brief)).isFalse();
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/VenueHardRequirementEvaluatorTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-204 — VenueHardRequirementEvaluatorTest#parkingRequirementAndMinimumSpacesFailClosed

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=VenueHardRequirementEvaluatorTest#parkingRequirementAndMinimumSpacesFailClosed`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        ParsedBrief brief = fromJson("{\"requiresParking\":true}");
        assertThat(evaluator.isEligible(venueWithParking(true, null), brief)).isTrue();
        assertThat(evaluator.isEligible(venueWithParking(false, null), brief)).isFalse();
        assertThat(evaluator.isEligible(venueWithParking(null, null), brief)).isFalse();

        ParsedBrief spacesBrief = fromJson("{\"minimumParkingSpaces\":3}");
        assertThat(evaluator.isEligible(venueWithParking(true, 3), spacesBrief)).isTrue();
        assertThat(evaluator.isEligible(venueWithParking(true, 2), spacesBrief)).isFalse();
        assertThat(evaluator.isEligible(venueWithParking(true, null), spacesBrief)).isFalse();
        assertThat(evaluator.isEligible(venueWithParking(null, 3), spacesBrief)).isFalse();
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/VenueHardRequirementEvaluatorTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-205 — VenueHardRequirementEvaluatorTest#preferencesNeverRemoveCandidatesEvenWhenFactsAreUnknown

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=VenueHardRequirementEvaluatorTest#preferencesNeverRemoveCandidatesEvenWhenFactsAreUnknown`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        ParsedBrief brief = fromJson("""
                {"prefersNaturalLight":true,"prefersParking":true,
                 "prefersLift":true,"prefersBlackout":true}
                """);

        assertThat(evaluator.isEligible(venue(), brief)).isTrue();
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/VenueHardRequirementEvaluatorTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-206 — VenueMatchAvailabilityFilterTest#completeIntervalInsideAvailabilityPasses

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=VenueMatchAvailabilityFilterTest#completeIntervalInsideAvailabilityPasses`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        Venue venue = venue();
        when(availabilityService.getAvailableIntervals(venue.getId(), date))
                .thenReturn(List.of(interval(9, 13)));

        assertThat(filter.retainAvailable(List.of(venue), exact(10, 12)))
                .containsExactly(venue);
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/VenueMatchAvailabilityFilterTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-207 — VenueMatchAvailabilityFilterTest#partialOverlapAndFullyUnavailableDateAreRejected

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=VenueMatchAvailabilityFilterTest#partialOverlapAndFullyUnavailableDateAreRejected`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        Venue partial = venue();
        Venue closed = venue();
        when(availabilityService.getAvailableIntervals(partial.getId(), date))
                .thenReturn(List.of(interval(9, 13)));
        when(availabilityService.getAvailableIntervals(closed.getId(), date))
                .thenReturn(List.of());

        assertThat(filter.retainAvailable(List.of(partial, closed), exact(12, 14))).isEmpty();
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/VenueMatchAvailabilityFilterTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-208 — VenueMatchAvailabilityFilterTest#noCompleteTimingPreservesCandidatesAndDoesNotQueryAvailability

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=VenueMatchAvailabilityFilterTest#noCompleteTimingPreservesCandidatesAndDoesNotQueryAvailability`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        Venue first = venue();
        Venue second = venue();
        ParsedBrief ambiguous = ParsedBriefFixtures.fromJson(
                "{\"timingDatePreferences\":\"Friday afternoon\"}"
        );

        assertThat(filter.retainAvailable(List.of(first, second), ambiguous))
                .containsExactly(first, second);
        verifyNoInteractions(availabilityService);
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/VenueMatchAvailabilityFilterTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-209 — VenueModerationServiceTest#verifiedOwnerSubmitsCompleteVenueButOtherOwnerCannot

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=VenueModerationServiceTest#verifiedOwnerSubmitsCompleteVenueButOtherOwnerCannot`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{when(current.getCurrentUser()).thenReturn(owner);assertThat(service.submit(venue.getId()).status()).isEqualTo(VenueStatus.PENDING);

venue.setStatus(VenueStatus.DRAFT);when(current.getCurrentUser()).thenReturn(User.builder().id(UUID.randomUUID()).role(UserRole.OWNER).emailVerified(true).
build());
        assertThatThrownBy(()->service.submit(venue.getId())).isInstanceOf(ForbiddenException.class);}
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/VenueModerationServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-210 — VenueModerationServiceTest#unverifiedOrIncompleteOwnerCannotSubmit

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=VenueModerationServiceTest#unverifiedOrIncompleteOwnerCannotSubmit`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{when(current.getCurrentUser()).thenReturn(owner);when(verification.isVerified(owner.getId())).thenReturn(false);
        assertThatThrownBy(()->service.submit(venue.getId())).isInstanceOf(ForbiddenException.class);
        when(verification.isVerified(owner.getId())).thenReturn(true);venue.setPhotoPaths(List.of());
        assertThatThrownBy(()->service.submit(venue.getId())).isInstanceOf(ConflictException.class).hasMessageContaining("photoPaths");}
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/VenueModerationServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-211 — VenueModerationServiceTest#adminApprovesRequestsChangesSuspendsAndReactivatesWithoutTouchingBookings

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=VenueModerationServiceTest#adminApprovesRequestsChangesSuspendsAndReactivatesWithoutTouchingBookings`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{when(current.getCurrentUser()).thenReturn(admin);venue.setStatus(VenueStatus.PENDING);
        assertThat(service.approve(venue.getId(),null).status()).isEqualTo(VenueStatus.ACTIVE);
        Booking existing=Booking.builder().id(UUID.randomUUID()).venueId(venue.getId()).status(BookingStatus.CONFIRMED).build();
        assertThat(service.suspend(venue.getId(),"Safety
review").status()).isEqualTo(VenueStatus.SUSPENDED);assertThat(existing.getStatus()).isEqualTo(BookingStatus.CONFIRMED);
        assertThat(service.reactivate(venue.getId(),null).status()).isEqualTo(VenueStatus.ACTIVE);
        venue.setStatus(VenueStatus.PENDING);assertThat(service.requestChanges(venue.getId(),"Add clearer rules").status()).isEqualTo(VenueStatus.DRAFT);}
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/VenueModerationServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-212 — VenueModerationServiceTest#approvalRechecksReadiness

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=VenueModerationServiceTest#approvalRechecksReadiness`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{when(current.getCurrentUser()).thenReturn(admin);venue.setStatus(VenueStatus.PENDING);venue.setProductionRules(null);
        assertThatThrownBy(()->service.approve(venue.getId(),null)).isInstanceOf(ConflictException.class).hasMessageContaining("productionRules");}
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/VenueModerationServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-213 — VenueProductionProfileValidatorTest#completeProfileIsReady

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=VenueProductionProfileValidatorTest#completeProfileIsReady`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        Venue venue = Venue.builder()
                .maxCrewSize(20).permittedActivities(List.of(ProductionActivity.PHOTO))
                .naturalLight(false).loadingAccess(false).bathroomsAvailable(true)
                .soundRecordingSuitable(true).productionRules("Keep common areas clear")
                .hourlyRateEur(new BigDecimal("100.00")).minimumBookingMinutes(180).minimumLeadTimeMinutes(1440)
                .timezone("Europe/Amsterdam").photoPaths(List.of("/api/upload/files/photo.jpg"))
                .build();

        assertTrue(validator.readinessIssues(venue).isEmpty());
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/VenueProductionProfileValidatorTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-214 — VenueProductionProfileValidatorTest#incompleteProfileReportsUnknownFieldsWithoutTreatingThemAsFalse

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=VenueProductionProfileValidatorTest#incompleteProfileReportsUnknownFieldsWithoutTreatingThemAsFalse`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        Venue venue = Venue.builder().hourlyRateEur(BigDecimal.TEN).minimumBookingMinutes(60)
                .timezone("Europe/Amsterdam").photoPaths(List.of()).build();

        List<String> issues = validator.readinessIssues(venue);
        assertTrue(issues.containsAll(List.of("maxCrewSize", "permittedActivities", "productionRules", "minimumLeadTimeMinutes",
                "naturalLight", "loadingAccess", "bathroomsAvailable", "soundRecordingSuitable", "photoPaths")));
        assertNull(venue.getNaturalLight());
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/VenueProductionProfileValidatorTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-215 — VenueProductionProfileValidatorTest#contradictoryActivitiesAndParkingAreRejected

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=VenueProductionProfileValidatorTest#contradictoryActivitiesAndParkingAreRejected`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        Venue activities = Venue.builder().permittedActivities(List.of(ProductionActivity.FILM))
                .prohibitedActivities(List.of(ProductionActivity.FILM)).build();
        assertThrows(BadRequestException.class, () -> validator.validateConsistency(activities));

        Venue parking = Venue.builder().parkingAvailable(false).parkingSpaces(2).build();
        assertThrows(BadRequestException.class, () -> validator.validateConsistency(parking));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/VenueProductionProfileValidatorTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-216 — VenueServiceTest#ownerUpdatesMetadataAndPersistsMediaOrderAndRemoval

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=VenueServiceTest#ownerUpdatesMetadataAndPersistsMediaOrderAndRemoval`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        when(currentUser.getCurrentUser()).thenReturn(owner);
        String primary = photo("33333333-3333-4333-8333-333333333333");
        String secondary = photo("11111111-1111-4111-8111-111111111111");

        var result = service.updateVenue(venueId, request(List.of(primary, secondary)));

        assertEquals("Updated Studio", result.name());
        assertEquals("Centre", result.neighbourhood());
        assertEquals(250, result.sizeM2());
        assertEquals(new BigDecimal("175.00"), result.hourlyRateEur());
        assertEquals(240, result.minimumBookingMinutes());
        assertEquals(new BigDecimal("1200.00"), result.fullDayRateEur());
        assertEquals(List.of(primary, secondary), result.photoPaths());
        assertFalse(result.photoPaths().contains(photo("22222222-2222-4222-8222-222222222222")));
        assertEquals(primary, result.photoPaths().getFirst());
        assertEquals("Europe/Paris", result.timezone());
        assertEquals(30, result.maxCrewSize());
        assertEquals(List.of(ProductionActivity.PHOTO, ProductionActivity.FILM), result.permittedActivities());
```

```
        assertEquals(new BigDecimal("4.50"), result.ceilingHeightM());
        assertTrue(result.loadingAccess());
        assertEquals(3, result.parkingSpaces());
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/VenueServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-217 — VenueServiceTest#anotherOwnerAndScoutCannotUpdateVenue

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=VenueServiceTest#anotherOwnerAndScoutCannotUpdateVenue`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        when(currentUser.getCurrentUser()).thenReturn(
                User.builder().id(UUID.randomUUID()).role(UserRole.OWNER).build());
```

```
        assertThrows(ForbiddenException.class, () -> service.updateVenue(venueId, request(List.of())));

        when(currentUser.getCurrentUser()).thenReturn(
                User.builder().id(UUID.randomUUID()).role(UserRole.SCOUT).build());
        assertThrows(ForbiddenException.class, () -> service.updateVenue(venueId, request(List.of())));
        verify(venues, never()).save(any());
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/VenueServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-218 — VenueServiceTest#ownerMayRemoveOptionalFullDayRate

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=VenueServiceTest#ownerMayRemoveOptionalFullDayRate`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        when(currentUser.getCurrentUser()).thenReturn(owner);
        UpdateVenueRequest withoutCap = requestWithPricing(new BigDecimal("175.00"), 240, null,
                30, new BigDecimal("4.50"), 3);

        var result = service.updateVenue(venueId, withoutCap);

        assertNull(result.fullDayRateEur());
        assertNull(result.naturalLight());
        assertNull(result.permittedActivities());
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/VenueServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-219 — VenueServiceTest#venuePricingRequestValidationRejectsInvalidRequiredValuesAndAcceptsNullCap

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=VenueServiceTest#venuePricingRequestValidationRejectsInvalidRequiredValuesAndAcceptsNullCap`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        try (var factory = jakarta.validation.Validation.buildDefaultValidatorFactory()) {
            var validator = factory.getValidator();
            UpdateVenueRequest valid = requestWithPricing(new BigDecimal("175.00"), 240, null,
                    null, null, null);
            assertTrue(validator.validate(valid).isEmpty());

            UpdateVenueRequest invalid = requestWithPricing(BigDecimal.ZERO, 0, BigDecimal.ZERO,
                    0, BigDecimal.ZERO, -1);
            assertEquals(6, validator.validate(invalid).stream()
                    .filter(v -> java.util.Set.of("hourlyRateEur", "minimumBookingMinutes", "fullDayRateEur",
                                    "maxCrewSize", "ceilingHeightM", "parkingSpaces")
                            .contains(v.getPropertyPath().toString())).count());
        }
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/VenueServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-220 — VenueServiceTest#invalidOrDuplicateStorageReferencesAreRejected

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=VenueServiceTest#invalidOrDuplicateStorageReferencesAreRejected`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        when(currentUser.getCurrentUser()).thenReturn(owner);
        assertThrows(BadRequestException.class,
                () -> service.updateVenue(venueId, request(List.of("/etc/passwd"))));
        String photo = photo("11111111-1111-4111-8111-111111111111");
        assertThrows(BadRequestException.class,
                () -> service.updateVenue(venueId, request(List.of(photo, photo))));
        verify(venues, never()).save(any());
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/VenueServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-221 — VenueServiceTest#scopedUploadIsAttachedAndCleanupRunsIfPersistenceFails

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=VenueServiceTest#scopedUploadIsAttachedAndCleanupRunsIfPersistenceFails`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        when(currentUser.getCurrentUser()).thenReturn(owner);
        String uploaded = photo("44444444-4444-4444-8444-444444444444");
        org.springframework.web.multipart.MultipartFile file = mock(org.springframework.web.multipart.MultipartFile.class);
        when(storage.store(file)).thenReturn(uploaded);

        var result = service.uploadAndAttachPhoto(venueId, file);
        assertEquals(uploaded, result.photoPaths().getLast());

        venue.setPhotoPaths(List.of());
        when(venues.saveAndFlush(any())).thenThrow(new IllegalStateException("database failure"));
        assertThrows(IllegalStateException.class, () -> service.uploadAndAttachPhoto(venueId, file));
        verify(storage).delete(uploaded);
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/VenueServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-222 — VerifiedUserPolicyTest#requiresCurrentDatabaseVerificationForMarketplaceUsers

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=VerifiedUserPolicyTest#requiresCurrentDatabaseVerificationForMarketplaceUsers`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        User user = User.builder().role(UserRole.SCOUT).active(true).emailVerified(false).build();
        when(currentUsers.getCurrentUser()).thenReturn(user);
        assertThat(policy.isVerified()).isFalse();

        user.setEmailVerified(true);
        assertThat(policy.isVerified()).isTrue();

        user.setActive(false);
        assertThat(policy.isVerified()).isFalse();
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/VerifiedUserPolicyTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-223 — VerifiedUserPolicyTest#activeAdminOperationsAreNotBlockedByMarketplaceVerification

Area: AUTO · **Priority:** P0 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=VerifiedUserPolicyTest#activeAdminOperationsAreNotBlockedByMarketplaceVerification`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        when(currentUsers.getCurrentUser()).thenReturn(
                User.builder().role(UserRole.ADMIN).active(true).emailVerified(false).build());
        assertThat(policy.isVerified()).isTrue();
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/VerifiedUserPolicyTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-224 — WebhookServiceTest#latePaidWebhookStartsRefundInsteadOfNormalConfirmationPath

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=WebhookServiceTest#latePaidWebhookStartsRefundInsteadOfNormalConfirmationPath`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        UUID bookingId = UUID.randomUUID();
        MolliePaymentResponse payment = new MolliePaymentResponse("tr_late", "paid",
                new MolliePaymentResponse.Amount("100.00", "EUR"), null,
                Map.of("bookingId", bookingId.toString()));
        when(mollie.getPayment("tr_late")).thenReturn(payment);
        when(bookings.processPaidPayment(payment)).thenReturn(
                new BookingService.PaidPaymentOutcome(bookingId, BookingService.PaidPaymentResult.LATE_PAYMENT));

        service.handlePaymentWebhook("tr_late");

        verify(refunds).requestLatePaymentRefund(bookingId);
        verify(refunds).reconcileForPayment("tr_late");
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/WebhookServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-225 — WorkflowObservabilityTest#recordsLifecycleCountersWithoutHighCardinalityLabels

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=WorkflowObservabilityTest#recordsLifecycleCountersWithoutHighCardinalityLabels`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        BookingRepository bookings = mock(BookingRepository.class);
        SimpleMeterRegistry registry = new SimpleMeterRegistry();
        WorkflowObservability visibility = new WorkflowObservability(bookings,
                mock(com.eema.backend.repository.OwnerPayoutRepository.class),
                mock(com.eema.backend.repository.OwnerFinancialLiabilityRepository.class),
                registry, new OperationalThresholds(60, 1440, 1440, 72));

        visibility.bookingLifecycle(new BookingLifecycleEvent(java.util.UUID.randomUUID(),
                BookingLifecycleEvent.Type.APPROVAL_EXPIRED));
        visibility.bookingLifecycle(new BookingLifecycleEvent(java.util.UUID.randomUUID(),
                BookingLifecycleEvent.Type.REQUEST_EXPIRED));
        visibility.bookingLifecycle(new BookingLifecycleEvent(java.util.UUID.randomUUID(),
                BookingLifecycleEvent.Type.REQUEST_BECAME_UNAVAILABLE));
        visibility.inquiryLifecycle(new AvailabilityInquiryLifecycleEvent(java.util.UUID.randomUUID(),
                AvailabilityInquiryLifecycleEvent.Type.OFFER_EXPIRED));
        visibility.latePayment(new LatePaymentDetectedEvent(java.util.UUID.randomUUID()));
```

```
        visibility.automaticRefund(new LatePaymentRefundInitiatedEvent(java.util.UUID.randomUUID()));

        assertEquals(1, registry.counter("eema.workflow.booking.approval.expired").count());
        assertEquals(1, registry.counter("eema.workflow.booking.request.expired").count());
        assertEquals(1, registry.counter("eema.workflow.booking.request.unavailable").count());
        assertEquals(1, registry.counter("eema.workflow.inquiry.offer.expired").count());
        assertEquals(1, registry.counter("eema.workflow.payment.late.detected").count());
        assertEquals(1, registry.counter("eema.workflow.refund.automatic.initiated").count());
        registry.getMeters().forEach(meter -> assertEquals(0, meter.getId().getTags().size()));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/WorkflowObservabilityTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-226 — WorkflowObservabilityTest#stuckScanPublishesReadOnlyGauges

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=WorkflowObservabilityTest#stuckScanPublishesReadOnlyGauges`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        BookingRepository bookings = mock(BookingRepository.class);
        when(bookings.countByPaymentCreationStatusAndPaymentCreationStartedAtBefore(
                eq(PaymentCreationStatus.CREATING), any())).thenReturn(2L);
        when(bookings.countByRefundStatusInAndRefundRequestedAtBefore(anyList(), any())).thenReturn(3L);
        when(bookings.countOldUnreconciledMollieFees(any())).thenReturn(4L);
        SimpleMeterRegistry registry = new SimpleMeterRegistry();
        WorkflowObservability visibility = new WorkflowObservability(bookings,
                mock(com.eema.backend.repository.OwnerPayoutRepository.class),
                mock(com.eema.backend.repository.OwnerFinancialLiabilityRepository.class),
                registry, new OperationalThresholds(60, 1440, 1440, 72));

        visibility.scanStuckStates();

        assertEquals(2, registry.get("eema.workflow.payment.creation.stuck").gauge().value());
        assertEquals(3, registry.get("eema.workflow.refund.stuck").gauge().value());
        assertEquals(4, registry.get("eema.workflow.mollie.fee.unreconciled").gauge().value());
        verify(bookings).countByRefundStatusInAndRefundRequestedAtBefore(
                eq(List.of(RefundStatus.CREATING, RefundStatus.QUEUED, RefundStatus.PENDING, RefundStatus.PROCESSING)), any());
        verify(bookings, never()).save(any());
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/WorkflowObservabilityTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-227 — GeminiAiServiceTest#ranksOnlyCandidatesReturnedByDeterministicFiltering

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=GeminiAiServiceTest#ranksOnlyCandidatesReturnedByDeterministicFiltering`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        BriefParser parser = mock(BriefParser.class);
        VenueFilterBuilder filter = mock(VenueFilterBuilder.class);
        VenueMatchAvailabilityFilter availability = mock(VenueMatchAvailabilityFilter.class);
        VenueRanker ranker = mock(VenueRanker.class);
        ParsedBrief parsed = ParsedBriefFixtures.empty();
        Venue eligible = mock(Venue.class);
        Venue incompatible = mock(Venue.class);
        MatchResponse ranked = new MatchResponse(List.of(), "brief", 0);

        when(parser.parseBrief("brief")).thenReturn(parsed);
        when(filter.findCandidateVenues(parsed)).thenReturn(List.of(eligible));
        when(availability.retainAvailable(List.of(eligible), parsed)).thenReturn(List.of(eligible));
        when(ranker.rank("brief", parsed, List.of(eligible))).thenReturn(ranked);

        MatchResponse result = new GeminiAiService(parser, filter, availability, ranker).searchVenues("brief");

        assertThat(result).isSameAs(ranked);
        verify(ranker).rank("brief", parsed, List.of(eligible));
        verifyNoInteractions(incompatible);
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/ai/GeminiAiServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-228 — GeminiAiServiceTest#zeroEligibleCandidatesReturnsEmptyResponseWithoutCallingGeminiRanker

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=GeminiAiServiceTest#zeroEligibleCandidatesReturnsEmptyResponseWithoutCallingGeminiRanker`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        BriefParser parser = mock(BriefParser.class);
        VenueFilterBuilder filter = mock(VenueFilterBuilder.class);
        VenueMatchAvailabilityFilter availability = mock(VenueMatchAvailabilityFilter.class);
        VenueRanker ranker = mock(VenueRanker.class);
        ParsedBrief parsed = ParsedBriefFixtures.empty();
        when(parser.parseBrief("brief")).thenReturn(parsed);
        when(filter.findCandidateVenues(parsed)).thenReturn(List.of());
        when(availability.retainAvailable(List.of(), parsed)).thenReturn(List.of());
```

```
        MatchResponse result = new GeminiAiService(parser, filter, availability, ranker).searchVenues("brief");

        assertThat(result.venues()).isEmpty();
        assertThat(result.totalMatches()).isZero();
        verifyNoInteractions(ranker);
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/ai/GeminiAiServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-229 — GeminiAiServiceTest#availabilityRemovalSkipsRankingAndIsNotRelaxed

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=GeminiAiServiceTest#availabilityRemovalSkipsRankingAndIsNotRelaxed`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        BriefParser parser = mock(BriefParser.class);
        VenueFilterBuilder filter = mock(VenueFilterBuilder.class);
        VenueMatchAvailabilityFilter availability = mock(VenueMatchAvailabilityFilter.class);
        VenueRanker ranker = mock(VenueRanker.class);
        ParsedBrief parsed = ParsedBriefFixtures.fromJson("""
                {"requestedDate":"2026-08-25","requestedStartTime":"10:00:00",
                 "requestedEndTime":"16:00:00"}
                """);
        Venue candidate = mock(Venue.class);
        when(parser.parseBrief("brief")).thenReturn(parsed);
        when(filter.findCandidateVenues(parsed)).thenReturn(List.of(candidate));
        when(availability.retainAvailable(List.of(candidate), parsed)).thenReturn(List.of());

        MatchResponse result = new GeminiAiService(parser, filter, availability, ranker).searchVenues("brief");

        assertThat(result.venues()).isEmpty();
        verifyNoInteractions(ranker);
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/ai/GeminiAiServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-230 — GeminiAiServiceTest#onlyAvailableCandidatesReachRanking

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=GeminiAiServiceTest#onlyAvailableCandidatesReachRanking`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        BriefParser parser = mock(BriefParser.class);
        VenueFilterBuilder filter = mock(VenueFilterBuilder.class);
        VenueMatchAvailabilityFilter availability = mock(VenueMatchAvailabilityFilter.class);
        VenueRanker ranker = mock(VenueRanker.class);
        ParsedBrief parsed = ParsedBriefFixtures.empty();
        Venue available = mock(Venue.class);
        Venue unavailable = mock(Venue.class);
        when(parser.parseBrief("brief")).thenReturn(parsed);
        when(filter.findCandidateVenues(parsed)).thenReturn(List.of(available, unavailable));
        when(availability.retainAvailable(List.of(available, unavailable), parsed)).thenReturn(List.of(available));
        when(ranker.rank("brief", parsed, List.of(available)))
                .thenReturn(new MatchResponse(List.of(), "brief", 0));

        new GeminiAiService(parser, filter, availability, ranker).searchVenues("brief");

        verify(ranker).rank("brief", parsed, List.of(available));
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/ai/GeminiAiServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-231 — GeminiBriefParserServiceTest#mapsStructuredProductionRequirementsWithoutLiveGemini

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=GeminiBriefParserServiceTest#mapsStructuredProductionRequirementsWithoutLiveGemini`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        GeminiClient client = mock(GeminiClient.class);
        ObjectMapper mapper = new ObjectMapper().findAndRegisterModules();
        String json = """
                {
                  "productionActivity": "INTERVIEW",
                  "numPeople": 6,
                  "requiresSoundRecordingSuitable": true,
                  "prefersNaturalLight": true,
                  "minimumCeilingHeightM": 3.5,
                  "requiresParking": true,
                  "minimumParkingSpaces": 2,
                  "mustHaveTags": ["studio"],
                  "confidenceScore": 0.93
                }
                """;
        when(client.generateContent(anyString())).thenReturn("raw");
        when(client.extractGeneratedText("raw")).thenReturn(json);
```

```
        MatchingReferenceDateProvider dates = mock(MatchingReferenceDateProvider.class);
        when(dates.today()).thenReturn(LocalDate.of(2026, 8, 20));
        ParsedBrief parsed = new GeminiBriefParserService(client, mapper, dates)
                .parseBrief("Interview for six people, needs clean audio");

        assertThat(parsed.productionActivity()).isEqualTo(ProductionActivity.INTERVIEW);
        assertThat(parsed.numPeople()).isEqualTo(6);
        assertThat(parsed.requiresSoundRecordingSuitable()).isTrue();
        assertThat(parsed.prefersNaturalLight()).isTrue();
        assertThat(parsed.minimumCeilingHeightM()).isEqualByComparingTo("3.5");
        assertThat(parsed.minimumParkingSpaces()).isEqualTo(2);
        assertThat(parsed.mustHaveTags()).containsExactly("studio");
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/ai/GeminiBriefParserServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-232 — GeminiBriefParserServiceTest#promptDefinesRequirementStrengthEnumsAndStructuredTagBoundary

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=GeminiBriefParserServiceTest#promptDefinesRequirementStrengthEnumsAndStructuredTagBoundary`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        GeminiClient client = mock(GeminiClient.class);
        when(client.generateContent(anyString())).thenReturn("raw");
        when(client.extractGeneratedText("raw")).thenReturn("{}");

        MatchingReferenceDateProvider dates = mock(MatchingReferenceDateProvider.class);
        when(dates.today()).thenReturn(LocalDate.of(2026, 8, 20));
        new GeminiBriefParserService(client, new ObjectMapper().findAndRegisterModules(), dates)
                .parseBrief("Industrial warehouse, parking would be nice");

        ArgumentCaptor<String> prompt = ArgumentCaptor.forClass(String.class);
        verify(client).generateContent(prompt.capture());
        assertThat(prompt.getValue())
                .contains("Requirement strength")
                .contains("PHOTO, FILM, VIDEO, INTERVIEW")
                .contains("COMMERCIAL, MUSIC_VIDEO, CONTENT_CREATION")
                .contains("Never put natural light")
                .contains("If ambiguous, return null")
                .contains("parking would be nice");
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/ai/GeminiBriefParserServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-233 — GeminiBriefParserServiceTest#relativeDateUsesExplicitApplicationReferenceDate

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=GeminiBriefParserServiceTest#relativeDateUsesExplicitApplicationReferenceDate`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        GeminiClient client = mock(GeminiClient.class);
        MatchingReferenceDateProvider dates = mock(MatchingReferenceDateProvider.class);
        when(dates.today()).thenReturn(LocalDate.of(2026, 8, 20));
        when(client.generateContent(anyString())).thenReturn("raw");
        when(client.extractGeneratedText("raw")).thenReturn("""
                {"requestedDate":"2026-08-21","requestedStartTime":"10:00:00",
                 "requestedEndTime":"16:00:00","timingDatePreferences":"tomorrow 10am-4pm"}
                """);

        ParsedBrief parsed = new GeminiBriefParserService(
                client, new ObjectMapper().findAndRegisterModules(), dates
        ).parseBrief("Tomorrow from 10am to 4pm");

        assertThat(parsed.requestedDate()).isEqualTo(LocalDate.of(2026, 8, 21));
        assertThat(parsed.requestedStartTime()).hasToString("10:00");
        assertThat(parsed.requestedEndTime()).hasToString("16:00");
        ArgumentCaptor<String> prompt = ArgumentCaptor.forClass(String.class);
        verify(client).generateContent(prompt.capture());
        assertThat(prompt.getValue()).contains("Current date: 2026-08-20")
                .contains("Europe/Amsterdam");
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/ai/GeminiBriefParserServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-234 — GeminiBriefParserServiceTest#ambiguousTimingRemainsNonExactContext

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=GeminiBriefParserServiceTest#ambiguousTimingRemainsNonExactContext`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        GeminiClient client = mock(GeminiClient.class);
        MatchingReferenceDateProvider dates = mock(MatchingReferenceDateProvider.class);
        when(dates.today()).thenReturn(LocalDate.of(2026, 8, 20));
        when(client.generateContent(anyString())).thenReturn("raw");
        when(client.extractGeneratedText("raw")).thenReturn("""
                {"requestedDate":null,"requestedStartTime":null,"requestedEndTime":null,
                 "timingDatePreferences":"Friday afternoon"}
```

```
                """);

        ParsedBrief parsed = new GeminiBriefParserService(
                client, new ObjectMapper().findAndRegisterModules(), dates
        ).parseBrief("Friday afternoon");

        assertThat(parsed.requestedDate()).isNull();
        assertThat(parsed.requestedStartTime()).isNull();
        assertThat(parsed.requestedEndTime()).isNull();
        assertThat(parsed.timingDatePreferences()).isEqualTo("Friday afternoon");
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/ai/GeminiBriefParserServiceTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

AUTO-235 — GeminiVenueRankerTest#rankingPromptReceivesAuthoritativeProductionFactsAndSafetyRules

Area: AUTO · **Priority:** P1 · **Test level:** UNIT · **Result:** NOT RUN

Roles involved: Roles constructed by the named test; SYSTEM/WEBHOOK where fixture invokes internal transition.

Preconditions: JDK 21 and Maven dependencies available; Docker required only for postgres-integration. Existing test class setup must run unmodified.

Test data: Use the exact existing test method and @BeforeEach/helper fixtures in linked file; fixture values are executable below.

Exact steps: From backend repository run `mvn test -Dtest=GeminiVenueRankerTest#rankingPromptReceivesAuthoritativeProductionFactsAndSafetyRules`. The existing class setup and the exact arrange/act/assert body below define fixture construction, provider stubs, invocation order and expected values; no live credentials required. Inspect surefire/failsafe XML for this named test and zero skips. Do not replace the real-Postgres profile with mocks for concurrency checks.

Expected UI behavior: N/A: this contract test does not mount the frontend. Mapped UX/JOURNEY scenarios remain NOT RUN.

Expected API/backend behavior: All explicit assertions, exception checks, repository/provider verifications and state comparisons in the executable contract below must pass. A passing method covers only these assertions, not full UI/staging behavior.

```
{
        GeminiClient client = mock(GeminiClient.class);
        VenueMapper mapper = mock(VenueMapper.class);
        ObjectMapper objectMapper = new ObjectMapper().findAndRegisterModules();
        Venue venue = mock(Venue.class);
        when(venue.getId()).thenReturn(UUID.randomUUID());
        when(venue.getName()).thenReturn("Sound Stage");
        when(venue.getHourlyRateEur()).thenReturn(new BigDecimal("175.00"));
        when(venue.getMinimumBookingMinutes()).thenReturn(240);
        when(venue.getMaxCrewSize()).thenReturn(30);
        when(venue.getPermittedActivities()).thenReturn(List.of(ProductionActivity.INTERVIEW));
        when(venue.getNaturalLight()).thenReturn(false);
        when(venue.getSoundRecordingSuitable()).thenReturn(true);
        when(venue.getProductionRules()).thenReturn("No smoke effects");
        when(venue.getPowerDetails()).thenReturn("3-phase power available");
        when(venue.getTags()).thenReturn(List.of("studio"));
        when(client.generateContent(anyString())).thenReturn("raw");
        when(client.extractGeneratedText("raw")).thenReturn("[]");

        ParsedBrief brief = ParsedBriefFixtures.fromJson("""
                {"requestedDate":"2026-08-25","requestedStartTime":"10:00:00",
                 "requestedEndTime":"16:00:00"}
                """);
        new GeminiVenueRanker(client, objectMapper, mapper)
                .rank("Interview", brief, List.of(venue));

        ArgumentCaptor<String> prompt = ArgumentCaptor.forClass(String.class);
        verify(client).generateContent(prompt.capture());
        assertThat(prompt.getValue())
                .contains("already passed all deterministic hard requirements")
                .contains("\"maxCrewSize\":30")
                .contains("\"permittedActivities\":[\"INTERVIEW\"]")
                .contains("\"soundRecordingSuitable\":true")
                .contains("No smoke effects")
                .contains("3-phase power available")
                .contains("Do not claim current availability")
                .contains("available for the exact requested interval 2026-08-25 10:00-16:00")
                .contains("Do not judge or claim a total budget fit")
```

```
                .contains("Do not claim excellent acoustics");
        assertThat(prompt.getValue()).doesNotContain("ownerEmail");
    }
```

Expected database/domain state: Mocked repository interactions only unless PostgreSQL integration profile; assertions below define persisted rows/constraints for real-Postgres cases.

Expected email/provider behavior: Existing test doubles/captured arguments only; no actual provider delivery implied.

Security/authorization expectation: Assert denial/ownership only where named test has those assertions; API-* security matrix covers remaining role boundaries.

Cleanup/reset steps: JUnit class fixture cleanup / disposable Testcontainer teardown; discard generated test output when no longer needed.

Automation recommendation: ALREADY AUTOMATED

Local/staging requirement: LOCAL MOCKED

Source / existing evidence: src/test/java/com/eema/backend/service/ai/GeminiVenueRankerTest.java

Pass/fail criteria: PASS only when every listed variant meets all expectations and cleanup/evidence are recorded; FAIL for any mismatch or forbidden side effect; BLOCKED when required harness/provider/fixture is unavailable. Existing source-level characterization does not close a separate launch-acceptance gap.

Full endpoint-by-role matrix

Each row has its own API test above. Cells describe a valid eligible fixture; body validation, object existence, role and state checks are not a promise of 200 for arbitrary inputs. `own` means the booking participant or owning venue/payout. Anonymous protected venue GET paths reach method security after the broad public GET matcher; GlobalExceptionHandler maps AccessDeniedException to 403. Other authenticated paths are denied with 401 by the filter chain. Webhook callback processing may deliberately respond 500 to trigger delivery retry; never mistake it for a business success.

Controller / signature path (prefix /api)	anonymous	SCOUT	OWNER	ADMIN	Test
AdminBookingOperationsController: `GET /admin/bookings`	401	403	403	200	API-001
AdminBookingOperationsController: `GET /admin/bookings/{id}`	401	403	403	200	API-002
AdminBookingOperationsController: `GET /admin/bookings/{id}/events`	401	403	403	200	API-003
AdminIncidentController: `GET /admin/incidents`	401	403	403	200	API-004

Controller / signature path (prefix /api)	anonymous	SCOUT	OWNER	ADMIN	Test
AdminIncidentController: `GET /admin/incidents/{id}`	401	403	403	200	API-005
AdminIncidentController: `POST /admin/incidents/{id}/start-review`	401	403	403	200	API-006
AdminIncidentController: `POST /admin/incidents/{id}/resolve`	401	403	403	200	API-007
AdminIncidentController: `POST /admin/incidents/{id}/close`	401	403	403	200	API-008
AdminOperationsExceptionController: `GET /admin/operations/exceptions`	401	403	403	200	API-009
AdminOwnerFinancialLiabilityController: `GET /admin/liabilities`	401	403	403	200	API-010
AdminOwnerFinancialLiabilityController: `GET /admin/liabilities/{id}`	401	403	403	200	API-011
AdminOwnerFinancialLiabilityController: `POST /admin/liabilities/{id}/settle`	401	403	403	200	API-012
AdminOwnerPayoutController: `GET /admin/payouts`	401	403	403	200	API-013
AdminOwnerPayoutController: `GET /admin/payouts/{id}`	401	403	403	200	API-014
AdminOwnerPayoutController: `POST /admin/payouts/{id}/mark-paid`	401	403	403	200	API-015

Controller / signature path (prefix /api)	anonymous	SCOUT	OWNER	ADMIN	Test
AdminOwnerVerificationController: `GET /admin/owner-verifications`	401	403	403	200	API-016
AdminOwnerVerificationController: `GET /admin/owner-verifications/{id}`	401	403	403	200	API-017
AdminOwnerVerificationController: `POST /admin/owner-verifications/{id}/approve`	401	403	403	200	API-018
AdminOwnerVerificationController: `POST /admin/owner-verifications/{id}/reject`	401	403	403	200	API-019
AdminPayoutDestinationController: `GET /admin/payout-destinations`	401	403	403	200	API-020
AdminPayoutDestinationController: `GET /admin/payout-destinations/{id}`	401	403	403	200	API-021
AdminPayoutDestinationController: `POST /admin/payout-destinations/{id}/verify`	401	403	403	200	API-022
AdminPayoutDestinationController: `POST /admin/payout-destinations/{id}/reject`	401	403	403	200	API-023
AdminPayoutDestinationController: `POST /admin/payout-destinations/{id}/reveal`	401	403	403	200	API-024
AdminPolicyController: `GET /admin/policies`	401	403	403	200	API-025
AdminPolicyController: `POST /admin/policies`	401	403	403	201	API-026

Controller / signature path (prefix /api)	anonymous	SCOUT	OWNER	ADMIN	Test
AdminPolicyController: POST /admin/policies/{id}/publish	401	403	403	200	API-027
AdminPolicyController: POST /admin/policies/{id}/retire	401	403	403	200	API-028
AdminVenueModerationController: GET /admin/venues	401	403	403	200	API-029
AdminVenueModerationController: GET /admin/venues/{id}	401	403	403	200	API-030
AdminVenueModerationController: GET /admin/venues/{id}/events	401	403	403	200	API-031
AdminVenueModerationController: POST /admin/venues/{id}/approve	401	403	403	200	API-032
AdminVenueModerationController: POST /admin/venues/{id}/request-changes	401	403	403	200	API-033
AdminVenueModerationController: POST /admin/venues/{id}/suspend	401	403	403	200	API-034
AdminVenueModerationController: POST /admin/venues/{id}/reactivate	401	403	403	200	API-035
AuthController: POST /auth/register	201	201	201	201	API-036
AuthController: POST /auth/login	200	200	200	200	API-037

Controller / signature path (prefix /api)	anonymous	SCOUT	OWNER	ADMIN	Test
AuthController: POST /auth/refresh	200	200	200	200	API-038
AuthController: POST /auth/verify-email	200	200	200	200	API-039
AuthController: POST /auth/resend-verification	200	200	200	200	API-040
AuthController: POST /auth/forgot-password	200	200	200	200	API-041
AuthController: POST /auth/reset-password	200	200	200	200	API-042
AuthController: GET /auth/me	401	200	200	200	API-043
AvailabilityInquiryController: POST /availability-inquiries	401	201	403	403	API-044
AvailabilityInquiryController: GET /availability-inquiries/mine	401	200	403	403	API-045
AvailabilityInquiryController: GET /availability-inquiries/owned-venues	401	403	200 own / 403 other	403	API-046
AvailabilityInquiryController: POST /availability-inquiries/{id}/decline	401	403	200 own / 403 other	403	API-047
AvailabilityInquiryController: POST /availability-inquiries/{id}/offer	401	403	200 own / 403 other	403	API-048
AvailabilityInquiryController: POST /availability-inquiries/{id}/accept	401	200	403	403	API-049
BookingController: POST /bookings	401	201	403	403	API-050
BookingController: GET /bookings	401	200 own / 403 other	200 own / 403 other	200	API-051

Controller / signature path (prefix /api)	anonymous	SCOUT	OWNER	ADMIN	Test
BookingController: GET /bookings/{id}	401	200 own / 403 other	200 own / 403 other	200	API-052
BookingController: POST /bookings/{id}/accept	401	403	200 own / 403 other	200	API-053
BookingController: POST /bookings/{id}/decline	401	403	200 own / 403 other	200	API-054
BookingController: POST /bookings/{id}/cancel	401	200 own / 403 other	200 own / 403 other	200	API-055
BookingController: GET /bookings/{id}/cancellation-preview	401	200 own / 403 other	200 own / 403 other	200	API-056
BookingController: POST /bookings/{id}/refund	401	403	403	202	API-057
CheckoutController: POST /bookings/{bookingId}/checkout	401	201 own / 403 other	403	403	API-058
FinancialDocumentController: GET /bookings/{id}/receipt	401	200 own / 403 other	403	200	API-059
IncidentController: POST /bookings/{bookingId}/incidents	401	201/200 own;403 other	201/200 own;403 other	201/200	API-060
IncidentController: GET /bookings/{bookingId}/incidents	401	201/200 own;403 other	201/200 own;403 other	201/200	API-061
MatchController: POST /match	401	200	200	200	API-062
OwnerPayoutController: GET /owner/payouts	401	403	200 own / 403 other	403	API-063
OwnerPayoutController: GET /owner/payouts/{id}	401	403	200 own / 403 other	403	API-064
OwnerPayoutController: GET /owner/payouts/{id}/statement	401	403	200 own / 403 other	200	API-065

Controller / signature path (prefix /api)	anonymous	SCOUT	OWNER	ADMIN	Test
OwnerPayoutDestinationController: `GET /owner/payout-destinations`	401	403	200	403	API-066
OwnerPayoutDestinationController: `POST /owner/payout-destinations`	401	403	200	403	API-067
OwnerVerificationController: `GET /owner/verification`	401	403	200	403	API-068
OwnerVerificationController: `POST /owner/verification/submit`	401	403	200	403	API-069
PolicyController: `GET /policies/current`	401	200	200	200	API-070
PolicyController: `GET /policies/{type}/current`	401	200	200	200	API-071
PolicyController: `GET /policies/status`	401	200	200	200	API-072
PolicyController: `POST /policies/accept`	401	200	200	200	API-073
UploadController: `POST /upload/venue-photo`	401	403	200	403	API-074
UploadController: `GET /upload/files/{filename:.+}`	200	200	200	200	API-075
VenueAvailabilityController: `GET /venues/{venueId}/availability`	200 ACTIVE / 403 other	200 ACTIVE / 403 other	200 ACTIVE or own / 403 other	200	API-076
VenueAvailabilityController: `GET /venues/{venueId}/availability/settings`	403	403	200 own / 403 other	200	API-077
VenueAvailabilityController: `PUT /venues/{venueId}/availability/settings`	401	403	200 own / 403 other	200	API-078

Controller / signature path (prefix /api)	anonymous	SCOUT	OWNER	ADMIN	Test
VenueAvailabilityController: PUT /venues/{venueId}/availability/weekly	401	403	200 own / 403 other	200	API-079
VenueAvailabilityController: POST /venues/{venueId}/availability/overrides	401	403	201 own / 403 other	201	API-080
VenueAvailabilityController: DELETE /venues/{venueId}/availability/overrides/{overrideId}	401	403	204 own / 403 other	204	API-081
VenueController: GET /venues	200	200	200	200	API-082
VenueController: GET /venues/{id}	200 ACTIVE / 403 other	200 ACTIVE / 403 other	200 ACTIVE or own / 403 other	200	API-083
VenueController: POST /venues	401	403	201	403	API-084
VenueController: GET /venues/my	403	403	200	403	API-085
VenueController: PUT /venues/{id}	401	403	200 own / 403 other	200	API-086
VenueController: POST /venues/{id}/photos	401	403	200 own / 403 other	200	API-087
VenueController: POST /venues/{id}/submit-for-review	401	403	200 own / 403 other	403	API-088
VenueController: GET /venues/{id}/moderation	403	403	200 own / 403 other	200	API-089
WebhookController: POST /webhooks/mollie/payment	200	200	200	200	API-090

Additional framework endpoints: GET `/api/actuator/health`, `/api/actuator/health/readiness`, `/api/actuator/health/liveness` public; `/api/actuator/metrics/**` ADMIN only (SEC-006). Frontend server endpoint POST `/api/verify-booker` is on Next origin and public, covered by CALA-001–CALA-001; do not confuse it with Spring `/api` routes.

Every frontend route and design QA

Every row attaches the same complete design, responsive, accessibility, navigation, loader/empty/error and role checklist through its unique UX scenario. Empty state for a singular record is missing/not-found, not a fabricated empty table. Admin home is static navigation behind auth. `/` also covers SearchBox, ExampleChips, VenueCard, VenueDetail, gallery, ProductionProfile, InstantQuote/calendar and inquiry fallback. No route inventory entry is omitted for being an admin/detail page.

Next page route	File	Full design/state checklist
`/admin/bookings/[id]`	frontend/app/admin/bookings/[id]/page.tsx	UX-001
`/admin/bookings`	frontend/app/admin/bookings/page.tsx	UX-002
`/admin/exceptions`	frontend/app/admin/exceptions/page.tsx	UX-003
`/admin/incidents/[id]`	frontend/app/admin/incidents/[id]/page.tsx	UX-004
`/admin/incidents`	frontend/app/admin/incidents/page.tsx	UX-005
`/admin/liabilities/[id]`	frontend/app/admin/liabilities/[id]/page.tsx	UX-006
`/admin/liabilities`	frontend/app/admin/liabilities/page.tsx	UX-007
`/admin/owner-verifications/[id]`	frontend/app/admin/owner-verifications/[id]/page.tsx	UX-008
`/admin/owner-verifications`	frontend/app/admin/owner-verifications/page.tsx	UX-009
`/admin`	frontend/app/admin/page.tsx	UX-010
`/admin/payout-destinations/[id]`	frontend/app/admin/payout-destinations/[id]/page.tsx	UX-011
`/admin/payout-destinations`	frontend/app/admin/payout-destinations/page.tsx	UX-012
`/admin/payouts/[id]`	frontend/app/admin/payouts/[id]/page.tsx	UX-013
`/admin/payouts`	frontend/app/admin/payouts/page.tsx	UX-014
`/admin/policies`	frontend/app/admin/policies/page.tsx	UX-015
`/admin/venues/[id]`	frontend/app/admin/venues/[id]/page.tsx	UX-016
`/admin/venues`	frontend/app/admin/venues/page.tsx	UX-017
`/booked`	frontend/app/booked/page.tsx	UX-018
`/bookings/[id]`	frontend/app/bookings/[id]/page.tsx	UX-019
`/forgot-password`	frontend/app/forgot-password/page.tsx	UX-020
`/owner/bookings`	frontend/app/owner/bookings/page.tsx	UX-021
`/owner/inquiries`	frontend/app/owner/inquiries/page.tsx	UX-022

Next page route	File	Full design/state checklist
`/owner/payout-destination`	frontend/app/owner/payout-destination/page.tsx	UX-023
`/owner/payouts/[id]`	frontend/app/owner/payouts/[id]/page.tsx	UX-024
`/owner/payouts`	frontend/app/owner/payouts/page.tsx	UX-025
`/owner/venues/[id]`	frontend/app/owner/venues/[id]/page.tsx	UX-026
`/owner/venues/new`	frontend/app/owner/venues/new/page.tsx	UX-027
`/owner/venues`	frontend/app/owner/venues/page.tsx	UX-028
`/owner/verification`	frontend/app/owner/verification/page.tsx	UX-029
`/`	frontend/app/page.tsx	UX-030
`/policies`	frontend/app/policies/page.tsx	UX-031
`/reset-password`	frontend/app/reset-password/page.tsx	UX-032
`/scout/bookings`	frontend/app/scout/bookings/page.tsx	UX-033
`/scout/inquiries`	frontend/app/scout/inquiries/page.tsx	UX-034
`/verify-email`	frontend/app/verify-email/page.tsx	UX-035

Shared navigation: anonymous Marketplace/Policies/Sign in; SCOUT My bookings/Inquiries; OWNER My venues/Verification/Payout details/Venue bookings/Payouts/Inquiries; ADMIN Operations/Bookings/Payouts/Payout destinations/Liabilities/Exceptions/Owner verification/Venue moderation/Incidents/Policies. Follow each header link, each Operations card, row detail link, back link, CTA and document download at all three widths. Verify logout and deep-link login across all roles. API-only paths are tested as API, not invented UI.

Copy and financial terminology checklist

Term	Required distinction
Request / booking	REQUESTED is a stored request, not paid/confirmed or a reserved slot. Review current expiry email saying “No booking was created” against this distinction.
Accepted / confirmed	Owner acceptance means AWAITING_PAYMENT; only authoritative successful payment confirms.
Payment / refund	Checkout/redirect is not payment proof; initiated/queued/processing refund is not completed or bank-settled.
Venue price / total paid	Booking snapshot versus actually received amount; never browser-controlled money.
Commission / provider fee	Eema percentage on authoritative retained amount versus actual reconciled provider cost. Unknown fee is not a zero-fee claim.
Owner payout / retained amount	Owner net entitlement after deductions versus amount not returned to scout.

Term	Required distinction
CALCULATED / HELD / READY / PAID	Calculation and readiness are not external transfer. Mark paid records a transfer and does not send money.
Receipt / statement	Payment record versus owner payout record; neither presented as tax invoice.
Owner verification / venue review / payout destination review	Separate manual trust decisions; not automated KYC, identity proof or bank ownership proof.
Incident / cancellation	Incident review has no automatic financial or venue effect.
Policy review / privacy acknowledgement	Exact published versions; final approved privacy semantics remain business/legal input.

Apply table to every UI state, notification, PDF and admin action; contradictory financial/trust copy fails the parent scenario. No automation or documentation should silently choose missing legal/business rules.

Complete email lifecycle matrix

Lifecycle	Recipient authority	Exact subject	CTA	Local content / staging delivery
Registration verification	active registering/resending account	Verify your Eema email	`/verify-email?token=…`	EMAIL-001; PROVIDER-002
Password reset	active existing account	Reset your Eema password	`/reset-password?token=…`	EMAIL-002; PROVIDER-002
Request submitted scout	scout contact snapshot	Your request was sent	`/bookings/{id}`	EMAIL-003; PROVIDER-002
Request submitted owner	owner contact snapshot	New booking request	`/bookings/{id}`	EMAIL-004; PROVIDER-002
Request accepted	scout contact snapshot	Your booking request was accepted	`/bookings/{id}`	EMAIL-005; PROVIDER-002
Request declined	scout contact snapshot	Your booking request was declined	`/bookings/{id}`	EMAIL-006; PROVIDER-002
Request expired	scout contact snapshot	Your booking request expired	`/bookings/{id}`	EMAIL-007; PROVIDER-002
Request unavailable	scout contact snapshot	The requested time is no longer available	`/bookings/{id}`	EMAIL-008; PROVIDER-002
Payment window expired	scout contact snapshot	Your payment window expired	`/bookings/{id}`	EMAIL-009; PROVIDER-002
Payment confirmed scout	scout contact snapshot	Your Eema booking is confirmed	`/bookings/{id}`	EMAIL-010; PROVIDER-002
Payment confirmed owner	owner contact snapshot	Confirmed booking for your Eema venue	`/bookings/{id}`	EMAIL-011; PROVIDER-002
Late payment	scout contact snapshot	Your late Eema payment is being refunded	`/bookings/{id}`	EMAIL-012; PROVIDER-002
Cancellation scout notification	scout contact snapshot for all cancellation actors	Your Eema booking was cancelled	`/bookings/{id}`	EMAIL-013; PROVIDER-002
Scout cancellation owner notification	owner contact snapshot only when SCOUT cancelled	A scout cancelled an Eema booking	`/bookings/{id}`	EMAIL-014; PROVIDER-002

Lifecycle	Recipient authority	Exact subject	CTA	Local content / staging delivery
Cancellation refund completed	scout contact snapshot	Your Eema refund has been processed	`/bookings/{id}`	EMAIL-015; PROVIDER-002
Inquiry created	current venue owner account	New availability inquiry	`/owner/inquiries`	EMAIL-016; PROVIDER-002
Inquiry offered	current scout account	The owner offered you a time	`/scout/inquiries`	EMAIL-017; PROVIDER-002
Inquiry declined	current scout account	Your availability inquiry was declined	`/scout/inquiries`	EMAIL-018; PROVIDER-002
Inquiry offer expired	current scout account	Your availability offer expired	`/scout/inquiries`	EMAIL-019; PROVIDER-002
Owner verification submitted	owner account	Owner verification submitted	`/owner/verification`	EMAIL-020; PROVIDER-002
Owner verification approved	owner account	Your Eema owner verification was approved	`/owner/verification`	EMAIL-021; PROVIDER-002
Owner verification rejected	owner account	Your Eema owner verification needs attention	`/owner/verification`	EMAIL-022; PROVIDER-002
Venue submitted	venue owner account	Venue submitted for review	`/owner/venues/{id}`	EMAIL-023; PROVIDER-002
Venue approved	venue owner account	Your venue is live	`/owner/venues/{id}`	EMAIL-024; PROVIDER-002
Venue changes requested	venue owner account	Changes requested for your venue	`/owner/venues/{id}`	EMAIL-025; PROVIDER-002
Venue suspended	venue owner account	Your venue was suspended	`/owner/venues/{id}`	EMAIL-026; PROVIDER-002
Venue reactivated	venue owner account	Your venue is live again	`/owner/venues/{id}`	EMAIL-027; PROVIDER-002
Destination submitted	owner account	Payout destination submitted	`/owner/payout-destination`	EMAIL-028; PROVIDER-002
Destination verified	owner account	Payout destination verified	`/owner/payout-destination`	EMAIL-029; PROVIDER-002
Destination rejected	owner account	Your payout destination needs attention	`/owner/payout-destination`	EMAIL-030; PROVIDER-002

Unimplemented email paths remain EMAIL-031. OPS-008 / OPS-DEC-001 now authorizes durable at-least-once dispatch for existing notifications; old generated after-commit-only descriptions are superseded by its explicit contract. Do not infer a generic refund-completed email for all refunds from the cancellation-specific completion listener. Verification/reset link tokens are intentionally present in their messages; other lifecycle messages contain no credentials.

Existing automated coverage mapped

The backend CI has fast `mvn test`, independent PostgreSQL `mvn verify -Ppostgres-integration`, and package after both. Frontend package scripts provide typecheck/lint/build; no test runner, browser suite or frontend CI workflow was found in the inspected repository. Static reflection/security-surface tests are narrower than the full actual HTTP role matrix, especially for newer controllers. SQL race tests are not service-level HTTP race proof.

Existing file	Named executable cases (each links to exact steps and assertions)
src/test/java/com/eema/backend/SimpleTest.java	AUTO-001 `testTrue`

Existing file	Named executable cases (each links to exact steps and assertions)
src/test/java/com/eema/backend/config/AsyncConfigurationTest.java	AUTO-002 `createsBoundedNamedExecutor`
src/test/java/com/eema/backend/config/FlywayLocationTest.java	AUTO-003 `developmentSeedsAreNotInCommonProductionLocation`
src/test/java/com/eema/backend/config/ProductionConfigurationValidatorTest.java	AUTO-004 `rejectsLocalStorageInProduction`; AUTO-005 `rejectsWeakJwtInProduction`; AUTO-006 `rejectsWildcardCorsInProduction`; AUTO-007 `rejectsSharedBuckets`; AUTO-008 `requiresS3Credentials`
src/test/java/com/eema/backend/controller/AdminOperationsSurfaceTest.java	AUTO-009 `everyAdminControllerHasDefenseInDepthRoleGuard`; AUTO-010 `surfaceContainsNoGenericPutPatchOrBookingStatusMutation`
src/test/java/com/eema/backend/controller/AdminOwnerFinancialLiabilityControllerTest.java	AUTO-011 `scoutAndOwnerAreDeniedAllLiabilityOperations`
src/test/java/com/eema/backend/controller/AdminOwnerPayoutControllerTest.java	AUTO-012 `scoutAndOwnerCannotReadOrMarkPayouts`; AUTO-013 `adminCanUseOperationsEndpoints`
src/test/java/com/eema/backend/controller/FinancialDocumentControllerTest.java	AUTO-014 `authorizedEndpointReturnsPdfAttachmentWithoutStoragePath`
src/test/java/com/eema/backend/controller/TrustWorkflowSecuritySurfaceTest.java	AUTO-015 `adminQueuesAndActionsAreAdminOnly`; AUTO-016 `ownerVerificationAndSubmissionAreOwnerOnly`
src/test/java/com/eema/backend/controller/VerifiedEmailEnforcementSurfaceTest.java	AUTO-017 `launchBlockerMarketplaceMutationsUseCentralVerifiedUserPolicy`
src/test/java/com/eema/backend/integration/PostgresCriticalPathIntegrationTest.java	AUTO-018 `commonFlywayChainAndHibernateValidationStartFromEmptyDatabase`; AUTO-019 `nativeEnumsAndProductionActivityArraysRoundTripThroughHibernate`; AUTO-020 `exclusionRejectsOneOverlappingBlockingInsert`; AUTO-021 `halfOpenBackToBackReservationsBothPersist`; AUTO-022 `overlappingRequestedRowsCoexistButConcurrentAcceptanceCannotDoubleReserve`; AUTO-023 `twoSkipLockedWorkersReceiveDisjointBoundedBookingBatches`; AUTO-024 `inquiryAndPayoutSkipLockedQueriesAreActuallyDisjoint`; AUTO-025 `refreshTokenRotationLockAllowsOnlyOneWinner`; AUTO-026 `payoutAndDocumentUniqueConstraintsHaveSingleAuthorities`; AUTO-027 `verifiedDestinationPartialIndexAndReplacementSemanticsHold`; AUTO-028 `readyPayoutDestinationSnapshotDoesNotFollowReplacement`; AUTO-029 `policyPublishRaceLeavesExactlyOneCurrentVersion`; AUTO-030 `policyAcceptanceBookingSnapshotStaysHistoricalAndIncidentHasNoFinancialSideEffects`
src/test/java/com/eema/backend/security/JwtServiceTest.java	AUTO-031 `validatesSignatureExpiryAndExpectedIssuer`
src/test/java/com/eema/backend/security/RequestCorrelationFilterTest.java	AUTO-032 `propagatesSafeRequestId`; AUTO-033 `replacesUnsafeRequestId`
src/test/java/com/eema/backend/service/AccountEmailNotificationServiceTest.java	AUTO-034 `verificationUsesAuthoritativeEmailAndFrontendVerificationRoute`; AUTO-035 `resetUsesAuthoritativeEmailAndFrontendResetRoute`

Existing file	Named executable cases (each links to exact steps and assertions)
src/test/java/com/eema/backend/service/AccountRecoveryServiceTest.java	AUTO-036 `issueStoresOnlyHashAndPublishesRawTokenAfterTransactionBoundary`; AUTO-037 `validVerificationIsSingleUseAndSameUsedTokenIsIdempotent`; AUTO-038 `expiredAndInactiveVerificationAreRejectedWithoutActivation`; AUTO-039 `resendIsGenericAndDoesNothingForVerifiedOrUnknownAccount`; AUTO-040 `forgotIsGenericAndResetChangesPasswordAndRevokesEveryRefreshSession`; AUTO-041 `unknownForgotResponseMatchesExistingAndExpiredResetIsRejected`
src/test/java/com/eema/backend/service/AdminOperationsServiceTest.java	AUTO-042 `eventTimelineIsChronologicalPagedAndRedactsUnknownSensitiveKeys`; AUTO-043 `exceptionQueueAggregatesAllRequiredOperationalCategories`; AUTO-044 `bookingSearchForwardsControlledFiltersAndCapsPageSize`; AUTO-045 `bookingDetailCombinesAuthoritativeRefundFeePayoutAndLiabilityState`
src/test/java/com/eema/backend/service/AuthServiceTest.java	AUTO-046 `loginRejectsInactiveUser`; AUTO-047 `registerAndLoginIssueTokensWithoutPersistingRawRefreshToken`; AUTO-048 `refreshRotatesAndRejectsReuse`
src/test/java/com/eema/backend/service/AvailabilityInquiryNotificationServiceTest.java	AUTO-049 `ownerReceivesNewInquiryWithExistingOwnerRoute`; AUTO-050 `scoutReceivesOfferWithDeadlineAndExistingScoutRoute`; AUTO-051 `scoutReceivesDeclineAndOfferExpiry`; AUTO-052 `providerFailureDoesNotEscapeNotificationBoundary`
src/test/java/com/eema/backend/service/AvailabilityInquiryServiceTest.java	AUTO-053 `inquiryIsAllowedForUnavailableInterval`; AUTO-054 `inquiryIsRejectedWhenNormalAvailabilityAlreadyContainsInterval`; AUTO-055 `unauthorizedOwnerCannotRespond`; AUTO-056 `ownerDeclineEmitsOneLifecycleEventAndDuplicateIsRejected`; AUTO-057 `ownerCannotOfferIntervalConflictingWithBooking`; AUTO-058 `inquiryAndOwnerOfferCannotBypassLeadTime`; AUTO-059 `offeredSlotThatBecomesTooLateExpiresWithoutCreatingBooking`; AUTO-060 `offerExpiresAndCannotBeAccepted`; AUTO-061 `onlyInquiryScoutCanAcceptOffer`; AUTO-062 `ownOfferCreatesApprovedBookingAndDoesNotChangeWeeklyAvailability`; AUTO-063 `offeringOneOffSlotDoesNotModifyWeeklyAvailability`; AUTO-064 `scheduledExpiryReleasesOutstandingOffers`
src/test/java/com/eema/backend/service/BookingApprovalExpirationJobTest.java	AUTO-065 `scheduledScanExpiresApprovedUnpaidBookings`
src/test/java/com/eema/backend/service/BookingAvailabilityValidatorTest.java	AUTO-066 `acceptsSlotContainedInEffectiveAvailability`; AUTO-067 `rejectsSlotOutsideEffectiveAvailability`; AUTO-068 `rejectsInvalidVenueTimezone`
src/test/java/com/eema/backend/service/BookingCancellationCoordinatorTest.java	AUTO-069 `scoutWithdrawsRequestedAndAwaitingPaymentWithoutRefund`; AUTO-070 `ownerCanCancelOwnConfirmedBookingButNotAnotherVenue`; AUTO-071 `adminRequiresReasonAndReceivesFullRefundPolicy`

Existing file	Named executable cases (each links to exact steps and assertions)
src/test/java/com/eema/backend/service/BookingCancellationPolicyTest.java	AUTO-072 `gracePeriodRequiresConfirmationWithin24HoursAndShootMoreThan48HoursAway`; AUTO-073 `exactSevenDaysAndExact48HoursUsePartialTier`; AUTO-074 `cancellationEconomicsUseRetainedAmountAndCommissionSnapshot`; AUTO-075 `ownerAndAdminAlwaysProduceFullCustomerRefundAndZeroSettlement`; AUTO-076 `snapshotUsesVenueTimezoneAcrossAmsterdamDstTransition`; AUTO-077 `moneyRoundingIsHalfUpAtTwoDecimals`
src/test/java/com/eema/backend/service/BookingFinancialsTest.java	AUTO-078 `calculatesCommissionAndActualFeePayout`; AUTO-079 `roundsMoneyExplicitlyAndEnforcesInvariants`
src/test/java/com/eema/backend/service/BookingNotificationServiceTest.java	AUTO-080 `usesSnapshotRecipientsAndRendersConfirmedDetails`; AUTO-081 `scoutFailureStillAttemptsOwner`; AUTO-082 `latePaymentMessageSaysNotConfirmedAndRefundInitiated`; AUTO-083 `cancellationEmailDistinguishesInitiatedFromCompletedRefund`; AUTO-084 `authoritativeCancellationRefundCompletionUsesProcessedCopy`; AUTO-085 `ownerAndScoutAreNotifiedWhenRequestIsSubmitted`; AUTO-086 `scoutAcceptanceEmailContainsDeadlinePayCtaAndDeepLink`; AUTO-087 `scoutReceivesDeclineAndApprovalExpiryMessages`; AUTO-088 `lifecycleEmailFailureIsContained`; AUTO-089 `requestExpiryAndUnavailableUseDistinctNonDeclineWording`
src/test/java/com/eema/backend/service/BookingOverlapRuleTest.java	AUTO-090 `detectsOverlappingStart`; AUTO-091 `detectsOverlappingEnd`; AUTO-092 `detectsCandidateContainingBooking`; AUTO-093 `detectsCandidateInsideBooking`; AUTO-094 `permitsBackToBackBefore`; AUTO-095 `permitsBackToBackAfter`; AUTO-096 `permitsSeparatedBooking`
src/test/java/com/eema/backend/service/BookingPriceCalculatorTest.java	AUTO-097 `appliesMinimumExactMinutesAndFullDayCap`; AUTO-098 `capAlsoAppliesToMinimumDerivedTotalAndRoundsHalfUpToCents`; AUTO-099 `exposesAuthoritativeQuoteDetails`; AUTO-100 `rejectsInvalidConfigurationAndIntervals`

Existing file	Named executable cases (each links to exact steps and assertions)
src/test/java/com/eema/backend/service/BookingServiceTest.java	AUTO-101 `scoutCanReadOwnBookingButNotAnotherScoutsBooking`; AUTO-102 `rejectsWebhookWhosePaymentIdDoesNotMatchStoredPayment`; AUTO-103 `rejectsWebhookWhoseAmountDoesNotMatchDatabasePrice`; AUTO-104 `duplicatePaidWebhookDoesNotCreateAnotherEvent`; AUTO-105 `firstPaidTransitionPublishesNotificationOnceAndDuplicateDoesNot`; AUTO-106 `ownerAndAdminCanReadAuthorizedBooking`; AUTO-107 `databaseConflictFromConcurrentAttemptCannotCreateSecondBooking`; AUTO-108 `requestAcceptPaymentEndsConfirmedWithoutSecondOwnerAction`; AUTO-109 `scoutCreatesRequestWithSelectedTimesWithoutStartingPayment`; AUTO-110 `ownerCanDeclineRequestAndDoubleAcceptanceIsRejected`; AUTO-111 `unauthorizedOwnerCannotAcceptRequest`; AUTO-112 `acceptedRequestMakesOnlyOverlappingRequestsUnavailable`; AUTO-113 `acceptanceAfterOwnerDeadlineExpiresRequestAndFailsClearly`; AUTO-114 `acceptanceThatHasBecomeTooLateExpiresRequest`; AUTO-115 `dueRequestExpirationIsIdempotentAndIgnoresAcceptedRows`; AUTO-116 `approvedRequestExpiresAndLatePaymentIsMarkedForRefund`; AUTO-117 `paymentAfterDeadlineExpiresBookingEvenBeforeSchedulerRuns`; AUTO-118 `acceptedInquiryBookingPublishesOnlyAcceptedBookingLifecycleEmail`
src/test/java/com/eema/backend/service/BookingStateMachineTest.java	AUTO-119 `permitsDefinedTransitions`; AUTO-120 `rejectsUserSkippingPaymentAndTerminalMutations`
src/test/java/com/eema/backend/service/CancellationServiceTest.java	AUTO-121 `openAwaitingPaymentIsCancelledAtMollieBeforeWithdrawal`; AUTO-122 `paidProviderStateIsAppliedBeforeConfirmedCancellationPolicy`; AUTO-123 `zeroRefundCancellationNeverCallsRefundProvider`; AUTO-124 `duplicateCancellationResumesAmbiguousRefundWithPersistedClaim`
src/test/java/com/eema/backend/service/CriticalBookingWorkflowTest.java	AUTO-125 `requestApprovalCheckoutAndAuthoritativePaymentCompleteOneReservedBooking`; AUTO-126 `declineAndExpiryReleaseSlotsAndRejectCheckout`; AUTO-127 `paidCallbackAfterDeadlineKeepsSlotReleasedAndInitiatesOneRefund`
src/test/java/com/eema/backend/service/EmailServiceTest.java	AUTO-128 `disabledEmailNeedsNoCredentials`; AUTO-129 `enabledEmailRequiresCredentials`
src/test/java/com/eema/backend/service/FinancialDocumentConcurrencyContractTest.java	AUTO-130 `issuanceLocksAuthoritativeAggregateRowsBeforeUniqueDocumentCreation`
src/test/java/com/eema/backend/service/FinancialDocumentPdfRendererTest.java	AUTO-131 `receiptContainsPaymentSnapshotAndExcludesInternalEconomics`; AUTO-132 `payoutStatementUsesExactLedgerValuesAndTransferReference`

Existing file	Named executable cases (each links to exact steps and assertions)
src/test/java/com/eema/backend/service/FinancialDocumentServiceTest.java	AUTO-133 `confirmedScoutReceiptIssuesOnceAndAdminCanReuseIt`; AUTO-134 `documentNumbersAreUniqueAndHistoricalReceiptIsNotRewritten`; AUTO-135 `unpaidStatesAndUnauthorizedPeopleCannotObtainReceipt`; AUTO-136 `confirmedPaymentRemainsReceiptEligibleAfterCancellationBeforeFirstDownload`; AUTO-137 `onlyPaidPayoutUsesExactLedgerAndIssuesOneStatement`; AUTO-138 `readyHeldScoutAndUnrelatedOwnerCannotObtainStatement`
src/test/java/com/eema/backend/service/HttpMollieProviderClientTest.java	AUTO-139 `omitsWebhookUrlForManualLocalWebhookMode`; AUTO-140 `sendsWebhookUrlWhenPublicTunnelIsConfigured`
src/test/java/com/eema/backend/service/IbanPolicyTest.java	AUTO-141 `normalizesAndAcceptsValidSepaIbans`; AUTO-142 `rejectsChecksumAndUnsupportedCountry`
src/test/java/com/eema/backend/service/IncidentReportTest.java	AUTO-143 `transitionsAreNarrowAndExplicit`; AUTO-144 `cannotResolveBeforeReview`
src/test/java/com/eema/backend/service/LocalPrivateDocumentStorageTest.java	AUTO-145 `storesOpaqueRelativeReferencesAndRejectsTraversal`; AUTO-146 `publicUploadControllerHasNoPrivateDocumentStorageDependency`
src/test/java/com/eema/backend/service/MollieFeeReconciliationJobTest.java	AUTO-147 `reconcilesAuthoritativeFeeOncePerPendingBooking`
src/test/java/com/eema/backend/service/OwnerFinancialLiabilityServiceTest.java	AUTO-148 `settleUsesStoredAmountAuditsAndIsIdempotent`; AUTO-149 `nonAdminCannotSettleAndWaivedCannotBeSettled`
src/test/java/com/eema/backend/service/OwnerPayoutServiceTest.java	AUTO-150 `otherwiseEligiblePayoutWaitsForVerifiedDestination`; AUTO-151 `normalEconomicsBecomeReadyOnlyAfterEndAndAuthoritativeFee`; AUTO-152 `scoutPartialCancellationUsesAuthoritativeSnapshot`; AUTO-153 `ownerCancellationCreatesProviderCostLiabilityWithoutNegativePayout`; AUTO-154 `adminCancellationIsCancelledWithoutOwnerLiability`; AUTO-155 `latePaymentCancelsAnyDefensivePayoutRecord`; AUTO-156 `paidHistoryIsImmutableAndReversalCreatesRecoveryLiability`; AUTO-157 `markPaidIsIdempotentOnlyForSameReference`; AUTO-158 `adminListForwardsAllFiltersAndCapsPageSize`; AUTO-159 `ownerHistoryIsScopedAndDoesNotExposeTransferReference`
src/test/java/com/eema/backend/service/OwnerVerificationServiceTest.java	AUTO-160 `ownerSubmitsAndCannotDowngradeVerifiedState`; AUTO-161 `adminApprovesPendingAndRejectsPendingWithReason`
src/test/java/com/eema/backend/service/PaymentCreationCoordinatorTest.java	AUTO-162 `concurrentClaimsReuseDurableProviderKeyAndCompletionIsIdempotent`; AUTO-163 `refusesUnsafeRetryOutsideMollieReplayWindow`; AUTO-164 `checkoutBeforeOwnerApprovalIsRejected`; AUTO-165 `checkoutAfterApprovalExpirationIsRejected`
src/test/java/com/eema/backend/service/PaymentServiceLeadTimeExemptionTest.java	AUTO-166 `acceptedBookingCanCheckoutWithoutReapplyingLeadTime`
src/test/java/com/eema/backend/service/PayoutDestinationCryptoTest.java	AUTO-167 `authenticatedEncryptionUsesRandomNonceAndRoundTrips`; AUTO-168 `tamperingFailsAuthentication`

Existing file	Named executable cases (each links to exact steps and assertions)
src/test/java/com/eema/backend/service/PolicyAcceptanceServiceTest.java	AUTO-169 `publishingANewVersionMakesTheOldAcceptanceIncomplete`; AUTO-170 `duplicateAcceptanceIsIdempotent`; AUTO-171 `snapshotReferencesExactFourCurrentVersions`
src/test/java/com/eema/backend/service/RefundCoordinatorLatePaymentTest.java	AUTO-172 `duplicateClaimReusesOneProviderIdempotencyKeyAndOneNotification`; AUTO-173 `authoritativeRefundCompletionKeepsBookingExpired`; AUTO-174 `cancellationRefundCompletionIsIdempotentAndKeepsCancelledStatus`
src/test/java/com/eema/backend/service/RefundServiceTest.java	AUTO-175 `onlyAdminMayRefund`; AUTO-176 `definitiveProviderRejectionIsRecordedAndSanitized`; AUTO-177 `latePaymentProviderFailureRetainsCoordinatorStateForSafeRetry`; AUTO-178 `cancellationRefundUsesExactClaimedPartialAmount`; AUTO-179 `duplicateCancellationClaimDoesNotCreateAnotherProviderRefund`
src/test/java/com/eema/backend/service/S3StorageTest.java	AUTO-180 `publicUploadUsesConfiguredPublicBucketAndGeneratedKey`; AUTO-181 `rejectsOversizedAndUnsupportedImages`; AUTO-182 `privateStorageRejectsTraversal`
src/test/java/com/eema/backend/service/TrustNotificationServiceTest.java	AUTO-183 `sendsVerificationApprovalAndSafeRejectionReason`; AUTO-184 `sendsApprovalChangesAndSuspensionMessages`
src/test/java/com/eema/backend/service/VenueAvailabilityServiceTest.java	AUTO-185 `ownerMayManageOwnVenueAndTimezoneIsValidated`; AUTO-186 `ownerCannotManageAnotherOwnersVenue`; AUTO-187 `scoutCannotMutateAvailability`; AUTO-188 `rejectsInvalidAndOverlappingWeeklyIntervals`; AUTO-189 `calculatesWeeklyAndEmptyDay`; AUTO-190 `blockedOverrideSubtractsPartialRange`; AUTO-191 `availableOverrideAddsAndNormalizesOutsideWeeklyHours`; AUTO-192 `leadTimeRemovesClipsAndPreservesIntervals`; AUTO-193 `blocksPartOfSpecialAvailabilityWhenNoWeeklyHoursExist`; AUTO-194 `confirmedBookingSplitsAvailabilityAndCancelledBookingDoesNotBlock`; AUTO-195 `onlyReservationStatusesBlockEffectiveAvailability`; AUTO-196 `rejectsExcessiveDateRange`
src/test/java/com/eema/backend/service/VenueBookingLeadTimePolicyTest.java	AUTO-197 `allowsTwentyFiveHoursAndExactBoundary`; AUTO-198 `rejectsOneMinuteInsideBoundary`; AUTO-199 `usesElapsedTimeAcrossAmsterdamDstChange`
src/test/java/com/eema/backend/service/VenueHardRequirementEvaluatorTest.java	AUTO-200 `crewCapacityMustBeKnownAndMeetTheRequirement`; AUTO-201 `explicitActivityMustBePermittedAndNotProhibited`; AUTO-202 `requiredBooleanFactsFailClosedWhenFalseOrUnknown`; AUTO-203 `minimumCeilingHeightMustBeKnownAndHighEnough`; AUTO-204 `parkingRequirementAndMinimumSpacesFailClosed`; AUTO-205 `preferencesNeverRemoveCandidatesEvenWhenFactsAreUnknown`
src/test/java/com/eema/backend/service/VenueMatchAvailabilityFilterTest.java	AUTO-206 `completeIntervalInsideAvailabilityPasses`; AUTO-207 `partialOverlapAndFullyUnavailableDateAreRejected`; AUTO-208 `noCompleteTimingPreservesCandidatesAndDoesNotQueryAvailability`

Existing file	Named executable cases (each links to exact steps and assertions)
src/test/java/com/eema/backend/service/VenueModerationServiceTest.java	AUTO-209 `verifiedOwnerSubmitsCompleteVenueButOtherOwnerCannot`; AUTO-210 `unverifiedOrIncompleteOwnerCannotSubmit`; AUTO-211 `adminApprovesRequestsChangesSuspendsAndReactivatesWithoutTouchingBookings`; AUTO-212 `approvalRechecksReadiness`
src/test/java/com/eema/backend/service/VenueProductionProfileValidatorTest.java	AUTO-213 `completeProfileIsReady`; AUTO-214 `incompleteProfileReportsUnknownFieldsWithoutTreatingThemAsFalse`; AUTO-215 `contradictoryActivitiesAndParkingAreRejected`
src/test/java/com/eema/backend/service/VenueServiceTest.java	AUTO-216 `ownerUpdatesMetadataAndPersistsMediaOrderAndRemoval`; AUTO-217 `anotherOwnerAndScoutCannotUpdateVenue`; AUTO-218 `ownerMayRemoveOptionalFullDayRate`; AUTO-219 `venuePricingRequestValidationRejectsInvalidRequiredValuesAndAcceptsNullCap`; AUTO-220 `invalidOrDuplicateStorageReferencesAreRejected`; AUTO-221 `scopedUploadIsAttachedAndCleanupRunsIfPersistenceFails`
src/test/java/com/eema/backend/service/VerifiedUserPolicyTest.java	AUTO-222 `requiresCurrentDatabaseVerificationForMarketplaceUsers`; AUTO-223 `activeAdminOperationsAreNotBlockedByMarketplaceVerification`
src/test/java/com/eema/backend/service/WebhookServiceTest.java	AUTO-224 `latePaidWebhookStartsRefundInsteadOfNormalConfirmationPath`
src/test/java/com/eema/backend/service/WorkflowObservabilityTest.java	AUTO-225 `recordsLifecycleCountersWithoutHighCardinalityLabels`; AUTO-226 `stuckScanPublishesReadOnlyGauges`
src/test/java/com/eema/backend/service/ai/GeminiAiServiceTest.java	AUTO-227 `ranksOnlyCandidatesReturnedByDeterministicFiltering`; AUTO-228 `zeroEligibleCandidatesReturnsEmptyResponseWithoutCallingGeminiRanker`; AUTO-229 `availabilityRemovalSkipsRankingAndIsNotRelaxed`; AUTO-230 `onlyAvailableCandidatesReachRanking`
src/test/java/com/eema/backend/service/ai/GeminiBriefParserServiceTest.java	AUTO-231 `mapsStructuredProductionRequirementsWithoutLiveGemini`; AUTO-232 `promptDefinesRequirementStrengthEnumsAndStructuredTagBoundary`; AUTO-233 `relativeDateUsesExplicitApplicationReferenceDate`; AUTO-234 `ambiguousTimingRemainsNonExactContext`
src/test/java/com/eema/backend/service/ai/GeminiVenueRankerTest.java	AUTO-235 `rankingPromptReceivesAuthoritativeProductionFactsAndSafetyRules`

Final capability traceability

Business capability	Backend endpoint/controller → service	Frontend route/component	Defined tests
Authentication	AuthController → AuthService / AccountRecoveryService; endpoint matrix above	/, /verify-email, /forgot-password, /reset-password	AUTH-001–AUTH-015; API-036, API-037, API-038, API-039, API-040, API-041, API-042, API-043; AUTO-036, AUTO-037, AUTO-038, AUTO-039, AUTO-040, AUTO-041, AUTO-046, AUTO-047, AUTO-048
Policies	PolicyController → PolicyAcceptanceService; endpoint matrix above	/policies	POL-001–POL-006; API-070, API-071, API-072, API-073; AUTO-169, AUTO-170, AUTO-171
Policies	AdminPolicyController → AdminPolicyService; endpoint matrix above	/admin/policies	POL-001–POL-006; API-025, API-026, API-027, API-028
Venues / production profile / media	VenueController → VenueService / VenueModerationService; endpoint matrix above	/ (detail panel), /owner/venues, /owner/venues/new, /owner/venues/[id]	VEN-001–VEN-018; API-082, API-083, API-084, API-085, API-086, API-087, API-088, API-089; AUTO-209, AUTO-210, AUTO-211, AUTO-212, AUTO-216, AUTO-217, AUTO-218, AUTO-219, AUTO-220, AUTO-221
Media	UploadController → FileStorage / StorageService / S3PublicMediaStorage; endpoint matrix above	/owner/venues/[id]; standalone upload API only	MEDIA-001–MEDIA-004; API-074, API-075
Availability	VenueAvailabilityController → VenueAvailabilityService / VenueBookingLeadTimePolicy; endpoint matrix above	/ (calendar), /owner/venues/[id]	AVL-001–AVL-015; API-076, API-077, API-078, API-079, API-080, API-081; AUTO-185, AUTO-186, AUTO-187, AUTO-188, AUTO-189, AUTO-190, AUTO-191, AUTO-192, AUTO-193, AUTO-194, AUTO-195, AUTO-196, AUTO-197, AUTO-198, AUTO-199
Matching	MatchController → MatchService / GeminiAiService / VenueFilterBuilder; endpoint matrix above	/	MATCH-001–MATCH-032; API-062; AUTO-227, AUTO-228, AUTO-229, AUTO-230
Booking / cancellation / refund	BookingController → BookingService / CancellationService / RefundService; endpoint matrix above	/bookings/[id], /scout/bookings, /owner/bookings; admin refund/cancel API only	BK-001–BK-008; API-050, API-051, API-052, API-053, API-054, API-055, API-056, API-057; AUTO-101, AUTO-102, AUTO-103, AUTO-104, AUTO-105, AUTO-106, AUTO-107, AUTO-108, AUTO-109, AUTO-110, AUTO-111, AUTO-112, AUTO-113, AUTO-114, AUTO-115, AUTO-116, AUTO-117, AUTO-118, AUTO-121, AUTO-122, AUTO-123, AUTO-124, AUTO-175, AUTO-176, AUTO-177, AUTO-178, AUTO-179
Checkout	CheckoutController → PaymentService / PaymentCreationCoordinator; endpoint matrix above	/bookings/[id], /booked	PAY-001–PAY-008; API-058; AUTO-162, AUTO-163, AUTO-164, AUTO-165, AUTO-166

Business capability	Backend endpoint/controller → service	Frontend route/component	Defined tests
Payment/refund callback	WebhookController → WebhookService / BookingService / RefundCoordinator; endpoint matrix above	/booked (reads state); webhook API only	PAY-001–PAY-008; API-090; AUTO-101, AUTO-102, AUTO-103, AUTO-104, AUTO-105, AUTO-106, AUTO-107, AUTO-108, AUTO-109, AUTO-110, AUTO-111, AUTO-112, AUTO-113, AUTO-114, AUTO-115, AUTO-116, AUTO-117, AUTO-118, AUTO-172, AUTO-173, AUTO-174, AUTO-224
Inquiries	AvailabilityInquiryController → AvailabilityInquiryService; endpoint matrix above	/ (calendar), /scout/inquiries, /owner/inquiries	INQ-001–INQ-004; API-044, API-045, API-046, API-047, API-048, API-049; AUTO-053, AUTO-054, AUTO-055, AUTO-056, AUTO-057, AUTO-058, AUTO-059, AUTO-060, AUTO-061, AUTO-062, AUTO-063, AUTO-064
Owner trust	OwnerVerificationController → OwnerVerificationService; endpoint matrix above	/owner/verification	TRUST-001–TRUST-003; API-068, API-069; AUTO-160, AUTO-161
Owner trust admin	AdminOwnerVerificationController → OwnerVerificationService; endpoint matrix above	/admin/owner-verifications, /admin/owner-verifications/[id]	TRUST-001–TRUST-003; API-016, API-017, API-018, API-019; AUTO-160, AUTO-161
Moderation	AdminVenueModerationController → VenueModerationService; endpoint matrix above	/admin/venues, /admin/venues/[id]	VEN-001–VEN-018; API-029, API-030, API-031, API-032, API-033, API-034, API-035; AUTO-209, AUTO-210, AUTO-211, AUTO-212
Owner payouts/statements	OwnerPayoutController → OwnerPayoutService / FinancialDocumentService; endpoint matrix above	/owner/payouts, /owner/payouts/[id]	PAYOUT-001–PAYOUT-006; API-063, API-064, API-065; AUTO-133, AUTO-134, AUTO-135, AUTO-136, AUTO-137, AUTO-138, AUTO-150, AUTO-151, AUTO-152, AUTO-153, AUTO-154, AUTO-155, AUTO-156, AUTO-157, AUTO-158, AUTO-159
Admin payouts	AdminOwnerPayoutController → OwnerPayoutService; endpoint matrix above	/admin/payouts, /admin/payouts/[id]	PAYOUT-001–PAYOUT-006; API-013, API-014, API-015; AUTO-150, AUTO-151, AUTO-152, AUTO-153, AUTO-154, AUTO-155, AUTO-156, AUTO-157, AUTO-158, AUTO-159
Liabilities	AdminOwnerFinancialLiabilityController → OwnerFinancialLiabilityService; endpoint matrix above	/admin/liabilities, /admin/liabilities/[id]	PAYOUT-001–PAYOUT-006; API-010, API-011, API-012; AUTO-148, AUTO-149
Payout destinations	OwnerPayoutDestinationController → OwnerPayoutDestinationService; endpoint matrix above	/owner/payout-destination	DEST-001–DEST-004; API-066, API-067
Payout destination review	AdminPayoutDestinationController → OwnerPayoutDestinationService; endpoint matrix above	/admin/payout-destinations, /admin/payout-destinations/[id]	DEST-001–DEST-004; API-020, API-021, API-022, API-023, API-024

Business capability	Backend endpoint/controller → service	Frontend route/component	Defined tests
Receipt	FinancialDocumentController → FinancialDocumentService; endpoint matrix above	/bookings/[id]	DOC-001–DOC-004; API-059; AUTO-133, AUTO-134, AUTO-135, AUTO-136, AUTO-137, AUTO-138
Incidents	IncidentController → IncidentService; endpoint matrix above	/bookings/[id]	INC-001–INC-002; API-060, API-061
Incident review	AdminIncidentController → IncidentService; endpoint matrix above	/admin/incidents, /admin/incidents/[id]	INC-001–INC-002; API-004, API-005, API-006, API-007, API-008
Booking operations	AdminBookingOperationsController → AdminOperationsService; endpoint matrix above	/admin/bookings, /admin/bookings/[id]	ADMIN-001–ADMIN-011; API-001, API-002, API-003; AUTO-042, AUTO-043, AUTO-044, AUTO-045
Exception queue	AdminOperationsExceptionController → AdminOperationsService / OperationalThresholds; endpoint matrix above	/admin/exceptions	ADMIN-001–ADMIN-011; API-009; AUTO-042, AUTO-043, AUTO-044, AUTO-045

Cross-cutting: pricing PRICE-001–PRICE-011; concurrency RACE-001–RACE-008; jobs JOB-001–JOB-006; operations OPS-001–OPS-009; security SEC-001–SEC-007; all request DTOs INPUT-001–INPUT-028; enums STATE-001–STATE-033; real providers PROVIDER-001–PROVIDER-005; optional Next Cala CALA-001–CALA-001; browser journeys JOURNEY-001–JOURNEY-009. Source-specific unexposed/edge paths are EDGE-001–EDGE-006.

Requested scope cross-check

Requested sections	Explicit coverage families
1–5 audit/map/levels/priorities/environments	Baseline, map, enum inventory, execution contract
6–10 account lifecycle	AUTH, INPUT auth DTOs, API auth, AUTO auth/recovery/security, UX recovery routes
11–12 policies	POL, API policy/admin, STATE, JOURNEY-009, AUTO policy
13–18 owner/venue/production/media/moderation	TRUST, VEN, READY, MEDIA, INPUT venue/verification, API, UX owner/admin
19–22 availability/lead/blocking	AVL, EDGE time boundaries, AUTO availability/overlap, API
23–26 parser/requirements/preferences/timing	MATCH, EDGE candidate cap, AUTO parser/ranker/filter
27–31 booking request/response	BK, API bookings, AUTO BookingService, RACE, EMAIL
32–34 inquiries	INQ, API inquiries, AUTO inquiry, JOB-002, EMAIL, JOURNEY-005
35–36 pricing/snapshots	PRICE explicit numeric rows, BK-001, AUTO price/financials
37–40 checkout/webhook/late payment	PAY, REF, AUTO coordinators/workflow, PROVIDER-001
41–45 cancellation	CANCEL boundary cases, AUTO cancellation, JOURNEY-004
46–48 refunds	REF, AUTO refund/coordinator, PROVIDER-001
49–55 payout/destination/liability	PAYOUT, DEST, AUTO payout/liability/crypto, API admin/owner

Requested sections	Explicit coverage families
56–57 documents	DOC, AUTO renderer/service/storage, RACE, API downloads
58–59 incidents	INC, STATE categories/transitions, INPUT incident, AUTO incident, API
60–70 every admin operation	ADMIN, API every controller endpoint, UX every admin page, domain trust/money/policy/incident cases
71–79 navigation/async/empty/error/responsive/accessibility/double-submit/stale/session	UX every route; UXACT every listed action/context; ADMIN-004, POL/CANCEL stale paths
80–87 security	Full API matrix; SEC, MEDIA, DEST, DOC, INPUT
88–95 concurrency	RACE, AUTO PostgreSQL, POL-006, UXACT, JOB
96–100 scheduled work	JOB plus extra auth-token-cleanup and fee job
101–104 email/metrics/logs	EMAIL, OPS-007/008, JOB-006, SEC-003
105–112 migrations/constraints/backup/restart	OPS-001..006, AUTO Flyway/Postgres/config
113–116 external providers/storage	PROVIDER, PAY/REF/EMAIL/MATCH/MEDIA/DOC/CALA, AUTO provider mocks
117–125 full journeys	JOURNEY-001..009
126–128 design/terminology/financial copy	UX all routes and copy table
129–132 personas/venues/time/reset	Fixture cookbook, OPS
133–140 automation/regression/gate/traceability/counts/gaps/no implementation	Every scenario automation field and final sections

TEST COVERAGE / PRODUCT GAPS DISCOVERED

These are source-observed limitations or focused validation hypotheses, not changes made by this task. Severity is proposed test priority; product/security/operations owners must decide acceptance. Source characterization may pass while the corresponding release acceptance remains FAIL/BLOCKED.

Gap	Priority	Source-observed behavior / hypothesis	Reproduction / coverage	Source
G01	P0	Historical gap resolved by VEN-D02; EDGE-004 reconfirms409 under venue lock with unchanged request/audit. Accepted commitments remain preserved.	EDGE-004, BK-006, JOURNEY-007	BookingService#acceptRequest; VenueAvailabilityService#getAvailableIntervalsExcludingBooking
G02	P0	Password reset removes refresh tokens but issued access JWT remains valid until expiry; logout does not revoke refresh tokens. “Old sessions invalid” must distinguish these cases.	AUTH-007, AUTH-015, JOURNEY-008	AccountRecoveryService; JwtAuthenticationFilter; AuthProvider

Gap	Priority	Source-observed behavior / hypothesis	Reproduction / coverage	Source
G03	P0	MEDIA-D01 fixed: both adapters now decode supported images with bounded dimensions/resources and reject fabricated content before storage. Reference shape still does not prove existence/exclusive ownership; EXIF and legacy objects remain unmodified. See MEDIA execution report.	MEDIA-002/003, SEC-002	StorageService; S3PublicMediaStorage; VenueMediaPolicy
G04	P1	Historical mismatch/cap superseded by prior fixes. OPS-007 verifies the shared positive-paid CONFIRMED/CANCELLED/REFUNDED predicate and strict cutoff, with205 additional exceptions fully visible and gauge/queue agreement. ADMIN complete-backlog pagination remains authoritative.	OPS-007, ADMIN exception cases	BookingRepository#countOldUnreconciledMollieFees/#findOldUnreconciledMollieFees; AdminOperationsService
G05	P1	Refresh single-flight remains per JS module/tab. The pre-launch stale-loser clearing defect is FIXED by generation comparison; simultaneous rotation still has one server winner. Second 401 after a successful refresh does not explicitly clear session.	AUTH-009/010, UXACT expiry cases	lib/api/client.ts; lib/auth/token.ts
G06	P0	Baseline harness gap corrected: local Chromium/CDP domain harnesses and real PostgreSQL-backed browser tests now exist and execute. UX has passing browser/client checks but required real screen-reader execution is BLOCKED (UX-ENV-001). Frontend CI/provider/JOURNEY launch coverage is not implied by these local runs.	All UX/JOURNEY; PROVIDER	UX and prior domain execution reports; scripts/qa; opt-in browser integration tests

Gap	Priority	Source-observed behavior / hypothesis	Reproduction / coverage	Source
G07	P0	JOB portion VERIFIED: actual concurrent scheduler/service tests prove singular committed expiry, rollback-safe counters/outbox, bounded token cleanup and idempotent fee reconciliation. Approval selection has short-lived locks with per-booking locked recheck; reporting lookups may overlap without duplicate financial effects. JOB-DEF-001–004 are fixed, including bounded fee/payout backlog progress. RACE-001–008 are also verified by audited HTTP/service races in the current complete PostgreSQL run. Each contract has its own method mapping; no blanket guarantee for untested races or deployed providers.	RACE, JOB-001/004/005	PostgresCriticalPathIntegrationTest; BookingApprovalExpirationJob; MollieFeeReconciliationJob; AuthTokenCleanupJob
G08	P0	RESOLVED LOCALLY under OPS-DEC-001: PostgreSQL transactional outbox with same-transaction intent, bounded fenced lease claims, retries, auth nonce/placeholder security and failed-delivery visibility. Full regression and packaged crash/restart dispatch passed. Historical process-memory crash loss is preserved. OPS-DEF-001 remains fixed; OPS-DEF-002 non-email refund routing regression is fixed. Real provider behavior remains staging-only; new email paths remain EMAIL-031 scope.	EMAIL-031, OPS-008	Event listeners; BookingNotificationService; IncidentService
G09	P0	Approved legal/business texts, final privacy acknowledgement, insurance and retention/support communication decisions are external prerequisites. No policy content in common migrations; publishing test text is not a legal gate.	POL, JOURNEY-009	PolicyAcceptanceService; PHASE_A_POLICY_AND_INCIDENT_CONTRACTS.md

Gap	Priority	Source-observed behavior / hypothesis	Reproduction / coverage	Source
G10	P1	Cancellation portion RESOLVED: signed preview proof, locked reevaluation, stale-term 409 and frontend refetch/renewed confirmation are locally verified in the CANCEL report (2026-09-19). Other review screens remain under their own ADMIN/UX scenarios.	CANCEL-003, ADMIN-004, UX all details	CancelBookingRequest; BookingCancellation; admin page handlers
G11	P1	Historical overdue decline resolved under EDGE-DEC-003: both owner decisions commit EXPIRED at/after the deadline. Scheduler only performs cleanup. Existing response/payment windows are unchanged; inquiry offer expiry remains capped at slot start.	EDGE-005, BK-004/005	BookingService#declineRequest/createBooking/acceptRequest
G12	P0	Historical midnight/sub-minute issues fixed by AVL/PRICE. EDGE v1 rejects nonexistent local times before reservation and requires positive resolved duration; explicit earlier-overlap policy preserves wall-clock pricing. Original P0 alias evidence is retained; all EDGE regression gates passed.	EDGE-001/002	VenueAvailabilityService#applyLeadTime; BookingPriceCalculator; VenueBookingLeadTimePolicy
G13	P1	EDGE-DEC-002 replaces the total candidate cap with bounded deterministic SQL batches until30 eligible available candidates or exhaustion. Preferences remain model-dependent. Genuine zero available candidates skips ranker only; parser may still call Gemini.	EDGE-003, MATCH preference case	VenueFilterBuilder; GeminiAiService; GeminiVenueRanker
G14	P1	Source README claims no owner editing, duplicated upload path and only two admin mutations are stale. Current frontend has listing editor, scoped upload and trust/policy/incident actions. Public detail is a panel, not a route.	VEN, API, UX, EDGE-006	Frontend README backend TODO/admin text versus current routes/components

Gap	Priority	Source-observed behavior / hypothesis	Reproduction / coverage	Source
G15	P1	Enum WAIVED has no waiver endpoint; venue delete/getAllVenues service methods have no public controller path; administrative cancellation/refund are API-only. Draft policy editing also has no endpoint.	STATE, EDGE-006, POL-002, REF-001	Controllers and service call sites
G16	P0	Payout destination service has no dedicated OwnerPayoutDestinationServiceTest file; crypto/IBAN and raw SQL uniqueness tests do not cover complete replacement/reveal review workflow. New policy/incident controllers are not automatically covered by older hand-enumerated security-surface tests.	DEST, API all endpoints	src/test/java inventory
G17	P1 — resolved locally	First-issuance venue/owner display details remain live, then bytes are immutable. Prior substitution, truncation and overflow were fixed under DOC: controlled embedded Unicode fonts preserve tested original names; unsupported glyph/complex sequences fail safely. Not every field is booking-time snapshotted.	DOC-002/004 — PASS	FinancialDocumentService; FinancialDocumentPdfRenderer; DOC execution report
G18	P0	Document and reveal responses have explicit no-store; DOC response tests verify the document header. Remaining session/storage concerns belong to SEC/AUTH. Tokens in localStorage remain accessible to same-origin JS. Frontend malformed stored session validation is partial.	SEC-005/007, AUTH	FinancialDocumentController; lib/auth/token.ts
G19	P1	Event payload allowlist filters only keys at first level, not nested values under allowed keys. Generic frontend 403 copy says ADMIN required even for owner/scout ownership denial; short validation text may include backend-provided content.	ADMIN-003, SEC-002, UX	AdminOperationsService#safePayload; lib/api/error-message.ts

Gap	Priority	Source-observed behavior / hypothesis	Reproduction / coverage	Source
G20	P0	No universal idempotency key for booking/inquiry/incident creation; UI busy states alone do not guarantee exactly one action under replay/network loss. Concurrent duplicate policy acceptance was reproduced as HTTP 500 and corrected with PostgreSQL conflict-safe inserts; see POL execution evidence for repeated real-transaction checks.	UXACT, POL-006	Create controllers/services; PolicyAcceptanceService#accept
G21	P1	Frontend verification gate applies broadly to required non-GET calls, while backend policy acceptance/incident report/matching do not all have identical verified-email enforcement. Preserve backend actual role matrix and test UI asymmetry.	AUTH-013, API, INC, POL	lib/api/client.ts; VerifiedEmailEnforcementSurfaceTest; controllers
G22	P1	Optional Cala BFF publicly accepts company input, logs company/provider error text and returns raw provider data on success; compatibility component reachability needs confirmation. It is not owner verification.	CALA-001, EDGE-006	app/api/verify-booker/route.ts; lib/cala.ts
G23	P0	Frontend README records dependency security findings and required patched-major migration before public/untrusted deployment. This documentation task does not re-audit advisories or resolve them.	Alpha SECURITY gate	Frontend README dependency section, package/package-lock baseline
G24	P0	No online payout-key rotation; losing encryption/fingerprint keys breaks recovery and duplicate detection. Local/public/private storage all need backup with database; removing media references does not guarantee object cleanup.	DEST-002, OPS-004/005, MEDIA-003	README; PayoutDestinationCrypto; storage implementations

Gap	Priority	Source-observed behavior / hypothesis	Reproduction / coverage	Source
G25	P1	Service compatibility constructors permit null policy acceptance collaborators in some unit fixtures. Passing those tests does not prove current policy gating is exercised.	POL, API, AUTO interpretation	BookingService / VenueModerationService constructor paths and test setup

No source-observed gap is silently fixed. No meaningful exposed controller/page/job/enum remains without a defined scenario in this baseline inventory; many lack automated or executed acceptance evidence. Future code changes must update this inventory and traceability before claiming completeness.

Regression packs and recommended automation order

Durations assume prepared fixtures and healthy environments; a failed gate stops release even if the time box expires. Do not force all P0 tests into a short smoke pack.

Pack	Ordered subset / evidence
10-minute smoke	Health (SEC-006), login/reload (AUTH-005/007), active catalogue/detail (VEN-008), read own booking and deny another (BK-008), admin exceptions read (ADMIN), one mocked request→accept check (BK-001/005) on prepared local fixtures. Production uses only read-only subset.
30-minute pre-commit/manual	Smoke + policy gate (POL-004), owner verification/readiness/submission (TRUST-001, READY, VEN-009), inquiry offer/accept (INQ-002/003), checkout boundary (PAY-001/002), cancel preview (CANCEL-003), receipt authorization (DOC-001), one mobile/keyboard route UX check. Run relevant Java tests separately.
2-hour release regression	Prepared JOURNEY-001/002/005/006/009; cancellation 100/50/0 and owner/admin; late payment/duplicate webhook; destination replacement/READY snapshot/mark-paid; incident isolation; one bad actor per admin controller; PostgreSQL suite; all changed-route error/responsive checks. Full suite if high-risk changes.
Full staging rehearsal	All P0/P1 and all STAGING provider cases; every route UX checklist, schema validation, backup/restore, restart, cross-origin/TLS, secret-log audit, exception/job recovery, legal/support gate. P2/P3 findings require disposition before beta.

Highest-priority manual sequence: prepare identities and current test policies → verify owner and listing → scout request → owner acceptance plus competing request → mocked or approved TEST payment and duplicate webhook → late payment/full refund → cancellation boundary/partial refund → private receipt access → destination verification/READY snapshot/mark-paid statement → incident isolation → expired session/ownership attacks → restore and operations. Use the JOURNEY IDs for full step sequences, and keep provider proof separate from mocked assertions.

Do not implement Playwright as part of this task. Recommended future automation order:

1. Preserve fast Java unit + PostgreSQL suites and add actual HTTP role/ownership checks for every API row, especially newer policy/incident/destination controllers. Add service-level transaction races beyond raw SQL guarantees.

2. Build a controlled provider test harness and deterministic fixture reset/clock boundary; then browser auth/register/verify/login/refresh/logout and policy gating.
3. Browser scout request→owner accept→checkout boundary→authoritative confirmed view with provider mocked; competing requests and late payment remain primarily API/integration tests plus small UI assertions.
4. Browser owner verification→venue editor/photos/readiness→moderation, inquiries, and stale-state refetch.
5. Browser cancellation preview/confirmation and receipt permission; exact cancellation/price permutations stay table-driven Java/API tests.
6. Browser payout destination reveal clearing/replacement, READY snapshot/mark-paid/statement and incident review; bank crypto/uniqueness/ledger remain integration/unit.
7. Error/offline/malformed response, responsive/a11y automation for all routes; manual keyboard/screen-reader/copy/design and real-provider delivery remain explicit human/staging work.

For each future automated browser test, assert observable business outcome and backend state, not a brittle sequence of DOM implementation details. Keep real Mollie/Resend/Gemini/S3 tests sandboxed and separately scheduled; do not make unit CI depend on provider availability. Exact money, token races, exclusion constraints and worker locks should remain lower-level tests, not browser timing races.

Private-alpha acceptance gate

Allowed status for each row: **PASS / FAIL / BLOCKED / NOT RUN**. All rows below are **NOT RUN** at plan creation. A gate needs named tester, evidence and accountable reviewer. P0 FAIL/BLOCKED/NOT RUN prevents alpha; business/infrastructure uncertainty is not waived by passing mocks. Any exception requires explicit accountable release decision recorded outside this document, not an automatic approval.

Category	Acceptance item	Required coverage	Status	Evidence / owner
FUNCTIONAL	All current P0/P1 marketplace journeys and state/ownership transitions executed; no undefined workflow outcome.	JOURNEY, BK, INQ, VEN, POL	NOT RUN	Unassigned / none
SECURITY	All endpoint roles/ownership, reset JWT limitation, uploads, document/IBAN confidentiality and token/log probes reviewed; dependency findings resolved or explicit controlled-release decision.	API, SEC, DEST; G01–G03/G18/G23	NOT RUN	Unassigned / none
PAYMENT	Mollie TEST create/paid/duplicate/late refund/full/partial/zero cancellation evidence; no duplicate charge/refund; fee reporting limitation has operating procedure.	PAY, REF, CANCEL, PROVIDER-001/005	NOT RUN	Unassigned / none
TRUST	Owner/venue/bank review verified; suspension acceptance gap resolved or explicit approved rule; audit and participant incident visibility correct.	TRUST, VEN, DEST, INC; G01	NOT RUN	Unassigned / none

Category	Acceptance item	Required coverage	Status	Evidence / owner
OPERATIONS	Jobs, exception queue/metric divergence, manual transfer recording, liabilities, provider ambiguity and lost email recovery have named operators and exercised runbooks.	JOB, OPS, PAYOUT, EMAIL	NOT RUN	Unassigned / none
UX	Every route tested at 375/768/1440; 11 representative actual screen-reader flows remain outstanding; no duplicate run per route is required. UXACT remains unstarted.	UX, UXACT, copy table	BLOCKED	UX-ENV-001; see UX execution report
LEGAL/BUSINESS	Approved five policies, privacy semantics, insurance/retention and incident/refund/payout communications confirmed; no development placeholder publication.	POL, EMAIL-031; G09	NOT RUN	Unassigned / none
INFRASTRUCTURE	Staging HTTPS/origins, production startup validation, DB/migrations, secret management, public/private storage, backup restore and restart validated.	OPS, SEC-004, PROVIDER-004	NOT RUN	Unassigned / none

Before staging and before beta

Before staging: source inventory refreshed, fast tests and real PostgreSQL suite executed, common-only fresh database and local backup restore rehearsed, P0 local contracts reviewed, approved staging secrets/origins/test-provider accounts ready. Missing legal texts may allow an isolated engineer-only test fixture rehearsal, never a claim that private-alpha legal acceptance passed.

Before beta: repeat full gate after code changes; close or explicitly disposition all P2/P3; test representative supported desktop/mobile browsers (record Chrome/Firefox/Safari versions rather than assume support); longer multi-session race/soak and queue backlog runs; restore from managed backup/PITR and object-store failure; actual provider outages/reconciliation; approved retention, incident communications and support response ownership. Traffic targets, RTO/RPO, supported regions/browser versions and service levels require product/operations input and must not be invented. Add those measurable targets before marking the beta operational gate PASS.

Test counts and verification ledger

Total: 703 distinct scenario IDs. Parameter variants (all roles, enum values, boundary values, 3 viewports and provider statuses) are required subchecks but are not multiplied into the count. Existing automated contracts and manual/API cases intentionally overlap capability coverage at distinct levels. No executed PASS count is claimed.

By priority

Priority	Scenarios
P0	413
P1	252
P2	37
P3	1

By scenario family/domain

Scenario family/domain	Scenarios
AUTH	15
POL	6
TRUST	3
VEN	18
READY	12
MEDIA	4
AVL	15
MATCH	32
BK	8
INQ	4
PRICE	11
PAY	8
CANCEL	14
REF	4
PAYOUT	6
DEST	4
DOC	4
INC	2

Scenario family/domain	Scenarios
ADMIN	11
SEC	7
EDGE	6
OPS	9
JOB	6
RACE	8
UX	35
UXACT	19
PROVIDER	5
CALA	1
EMAIL	31
JOURNEY	9
INPUT	28
API	90
STATE	33
AUTO	235

By primary environment requirement

Primary environment requirement	Scenarios
LOCAL REAL POSTGRES	379
LOCAL MOCKED	319
STAGING	5

By automation recommendation

Automation recommendation	Scenarios
SHOULD AUTOMATE	428
MANUAL ONLY	35

Automation recommendation	Scenarios
STAGING ONLY	5
ALREADY AUTOMATED	235

By test level (overlapping)

Test level (overlapping)	Scenarios
API	348
LOCAL E2E	181
UNIT	317
INTEGRATION	126
SECURITY	166
OPERATIONAL	20
CONCURRENCY	35
MANUAL UX	93
STAGING E2E	14

Inventory reconciled: **24 backend controllers / 90 HTTP method-path endpoints**, **35 frontend pages / 1 Next server endpoint**, **33 Java enums / 176 declared values**, **6 scheduled methods**, **30 implemented email recipient/lifecycle variants** and one explicit unsupported-email case. Existing automation mapped: **235 named methods in 64 files**, including **13 real-Postgres methods**. Helpers/fixtures without @Test are dependencies, not counted tests.

Local-only primary tests = LOCAL MOCKED count; local real-Postgres primary = LOCAL REAL POSTGRES count; mandatory staging-only primary = STAGING count. LOCAL MOLLIE TEST is an optional execution alternative to sandbox payment scenarios, with zero tests requiring it exclusively. Private alpha is a gate/rehearsal environment, not a separate duplicate count. Production smoke is read-only subset, not another paid-flow count.

Provider-specific cases: PAY/REF (Mollie mock), EMAIL (Resend content), MATCH (Gemini mocked extraction/ranking), MEDIA/DOC (local/mock S3), CALA (Next server provider), and five explicit real STAGING PROVIDER cases. Existing AUTO provider methods supply exact assertions without implying live delivery. Security and concurrency counts are by explicit test-level tags; all API cases include the full four-role matrix and object variants. UX/design includes one complete parameterized checklist per page plus UXACT double-submit/session cases and journeys.

Verification performed for this document: inventory/set comparison for every controller mapping, every Next page/route, every Java enum (including nested lifecycle/payment-result enums), every @Scheduled method and named existing @Test method; all IDs/anchors and priority/environment counts checked; output contains only fictional fixture data and repository test fixtures, no environment secret values. `git diff --check` and the explicit untracked-file check `git diff --no-index --check /dev/null docs/EEMA_COMPLETE_TEST_MASTER_PLAN.md` reported no whitespace errors. Application tests/providers were not executed for this planning-only task.

Re-run inventory before each release: new endpoint/page/state/job/event/field must receive a test ID, dependencies and traceability mapping. **No inventoried implemented capability remains without a defined test; completion of this document does not mean those tests have been executed or that discovered gaps are launch-safe.**

AUTH launch-fix follow-up — 2026-09-10

AUTH-001 through AUTH-015 were rerun: all15 LOCAL PASS. Argon2id resolved the historical AUTH-004/AUTH-014 BCrypt failure; cross-tab failures now preserve newer sessions; unfiltered admin booking search and all filter combinations pass real PostgreSQL. See execution report and design/parameters.

P1 owner stale-auth UX — RESOLVED: required reads use non-secret session generations. A stale403 joins recovery or checks `/auth/me`; only successful generation change permits one retry. Genuine403 remains forbidden with owner-appropriate copy. Three browser recovery repetitions passed without reload, alongside real ownership denial and role guards. See owner recovery report. Backend application code remained unchanged; no unrelated domain gate is marked PASS.